THIRSTY'S CODEX

STANDARDS EDITION

VOLUME XIX / 19 OF 20

STATIC ANALYSIS AND TOOLING EVIDENCE

Codacy configuration, rulesets, analysis logs, and repository tooling

NO PROOF, NO CLAIM

CONTINUOUS MASTER FOLIO / SOURCE-PRESERVING EDITION

Edition Statement

THIRSTY'S CODEX / STANDARDS EDITION / SOURCE-PRESERVING SET

This volume is one part of a continuous 20-volume edition. Canonical lifecycle section numbers remain stable across the set; no volume restarts or renumbers them.

The visual treatment has been rebuilt as a unified standards library: coordinated covers, volume architecture, source dividers, running headers, continuous master folios, local page numbers, and source-identity marks.

The source corpus remains authoritative over the presentation layer. Status terms are preserved exactly. CONTENT-COMPLETE does not silently become source-reconciled, hostile-reviewed, owner-accepted, ratified, or published.

Every registered source witness is identified by path, byte count, and SHA-256 digest. The complete source archive is preserved as an embedded attachment in Volume XX, while the two supplied foundational PDFs are preserved as attachments in Volume I.

CONTINUITY RULE

Read the master folio and canonical section sequence as one uninterrupted work. Each succeeding volume begins where the preceding volume ends.

Standards Edition Set Map

VOLUME I
Foundations and Governance
Doctrine, prompts, front matter, shared governance, roles, and output contracts

VOLUME II
Research, Strategy, and Risk
Lifecycle Sections 001-018

VOLUME III
Requirements through Hardware Architecture
Lifecycle Sections 019-060

VOLUME IV
Electrical Engineering through User Validation
Lifecycle Sections 061-106

VOLUME V
Software Systems through Smart Contracts
Lifecycle Sections 107-150

VOLUME VI
Data, Intelligence, and Human Oversight
Lifecycle Sections 151-188

VOLUME VII
Cybersecurity through Privacy Engineering
Lifecycle Sections 189-218

VOLUME VIII
Law, Assurance, and Validation
Lifecycle Sections 219-257

VOLUME IX
Configuration through Delivery Infrastructure
Lifecycle Sections 258-300

VOLUME X
Production Operations and Command
Lifecycle Sections 301-332

VOLUME XI
Support, Enterprise, and Roadmaps
Lifecycle Sections 333-373

VOLUME XII
Maintenance, Decommissioning, and Retirement
Lifecycle Sections 374-406

VOLUME XIII
Reference Registers and Working Documentation
Indexes, cross-references, source registers, checkpoints, and operational guides

VOLUME XIV
Standards Source Library
Preserved standards, command editions, blueprints, and source artifacts

VOLUME XV
Canonical Authority and Assurance
Authority, constitutions, policies, schemas, provenance, evaluation, and verifica...

VOLUME XVI
Applied Corpus and V3Q
Agent-output corpus, V3Q implementation, fixtures, tests, and governed examples

VOLUME XVII
Training Views and Indexes
Continuous-pretraining, evaluation, retrieval, and training views with canonical ...

VOLUME XVIII
Agents and Owner Command Standards
Agents 101 and Owner's Cockpit source-standard facsimiles

VOLUME XIX
Static Analysis and Tooling Evidence
Codacy configuration, rulesets, analysis logs, and repository tooling

Volume Contents

PAGE 1 OF 1

003 .Gitignore 5
004 Cli Config 6
005 Cli 7
006 Codacy.Config.Baseline 10
007 Codacy.Config 11
008 Codacy 12
009 Codacy Cli 13
010 Analysis Options 42
011 Eslint.Config 46
012 Languages Config 48
013 Lizard 49
014 Pylint 50
015 Revive 51
016 Ruleset 52
017 Semgrep 55
018 Trivy 577

SOURCE WITNESS 003

.Gitignore

SHA-256 86b825156981777eb73068ae7bd9ad725a571a73891883af7883541c294ddae8

Codacy CLI

tools-configs/
.gitignore
cli-config.yaml
logs/
generated/

CODEX/.codacy/.gitignore

SOURCE WITNESS 004

Cli Config

SHA-256 a833e2b03af5150491ff490cedb5bdff901ba7d4a28924c811e6a936b9399eed

```
mode: local
```

CODEX/.codacy/cli-config.yaml

SOURCE WITNESS 005

Cli

SHA-256 1bb82234e74e5385ae6a6e93cb61cbc3356fdffdeb36f907686b88679c0cd82c

```
#!/usr/bin/env bash


set -e +o pipefail

# Set up paths first
bin_name="codacy-cli-v2"

# Determine OS-specific paths
os_name=$(uname)
arch=$(uname -m)

case "$arch" in
"x86_64")
  arch="amd64"
  ;;
"x86")
  arch="386"
  ;;
"aarch64"|"arm64")
  arch="arm64"
  ;;
esac

if [ -z "$CODACY_CLI_V2_TMP_FOLDER" ]; then
    if [ "$(uname)" = "Linux" ]; then
        CODACY_CLI_V2_TMP_FOLDER="$HOME/.cache/codacy/codacy-cli-v2"
    elif [ "$(uname)" = "Darwin" ]; then
        CODACY_CLI_V2_TMP_FOLDER="$HOME/Library/Caches/Codacy/codacy-cli-v2"
    else
        CODACY_CLI_V2_TMP_FOLDER=".codacy-cli-v2"
    fi
fi

version_file="$CODACY_CLI_V2_TMP_FOLDER/version.yaml"


get_version_from_yaml() {
    if [ -f "$version_file" ]; then
        local version=$(grep -o 'version: *"[^"]*"' "$version_file" | cut -d'"' -f2)
        if [ -n "$version" ]; then
            echo "$version"
            return 0
        fi
    fi
    return 1
}

get_latest_version() {
    local response
    if [ -n "$GH_TOKEN" ]; then
        response=$(curl -Lq --header "Authorization: Bearer $GH_TOKEN" "https://api.github.com/r
epos/codacy/codacy-cli-v2/releases/latest" 2>/dev/null)
    else
        response=$(curl -Lq "https://api.github.com/repos/codacy/codacy-cli-v2/releases/latest"
2>/dev/null)
    fi

    handle_rate_limit "$response"
    local version=$(echo "$response" | grep -m 1 tag_name | cut -d'"' -f4)
    echo "$version"
}

handle_rate_limit() {
    local response="$1"
    if echo "$response" | grep -q "API rate limit exceeded"; then
          fatal "Error: GitHub API rate limit exceeded. Please try again later"
```

CODEX/.codacy/cli.sh

```
    fi
}

download_file() {
    local url="$1"

    echo "Downloading from URL: ${url}"
    if command -v curl > /dev/null 2>&1; then
        curl -# -LS "$url" -O
    elif command -v wget > /dev/null 2>&1; then
        wget "$url"
    else
        fatal "Error: Could not find curl or wget, please install one."
    fi
}

download() {
    local url="$1"
    local output_folder="$2"

    ( cd "$output_folder" && download_file "$url" )
}

download_cli() {
    # OS name lower case
    suffix=$(echo "$os_name" | tr '[:upper:]' '[:lower:]')

    local bin_folder="$1"
    local bin_path="$2"
    local version="$3"

    if [ ! -f "$bin_path" ]; then
        echo "□ Downloading CLI version $version..."

        remote_file="codacy-cli-v2_${version}_${suffix}_${arch}.tar.gz"
        url="https://github.com/codacy/codacy-cli-v2/releases/download/${version}/${remote_file}
"

        download "$url" "$bin_folder"
        tar xzfv "${bin_folder}/${remote_file}" -C "${bin_folder}"
    fi
}

# Warn if CODACY_CLI_V2_VERSION is set and update is requested
if [ -n "$CODACY_CLI_V2_VERSION" ] && [ "$1" = "update" ]; then
    echo "⚠□  Warning: Performing update with forced version $CODACY_CLI_V2_VERSION"
    echo "    Unset CODACY_CLI_V2_VERSION to use the latest version"
fi

# Ensure version.yaml exists and is up to date
if [ ! -f "$version_file" ] || [ "$1" = "update" ]; then
    echo "□□  Fetching latest version..."
    version=$(get_latest_version)
    mkdir -p "$CODACY_CLI_V2_TMP_FOLDER"
    echo "version: \"$version\"" > "$version_file"
fi

# Set the version to use
if [ -n "$CODACY_CLI_V2_VERSION" ]; then
    version="$CODACY_CLI_V2_VERSION"
else
    version=$(get_version_from_yaml)
fi


# Set up version-specific paths
bin_folder="${CODACY_CLI_V2_TMP_FOLDER}/${version}"

mkdir -p "$bin_folder"
bin_path="$bin_folder"/"$bin_name"

# Download the tool if not already installed
download_cli "$bin_folder" "$bin_path" "$version"
chmod +x "$bin_path"
```

CODEX/.codacy/cli.sh

```
run_command="$bin_path"
if [ -z "$run_command" ]; then
    fatal "Codacy cli v2 binary could not be found."
fi

if [ "$#" -eq 1 ] && [ "$1" = "download" ]; then
    echo "Codacy cli v2 download succeeded"
else
    eval "$run_command $*"
fi
```

CODEX/.codacy/cli.sh

SOURCE WITNESS 006

Codacy.Config.Baseline

SHA-256 93bcd153a2a3485b66e313f370f367ff962b8e8aa4a1a574159aefb0fdff12d9


```
{
  "version": 1,
  "metadata": {
    "source": "auto",
    "provider": null,
    "organization": null,
    "repositoryId": null,
    "repositoryName": null,
    "createdAt": "2026-08-06T11:14:05.881Z",
    "updatedAt": "2026-08-06T11:14:05.881Z",
    "languages": []
  },
  "tools": [],
  "exclude": []
}
```


CODEX/.codacy/codacy.config.baseline.json

SOURCE WITNESS 007

Codacy.Config

SHA-256 d302cf8f5968d5f4cfcff368c627c84ef63eed63d5da3dba7f323bacce359eaf


```
{
  "version": 1,
  "metadata": {
    "source": "auto",
    "provider": null,
    "organization": null,
    "repositoryId": null,
    "repositoryName": null,
    "createdAt": "2026-08-06T11:14:06.057Z",
    "updatedAt": "2026-08-06T11:14:06.057Z",
    "languages": []
  },
  "tools": [],
  "exclude": []
}
```


CODEX/.codacy/codacy.config.json

SOURCE WITNESS 008

Codacy

SHA-256 3f1bb0b1e5735b764738a0f349447c5f04589a18083ed8af7da6ff408706ff57

```
runtimes:
    - go@1.24.13
0
```

CODEX/.codacy/codacy.yaml

SOURCE WITNESS 009

Codacy Cli

SHA-256 87e2f485044b759fb4e9ee1af31cd34930f5a3dca5b19db4b9b6ab3bb760ce4b

```
2026-08-06T03:12:50-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:12:50-06:00 [INFO] (codacy-cli-v2/cmd/root.go:34) Executing CLI command args=[] co
mmand=init full_command=[/home/quencher/.cache/codacy/codacy-cli-v2/1.0.0-main.375.sha.df55ffa/c
odacy-cli-v2 init]
2026-08-06T03:13:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:13:07-06:00 [INFO] (codacy-cli-v2/cmd/root.go:34) Executing CLI command command=in
stall full_command=[/home/quencher/.cache/codacy/codacy-cli-v2/1.0.0-main.375.sha.df55ffa/codacy
-cli-v2 install] args=[]
2026-08-06T03:13:08-06:00 [INFO] (codacy-cli-v2/cmd/install.go:78) Starting installation process
2026-08-06T03:13:08-06:00 [INFO] (codacy-cli-v2/cmd/install.go:116) Tool scheduled for installat
ion tool=revive version=1.7.0
2026-08-06T03:13:08-06:00 [INFO] (codacy-cli-v2/cmd/install.go:185) Installing tool tool=revive
version=1.7.0
2026-08-06T03:13:08-06:00 [DEBUG] (codacy-cli-v2/config/tools-installer.go:135) Installing runti
me-based tool runtime=go tool=revive version=1.7.0
2026-08-06T03:13:08-06:00 [DEBUG] (codacy-cli-v2/config/tools-installer.go:204) Installing tool
packageManager=go packageManagerBin=/home/quencher/.cache/codacy/runtimes/go/bin/go command=inst
all github.com/mgechev/revive@v1.7.0 tool=revive version=1.7.0
2026-08-06T03:13:08-06:00 [DEBUG] (codacy-cli-v2/config/tools-installer.go:243) Running command
command=/home/quencher/.cache/codacy/runtimes/go/bin/go install github.com/mgechev/revive@v1.7.0
2026-08-06T03:13:09-06:00 [ERROR] (codacy-cli-v2/cmd/install.go:191) Failed to install tool vers
ion=1.7.0 error=failed to install tool: /home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7
.0/cli/main.go:5:2: package flag is not in std (/home/quencher/.cache/codacy/runtimes/go/src/fla
g)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/cli/main.go:6:2: package fmt is not i
n std (/home/quencher/.cache/codacy/runtimes/go/src/fmt)
/home/quencher/.cache/codacy/runtimes/go/src/bytes/buffer.go:10:2: package errors is not in std
(/home/quencher/.cache/codacy/runtimes/go/src/errors)
/home/quencher/.cache/codacy/runtimes/go/src/bytes/bytes.go:10:2: package internal/bytealg is no
t in std (/home/quencher/.cache/codacy/runtimes/go/src/internal/bytealg)
/home/quencher/.cache/codacy/runtimes/go/src/bytes/buffer.go:11:2: package io is not in std (/ho
me/quencher/.cache/codacy/runtimes/go/src/io)
/home/quencher/.cache/codacy/runtimes/go/src/bytes/iter.go:8:2: package iter is not in std (/hom
e/quencher/.cache/codacy/runtimes/go/src/iter)
/home/quencher/.cache/codacy/runtimes/go/src/bytes/bytes.go:11:2: package math/bits is not in st
d (/home/quencher/.cache/codacy/runtimes/go/src/math/bits)
/home/quencher/.cache/codacy/runtimes/go/src/bytes/bytes.go:12:2: package unicode is not in std
(/home/quencher/.cache/codacy/runtimes/go/src/unicode)
/home/quencher/.cache/codacy/runtimes/go/src/bytes/buffer.go:12:2: package unicode/utf8 is not i
n std (/home/quencher/.cache/codacy/runtimes/go/src/unicode/utf8)
/home/quencher/.cache/codacy/runtimes/go/src/bytes/bytes.go:14:2: package unsafe is not in std (
/home/quencher/.cache/codacy/runtimes/go/src/unsafe)
/home/quencher/go/pkg/mod/golang.org/x/sys@v0.30.0/unix/syscall_linux.go:15:2: package encoding/
binary is not in std (/home/quencher/.cache/codacy/runtimes/go/src/encoding/binary)
/home/quencher/go/pkg/mod/golang.org/x/sys@v0.30.0/unix/sockcmsg_unix_other.go:10:2: package run
time is not in std (/home/quencher/.cache/codacy/runtimes/go/src/runtime)
/home/quencher/go/pkg/mod/golang.org/x/sys@v0.30.0/unix/syscall_unix.go:11:2: package sort is no
t in std (/home/quencher/.cache/codacy/runtimes/go/src/sort)
/home/quencher/go/pkg/mod/golang.org/x/sys@v0.30.0/unix/syscall_linux.go:16:2: package strconv i
s not in std (/home/quencher/.cache/codacy/runtimes/go/src/strconv)
/home/quencher/go/pkg/mod/golang.org/x/sys@v0.30.0/unix/syscall.go:29:2: package strings is not
in std (/home/quencher/.cache/codacy/runtimes/go/src/strings)
/home/quencher/go/pkg/mod/golang.org/x/sys@v0.30.0/unix/syscall_unix.go:12:2: package sync is no
t in std (/home/quencher/.cache/codacy/runtimes/go/src/sync)
/home/quencher/go/pkg/mod/golang.org/x/sys@v0.30.0/unix/aliases.go:9:8: package syscall is not i
n std (/home/quencher/.cache/codacy/runtimes/go/src/syscall)
/home/quencher/go/pkg/mod/golang.org/x/sys@v0.30.0/unix/syscall_linux.go:18:2: package time is n
ot in std (/home/quencher/.cache/codacy/runtimes/go/src/time)
/home/quencher/go/pkg/mod/github.com/mattn/go-colorable@v0.1.14/colorable_others.go:8:2: package
 os is not in std (/home/quencher/.cache/codacy/runtimes/go/src/os)
/home/quencher/go/pkg/mod/github.com/!burnt!sushi/toml@v1.4.0/decode.go:5:2: package encoding is
 not in std (/home/quencher/.cache/codacy/runtimes/go/src/encoding)
```

CODEX/.codacy/logs/codacy-cli.log

```
/home/quencher/go/pkg/mod/github.com/!burnt!sushi/toml@v1.4.0/decode.go:6:2: package encoding/js
on is not in std (/home/quencher/.cache/codacy/runtimes/go/src/encoding/json)
/home/quencher/go/pkg/mod/github.com/!burnt!sushi/toml@v1.4.0/decode.go:9:2: package io/fs is no
t in std (/home/quencher/.cache/codacy/runtimes/go/src/io/fs)
/home/quencher/go/pkg/mod/github.com/!burnt!sushi/toml@v1.4.0/decode.go:10:2: package math is no
t in std (/home/quencher/.cache/codacy/runtimes/go/src/math)
/home/quencher/go/pkg/mod/github.com/!burnt!sushi/toml@v1.4.0/decode.go:12:2: package reflect is
 not in std (/home/quencher/.cache/codacy/runtimes/go/src/reflect)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/formatter/friendly.go:5:2: package cm
p is not in std (/home/quencher/.cache/codacy/runtimes/go/src/cmp)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/formatter/checkstyle.go:5:2: package
encoding/xml is not in std (/home/quencher/.cache/codacy/runtimes/go/src/encoding/xml)
/home/quencher/go/pkg/mod/github.com/hashicorp/go-version@v1.7.0/version.go:8:2: package databas
e/sql/driver is not in std (/home/quencher/.cache/codacy/runtimes/go/src/database/sql/driver)
/home/quencher/go/pkg/mod/github.com/hashicorp/go-version@v1.7.0/constraint.go:8:2: package rege
xp is not in std (/home/quencher/.cache/codacy/runtimes/go/src/regexp)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/internal/astutils/ast_utils.go:5:2: p
ackage go/ast is not in std (/home/quencher/.cache/codacy/runtimes/go/src/go/ast)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/lint/package.go:6:2: package go/impor
ter is not in std (/home/quencher/.cache/codacy/runtimes/go/src/go/importer)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/lint/file.go:7:2: package go/parser i
s not in std (/home/quencher/.cache/codacy/runtimes/go/src/go/parser)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/lint/file.go:8:2: package go/printer
is not in std (/home/quencher/.cache/codacy/runtimes/go/src/go/printer)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/lint/failure.go:5:2: package go/token
 is not in std (/home/quencher/.cache/codacy/runtimes/go/src/go/token)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/lint/file.go:10:2: package go/types i
s not in std (/home/quencher/.cache/codacy/runtimes/go/src/go/types)
/home/quencher/go/pkg/mod/golang.org/x/mod@v0.23.0/module/module.go:101:2: package path is not i
n std (/home/quencher/.cache/codacy/runtimes/go/src/path)
/home/quencher/go/pkg/mod/golang.org/x/mod@v0.23.0/modfile/rule.go:25:2: package path/filepath i
s not in std (/home/quencher/.cache/codacy/runtimes/go/src/path/filepath)
/home/quencher/go/pkg/mod/golang.org/x/sync@v0.11.0/errgroup/errgroup.go:13:2: package context i
s not in std (/home/quencher/.cache/codacy/runtimes/go/src/context)
/home/quencher/go/pkg/mod/github.com/olekukonko/tablewriter@v0.0.5/csv.go:11:2: package encoding
/csv is not in std (/home/quencher/.cache/codacy/runtimes/go/src/encoding/csv)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/formatter/friendly.go:8:2: package sl
ices is not in std (/home/quencher/.cache/codacy/runtimes/go/src/slices)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/formatter/checkstyle.go:6:2: package
text/template is not in std (/home/quencher/.cache/codacy/runtimes/go/src/text/template)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/rule/redefines_builtin_id.go:7:2: pac
kage maps is not in std (/home/quencher/.cache/codacy/runtimes/go/src/maps)
/home/quencher/go/pkg/mod/github.com/mgechev/dots@v0.0.0-20210922191527-e955255bf517/resolve.go:
4:2: package go/build is not in std (/home/quencher/.cache/codacy/runtimes/go/src/go/build)
/home/quencher/go/pkg/mod/github.com/mgechev/dots@v0.0.0-20210922191527-e955255bf517/resolve.go:
5:2: package log is not in std (/home/quencher/.cache/codacy/runtimes/go/src/log)
/home/quencher/go/pkg/mod/github.com/spf13/afero@v1.12.0/mem/file.go:25:2: package sync/atomic i
s not in std (/home/quencher/.cache/codacy/runtimes/go/src/sync/atomic)
/home/quencher/go/pkg/mod/github.com/spf13/afero@v1.12.0/httpFs.go:18:2: package net/http is not
 in std (/home/quencher/.cache/codacy/runtimes/go/src/net/http)
/home/quencher/go/pkg/mod/github.com/mgechev/revive@v1.7.0/cli/main.go:9:2: package runtime/debu
g is not in std (/home/quencher/.cache/codacy/runtimes/go/src/runtime/debug)
: exit status 1 tool=revive
2026-08-06T03:13:09-06:00 [WARNING] (codacy-cli-v2/cmd/install.go:235) Installation completed wi
th some failures
2026-08-06T03:13:40-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:13:40-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:13:40-06:00 [INFO] (codacy-cli-v2/cmd/root.go:34) Executing CLI command command=an
alyze full_command=[/home/quencher/.cache/codacy/codacy-cli-v2/1.0.0-main.375.sha.df55ffa/codacy
-cli-v2 analyze /mnt/x/03_SECTIONS/027_PROCESS_ARCHITECTURE.txt --format sarif] args=[/mnt/x/03_
SECTIONS/027_PROCESS_ARCHITECTURE.txt]
2026-08-06T03:13:40-06:00 [ERROR] (codacy-cli-v2/cmd/analyze.go:494) Analysis failed because pat
h does not exist path=/mnt/x/03_SECTIONS/027_PROCESS_ARCHITECTURE.txt
2026-08-06T03:13:40-06:00 [INFO] (codacy-cli-v2/cmd/root.go:34) Executing CLI command command=an
alyze full_command=[/home/quencher/.cache/codacy/codacy-cli-v2/1.0.0-main.375.sha.df55ffa/codacy
-cli-v2 analyze /mnt/x/03_SECTIONS/028_FUNCTIONAL_ARCHITECTURE.txt --format sarif] args=[/mnt/x/
03_SECTIONS/028_FUNCTIONAL_ARCHITECTURE.txt]
2026-08-06T03:13:40-06:00 [ERROR] (codacy-cli-v2/cmd/analyze.go:494) Analysis failed because pat
h does not exist path=/mnt/x/03_SECTIONS/028_FUNCTIONAL_ARCHITECTURE.txt
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:15:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:15:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:15:54-06:00 [INFO] (codacy-cli-v2/cmd/root.go:34) Executing CLI command full_comma
nd=[/home/quencher/.cache/codacy/codacy-cli-v2/1.0.0-main.375.sha.df55ffa/codacy-cli-v2 analyze
/mnt/t/00-Active/Thirstys Codex/03_SECTIONS/028_FUNCTIONAL_ARCHITECTURE.txt --format sarif] args
=[/mnt/t/00-Active/Thirstys Codex/03_SECTIONS/028_FUNCTIONAL_ARCHITECTURE.txt] command=analyze
2026-08-06T03:15:54-06:00 [INFO] (codacy-cli-v2/cmd/root.go:34) Executing CLI command command=an
alyze full_command=[/home/quencher/.cache/codacy/codacy-cli-v2/1.0.0-main.375.sha.df55ffa/codacy
-cli-v2 analyze /mnt/t/00-Active/Thirstys Codex/03_SECTIONS/027_PROCESS_ARCHITECTURE.txt --forma
t sarif] args=[/mnt/t/00-Active/Thirstys Codex/03_SECTIONS/027_PROCESS_ARCHITECTURE.txt]
2026-08-06T03:26:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:26:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:26:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:26:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:26:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=failed
to update codacy.yaml with runtime java: error parsing codacy.yaml: yaml: line 3: could not find
 expected ':'
2026-08-06T03:26:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=failed
to update codacy.yaml with runtime python: error parsing codacy.yaml: yaml: line 3: could not fi
nd expected ':'
2026-08-06T03:26:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=failed
to update codacy.yaml with tool eslint: error parsing codacy.yaml: yaml: line 3: could not find
expected ':'
2026-08-06T03:26:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=failed
to update codacy.yaml with tool dartanalyzer: error parsing codacy.yaml: yaml: line 3: could not
 find expected ':'
2026-08-06T03:29:39-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:29:39-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:29:39-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:29:39-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:29:39-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:29:39-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:29:39-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:29:39-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:29:39-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:29:40-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:29:40-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:29:40-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:29:40-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:29:40-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:29:40-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:29:40-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:29:40-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:29:40-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:30:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:30:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:30:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:30:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:31:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:31:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:31:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:31:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:31:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:31:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:31:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:31:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:31:15-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:31:15-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:31:15-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:31:15-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:31:15-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:31:15-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:31:15-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:31:15-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:21-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:21-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:21-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:21-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:21-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:21-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:32:21-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:21-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:58-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:58-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:32:58-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:32:58-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:33:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:33:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:33:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:33:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:33:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:33:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:33:53-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:33:53-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:33:53-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:33:53-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:33:53-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:35:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:35:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:35:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:35:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:35:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:35:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:35:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:35:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:36:54-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:36:54-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:36:54-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:36:54-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:36:54-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:36:54-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:36:54-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:36:54-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:38:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:38:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:38:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:38:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:38:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:38:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:38:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:38:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:39:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:39:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:39:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:39:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:39:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:39:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:39:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:39:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:39:44-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:39:44-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:39:44-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:39:44-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:39:44-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:39:44-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:39:44-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:39:44-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:42:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:42:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:42:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:42:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:42:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:42:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:42:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:42:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:42:30-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:42:30-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:42:30-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:42:30-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:42:30-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:42:30-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:42:30-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:42:30-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:20-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:20-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:20-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:20-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:20-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:20-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:20-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:20-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:43:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:43:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:44:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:44:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:44:15-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:44:15-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:44:15-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:44:15-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:44:15-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:44:15-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:44:15-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:44:15-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:27-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:27-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:27-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:27-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:27-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:45:27-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:45:27-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:46:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:46:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:46:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:46:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:46:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:46:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:46:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:46:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:46:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:46:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:46:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:46:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:46:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:46:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:46:25-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:46:25-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:37-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:37-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:37-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:37-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:37-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:37-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:37-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:47:37-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:54-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:54-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:54-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:54-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:54-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:54-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:47:54-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:47:54-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:48:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:48:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:48:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:48:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:48:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:48:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:48:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:48:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:00-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:00-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:00-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:00-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:00-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:00-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:00-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:00-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:49:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:49:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:49:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:50:47-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:47-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:50:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:50:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:51:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:51:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:51:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:51:48-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:51:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:51:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:51:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:51:48-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:52:02-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:52:02-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:52:02-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:52:02-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:52:02-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:52:02-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:52:02-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:52:02-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:52:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:52:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:52:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:52:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:52:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:52:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:52:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:52:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:53:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:53:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:53:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:53:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:53:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:53:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:53:26-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:53:26-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:54:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:54:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:54:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:54:19-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:54:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:54:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:54:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:54:19-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:54:25-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:54:25-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:54:25-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:54:25-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:54:25-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:54:25-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:54:25-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:54:25-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:55:27-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:55:27-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:55:27-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:55:27-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:55:27-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:55:27-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:55:27-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:55:27-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:56:25-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:56:25-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:56:25-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:56:25-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:56:25-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:56:25-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:56:25-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:56:25-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:56:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:56:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:56:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:56:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:56:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:56:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T03:56:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:56:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:57:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:57:41-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:57:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:57:41-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:57:52-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:57:52-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:57:52-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:57:52-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:57:52-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:57:52-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T03:57:52-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T03:57:52-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:00:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:00:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:00:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:00:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:00:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:00:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:00:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:00:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:00:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:00:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:00:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:00:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:00:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:00:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:00:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:00:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:01:16-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:01:16-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:01:16-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:01:16-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:03:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:03:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:03:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:03:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:03:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:03:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:03:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:03:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:09:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:09:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:09:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:09:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:10:01-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:10:01-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:10:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:10:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:11:23-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:11:23-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:15:45-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:19:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:22:08-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:25:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:27:59-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:30:49-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:34:32-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:37:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:40:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:43:03-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:08-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:46:09-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:49:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:10-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:49:11-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:52:12-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T04:55:22-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T04:55:22-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

```
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:17:24-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:18:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:18:53-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:18:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:18:53-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:18:53-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:18:53-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
2026-08-06T05:19:07-06:00 [INFO] (codacy-cli-v2/cli-v2.go:27) Starting Codacy CLI version=1.0.0-
main.375.sha.df55ffa (df55ffa) built at 2026-04-08T11:10:45Z
2026-08-06T05:19:07-06:00 [ERROR] (codacy-cli-v2/cli-v2.go:58) Configuration error error=yaml: l
ine 3: could not find expected ':'
```

CODEX/.codacy/logs/codacy-cli.log

SOURCE WITNESS 010

Analysis Options

SHA-256 ba43f33d1007c17b7596e6f9c3c7a06711df4fad7a4ecfa37d381e3a372c58ae

```
analyzer:
    errors:
        avoid_as: warning
        avoid_catches_without_on_clauses: high
        avoid_catching_errors: high
        avoid_double_and_int_checks: warning
        avoid_dynamic_calls: high
        avoid_equals_and_hash_code_on_mutable_classes: high
        avoid_field_initializers_in_const_classes: warning
        avoid_implementing_value_types: high
        avoid_js_rounded_ints: high
        avoid_returning_null: high
        avoid_returning_null_for_future: high
        avoid_slow_async_io: warning
        await_only_futures: warning
        cast_nullable_to_non_nullable: high
        close_sinks: high
        collection_methods_unrelated_type: warning
        conditional_uri_does_not_exist: high
        control_flow_in_finally: high
        discarded_futures: high
        empty_statements: high
        exhaustive_cases: high
        hash_and_equals: high
        invariant_booleans: warning
        iterable_contains_unrelated_type: high
        list_remove_unrelated_type: warning
        no_adjacent_strings_in_list: warning
        no_duplicate_case_values: high
        no_runtimeType_toString: warning
        null_check_on_nullable_type_parameter: high
        null_closures: high
        prefer_bool_in_asserts: info
        prefer_contains: info
        prefer_for_elements_to_map_fromIterable: info
        prefer_is_empty: warning
        recursive_getters: high
        secure_pubspec_urls: high
        sized_box_for_whitespace: info
        test_types_in_equals: high
        throw_in_finally: high
        unawaited_futures: high
        unnecessary_await_in_return: info
        unnecessary_statements: warning
        unrelated_type_equality_checks: warning
        unsafe_html: high
        use_build_context_synchronously: high
        use_colored_box: info
        use_decorated_box: info
        use_string_buffers: warning
        valid_regexps: high
        void_checks: high
linter:
    rules:
        always_declare_return_types: "true"
        always_put_control_body_on_new_line: "true"
        always_put_required_named_parameters_first: "true"
        always_require_non_null_named_parameters: "true"
        always_specify_types: "true"
        always_use_package_imports: "true"
        annotate_overrides: "true"
        avoid_annotating_with_dynamic: "true"
        avoid_bool_literals_in_conditional_expressions: "true"
        avoid_classes_with_only_static_members: "true"
```

CODEX/.codacy/tools-configs/analysis_options.yaml

```
avoid_empty_else: "true"
avoid_escaping_inner_quotes: "true"
avoid_final_parameters: "true"
avoid_function_literals_in_foreach_calls: "true"
avoid_init_to_null: "true"
avoid_multiple_declarations_per_line: "true"
avoid_null_checks_in_equality_operators: "true"
avoid_positional_boolean_parameters: "true"
avoid_print: "true"
avoid_private_typedef_functions: "true"
avoid_redundant_argument_values: "true"
avoid_relative_lib_imports: "true"
avoid_renaming_method_parameters: "true"
avoid_return_types_on_setters: "true"
avoid_returning_null_for_void: "true"
avoid_returning_this: "true"
avoid_setters_without_getters: "true"
avoid_shadowing_type_parameters: "true"
avoid_single_cascade_in_expression_statements: "true"
avoid_type_to_string: "true"
avoid_types_as_parameter_names: "true"
avoid_types_on_closure_parameters: "true"
avoid_unnecessary_containers: "true"
avoid_unused_constructor_parameters: "true"
avoid_void_async: "true"
avoid_web_libraries_in_flutter: "true"
camel_case_extensions: "true"
camel_case_types: "true"
cancel_subscriptions: "true"
cascade_invocations: "true"
combinators_ordering: "true"
comment_references: "true"
constant_identifier_names: "true"
curly_braces_in_flow_control_structures: "true"
dangling_library_doc_comments: "true"
depend_on_referenced_packages: "true"
deprecated_consistency: "true"
diagnostic_describe_all_properties: "true"
directives_ordering: "true"
do_not_use_environment: "true"
empty_catches: "true"
empty_constructor_bodies: "true"
enable_null_safety: "true"
eol_at_end_of_file: "true"
file_names: "true"
flutter_style_todos: "true"
implementation_imports: "true"
implicit_call_tearoffs: "true"
join_return_with_assignment: "true"
leading_newlines_in_multiline_strings: "true"
library_annotations: "true"
library_names: "true"
library_prefixes: "true"
library_private_types_in_public_api: "true"
lines_longer_than_80_chars: "true"
literal_only_boolean_expressions: "true"
missing_whitespace_between_adjacent_strings: "true"
no_default_cases: "true"
no_leading_underscores_for_library_prefixes: "true"
no_leading_underscores_for_local_identifiers: "true"
no_logic_in_create_state: "true"
non_constant_identifier_names: "true"
noop_primitive_operations: "true"
omit_local_variable_types: "true"
one_member_abstracts: "true"
only_throw_errors: "true"
overridden_fields: "true"
package_api_docs: "true"
package_names: "true"
package_prefixed_library_names: "true"
parameter_assignments: "true"
```

CODEX/.codacy/tools-configs/analysis_options.yaml

```
prefer_adjacent_string_concatenation: "true"
prefer_asserts_in_initializer_lists: "true"
prefer_asserts_with_message: "true"
prefer_collection_literals: "true"
prefer_conditional_assignment: "true"
prefer_const_constructors: "true"
prefer_const_constructors_in_immutables: "true"
prefer_const_declarations: "true"
prefer_const_literals_to_create_immutables: "true"
prefer_constructors_over_static_methods: "true"
prefer_double_quotes: "true"
prefer_equal_for_default_values: "true"
prefer_expression_function_bodies: "true"
prefer_final_fields: "true"
prefer_final_in_for_each: "true"
prefer_final_locals: "true"
prefer_final_parameters: "true"
prefer_foreach: "true"
prefer_function_declarations_over_variables: "true"
prefer_generic_function_type_aliases: "true"
prefer_if_elements_to_conditional_expressions: "true"
prefer_if_null_operators: "true"
prefer_initializing_formals: "true"
prefer_inlined_adds: "true"
prefer_int_literals: "true"
prefer_interpolation_to_compose_strings: "true"
prefer_is_not_empty: "true"
prefer_is_not_operator: "true"
prefer_iterable_whereType: "true"
prefer_mixin: "true"
prefer_null_aware_method_calls: "true"
prefer_null_aware_operators: "true"
prefer_relative_imports: "true"
prefer_single_quotes: "true"
prefer_spread_collections: "true"
prefer_typing_uninitialized_variables: "true"
prefer_void_to_null: "true"
provide_deprecation_message: "true"
public_member_api_docs: "true"
require_trailing_commas: "true"
sized_box_shrink_expand: "true"
slash_for_doc_comments: "true"
sort_child_properties_last: "true"
sort_constructors_first: "true"
sort_pub_dependencies: "true"
sort_unnamed_constructors_first: "true"
super_goes_last: "true"
tighten_type_of_initializing_formals: "true"
type_annotate_public_apis: "true"
type_init_formals: "true"
unnecessary_brace_in_string_interps: "true"
unnecessary_const: "true"
unnecessary_constructor_name: "true"
unnecessary_final: "true"
unnecessary_getters_setters: "true"
unnecessary_lambdas: "true"
unnecessary_late: "true"
unnecessary_library_directive: "true"
unnecessary_new: "true"
unnecessary_null_aware_assignments: "true"
unnecessary_null_aware_operator_on_extension_on_nullable: "true"
unnecessary_null_checks: "true"
unnecessary_null_in_if_null_operators: "true"
unnecessary_nullable_for_final_variable_declarations: "true"
unnecessary_overrides: "true"
unnecessary_parenthesis: "true"
unnecessary_raw_strings: "true"
unnecessary_string_escapes: "true"
unnecessary_string_interpolations: "true"
unnecessary_this: "true"
unnecessary_to_list_in_spreads: "true"
```

CODEX/.codacy/tools-configs/analysis_options.yaml

```
unreachable_from_main: "true"
use_enums: "true"
use_full_hex_values_for_flutter_colors: "true"
use_function_type_syntax_for_parameters: "true"
use_if_null_to_convert_nulls_to_bools: "true"
use_is_even_rather_than_modulo: "true"
use_key_in_widget_constructors: "true"
use_late_for_private_fields_and_variables: "true"
use_named_constants: "true"
use_raw_strings: "true"
use_rethrow_when_possible: "true"
use_setters_to_change_properties: "true"
use_string_in_part_of_directives: "true"
use_super_parameters: "true"
use_test_throws_matchers: "true"
use_to_and_as_if_applicable: "true"
```

CODEX/.codacy/tools-configs/analysis_options.yaml

SOURCE WITNESS 011

Eslint.Config

SHA-256 ce531a2fbb24eaeccc658d9f84b173d0246fe9096057b8c7fd0a4016aa665826

```
export default [
    {
        files: ["**/*.js", "**/*.jsx", "**/*.ts", "**/*.tsx", "**/*.mjs", "**/*.cjs", "**/*.vue"
],
        rules: {
          "constructor-super": ["error"],
          "for-direction": ["error"],
          "getter-return": ["error", {"allowImplicit": false}],
          "no-async-promise-executor": ["error"],
          "no-case-declarations": ["error"],
          "no-class-assign": ["error"],
          "no-compare-neg-zero": ["error"],
          "no-cond-assign": ["error", "except-parens"],
          "no-constant-condition": ["error", {"checkLoops": true}],
          "no-const-assign": ["error"],
          "no-control-regex": ["error"],
          "no-debugger": ["error"],
          "no-delete-var": ["error"],
          "no-dupe-args": ["error"],
          "no-dupe-class-members": ["error"],
          "no-dupe-else-if": ["error"],
          "no-dupe-keys": ["error"],
          "no-duplicate-case": ["error"],
          "no-empty": ["error", {"allowEmptyCatch": false}],
          "no-empty-character-class": ["error"],
          "no-empty-pattern": ["error", {"allowObjectPatternsAsParameters": false}],
          "no-ex-assign": ["error"],
          "no-extra-boolean-cast": ["error", {"enforceForLogicalOperands": false}],
          "no-extra-semi": ["error"],
          "no-fallthrough": ["error", {"allowEmptyCase": false}],
          "no-func-assign": ["error"],
          "no-global-assign": ["error"],
          "no-import-assign": ["error"],
          "no-inner-declarations": ["error", "functions"],
          "no-invalid-regexp": ["error"],
          "no-irregular-whitespace": ["error", {"skipComments": false, "skipJSXText": false, "sk
ipRegExps": false, "skipStrings": true, "skipTemplates": false}],
          "no-loss-of-precision": ["error"],
          "no-misleading-character-class": ["error"],
          "no-mixed-spaces-and-tabs": ["error"],
          "no-new-symbol": ["error"],
          "no-nonoctal-decimal-escape": ["error"],
          "no-obj-calls": ["error"],
          "no-octal": ["error"],
          "no-prototype-builtins": ["error"],
          "no-redeclare": ["error", {"builtinGlobals": true}],
          "no-regex-spaces": ["error"],
          "no-self-assign": ["error", {"props": true}],
          "no-setter-return": ["error"],
          "no-shadow-restricted-names": ["error"],
          "no-sparse-arrays": ["error"],
          "no-this-before-super": ["error"],
          "no-undef": ["error", {"typeof": false}],
          "no-unexpected-multiline": ["error"],
          "no-unreachable": ["error"],
          "no-unsafe-finally": ["error"],
          "no-unsafe-negation": ["error", {"enforceForOrderingRelations": false}],
          "no-unsafe-optional-chaining": ["error", {"disallowArithmeticOperators": false}],
          "no-unused-labels": ["error"],
          "no-unused-vars": ["error"],
          "no-useless-backreference": ["error"],
          "no-useless-catch": ["error"],
          "no-useless-escape": ["error"],
          "no-with": ["error"],
```

CODEX/.codacy/tools-configs/eslint.config.mjs

```
          "require-yield": ["error"],
          "use-isnan": ["error", {"enforceForIndexOf": false, "enforceForSwitchCase": true}],
          "valid-typeof": ["error", {"requireStringLiterals": false}],
        }
    }
];
```

CODEX/.codacy/tools-configs/eslint.config.mjs

SOURCE WITNESS 012

Languages Config

SHA-256 6a0ad1ec395c4b6c81831c5305be28b54968f464f2e553e729de997b012c13db

```
tools:
    - name: dartanalyzer
      languages: [Dart]
      extensions: [.dart]
      files: []
    - name: eslint
      languages: [Javascript, TypeScript]
      extensions: [.js, .jsm, .jsx, .mjs, .ts, .tsx, .vue]
      files: []
    - name: lizard
      languages: [C, CPP, CSharp, Erlang, Fortran, Go, Java, Javascript, Kotlin, Lua, Objective
C, PHP, Python, Ruby, Rust, Scala, Solidity, Swift, TypeScript]
      extensions: [.c, .cc, .cpp, .cs, .cxx, .gemspec, .go, .h, .hpp, .ino, .java, .jbuilder, .j
s, .jsm, .jsx, .kt, .kts, .m, .mjs, .opal, .php, .podspec, .py, .rake, .rb, .rlib, .rs, .scala,
.swift, .ts, .tsx, .vue]
      files: []
    - name: opengrep
      languages: [Apex, C, CPP, CSharp, Dockerfile, Go, Java, Javascript, Kotlin, PHP, PLSQL, Py
thon, Ruby, Rust, SQL, Scala, Shell, Swift, Terraform, TypeScript, YAML]
      extensions: [.bash, .c, .cc, .cls, .cpp, .cs, .cxx, .dockerfile, .env, .fnc, .gemspec, .go
, .h, .hpp, .ino, .java, .jbuilder, .js, .jsm, .jsx, .kt, .kts, .mjs, .opal, .pck, .php, .pkb, .
pkh, .pks, .plb, .pld, .plh, .pls, .podspec, .prc, .py, .rake, .rb, .rlib, .rs, .scala, .sh, .sq
l, .swift, .tf, .tpb, .tps, .trg, .trigger, .ts, .tsx, .tyb, .typ, .vue, .yaml, .yml]
      files: []
    - name: pmd
      languages: [Apex, JSP, Java, Javascript, PLSQL, SQL, Velocity, VisualForce, XML]
      extensions: [.cls, .component, .fnc, .java, .js, .jsm, .jsp, .jsx, .mjs, .page, .pck, .pkb
, .pkh, .pks, .plb, .pld, .plh, .pls, .pom, .prc, .sql, .tpb, .tps, .trg, .trigger, .tyb, .typ,
.vm, .vue, .wsdl, .xml, .xsl]
      files: []
    - name: pylint
      languages: [Python]
      extensions: [.py]
      files: []
    - name: revive
      languages: [Go]
      extensions: [.go]
      files: []
    - name: trivy
      languages: [C, CPP, CSharp, Dart, Dockerfile, Elixir, Go, JSON, Java, Javascript, PHP, Pyt
hon, Ruby, Rust, Scala, Swift, Terraform, TypeScript, XML, YAML]
      extensions: [.c, .cc, .cpp, .cs, .cxx, .dart, .dockerfile, .env, .ex, .exs, .gemspec, .go,
 .h, .hpp, .ino, .java, .jbuilder, .js, .jsm, .json, .jsx, .mjs, .opal, .php, .podspec, .pom, .p
y, .rake, .rb, .rlib, .rs, .scala, .swift, .tf, .ts, .tsx, .vue, .wsdl, .xml, .xsl, .yaml, .yml]
      files: [.deps.json, .env, .env.dev, .env.development, .env.prod, .env.production, .env.sta
ging, Berksfile, Capfile, Cargo.lock, Cheffile, Directory.Packages.props, Dockerfile, Fastfile,
Gemfile, Gemfile.lock, Guardfile, Package.resolved, Packages.props, Pipfile.lock, Podfile, Podfi
le.lock, Rakefile, Thorfile, Vagabondfile, Vagrantfile, build.sbt.lock, composer.lock, conan.loc
k, config.ru, go.mod, go.sum, gradle.lockfile, mix.lock, package-lock.json, package.json, packag
es.config, packages.lock.json, pnpm-lock.yaml, poetry.lock, pom.xml, pubspec.lock, requirements.
txt, uv.lock, yarn.lock]
```

CODEX/.codacy/tools-configs/languages-config.yaml

SOURCE WITNESS 013

Lizard

SHA-256 62af342763d0dc93c0428e96509063dfc9ead73aeee175ef11637564083de791

```
patterns:
    Lizard_ccn-minor:
        category: Complexity
        description: Reports a Minor issue when a function's cyclomatic complexity (the number o
f independent decision paths through it) reaches the configured threshold, which defaults to 10.
 This is the earliest warning tier, flagging functions that are beginning to accumulate branchin
g logic.
        explanation: |-
            # Minor Cyclomatic Complexity control

            Check the Cyclomatic Complexity value of a function or logic block. If the threshold
 is exceeded, raise a Minor issue. The default threshold is 10.
        id: Lizard_ccn-minor
        level: Info
        severityLevel: Info
        threshold: 10
        timeToFix: 5
        title: Limit Function Cyclomatic Complexity (Minor)
    Lizard_file-nloc-medium:
        category: Complexity
        description: Reports a Medium issue when a file's number of lines of code (excluding com
ments) reaches the configured threshold, which defaults to 500. Files this large typically mix s
everal concerns and are noticeably harder to navigate and maintain.
        explanation: ""
        id: Lizard_file-nloc-medium
        level: Warning
        severityLevel: Warning
        threshold: 500
        timeToFix: 10
        title: Limit File Length (Medium)
    Lizard_nloc-medium:
        category: Complexity
        description: Reports a Medium issue when a function's number of lines of code (excluding
 comments) reaches the configured threshold, which defaults to 50. Functions this long usually h
andle more than one responsibility and are noticeably harder to follow.
        explanation: |-
            # Medium NLOC control - Number of Lines of Code (without comments)

            Check the number of lines of code (without comments) in a function. If the threshold
 is not met, raise a Medium issue. The default threshold is 50.
        id: Lizard_nloc-medium
        level: Warning
        severityLevel: Warning
        threshold: 50
        timeToFix: 10
        title: Limit Function Length (Medium)
    Lizard_parameter-count-medium:
        category: Complexity
        description: Reports a Medium issue when a function's parameter count reaches the config
ured threshold, which defaults to 8. A signature this wide is hard to call correctly and usually
 signals that the function has taken on too many responsibilities.
        explanation: |-
            # Medium Parameter count control

            Check the number of parameters sent to a function. If the threshold is not met, rais
e a Medium issue. The default threshold is 5.
        id: Lizard_parameter-count-medium
        level: Warning
        severityLevel: Warning
        threshold: 8
        timeToFix: 10
        title: Limit Function Parameter Count (Medium)
```

CODEX/.codacy/tools-configs/lizard.yaml

SOURCE WITNESS 014

Pylint

SHA-256 e1c930b6a53b7d62cff8411616bc6723027eead4f6dedb8ec7666ea7ddbf755c

```
[MASTER]
ignore=CVS
persistent=yes
load-plugins=

[MESSAGES CONTROL]
disable=all
enable=C0123,C0200,E0100,E0101,E0102,E0103,E0104,E0105,E0106,E0107,E0108,E0110,E0112,E0113,E0114
,E0115,E0117,E0202,E0203,E0211,E0236,E0238,E0239,E0240,E0241,E0301,E0302,E0601,E0603,E0604,E0701
,E0702,E0704,E0710,E0711,E0712,E1003,E1102,E1111,E1120,E1121,E1123,E1124,E1125,E1126,E1127,E1132
,E1200,E1201,E1205,E1206,E1300,E1301,E1302,E1303,E1304,E1305,E1306,R0202,R0203,W0101,W0102,W0104
,W0105,W0106,W0107,W0108,W0109,W0120,W0122,W0124,W0150,W0199,W0221,W0222,W0233,W0404,W0410,W0601
,W0602,W0604,W0611,W0612,W0622,W0702,W0705,W0711,W1300,W1301,W1302,W1303,W1305,W1306,W1307
```

CODEX/.codacy/tools-configs/pylint.rc

SOURCE WITNESS 015

Revive

SHA-256 33b1766d160328e525376212af104820049b93460598452a0874493e598a6a6c

```
[revive]
ignoreGeneratedHeader = true
severity = "warning"
confidence = 0.8
errorCode = 0
warningCode = 0

rules = ["blank-imports", "context-as-argument", "context-keys-type", "dot-imports", "empty-bloc
k", "errorf", "error-naming", "error-return", "error-strings", "exported", "increment-decrement"
, "indent-error-flow", "package-comments", "range", "receiver-naming", "redefines-builtin-id", "
superfluous-else", "time-naming", "unexported-return", "unreachable-code", "unused-parameter", "
var-declaration", "var-naming"]

[rule.blank-imports]

[rule.context-as-argument]

[rule.context-keys-type]

[rule.dot-imports]

[rule.empty-block]

[rule.errorf]

[rule.error-naming]

[rule.error-return]

[rule.error-strings]

[rule.exported]

[rule.increment-decrement]

[rule.indent-error-flow]

[rule.package-comments]

[rule.range]

[rule.receiver-naming]

[rule.redefines-builtin-id]

[rule.superfluous-else]
arguments = [""]

[rule.time-naming]

[rule.unexported-return]

[rule.unreachable-code]

[rule.unused-parameter]

[rule.var-declaration]

[rule.var-naming]
```

CODEX/.codacy/tools-configs/revive.toml

SOURCE WITNESS 016

Ruleset

SHA-256 f69570c90869065426feb11bcf3313ce5f323dff801e1ff1019f47d9e0ac76fc

```
<?xml version="1.0" encoding="UTF-8"?>
<ruleset name="Codacy PMD 7 Ruleset"
    xmlns="https://pmd.github.io/ruleset/2.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://pmd.github.io/ruleset/2.0.0 https://pmd.github.io/schemas/pmd-7.
0.0.xsd">
    <description>Codacy PMD 7 Ruleset</description>
    <rule ref="category/apex/design.xml/AvoidDeeplyNestedIfStmts"/>
    <rule ref="category/apex/design.xml/ExcessiveParameterList"/>
    <rule ref="category/apex/design.xml/ExcessivePublicCount"/>
    <rule ref="category/apex/security.xml/ApexBadCrypto"/>
    <rule ref="category/apex/security.xml/ApexCRUDViolation"/>
    <rule ref="category/apex/security.xml/ApexDangerousMethods"/>
    <rule ref="category/apex/security.xml/ApexInsecureEndpoint"/>
    <rule ref="category/apex/security.xml/ApexOpenRedirect"/>
    <rule ref="category/apex/security.xml/ApexSharingViolations"/>
    <rule ref="category/apex/security.xml/ApexSOQLInjection"/>
    <rule ref="category/apex/security.xml/ApexSuggestUsingNamedCred"/>
    <rule ref="category/apex/security.xml/ApexXSSFromEscapeFalse"/>
    <rule ref="category/apex/security.xml/ApexXSSFromURLParam"/>
    <rule ref="category/ecmascript/bestpractices.xml/AvoidWithStatement"/>
    <rule ref="category/ecmascript/bestpractices.xml/ConsistentReturn"/>
    <rule ref="category/ecmascript/bestpractices.xml/UseBaseWithParseInt"/>
    <rule ref="category/ecmascript/codestyle.xml/AssignmentInOperand">
        <properties>
            <property name="allowFor" value="false"/>
            <property name="allowIf" value="false"/>
            <property name="allowIncrementDecrement" value="false"/>
            <property name="allowTernary" value="false"/>
            <property name="allowTernaryResults" value="false"/>
            <property name="allowWhile" value="false"/>
        </properties>
    </rule>
    <rule ref="category/ecmascript/codestyle.xml/UnnecessaryBlock"/>
    <rule ref="category/ecmascript/errorprone.xml/EqualComparison"/>
    <rule ref="category/java/bestpractices.xml/AvoidReassigningParameters"/>
    <rule ref="category/java/bestpractices.xml/CheckResultSet"/>
    <rule ref="category/java/bestpractices.xml/OneDeclarationPerLine">
        <properties>
            <property name="strictMode" value="false"/>
        </properties>
    </rule>
    <rule ref="category/java/bestpractices.xml/UnusedFormalParameter"/>
    <rule ref="category/java/bestpractices.xml/UnusedLocalVariable"/>
    <rule ref="category/java/bestpractices.xml/UnusedPrivateField"/>
    <rule ref="category/java/bestpractices.xml/UnusedPrivateMethod"/>
    <rule ref="category/java/codestyle.xml/ClassNamingConventions">
        <properties>
            <property name="abstractClassPattern" value="[A-Z][a-zA-Z0-9]*"/>
            <property name="annotationPattern" value="[A-Z][a-zA-Z0-9]*"/>
            <property name="classPattern" value="[A-Z][a-zA-Z0-9]*"/>
            <property name="enumPattern" value="[A-Z][a-zA-Z0-9]*"/>
            <property name="interfacePattern" value="[A-Z][a-zA-Z0-9]*"/>
            <property name="testClassPattern" value="^Test.*$|^[A-Z][a-zA-Z0-9]*Test(s|Case)?$"/
>
            <property name="utilityClassPattern" value="[A-Z][a-zA-Z0-9]*"/>
        </properties>
    </rule>
    <rule ref="category/java/codestyle.xml/ExtendsObject"/>
    <rule ref="category/java/codestyle.xml/FieldDeclarationsShouldBeAtStartOfClass"/>
    <rule ref="category/java/codestyle.xml/MethodNamingConventions">
        <properties>
            <property name="junit3TestPattern" value="test[A-Z0-9][a-zA-Z0-9]*"/>
            <property name="junit4TestPattern" value="[a-z][a-zA-Z0-9]*"/>
```

CODEX/.codacy/tools-configs/ruleset.xml

```
            <property name="junit5TestPattern" value="[a-z][a-zA-Z0-9]*"/>
            <property name="methodPattern" value="[a-z][a-zA-Z0-9]*"/>
            <property name="nativePattern" value="[a-z][a-zA-Z0-9]*"/>
            <property name="staticPattern" value="[a-z][a-zA-Z0-9]*"/>
        </properties>
    </rule>
    <rule ref="category/java/codestyle.xml/NoPackage"/>
    <rule ref="category/java/codestyle.xml/PackageCase"/>
    <rule ref="category/java/codestyle.xml/UnnecessaryConstructor"/>
    <rule ref="category/java/codestyle.xml/UnnecessaryFullyQualifiedName"/>
    <rule ref="category/java/codestyle.xml/UnnecessaryReturn"/>
    <rule ref="category/java/design.xml/AvoidThrowingNullPointerException"/>
    <rule ref="category/java/design.xml/AvoidThrowingRawExceptionTypes"/>
    <rule ref="category/java/design.xml/CollapsibleIfStatements"/>
    <rule ref="category/java/design.xml/ExcessiveParameterList"/>
    <rule ref="category/java/design.xml/LogicInversion"/>
    <rule ref="category/java/design.xml/NPathComplexity"/>
    <rule ref="category/java/design.xml/SimplifyBooleanExpressions"/>
    <rule ref="category/java/design.xml/SimplifyBooleanReturns"/>
    <rule ref="category/java/design.xml/SingularField"/>
    <rule ref="category/java/documentation.xml/UncommentedEmptyMethodBody"/>
    <rule ref="category/java/errorprone.xml/AssignmentToNonFinalStatic"/>
    <rule ref="category/java/errorprone.xml/AvoidAccessibilityAlteration"/>
    <rule ref="category/java/errorprone.xml/AvoidBranchingStatementAsLastInLoop"/>
    <rule ref="category/java/errorprone.xml/AvoidCallingFinalize"/>
    <rule ref="category/java/errorprone.xml/AvoidDecimalLiteralsInBigDecimalConstructor"/>
    <rule ref="category/java/errorprone.xml/AvoidInstanceofChecksInCatchClause"/>
    <rule ref="category/java/errorprone.xml/AvoidMultipleUnaryOperators"/>
    <rule ref="category/java/errorprone.xml/BrokenNullCheck"/>
    <rule ref="category/java/errorprone.xml/CheckSkipResult"/>
    <rule ref="category/java/errorprone.xml/CompareObjectsWithEquals">
        <properties>
            <property name="typesThatCompareByReference" value="java.lang.Enum,java.lang.Class"/
>
        </properties>
    </rule>
    <rule ref="category/java/errorprone.xml/DoNotHardCodeSDCard"/>
    <rule ref="category/java/errorprone.xml/DontUseFloatTypeForLoopIndices"/>
    <rule ref="category/java/errorprone.xml/EmptyFinalizer"/>
    <rule ref="category/java/errorprone.xml/EqualsNull"/>
    <rule ref="category/java/errorprone.xml/JumbledIncrementer"/>
    <rule ref="category/java/errorprone.xml/JUnitSpelling"/>
    <rule ref="category/java/errorprone.xml/JUnitStaticSuite"/>
    <rule ref="category/java/errorprone.xml/MethodWithSameNameAsEnclosingClass"/>
    <rule ref="category/java/errorprone.xml/MisplacedNullCheck"/>
    <rule ref="category/java/errorprone.xml/MissingStaticMethodInNonInstantiatableClass">
        <properties>
            <property name="annotations" value="org.springframework.beans.factory.annotation.Aut
owired,javax.inject.Inject,com.google.inject.Inject,lombok.Builder"/>
        </properties>
    </rule>
    <rule ref="category/java/errorprone.xml/NonStaticInitializer"/>
    <rule ref="category/java/errorprone.xml/ReturnFromFinallyBlock"/>
    <rule ref="category/java/errorprone.xml/UnconditionalIfStatement"/>
    <rule ref="category/java/errorprone.xml/UnnecessaryBooleanAssertion"/>
    <rule ref="category/java/errorprone.xml/UnnecessaryCaseChange"/>
    <rule ref="category/java/errorprone.xml/UseEqualsToCompareStrings"/>
    <rule ref="category/java/multithreading.xml/AvoidThreadGroup"/>
    <rule ref="category/java/multithreading.xml/DontCallThreadRun"/>
    <rule ref="category/java/performance.xml/StringInstantiation"/>
    <rule ref="category/java/performance.xml/StringToString"/>
    <rule ref="category/java/performance.xml/UseStringBufferLength"/>
    <rule ref="category/jsp/bestpractices.xml/DontNestJsfInJstlIteration"/>
    <rule ref="category/jsp/bestpractices.xml/NoClassAttribute"/>
    <rule ref="category/jsp/bestpractices.xml/NoHtmlComments"/>
    <rule ref="category/jsp/bestpractices.xml/NoJspForward"/>
    <rule ref="category/jsp/codestyle.xml/DuplicateJspImports"/>
    <rule ref="category/jsp/design.xml/NoInlineScript"/>
    <rule ref="category/jsp/design.xml/NoInlineStyleInformation"/>
    <rule ref="category/jsp/design.xml/NoLongScripts"/>
    <rule ref="category/jsp/design.xml/NoScriptlets"/>
```

CODEX/.codacy/tools-configs/ruleset.xml

```
    <rule ref="category/jsp/errorprone.xml/JspEncoding"/>
    <rule ref="category/jsp/security.xml/IframeMissingSrcAttribute"/>
    <rule ref="category/jsp/security.xml/NoUnsanitizedJSPExpression"/>
    <rule ref="category/plsql/bestpractices.xml/TomKytesDespair"/>
    <rule ref="category/plsql/codestyle.xml/MisplacedPragma"/>
    <rule ref="category/plsql/design.xml/ExcessiveParameterList"/>
    <rule ref="category/plsql/design.xml/TooManyMethods">
        <properties>
            <property name="maxmethods" value="1"/>
        </properties>
    </rule>
    <rule ref="category/plsql/errorprone.xml/TO_DATE_TO_CHAR"/>
    <rule ref="category/plsql/errorprone.xml/TO_DATEWithoutDateFormat"/>
    <rule ref="category/plsql/errorprone.xml/TO_TIMESTAMPWithoutDateFormat"/>
    <rule ref="category/pom/errorprone.xml/InvalidDependencyTypes">
        <properties>
            <property name="validTypes" value="pom,jar,maven-plugin,ejb,war,ear,rar,par"/>
        </properties>
    </rule>
    <rule ref="category/pom/errorprone.xml/ProjectVersionAsDependencyVersion"/>
    <rule ref="category/xml/errorprone.xml/MistypedCDATASection"/>
    <rule ref="category/xsl/codestyle.xml/UseConcatOnce"/>
    <rule ref="category/xsl/performance.xml/AvoidAxisNavigation">
        <properties>
            <property name="checkSelfDescendantAbreviation" value="false"/>
        </properties>
    </rule>
</ruleset>
```

CODEX/.codacy/tools-configs/ruleset.xml

SOURCE WITNESS 017

Semgrep

SHA-256 a2c286a52f0409d8bb885f17c317cc0fba1f0cfcad1599ed599bf6974ecc2aa4

```
rules:
    - id: bash.curl.security.curl-eval.curl-eval
      languages:
        - bash
      message: Data is being eval'd from a `curl` command. An attacker with control of the serve
r in the `curl` command could inject malicious code into the `eval`, resulting in a system comrp
omise. Avoid eval'ing untrusted data if you can. If you must do this, consider checking the SHA
sum of the content returned by the server to verify its integrity.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - bash
            - curl
      mode: taint
      pattern-sinks:
        - pattern: eval ...
      pattern-sources:
        - pattern: |
            $(curl ...)
        - pattern: |
            `curl ...`
      severity: WARNING
    - id: c.lang.security.insecure-use-gets-fn.insecure-use-gets-fn
      languages:
        - c
        - cpp
      message: Avoid 'gets()'. This function does not consider buffer boundaries and can lead to
 buffer overflows. Use 'fgets()' or 'gets_s()' instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-676: Use of Potentially Dangerous Function'
        impact: HIGH
        likelihood: LOW
        references:
            - https://us-cert.cisa.gov/bsi/articles/knowledge/coding-practices/fgets-and-gets_s
        subcategory:
            - audit
        technology:
            - c
            - cpp
      pattern: gets(...)
      severity: ERROR
    - id: c.lang.security.random-fd-exhaustion.random-fd-exhaustion
      languages:
        - c
        - cpp
      message: Call to 'read()' without error checking is susceptible to file descriptor exhaust
ion. Consider using the 'getrandom()' function.
      metadata:
        category: security
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        confidence: MEDIUM
        cwe:
            - 'CWE-774: Allocation of File Descriptors or Handles Without Limits or Throttling'
        impact: HIGH
        likelihood: LOW
        references:
            - https://lwn.net/Articles/606141/
        subcategory:
            - audit
        technology:
            - c
            - cpp
      pattern-either:
        - patterns:
            - pattern: |
                $FD = open("/dev/urandom", ...);
                ...
                read($FD, ...);
            - pattern-not: |
                $FD = open("/dev/urandom", ...);
                ...
                $BYTES_READ = read($FD, ...);
        - patterns:
            - pattern: |
                $FD = open("/dev/random", ...);
                ...
                read($FD, ...);
            - pattern-not: |
                $FD = open("/dev/random", ...);
                ...
                $BYTES_READ = read($FD, ...);
      severity: WARNING
    - id: clojure.lang.security.documentbuilderfactory-xxe.documentbuilderfactory-xxe
      languages:
        - clojure
      message: DOCTYPE declarations are enabled for javax.xml.parsers.SAXParserFactory. Without
prohibiting external entity declarations, this is vulnerable to XML external entity attacks. Dis
able this by setting the feature "http://apache.org/xml/features/disallow-doctype-decl" to true.
 Alternatively, allow DOCTYPE declarations and only prohibit external entities declarations. Thi
s can be done by setting the features "http://xml.org/sax/features/external-general-entities" an
d "http://xml.org/sax/features/external-parameter-entities" to false.
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://xerces.apache.org/xerces2-j/features.html
        source-rule-url: https://github.com/clj-holmes/clj-holmes-rules/blob/main/security/xxe-c
lojure-xml/xxe-clojure-xml.yml
        subcategory:
            - vuln
        technology:
            - clojure
            - xml
      patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-inside: |
            (ns ... (:require [clojure.xml :as ...]))
            ...
        - pattern-either:
            - pattern-inside: |
                (def ... ... ( ... ))
            - pattern-inside: |
                (defn ... ... ( ... ))
        - pattern-either:
            - pattern: (clojure.xml/parse $INPUT)
            - patterns:
                - pattern-inside: |
                    (doto (javax.xml.parsers.SAXParserFactory/newInstance) ...)
                - pattern: (.setFeature "http://apache.org/xml/features/disallow-doctype-decl" f
alse)
                - pattern-not-inside: |
                    (doto (javax.xml.parsers.SAXParserFactory/newInstance)
                      ...
                      (.setFeature "http://xml.org/sax/features/external-general-entities" false
)
                      ...
                      (.setFeature "http://xml.org/sax/features/external-parameter-entities" fal
se)
                      ...)
                - pattern-not-inside: |
                    (doto (javax.xml.parsers.SAXParserFactory/newInstance)
                      ...
                      (.setFeature "http://xml.org/sax/features/external-parameter-entities" fal
se)
                      ...
                      (.setFeature "http://xml.org/sax/features/external-general-entities" false
)
                      ...)
      severity: ERROR
    - id: clojure.lang.security.use-of-md5.use-of-md5
      languages:
        - clojure
      message: MD5 hash algorithm detected. This is not collision resistant and leads to easily-
cracked password hashes. Replace with current recommended hashing algorithms.
      metadata:
        author: Gabriel Marquet <gab.marquet@gmail.com>
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-328: Use of Weak Hash'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.h
tml
            - https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
        source-rule-url: https://github.com/clj-holmes/clj-holmes-rules/blob/main/security/weak-
hash-function-md5.yml
        subcategory:
            - vuln
        technology:
            - clojure
      pattern-either:
        - pattern: (MessageDigest/getInstance "MD5")
        - pattern: (MessageDigest/getInstance MessageDigestAlgorithms/MD5)
        - pattern: (MessageDigest/getInstance org.apache.commons.codec.digest.MessageDigestAlgor
ithms/MD5)
        - pattern: (java.security.MessageDigest/getInstance "MD5")
        - pattern: (java.security.MessageDigest/getInstance MessageDigestAlgorithms/MD5)
        - pattern: (java.security.MessageDigest/getInstance org.apache.commons.codec.digest.Mess
ageDigestAlgorithms/MD5)
      severity: WARNING
    - id: clojure.lang.security.use-of-sha1.use-of-sha1
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      languages:
        - clojure
      message: Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision
resistant and is therefore not suitable as a cryptographic signature. Instead, use PBKDF2 for pa
ssword hashing or SHA256 or SHA512 for other hash function applications.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
            - 'CWE-328: Use of Weak Hash'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.h
tml
            - https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - clojure
      patterns:
        - pattern-either:
            - pattern: (MessageDigest/getInstance $ALGO)
            - pattern: (java.security.MessageDigest/getInstance $ALGO)
        - metavariable-regex:
            metavariable: $ALGO
            regex: (((org\.apache\.commons\.codec\.digest\.)?MessageDigestAlgorithms/)?"?(SHA-1|
SHA1)"?)
      severity: WARNING
    - id: csharp.dotnet.security.audit.ldap-injection.ldap-injection
      languages:
        - csharp
      message: LDAP queries are constructed dynamically on user-controlled input. This vulnerabi
lity in code could lead to an arbitrary LDAP query execution.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-90: Improper Neutralization of Special Elements used in an LDAP Query (''LDAP
 Injection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection/
            - https://cwe.mitre.org/data/definitions/90
            - https://cheatsheetseries.owasp.org/cheatsheets/LDAP_Injection_Prevention_Cheat_She
et.html#safe-c-sharp-net-tba-example
        subcategory:
            - vuln
        technology:
            - .net
      mode: taint
      options:
        taint_unify_mvars: true
      pattern-sanitizers:
        - pattern-either:
            - pattern: Regex.Replace($INPUT, ...)
            - pattern: $ENCODER.LdapFilterEncode($INPUT)
            - pattern: $ENCODER.LdapDistinguishedNameEncode($INPUT)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: $S.Filter = ... + $INPUT + ...
                - pattern: $S.Filter = String.Format(...,$INPUT)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              - pattern: $S.Filter = String.Concat(...,$INPUT)
      pattern-sources:
        - patterns:
            - focus-metavariable: $INPUT
            - pattern-inside: $T $M($INPUT,...) {...}
      severity: ERROR
    - id: csharp.dotnet.security.audit.mass-assignment.mass-assignment
      languages:
        - csharp
      message: Mass assignment or Autobinding vulnerability in code allows an attacker to execut
e over-posting attacks, which could create a new parameter in the binding request and manipulate
 the underlying object in the application.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attr
ibutes'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://cwe.mitre.org/data/definitions/915.html
            - https://github.com/OWASP/API-Security/blob/master/2019/en/src/0xa6-mass-assignment
.md
        subcategory:
            - vuln
        technology:
            - .net
      mode: taint
      pattern-sinks:
        - pattern: View(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    public IActionResult $METHOD(..., $TYPE $ARG, ...){
                      ...
                    }
                - pattern: |
                    public ActionResult $METHOD(..., $TYPE $ARG, ...){
                      ...
                    }
            - pattern-inside: |
                using Microsoft.AspNetCore.Mvc;
                ...
            - pattern-not: |
                public IActionResult $METHOD(..., [Bind(...)] $TYPE $ARG, ...){
                  ...
                }
            - pattern-not: |
                public ActionResult $METHOD(..., [Bind(...)] $TYPE $ARG, ...){
                  ...
                }
            - focus-metavariable: $ARG
      severity: WARNING
    - id: csharp.dotnet.security.audit.missing-or-broken-authorization.missing-or-broken-authori
zation
      languages:
        - csharp
      message: Anonymous access shouldn't be allowed unless explicit by design. Access control c
hecks are missing and potentially can be bypassed. This finding violates the principle of least
privilege or deny by default, where access should only be permitted for a specific set of roles
or conforms to a custom policy or users.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-862: Missing Authorization'
        cwe2021-top25: true
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe2022-top25: true
        cwe2023-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
            - https://cwe.mitre.org/data/definitions/862.html
            - https://docs.microsoft.com/en-us/aspnet/core/security/authorization/simple?view=as
pnetcore-7.0
        subcategory:
            - vuln
        technology:
            - .net
            - mvc
      patterns:
        - pattern: |
            public class $CLASS : Controller {
              ...
            }
        - pattern-inside: |
            using Microsoft.AspNetCore.Mvc;
            ...
        - pattern-not: |
            [AllowAnonymous]
            public class $CLASS : Controller {
              ...
            }
        - pattern-not: |
            [Authorize]
            public class $CLASS : Controller {
              ...
            }
        - pattern-not: |
            [Authorize(Roles = ...)]
            public class $CLASS : Controller {
              ...
            }
        - pattern-not: |
            [Authorize(Policy = ...)]
            public class $CLASS : Controller {
              ...
            }
      severity: INFO
    - id: csharp.dotnet.security.audit.open-directory-listing.open-directory-listing
      languages:
        - csharp
      message: An open directory listing is potentially exposed, potentially revealing sensitive
 information to attackers.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-548: Exposure of Information Through Directory Listing'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A06:2017 - Security Misconfiguration
            - A01:2021 - Broken Access Control
        references:
            - https://cwe.mitre.org/data/definitions/548.html
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
            - https://docs.microsoft.com/en-us/aspnet/core/fundamentals/static-files?view=aspnet
core-7.0#directory-browsing
        subcategory:
            - vuln
        technology:
            - .net
            - mvc
      patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-either:
            - pattern: (IApplicationBuilder $APP).UseDirectoryBrowser(...);
            - pattern: $BUILDER.Services.AddDirectoryBrowser(...);
        - pattern-inside: |
            public void Configure(...) {
              ...
            }
      severity: INFO
    - id: csharp.dotnet.security.audit.xpath-injection.xpath-injection
      languages:
        - csharp
      message: XPath queries are constructed dynamically on user-controlled input. This vulnerab
ility in code could lead to an XPath Injection exploitation.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-643: Improper Neutralization of Data within XPath Expressions (''XPath Inject
ion'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection/
            - https://cwe.mitre.org/data/definitions/643.html
        subcategory:
            - vuln
        technology:
            - .net
      mode: taint
      pattern-sinks:
        - pattern-either:
            - pattern: XPathExpression $EXPR = $NAV.Compile("..." + $INPUT + "...");
            - pattern: var $EXPR = $NAV.Compile("..." + $INPUT + "...");
            - pattern: XPathNodeIterator $NODE = $NAV.Select("..." + $INPUT + "...");
            - pattern: var $NODE = $NAV.Select("..." + $INPUT + "...");
            - pattern: Object $OBJ = $NAV.Evaluate("..." + $INPUT + "...");
            - pattern: var $OBJ = $NAV.Evaluate("..." + $INPUT + "...");
      pattern-sources:
        - pattern-either:
            - pattern: $T $M($INPUT,...) {...}
            - pattern: |
                $T $M(...) {
                  ...
                  string $INPUT;
                }
      severity: ERROR
    - id: csharp.dotnet.security.razor-template-injection.razor-template-injection
      languages:
        - csharp
      message: User-controllable string passed to Razor.Parse. This leads directly to code execu
tion in the context of the process.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://clement.notin.org/blog/2020/04/15/Server-Side-Template-Injection-(SSTI)-in
-ASP.NET-Razor/
        subcategory:
            - vuln
        technology:
            - .net
            - razor
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - asp
      mode: taint
      pattern-sanitizers:
        - not_conflicting: true
          pattern: $F(...)
      pattern-sinks:
        - pattern: |
            Razor.Parse(...)
      pattern-sources:
        - patterns:
            - focus-metavariable: $ARG
            - pattern-inside: |
                public ActionResult $METHOD(..., string $ARG,...){...}
      severity: WARNING
    - id: csharp.dotnet.security.use_deprecated_cipher_algorithm.use_deprecated_cipher_algorithm
      languages:
        - csharp
      message: Usage of deprecated cipher algorithm detected. Use Aes or ChaCha20Poly1305 instea
d.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.des?view
=net-6.0#remarks
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.rc2?view
=net-6.0#remarks
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.aes?view
=net-6.0
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.chacha20
poly1305?view=net-6.0
        subcategory:
            - vuln
        technology:
            - .net
      patterns:
        - pattern: $KEYTYPE.Create(...);
        - metavariable-pattern:
            metavariable: $KEYTYPE
            pattern-either:
                - pattern: DES
                - pattern: RC2
      severity: ERROR
    - id: csharp.dotnet.security.use_ecb_mode.use_ecb_mode
      languages:
        - csharp
      message: Usage of the insecure ECB mode detected. You should use an authenticated encrypti
on mode instead, which is implemented by the classes AesGcm or ChaCha20Poly1305.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.chacha20
poly1305?view=net-6.0
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.aesgcm?v
iew=net-6.0
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.ciphermo
de?view=net-6.0
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.h
tml#cipher-modes
        subcategory:
            - vuln
        technology:
            - .net
      patterns:
        - pattern-either:
            - pattern: ($KEYTYPE $KEY).EncryptEcb(...);
            - pattern: ($KEYTYPE $KEY).DecryptEcb(...);
            - pattern: ($KEYTYPE $KEY).Mode = CipherMode.ECB;
        - metavariable-pattern:
            metavariable: $KEYTYPE
            pattern-either:
                - pattern: SymmetricAlgorithm
                - pattern: Aes
                - pattern: Rijndael
                - pattern: DES
                - pattern: TripleDES
                - pattern: RC2
      severity: WARNING
    - id: csharp.dotnet.security.use_weak_rng_for_keygeneration.use_weak_rng_for_keygeneration
      languages:
        - csharp
      message: You are using an insecure random number generator (RNG) to create a cryptographic
 key. System.Random must never be used for cryptographic purposes. Use System.Security.Cryptogra
phy.RandomNumberGenerator instead.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-338: Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG)'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://learn.microsoft.com/en-us/dotnet/api/system.random?view=net-6.0#remarks
            - https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.randomnu
mbergenerator?view=net-6.0
            - https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.aesgcm?v
iew=net-6.0#constructors
            - https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.symmetri
calgorithm.key?view=net-6.0#system-security-cryptography-symmetricalgorithm-key
        subcategory:
            - vuln
        technology:
            - .net
      mode: taint
      pattern-sinks:
        - pattern-either:
            - patterns:
                - pattern: ($KEYTYPE $CIPHER).Key = $SINK;
                - focus-metavariable: $SINK
                - metavariable-pattern:
                    metavariable: $KEYTYPE
                    pattern-either:
                        - pattern: SymmetricAlgorithm
                        - pattern: Aes
                        - pattern: Rijndael
                        - pattern: DES
                        - pattern: TripleDES
                        - pattern: RC2
            - pattern: new AesGcm(...)
            - pattern: new AesCcm(...)
            - pattern: new ChaCha20Poly1305(...)
      pattern-sources:
        - patterns:
            - pattern-inside: (System.Random $RNG).NextBytes($KEY); ...
            - pattern: $KEY
      severity: ERROR
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: csharp.dotnet.security.use_weak_rsa_encryption_padding.use_weak_rsa_encryption_padding
      languages:
        - csharp
      message: You are using the outdated PKCS#1 v1.5 encryption padding for your RSA key. Use t
he OAEP padding instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-780: Use of RSA Algorithm without OAEP'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.rsapkcs1
keyexchangeformatter
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.rsaoaepk
eyexchangeformatter
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.rsapkcs1
keyexchangedeformatter
            - https://learn.microsoft.com/en-gb/dotnet/api/system.security.cryptography.rsaoaepk
eyexchangedeformatter
        subcategory:
            - vuln
        technology:
            - .net
      pattern-either:
        - pattern: (RSAPKCS1KeyExchangeFormatter $FORMATER).CreateKeyExchange(...);
        - pattern: (RSAPKCS1KeyExchangeDeformatter $DEFORMATER).DecryptKeyExchange(...);
      severity: WARNING
    - id: csharp.lang.correctness.double.double-epsilon-equality.correctness-double-epsilon-equa
lity
      languages:
        - csharp
      message: Double.Epsilon is defined by .NET as the smallest value that can be added to or s
ubtracted from a zero-value Double. It is unsuitable for equality comparisons of non-zero Double
 values. Furthermore, the value of Double.Epsilon is framework and processor architecture depend
ent. Wherever possible, developers should prefer the framework Equals() method over custom equal
ity implementations.
      metadata:
        category: correctness
        confidence: MEDIUM
        references:
            - https://docs.microsoft.com/en-us/dotnet/api/system.double?view=net-6.0#testing-for
-equality
            - https://docs.microsoft.com/en-us/dotnet/api/system.double.epsilon?view=net-6.0#pla
tform-notes
        technology:
            - .net
      patterns:
        - pattern: |
            $V1 - $V2
        - pattern-either:
            - pattern-inside: |
                ... <= Double.Epsilon
            - pattern-inside: |
                Double.Epsilon <= ...
        - pattern-not-inside: |
            double $V1 = 0;
            ...
        - pattern-not-inside: |
            double $V2 = 0;
            ...
        - pattern-not-inside: |
            $V1 = 0;
            ...
        - pattern-not-inside: |
            $V2 = 0;
            ...
      severity: WARNING
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: csharp.lang.correctness.regioninfo.regioninfo-interop.correctness-regioninfo-interop
      languages:
        - csharp
      message: Potential inter-process write of RegionInfo $RI via $PIPESTREAM $P that was insta
ntiated with a two-character culture code $REGION.  Per .NET documentation, if you want to persi
st a RegionInfo object or communicate it between processes, you should instantiate it by using a
 full culture name rather than a two-letter ISO region code.
      metadata:
        category: correctness
        confidence: MEDIUM
        references:
            - https://docs.microsoft.com/en-us/dotnet/api/system.globalization.regioninfo.twolet
terisoregionname?view=net-6.0#remarks
        technology:
            - .net
      patterns:
        - pattern-either:
            - pattern: |
                $WRITER.Write($RI);
            - pattern: |
                $WRITER.WriteAsync($RI);
            - pattern: |
                $WRITER.WriteLine($RI);
            - pattern: |
                $WRITER.WriteLineAsync($RI);
        - pattern-inside: |
            RegionInfo $RI = new RegionInfo($REGION);
            ...
            using($PIPESTREAM $P = ...){
              ...
            }
        - metavariable-regex:
            metavariable: $REGION
            regex: ^"\w{2}"$
        - metavariable-regex:
            metavariable: $PIPESTREAM
            regex: (Anonymous|Named)Pipe(Server|Client)Stream
      severity: WARNING
    - fix: SslCertificateTrust.$METHOD($COLLECTION,false)
      id: csharp.lang.correctness.sslcertificatetrust.sslcertificatetrust-handshake-no-trust.cor
rectness-sslcertificatetrust-handshake-no-trust
      languages:
        - csharp
      message: Sending the trusted CA list increases the size of the handshake request and can l
eak system configuration information.
      metadata:
        category: correctness
        confidence: HIGH
        cwe: 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://docs.microsoft.com/en-us/dotnet/api/system.net.security.sslcertificatetrus
t.createforx509collection?view=net-6.0#remarks
            - https://docs.microsoft.com/en-us/dotnet/api/system.net.security.sslcertificatetrus
t.createforx509store?view=net-6.0#remarks
        technology:
            - .net
      patterns:
        - pattern-either:
            - pattern: SslCertificateTrust.$METHOD($COLLECTION,sendTrustInHandshake=true)
            - pattern: SslCertificateTrust.$METHOD($COLLECTION,true)
        - metavariable-regex:
            metavariable: $METHOD
            regex: CreateForX509(Collection|Store)
      severity: WARNING
    - fix: |
        true
      id: csharp.lang.security.ad.jwt-tokenvalidationparameters-no-expiry-validation.jwt-tokenva
lidationparameters-no-expiry-validation
      languages:
        - csharp
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: The TokenValidationParameters.$LIFETIME is set to $FALSE, this means the JWT toke
ns lifetime is not validated. This can lead to an JWT token being used after it has expired, whi
ch has security implications. It is recommended to validate the JWT lifetime to ensure only vali
d tokens are used.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-613: Insufficient Session Expiration'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A02:2017 - Broken Authentication
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
            - https://cwe.mitre.org/data/definitions/613.html
            - https://docs.microsoft.com/en-us/dotnet/api/microsoft.identitymodel.tokens.tokenva
lidationparameters?view=azure-dotnet
        subcategory:
            - audit
        technology:
            - csharp
      patterns:
        - pattern-either:
            - patterns:
                - pattern: $LIFETIME = $FALSE
                - pattern-inside: new TokenValidationParameters {...}
            - patterns:
                - pattern: |
                    (TokenValidationParameters $OPTS). ... .$LIFETIME = $FALSE
        - metavariable-regex:
            metavariable: $LIFETIME
            regex: (RequireExpirationTime|ValidateLifetime)
        - metavariable-regex:
            metavariable: $FALSE
            regex: (false)
        - focus-metavariable: $FALSE
      severity: WARNING
    - id: csharp.lang.security.cryptography.x509-subject-name-validation.x509-subject-name-valid
ation
      languages:
        - csharp
      message: Validating certificates based on subject name is bad practice. Use the X509Certif
icate2.Verify() method instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-295: Improper Certificate Validation'
        impact: LOW
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://docs.microsoft.com/en-us/dotnet/api/system.identitymodel.tokens.issuername
registry?view=netframework-4.8
        subcategory:
            - vuln
        technology:
            - .net
      patterns:
        - pattern-inside: |
            using System.IdentityModel.Tokens;
            ...
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern-inside: |
                        X509SecurityToken $TOK = $RHS;
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    ...
                - pattern-inside: |
                    $T $M(..., X509SecurityToken $TOK, ...) {
                        ...
                    }
            - metavariable-pattern:
                metavariable: $RHS
                pattern-either:
                    - pattern: $T as X509SecurityToken
                    - pattern: new X509SecurityToken(...)
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    X509Certificate2 $CERT = new X509Certificate2(...);
                    ...
                - pattern-inside: |
                    $T $M(..., X509Certificate2 $CERT, ...) {
                        ...
                    }
                - pattern-inside: |
                    foreach (X509Certificate2 $CERT in $COLLECTION) {
                        ...
                    }
      - patterns:
          - pattern-either:
              - pattern: String.Equals($NAME, "...")
              - pattern: String.Equals("...", $NAME)
              - pattern: $NAME.Equals("...")
              - pattern: $NAME == "..."
              - pattern: $NAME != "..."
              - pattern: |
                  "..." == $NAME
              - pattern: |
                  "..." != $NAME
          - metavariable-pattern:
              metavariable: $NAME
              pattern-either:
                  - pattern: $TOK.Certificate.SubjectName.Name
                  - pattern: $CERT.SubjectName.Name
                  - pattern: $CERT.GetNameInfo(...)
    severity: WARNING
  - fix: RequireSignedTokens = true
    id: csharp.lang.security.cryptography.unsigned-security-token.unsigned-security-token
    languages:
      - csharp
    message: Accepting unsigned security tokens as valid security tokens allows an attacker to
remove its signature and potentially forge an identity. As a fix, set RequireSignedTokens to be
true.
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
          - 'CWE-347: Improper Verification of Cryptographic Signature'
      impact: MEDIUM
      likelihood: MEDIUM
      owasp:
          - A02:2021 - Cryptographic Failures
      references:
          - https://owasp.org/Top10/A01_2021-Broken_Access_Control/
          - https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
          - https://cwe.mitre.org/data/definitions/347
      subcategory:
          - vuln
      technology:
          - csharp
    patterns:
      - pattern: RequireSignedTokens = false
      - pattern-inside: |
          new TokenValidationParameters {
            ...
          }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: ERROR
    - id: csharp.lang.security.filesystem.unsafe-path-combine.unsafe-path-combine
      languages:
        - csharp
      message: String argument $A is used to read or write data from a file via Path.Combine wit
hout direct sanitization via Path.GetFileName. If the path is user-supplied data this can lead t
o path traversal.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://www.praetorian.com/blog/pathcombine-security-issues-in-aspnet-applications
/
            - https://docs.microsoft.com/en-us/dotnet/api/system.io.path.combine?view=net-6.0#re
marks
        subcategory:
            - vuln
        technology:
            - .net
      mode: taint
      pattern-sanitizers:
        - pattern: |
            Path.GetFileName(...)
        - patterns:
            - pattern-inside: |
                $X = Path.GetFileName(...);
                ...
            - pattern: $X
        - patterns:
            - pattern: $X
            - pattern-inside: |
                if(<... Path.GetFileName($X) != $X ...>){
                  ...
                  throw new $EXCEPTION(...);
                }
                ...
      pattern-sinks:
        - patterns:
            - focus-metavariable: $X
            - pattern: |
                File.$METHOD($X,...)
            - metavariable-regex:
                metavariable: $METHOD
                regex: (?i)^(read|write)
      pattern-sources:
        - patterns:
            - pattern: $A
            - pattern-inside: |
                Path.Combine(...,$A,...)
            - pattern-inside: |
                public $TYPE $M(...,$A,...){...}
            - pattern-not-inside: |
                <... Path.GetFileName($A) != $A ...>
      severity: WARNING
    - id: csharp.lang.security.http.http-listener-wildcard-bindings.http-listener-wildcard-bindi
ngs
      languages:
        - C#
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: The top level wildcard bindings $PREFIX leaves your application open to security
vulnerabilities and give attackers more control over where traffic is routed. If you must use wi
ldcards, consider using subdomain wildcard binding. For example, you can use "*.asdf.gov" if you
 own all of "asdf.gov".
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-706: Use of Incorrectly-Resolved Name or Reference'
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://docs.microsoft.com/en-us/dotnet/api/system.net.httplistener?view=net-6.0
        subcategory:
            - vuln
        technology:
            - .net
      patterns:
        - pattern-inside: |
            using System.Net;
            ...
        - pattern: $LISTENER.Prefixes.Add("$PREFIX")
        - metavariable-regex:
            metavariable: $PREFIX
            regex: (http|https)://(\*|\+)(.[a-zA-Z]{2,})?:[0-9]+
      severity: WARNING
    - id: csharp.lang.security.insecure-deserialization.binary-formatter.insecure-binaryformatte
r-deserialization
      languages:
        - C#
      message: The BinaryFormatter type is dangerous and is not recommended for data processing.
 Applications should stop using BinaryFormatter as soon as possible, even if they believe the da
ta they're processing to be trustworthy. BinaryFormatter is insecure and can't be made secure
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://docs.microsoft.com/en-us/dotnet/standard/serialization/binaryformatter-sec
urity-guide
        subcategory:
            - vuln
        technology:
            - .net
      patterns:
        - pattern-inside: |
            using System.Runtime.Serialization.Formatters.Binary;
            ...
        - pattern: |
            new BinaryFormatter();
      severity: WARNING
    - id: csharp.lang.security.insecure-deserialization.fs-pickler.insecure-fspickler-deserializ
ation
      languages:
        - C#
      message: The FsPickler is dangerous and is not recommended for data processing. Default co
nfiguration tend to insecure deserialization vulnerability.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://mbraceproject.github.io/FsPickler/tutorial.html#Disabling-Subtype-Resoluti
on
        subcategory:
            - vuln
        technology:
            - .net
      patterns:
        - pattern-inside: |
            using MBrace.FsPickler.Json;
            ...
        - pattern: |
            FsPickler.CreateJsonSerializer();
      severity: WARNING
    - id: csharp.lang.security.insecure-deserialization.los-formatter.insecure-losformatter-dese
rialization
      languages:
        - C#
      message: The LosFormatter type is dangerous and is not recommended for data processing. Ap
plications should stop using LosFormatter as soon as possible, even if they believe the data the
y're processing to be trustworthy. LosFormatter is insecure and can't be made secure
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://docs.microsoft.com/en-us/dotnet/api/system.web.ui.losformatter?view=netfra
mework-4.8
        subcategory:
            - vuln
        technology:
            - .net
      patterns:
        - pattern-inside: |
            using System.Web.UI;
            ...
        - pattern: |
            new LosFormatter();
      severity: WARNING
    - id: csharp.lang.security.insecure-deserialization.net-data-contract.insecure-netdatacontra
ct-deserialization
      languages:
        - C#
      message: The NetDataContractSerializer type is dangerous and is not recommended for data p
rocessing. Applications should stop using NetDataContractSerializer as soon as possible, even if
 they believe the data they're processing to be trustworthy. NetDataContractSerializer is insecu
re and can't be made secure
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: LOW
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.netdataco
ntractserializer?view=netframework-4.8#security
        subcategory:
            - vuln
        technology:
            - .net
      patterns:
        - pattern-inside: |
            using System.Runtime.Serialization;
            ...
        - pattern: |
            new NetDataContractSerializer();
      severity: WARNING
    - id: csharp.lang.security.insecure-deserialization.soap-formatter.insecure-soapformatter-de
serialization
      languages:
        - C#
      message: The SoapFormatter type is dangerous and is not recommended for data processing. A
pplications should stop using SoapFormatter as soon as possible, even if they believe the data t
hey're processing to be trustworthy. SoapFormatter is insecure and can't be made secure
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.formatter
s.soap.soapformatter?view=netframework-4.8#remarks
        subcategory:
            - vuln
        technology:
            - .net
      patterns:
        - pattern-inside: |
            using System.Runtime.Serialization.Formatters.Soap;
            ...
        - pattern: |
            new SoapFormatter();
      severity: WARNING
    - id: csharp.lang.security.regular-expression-dos.regular-expression-dos-infinite-timeout.re
gular-expression-dos-infinite-timeout
      languages:
        - C#
      message: 'Specifying the regex timeout leaves the system vulnerable to a regex-based Denia
l of Service (DoS) attack. Consider setting the timeout to a short amount of time like 2 or 3 se
conds. If you are sure you need an infinite timeout, double check that your context meets the co
nditions outlined in the "Notes to Callers" section at the bottom of this page: https://docs.mic
rosoft.com/en-us/dotnet/api/system.text.regularexpressions.regex.-ctor?view=net-6.0'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1333: Inefficient Regular Expression Complexity'
        impact: MEDIUM
        likelihood: LOW
        owasp: A01:2017 - Injection
        references:
            - https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReD
oS
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://docs.microsoft.com/en-us/dotnet/api/system.text.regularexpressions.regex.i
nfinitematchtimeout
            - https://docs.microsoft.com/en-us/dotnet/api/system.text.regularexpressions.regex.-
ctor?view=net-6.0
        subcategory:
            - audit
        technology:
            - .net
      patterns:
        - pattern-inside: |
            using System.Text.RegularExpressions;
            ...
        - pattern-either:
            - pattern: new Regex(..., TimeSpan.InfiniteMatchTimeout)
            - patterns:
                - pattern: new Regex(..., TimeSpan.FromSeconds($TIME))
                - metavariable-comparison:
                    comparison: $TIME > 5
                    metavariable: $TIME
            - pattern: new Regex(..., TimeSpan.FromMinutes(...))
            - pattern: new Regex(..., TimeSpan.FromHours(...))
      severity: WARNING
    - id: csharp.lang.security.regular-expression-dos.regular-expression-dos.regular-expression-
dos
      languages:
        - C#
      message: When using `System.Text.RegularExpressions` to process untrusted input, pass a ti
meout.  A malicious user can provide input to `RegularExpressions` that abuses the backtracking
behaviour of this regular expression engine. This will lead to excessive CPU usage, causing a De
nial-of-Service attack
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1333: Inefficient Regular Expression Complexity'
        impact: MEDIUM
        likelihood: LOW
        owasp: A01:2017 - Injection
        references:
            - https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReD
oS
            - https://docs.microsoft.com/en-us/dotnet/standard/base-types/regular-expressions#re
gular-expression-examples
        subcategory:
            - audit
        technology:
            - .net
      patterns:
        - pattern-inside: |
            using System.Text.RegularExpressions;
            ...
        - pattern-either:
            - pattern: |
                public $T $F($X)
                {
                  Regex $Y = new Regex($P);
                  ...
                  $Y.Match($X);
                }
            - pattern: |
                public $T $F($X)
                {
                  Regex $Y = new Regex($P, $O);
                  ...
                  $Y.Match($X);
                }
            - pattern: |
                public $T $F($X)
                {
                  ... Regex.Match($X, $P);
                }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                public $T $F($X)
                {
                  ... Regex.Match($X, $P, $O);
                }
      severity: WARNING
    - id: csharp.lang.security.sqli.csharp-sqli.csharp-sqli
      languages:
        - csharp
      message: Detected a formatted string in a SQL statement. This could lead to SQL injection
if variables in the SQL statement are not properly sanitized. Use a prepared statements instead.
 You can obtain a PreparedStatement using 'SqlCommand' and 'SqlParameter'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - audit
        technology:
            - csharp
      mode: taint
      pattern-propagators:
        - from: $X
          pattern: (StringBuilder $B).$ANY(...,(string $X),...)
          to: $B
      pattern-sanitizers:
        - by-side-effect: true
          pattern-either:
            - pattern: |
                $CMD.Parameters.add(...)
            - pattern: |
                $CMD.Parameters[$IDX] = ...
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: |
                        new $PATTERN($CMD,...)
                    - focus-metavariable: $CMD
                - pattern: |
                    $CMD.$PATTERN = ...;
            - metavariable-regex:
                metavariable: $PATTERN
                regex: ^(SqlCommand|CommandText|OleDbCommand|OdbcCommand|OracleCommand)$
      pattern-sources:
        - patterns:
            - pattern: |
                (string $X)
            - pattern-not: |
                "..."
      severity: ERROR
    - id: csharp.lang.security.stacktrace-disclosure.stacktrace-disclosure
      languages:
        - csharp
      message: Stacktrace information is displayed in a non-Development environment. Accidentall
y disclosing sensitive stack trace information in a production environment aids an attacker in r
econnaissance and information gathering.
      metadata:
        category: security
        confidence: HIGH
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe:
            - 'CWE-209: Generation of Error Message Containing Sensitive Information'
        impact: LOW
        likelihood: LOW
        owasp:
            - A06:2017 - Security Misconfiguration
            - A04:2021 - Insecure Design
        references:
            - https://cwe.mitre.org/data/definitions/209.html
            - https://owasp.org/Top10/A04_2021-Insecure_Design/
        subcategory:
            - audit
        technology:
            - csharp
      patterns:
        - pattern: $APP.UseDeveloperExceptionPage(...);
        - pattern-not-inside: "if ($ENV.IsDevelopment(...)) {\n  ... \n  $APP.UseDeveloperExcept
ionPage(...); \n  ...\n}\n"
      severity: WARNING
    - id: csharp.lang.security.xxe.xmldocument-unsafe-parser-override.xmldocument-unsafe-parser-
override
      languages:
        - csharp
      message: XmlReaderSettings found with DtdProcessing.Parse on an XmlReader handling a strin
g argument from a public method. Enabling Document Type Definition (DTD) parsing may cause XML E
xternal Entity (XXE) injection if supplied with user-controllable data.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://www.jardinesoftware.net/2016/05/26/xxe-and-net/
            - https://docs.microsoft.com/en-us/dotnet/api/system.xml.xmldocument.xmlresolver?vie
w=net-6.0#remarks
        subcategory:
            - vuln
        technology:
            - .net
            - xml
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                $XMLDOCUMENT.$METHOD(...)
            - pattern-inside: "XmlDocument $XMLDOCUMENT = new XmlDocument(...);\n...\n$XMLDOCUME
NT.XmlResolver = new XmlUrlResolver(...);\n...  \n"
      pattern-sources:
        - patterns:
            - focus-metavariable: $ARG
            - pattern-inside: |
                public $T $M(...,string $ARG,...){...}
      severity: WARNING
    - id: csharp.lang.security.xxe.xmlreadersettings-unsafe-parser-override.xmlreadersettings-un
safe-parser-override
      languages:
        - csharp
      message: XmlReaderSettings found with DtdProcessing.Parse on an XmlReader handling a strin
g argument from a public method. Enabling Document Type Definition (DTD) parsing may cause XML E
xternal Entity (XXE) injection if supplied with user-controllable data.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://www.jardinesoftware.net/2016/05/26/xxe-and-net/
            - https://docs.microsoft.com/en-us/dotnet/api/system.xml.xmldocument.xmlresolver?vie
w=net-6.0#remarks
        subcategory:
            - vuln
        technology:
            - .net
            - xml
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                XmlReader $READER = XmlReader.Create(...,$RS,...);
            - pattern-inside: "XmlReaderSettings $RS = new XmlReaderSettings();\n...\n$RS.DtdPro
cessing = DtdProcessing.Parse;\n...        \n"
      pattern-sources:
        - patterns:
            - focus-metavariable: $ARG
            - pattern-inside: |
                public $T $M(...,string $ARG,...){...}
      severity: WARNING
    - id: csharp.lang.security.xxe.xmltextreader-unsafe-defaults.xmltextreader-unsafe-defaults
      languages:
        - csharp
      message: XmlReaderSettings found with DtdProcessing.Parse on an XmlReader handling a strin
g argument from a public method. Enabling Document Type Definition (DTD) parsing may cause XML E
xternal Entity (XXE) injection if supplied with user-controllable data.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://www.jardinesoftware.net/2016/05/26/xxe-and-net/
            - https://docs.microsoft.com/en-us/dotnet/api/system.xml.xmldocument.xmlresolver?vie
w=net-6.0#remarks
        subcategory:
            - vuln
        technology:
            - .net
            - xml
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                $READER.$METHOD(...)
            - pattern-not-inside: |
                $READER.DtdProcessing = DtdProcessing.Prohibit;
                ...
            - pattern-inside: |
                XmlTextReader $READER = new XmlTextReader(...);
                ...
      pattern-sources:
        - patterns:
            - focus-metavariable: $ARG
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern-inside: |
                public $T $M(...,string $ARG,...){...}
      severity: WARNING
    - id: dockerfile.security.last-user-is-root.last-user-is-root
      languages:
        - dockerfile
      message: The last user in the container is 'root'. This is a security hazard because if an
 attacker gains control of the container they will have root access. Switch back to another user
 after running commands as 'root'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-269: Improper Privilege Management'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A04:2021 - Insecure Design
        references:
            - https://github.com/hadolint/hadolint/wiki/DL3002
        source-rule-url: https://github.com/hadolint/hadolint/wiki/DL3002
        subcategory:
            - audit
        technology:
            - dockerfile
      patterns:
        - pattern: USER root
        - pattern-not-inside:
            patterns:
                - pattern: |
                    USER root
                    ...
                    USER $X
                - metavariable-pattern:
                    metavariable: $X
                    patterns:
                        - pattern-not: root
      severity: ERROR
    - fix: |
        USER non-root
        ENTRYPOINT $...VARS
      id: dockerfile.security.missing-user-entrypoint.missing-user-entrypoint
      languages:
        - dockerfile
      message: By not specifying a USER, a program in the container may run as 'root'. This is a
 security hazard. If an attacker can control a process running as root, they may have control ov
er the container. Ensure that the last USER in a Dockerfile is a USER other than 'root'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-269: Improper Privilege Management'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        subcategory:
            - audit
        technology:
            - dockerfile
      patterns:
        - pattern: |
            ENTRYPOINT $...VARS
        - pattern-not-inside: |
            USER $USER
            ...
      severity: ERROR
    - fix: |
        USER non-root
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        CMD $...VARS
      id: dockerfile.security.missing-user.missing-user
      languages:
        - dockerfile
      message: By not specifying a USER, a program in the container may run as 'root'. This is a
 security hazard. If an attacker can control a process running as root, they may have control ov
er the container. Ensure that the last USER in a Dockerfile is a USER other than 'root'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-269: Improper Privilege Management'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        subcategory:
            - audit
        technology:
            - dockerfile
      patterns:
        - pattern: |
            CMD $...VARS
        - pattern-not-inside: |
            USER $USER
            ...
      severity: ERROR
    - id: dockerfile.security.no-sudo-in-dockerfile.no-sudo-in-dockerfile
      languages:
        - dockerfile
      message: Avoid using sudo in Dockerfiles. Running processes as a non-root user can help  r
educe the potential impact of configuration errors and security vulnerabilities.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-250: Execution with Unnecessary Privileges'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://cwe.mitre.org/data/definitions/250.html
            - https://docs.docker.com/develop/develop-images/dockerfile_best-practices/#user
        subcategory:
            - audit
        technology:
            - dockerfile
      patterns:
        - pattern: |
            RUN sudo ...
      severity: WARNING
    - id: generic.secrets.security.detected-stripe-restricted-api-key.detected-stripe-restricted
-api-key
      languages:
        - regex
      message: Stripe Restricted API Key detected
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: LOW
        likelihood: LOW
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures
        source-rule-url: https://github.com/dxa4481/truffleHogRegexes/blob/master/truffleHogRege
xes/regexes.json
        subcategory:
            - audit
        technology:
            - secrets
            - stripe
      pattern-regex: rk_live_[0-9a-zA-Z]{24}
      severity: ERROR
    - id: generic.secrets.security.detected-username-and-password-in-uri.detected-username-and-p
assword-in-uri
      languages:
        - generic
      message: Username and password in URI detected
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://github.com/grab/secret-scanner/blob/master/scanner/signatures/pattern.go
        subcategory:
            - vuln
        technology:
            - secrets
      patterns:
        - pattern: $PROTOCOL://$...USERNAME:$...PASSWORD@$END
        - metavariable-regex:
            metavariable: $...USERNAME
            regex: \A({?)([A-Za-z])([A-Za-z0-9_-]){5,31}(}?)\Z
        - metavariable-regex:
            metavariable: $...PASSWORD
            regex: (?!.*[\s])(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[!"#$%&'()*+,-./:;<=>?@[\]^_`
{|}~]){6,32}
        - metavariable-regex:
            metavariable: $PROTOCOL
            regex: (.*http.*)|(.*sql.*)|(.*ftp.*)|(.*smtp.*)
      severity: ERROR
    - id: generic.secrets.security.google-maps-apikeyleak.google-maps-apikeyleak
      languages:
        - generic
      message: Detects potential Google Maps API keys in code
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-538: Insertion of Sensitive Information into Externally-Accessible File or Di
rectory'
        description: Detects potential Google Maps API keys in code
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A3:2017 Sensitive Data Exposure
        references:
            - https://ozguralp.medium.com/unauthorized-google-maps-api-key-usage-cases-and-why-y
ou-need-to-care-1ccb28bf21e
        severity: MEDIUM
        subcategory:
            - audit
        technology:
            - Google Maps
      patterns:
        - pattern-regex: ^(AIza[0-9A-Za-z_-]{35}(?!\S))$
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: WARNING
    - id: generic.visualforce.security.ncino.html.usesriforcdns.use-sri-for-cdns
      languages:
        - generic
      message: 'Consuming CDNs without including a SubResource Integrity (SRI) can expose your a
pplication and its users to compromised code. SRIs allow you to consume specific versions of con
tent where if even a single byte is compromised, the resource will not be loaded. Add an integri
ty attribute to your <script> and <link> tags pointing to CDN content to ensure the resources ha
ve not been compromised. A crossorigin attribute should also be added. For a more thorough expla
nation along with explicit instructions on remediating, follow the directions from Mozilla here:
 https://developer.mozilla.org/en-US/blog/securing-cdn-using-sri-why-how/'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-346: Origin Validation Error'
        cwe2020-top25': true
        cwe2021-top25': true
        cwe2022-top25': true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cwe.mitre.org/data/definitions/352.html
            - https://developer.mozilla.org/en-US/blog/securing-cdn-using-sri-why-how/
        subcategory:
            - vuln
        technology:
            - salesforce
            - visualforce
      paths:
        include:
            - '*.component'
            - '*.page'
      patterns:
        - pattern-either:
            - pattern: <link...href="$URL..."...>
            - pattern: <script...src="$URL..."...>
        - metavariable-regex:
            metavariable: $URL
            regex: http[A-Za-z0-9\/\.\-\:]
        - pattern-not: <script...integrity="..."...src="..."...>
        - pattern-not: <script...src="..."...integrity="..."...>
        - pattern-not: <link...integrity="..."...href="..."...>
        - pattern-not: <link...href="..."...integrity="..."...>
      severity: WARNING
    - id: generic.visualforce.security.ncino.vf.xssfromunescapedurlparam.xss-from-unescaped-url-
param
      languages:
        - generic
      message: To remediate this issue, ensure that all URL parameters are properly escaped befo
re including them in scripts. Please update your code to use either the JSENCODE method to escap
e URL parameters or the escape="true" attribute on <apex:outputText> tags. Passing URL parameter
s directly into scripts and DOM sinks creates an opportunity for Cross-Site Scripting attacks. C
ross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are inject
ed into otherwise benign and trusted websites. To remediate this issue, ensure that all URL para
meters are properly escaped before including them in scripts.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        references:
            - https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/pages_sec
urity_tips_xss.htm
        subcategory:
            - vuln
        technology:
            - salesforce
            - visualforce
      paths:
        include:
            - '*.component'
            - '*.page'
      patterns:
        - pattern-either:
            - pattern: <apex:outputText...escape="false"...value="{!...CurrentPage.parameters.$U
RL_PARAM}"...>
            - pattern: <apex:outputText...value="{!...CurrentPage.parameters.$URL_PARAM}"...esca
pe="false"...>
            - pattern: <script>...'{!...CurrentPage.parameters.$URL_PARAM}'...</script>
        - pattern-not: <script>...'{!...JSENCODE(...CurrentPage.parameters.$URL_PARAM})'...</scr
ipt>
      severity: ERROR
    - id: generic.visualforce.security.ncino.xml.cspheaderattribute.csp-header-attribute
      languages:
        - generic
      message: Visualforce Pages must have the cspHeader attribute set to true. This attribute i
s available in API version 55 or higher.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://help.salesforce.com/s/articleView?id=sf.csp_trusted_sites.htm&type=5
        subcategory:
            - vuln
        technology:
            - salesforce
            - visualforce
      paths:
        include:
            - '*.page'
      patterns:
        - pattern: <apex:page...>...</apex:page>
        - pattern-not: <apex:page...cspHeader="true"...>...</apex:page>
        - pattern-not: <apex:page...>...<!--deprecated-->...</apex:page>
        - pattern-not: <apex:page...>...<!-- deprecated -->...</apex:page>
      severity: INFO
    - id: generic.visualforce.security.ncino.xml.visualforceapiversion.visualforce-page-api-vers
ion
      languages:
        - generic
      message: Visualforce Pages must use API version 55 or higher for required use of the cspHe
ader attribute set to true.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://developer.salesforce.com/docs/atlas.en-us.api_meta.meta/api_meta/meta_page
s.htm
        subcategory:
            - vuln
        technology:
            - salesforce
            - visualforce
      paths:
        include:
            - '*.page-meta.xml'
      patterns:
        - pattern-inside: <apiVersion.../apiVersion>
        - pattern-either:
            - pattern-regex: '[>][0-9].[0-9][<]'
            - pattern-regex: '[>][1-4][0-9].[0-9][<]'
            - pattern-regex: '[>][5][0-4].[0-9][<]'
      severity: WARNING
    - id: go.aws-lambda.security.database-sqli.database-sqli
      languages:
        - go
      message: Detected SQL statement that is tainted by `$EVENT` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use prepared stat
ements with the 'Prepare' and 'PrepareContext' calls.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://pkg.go.dev/database/sql#DB.Query
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - database
            - sql
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $QUERY
            - pattern-either:
                - pattern: $DB.Exec($QUERY,...)
                - pattern: $DB.ExecContent($QUERY,...)
                - pattern: $DB.Query($QUERY,...)
                - pattern: $DB.QueryContext($QUERY,...)
                - pattern: $DB.QueryRow($QUERY,...)
                - pattern: $DB.QueryRowContext($QUERY,...)
            - pattern-inside: |
                import "database/sql"
                ...
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    func $HANDLER($CTX $CTXTYPE, $EVENT $TYPE, ...) {...}
                    ...
                    lambda.Start($HANDLER, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - patterns:
                    - pattern-inside: |
                        func $HANDLER($EVENT $TYPE) {...}
                        ...
                        lambda.Start($HANDLER, ...)
                    - pattern-not-inside: |
                        func $HANDLER($EVENT context.Context) {...}
                        ...
                        lambda.Start($HANDLER, ...)
            - focus-metavariable: $EVENT
      severity: WARNING
    - id: go.aws-lambda.security.tainted-sql-string.tainted-sql-string
      languages:
        - go
      message: Detected user input used to manually construct a SQL string. This is usually bad
practice because manual construction could accidentally result in a SQL injection. An attacker c
ould use a SQL injection to steal or modify contents of the database. Instead, use a parameteriz
ed query which is available by default in most database engines. Alternatively, consider using a
n object-relational mapper (ORM) such as Sequelize which will protect your queries.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/SQL_Injection
        subcategory:
            - vuln
        technology:
            - aws-lambda
      mode: taint
      pattern-sanitizers:
        - pattern: strconv.Atoi(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: |
                        "$SQLSTR" + ...
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: (?i)(\s*select|\s*delete|\s*insert|\s*create|\s*update|\s*alter|\
s*drop).*
                - patterns:
                    - pattern-either:
                        - pattern: fmt.Fprintf($F, "$SQLSTR", ...)
                        - pattern: fmt.Sprintf("$SQLSTR", ...)
                        - pattern: fmt.Printf("$SQLSTR", ...)
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: \s*(?i)(select|delete|insert|create|update|alter|drop)\b.*%(v|s|q
).*
            - pattern-not-inside: |
                log.$PRINT(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    func $HANDLER($CTX $CTXTYPE, $EVENT $TYPE, ...) {...}
                    ...
                    lambda.Start($HANDLER, ...)
                - patterns:
                    - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        func $HANDLER($EVENT $TYPE) {...}
                        ...
                        lambda.Start($HANDLER, ...)
                    - pattern-not-inside: |
                        func $HANDLER($EVENT context.Context) {...}
                        ...
                        lambda.Start($HANDLER, ...)
            - focus-metavariable: $EVENT
      severity: ERROR
    - id: go.gorilla.security.audit.handler-assignment-from-multiple-sources.handler-assignment-
from-multiple-sources
      languages:
        - go
      message: 'Variable $VAR is assigned from two different sources: ''$Y'' and ''$R''. Make su
re this is intended, as this could cause logic bugs if they are treated as they are the same obj
ect.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-289: Authentication Bypass by Alternate Name'
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        references:
            - https://cwe.mitre.org/data/definitions/289.html
        subcategory:
            - audit
        technology:
            - gorilla
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                $Y, err := store.Get(...)
                ...
                $VAR := $Y.Values[...]
                ...
                $VAR = $R
            - focus-metavariable: $R
        - patterns:
            - pattern: |
                $Y, err := store.Get(...)
                ...
                var $VAR $INT = $Y.Values["..."].($INT)
                ...
                $VAR = $R
            - focus-metavariable: $R
      pattern-sources:
        - patterns:
            - pattern-inside: |
                func $HANDLER(..., $R *http.Request, ...) {
                  ...
                }
            - focus-metavariable: $R
            - pattern-either:
                - pattern: $R.query
      severity: WARNING
    - fix-regex:
        regex: (HttpOnly\s*:\s+)false
        replacement: \1true
      id: go.gorilla.security.audit.session-cookie-missing-httponly.session-cookie-missing-httpo
nly
      languages:
        - go
      message: A session cookie was detected without setting the 'HttpOnly' flag. The 'HttpOnly'
 flag for cookies instructs the browser to forbid client-side scripts from reading the cookie wh
ich mitigates XSS attacks. Set the 'HttpOnly' flag by setting 'HttpOnly' to 'true' in the Option
s struct.
      metadata:
        category: security
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        confidence: MEDIUM
        cwe:
            - 'CWE-1004: Sensitive Cookie Without ''HttpOnly'' Flag'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://github.com/0c34/govwa/blob/139693e56406b5684d2a6ae22c0af90717e149b8/user/s
ession/session.go#L69
        subcategory:
            - audit
        technology:
            - gorilla
      patterns:
        - pattern-not-inside: |
            &sessions.Options{
              ...,
              HttpOnly: true,
              ...,
            }
        - pattern: |
            &sessions.Options{
              ...,
            }
      severity: WARNING
    - fix-regex:
        regex: (Secure\s*:\s+)false
        replacement: \1true
      id: go.gorilla.security.audit.session-cookie-missing-secure.session-cookie-missing-secure
      languages:
        - go
      message: A session cookie was detected without setting the 'Secure' flag. The 'secure' fla
g for cookies prevents the client from transmitting the cookie over insecure channels such as HT
TP. Set the 'Secure' flag by setting 'Secure' to 'true' in the Options struct.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-614: Sensitive Cookie in HTTPS Session Without ''Secure'' Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://github.com/0c34/govwa/blob/139693e56406b5684d2a6ae22c0af90717e149b8/user/s
ession/session.go#L69
        subcategory:
            - audit
        technology:
            - gorilla
      patterns:
        - pattern-not-inside: |
            &sessions.Options{
              ...,
              Secure: true,
              ...,
            }
        - pattern: |
            &sessions.Options{
              ...,
            }
      severity: WARNING
    - fix-regex:
        regex: (SameSite\s*:\s+)http.SameSiteNoneMode
        replacement: \1http.SameSiteDefaultMode
      id: go.gorilla.security.audit.session-cookie-samesitenone.session-cookie-samesitenone
      languages:
        - go
      message: Found SameSiteNoneMode setting in Gorilla session options. Consider setting SameS
ite to Lax, Strict or Default for enhanced security.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1275: Sensitive Cookie with Improper SameSite Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://pkg.go.dev/github.com/gorilla/sessions#Options
        subcategory:
            - audit
        technology:
            - gorilla
      patterns:
        - pattern-inside: |
            &sessions.Options{
              ...,
              SameSite: http.SameSiteNoneMode,
              ...,
            }
        - pattern: |
            &sessions.Options{
              ...,
            }
      severity: WARNING
    - id: go.gorilla.security.audit.websocket-missing-origin-check.websocket-missing-origin-chec
k
      languages:
        - go
      message: 'The Origin header in the HTTP WebSocket handshake is used to guarantee that the
connection accepted by the WebSocket is from a trusted origin domain. Failure to enforce can lea
d to Cross Site Request Forgery (CSRF). As per "gorilla/websocket" documentation: "A CheckOrigin
 function should carefully validate the request origin to prevent cross-site request forgery."'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-352: Cross-Site Request Forgery (CSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://pkg.go.dev/github.com/gorilla/websocket#Upgrader
        subcategory:
            - audit
        technology:
            - gorilla
      patterns:
        - pattern-inside: |
            import ("github.com/gorilla/websocket")
            ...
        - patterns:
            - pattern-not-inside: |
                $UPGRADER = websocket.Upgrader{..., CheckOrigin: $FN ,...}
                ...
            - pattern-not-inside: |
                $UPGRADER.CheckOrigin = $FN2
                ...
            - pattern: |
                $UPGRADER.Upgrade(...)
      severity: WARNING
    - id: go.gorm.security.audit.gorm-dangerous-methods-usage.gorm-dangerous-method-usage
      languages:
        - go
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Detected usage of dangerous method $METHOD which does not escape inputs (see link
 in references). If the argument is user-controlled, this can lead to SQL injection. When using
$METHOD function, do not trust user-submitted data and only allow approved list of input (possib
ly, use an allowlist approach).
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://gorm.io/docs/security.html#SQL-injection-Methods
            - https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Shee
t.html
        subcategory:
            - vuln
        technology:
            - gorm
      mode: taint
      options:
        interfile: true
      pattern-sanitizers:
        - pattern-either:
            - pattern: strconv.Atoi(...)
            - pattern: |
                ($X: bool)
      pattern-sinks:
        - patterns:
            - pattern-inside: |
                import ("gorm.io/gorm")
                ...
            - patterns:
                - pattern-inside: |
                    func $VAL(..., $GORM *gorm.DB,... ) {
                      ...
                    }
                - pattern-either:
                    - pattern: |
                        $GORM. ... .$METHOD($VALUE)
                    - pattern: |
                        $DB := $GORM. ... .$ANYTHING(...)
                        ...
                        $DB. ... .$METHOD($VALUE)
            - focus-metavariable: $VALUE
            - metavariable-regex:
                metavariable: $METHOD
                regex: ^(Order|Exec|Raw|Group|Having|Distinct|Select|Pluck)$
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    ($REQUEST : http.Request).$ANYTHING
                - pattern: |
                    ($REQUEST : *http.Request).$ANYTHING
            - metavariable-regex:
                metavariable: $ANYTHING
                regex: ^(BasicAuth|Body|Cookie|Cookies|Form|FormValue|GetBody|Host|MultipartRead
er|ParseForm|ParseMultipartForm|PostForm|PostFormValue|Referer|RequestURI|Trailer|TransferEncodi
ng|UserAgent|URL)$
      severity: WARNING
    - fix-regex:
        regex: (.*)WithInsecure\(.*?\)
        replacement: \1WithTransportCredentials(credentials.NewTLS(<your_tls_config_here>))
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      id: go.grpc.security.grpc-client-insecure-connection.grpc-client-insecure-connection
      languages:
        - go
      message: 'Found an insecure gRPC connection using ''grpc.WithInsecure()''. This creates a
connection without encryption to a gRPC server. A malicious attacker could tamper with the gRPC
message, which could compromise the machine. Instead, establish a secure connection with an SSL
certificate using the ''grpc.WithTransportCredentials()'' function. You can create a create cred
entials using a ''tls.Config{}'' struct with ''credentials.NewTLS()''. The final fix looks like
this: ''grpc.WithTransportCredentials(credentials.NewTLS(<config>))''.'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-300: Channel Accessible by Non-Endpoint'
        impact: LOW
        likelihood: LOW
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://blog.gopheracademy.com/advent-2019/go-grps-and-tls/#connection-without-enc
ryption
        subcategory:
            - audit
        technology:
            - grpc
      pattern: $GRPC.Dial($ADDR, ..., $GRPC.WithInsecure(...), ...)
      severity: ERROR
    - id: go.grpc.security.grpc-server-insecure-connection.grpc-server-insecure-connection
      languages:
        - go
      message: Found an insecure gRPC server without 'grpc.Creds()' or options with credentials.
 This allows for a connection without encryption to this server. A malicious attacker could tamp
er with the gRPC message, which could compromise the machine. Include credentials derived from a
n SSL certificate in order to create a secure gRPC connection. You can create credentials using
'credentials.NewServerTLSFromFile("cert.pem", "cert.key")'.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-300: Channel Accessible by Non-Endpoint'
        impact: LOW
        likelihood: LOW
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://blog.gopheracademy.com/advent-2019/go-grps-and-tls/#connection-without-enc
ryption
        subcategory:
            - audit
        technology:
            - grpc
      mode: taint
      pattern-sinks:
        - pattern: grpc.NewServer($OPT, ...)
          requires: OPTIONS and not CREDS
        - pattern: grpc.NewServer()
          requires: EMPTY_CONSTRUCTOR
      pattern-sources:
        - label: OPTIONS
          pattern: grpc.ServerOption{ ... }
        - label: CREDS
          pattern: grpc.Creds(...)
        - label: EMPTY_CONSTRUCTOR
          pattern: grpc.NewServer()
      severity: ERROR
    - id: go.jwt-go.security.audit.jwt-parse-unverified.jwt-go-parse-unverified
      languages:
        - go
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Detected the decoding of a JWT token without a verify step. Don't use `ParseUnver
ified` unless you know what you're doing This method parses the token but doesn't validate the s
ignature. It's only ever useful in cases where you know the signature is valid (because it has b
een checked previously in the stack) and you want to extract values from it.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-345: Insufficient Verification of Data Authenticity'
        impact: LOW
        likelihood: LOW
        owasp:
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory:
            - audit
        technology:
            - jwt
      patterns:
        - pattern-inside: |
            import "github.com/dgrijalva/jwt-go"
            ...
        - pattern: |
            $JWT.ParseUnverified(...)
      severity: WARNING
    - id: go.jwt-go.security.jwt-none-alg.jwt-go-none-algorithm
      languages:
        - go
      message: Detected use of the 'none' algorithm in a JWT token. The 'none' algorithm assumes
 the integrity of the token has already been verified. This would allow a malicious actor to for
ge a JWT token that will automatically be verified. Do not explicitly use the 'none' algorithm.
Instead, use an algorithm such as 'HS256'.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: LOW
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory:
            - audit
        technology:
            - jwt
      patterns:
        - pattern-either:
            - pattern-inside: |
                import "github.com/golang-jwt/jwt"
                ...
            - pattern-inside: |
                import "github.com/dgrijalva/jwt-go"
                ...
        - pattern-either:
            - pattern: |
                jwt.SigningMethodNone
            - pattern: jwt.UnsafeAllowNoneSignatureType
      severity: ERROR
    - id: go.jwt-go.security.jwt.hardcoded-jwt-key
      languages:
        - go
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        likelihood: HIGH
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - jwt
            - secrets
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $TOKEN.SignedString($F)
            - focus-metavariable: $F
      pattern-sources:
        - patterns:
            - pattern-inside: |
                []byte("$F")
      severity: WARNING
    - id: go.lang.security.audit.crypto.bad_imports.insecure-module-used
      languages:
        - go
      message: The package `net/http/cgi` is on the import blocklist.  The package is vulnerable
 to httpoxy attacks (CVE-2015-5386). It is recommended to use `net/http` or a web framework to b
uild a web application instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://godoc.org/golang.org/x/crypto/sha3
        source-rule-url: https://github.com/securego/gosec
        subcategory:
            - audit
        technology:
            - go
      pattern-either:
        - patterns:
            - pattern-inside: |
                import "net/http/cgi"
                ...
            - pattern: |
                cgi.$FUNC(...)
      severity: WARNING
    - id: go.lang.security.audit.crypto.insecure_ssh.avoid-ssh-insecure-ignore-host-key
      languages:
        - go
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Disabled host key verification detected. This allows man-in-the-middle attacks. U
se the 'golang.org/x/crypto/ssh/knownhosts' package to do host key verification. See https://ska
rlso.github.io/2019/02/17/go-ssh-with-host-key-verification/ to learn more about the problem and
 how to fix it.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-322: Key Exchange without Entity Authentication'
        impact: LOW
        likelihood: LOW
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://skarlso.github.io/2019/02/17/go-ssh-with-host-key-verification/
            - https://gist.github.com/Skarlso/34321a230cf0245018288686c9e70b2d
        source-rule-url: https://github.com/securego/gosec
        subcategory:
            - audit
        technology:
            - go
      pattern: ssh.InsecureIgnoreHostKey()
      severity: WARNING
    - fix: |
        crypto/rand
      id: go.lang.security.audit.crypto.math_random.math-random-used
      languages:
        - go
      message: Do not use `math/rand`. Use `crypto/rand` instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-338: Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG)'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.h
tml#secure-random-number-generation
        subcategory:
            - vuln
        technology:
            - go
      patterns:
        - pattern-either:
            - pattern: |
                import $RAND "$MATH"
            - pattern: |
                import "$MATH"
        - metavariable-regex:
            metavariable: $MATH
            regex: ^(math/rand(\/v[0-9]+)*)$
        - pattern-either:
            - pattern-inside: |
                ...
                rand.$FUNC(...)
            - pattern-inside: |
                ...
                $RAND.$FUNC(...)
        - focus-metavariable:
            - $MATH
      severity: WARNING
    - fix: |
        tls.Config{ $...CONF, MinVersion: tls.VersionTLS13 }
      id: go.lang.security.audit.crypto.missing-ssl-minversion.missing-ssl-minversion
      languages:
        - go
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: ``MinVersion` is missing from this TLS configuration.  By default, TLS 1.2 is cur
rently used as the minimum when acting as a client, and TLS 1.0 when acting as a server. General
 purpose web applications should default to TLS 1.3 with all other protocols disabled.  Only whe
re it is known that a web server must support legacy clients with unsupported an insecure browse
rs (such as Internet Explorer 10), it may be necessary to enable TLS 1.0 to provide support. Add
 `MinVersion: tls.VersionTLS13'' to the TLS configuration to bump the minimum version to TLS 1.3
.'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: LOW
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://golang.org/doc/go1.14#crypto/tls
            - https://golang.org/pkg/crypto/tls/#:~:text=MinVersion
            - https://www.us-cert.gov/ncas/alerts/TA14-290A
        source-rule-url: https://github.com/securego/gosec/blob/master/rules/tls_config.go
        subcategory:
            - guardrail
        technology:
            - go
      patterns:
        - pattern: |
            tls.Config{ $...CONF }
        - pattern-not: |
            tls.Config{..., MinVersion: ..., ...}
      severity: WARNING
    - fix-regex:
        regex: VersionSSL30
        replacement: VersionTLS13
      id: go.lang.security.audit.crypto.ssl.ssl-v3-is-insecure
      languages:
        - go
      message: SSLv3 is insecure because it has known vulnerabilities. Starting with go1.14, SSL
v3 will be removed. Instead, use 'tls.VersionTLS13'.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: LOW
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://golang.org/doc/go1.14#crypto/tls
            - https://www.us-cert.gov/ncas/alerts/TA14-290A
        source-rule-url: https://github.com/securego/gosec/blob/master/rules/tls_config.go
        subcategory:
            - vuln
        technology:
            - go
      pattern: 'tls.Config{..., MinVersion: $TLS.VersionSSL30, ...}'
      severity: WARNING
    - id: go.lang.security.audit.crypto.tls.tls-with-insecure-cipher
      languages:
        - go
      message: Detected an insecure CipherSuite via the 'tls' module. This suite is considered w
eak. Use the function 'tls.CipherSuites()' to get a list of good cipher suites. See https://gola
ng.org/pkg/crypto/tls/#InsecureCipherSuites for why and what other cipher suites to use.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        impact: LOW
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://golang.org/pkg/crypto/tls/#InsecureCipherSuites
        source-rule-url: https://github.com/securego/gosec/blob/master/rules/tls.go
        subcategory:
            - vuln
        technology:
            - go
      pattern-either:
        - pattern: |
            tls.Config{..., CipherSuites: []$TYPE{..., tls.TLS_RSA_WITH_RC4_128_SHA, ...}}
        - pattern: |
            tls.Config{..., CipherSuites: []$TYPE{..., tls.TLS_RSA_WITH_3DES_EDE_CBC_SHA, ...}}
        - pattern: |
            tls.Config{..., CipherSuites: []$TYPE{..., tls.TLS_RSA_WITH_AES_128_CBC_SHA256, ...}
}
        - pattern: |
            tls.Config{..., CipherSuites: []$TYPE{..., tls.TLS_ECDHE_ECDSA_WITH_RC4_128_SHA, ...
}}
        - pattern: |
            tls.Config{..., CipherSuites: []$TYPE{..., tls.TLS_ECDHE_RSA_WITH_RC4_128_SHA, ...}}
        - pattern: |
            tls.Config{..., CipherSuites: []$TYPE{..., tls.TLS_ECDHE_RSA_WITH_3DES_EDE_CBC_SHA,
...}}
        - pattern: |
            tls.Config{..., CipherSuites: []$TYPE{..., tls.TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA2
56, ...}}
        - pattern: |
            tls.Config{..., CipherSuites: []$TYPE{..., tls.TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
, ...}}
        - pattern: |
            tls.CipherSuite{..., TLS_RSA_WITH_RC4_128_SHA, ...}
        - pattern: |
            tls.CipherSuite{..., TLS_RSA_WITH_3DES_EDE_CBC_SHA, ...}
        - pattern: |
            tls.CipherSuite{..., TLS_RSA_WITH_AES_128_CBC_SHA256, ...}
        - pattern: |
            tls.CipherSuite{..., TLS_ECDHE_ECDSA_WITH_RC4_128_SHA, ...}
        - pattern: |
            tls.CipherSuite{..., TLS_ECDHE_RSA_WITH_RC4_128_SHA, ...}
        - pattern: |
            tls.CipherSuite{..., TLS_ECDHE_RSA_WITH_3DES_EDE_CBC_SHA, ...}
        - pattern: |
            tls.CipherSuite{..., TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256, ...}
        - pattern: |
            tls.CipherSuite{..., TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256, ...}
      severity: WARNING
    - id: go.lang.security.audit.crypto.use_of_weak_crypto.use-of-md5
      languages:
        - go
      message: Detected MD5 hash algorithm which is considered insecure. MD5 is not collision re
sistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-328: Use of Weak Hash'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://github.com/securego/gosec#available-rules
        subcategory:
            - vuln
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        technology:
            - go
      patterns:
        - pattern-inside: |
            import "crypto/md5"
            ...
        - pattern-either:
            - pattern: |
                md5.New()
            - pattern: |
                md5.Sum(...)
      severity: WARNING
    - id: go.lang.security.audit.crypto.use_of_weak_crypto.use-of-sha1
      languages:
        - go
      message: Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision
resistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead
.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-328: Use of Weak Hash'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://github.com/securego/gosec#available-rules
        subcategory:
            - vuln
        technology:
            - go
      patterns:
        - pattern-inside: |
            import "crypto/sha1"
            ...
        - pattern-either:
            - pattern: |
                sha1.New()
            - pattern: |
                sha1.Sum(...)
      severity: WARNING
    - id: go.lang.security.audit.crypto.use_of_weak_crypto.use-of-des
      languages:
        - go
      message: Detected DES cipher algorithm which is insecure. The algorithm is considered weak
 and has been deprecated. Use AES instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://github.com/securego/gosec#available-rules
        subcategory:
            - vuln
        technology:
            - go
      patterns:
        - pattern-inside: |
            import "crypto/des"
            ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-either:
            - pattern: |
                des.NewTripleDESCipher(...)
            - pattern: |
                des.NewCipher(...)
      severity: WARNING
    - id: go.lang.security.audit.crypto.use_of_weak_crypto.use-of-rc4
      languages:
        - go
      message: Detected RC4 cipher algorithm which is insecure. The algorithm has many known vul
nerabilities. Use AES instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://github.com/securego/gosec#available-rules
        subcategory:
            - vuln
        technology:
            - go
      patterns:
        - pattern-inside: |
            import "crypto/rc4"
            ...
        - pattern: rc4.NewCipher(...)
      severity: WARNING
    - fix: |
        2048
      id: go.lang.security.audit.crypto.use_of_weak_rsa_key.use-of-weak-rsa-key
      languages:
        - go
      message: RSA keys should be at least 2048 bits
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.h
tml#algorithms
        source-rule-url: https://github.com/securego/gosec/blob/master/rules/rsa.go
        subcategory:
            - audit
        technology:
            - go
      patterns:
        - pattern-either:
            - pattern: |
                rsa.GenerateKey(..., $BITS)
            - pattern: |
                rsa.GenerateMultiPrimeKey(..., $BITS)
        - metavariable-comparison:
            comparison: $BITS < 2048
            metavariable: $BITS
        - focus-metavariable:
            - $BITS
      severity: WARNING
    - id: go.lang.security.audit.dangerous-exec-cmd.dangerous-exec-cmd
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      languages:
        - go
      message: Detected non-static command inside exec.Cmd. Audit the input to 'exec.Cmd'. If un
verified user data can reach this call site, this is a code injection vulnerability. A malicious
 actor can inject a malicious script to execute arbitrary code.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - audit
        technology:
            - go
      patterns:
        - pattern-either:
            - patterns:
                - pattern: |
                    exec.Cmd {...,Path: $CMD,...}
                - pattern-not: |
                    exec.Cmd {...,Path: "...",...}
                - pattern-not-inside: |
                    $CMD,$ERR := exec.LookPath("...");
                    ...
                - pattern-not-inside: |
                    $CMD = "...";
                    ...
            - patterns:
                - pattern: |
                    exec.Cmd {...,Args: $ARGS,...}
                - pattern-not: |
                    exec.Cmd {...,Args: []string{...},...}
                - pattern-not-inside: |
                    $ARGS = []string{"...",...};
                    ...
                - pattern-not-inside: |
                    $CMD = "...";
                    ...
                    $ARGS = []string{$CMD,...};
                    ...
                - pattern-not-inside: |
                    $CMD = exec.LookPath("...");
                    ...
                    $ARGS = []string{$CMD,...};
                    ...
            - patterns:
                - pattern: |
                    exec.Cmd {...,Args: []string{$CMD,...},...}
                - pattern-not: |
                    exec.Cmd {...,Args: []string{"...",...},...}
                - pattern-not-inside: |
                    $CMD,$ERR := exec.LookPath("...");
                    ...
                - pattern-not-inside: |
                    $CMD = "...";
                    ...
            - patterns:
                - pattern-either:
                    - pattern: |
                        exec.Cmd {...,Args: []string{"=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c",$EXE,
...},...}
                    - patterns:
                        - pattern: |
                            exec.Cmd {...,Args: []string{$CMD,"-c",$EXE,...},...}
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern-inside: |
                        $CMD,$ERR := exec.LookPath("=~/(sh|bash|ksh|csh|tcsh|zsh)/");
                        ...
                - pattern-not: |
                    exec.Cmd {...,Args: []string{"...","...","...",...},...}
                - pattern-not-inside: |
                    $EXE = "...";
                    ...
        - pattern-inside: |
            import "os/exec"
            ...
      severity: ERROR
    - id: go.lang.security.audit.md5-used-as-password.md5-used-as-password
      languages:
        - go
      message: It looks like MD5 is used as a password hash. MD5 is not considered a secure pass
word hash because it can be cracked by an attacker in a short amount of time. Use a suitable pas
sword hashing function such as bcrypt. You can use the `golang.org/x/crypto/bcrypt` package.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://tools.ietf.org/id/draft-lvelvindron-tls-md5-sha1-deprecate-01.html
            - https://security.stackexchange.com/questions/211/how-to-securely-hash-passwords
            - https://github.com/returntocorp/semgrep-rules/issues/1609
            - https://pkg.go.dev/golang.org/x/crypto/bcrypt
        subcategory:
            - vuln
        technology:
            - md5
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern: $FUNCTION(...)
            - metavariable-regex:
                metavariable: $FUNCTION
                regex: (?i)(.*password.*)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: md5.New
                - pattern: md5.Sum
      severity: WARNING
    - id: go.lang.security.audit.net.bind_all.avoid-bind-to-all-interfaces
      languages:
        - go
      message: Detected a network listener listening on 0.0.0.0 or an empty string. This could u
nexpectedly expose the server publicly as it binds to all available interfaces. Instead, specify
 another IP address that is not 0.0.0.0 nor the empty string.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        source-rule-url: https://github.com/securego/gosec
        subcategory:
            - audit
        technology:
            - go
      pattern-either:
        - pattern: tls.Listen($NETWORK, "=~/^0.0.0.0:.*$/", ...)
        - pattern: net.Listen($NETWORK, "=~/^0.0.0.0:.*$/", ...)
        - pattern: tls.Listen($NETWORK, "=~/^:.*$/", ...)
        - pattern: net.Listen($NETWORK, "=~/^:.*$/", ...)
      severity: WARNING
    - fix-regex:
        regex: (HttpOnly\s*:\s+)false
        replacement: \1true
      id: go.lang.security.audit.net.cookie-missing-httponly.cookie-missing-httponly
      languages:
        - go
      message: A session cookie was detected without setting the 'HttpOnly' flag. The 'HttpOnly'
 flag for cookies instructs the browser to forbid client-side scripts from reading the cookie wh
ich mitigates XSS attacks. Set the 'HttpOnly' flag by setting 'HttpOnly' to 'true' in the Cookie
.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1004: Sensitive Cookie Without ''HttpOnly'' Flag'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://github.com/0c34/govwa/blob/139693e56406b5684d2a6ae22c0af90717e149b8/util/c
ookie.go
            - https://golang.org/src/net/http/cookie.go
        subcategory:
            - vuln
        technology:
            - go
      patterns:
        - pattern-not-inside: |
            http.Cookie{
              ...,
              HttpOnly: true,
              ...,
            }
        - pattern: |
            http.Cookie{
              ...,
            }
      severity: WARNING
    - fix-regex:
        regex: (Secure\s*:\s+)false
        replacement: \1true
      id: go.lang.security.audit.net.cookie-missing-secure.cookie-missing-secure
      languages:
        - go
      message: A session cookie was detected without setting the 'Secure' flag. The 'secure' fla
g for cookies prevents the client from transmitting the cookie over insecure channels such as HT
TP. Set the 'Secure' flag by setting 'Secure' to 'true' in the Options struct.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-614: Sensitive Cookie in HTTPS Session Without ''Secure'' Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://github.com/0c34/govwa/blob/139693e56406b5684d2a6ae22c0af90717e149b8/util/c
ookie.go
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://golang.org/src/net/http/cookie.go
          subcategory:
            - vuln
          technology:
            - go
        patterns:
          - pattern-not-inside: |
              http.Cookie{
                ...,
                Secure: true,
                ...,
              }
          - pattern: |
              http.Cookie{
                ...,
              }
        severity: WARNING
      - id: go.lang.security.audit.net.dynamic-httptrace-clienttrace.dynamic-httptrace-clienttrace
        languages:
          - go
        message: Detected a potentially dynamic ClientTrace. This occurred because semgrep could n
ot find a static definition for '$TRACE'. Dynamic ClientTraces are dangerous because they deseri
alize function code to run when certain Request events occur, which could lead to code being run
 without your knowledge. Ensure that your ClientTrace is statically defined.
        metadata:
          category: security
          confidence: MEDIUM
          cwe:
            - 'CWE-913: Improper Control of Dynamically-Managed Code Resources'
          impact: LOW
          likelihood: LOW
          owasp:
            - A01:2021 - Broken Access Control
          references:
            - https://github.com/returntocorp/semgrep-rules/issues/518
          subcategory:
            - vuln
          technology:
            - go
        patterns:
          - pattern-not-inside: |
              package $PACKAGE
              ...
              &httptrace.ClientTrace { ... }
              ...
          - pattern: httptrace.WithClientTrace($ANY, $TRACE)
        severity: WARNING
      - id: go.lang.security.audit.net.formatted-template-string.formatted-template-string
        languages:
          - go
        message: Found a formatted template string passed to 'template.HTML()'. 'template.HTML()'
does not escape contents. Be absolutely sure there is no user-controlled data in this template.
If user data can reach this template, you may have a XSS vulnerability.
        metadata:
          category: security
          confidence: MEDIUM
          cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
          cwe2021-top25: true
          cwe2022-top25: true
          impact: MEDIUM
          likelihood: LOW
          owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
          references:
            - https://golang.org/pkg/html/template/#HTML
          subcategory:
            - audit
          technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - go
      patterns:
        - pattern-not: template.HTML("..." + "...")
        - pattern-either:
            - pattern: template.HTML($T + $X, ...)
            - pattern: template.HTML(fmt.$P("...", ...), ...)
            - pattern: |
                $T = "..."
                ...
                $T = $FXN(..., $T, ...)
                ...
                template.HTML($T, ...)
            - pattern: |
                $T = fmt.$P("...", ...)
                ...
                template.HTML($T, ...)
            - pattern: |
                $T, $ERR = fmt.$P("...", ...)
                ...
                template.HTML($T, ...)
            - pattern: |
                $T = $X + $Y
                ...
                template.HTML($T, ...)
            - pattern: |-
                $T = "..."
                ...
                $OTHER, $ERR = fmt.$P(..., $T, ...)
                ...
                template.HTML($OTHER, ...)
      severity: WARNING
    - id: go.lang.security.audit.net.fs-directory-listing.fs-directory-listing
      languages:
        - go
      message: 'Detected usage of ''http.FileServer'' as handler: this allows directory listing
and an attacker could navigate through directories looking for sensitive files. Be sure to disab
le directory listing or restrict access to specific directories/files.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-548: Exposure of Information Through Directory Listing'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A06:2017 - Security Misconfiguration
            - A01:2021 - Broken Access Control
        references:
            - https://github.com/OWASP/Go-SCP
            - https://cwe.mitre.org/data/definitions/548.html
        subcategory:
            - vuln
        technology:
            - go
      patterns:
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    $FS := http.FileServer(...)
                    ...
                - pattern-either:
                    - pattern: |
                        http.ListenAndServe(..., $FS)
                    - pattern: |
                        http.ListenAndServeTLS(..., $FS)
                    - pattern: |
                        http.Handle(..., $FS)
                    - pattern: |
                        http.HandleFunc(..., $FS)
            - patterns:
                - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    http.$FN(..., http.FileServer(...))
                - metavariable-regex:
                    metavariable: $FN
                    regex: (ListenAndServe|ListenAndServeTLS|Handle|HandleFunc)
      severity: WARNING
    - fix: http.ListenAndServeTLS($ADDR, certFile, keyFile, $HANDLER)
      id: go.lang.security.audit.net.use-tls.use-tls
      languages:
        - go
      message: Found an HTTP server without TLS. Use 'http.ListenAndServeTLS' instead. See https
://golang.org/pkg/net/http/#ListenAndServeTLS for more information.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://golang.org/pkg/net/http/#ListenAndServeTLS
        subcategory:
            - audit
        technology:
            - go
      pattern: http.ListenAndServe($ADDR, $HANDLER)
      severity: WARNING
    - id: go.lang.security.audit.net.wip-xss-using-responsewriter-and-printf.wip-xss-using-respo
nsewriter-and-printf
      languages:
        - go
      message: Found data going from url query parameters into formatted data written to Respons
eWriter. This could be XSS and should not be done. If you must do this, ensure your data is sani
tized or escaped.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - go
      patterns:
        - pattern-inside: |
            func $FUNC(..., $W http.ResponseWriter, ...) {
              ...
              var $TEMPLATE = "..."
              ...
              $W.Write([]byte(fmt.$PRINTF($TEMPLATE, ...)), ...)
              ...
            }
        - pattern-either:
            - pattern: |
                $PARAMS = r.URL.Query()
                ...
                $DATA, $ERR := $PARAMS[...]
                ...
                $INTERM = $ANYTHING(..., $DATA, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                $W.Write([]byte(fmt.$PRINTF(..., $INTERM, ...)))
            - pattern: |
                $PARAMS = r.URL.Query()
                ...
                $DATA, $ERR := $PARAMS[...]
                ...
                $INTERM = $DATA[...]
                ...
                $W.Write([]byte(fmt.$PRINTF(..., $INTERM, ...)))
            - pattern: |
                $DATA, $ERR := r.URL.Query()[...]
                ...
                $INTERM = $DATA[...]
                ...
                $W.Write([]byte(fmt.$PRINTF(..., $INTERM, ...)))
            - pattern: |
                $DATA, $ERR := r.URL.Query()[...]
                ...
                $INTERM = $ANYTHING(..., $DATA, ...)
                ...
                $W.Write([]byte(fmt.$PRINTF(..., $INTERM, ...)))
            - pattern: |
                $PARAMS = r.URL.Query()
                ...
                $DATA, $ERR := $PARAMS[...]
                ...
                $W.Write([]byte(fmt.$PRINTF(..., $DATA, ...)))
      severity: WARNING
    - fix: filepath.FromSlash(filepath.Clean("/"+strings.Trim($...INNER, "/")))
      id: go.lang.security.filepath-clean-misuse.filepath-clean-misuse
      languages:
        - go
      message: '`Clean` is not intended to sanitize against path traversal attacks. This functio
n is for finding the shortest path name equivalent to the given input. Using `Clean` to sanitize
 file reads may expose this application to path traversal attacks, where an attacker could acces
s arbitrary files on the server. To fix this easily, write this: `filepath.FromSlash(path.Clean(
"/"+strings.Trim(req.URL.Path, "/")))` However, a better solution is using the `SecureJoin` func
tion in the package `filepath-securejoin`. See https://pkg.go.dev/github.com/cyphar/filepath-sec
urejoin#section-readme.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://pkg.go.dev/path#Clean
            - http://technosophos.com/2016/03/31/go-quickly-cleaning-filepaths.html
            - https://labs.detectify.com/2021/12/15/zero-day-path-traversal-grafana/
            - https://dzx.cz/2021/04/02/go_path_traversal/
            - https://pkg.go.dev/github.com/cyphar/filepath-securejoin#section-readme
        subcategory:
            - vuln
        technology:
            - go
      mode: taint
      options:
        interfile: true
      pattern-sanitizers:
        - pattern-either:
            - pattern: |
                "/" + ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: filepath.Clean($...INNER)
                - pattern: path.Clean($...INNER)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    ($REQUEST : *http.Request).$ANYTHING
                - pattern: |
                    ($REQUEST : http.Request).$ANYTHING
            - metavariable-regex:
                metavariable: $ANYTHING
                regex: ^(BasicAuth|Body|Cookie|Cookies|Form|FormValue|GetBody|Host|MultipartRead
er|ParseForm|ParseMultipartForm|PostForm|PostFormValue|Referer|RequestURI|Trailer|TransferEncodi
ng|UserAgent|URL)$
      severity: ERROR
    - id: go.lang.security.injection.open-redirect.open-redirect
      languages:
        - go
      message: An HTTP redirect was found to be crafted from user-input `$REQUEST`. This can lea
d to open redirect vulnerabilities, potentially allowing attackers to redirect users to maliciou
s web sites. It is recommend where possible to not allow user-input to craft the redirect URL. W
hen user-input is necessary to craft the request, it is recommended to follow OWASP best practic
es to restrict the URL to domains in an allowlist.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-601: URL Redirection to Untrusted Site (''Open Redirect'')'
        description: An HTTP redirect was found to be crafted from user-input leading to an open
 redirect vulnerability
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        references:
            - https://knowledge-base.secureflag.com/vulnerabilities/unvalidated_redirects___forw
ards/open_redirect_go_lang.html
        subcategory:
            - vuln
        technology:
            - go
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern: http.Redirect($W, $REQ, $URL, ...)
            - focus-metavariable: $URL
          requires: INPUT and not CLEAN
      pattern-sources:
        - label: INPUT
          patterns:
            - pattern-either:
                - pattern: |
                    ($REQUEST : *http.Request).$ANYTHING
                - pattern: |
                    ($REQUEST : http.Request).$ANYTHING
            - metavariable-regex:
                metavariable: $ANYTHING
                regex: ^(BasicAuth|Body|Cookie|Cookies|Form|FormValue|GetBody|Host|MultipartRead
er|ParseForm|ParseMultipartForm|PostForm|PostFormValue|Referer|RequestURI|Trailer|TransferEncodi
ng|UserAgent|URL)$
        - label: CLEAN
          patterns:
            - pattern-either:
                - pattern: |
                    "$URLSTR" + $INPUT
                - patterns:
                    - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern: fmt.Fprintf($F, "$URLSTR", $INPUT, ...)
                        - pattern: fmt.Sprintf("$URLSTR", $INPUT, ...)
                        - pattern: fmt.Printf("$URLSTR", $INPUT, ...)
            - metavariable-regex:
                metavariable: $URLSTR
                regex: .*//[a-zA-Z0-10]+\..*
        requires: INPUT
    severity: WARNING
  - id: go.lang.security.injection.raw-html-format.raw-html-format
    languages:
      - go
    message: Detected user input flowing into a manually constructed HTML string. You may be a
ccidentally bypassing secure methods of rendering HTML by manually constructing HTML and this co
uld create a cross-site scripting vulnerability, which could let attackers steal sensitive user
data. Use the `html/template` package which will safely render HTML instead, or inspect that the
 HTML is rendered safely.
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
          - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
      cwe2021-top25: true
      cwe2022-top25: true
      impact: MEDIUM
      license: Commons Clause License Condition v1.0[LGPL-2.1-only]
      likelihood: HIGH
      owasp:
          - A07:2017 - Cross-Site Scripting (XSS)
          - A03:2021 - Injection
      references:
          - https://blogtitle.github.io/robn-go-security-pearls-cross-site-scripting-xss/
      subcategory:
          - vuln
      technology:
          - go
    mode: taint
    pattern-sanitizers:
      - pattern: html.EscapeString(...)
    pattern-sinks:
      - patterns:
          - pattern-either:
              - pattern: fmt.Printf("$HTMLSTR", ...)
              - pattern: fmt.Sprintf("$HTMLSTR", ...)
              - pattern: fmt.Fprintf($W, "$HTMLSTR", ...)
              - pattern: '"$HTMLSTR" + ...'
          - metavariable-pattern:
              language: generic
              metavariable: $HTMLSTR
              pattern: <$TAG ...
    pattern-sources:
      - patterns:
          - pattern-either:
              - pattern: |
                  ($REQUEST : *http.Request).$ANYTHING
              - pattern: |
                  ($REQUEST : http.Request).$ANYTHING
          - metavariable-regex:
              metavariable: $ANYTHING
              regex: ^(BasicAuth|Body|Cookie|Cookies|Form|FormValue|GetBody|Host|MultipartRead
er|ParseForm|ParseMultipartForm|PostForm|PostFormValue|Referer|RequestURI|Trailer|TransferEncodi
ng|UserAgent|URL)$
    severity: WARNING
  - id: go.lang.security.injection.tainted-sql-string.tainted-sql-string
    languages:
      - go
    message: User data flows into this manually-constructed SQL string. User data can be safel
y inserted into SQL strings using prepared statements or an object-relational mapper (ORM). Manu
ally-constructed SQL strings is a possible indicator of SQL injection, which could let an attack
er steal or manipulate data from the database. Instead, use prepared statements (`db.Query("SELE
CT * FROM t WHERE id = ?", id)`) or a safe library.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://golang.org/doc/database/sql-injection
            - https://www.stackhawk.com/blog/golang-sql-injection-guide-examples-and-prevention/
        subcategory:
            - vuln
        technology:
            - go
      mode: taint
      options:
        interfile: true
      pattern-sanitizers:
        - pattern-either:
            - pattern: strconv.Atoi(...)
            - pattern: |
                ($X: bool)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: |
                            "$SQLSTR" + ...
                        - patterns:
                            - pattern-inside: |
                                $VAR = "$SQLSTR";
                                ...
                            - pattern: $VAR += ...
                        - patterns:
                            - pattern-inside: |
                                var $SB strings.Builder
                                ...
                            - pattern-inside: |
                                $SB.WriteString("$SQLSTR")
                                ...
                                $SB.String(...)
                            - pattern: |
                                $SB.WriteString(...)
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: (?i)(select|delete|insert|create|update|alter|drop).*
                - patterns:
                    - pattern-either:
                        - pattern: fmt.Fprintf($F, "$SQLSTR", ...)
                        - pattern: fmt.Sprintf("$SQLSTR", ...)
                        - pattern: fmt.Printf("$SQLSTR", ...)
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: \s*(?i)(select|delete|insert|create|update|alter|drop)\b.*%(v|s|q
).*
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    ($REQUEST : *http.Request).$ANYTHING
                - pattern: |
                    ($REQUEST : http.Request).$ANYTHING
            - metavariable-regex:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              metavariable: $ANYTHING
              regex: ^(BasicAuth|Body|Cookie|Cookies|Form|FormValue|GetBody|Host|MultipartRead
er|ParseForm|ParseMultipartForm|PostForm|PostFormValue|Referer|RequestURI|Trailer|TransferEncodi
ng|UserAgent|URL)$
      severity: ERROR
    - id: go.lang.security.injection.tainted-url-host.tainted-url-host
      languages:
        - go
      message: A request was found to be crafted from user-input `$REQUEST`. This can lead to Se
rver-Side Request Forgery (SSRF) vulnerabilities, potentially exposing sensitive data. It is rec
ommend where possible to not allow user-input to craft the base request, but to be treated as pa
rt of the path or query parameter. When user-input is necessary to craft the request, it is reco
mmended to follow OWASP best practices to prevent abuse, including using an allowlist.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://goteleport.com/blog/ssrf-attacks/
        subcategory:
            - vuln
        technology:
            - go
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-inside: |
                                $CLIENT := &http.Client{...}
                                ...
                            - pattern: $CLIENT.$METHOD($URL, ...)
                        - pattern: http.$METHOD($URL, ...)
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(Get|Head|Post|PostForm)$
                - patterns:
                    - pattern: |
                        http.NewRequest("$METHOD", $URL, ...)
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(GET|HEAD|POST|POSTFORM)$
            - focus-metavariable: $URL
          requires: INPUT and not CLEAN
      pattern-sources:
        - label: INPUT
          patterns:
            - pattern-either:
                - pattern: |
                    ($REQUEST : *http.Request).$ANYTHING
                - pattern: |
                    ($REQUEST : http.Request).$ANYTHING
            - metavariable-regex:
                metavariable: $ANYTHING
                regex: ^(BasicAuth|Body|Cookie|Cookies|Form|FormValue|GetBody|Host|MultipartRead
er|ParseForm|ParseMultipartForm|PostForm|PostFormValue|Referer|RequestURI|Trailer|TransferEncodi
ng|UserAgent|URL)$
        - label: CLEAN
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          patterns:
            - pattern-either:
                - pattern: |
                    "$URLSTR" + $INPUT
                - patterns:
                    - pattern-either:
                        - pattern: fmt.Fprintf($F, "$URLSTR", $INPUT, ...)
                        - pattern: fmt.Sprintf("$URLSTR", $INPUT, ...)
                        - pattern: fmt.Printf("$URLSTR", $INPUT, ...)
            - metavariable-regex:
                metavariable: $URLSTR
                regex: .*//[a-zA-Z0-10]+\..*
          requires: INPUT
      severity: WARNING
    - id: go.template.security.ssti.go-ssti
      languages:
        - go
      message: A server-side template injection occurs when an attacker is able to use native te
mplate syntax to inject a malicious payload into a template, which is then executed server-side.
 When using "html/template" always check that user inputs are validated and sanitized before inc
luded within the template.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1336: Improper Neutralization of Special Elements Used in a Template Engine'
        impact: HIGH
        likelihood: LOW
        references:
            - https://www.onsecurity.io/blog/go-ssti-method-research/
            - http://blog.takemyhand.xyz/2020/05/ssti-breaking-gos-template-engine-to.html
        subcategory:
            - vuln
        technology:
            - go
      patterns:
        - pattern-inside: |
            import ("html/template")
            ...
        - pattern: $TEMPLATE = fmt.Sprintf("...", $ARG, ...)
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    func $FN(..., $REQ *http.Request, ...){
                    ...
                    }
                - pattern-inside: |
                    func $FN(..., $REQ http.Request, ...){
                    ...
                    }
                - pattern-inside: |
                    func(..., $REQ *http.Request, ...){
                    ...
                    }
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $ARG := $REQ.URL.Query().Get(...)
                    ...
                    $T, $ERR := $TMPL.Parse($TEMPLATE)
                - pattern-inside: |
                    $ARG := $REQ.Form.Get(...)
                    ...
                    $T, $ERR := $TMPL.Parse($TEMPLATE)
                - pattern-inside: |
                    $ARG := $REQ.PostForm.Get(...)
                    ...
                    $T, $ERR := $TMPL.Parse($TEMPLATE)
      severity: ERROR
    - id: java.android.security.exported_activity.exported_activity
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - generic
      message: The application exports an activity. Any application on the device can launch the
 exported activity which may compromise the integrity of your application or its data.  Ensure t
hat any exported activities do not have privileged access to your application's control plane.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-926: Improper Export of Android Application Components'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A5:2021 Security Misconfiguration
        references:
            - https://cwe.mitre.org/data/definitions/926.html
        subcategory:
            - vuln
        technology:
            - Android
      paths:
        exclude:
            - sources/
            - classes3.dex
            - '*.so'
        include:
            - '*AndroidManifest.xml'
      patterns:
        - pattern-not-inside: <activity ... android:exported="false" ... />
        - pattern-inside: "<activity  ... /> \n"
        - pattern-either:
            - pattern: |
                <activity ... android:exported="true" ... />
            - pattern: |
                <activity ... <intent-filter> ... />
      severity: WARNING
    - id: java.aws-lambda.security.tainted-sql-string.tainted-sql-string
      languages:
        - java
      message: Detected user input used to manually construct a SQL string. This is usually bad
practice because manual construction could accidentally result in a SQL injection. An attacker c
ould use a SQL injection to steal or modify contents of the database. Instead, use a parameteriz
ed query which is available by default in most database engines. Alternatively, consider using a
n object-relational mapper (ORM) such as Sequelize which will protect your queries.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/SQL_Injection
        subcategory:
            - vuln
        technology:
            - aws-lambda
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    "$SQLSTR" + ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: |
                    "$SQLSTR".concat(...)
                - patterns:
                    - pattern-inside: |
                        StringBuilder $SB = new StringBuilder("$SQLSTR");
                        ...
                    - pattern: $SB.append(...)
                - patterns:
                    - pattern-inside: |
                        $VAR = "$SQLSTR";
                        ...
                    - pattern: $VAR += ...
                - pattern: String.format("$SQLSTR", ...)
            - metavariable-regex:
                metavariable: $SQLSTR
                regex: (?i)(select|delete|insert|create|update|alter|drop)\b
            - pattern-not-inside: |
                System.out.$PRINTLN(...)
      pattern-sources:
        - patterns:
            - focus-metavariable: $EVENT
            - pattern-either:
                - pattern: |
                    $HANDLERTYPE $HANDLER($TYPE $EVENT, com.amazonaws.services.lambda.runtime.Co
ntext $CONTEXT) {
                      ...
                    }
                - pattern: |
                    $HANDLERTYPE $HANDLER(InputStream $EVENT, OutputStream $OUT, com.amazonaws.s
ervices.lambda.runtime.Context $CONTEXT) {
                      ...
                    }
      severity: ERROR
    - id: java.aws-lambda.security.tainted-sqli.tainted-sqli
      languages:
        - java
      message: Detected SQL statement that is tainted by `$EVENT` object. This could lead to SQL
 injection if variables in the SQL statement are not properly sanitized. Use parameterized SQL q
ueries or properly sanitize user input instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - sql
            - java
            - aws-lambda
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: "(java.sql.CallableStatement $STMT) = ...; \n"
                - pattern: |
                    (java.sql.Statement $STMT) = ...;
                - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    (java.sql.PreparedStatement $STMT) = ...;
                - pattern: |
                    $VAR = $CONN.prepareStatement(...)
                - pattern: |
                    $PATH.queryForObject(...);
                - pattern: |
                    (java.util.Map<String, Object> $STMT) = $PATH.queryForMap(...);
                - pattern: |
                    (org.springframework.jdbc.support.rowset.SqlRowSet $STMT) = ...;
                - patterns:
                    - pattern-inside: |
                        (String $SQL) = "$SQLSTR" + ...;
                        ...
                    - pattern: $PATH.$SQLCMD(..., $SQL, ...);
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: (?i)(^SELECT.* | ^INSERT.* | ^UPDATE.*)
                    - metavariable-regex:
                        metavariable: $SQLCMD
                        regex: (execute|query|executeUpdate|batchUpdate)
      pattern-sources:
        - patterns:
            - focus-metavariable: $EVENT
            - pattern-either:
                - pattern: |
                    $HANDLERTYPE $HANDLER($TYPE $EVENT, com.amazonaws.services.lambda.runtime.Co
ntext $CONTEXT) {
                      ...
                    }
                - pattern: |
                    $HANDLERTYPE $HANDLER(InputStream $EVENT, OutputStream $OUT, com.amazonaws.s
ervices.lambda.runtime.Context $CONTEXT) {
                      ...
                    }
      severity: WARNING
    - id: java.java-jwt.security.audit.jwt-decode-without-verify.java-jwt-decode-without-verify
      languages:
        - java
      message: Detected the decoding of a JWT token without a verify step. JWT tokens must be ve
rified before use, otherwise the token's integrity is unknown. This means a malicious actor coul
d forge a JWT token with any claims. Call '.verify()' before using the token.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-345: Insufficient Verification of Data Authenticity'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory:
            - vuln
        technology:
            - jwt
      patterns:
        - pattern: |
            com.auth0.jwt.JWT.decode(...);
        - pattern-not-inside: |-
            class $CLASS {
              ...
              $RETURNTYPE $FUNC (...) {
                ...
                $VERIFIER.verify(...);
                ...
              }
            }
      severity: WARNING
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: java.java-jwt.security.jwt-hardcode.java-jwt-hardcoded-secret
      languages:
        - java
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - java
            - secrets
            - jwt
      patterns:
        - pattern-either:
            - pattern: |
                (Algorithm $ALG) = $ALGO.$HMAC("$Y");
            - pattern: |
                $SECRET = "$Y";
                ...
                (Algorithm $ALG) = $ALGO.$HMAC($SECRET);
            - pattern: |
                class $CLASS {
                  ...
                  $TYPE $SECRET = "$Y";
                  ...
                  $RETURNTYPE $FUNC (...) {
                    ...
                    (Algorithm $ALG) = $ALGO.$HMAC($SECRET);
                    ...
                  }
                  ...
                }
        - focus-metavariable: $Y
        - metavariable-regex:
            metavariable: $HMAC
            regex: (HMAC384|HMAC256|HMAC512)
      severity: WARNING
    - id: java.java-jwt.security.jwt-none-alg.java-jwt-none-alg
      languages:
        - java
      message: Detected use of the 'none' algorithm in a JWT token. The 'none' algorithm assumes
 the integrity of the token has already been verified. This would allow a malicious actor to for
ge a JWT token that will automatically be verified. Do not explicitly use the 'none' algorithm.
Instead, use an algorithm such as 'HS256'.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory:
            - vuln
        technology:
            - jwt
      pattern-either:
        - pattern: |
            $JWT.sign(com.auth0.jwt.algorithms.Algorithm.none());
        - pattern: |
            $NONE = com.auth0.jwt.algorithms.Algorithm.none();
            ...
            $JWT.sign($NONE);
        - pattern: |-
            class $CLASS {
              ...
              $TYPE $NONE = com.auth0.jwt.algorithms.Algorithm.none();
              ...
              $RETURNTYPE $FUNC (...) {
                ...
                $JWT.sign($NONE);
                ...
              }
              ...
            }
      severity: ERROR
    - id: java.jax-rs.security.jax-rs-path-traversal.jax-rs-path-traversal
      languages:
        - java
      message: Detected a potential path traversal. A malicious actor could control the location
 of this file, to include going backwards in the directory with '../'. To address this, ensure t
hat user-controlled variables in file paths are sanitized. You may also consider using a utility
 method such as org.apache.commons.io.FilenameUtils.getName(...) to only retrieve the file name
from the path.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://www.owasp.org/index.php/Path_Traversal
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#PATH_TRAVERSAL_IN
        subcategory:
            - vuln
        technology:
            - jax-rs
      pattern-either:
        - pattern: |
            $RETURNTYPE $FUNC (..., @PathParam(...) $TYPE $VAR, ...) {
              ...
              new File(..., $VAR, ...);
              ...
            }
        - pattern: |-
            $RETURNTYPE $FUNC (..., @javax.ws.rs.PathParam(...) $TYPE $VAR, ...) {
              ...
              new File(..., $VAR, ...);
              ...
            }
      severity: WARNING
    - id: java.jboss.security.session_sqli.find-sql-string-concatenation
      languages:
        - java
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: In $METHOD, $X is used to construct a SQL query via string concatenation.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - jboss
      pattern-either:
        - pattern: |
            $RETURN $METHOD(...,String $X,...){
              ...
              Session $SESSION = ...;
              ...
              String $QUERY = ... + $X + ...;
              ...
              PreparedStatement $PS = $SESSION.connection().prepareStatement($QUERY);
              ...
              ResultSet $RESULT = $PS.executeQuery();
              ...
            }
        - pattern: |
            $RETURN $METHOD(...,String $X,...){
              ...
              String $QUERY = ... + $X + ...;
              ...
              Session $SESSION = ...;
              ...
              PreparedStatement $PS = $SESSION.connection().prepareStatement($QUERY);
              ...
              ResultSet $RESULT = $PS.executeQuery();
              ...
            }
      severity: ERROR
    - id: java.lang.security.audit.blowfish-insufficient-key-size.blowfish-insufficient-key-size
      languages:
        - java
      message: Using less than 128 bits for Blowfish is considered insecure. Use 128 bits or mor
e, or switch to use AES instead.
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#BLOWFISH_KEY_SIZE
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - audit
        technology:
            - java
      patterns:
        - pattern: |
            $KEYGEN = KeyGenerator.getInstance("Blowfish");
            ...
            $KEYGEN.init($SIZE);
        - metavariable-comparison:
            comparison: $SIZE < 128
            metavariable: $SIZE
      severity: WARNING
    - fix: |
        "AES/GCM/NoPadding"
      id: java.lang.security.audit.cbc-padding-oracle.cbc-padding-oracle
      languages:
        - java
      message: Using CBC with PKCS5Padding is susceptible to padding oracle attacks. A malicious
 actor could discern the difference between plaintext with valid or invalid padding. Further, CB
C mode does not include any integrity checks. Use 'AES/GCM/NoPadding' instead.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://capec.mitre.org/data/definitions/463.html
            - https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.h
tml#cipher-modes
            - https://find-sec-bugs.github.io/bugs.htm#CIPHER_INTEGRITY
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#PADDING_ORACLE
        subcategory:
            - audit
        technology:
            - java
      patterns:
        - pattern-inside: Cipher.getInstance("=~/.*\/CBC\/PKCS5Padding/")
        - pattern: |
            "=~/.*\/CBC\/PKCS5Padding/"
      severity: WARNING
    - id: java.lang.security.audit.crlf-injection-logs.crlf-injection-logs
      languages:
        - java
      message: When data from an untrusted source is put into a logger and not neutralized corre
ctly, an attacker could forge log entries or include malicious content.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-93: Improper Neutralization of CRLF Sequences (''CRLF Injection'')'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#CRLF_INJECTION_LOGS
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    class $CLASS {
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    ...
                    Logger $LOG = ...;
                    ...
                  }
              - pattern-either:
                  - pattern-inside: |
                      $X $METHOD(...,HttpServletRequest $REQ,...) {
                        ...
                      }
                  - pattern-inside: |
                      $X $METHOD(...,ServletRequest $REQ,...) {
                        ...
                      }
                  - pattern-inside: |
                      $X $METHOD(...) {
                        ...
                        HttpServletRequest $REQ = ...;
                        ...
                      }
                  - pattern-inside: |
                      $X $METHOD(...) {
                        ...
                        ServletRequest $REQ = ...;
                        ...
                      }
          - pattern-inside: |
              $X $METHOD(...) {
                ...
                Logger $LOG = ...;
                ...
                HttpServletRequest $REQ = ...;
                ...
              }
          - pattern-inside: |
              $X $METHOD(...) {
                ...
                Logger $LOG = ...;
                ...
                ServletRequest $REQ = ...;
                ...
              }
        - pattern-either:
            - pattern: |
                String $VAL = $REQ.getParameter(...);
                ...
                $LOG.$LEVEL(<... $VAL ...>);
            - pattern: |
                String $VAL = $REQ.getParameter(...);
                ...
                $LOG.log($LEVEL,<... $VAL ...>);
            - pattern: |
                $LOG.$LEVEL(<... $REQ.getParameter(...) ...>);
            - pattern: |
                $LOG.log($LEVEL,<... $REQ.getParameter(...) ...>);
      severity: WARNING
    - fix: |
        "AES/GCM/NoPadding"
      id: java.lang.security.audit.crypto.des-is-deprecated.des-is-deprecated
      languages:
        - java
        - kt
      message: DES is considered deprecated. AES is the recommended cipher. Upgrade to use AES.
See https://www.nist.gov/news-events/news/2005/06/nist-withdraws-outdated-data-encryption-standa
rd for more information.
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        functional-categories:
            - crypto::search::symmetric-algorithm::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.nist.gov/news-events/news/2005/06/nist-withdraws-outdated-data-encrypt
ion-standard
            - https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.h
tml#algorithms
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#DES_USAGE
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern-either:
            - pattern-inside: $CIPHER.getInstance("=~/DES/.*/")
            - pattern-inside: $CIPHER.getInstance("DES")
        - pattern-either:
            - pattern: |
                "=~/DES/.*/"
            - pattern: |
                "DES"
      severity: WARNING
    - id: java.lang.security.audit.crypto.desede-is-deprecated.desede-is-deprecated
      languages:
        - java
        - kt
      message: Triple DES (3DES or DESede) is considered deprecated. AES is the recommended ciph
er. Upgrade to use AES.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        functional-categories:
            - crypto::search::symmetric-algorithm::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://csrc.nist.gov/News/2017/Update-to-Current-Use-and-Deprecation-of-TDEA
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#TDES_USAGE
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern-either:
            - pattern: |
                $CIPHER.getInstance("=~/DESede.*/")
            - pattern: |
                $CRYPTO.KeyGenerator.getInstance("DES")
      severity: WARNING
    - id: java.lang.security.audit.crypto.ecb-cipher.ecb-cipher
      languages:
        - java
      message: Cipher in ECB mode is detected. ECB mode produces the same output for the same in
put each time which allows an attacker to intercept and replay the data. Further, ECB mode does
not provide any integrity checking. See https://find-sec-bugs.github.io/bugs.htm#CIPHER_INTEGRIT
Y.
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::mode::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#ECB_MODE
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern: |
            Cipher $VAR = $CIPHER.getInstance($MODE);
        - metavariable-regex:
            metavariable: $MODE
            regex: .*ECB.*
      severity: WARNING
    - id: java.lang.security.audit.crypto.gcm-nonce-reuse.gcm-nonce-reuse
      languages:
        - java
      message: 'GCM IV/nonce is reused: encryption can be totally useless'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-323: Reusing a Nonce, Key Pair in Encryption'
        functional-categories:
            - crypto::search::randomness::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://www.youtube.com/watch?v=r1awgAl90wM
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern-either:
            - pattern: new GCMParameterSpec(..., "...".getBytes(...), ...);
            - pattern: byte[] $NONCE = "...".getBytes(...); ... new GCMParameterSpec(..., $NONCE
, ...);
      severity: ERROR
    - id: java.lang.security.audit.crypto.no-null-cipher.no-null-cipher
      languages:
        - java
      message: 'NullCipher was detected. This will not encrypt anything; the cipher text will be
 the same as the plain text. Use a valid, secure cipher: Cipher.getInstance("AES/CBC/PKCS7PADDIN
G"). See https://owasp.org/www-community/Using_the_Java_Cryptographic_Extensions for more inform
ation.'
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#NULL_CIPHER
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern-either:
            - pattern: new NullCipher(...);
            - pattern: new javax.crypto.NullCipher(...);
      severity: WARNING
    - id: java.lang.security.audit.crypto.no-static-initialization-vector.no-static-initializati
on-vector
      languages:
        - java
      message: Initialization Vectors (IVs) for block ciphers should be randomly generated each
time they are used. Using a static IV means the same plaintext encrypts to the same ciphertext e
very time, weakening the strength of the encryption.
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-329: Generation of Predictable IV with CBC Mode'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://cwe.mitre.org/data/definitions/329.html
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#STATIC_IV
        subcategory:
            - vuln
        technology:
            - java
      pattern-either:
        - pattern: |
            byte[] $IV = {
                ...
            };
            ...
            new IvParameterSpec($IV, ...);
        - pattern: |
            class $CLASS {
                byte[] $IV = {
                    ...
                };
                ...
                $METHOD(...) {
                    ...
                    new IvParameterSpec($IV, ...);
                    ...
                }
            }
      severity: WARNING
    - id: java.lang.security.audit.crypto.rsa-no-padding.rsa-no-padding
      languages:
        - java
        - kt
      message: Using RSA without OAEP mode weakens the encryption.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        functional-categories:
            - crypto::search::mode::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://rdist.root.org/2009/10/06/why-rsa-encryption-padding-is-critical/
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#RSA_NO_PADDING
        subcategory:
            - vuln
        technology:
            - java
            - kotlin
      pattern: $CIPHER.getInstance("=~/RSA/[Nn][Oo][Nn][Ee]/NoPadding/")
      severity: WARNING
    - id: java.lang.security.audit.crypto.unencrypted-socket.unencrypted-socket
      languages:
        - java
      message: Detected use of a Java socket that is not encrypted. As a result, the traffic cou
ld be read by an attacker intercepting the network traffic. Use an SSLSocket created by 'SSLSock
etFactory' or 'SSLServerSocketFactory' instead.
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        functional-categories:
            - net::search::crypto-config::java.net
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#UNENCRYPTED_SOCKET
        subcategory:
            - vuln
        technology:
            - java
      pattern-either:
        - pattern: new ServerSocket(...)
        - pattern: new Socket(...)
      severity: WARNING
    - id: java.lang.security.audit.crypto.use-of-aes-ecb.use-of-aes-ecb
      languages:
        - java
      message: 'Use of AES with ECB mode detected. ECB doesn''t provide message confidentiality
and  is not semantically secure so should not be used. Instead, use a strong, secure cipher: Cip
her.getInstance("AES/CBC/PKCS7PADDING"). See https://owasp.org/www-community/Using_the_Java_Cryp
tographic_Extensions for more information.'
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::mode::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
            - https://googleprojectzero.blogspot.com/2022/10/rc4-is-still-considered-harmful.htm
l
        subcategory:
            - vuln
        technology:
            - java
      pattern: $CIPHER.getInstance("=~/AES/ECB.*/")
      severity: WARNING
    - id: java.lang.security.audit.crypto.use-of-blowfish.use-of-blowfish
      languages:
        - java
      message: 'Use of Blowfish was detected. Blowfish uses a 64-bit block size that  makes it v
ulnerable to birthday attacks, and is therefore considered non-compliant.  Instead, use a strong
, secure cipher: Cipher.getInstance("AES/CBC/PKCS7PADDING"). See https://owasp.org/www-community
/Using_the_Java_Cryptographic_Extensions for more information.'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::symmetric-algorithm::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
            - https://googleprojectzero.blogspot.com/2022/10/rc4-is-still-considered-harmful.htm
l
        subcategory:
            - vuln
        technology:
            - java
      pattern: $CIPHER.getInstance("Blowfish")
      severity: WARNING
    - id: java.lang.security.audit.crypto.use-of-default-aes.use-of-default-aes
      languages:
        - java
      message: 'Use of AES with no settings detected. By default, java.crypto.Cipher uses ECB mo
de. ECB doesn''t  provide message confidentiality and is not semantically secure so should not b
e used. Instead, use a strong, secure cipher: java.crypto.Cipher.getInstance("AES/CBC/PKCS7PADDI
NG"). See https://owasp.org/www-community/Using_the_Java_Cryptographic_Extensions for more infor
mation.'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::mode::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
            - https://googleprojectzero.blogspot.com/2022/10/rc4-is-still-considered-harmful.htm
l
        subcategory:
            - vuln
        technology:
            - java
      pattern-either:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import javax;
                    ...
            - pattern-either:
                - pattern: javax.crypto.Cipher.getInstance("AES")
                - pattern: (javax.crypto.Cipher $CIPHER).getInstance("AES")
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import javax.*;
                    ...
                - pattern-inside: |
                    import javax.crypto;
                    ...
            - pattern-either:
                - pattern: crypto.Cipher.getInstance("AES")
                - pattern: (crypto.Cipher $CIPHER).getInstance("AES")
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import javax.crypto.*;
                    ...
                - pattern-inside: |
                    import javax.crypto.Cipher;
                    ...
            - pattern-either:
                - pattern: Cipher.getInstance("AES")
                - pattern: (Cipher $CIPHER).getInstance("AES")
      severity: WARNING
    - fix: |
        getSha512Digest
      id: java.lang.security.audit.crypto.use-of-md5-digest-utils.use-of-md5-digest-utils
      languages:
        - java
      message: Detected MD5 hash algorithm which is considered insecure. MD5 is not collision re
sistant and is therefore not suitable as a cryptographic signature. Use HMAC instead.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-328: Use of Weak Hash'
        functional-categories:
            - crypto::search::hash-algorithm::org.apache.commons
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#WEAK_MESSAGE_DIGEST_MD5
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern: |
            $DU.$GET_ALGO().digest(...)
        - metavariable-pattern:
            metavariable: $GET_ALGO
            pattern: getMd5Digest
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - metavariable-pattern:
            metavariable: $DU
            pattern: DigestUtils
        - focus-metavariable: $GET_ALGO
      severity: WARNING
    - fix: |
        "SHA-512"
      id: java.lang.security.audit.crypto.use-of-md5.use-of-md5
      languages:
        - java
      message: Detected MD5 hash algorithm which is considered insecure. MD5 is not collision re
sistant and is therefore not suitable as a cryptographic signature. Use HMAC instead.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-328: Use of Weak Hash'
        functional-categories:
            - crypto::search::hash-algorithm::java.security
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#WEAK_MESSAGE_DIGEST_MD5
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern: |
            java.security.MessageDigest.getInstance($ALGO, ...);
        - metavariable-regex:
            metavariable: $ALGO
            regex: (.MD5.)
        - focus-metavariable: $ALGO
      severity: WARNING
    - id: java.lang.security.audit.crypto.use-of-rc2.use-of-rc2
      languages:
        - java
      message: 'Use of RC2 was detected. RC2 is vulnerable to related-key attacks, and is theref
ore considered non-compliant. Instead, use a strong, secure cipher: Cipher.getInstance("AES/CBC/
PKCS7PADDING"). See https://owasp.org/www-community/Using_the_Java_Cryptographic_Extensions for
more information.'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::symmetric-algorithm::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
            - https://googleprojectzero.blogspot.com/2022/10/rc4-is-still-considered-harmful.htm
l
        subcategory:
            - vuln
        technology:
            - java
      pattern: $CIPHER.getInstance("RC2")
      severity: WARNING
    - id: java.lang.security.audit.crypto.use-of-rc4.use-of-rc4
      languages:
        - java
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: 'Use of RC4 was detected. RC4 is vulnerable to several attacks, including stream
cipher attacks and bit flipping attacks. Instead, use a strong, secure cipher: Cipher.getInstanc
e("AES/CBC/PKCS7PADDING"). See https://owasp.org/www-community/Using_the_Java_Cryptographic_Exte
nsions for more information.'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::symmetric-algorithm::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
            - https://googleprojectzero.blogspot.com/2022/10/rc4-is-still-considered-harmful.htm
l
        subcategory:
            - vuln
        technology:
            - java
      pattern: $CIPHER.getInstance("RC4")
      severity: WARNING
    - id: java.lang.security.audit.crypto.use-of-sha1.use-of-sha1
      languages:
        - java
      message: Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision
resistant and is therefore not suitable as a cryptographic signature. Instead, use PBKDF2 for pa
ssword hashing or SHA256 or SHA512 for other hash function applications.
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-328: Use of Weak Hash'
        functional-categories:
            - crypto::search::hash-algorithm::javax.crypto
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#WEAK_MESSAGE_DIGEST_SHA1
        subcategory:
            - vuln
        technology:
            - java
      pattern-either:
        - patterns:
            - pattern: |
                java.security.MessageDigest.getInstance("$ALGO", ...);
            - metavariable-regex:
                metavariable: $ALGO
                regex: (SHA1|SHA-1)
        - pattern: |
            $DU.getSha1Digest().digest(...)
      severity: WARNING
    - id: java.lang.security.audit.crypto.weak-rsa.use-of-weak-rsa-key
      languages:
        - java
      message: RSA keys should be at least 2048 bits based on NIST recommendation.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        functional-categories:
            - crypto::search::key-length::java.security
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.h
tml#algorithms
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#RSA_KEY_SIZE
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern: |
            KeyPairGenerator $KEY = $G.getInstance("RSA");
            ...
            $KEY.initialize($BITS);
        - metavariable-comparison:
            comparison: $BITS < 2048
            metavariable: $BITS
      severity: WARNING
    - id: java.lang.security.audit.formatted-sql-string.formatted-sql-string
      languages:
        - java
      message: Detected a formatted string in a SQL statement. This could lead to SQL injection
if variables in the SQL statement are not properly sanitized. Use a prepared statements (java.sq
l.PreparedStatement) instead. You can obtain a PreparedStatement using 'connection.prepareStatem
ent'.
      metadata:
        asvs:
            control_id: 5.3.5 Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Shee
t.html
            - https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html#create_ps
            - https://software-security.sans.org/developer-how-to/fix-sql-injection-in-java-usin
g-prepared-callable-statement
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#SQL_INJECTION
        subcategory:
            - vuln
        technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - java
      mode: taint
      options:
        taint_assume_safe_booleans: true
        taint_assume_safe_numbers: true
      pattern-propagators:
        - from: $X
          pattern: (StringBuffer $S).append($X)
          to: $S
        - from: $X
          pattern: (StringBuilder $S).append($X)
          to: $S
      pattern-sanitizers:
        - patterns:
            - pattern: (CriteriaBuilder $CB).$ANY(...)
      pattern-sinks:
        - patterns:
            - pattern-not: $S.$SQLFUNC(<... "=~/.*TABLE *$/" ...>)
            - pattern-not: $S.$SQLFUNC(<... "=~/.*TABLE %s$/" ...>)
            - pattern-either:
                - pattern: (Statement $S).$SQLFUNC(...)
                - pattern: (PreparedStatement $P).$SQLFUNC(...)
                - pattern: (Connection $C).createStatement(...).$SQLFUNC(...)
                - pattern: (Connection $C).prepareStatement(...).$SQLFUNC(...)
                - pattern: (EntityManager $EM).$SQLFUNC(...)
            - metavariable-regex:
                metavariable: $SQLFUNC
                regex: execute|executeQuery|createQuery|query|addBatch|nativeSQL|create|prepare
          requires: CONCAT
      pattern-sources:
        - label: INPUT
          patterns:
            - pattern-either:
                - pattern: |
                    (HttpServletRequest $REQ)
                - patterns:
                    - pattern-inside: |
                        $ANNOT $FUNC (..., $INPUT, ...) {
                          ...
                        }
                    - pattern: (String $INPUT)
            - focus-metavariable: $INPUT
        - label: CONCAT
          patterns:
            - pattern-either:
                - pattern: $X + $INPUT
                - pattern: $X += $INPUT
                - pattern: $STRB.append($INPUT)
                - pattern: String.format(..., $INPUT, ...)
                - pattern: String.join(..., $INPUT, ...)
                - pattern: (String $STR).concat($INPUT)
                - pattern: $INPUT.concat(...)
                - pattern: new $STRB(..., $INPUT, ...)
          requires: INPUT
      severity: ERROR
    - id: java.lang.security.audit.http-response-splitting.http-response-splitting
      languages:
        - java
      message: Older Java application servers are vulnerable to HTTP response splitting, which m
ay occur if an HTTP request can be injected with CRLF characters. This finding is reported for c
ompleteness; it is recommended to ensure your environment is not affected by testing this yourse
lf.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-113: Improper Neutralization of CRLF Sequences in HTTP Headers (''HTTP Reques
t/Response Splitting'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - A03:2021 - Injection
        references:
          - https://www.owasp.org/index.php/HTTP_Response_Splitting
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#HTTP_RESPONSE_SPLITTING
        subcategory:
          - vuln
        technology:
          - java
      pattern-either:
        - pattern: |
            $VAR = $REQ.getParameter(...);
            ...
            $COOKIE = new Cookie(..., $VAR, ...);
            ...
            $RESP.addCookie($COOKIE, ...);
        - patterns:
            - pattern-inside: |
                $RETTYPE $FUNC(...,@PathVariable $TYPE $VAR, ...) {
                  ...
                }
            - pattern: |
                $COOKIE = new Cookie(..., $VAR, ...);
                ...
                $RESP.addCookie($COOKIE, ...);
      severity: INFO
    - id: java.lang.security.audit.insecure-smtp-connection.insecure-smtp-connection
      languages:
        - java
      message: Insecure SMTP connection detected. This connection will trust any SSL certificate
. Enable certificate verification by setting 'email.setSSLCheckServerIdentity(true)'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
          - 'CWE-297: Improper Validation of Certificate with Host Mismatch'
        impact: MEDIUM
        likelihood: LOW
        owasp:
          - A07:2021 - Identification and Authentication Failures
        references:
          - https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#INSECURE_SMTP_SSL
        subcategory:
          - vuln
        technology:
          - java
      patterns:
        - pattern-not-inside: |
            $EMAIL.setSSLCheckServerIdentity(true);
            ...
        - pattern-inside: |
            $EMAIL = new SimpleEmail(...);
            ...
        - pattern: $EMAIL.send(...);
      severity: WARNING
    - id: java.lang.security.audit.md5-used-as-password.md5-used-as-password
      languages:
        - java
      message: It looks like MD5 is used as a password hash. MD5 is not considered a secure pass
word hash because it can be cracked by an attacker in a short amount of time. Use a suitable pas
sword hashing function such as PBKDF2 or bcrypt. You can use `javax.crypto.SecretKeyFactory` wit
h `SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1")` or, if using Spring, `org.springframework
.security.crypto.bcrypt`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
          - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://tools.ietf.org/id/draft-lvelvindron-tls-md5-sha1-deprecate-01.html
            - https://github.com/returntocorp/semgrep-rules/issues/1609
            - https://docs.oracle.com/javase/7/docs/technotes/guides/security/StandardNames.html
#SecretKeyFactory
            - https://docs.spring.io/spring-security/site/docs/current/api/org/springframework/s
ecurity/crypto/bcrypt/BCryptPasswordEncoder.html
        subcategory:
            - vuln
        technology:
            - java
            - md5
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: $MODEL.$METHOD(...);
            - metavariable-regex:
                metavariable: $METHOD
                regex: (?i)(.*password.*)
      pattern-sources:
        - patterns:
            - pattern-inside: |
                $TYPE $MD = MessageDigest.getInstance("MD5");
                ...
            - pattern: $MD.digest(...);
      severity: WARNING
    - id: java.lang.security.audit.sqli.tainted-sql-from-http-request.tainted-sql-from-http-requ
est
      languages:
        - java
      message: Detected input from a HTTPServletRequest going into a SQL sink or statement. This
 could lead to SQL injection if variables in the SQL statement are not properly sanitized. Use p
arameterized SQL queries or properly sanitize user input instead.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Shee
t.html
            - https://owasp.org/www-community/attacks/SQL_Injection
        subcategory:
            - vuln
        technology:
            - sql
            - java
            - servlets
            - spring
      mode: taint
      options:
        taint_assume_safe_booleans: true
        taint_assume_safe_numbers: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: "(java.sql.CallableStatement $STMT) = ...; \n"
                - pattern: |
                    (java.sql.Statement $STMT) = ...;
                    ...
                    $OUTPUT = $STMT.$FUNC(...);
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: |
                    (java.sql.PreparedStatement $STMT) = ...;
                - pattern: |
                    $VAR = $CONN.prepareStatement(...)
                - pattern: |
                    $PATH.queryForObject(...);
                - pattern: |
                    (java.util.Map<String, Object> $STMT) = $PATH.queryForMap(...);
                - pattern: |
                    (org.springframework.jdbc.support.rowset.SqlRowSet $STMT) = ...;
                - pattern: |
                    (org.springframework.jdbc.core.JdbcTemplate $TEMPL).batchUpdate(...)
                - patterns:
                    - pattern-inside: |
                        (String $SQL) = "$SQLSTR" + ...;
                        ...
                    - pattern: $PATH.$SQLCMD(..., $SQL, ...);
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: (?i)(^SELECT.* | ^INSERT.* | ^UPDATE.*)
                    - metavariable-regex:
                        metavariable: $SQLCMD
                        regex: (execute|query|executeUpdate|batchUpdate)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    (HttpServletRequest $REQ).$REQFUNC(...)
                - pattern: "(ServletRequest $REQ).$REQFUNC(...) \n"
            - metavariable-regex:
                metavariable: $REQFUNC
                regex: (getInputStream|getParameter|getParameterMap|getParameterValues|getReader
|getCookies|getHeader|getHeaderNames|getHeaders|getPart|getParts|getQueryString)
      severity: WARNING
    - id: java.lang.security.audit.tainted-cmd-from-http-request.tainted-cmd-from-http-request
      languages:
        - java
      message: Detected input from a HTTPServletRequest going into a 'ProcessBuilder' or 'exec'
command. This could lead to command injection if variables passed into the exec commands are not
 properly sanitized. Instead, avoid using these OS commands with user-supplied input, or, if you
 must use these commands, use a whitelist of specific values.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - java
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    (ProcessBuilder $PB) = ...;
                - patterns:
                    - pattern: |
                        (Process $P) = ...;
                    - pattern-not: |
                        (Process $P) = (java.lang.Runtime $R).exec(...);
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - patterns:
                    - pattern: (java.lang.Runtime $R).exec($CMD, ...);
                    - focus-metavariable: $CMD
                - patterns:
                    - pattern-either:
                        - pattern-inside: "(java.util.List<$TYPE> $ARGLIST) = ...;  \n...\n(Proc
essBuilder $PB) = ...;\n...\n$PB.command($ARGLIST);\n"
                        - pattern-inside: "(java.util.List<$TYPE> $ARGLIST) = ...;  \n...\n(Proc
essBuilder $PB) = ...;\n"
                        - pattern-inside: "(java.util.List<$TYPE> $ARGLIST) = ...;  \n...\n(Proc
ess $P) = ...;\n"
                    - pattern: |
                        $ARGLIST.add(...);
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    (HttpServletRequest $REQ)
                - patterns:
                    - pattern-inside: |
                        (javax.servlet.http.Cookie[] $COOKIES) = (HttpServletRequest $REQ).getCo
okies(...);
                        ...
                        for (javax.servlet.http.Cookie $COOKIE: $COOKIES) {
                          ...
                        }
                    - pattern: |
                        $COOKIE.getValue(...)
      severity: ERROR
    - id: java.lang.security.audit.tainted-env-from-http-request.tainted-env-from-http-request
      languages:
        - java
      message: Detected input from a HTTPServletRequest going into the environment variables of
an 'exec' command.  Instead, call the command with user-supplied arguments by using the overload
ed method with one String array as the argument. `exec({"command", "arg1", "arg2"})`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-454: External Initialization of Trusted Variables or Data Stores'
        cwe2021-top25: false
        cwe2022-top25: false
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - java
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: (java.lang.Runtime $R).exec($CMD, $ENV_ARGS, ...);
            - focus-metavariable: $ENV_ARGS
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    (HttpServletRequest $REQ)
                - patterns:
                    - pattern-inside: |
                        (javax.servlet.http.Cookie[] $COOKIES) = (HttpServletRequest $REQ).getCo
okies(...);
                        ...
                        for (javax.servlet.http.Cookie $COOKIE: $COOKIES) {
                          ...
                        }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern: |
                        $COOKIE.getValue(...)
      severity: ERROR
    - id: java.lang.security.audit.tainted-ldapi-from-http-request.tainted-ldapi-from-http-reque
st
      languages:
        - java
      message: Detected input from a HTTPServletRequest going into an LDAP query. This could lea
d to LDAP injection if the input is not properly sanitized, which could result in attackers modi
fying objects in the LDAP tree structure. Ensure data passed to an LDAP query is not controllabl
e or properly sanitize the data.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-90: Improper Neutralization of Special Elements used in an LDAP Query (''LDAP
 Injection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://sensei.securecodewarrior.com/recipes/scw%3Ajava%3ALDAP-injection
        subcategory:
            - vuln
        technology:
            - java
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    (javax.naming.directory.InitialDirContext $IDC).search(...)
                - pattern: |
                    (javax.naming.directory.DirContext $CTX).search(...)
            - pattern-not: |
                (javax.naming.directory.InitialDirContext $IDC).search($Y, "...", ...)
            - pattern-not: |
                (javax.naming.directory.DirContext $CTX).search($Y, "...", ...)
      pattern-sources:
        - patterns:
            - pattern: (HttpServletRequest $REQ)
      severity: WARNING
    - id: java.lang.security.audit.tainted-session-from-http-request.tainted-session-from-http-r
equest
      languages:
        - java
      message: Detected input from a HTTPServletRequest going into a session command, like `setA
ttribute`. User input into such a command could lead to an attacker inputting malicious code int
o your session parameters, blurring the line between what's trusted and untrusted, and therefore
 leading to a trust boundary violation. This could lead to programmers trusting unvalidated data
. Instead, thoroughly sanitize user input before passing it into such function calls.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-501: Trust Boundary Violation'
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        subcategory:
            - vuln
        technology:
            - java
      mode: taint
      options:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern: (HttpServletRequest $REQ).getSession().$FUNC($NAME, $VALUE);
            - metavariable-regex:
                metavariable: $FUNC
                regex: ^(putValue|setAttribute)$
            - focus-metavariable: $VALUE
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: |
                        (HttpServletRequest $REQ).$FUNC(...)
                    - pattern-not: |
                        (HttpServletRequest $REQ).getSession()
                - patterns:
                    - pattern-inside: |
                        (javax.servlet.http.Cookie[] $COOKIES) = (HttpServletRequest $REQ).getCo
okies(...);
                        ...
                        for (javax.servlet.http.Cookie $COOKIE: $COOKIES) {
                          ...
                        }
                    - pattern: |
                        $COOKIE.getValue(...)
                - patterns:
                    - pattern-inside: |
                        $TYPE[] $VALS = (HttpServletRequest $REQ).$GETFUNC(... );
                        ...
                    - pattern: |
                        $PARAM = $VALS[$INDEX];
                - patterns:
                    - pattern-inside: |
                        $HEADERS = (HttpServletRequest $REQ).getHeaders(...);
                        ...
                        $PARAM = $HEADERS.$FUNC(...);
                        ...
                    - pattern: |
                        java.net.URLDecoder.decode($PARAM, ...)
      severity: WARNING
    - id: java.lang.security.audit.tainted-xpath-from-http-request.tainted-xpath-from-http-reque
st
      languages:
        - java
      message: Detected input from a HTTPServletRequest going into a XPath evaluate or compile c
ommand. This could lead to xpath injection if variables passed into the evaluate or compile comm
ands are not properly sanitized. Xpath injection could lead to unauthorized access to sensitive
information in XML documents. Instead, thoroughly sanitize user input or use parameterized xpath
 queries if you can.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-643: Improper Neutralization of Data within XPath Expressions (''XPath Inject
ion'')'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - java
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    (javax.xml.xpath.XPath $XP).evaluate(...)
                - pattern: |
                    (javax.xml.xpath.XPath $XP).compile(...).evaluate(...)
      pattern-sources:
        - patterns:
            - pattern: |
                (HttpServletRequest $REQ).$FUNC(...)
      severity: WARNING
    - id: java.lang.security.audit.unvalidated-redirect.unvalidated-redirect
      languages:
        - java
      message: Application redirects to a destination URL specified by a user-supplied parameter
 that is not validated. This could direct users to malicious locations. Consider using an allowl
ist to validate URLs.
      metadata:
        asvs:
            control_id: 5.1.5 Open Redirect
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v51-input-validation-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-601: URL Redirection to Untrusted Site (''Open Redirect'')'
        impact: LOW
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#UNVALIDATED_REDIRECT
        subcategory:
            - vuln
        technology:
            - java
      pattern-either:
        - pattern: |
            $X $METHOD(...,HttpServletResponse $RES,...,String $URL,...) {
              ...
              $RES.sendRedirect($URL);
              ...
            }
        - pattern: |
            $X $METHOD(...,String $URL,...,HttpServletResponse $RES,...) {
              ...
              $RES.sendRedirect($URL);
              ...
            }
        - pattern: |
            $X $METHOD(...,HttpServletRequest $REQ,...,HttpServletResponse $RES,...) {
              ...
              String $URL = $REQ.getParameter(...);
              ...
              $RES.sendRedirect($URL);
              ...
            }
        - pattern: |
            $X $METHOD(...,HttpServletResponse $RES,...,HttpServletRequest $REQ,...) {
              ...
              String $URL = $REQ.getParameter(...);
              ...
              $RES.sendRedirect($URL);
              ...
            }
        - pattern: |
            $X $METHOD(...,String $URL,...) {
              ...
              HttpServletResponse $RES = ...;
              ...
              $RES.sendRedirect($URL);
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          ...
        }
    - pattern: |
        $X $METHOD(...,HttpServletRequest $REQ,...,HttpServletResponse $RES,...) {
          ...
          $RES.sendRedirect($REQ.getParameter(...));
          ...
        }
    - pattern: |
        $X $METHOD(...,HttpServletResponse $RES,...,HttpServletRequest $REQ,...) {
          ...
          $RES.sendRedirect($REQ.getParameter(...));
          ...
        }
    - pattern: |
        $X $METHOD(...,HttpServletResponse $RES,...,String $URL,...) {
          ...
          $RES.addHeader("Location",$URL);
          ...
        }
    - pattern: |
        $X $METHOD(...,String $URL,...,HttpServletResponse $RES,...) {
          ...
          $RES.addHeader("Location",$URL);
          ...
        }
    - pattern: |
        $X $METHOD(...,HttpServletRequest $REQ,...,HttpServletResponse $RES,...) {
          ...
          String $URL = $REQ.getParameter(...);
          ...
          $RES.addHeader("Location",$URL);
          ...
        }
    - pattern: |
        $X $METHOD(...,HttpServletResponse $RES,...,HttpServletRequest $REQ,...) {
          ...
          String $URL = $REQ.getParameter(...);
          ...
          $RES.addHeader("Location",$URL);
          ...
        }
    - pattern: |
        $X $METHOD(...,String $URL,...) {
          ...
          HttpServletResponse $RES = ...;
          ...
          $RES.addHeader("Location",$URL);
          ...
        }
    - pattern: |
        $X $METHOD(...,HttpServletRequest $REQ,...,HttpServletResponse $RES,...) {
          ...
          $RES.addHeader("Location",$REQ.getParameter(...));
          ...
        }
    - pattern: |-
        $X $METHOD(...,HttpServletResponse $RES,...,HttpServletRequest $REQ,...) {
          ...
          $RES.addHeader("Location",$REQ.getParameter(...));
          ...
        }
  severity: WARNING
- fix-regex:
    regex: (.*?)\.getInstance\(.*?\)
    replacement: \1.getInstance("TLSv1.2")
  id: java.lang.security.audit.weak-ssl-context.weak-ssl-context
  languages:
    - java
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: An insecure SSL context was detected. TLS versions 1.0, 1.1, and all SSL versions
 are considered weak encryption and are deprecated. Use SSLContext.getInstance("TLSv1.2") for th
e best security.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://tools.ietf.org/html/rfc7568
            - https://tools.ietf.org/id/draft-ietf-tls-oldversions-deprecate-02.html
        source_rule_url: https://find-sec-bugs.github.io/bugs.htm#SSL_CONTEXT
        subcategory:
            - audit
        technology:
            - java
      patterns:
        - pattern-not: SSLContext.getInstance("TLSv1.3")
        - pattern-not: SSLContext.getInstance("TLSv1.2")
        - pattern: SSLContext.getInstance("...")
      severity: WARNING
    - id: java.lang.security.audit.xss.no-direct-response-writer.no-direct-response-writer
      languages:
        - java
      message: Detected a request with potential user-input going into a OutputStream or Writer
object. This bypasses any view or template environments, including HTML escaping, which may expo
se this application to cross-site scripting (XSS) vulnerabilities. Consider using a view technol
ogy such as JavaServer Faces (JSFs) which automatically escapes HTML views.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        license: proprietary license - copyright © Semgrep, Inc.
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://www3.ntu.edu.sg/home/ehchua/programming/java/JavaServerFaces.html
        subcategory:
            - vuln
        technology:
            - java
            - servlets
      mode: taint
      options:
        interfile: true
      pattern-sanitizers:
        - pattern-either:
            - pattern: Encode.forHtml(...)
            - pattern: (PolicyFactory $POLICY).sanitize(...)
            - pattern: (AntiSamy $AS).scan(...)
            - pattern: JSoup.clean(...)
            - pattern: org.apache.commons.lang.StringEscapeUtils.escapeHtml(...)
            - pattern: org.springframework.web.util.HtmlUtils.htmlEscape(...)
            - pattern: org.owasp.esapi.ESAPI.encoder().encodeForHTML(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    (HttpServletResponse $RESPONSE).getWriter(...).$WRITE(...)
                - pattern: |
                    (HttpServletResponse $RESPONSE).getOutputStream(...).$WRITE(...)
                - pattern: |
                    (java.io.PrintWriter $WRITER).$WRITE(...)
                - pattern: |
                    (PrintWriter $WRITER).$WRITE(...)
                - pattern: |
                    (javax.servlet.ServletOutputStream $WRITER).$WRITE(...)
                - pattern: |
                    (ServletOutputStream $WRITER).$WRITE(...)
                - pattern: |
                    (java.io.OutputStream $WRITER).$WRITE(...)
                - pattern: |
                    (OutputStream $WRITER).$WRITE(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    (HttpServletRequest $REQ).$REQFUNC(...)
                - pattern: "(ServletRequest $REQ).$REQFUNC(...) \n"
            - metavariable-regex:
                metavariable: $REQFUNC
                regex: (getInputStream|getParameter|getParameterMap|getParameterValues|getReader
|getCookies|getHeader|getHeaderNames|getHeaders|getPart|getParts|getQueryString)
      severity: WARNING
    - id: java.lang.security.audit.xxe.documentbuilderfactory-disallow-doctype-decl-false.docume
ntbuilderfactory-disallow-doctype-decl-false
      languages:
        - java
      message: DOCTYPE declarations are enabled for $DBFACTORY. Without prohibiting external ent
ity declarations, this is vulnerable to XML external entity attacks. Disable this by setting the
 feature "http://apache.org/xml/features/disallow-doctype-decl" to true. Alternatively, allow DO
CTYPE declarations and only prohibit external entities declarations. This can be done by setting
 the features "http://xml.org/sax/features/external-general-entities" and "http://xml.org/sax/fe
atures/external-parameter-entities" to false.
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://blog.sonarsource.com/secure-xml-processor
            - https://xerces.apache.org/xerces2-j/features.html
        subcategory:
            - vuln
        technology:
            - java
            - xml
      patterns:
        - pattern: $DBFACTORY.setFeature("http://apache.org/xml/features/disallow-doctype-decl",
 false);
        - pattern-not-inside: |
            $RETURNTYPE $METHOD(...){
              ...
              $DBF.setFeature("http://xml.org/sax/features/external-general-entities", false);
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
            $DBF.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
            ...
          }
        - pattern-not-inside: |
            $RETURNTYPE $METHOD(...){
              ...
              $DBF.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
              ...
              $DBF.setFeature("http://xml.org/sax/features/external-general-entities", false);
              ...
            }
        - pattern-not-inside: |
            $RETURNTYPE $METHOD(...){
              ...
              $DBF.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
              ...
              $DBF.setAttribute(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");
              ...
            }
        - pattern-not-inside: |
            $RETURNTYPE $METHOD(...){
              ...
              $DBF.setAttribute(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");
              ...
              $DBF.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
              ...
            }
      severity: ERROR
    - fix: |
        $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
        $FACTORY.newDocumentBuilder();
      id: java.lang.security.audit.xxe.documentbuilderfactory-disallow-doctype-decl-missing.docu
mentbuilderfactory-disallow-doctype-decl-missing
      languages:
        - java
      message: DOCTYPE declarations are enabled for this DocumentBuilderFactory. This is vulnera
ble to XML external entity attacks. Disable this by setting the feature "http://apache.org/xml/f
eatures/disallow-doctype-decl" to true. Alternatively, allow DOCTYPE declarations and only prohi
bit external entities declarations. This can be done by setting the features "http://xml.org/sax
/features/external-general-entities" and "http://xml.org/sax/features/external-parameter-entitie
s" to false.
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://blog.sonarsource.com/secure-xml-processor
            - https://xerces.apache.org/xerces2-j/features.html
        subcategory:
            - vuln
        technology:
            - java
            - xml
      mode: taint
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      pattern-sanitizers:
        - by-side-effect: true
          pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: |
                        $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype-dec
l", true);
                    - pattern: |
                        $FACTORY.setFeature("http://xml.org/sax/features/external-general-entiti
es", false);
                        ...
                        $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-enti
ties", false);
                    - pattern: |
                        $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-enti
ties", false);
                        ...
                        $FACTORY.setFeature("http://xml.org/sax/features/external-general-entiti
es", false);
                - focus-metavariable: $FACTORY
            - patterns:
                - pattern-either:
                    - pattern-inside: |
                        class $C {
                          ...
                          $T $M(...) {
                            ...
                            $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype
-decl",
                            true);
                            ...
                          }
                          ...
                        }
                    - pattern-inside: |
                        class $C {
                          ...
                          $T $M(...) {
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities", false);
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                            ...
                          }
                          ...
                        }
                    - pattern-inside: |
                        class $C {
                          ...
                          $T $M(...) {
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities",false);
                            ...
                          }
                          ...
                        }
                - pattern: $M($X)
                - focus-metavariable: $X
      pattern-sinks:
        - patterns:
            - pattern: $FACTORY.newDocumentBuilder();
      pattern-sources:
        - by-side-effect: true
          patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern-either:
                - pattern: |
                    $FACTORY = DocumentBuilderFactory.newInstance();
                - patterns:
                    - pattern: $FACTORY
                    - pattern-inside: |
                        class $C {
                          ...
                          $V $FACTORY = DocumentBuilderFactory.newInstance();
                          ...
                        }
                    - pattern-not-inside: |
                        class $C {
                          ...
                          $V $FACTORY = DocumentBuilderFactory.newInstance();
                          static {
                            ...
                            $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype
-decl", true);
                            ...
                          }
                          ...
                        }
                    - pattern-not-inside: |
                        class $C {
                          ...
                          $V $FACTORY = DocumentBuilderFactory.newInstance();
                          static {
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities", false);
                            ...
                          }
                          ...
                        }
                    - pattern-not-inside: |
                        class $C {
                          ...
                          $V $FACTORY = DocumentBuilderFactory.newInstance();
                          static {
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities", false);
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                            ...
                          }
                          ...
                        }
      severity: ERROR
    - fix: $DBFACTORY.setFeature("http://xml.org/sax/features/external-general-entities", false)
;
      id: java.lang.security.audit.xxe.documentbuilderfactory-external-general-entities-true.doc
umentbuilderfactory-external-general-entities-true
      languages:
        - java
      message: External entities are allowed for $DBFACTORY. This is vulnerable to XML external
entity attacks. Disable this by setting the feature "http://xml.org/sax/features/external-genera
l-entities" to false.
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://blog.sonarsource.com/secure-xml-processor
        subcategory:
            - vuln
        technology:
            - java
            - xml
      pattern: $DBFACTORY.setFeature("http://xml.org/sax/features/external-general-entities", tr
ue);
      severity: ERROR
    - fix: $DBFACTORY.setFeature("http://xml.org/sax/features/external-parameter-entities", fals
e);
      id: java.lang.security.audit.xxe.documentbuilderfactory-external-parameter-entities-true.d
ocumentbuilderfactory-external-parameter-entities-true
      languages:
        - java
      message: External entities are allowed for $DBFACTORY. This is vulnerable to XML external
entity attacks. Disable this by setting the feature "http://xml.org/sax/features/external-parame
ter-entities" to false.
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://blog.sonarsource.com/secure-xml-processor
        subcategory:
            - vuln
        technology:
            - java
            - xml
      pattern: $DBFACTORY.setFeature("http://xml.org/sax/features/external-parameter-entities",
true);
      severity: ERROR
    - fix: |
        $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
        $FACTORY.newSAXParser();
      id: java.lang.security.audit.xxe.saxparserfactory-disallow-doctype-decl-missing.saxparserf
actory-disallow-doctype-decl-missing
      languages:
        - java
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: DOCTYPE declarations are enabled for this SAXParserFactory. This is vulnerable to
 XML external entity attacks. Disable this by setting the feature `http://apache.org/xml/feature
s/disallow-doctype-decl` to true. Alternatively, allow DOCTYPE declarations and only prohibit ex
ternal entities declarations. This can be done by setting the features `http://xml.org/sax/featu
res/external-general-entities` and `http://xml.org/sax/features/external-parameter-entities` to
false. NOTE - The previous links are not meant to be clicked. They are the literal config key va
lues that are supposed to be used to disable these features. For more information, see https://s
emgrep.dev/docs/cheat-sheets/java-xxe/#3a-documentbuilderfactory.
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://blog.sonarsource.com/secure-xml-processor
            - https://xerces.apache.org/xerces2-j/features.html
        subcategory:
            - vuln
        technology:
            - java
            - xml
      mode: taint
      pattern-sanitizers:
        - by-side-effect: true
          pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: |
                        $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype-dec
l", true);
                    - pattern: |
                        $FACTORY.setFeature("http://xml.org/sax/features/external-general-entiti
es", false);
                        ...
                        $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-enti
ties", false);
                    - pattern: |
                        $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-enti
ties", false);
                        ...
                        $FACTORY.setFeature("http://xml.org/sax/features/external-general-entiti
es", false);
                - focus-metavariable: $FACTORY
            - patterns:
                - pattern-either:
                    - pattern-inside: |
                        class $C {
                          ...
                          $T $M(...) {
                            ...
                            $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype
-decl",
                            true);
                            ...
                          }
                          ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    }
                - pattern-inside: |
                    class $C {
                      ...
                      $T $M(...) {
                        ...
                        $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities", false);
                        ...
                        $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                        ...
                      }
                      ...
                    }
                - pattern-inside: |
                    class $C {
                      ...
                      $T $M(...) {
                        ...
                        $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                        ...
                        $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities",false);
                        ...
                      }
                      ...
                    }
            - pattern: $M($X)
            - focus-metavariable: $X
    pattern-sinks:
      - patterns:
          - pattern: $FACTORY.newSAXParser();
    pattern-sources:
      - by-side-effect: true
        patterns:
          - pattern-either:
              - pattern: |
                  $FACTORY = SAXParserFactory.newInstance();
              - patterns:
                  - pattern: $FACTORY
                  - pattern-inside: |
                      class $C {
                        ...
                        $V $FACTORY = SAXParserFactory.newInstance();
                        ...
                      }
                  - pattern-not-inside: |
                      class $C {
                        ...
                        $V $FACTORY = SAXParserFactory.newInstance();
                        static {
                          ...
                          $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype
-decl", true);
                          ...
                        }
                        ...
                      }
                  - pattern-not-inside: |
                      class $C {
                        ...
                        $V $FACTORY = SAXParserFactory.newInstance();
                        static {
                          ...
                          $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                          ...
                          $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities", false);
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                          ...
                        }
                        ...
                      }
                  - pattern-not-inside: |
                      class $C {
                        ...
                        $V $FACTORY = SAXParserFactory.newInstance();
                        static {
                          ...
                          $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities", false);
                          ...
                          $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                          ...
                        }
                        ...
                      }
      severity: ERROR
    - fix: |
        $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, ""); $FACTORY.setAttribute(XMLCo
nstants.ACCESS_EXTERNAL_STYLESHEET, "");
        $FACTORY.newTransformer(...);
      id: java.lang.security.audit.xxe.transformerfactory-dtds-not-disabled.transformerfactory-d
tds-not-disabled
      languages:
        - java
      message: DOCTYPE declarations are enabled for this TransformerFactory. This is vulnerable
to XML external entity attacks. Disable this by setting the attributes "accessExternalDTD" and "
accessExternalStylesheet" to "".
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://blog.sonarsource.com/secure-xml-processor
            - https://xerces.apache.org/xerces2-j/features.html
        subcategory:
            - vuln
        technology:
            - java
            - xml
      mode: taint
      pattern-sanitizers:
        - by-side-effect: true
          pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: |
                        $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_STYLESHEET, ""); ...
                        $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
                    - pattern: |
                        $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
                        ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_STYLESHEET, "");
                - pattern: |
                    $FACTORY.setAttribute("=~/.*accessExternalStylesheet.*/", ""); ...
                    $FACTORY.setAttribute("=~/.*accessExternalDTD.*/", "");
                - pattern: |
                    $FACTORY.setAttribute("=~/.*accessExternalDTD.*/", "");
                    ...
                    $FACTORY.setAttribute("=~/.*accessExternalStylesheet.*/", "");
            - focus-metavariable: $FACTORY
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    class $C {
                      ...
                      $T $M(...) {
                        ...
                        $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_STYLESHEET, "");
                        ...
                        $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
                        ...
                      }
                      ...
                    }
                - pattern-inside: |
                    class $C {
                      ...
                      $T $M(...) {
                        ...
                        $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
                        ...
                        $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_STYLESHEET, "");
                        ...
                      }
                      ...
                    }
                - pattern-inside: |
                    class $C {
                      ...
                      $T $M(...) {
                        ...
                        $FACTORY.setAttribute("=~/.*accessExternalStylesheet.*/", "");
                        ...
                        $FACTORY.setAttribute("=~/.*accessExternalDTD.*/", "");
                        ...
                      }
                      ...
                    }
                - pattern-inside: |
                    class $C {
                      ...
                      $T $M(...) {
                        ...
                        $FACTORY.setAttribute("=~/.*accessExternalDTD.*/", "");
                        ...
                        $FACTORY.setAttribute("=~/.*accessExternalStylesheet.*/", "");
                        ...
                      }
                      ...
                    }
            - pattern: $M($X)
            - focus-metavariable: $X
    pattern-sinks:
      - patterns:
          - pattern: $FACTORY.newTransformer(...);
    pattern-sources:
      - by-side-effect: true
        patterns:
          - pattern-either:
              - pattern: |
                  $FACTORY = TransformerFactory.newInstance();
              - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern: $FACTORY
                    - pattern-inside: |
                        class $C {
                          ...
                          $V $FACTORY = TransformerFactory.newInstance();
                          ...
                        }
                    - pattern-not-inside: |
                        class $C {
                          ...
                          $V $FACTORY = TransformerFactory.newInstance();
                          static {
                            ...
                            $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
                            ...
                            $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_STYLESHEET, "");
                            ...
                          }
                          ...
                        }
                    - pattern-not-inside: |
                        class $C {
                          ...
                          $V $FACTORY = TransformerFactory.newInstance();
                          static {
                            ...
                            $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_STYLESHEET, "");
                            ...
                            $FACTORY.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
                            ...
                          }
                          ...
                        }
                    - pattern-not-inside: |
                        class $C {
                          ...
                          $V $FACTORY = TransformerFactory.newInstance();
                          static {
                            ...
                            $FACTORY.setAttribute("=~/.*accessExternalDTD.*/", "");
                            ...
                            $FACTORY.setAttribute("=~/.*accessExternalStylesheet.*/", "");
                            ...
                          }
                          ...
                        }
                    - pattern-not-inside: |
                        class $C {
                          ...
                          $V $FACTORY = TransformerFactory.newInstance();
                          static {
                            ...
                            $FACTORY.setAttribute("=~/.*accessExternalStylesheet.*/", "");
                            ...
                            $FACTORY.setAttribute("=~/.*accessExternalDTD.*/", "");
                            ...
                          }
                          ...
                        }
      severity: ERROR
    - id: java.lang.security.httpservlet-path-traversal.httpservlet-path-traversal
      languages:
        - java
      message: Detected a potential path traversal. A malicious actor could control the location
 of this file, to include going backwards in the directory with '../'. To address this, ensure t
hat user-controlled variables in file paths are sanitized. You may also consider using a utility
 method such as org.apache.commons.io.FilenameUtils.getName(...) to only retrieve the file name
from the path.
      metadata:
        category: security
        confidence: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://www.owasp.org/index.php/Path_Traversal
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#PATH_TRAVERSAL_IN
        subcategory:
            - vuln
        technology:
            - java
      mode: taint
      pattern-sanitizers:
        - pattern: org.apache.commons.io.FilenameUtils.getName(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    (java.io.File $FILE) = ...
                - pattern: |
                    (java.io.FileOutputStream $FOS) = ...
                - pattern: |
                    new java.io.FileInputStream(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    (HttpServletRequest $REQ)
                - patterns:
                    - pattern-inside: |
                        (javax.servlet.http.Cookie[] $COOKIES) = (HttpServletRequest $REQ).getCo
okies(...);
                        ...
                        for (javax.servlet.http.Cookie $COOKIE: $COOKIES) {
                          ...
                        }
                    - pattern: |
                        $COOKIE.getValue(...)
                - patterns:
                    - pattern-inside: |
                        $TYPE[] $VALS = (HttpServletRequest $REQ).$GETFUNC(...);
                        ...
                    - pattern: |
                        $PARAM = $VALS[$INDEX];
      severity: ERROR
    - id: java.lang.security.insecure-jms-deserialization.insecure-jms-deserialization
      languages:
        - java
      message: JMS Object messages depend on Java Serialization for marshalling/unmarshalling of
 the message payload when ObjectMessage.getObject() is called. Deserialization of untrusted data
 can lead to security flaws; a remote attacker could via a crafted JMS ObjectMessage to execute
arbitrary code with the permissions of the application listening/consuming JMS Messages. In this
 case, the JMS MessageListener consume an ObjectMessage type received inside the onMessage metho
d, which may lead to arbitrary code execution when calling the $Y.getObject method.
      metadata:
        asvs:
            control_id: 5.5.3 Insecue Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://www.blackhat.com/docs/us-16/materials/us-16-Kaiser-Pwning-Your-Java-Messag
ing-With-Deserialization-Vulnerabilities-wp.pdf
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern-inside: |
            public class $JMS_LISTENER implements MessageListener {
              ...
              public void onMessage(Message $JMS_MSG) {
                  ...
              }
            }
        - pattern-either:
            - pattern-inside: $X = $Y.getObject(...);
            - pattern-inside: $X = ($Z) $Y.getObject(...);
      severity: WARNING
    - id: java.lang.security.jackson-unsafe-deserialization.jackson-unsafe-deserialization
      languages:
        - java
      message: When using Jackson to marshall/unmarshall JSON to Java objects, enabling default
typing is dangerous and can lead to RCE. If an attacker can control `$JSON` it might be possible
 to provide a malicious JSON which can be used to exploit unsecure deserialization. In order to
prevent this issue, avoid to enable default typing (globally or by using "Per-class" annotations
) and avoid using `Object` and other dangerous types for member variable declaration which creat
ing classes for Jackson based deserialization.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A8:2017 Insecure Deserialization
            - A8:2021 Software and Data Integrity Failures
        references:
            - https://swapneildash.medium.com/understanding-insecure-implementation-of-jackson-d
eserialization-7b3d409d2038
            - https://cowtowncoder.medium.com/on-jackson-cves-dont-panic-here-is-what-you-need-t
o-know-54cd0d6e8062
            - https://adamcaudill.com/2017/10/04/exploiting-jackson-rce-cve-2017-7525/
        subcategory:
            - audit
        technology:
            - jackson
      patterns:
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    ObjectMapper $OM = new ObjectMapper(...);
                    ...
                - pattern-inside: |
                    $OM.enableDefaultTyping();
                    ...
                - pattern: $OM.readValue($JSON, ...);
            - patterns:
                - pattern-inside: |
                    class $CLASS {
                      ...
                      @JsonTypeInfo(use = Id.CLASS,...)
                      $TYPE $VAR;
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    ...
                  }
              - metavariable-regex:
                  metavariable: $TYPE
                  regex: (Object|Serializable|Comparable)
              - pattern: $OM.readValue($JSON, $CLASS.class);
          - patterns:
              - pattern-inside: |
                  class $CLASS {
                    ...
                    ObjectMapper $OM;
                    ...
                    $INITMETHODTYPE $INITMETHOD(...) {
                      ...
                      $OM = new ObjectMapper();
                      ...
                      $OM.enableDefaultTyping();
                      ...
                    }
                    ...
                  }
              - pattern-inside: "$METHODTYPE $METHOD(...) {\n  ...  \n}\n"
              - pattern: $OM.readValue($JSON, ...);
      severity: WARNING
    - id: java.lang.security.servletresponse-writer-xss.servletresponse-writer-xss
      languages:
        - java
      message: 'Cross-site scripting detected in HttpServletResponse writer with variable ''$VAR
''. User input was detected going directly from the HttpServletRequest into output. Ensure your
data is properly encoded using org.owasp.encoder.Encode.forHtml: ''Encode.forHtml($VAR)''.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#XSS_SERVLET
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern-inside: $TYPE $FUNC(..., HttpServletResponse $RESP, ...) { ... }
        - pattern-inside: $VAR = $REQ.getParameter(...); ...
        - pattern-either:
            - pattern: $RESP.getWriter(...).write(..., $VAR, ...);
            - pattern: |
                $WRITER = $RESP.getWriter(...);
                ...
                $WRITER.write(..., $VAR, ...);
      severity: ERROR
    - id: java.lang.security.xmlinputfactory-possible-xxe.xmlinputfactory-possible-xxe
      languages:
        - java
      message: XML external entities are not explicitly disabled for this XMLInputFactory. This
could be vulnerable to XML external entity vulnerabilities. Explicitly disable external entities
 by setting "javax.xml.stream.isSupportingExternalEntities" to false.
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://www.blackhat.com/docs/us-15/materials/us-15-Wang-FileCry-The-New-Age-Of-XX
E-java-wp.pdf
            - https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Chea
t_Sheet.html#xmlinputfactory-a-stax-parser
        subcategory:
            - vuln
        technology:
            - java
      patterns:
        - pattern-not-inside: |
            $METHOD(...) {
              ...
              $XMLFACTORY.setProperty("javax.xml.stream.isSupportingExternalEntities", false);
              ...
            }
        - pattern-not-inside: |
            $METHOD(...) {
              ...
              $XMLFACTORY.setProperty(javax.xml.stream.XMLInputFactory.IS_SUPPORTING_EXTERNAL_EN
TITIES, false);
              ...
            }
        - pattern-not-inside: |
            $METHOD(...) {
              ...
              $XMLFACTORY.setProperty("javax.xml.stream.isSupportingExternalEntities", Boolean.F
ALSE);
              ...
            }
        - pattern-not-inside: |
            $METHOD(...) {
              ...
              $XMLFACTORY.setProperty(javax.xml.stream.XMLInputFactory.IS_SUPPORTING_EXTERNAL_EN
TITIES, Boolean.FALSE);
              ...
            }
        - pattern-either:
            - pattern: javax.xml.stream.XMLInputFactory.newFactory(...)
            - pattern: new XMLInputFactory(...)
      severity: WARNING
    - id: java.spring.security.audit.spring-actuator-fully-enabled-yaml.spring-actuator-fully-en
abled-yaml
      languages:
        - yaml
      message: Spring Boot Actuator is fully enabled. This exposes sensitive endpoints such as /
actuator/env, /actuator/logfile, /actuator/heapdump and others. Unless you have Spring Security
enabled or another means to protect these endpoints, this functionality is available without aut
hentication, causing a severe security risk.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        cwe2021-top25: true
        impact: HIGH
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://docs.spring.io/spring-boot/docs/current/reference/html/production-ready-fe
atures.html#production-ready-endpoints-exposing-endpoints
            - https://medium.com/walmartglobaltech/perils-of-spring-boot-actuators-misconfigurat
ion-185c43a0f785
            - https://blog.maass.xyz/spring-actuator-security-part-1-stealing-secrets-using-spri
ng-actuators
        subcategory:
            - vuln
        technology:
            - spring
      patterns:
        - pattern-inside: |
            management:
              ...
              endpoints:
                ...
                web:
                  ...
                  exposure:
                    ...
        - pattern: |
            include: "*"
      severity: WARNING
    - id: java.spring.security.audit.spring-actuator-fully-enabled.spring-actuator-fully-enabled
      languages:
        - generic
      message: Spring Boot Actuator is fully enabled. This exposes sensitive endpoints such as /
actuator/env, /actuator/logfile, /actuator/heapdump and others. Unless you have Spring Security
enabled or another means to protect these endpoints, this functionality is available without aut
hentication, causing a significant security risk.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        cwe2021-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://docs.spring.io/spring-boot/docs/current/reference/html/production-ready-fe
atures.html#production-ready-endpoints-exposing-endpoints
            - https://medium.com/walmartglobaltech/perils-of-spring-boot-actuators-misconfigurat
ion-185c43a0f785
            - https://blog.maass.xyz/spring-actuator-security-part-1-stealing-secrets-using-spri
ng-actuators
        subcategory:
            - vuln
        technology:
            - spring
      paths:
        include:
            - '*properties'
      pattern: management.endpoints.web.exposure.include=*
      severity: ERROR
    - id: java.spring.security.audit.spring-actuator-non-health-enabled-yaml.spring-actuator-dan
gerous-endpoints-enabled-yaml
      languages:
        - yaml
      message: Spring Boot Actuator "$ACTUATOR" is enabled. Depending on the actuator, this can
pose a significant security risk. Please double-check if the actuator is needed and properly sec
ured.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        cwe2021-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://docs.spring.io/spring-boot/docs/current/reference/html/production-ready-fe
atures.html#production-ready-endpoints-exposing-endpoints
            - https://medium.com/walmartglobaltech/perils-of-spring-boot-actuators-misconfigurat
ion-185c43a0f785
            - https://blog.maass.xyz/spring-actuator-security-part-1-stealing-secrets-using-spri
ng-actuators
        subcategory:
            - vuln
        technology:
            - spring
      patterns:
        - pattern-inside: |
            management:
              ...
              endpoints:
                ...
                web:
                  ...
                  exposure:
                    ...
                    include:
                      ...
        - pattern: |
            include: [..., $ACTUATOR, ...]
        - metavariable-comparison:
            comparison: not str($ACTUATOR) in ["health","*"]
            metavariable: $ACTUATOR
      severity: WARNING
    - id: java.spring.security.audit.spring-actuator-non-health-enabled.spring-actuator-dangerou
s-endpoints-enabled
      languages:
        - generic
      message: Spring Boot Actuators "$...ACTUATORS" are enabled. Depending on the actuators, th
is can pose a significant security risk. Please double-check if the actuators are needed and pro
perly secured.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        cwe2021-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://docs.spring.io/spring-boot/docs/current/reference/html/production-ready-fe
atures.html#production-ready-endpoints-exposing-endpoints
            - https://medium.com/walmartglobaltech/perils-of-spring-boot-actuators-misconfigurat
ion-185c43a0f785
            - https://blog.maass.xyz/spring-actuator-security-part-1-stealing-secrets-using-spri
ng-actuators
        subcategory:
            - vuln
        technology:
            - spring
      options:
        generic_ellipsis_max_span: 0
      patterns:
        - pattern: management.endpoints.web.exposure.include=$...ACTUATORS
        - metavariable-comparison:
            comparison: not str($...ACTUATORS) in ["health","*"]
            metavariable: $...ACTUATORS
      severity: WARNING
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: java.spring.security.audit.spring-sqli.spring-sqli
      languages:
        - java
      message: Detected a string argument from a public method contract in a raw SQL statement.
This could lead to SQL injection if variables in the SQL statement are not properly sanitized. U
se a prepared statements (java.sql.PreparedStatement) instead. You can obtain a PreparedStatemen
t using 'connection.prepareStatement'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - spring
      mode: taint
      options:
        taint_assume_safe_booleans: true
        taint_assume_safe_numbers: true
      pattern-sanitizers:
        - not_conflicting: true
          pattern-either:
            - patterns:
                - focus-metavariable: $A
                - pattern-inside: |
                    new $TYPE(...,$A,...);
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - focus-metavariable: $A
                    - pattern: |
                        new PreparedStatementCreatorFactory($A,...);
                - patterns:
                    - focus-metavariable: $A
                    - pattern: |
                        (JdbcTemplate $T).$M($A,...)
                - patterns:
                    - pattern: (String $A)
                    - pattern-inside: |
                        (JdbcTemplate $T).batchUpdate(...)
                - patterns:
                    - focus-metavariable: $A
                    - pattern: |
                        NamedParameterBatchUpdateUtils.$M($A,...)
                - patterns:
                    - focus-metavariable: $A
                    - pattern: |
                        BatchUpdateUtils.$M($A,...)
      pattern-sources:
        - patterns:
            - pattern: $ARG
            - pattern-inside: |
                public $T $M (..., String $ARG,...){...}
      severity: WARNING
    - id: java.spring.security.audit.spring-unvalidated-redirect.spring-unvalidated-redirect
      languages:
        - java
      message: Application redirects a user to a destination URL specified by a user supplied pa
rameter that is not validated.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-601: URL Redirection to Untrusted Site (''Open Redirect'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#UNVALIDATED_REDIRECT
        subcategory:
            - vuln
        technology:
            - spring
      pattern-either:
        - pattern: |
            $X $METHOD(...,String $URL,...) {
              return "redirect:" + $URL;
            }
        - pattern: |
            $X $METHOD(...,String $URL,...) {
              ...
              String $REDIR = "redirect:" + $URL;
              ...
              return $REDIR;
              ...
            }
        - pattern: |
            $X $METHOD(...,String $URL,...) {
              ...
              new ModelAndView("redirect:" + $URL);
              ...
            }
        - pattern: |-
            $X $METHOD(...,String $URL,...) {
              ...
              String $REDIR = "redirect:" + $URL;
              ...
              new ModelAndView($REDIR);
              ...
            }
      severity: WARNING
    - id: java.spring.security.injection.tainted-file-path.tainted-file-path
      languages:
        - java
      message: Detected user input controlling a file path. An attacker could control the locati
on of this file, to include going backwards in the directory with '../'. To address this, ensure
 that user-controlled variables in file paths are sanitized. You may also consider using a utili
ty method such as org.apache.commons.io.FilenameUtils.getName(...) to only retrieve the file nam
e from the path.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-23: Relative Path Traversal'
        impact: HIGH
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/www-community/attacks/Path_Traversal
        subcategory:
            - vuln
        technology:
            - java
            - spring
      mode: taint
      options:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        interfile: true
      pattern-sanitizers:
        - pattern: org.apache.commons.io.FilenameUtils.getName(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: new File(...)
                - pattern: new java.io.File(...)
                - pattern: new FileReader(...)
                - pattern: new java.io.FileReader(...)
                - pattern: new FileInputStream(...)
                - pattern: new java.io.FileInputStream(...)
                - pattern: (Paths $PATHS).get(...)
                - patterns:
                    - pattern: |
                        $CLASS.$FUNC(...)
                    - metavariable-regex:
                        metavariable: $FUNC
                        regex: ^(getResourceAsStream|getResource)$
                - patterns:
                    - pattern-either:
                        - pattern: new ClassPathResource($FILE, ...)
                        - pattern: ResourceUtils.getFile($FILE, ...)
                        - pattern: new FileOutputStream($FILE, ...)
                        - pattern: new java.io.FileOutputStream($FILE, ...)
                        - pattern: new StreamSource($FILE, ...)
                        - pattern: new javax.xml.transform.StreamSource($FILE, ...)
                        - pattern: FileUtils.openOutputStream($FILE, ...)
                    - focus-metavariable: $FILE
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $METHODNAME(..., @$REQ(...) $TYPE $SOURCE,...) {
                      ...
                    }
                - pattern-inside: |
                    $METHODNAME(..., @$REQ $TYPE $SOURCE,...) {
                      ...
                    }
            - metavariable-regex:
                metavariable: $TYPE
                regex: ^(?!(Integer|Long|Float|Double|Char|Boolean|int|long|float|double|char|bo
olean))
            - metavariable-regex:
                metavariable: $REQ
                regex: (RequestBody|PathVariable|RequestParam|RequestHeader|CookieValue|ModelAtt
ribute)
            - focus-metavariable: $SOURCE
      severity: ERROR
    - id: java.spring.security.injection.tainted-html-string.tainted-html-string
      languages:
        - java
      message: Detected user input flowing into a manually constructed HTML string. You may be a
ccidentally bypassing secure methods of rendering HTML by manually constructing HTML and this co
uld create a cross-site scripting vulnerability, which could let attackers steal sensitive user
data. To be sure this is safe, check that the HTML is rendered safely. You can use the OWASP ESA
PI encoder if you must render user data.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Che
at_Sheet.html
        subcategory:
            - vuln
        technology:
            - java
            - spring
      mode: taint
      pattern-propagators:
        - from: $...TAINTED
          pattern: (StringBuilder $SB).append($...TAINTED)
          to: $SB
        - from: $...TAINTED
          pattern: $VAR += $...TAINTED
          to: $VAR
      pattern-sanitizers:
        - pattern-either:
            - pattern: Encode.forHtml(...)
            - pattern: (PolicyFactory $POLICY).sanitize(...)
            - pattern: (AntiSamy $AS).scan(...)
            - pattern: JSoup.clean(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: new ResponseEntity<>($PAYLOAD, ...)
                - pattern: new ResponseEntity<$ERROR>($PAYLOAD, ...)
                - pattern: ResponseEntity. ... .body($PAYLOAD)
                - patterns:
                    - pattern: |
                        ResponseEntity.$RESPFUNC($PAYLOAD). ...
                    - metavariable-regex:
                        metavariable: $RESPFUNC
                        regex: ^(ok|of)$
            - focus-metavariable: $PAYLOAD
          requires: CONCAT
      pattern-sources:
        - label: INPUT
          patterns:
            - pattern-either:
                - pattern-inside: |
                    $METHODNAME(..., @$REQ(...) $TYPE $SOURCE,...) {
                      ...
                    }
                - pattern-inside: |
                    $METHODNAME(..., @$REQ $TYPE $SOURCE,...) {
                      ...
                    }
            - metavariable-regex:
                metavariable: $TYPE
                regex: ^(?!(Integer|Long|Float|Double|Char|Boolean|int|long|float|double|char|bo
olean))
            - metavariable-regex:
                metavariable: $REQ
                regex: (RequestBody|PathVariable|RequestParam|RequestHeader|CookieValue|ModelAtt
ribute)
            - focus-metavariable: $SOURCE
        - by-side-effect: true
          label: CONCAT
          patterns:
            - pattern-either:
                - pattern: |
                    "$HTMLSTR" + ...
                - pattern: |
                    "$HTMLSTR".concat(...)
                - patterns:
                    - pattern-inside: |
                        StringBuilder $SB = new StringBuilder("$HTMLSTR");
                        ...
                    - pattern: $SB.append(...)
                - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern-inside: |
                        $VAR = "$HTMLSTR";
                        ...
                    - pattern: $VAR += ...
                - pattern: String.format("$HTMLSTR", ...)
                - patterns:
                    - pattern-inside: |
                        String $VAR = "$HTMLSTR";
                        ...
                    - pattern: String.format($VAR, ...)
            - metavariable-regex:
                metavariable: $HTMLSTR
                regex: ^<\w+
          requires: INPUT
      severity: ERROR
    - id: java.spring.security.injection.tainted-sql-string.tainted-sql-string
      languages:
        - java
      message: User data flows into this manually-constructed SQL string. User data can be safel
y inserted into SQL strings using prepared statements or an object-relational mapper (ORM). Manu
ally-constructed SQL strings is a possible indicator of SQL injection, which could let an attack
er steal or manipulate data from the database. Instead, use prepared statements (`connection.Pre
paredStatement`) or a safe library.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.oracle.com/javase/7/docs/api/java/sql/PreparedStatement.html
        subcategory:
            - vuln
        technology:
            - spring
      mode: taint
      options:
        interfile: true
        taint_assume_safe_booleans: true
        taint_assume_safe_numbers: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    "$SQLSTR" + ...
                - pattern: |
                    "$SQLSTR".concat(...)
                - patterns:
                    - pattern-inside: |
                        StringBuilder $SB = new StringBuilder("$SQLSTR");
                        ...
                    - pattern: $SB.append(...)
                - patterns:
                    - pattern-inside: |
                        $VAR = "$SQLSTR";
                        ...
                    - pattern: $VAR += ...
                - pattern: String.format("$SQLSTR", ...)
                - patterns:
                    - pattern-inside: |
                        String $VAR = "$SQLSTR";
                        ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern: String.format($VAR, ...)
            - pattern-not-inside: System.out.println(...)
            - pattern-not-inside: $LOG.info(...)
            - pattern-not-inside: $LOG.warn(...)
            - pattern-not-inside: $LOG.warning(...)
            - pattern-not-inside: $LOG.debug(...)
            - pattern-not-inside: $LOG.debugging(...)
            - pattern-not-inside: $LOG.error(...)
            - pattern-not-inside: new Exception(...)
            - pattern-not-inside: throw ...;
            - metavariable-regex:
                metavariable: $SQLSTR
                regex: (?i)(select|delete|insert|create|update|alter|drop)\b
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $METHODNAME(..., @$REQ(...) $TYPE $SOURCE,...) {
                      ...
                    }
                - pattern-inside: |
                    $METHODNAME(..., @$REQ $TYPE $SOURCE,...) {
                      ...
                    }
            - metavariable-regex:
                metavariable: $REQ
                regex: (RequestBody|PathVariable|RequestParam|RequestHeader|CookieValue)
            - metavariable-regex:
                metavariable: $TYPE
                regex: ^(?!(Integer|Long|Float|Double|Char|Boolean|int|long|float|double|char|bo
olean))
            - focus-metavariable: $SOURCE
      severity: ERROR
    - id: java.spring.security.injection.tainted-system-command.tainted-system-command
      languages:
        - java
      message: 'Detected user input entering a method which executes a system command. This coul
d result in a command injection vulnerability, which allows an attacker to inject an arbitrary s
ystem command onto the server. The attacker could download malware onto or steal data from the s
erver. Instead, use ProcessBuilder, separating the command into individual arguments, like this:
 `new ProcessBuilder("ls", "-al", targetDirectory)`. Further, make sure you hardcode or allowlis
t the actual command so that attackers can''t run arbitrary commands.'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://www.stackhawk.com/blog/command-injection-java/
            - https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_
Sheet.html
            - https://github.com/github/codeql/blob/main/java/ql/src/Security/CWE/CWE-078/ExecUn
escaped.java
        subcategory:
            - vuln
        technology:
            - java
            - spring
      mode: taint
      pattern-propagators:
        - from: $INPUT
          label: CONCAT
          pattern: (StringBuilder $STRB).append($INPUT)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          requires: INPUT
          to: $STRB
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    (Process $P) = new Process(...);
                - pattern: |
                    (ProcessBuilder $PB).command(...);
                - patterns:
                    - pattern-either:
                        - pattern: |
                            (Runtime $R).$EXEC(...);
                        - pattern: |
                            Runtime.getRuntime(...).$EXEC(...);
                    - metavariable-regex:
                        metavariable: $EXEC
                        regex: (exec|loadLibrary|load)
                - patterns:
                    - pattern: |
                        (ProcessBuilder $PB).command(...).$ADD(...);
                    - metavariable-regex:
                        metavariable: $ADD
                        regex: (add|addAll)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-inside: |
                                $BUILDER = new ProcessBuilder(...);
                                ...
                            - pattern: $BUILDER.start(...)
                        - pattern: |
                            new ProcessBuilder(...). ... .start(...);
          requires: CONCAT
      pattern-sources:
        - label: INPUT
          patterns:
            - pattern-either:
                - pattern-inside: |
                    $METHODNAME(..., @$REQ(...) $TYPE $SOURCE,...) {
                      ...
                    }
                - pattern-inside: |
                    $METHODNAME(..., @$REQ $TYPE $SOURCE,...) {
                      ...
                    }
            - metavariable-regex:
                metavariable: $TYPE
                regex: ^(?!(Integer|Long|Float|Double|Char|Boolean|int|long|float|double|char|bo
olean))
            - metavariable-regex:
                metavariable: $REQ
                regex: (RequestBody|PathVariable|RequestParam|RequestHeader|CookieValue|ModelAtt
ribute)
            - focus-metavariable: $SOURCE
        - label: CONCAT
          patterns:
            - pattern-either:
                - pattern: $X + $SOURCE
                - pattern: $SOURCE + $Y
                - pattern: String.format("...", ..., $SOURCE, ...)
                - pattern: String.join("...", ..., $SOURCE, ...)
                - pattern: (String $STR).concat($SOURCE)
                - pattern: $SOURCE.concat(...)
                - pattern: $X += $SOURCE
                - pattern: $SOURCE += $X
          requires: INPUT
      severity: ERROR
    - id: java.spring.security.injection.tainted-url-host.tainted-url-host
      languages:
        - java
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: User data flows into the host portion of this manually-constructed URL. This coul
d allow an attacker to send data to their own server, potentially exposing sensitive data such a
s cookies or authorization information sent with this request. They could also probe internal se
rvers or other resources that the server running this code can access. (This is called server-si
de request forgery, or SSRF.) Do not allow arbitrary hosts. Instead, create an allowlist for app
roved hosts, hardcode the correct host, or ensure that the user data can only affect the path or
 parameters.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevent
ion_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - java
            - spring
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - pattern-either:
            - pattern: new URL($ONEARG)
            - patterns:
                - pattern-either:
                    - pattern: |
                        "$URLSTR" + ...
                    - pattern: |
                        "$URLSTR".concat(...)
                    - patterns:
                        - pattern-inside: |
                            StringBuilder $SB = new StringBuilder("$URLSTR");
                            ...
                        - pattern: $SB.append(...)
                    - patterns:
                        - pattern-inside: |
                            $VAR = "$URLSTR";
                            ...
                        - pattern: $VAR += ...
                    - patterns:
                        - pattern: String.format("$URLSTR", ...)
                        - pattern-not: String.format("$URLSTR", "...", ...)
                    - patterns:
                        - pattern-inside: |
                            String $VAR = "$URLSTR";
                            ...
                        - pattern: String.format($VAR, ...)
                - metavariable-regex:
                    metavariable: $URLSTR
                    regex: http(s?)://%(v|s|q).*
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $METHODNAME(..., @$REQ(...) $TYPE $SOURCE,...) {
                      ...
                    }
                - pattern-inside: |
                    $METHODNAME(..., @$REQ $TYPE $SOURCE,...) {
                      ...
                    }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - metavariable-regex:
                metavariable: $TYPE
                regex: ^(?!(Integer|Long|Float|Double|Char|Boolean|int|long|float|double|char|bo
olean))
            - metavariable-regex:
                metavariable: $REQ
                regex: (RequestBody|PathVariable|RequestParam|RequestHeader|CookieValue|ModelAtt
ribute)
            - focus-metavariable: $SOURCE
      severity: ERROR
    - id: javascript.angular.security.detect-angular-element-taint.detect-angular-element-taint
      languages:
        - javascript
        - typescript
      message: Use of angular.element can lead to XSS if user-input is treated as part of the HT
ML element within `$SINK`. It is recommended to contextually output encode user-input, before in
serting into `$SINK`. If the HTML needs to be preserved it is recommended to sanitize the input
using $sce.getTrustedHTML or $sanitize.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://docs.angularjs.org/api/ng/function/angular.element
            - https://owasp.org/www-chapter-london/assets/slides/OWASPLondon20170727_AngularJS.p
df
        subcategory:
            - vuln
        technology:
            - angularjs
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern: $sce.getTrustedHtml(...)
                - pattern: $sanitize(...)
                - pattern: DOMPurify.sanitize(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    angular.element(...). ... .$SINK($QUERY)
                - pattern-inside: |
                    $ANGULAR = angular.element(...)
                    ...
                    $ANGULAR. ... .$SINK($QUERY)
            - metavariable-regex:
                metavariable: $SINK
                regex: ^(after|append|html|prepend|replaceWith|wrap)$
            - focus-metavariable: $QUERY
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: window.location.search
                - pattern: window.document.location.search
                - pattern: document.location.search
                - pattern: location.search
                - pattern: $location.search(...)
        - patterns:
            - pattern-either:
                - pattern: $DECODE(<... location.hash ...>)
                - pattern: $DECODE(<... window.location.hash ...>)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: $DECODE(<... document.location.hash ...>)
                - pattern: $DECODE(<... location.href ...>)
                - pattern: $DECODE(<... window.location.href ...>)
                - pattern: $DECODE(<... document.location.href ...>)
                - pattern: $DECODE(<... document.URL ...>)
                - pattern: $DECODE(<... window.document.URL ...>)
                - pattern: $DECODE(<... document.location.href ...>)
                - pattern: $DECODE(<... document.location.href ...>)
                - pattern: $DECODE(<... $location.absUrl() ...>)
                - pattern: $DECODE(<... $location.url() ...>)
                - pattern: $DECODE(<... $location.hash() ...>)
            - metavariable-regex:
                metavariable: $DECODE
                regex: ^(unescape|decodeURI|decodeURIComponent)$
        - patterns:
            - pattern-inside: $http.$METHOD(...).$CONTINUE(function $FUNC($RES) {...})
            - metavariable-regex:
                metavariable: $METHOD
                regex: ^(get|delete|head|jsonp|post|put|patch)
            - pattern: $RES.data
      severity: WARNING
    - id: javascript.angular.security.detect-angular-sce-disabled.detect-angular-sce-disabled
      languages:
        - javascript
        - typescript
      message: $sceProvider is set to false. Disabling Strict Contextual escaping (SCE) in an An
gularJS application could provide additional attack surface for XSS vulnerabilities.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://docs.angularjs.org/api/ng/service/$sce
            - https://owasp.org/www-chapter-london/assets/slides/OWASPLondon20170727_AngularJS.p
df
        subcategory:
            - vuln
        technology:
            - angular
      pattern: |
        $sceProvider.enabled(false);
      severity: ERROR
    - id: javascript.angular.security.detect-angular-trust-as-method.detect-angular-trust-as-met
hod
      languages:
        - javascript
        - typescript
      message: The use of $sce.trustAs can be dangerous if unsanitized user input flows through
this API.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        references:
            - https://docs.angularjs.org/api/ng/service/$sce
            - https://owasp.org/www-chapter-london/assets/slides/OWASPLondon20170727_AngularJS.p
df
        subcategory:
            - vuln
        technology:
            - angular
      mode: taint
      pattern-sinks:
        - pattern: $sce.trustAs(...)
        - pattern: $sce.trustAsHtml(...)
      pattern-sources:
        - patterns:
            - pattern-inside: |
                app.controller(..., function($scope,$sce) {
                ...
                });
            - pattern: $scope.$X
      severity: WARNING
    - id: javascript.argon2.security.unsafe-argon2-config.unsafe-argon2-config
      languages:
        - javascript
        - typescript
      message: Prefer Argon2id where possible. Per RFC9016, section 4 IETF recommends selecting
Argon2id unless you can guarantee an adversary has no direct access to the computing environment
.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-916: Use of Password Hash With Insufficient Computational Effort'
        impact: LOW
        likelihood: HIGH
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
            - https://eprint.iacr.org/2016/759.pdf
            - https://www.cs.tau.ac.il/~tromer/papers/cache-joc-20090619.pdf
            - https://datatracker.ietf.org/doc/html/rfc9106#section-4
        subcategory:
            - vuln
        technology:
            - argon2
            - cryptography
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern: |
                {type: $ARGON.argon2id}
                ...
      pattern-sinks:
        - patterns:
            - pattern: |
                $Y
            - pattern-inside: |
                $ARGON.hash(...,$Y)
      pattern-sources:
        - patterns:
            - pattern-inside: |
                $ARGON = require('argon2');
                ...
            - pattern: |
                {type: ...}
      severity: WARNING
    - id: javascript.aws-lambda.security.detect-child-process.detect-child-process
      languages:
        - javascript
        - typescript
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Allowing spawning arbitrary programs or running shell processes with arbitrary ar
guments may end up in a command injection vulnerability. Try to avoid non-literal values for the
 command string. If it is not possible, then do not let running arbitrary commands, use a white
list for inputs.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - javascript
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $CMD
            - pattern-either:
                - pattern: exec($CMD,...)
                - pattern: execSync($CMD,...)
                - pattern: spawn($CMD,...)
                - pattern: spawnSync($CMD,...)
                - pattern: $CP.exec($CMD,...)
                - pattern: $CP.execSync($CMD,...)
                - pattern: $CP.spawn($CMD,...)
                - pattern: $CP.spawnSync($CMD,...)
            - pattern-either:
                - pattern-inside: |
                    require('child_process')
                    ...
                - pattern-inside: |
                    import 'child_process'
                    ...
      pattern-sources:
        - patterns:
            - pattern: $EVENT
            - pattern-either:
                - pattern-inside: |
                    exports.handler = function ($EVENT, ...) {
                      ...
                    }
                - pattern-inside: |
                    function $FUNC ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
                - pattern-inside: |
                    $FUNC = function ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
      severity: ERROR
    - id: javascript.aws-lambda.security.dynamodb-request-object.dynamodb-request-object
      languages:
        - javascript
        - typescript
      message: Detected DynamoDB query params that are tainted by `$EVENT` object. This could le
ad to NoSQL injection if the variable is user-controlled and not properly sanitized. Explicitly
assign query params instead of passing data from `$EVENT` directly to DynamoDB client.
      metadata:
        category: security
        confidence: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe:
            - 'CWE-943: Improper Neutralization of Special Elements in Data Query Logic'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - javascript
            - aws-lambda
            - dynamodb
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern: |
                {...}
      pattern-sinks:
        - patterns:
            - focus-metavariable: $SINK
            - pattern: |
                $DC.$METHOD($SINK, ...)
            - metavariable-regex:
                metavariable: $METHOD
                regex: (query|send|scan|delete|put|transactWrite|update|batchExecuteStatement|ex
ecuteStatement|executeTransaction|transactWriteItems)
            - pattern-either:
                - pattern-inside: |
                    $DC = new $AWS.DocumentClient(...);
                    ...
                - pattern-inside: |
                    $DC = new $AWS.DynamoDB(...);
                    ...
                - pattern-inside: |
                    $DC = new DynamoDBClient(...);
                    ...
                - pattern-inside: |
                    $DC = DynamoDBDocumentClient.from(...);
                    ...
      pattern-sources:
        - patterns:
            - pattern: $EVENT
            - pattern-either:
                - pattern-inside: |
                    exports.handler = function ($EVENT, ...) {
                      ...
                    }
                - pattern-inside: |
                    function $FUNC ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
                - pattern-inside: |
                    $FUNC = function ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
      severity: ERROR
    - id: javascript.aws-lambda.security.knex-sqli.knex-sqli
      languages:
        - javascript
        - typescript
      message: 'Detected SQL statement that is tainted by `$EVENT` object. This could lead to SQ
L injection if the variable is user-controlled and not properly sanitized. In order to prevent S
QL injection, use parameterized queries or prepared statements instead. You can use parameterize
d statements like so: `knex.raw(''SELECT $1 from table'', [userinput])`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
       cwe2021-top25: true
       cwe2022-top25: true
       impact: HIGH
       likelihood: MEDIUM
       owasp:
           - A01:2017 - Injection
           - A03:2021 - Injection
       references:
           - https://knexjs.org/#Builder-fromRaw
           - https://knexjs.org/#Builder-whereRaw
       subcategory:
           - vuln
       technology:
           - aws-lambda
           - knex
     mode: taint
     pattern-sinks:
       - patterns:
           - focus-metavariable: $QUERY
           - pattern-either:
               - pattern: $KNEX.fromRaw($QUERY, ...)
               - pattern: $KNEX.whereRaw($QUERY, ...)
               - pattern: $KNEX.raw($QUERY, ...)
           - pattern-either:
               - pattern-inside: |
                   require('knex')
                   ...
               - pattern-inside: |
                   import 'knex'
                   ...
     pattern-sources:
       - patterns:
           - pattern-either:
               - pattern-inside: |
                   exports.handler = function ($EVENT, ...) {
                     ...
                   }
               - pattern-inside: |
                   function $FUNC ($EVENT, ...) {...}
                   ...
                   exports.handler = $FUNC
               - pattern-inside: |
                   $FUNC = function ($EVENT, ...) {...}
                   ...
                   exports.handler = $FUNC
           - pattern: $EVENT
     severity: WARNING
   - id: javascript.aws-lambda.security.mysql-sqli.mysql-sqli
     languages:
       - javascript
       - typescript
     message: 'Detected SQL statement that is tainted by `$EVENT` object. This could lead to SQ
L injection if the variable is user-controlled and not properly sanitized. In order to prevent S
QL injection, use parameterized queries or prepared statements instead. You can use parameterize
d statements like so: `connection.query(''SELECT $1 from table'', [userinput])`'
     metadata:
       category: security
       confidence: MEDIUM
       cwe:
           - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
       cwe2021-top25: true
       cwe2022-top25: true
       impact: HIGH
       likelihood: MEDIUM
       owasp:
           - A01:2017 - Injection
           - A03:2021 - Injection
       references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - https://www.npmjs.com/package/mysql2
        subcategory:
          - vuln
        technology:
          - aws-lambda
          - mysql
          - mysql2
      mode: taint
      pattern-sinks:
        - patterns:
          - focus-metavariable: $QUERY
          - pattern-either:
            - pattern: $POOL.query($QUERY, ...)
            - pattern: $POOL.execute($QUERY, ...)
          - pattern-either:
            - pattern-inside: |
                require('mysql')
                ...
            - pattern-inside: |
                require('mysql2')
                ...
            - pattern-inside: |
                require('mysql2/promise')
                ...
            - pattern-inside: |
                import 'mysql'
                ...
            - pattern-inside: |
                import 'mysql2'
                ...
            - pattern-inside: |
                import 'mysql2/promise'
                ...
      pattern-sources:
        - patterns:
          - pattern-either:
            - pattern-inside: |
                exports.handler = function ($EVENT, ...) {
                  ...
                }
            - pattern-inside: |
                function $FUNC ($EVENT, ...) {...}
                ...
                exports.handler = $FUNC
            - pattern-inside: |
                $FUNC = function ($EVENT, ...) {...}
                ...
                exports.handler = $FUNC
          - pattern: $EVENT
      severity: WARNING
    - id: javascript.aws-lambda.security.pg-sqli.pg-sqli
      languages:
        - javascript
        - typescript
      message: 'Detected SQL statement that is tainted by `$EVENT` object. This could lead to SQ
L injection if the variable is user-controlled and not properly sanitized. In order to prevent S
QL injection, use parameterized queries or prepared statements instead. You can use parameterize
d statements like so: `connection.query(''SELECT $1 from table'', [userinput])`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
          - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
          - A01:2017 - Injection
          - A03:2021 - Injection
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        references:
            - https://node-postgres.com/features/queries
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - postgres
            - pg
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $QUERY
            - pattern-either:
                - pattern: $DB.query($QUERY, ...)
            - pattern-either:
                - pattern-inside: |
                    require('pg')
                    ...
                - pattern-inside: |
                    import 'pg'
                    ...
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    exports.handler = function ($EVENT, ...) {
                      ...
                    }
                - pattern-inside: |
                    function $FUNC ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
                - pattern-inside: |
                    $FUNC = function ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
            - pattern: $EVENT
      severity: WARNING
    - id: javascript.aws-lambda.security.sequelize-sqli.sequelize-sqli
      languages:
        - javascript
        - typescript
      message: 'Detected SQL statement that is tainted by `$EVENT` object. This could lead to SQ
L injection if the variable is user-controlled and not properly sanitized. In order to prevent S
QL injection, use parameterized queries or prepared statements instead. You can use parameterize
d statements like so: `sequelize.query(''SELECT * FROM projects WHERE status = ?'', { replacemen
ts: [''active''], type: QueryTypes.SELECT });`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://sequelize.org/master/manual/raw-queries.html
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - sequelize
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $QUERY
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern-either:
                - pattern: $DB.query($QUERY, ...)
            - pattern-either:
                - pattern-inside: |
                    require('sequelize')
                    ...
                - pattern-inside: |
                    import 'sequelize'
                    ...
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    exports.handler = function ($EVENT, ...) {
                      ...
                    }
                - pattern-inside: |
                    function $FUNC ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
                - pattern-inside: |
                    $FUNC = function ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
            - pattern: $EVENT
      severity: WARNING
    - id: javascript.aws-lambda.security.tainted-html-response.tainted-html-response
      languages:
        - javascript
        - typescript
      message: Detected user input flowing into an HTML response. You may be accidentally bypass
ing secure methods of rendering HTML by manually constructing HTML and this could create a cross
-site scripting vulnerability, which could let attackers steal sensitive user data.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $BODY
            - pattern-inside: |
                {..., headers: {..., 'Content-Type': 'text/html', ...}, body: $BODY, ... }
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    exports.handler = function ($EVENT, ...) {
                      ...
                    }
                - pattern-inside: |
                    function $FUNC ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
                - pattern-inside: |
                    $FUNC = function ($EVENT, ...) {...}
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    ...
                    exports.handler = $FUNC
            - pattern: $EVENT
      severity: WARNING
    - id: javascript.aws-lambda.security.tainted-html-string.tainted-html-string
      languages:
        - javascript
        - typescript
      message: Detected user input flowing into a manually constructed HTML string. You may be a
ccidentally bypassing secure methods of rendering HTML by manually constructing HTML and this co
uld create a cross-site scripting vulnerability, which could let attackers steal sensitive user
data. To be sure this is safe, check that the HTML is rendered safely. Otherwise, use templates
which will safely render HTML instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: |
                            "$HTMLSTR" + $EXPR
                        - pattern: |
                            "$HTMLSTR".concat(...)
                        - pattern: $UTIL.format($HTMLSTR, ...)
                        - pattern: format($HTMLSTR, ...)
                    - metavariable-pattern:
                        language: generic
                        metavariable: $HTMLSTR
                        pattern: <$TAG ...
                - patterns:
                    - pattern: |
                        `...${...}...`
                    - pattern-regex: |
                        .*<\w+.*
            - pattern-not-inside: |
                console.$LOG(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    exports.handler = function ($EVENT, ...) {
                      ...
                    }
                - pattern-inside: |
                    function $FUNC ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
                - pattern-inside: |
                    $FUNC = function ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
            - pattern: $EVENT
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: WARNING
    - id: javascript.aws-lambda.security.vm-runincontext-injection.vm-runincontext-injection
      languages:
        - javascript
        - typescript
      message: The `vm` module enables compiling and running code within V8 Virtual Machine cont
exts. The `vm` module is not a security mechanism. Do not use it to run untrusted code. If code
passed to `vm` functions is controlled by user input it could result in command injection. Do no
t let user input in `vm` functions.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - javascript
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    require('vm');
                    ...
                - pattern-inside: |
                    import 'vm'
                    ...
            - pattern-either:
                - pattern: $VM.runInContext($X,...)
                - pattern: $VM.runInNewContext($X,...)
                - pattern: $VM.runInThisContext($X,...)
                - pattern: $VM.compileFunction($X,...)
                - pattern: new $VM.Script($X,...)
                - pattern: new $VM.SourceTextModule($X,...)
                - pattern: runInContext($X,...)
                - pattern: runInNewContext($X,...)
                - pattern: runInThisContext($X,...)
                - pattern: compileFunction($X,...)
                - pattern: new Script($X,...)
                - pattern: new SourceTextModule($X,...)
      pattern-sources:
        - patterns:
            - pattern: $EVENT
            - pattern-either:
                - pattern-inside: |
                    exports.handler = function ($EVENT, ...) {
                      ...
                    }
                - pattern-inside: |
                    function $FUNC ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
                - pattern-inside: |
                    $FUNC = function ($EVENT, ...) {...}
                    ...
                    exports.handler = $FUNC
      severity: ERROR
    - id: javascript.browser.security.open-redirect.js-open-redirect
      languages:
        - javascript
        - typescript
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: The application accepts potentially user-controlled input `$PROP` which can contr
ol the location of the current window context. This can lead two types of vulnerabilities open-r
edirection and Cross-Site-Scripting (XSS) with JavaScript URIs. It is recommended to validate us
er-controllable input before allowing it to control the redirection.
      metadata:
        asvs:
            control_id: 5.5.1 Insecue Redirect
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v51-input-validation
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-601: URL Redirection to Untrusted Site (''Open Redirect'')'
        impact: MEDIUM
        interfile: true
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_
Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - browser
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: location.href = $SINK
                - pattern: $THIS. ... .location.href = $SINK
                - pattern: location.replace($SINK)
                - pattern: $THIS. ... .location.replace($SINK)
                - pattern: location = $SINK
                - pattern: $WINDOW. ... .location = $SINK
            - focus-metavariable: $SINK
            - metavariable-pattern:
                metavariable: $SINK
                patterns:
                    - pattern-not: |
                        "..." + $VALUE
                    - pattern-not: |
                        `...${$VALUE}`
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $PROP = new URLSearchParams($WINDOW. ... .location.search).get('...')
                    ...
                - pattern-inside: |
                    $PROP = new URLSearchParams(location.search).get('...')
                    ...
                - pattern-inside: |
                    $PROP = new URLSearchParams($WINDOW. ... .location.hash.substring(1)).get('.
..')
                    ...
                - pattern-inside: |
                    $PROP = new URLSearchParams(location.hash.substring(1)).get('...')
                    ...
            - pattern: $PROP
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $PROPS = new URLSearchParams($WINDOW. ... .location.search)
                    ...
                - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    $PROPS = new URLSearchParams(location.search)
                    ...
                - pattern-inside: |
                    $PROPS = new URLSearchParams($WINDOW. ... .location.hash.substring(1))
                    ...
                - pattern-inside: |
                    $PROPS = new URLSearchParams(location.hash.substring(1))
                    ...
            - pattern: $PROPS.get('...')
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $PROPS = new URL($WINDOW. ... .location.href)
                    ...
                - pattern-inside: |
                    $PROPS = new URL(location.href)
                    ...
            - pattern: $PROPS.searchParams.get('...')
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $PROPS = new URL($WINDOW. ... .location.href).searchParams.get('...')
                    ...
                - pattern-inside: |
                    $PROPS = new URL(location.href).searchParams.get('...')
                    ...
            - pattern: $PROPS
      severity: WARNING
    - id: javascript.browser.security.raw-html-concat.raw-html-concat
      languages:
        - javascript
        - typescript
      message: User controlled data in a HTML string may result in XSS
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/xss/
        subcategory:
            - vuln
        technology:
            - browser
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import $S from "underscore.string"
                    ...
                - pattern-inside: |
                    import * as $S from "underscore.string"
                    ...
                - pattern-inside: |
                    import $S from "underscore.string"
                    ...
                - pattern-inside: |
                    $S = require("underscore.string")
                    ...
            - pattern-either:
                - pattern: $S.escapeHTML(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - patterns:
        - pattern-either:
            - pattern-inside: |
                import $S from "dompurify"
                ...
            - pattern-inside: |
                import { ..., $S,... } from "dompurify"
                ...
            - pattern-inside: |
                import * as $S from "dompurify"
                ...
            - pattern-inside: |
                $S = require("dompurify")
                ...
            - pattern-inside: |
                import $S from "isomorphic-dompurify"
                ...
            - pattern-inside: |
                import * as $S from "isomorphic-dompurify"
                ...
            - pattern-inside: |
                $S = require("isomorphic-dompurify")
                ...
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    $VALUE = $S(...)
                    ...
                - pattern: $VALUE.sanitize(...)
            - patterns:
                - pattern-inside: |
                    $VALUE = $S.sanitize
                    ...
                - pattern: $S(...)
            - pattern: $S.sanitize(...)
            - pattern: $S(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                import $S from 'xss';
                ...
            - pattern-inside: |
                import * as $S from 'xss';
                ...
            - pattern-inside: |
                $S = require("xss")
                ...
        - pattern: $S(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                import $S from 'sanitize-html';
                ...
            - pattern-inside: |
                import * as $S from "sanitize-html";
                ...
            - pattern-inside: |
                $S = require("sanitize-html")
                ...
        - pattern: $S(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                $S = new Remarkable()
                ...
        - pattern: $S.render(...)
  pattern-sinks:
    - patterns:
        - pattern-either:
            - patterns:
                - pattern: $STRING + $EXPR
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern-not: $STRING + "..."
                    - metavariable-pattern:
                        language: generic
                        metavariable: $STRING
                        patterns:
                            - pattern: <$TAG ...
                            - pattern-not: <$TAG ...>...</$TAG>...
                - patterns:
                    - pattern: $EXPR + $STRING
                    - pattern-not: '"..." + $STRING'
                    - metavariable-pattern:
                        language: generic
                        metavariable: $STRING
                        patterns:
                            - pattern: '... </$TAG'
                - patterns:
                    - pattern: '[..., $STRING, ...].join(...)'
                    - metavariable-pattern:
                        language: generic
                        metavariable: $STRING
                        patterns:
                            - pattern: <$TAG ...
                - patterns:
                    - pattern: '[..., $STRING, ...].join(...)'
                    - metavariable-pattern:
                        language: generic
                        metavariable: $STRING
                        patterns:
                            - pattern: '... </$TAG'
                - patterns:
                    - pattern: $VAR += $STRING
                    - metavariable-pattern:
                        language: generic
                        metavariable: $STRING
                        patterns:
                            - pattern: <$TAG ...
                - patterns:
                    - pattern: $VAR += $STRING
                    - metavariable-pattern:
                        language: generic
                        metavariable: $STRING
                        patterns:
                            - pattern: '... </$TAG'
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: location.href
                - pattern: location.hash
                - pattern: location.search
                - pattern: $WINDOW. ... .location.href
                - pattern: $WINDOW. ... .location.hash
                - pattern: $WINDOW. ... .location.search
      severity: WARNING
    - id: javascript.browser.security.wildcard-postmessage-configuration.wildcard-postmessage-co
nfiguration
      languages:
        - javascript
        - typescript
      message: The target origin of the window.postMessage() API is set to "*". This could allow
 for information disclosure due to the possibility of any origin allowed to receive the message.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-345: Insufficient Verification of Data Authenticity'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        subcategory:
            - audit
        technology:
            - browser
      pattern: $OBJECT.postMessage(...,'*')
      severity: WARNING
    - id: javascript.chrome-remote-interface.security.audit.chrome-remote-interface-compilescrip
t-injection.chrome-remote-interface-compilescript-injection
      languages:
        - javascript
        - typescript
      message: If unverified user data can reach the `compileScript` method it can result in Ser
ver-Side Request Forgery vulnerabilities
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://github.com/cyrus-and/chrome-remote-interface
        subcategory:
            - vuln
        technology:
            - chrome-remote-interface
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    require('chrome-remote-interface');
                    ...
                - pattern-inside: |
                    import 'chrome-remote-interface';
                    ...
            - pattern-either:
                - pattern: |
                    $RUNTIME.compileScript({expression: $SINK},...)
                - pattern: |
                    $RUNTIME.evaluate({expression: $SINK},...)
                - pattern: |
                    $PAGE.navigate({url: $SINK},...)
                - pattern: |
                    $RUNTIME.printToPDF({headerTemplate: $SINK},...)
                - pattern: |
                    $RUNTIME.printToPDF({footerTemplate: $SINK},...)
                - pattern: |
                    $PAGE.setDocumentContent({html: $SINK},...)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern-inside: function ... (..., $ARG,...) {...}
            - focus-metavariable: $ARG
      severity: WARNING
    - id: javascript.deno.security.audit.deno-dangerous-run.deno-dangerous-run
      languages:
        - javascript
        - typescript
      message: Detected non-literal calls to Deno.run(). This could lead to a command injection
vulnerability.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://deno.land/manual/examples/subprocess#simple-example
        subcategory:
            - vuln
        technology:
            - deno
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    Deno.run({cmd: [$INPUT,...]},...)
                - pattern: |
                    Deno.run({cmd: ["=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c",$INPUT,...]},...)
                - patterns:
                    - pattern: |
                        Deno.run({cmd: [$CMD,"-c",$INPUT,...]},...)
                    - pattern-inside: |
                        $CMD = "=~/(sh|bash|ksh|csh|tcsh|zsh)/"
                        ...
            - focus-metavariable: $INPUT
      pattern-sources:
        - patterns:
            - pattern-inside: function ... (..., $ARG,...) {...}
            - focus-metavariable: $ARG
      severity: ERROR
    - id: javascript.express.security.audit.express-check-directory-listing.express-check-direct
ory-listing
      languages:
        - javascript
        - typescript
      message: Directory listing/indexing is enabled, which may lead to disclosure of sensitive
directories and files. It is recommended to disable directory listing unless it is a public reso
urce. If you need directory listing, ensure that sensitive files are inaccessible when querying
the resource.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-548: Exposure of Information Through Directory Listing'
        impact: MEDIUM
        interfile: true
        likelihood: HIGH
        owasp:
            - A06:2017 - Security Misconfiguration
            - A01:2021 - Broken Access Control
        references:
            - https://www.npmjs.com/package/serve-index
            - https://www.acunetix.com/blog/articles/directory-listing-information-disclosure/
        subcategory:
            - vuln
        technology:
            - express
      options:
        interfile: true
      patterns:
        - pattern-either:
            - pattern: |
                $APP.use(require('serve-index')(...))
            - patterns:
                - pattern-either:
                    - pattern-inside: |
                        $SERVEINDEX = require('serve-index')
                        ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern-inside: |
                        import $SERVEINDEX from 'serve-index'
                        ...
                    - pattern-inside: |
                        import * as $SERVEINDEX from 'serve-index'
                        ...
                - pattern-either:
                    - patterns:
                        - pattern-inside: |
                            $VALUE = $SERVEINDEX(...)
                            ...
                        - pattern: |
                            $VALUE(...)
                    - pattern: |
                        $APP.use(..., $SERVEINDEX(...), ...)
      severity: WARNING
    - id: javascript.express.security.audit.express-cookie-settings.express-cookie-session-defau
lt-name
      languages:
        - javascript
        - typescript
      message: 'Don’t use the default session cookie name Using the default session cookie name
can open your app to attacks. The security issue posed is similar to X-Powered-By: a potential a
ttacker can use it to fingerprint the server and target attacks accordingly.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: LOW
        likelihood: HIGH
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        source-rule-url: https://expressjs.com/en/advanced/best-practice-security.html
        subcategory:
            - vuln
        technology:
            - express
      patterns:
        - pattern-either:
            - pattern-inside: |
                $SESSION = require('cookie-session');
                ...
            - pattern-inside: |
                $SESSION = require('express-session');
                ...
        - pattern: $SESSION(...)
        - pattern-not-inside: $SESSION(<... {name:...} ...>,...)
        - pattern-not-inside: |
            $OPTS = <... {name:...} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.name = ...;
            ...
            $SESSION($OPTS,...);
      severity: WARNING
    - id: javascript.express.security.audit.express-cookie-settings.express-cookie-session-no-se
cure
      languages:
        - javascript
        - typescript
      message: 'Default session middleware settings: `secure` not set. It ensures the browser on
ly sends the cookie over HTTPS.'
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: LOW
        likelihood: HIGH
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        source-rule-url: https://expressjs.com/en/advanced/best-practice-security.html
        subcategory:
            - vuln
        technology:
            - express
      patterns:
        - pattern-either:
            - pattern-inside: |
                $SESSION = require('cookie-session');
                ...
            - pattern-inside: |
                $SESSION = require('express-session');
                ...
        - pattern: $SESSION(...)
        - pattern-not-inside: $SESSION(<... {cookie:{secure:true}} ...>,...)
        - pattern-not-inside: |
            $OPTS = <... {cookie:{secure:true}} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE = <... {secure:true} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.cookie = <... {secure:true} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE.secure = true;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.cookie.secure = true;
            ...
            $SESSION($OPTS,...);
      severity: WARNING
    - id: javascript.express.security.audit.express-cookie-settings.express-cookie-session-no-ht
tponly
      languages:
        - javascript
        - typescript
      message: 'Default session middleware settings: `httpOnly` not set. It ensures the cookie i
s sent only over HTTP(S), not client JavaScript, helping to protect against cross-site scripting
 attacks.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: LOW
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: HIGH
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        source-rule-url: https://expressjs.com/en/advanced/best-practice-security.html
        subcategory:
            - vuln
        technology:
            - express
      patterns:
        - pattern-either:
            - pattern-inside: |
                $SESSION = require('cookie-session');
                ...
            - pattern-inside: |
                $SESSION = require('express-session');
                ...
        - pattern: $SESSION(...)
        - pattern-not-inside: $SESSION(<... {cookie:{httpOnly:true}} ...>,...)
        - pattern-not-inside: |
            $OPTS = <... {cookie:{httpOnly:true}} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE = <... {httpOnly:true} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.cookie = <... {httpOnly:true} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE.httpOnly = true;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.cookie.httpOnly = true;
            ...
            $SESSION($OPTS,...);
      severity: WARNING
    - id: javascript.express.security.audit.express-cookie-settings.express-cookie-session-no-do
main
      languages:
        - javascript
        - typescript
      message: 'Default session middleware settings: `domain` not set. It indicates the domain o
f the cookie; use it to compare against the domain of the server in which the URL is being reque
sted. If they match, then check the path attribute next.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: LOW
        likelihood: HIGH
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        source-rule-url: https://expressjs.com/en/advanced/best-practice-security.html
        subcategory:
            - vuln
        technology:
            - express
      patterns:
        - pattern-either:
            - pattern-inside: |
                $SESSION = require('cookie-session');
                ...
            - pattern-inside: |
                $SESSION = require('express-session');
                ...
        - pattern: $SESSION(...)
        - pattern-not-inside: $SESSION(<... {cookie:{domain:...}} ...>,...)
        - pattern-not-inside: |
            $OPTS = <... {cookie:{domain:...}} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE = <... {domain:...} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.cookie = <... {domain:...} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE.domain = ...;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.cookie.domain = ...;
            ...
            $SESSION($OPTS,...);
      severity: WARNING
    - id: javascript.express.security.audit.express-cookie-settings.express-cookie-session-no-pa
th
      languages:
        - javascript
        - typescript
      message: 'Default session middleware settings: `path` not set. It indicates the path of th
e cookie; use it to compare against the request path. If this and domain match, then send the co
okie in the request.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: LOW
        likelihood: HIGH
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        source-rule-url: https://expressjs.com/en/advanced/best-practice-security.html
        subcategory:
            - vuln
        technology:
            - express
      patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-either:
            - pattern-inside: |
                $SESSION = require('cookie-session');
                ...
            - pattern-inside: |
                $SESSION = require('express-session');
                ...
        - pattern: $SESSION(...)
        - pattern-not-inside: $SESSION(<... {cookie:{path:...}} ...>,...)
        - pattern-not-inside: |
            $OPTS = <... {cookie:{path:...}} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE = <... {path:...} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.cookie = <... {path:...} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE.path = ...;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.cookie.path = ...;
            ...
            $SESSION($OPTS,...);
      severity: WARNING
    - id: javascript.express.security.audit.express-cookie-settings.express-cookie-session-no-ex
pires
      languages:
        - javascript
        - typescript
      message: 'Default session middleware settings: `expires` not set. Use it to set expiration
 date for persistent cookies.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: LOW
        likelihood: HIGH
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        source-rule-url: https://expressjs.com/en/advanced/best-practice-security.html
        subcategory:
            - vuln
        technology:
            - express
      patterns:
        - pattern-either:
            - pattern-inside: |
                $SESSION = require('cookie-session');
                ...
            - pattern-inside: |
                $SESSION = require('express-session');
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern: $SESSION(...)
        - pattern-not-inside: $SESSION(<... {cookie:{expires:...}} ...>,...)
        - pattern-not-inside: |
            $OPTS = <... {cookie:{expires:...}} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE = <... {expires:...} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $OPTS.cookie = <... {expires:...} ...>;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |
            $OPTS = ...;
            ...
            $COOKIE.expires = ...;
            ...
            $SESSION($OPTS,...);
        - pattern-not-inside: |-
            $OPTS = ...;
            ...
            $OPTS.cookie.expires = ...;
            ...
            $SESSION($OPTS,...);
      severity: WARNING
    - id: javascript.express.security.audit.express-jwt-not-revoked.express-jwt-not-revoked
      languages:
        - javascript
        - typescript
      message: No token revoking configured for `express-jwt`. A leaked token could still be use
d and unable to be revoked. Consider using function as the `isRevoked` option.
      metadata:
        asvs:
            control_id: 3.5.3 Insecue Stateless Session Tokens
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x12-V3-Session-manage
ment.md#v35-token-based-session-management
            section: 'V3: Session Management Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        source-rule-url: https://github.com/goldbergyoni/nodebestpractices/blob/master/sections/
security/expirejwt.md
        subcategory:
            - vuln
        technology:
            - express
      patterns:
        - pattern-inside: |
            $JWT = require('express-jwt');
            ...
        - pattern: $JWT(...)
        - pattern-not-inside: $JWT(<... {isRevoked:...} ...>,...)
        - pattern-not-inside: |-
            $OPTS = <... {isRevoked:...} ...>;
            ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            $JWT($OPTS,...);
      severity: WARNING
    - id: javascript.express.security.audit.express-libxml-noent.express-libxml-noent
      languages:
        - javascript
        - typescript
      message: The libxml library processes user-input with the `noent` attribute is set to `tru
e` which can lead to being vulnerable to XML External Entities (XXE) type attacks. It is recomme
nded to set `noent` to `false` when using this feature to ensure you are protected.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        interfile: true
        likelihood: HIGH
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Chea
t_Sheet.html
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern-inside: |
                        $XML = require('$IMPORT')
                        ...
                    - pattern-inside: |
                        import $XML from '$IMPORT'
                          ...
                    - pattern-inside: |
                        import * as $XML from '$IMPORT'
                        ...
                - metavariable-regex:
                    metavariable: $IMPORT
                    regex: ^(libxmljs|libxmljs2)$
                - pattern-inside: $XML.$FUNC($QUERY, {...,noent:true,...})
                - metavariable-regex:
                    metavariable: $FUNC
                    regex: ^(parseXmlString|parseXml)$
                - focus-metavariable: $QUERY
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: $REQ.headers
                - pattern: $REQ.files.$ANYTHING.data.toString('utf8')
                - pattern: $REQ.files.$ANYTHING['data'].toString('utf8')
          - patterns:
              - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
              - focus-metavariable: $REQ
              - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
                - pattern: files.$ANYTHING.data.toString('utf8')
                - pattern: files.$ANYTHING['data'].toString('utf8')
        severity: ERROR
      - id: javascript.express.security.audit.express-open-redirect.express-open-redirect
        languages:
          - javascript
          - typescript
        message: The application redirects to a URL specified by user-supplied input `$REQ` that i
s not validated. This could redirect users to malicious locations. Consider using an allow-list
approach to validate URLs, or warn users they are being redirected to a third-party website.
        metadata:
          category: security
          confidence: HIGH
          cwe:
              - 'CWE-601: URL Redirection to Untrusted Site (''Open Redirect'')'
          impact: MEDIUM
          license: Commons Clause License Condition v1.0[LGPL-2.1-only]
          likelihood: HIGH
          owasp:
              - A01:2021 - Broken Access Control
          references:
              - https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_
Cheat_Sheet.html
          subcategory:
              - vuln
          technology:
              - express
        mode: taint
        options:
          symbolic_propagation: true
          taint_unify_mvars: true
        pattern-sinks:
          - patterns:
              - pattern-either:
                - pattern: $RES.redirect("$HTTP"+$REQ. ... .$VALUE)
                - pattern: $RES.redirect("$HTTP"+$REQ. ... .$VALUE + $...A)
                - pattern: $RES.redirect(`$HTTP${$REQ. ... .$VALUE}...`)
                - pattern: $RES.redirect("$HTTP"+$REQ.$VALUE[...])
                - pattern: $RES.redirect("$HTTP"+$REQ.$VALUE[...] + $...A)
                - pattern: $RES.redirect(`$HTTP${$REQ.$VALUE[...]}...`)
              - metavariable-regex:
                  metavariable: $HTTP
                  regex: ^https?:\/\/$
              - pattern-either:
                - pattern: $REQ. ... .$VALUE
          - patterns:
              - pattern-either:
                - pattern: $RES.redirect($REQ. ... .$VALUE)
                - pattern: $RES.redirect($REQ. ... .$VALUE + $...A)
                - pattern: $RES.redirect(`${$REQ. ... .$VALUE}...`)
              - pattern: $REQ. ... .$VALUE
          - patterns:
              - pattern-either:
                - pattern: $RES.redirect($REQ.$VALUE['...'])
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: $RES.redirect($REQ.$VALUE['...'] + $...A)
                - pattern: $RES.redirect(`${$REQ.$VALUE['...']}...`)
            - pattern: $REQ.$VALUE
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $ASSIGN = $REQ. ... .$VALUE
                    ...
                - pattern-inside: |
                    $ASSIGN = $REQ.$VALUE['...']
                    ...
                - pattern-inside: |
                    $ASSIGN = $REQ. ... .$VALUE + $...A
                    ...
                - pattern-inside: "$ASSIGN = $REQ.$VALUE['...'] + $...A\n...     \n"
                - pattern-inside: |
                    $ASSIGN = `${$REQ. ... .$VALUE}...`
                    ...
                - pattern-inside: "$ASSIGN = `${$REQ.$VALUE['...']}...`\n...
\n"
            - pattern-either:
                - pattern: $RES.redirect($ASSIGN)
                - pattern: $RES.redirect($ASSIGN + $...FOO)
                - pattern: $RES.redirect(`${$ASSIGN}...`)
            - focus-metavariable: $ASSIGN
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: WARNING
    - id: javascript.express.security.audit.express-path-join-resolve-traversal.express-path-joi
n-resolve-traversal
      languages:
        - javascript
        - typescript
      message: Possible writing outside of the destination, make sure that the target path is ne
sted in the intended destination
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/www-community/attacks/Path_Traversal
        subcategory:
            - vuln
        technology:
            - express
            - node.js
      mode: taint
      pattern-sanitizers:
        - pattern: $Y.replace(...)
        - pattern: $Y.indexOf(...)
        - pattern: |
            function ... (...) {
                ...
                <... $Y.indexOf(...) ...>
                ...
            }
        - patterns:
            - pattern: $FUNC(...)
            - metavariable-regex:
                metavariable: $FUNC
                regex: sanitize
      pattern-sinks:
        - patterns:
            - focus-metavariable: $SINK
            - pattern-either:
                - pattern-inside: |
                    $PATH = require('path');
                    ...
                - pattern-inside: |
                    import $PATH from 'path';
                    ...
            - pattern-either:
                - pattern: $PATH.join(...,$SINK,...)
                - pattern: $PATH.resolve(...,$SINK,...)
        - patterns:
            - focus-metavariable: $SINK
            - pattern-inside: |
                import 'path';
                ...
            - pattern-either:
                - pattern: path.join(...,$SINK,...)
                - pattern: path.resolve(...,$SINK,...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: WARNING
    - id: javascript.express.security.audit.express-res-sendfile.express-res-sendfile
      languages:
        - javascript
        - typescript
      message: The application processes user-input, this is passed to res.sendFile which can al
low an attacker to arbitrarily read files on the system through path traversal. It is recommende
d to perform input validation in addition to canonicalizing the path. This allows you to validat
e the path against the intended directory it should be accessing.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-73: External Control of File Name or Path'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A04:2021 - Insecure Design
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: $RES.$METH($QUERY,...)
            - pattern-not-inside: $RES.$METH($QUERY,$OPTIONS)
            - metavariable-regex:
                metavariable: $METH
                regex: ^(sendfile|sendFile)$
            - focus-metavariable: $QUERY
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            function ... (...,$REQ: $TYPE, ...) {...}
                    - metavariable-regex:
                        metavariable: $TYPE
                        regex: ^(string|String)
      severity: WARNING
    - id: javascript.express.security.audit.express-session-hardcoded-secret.express-session-har
dcoded-secret
      languages:
        - javascript
        - typescript
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        interfile: true
        likelihood: HIGH
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - express
            - secrets
      options:
        interfile: true
      patterns:
        - pattern-either:
            - pattern-inside: |
                $SESSION = require('express-session');
                ...
            - pattern-inside: |
                import $SESSION from 'express-session'
                ...
            - pattern-inside: |
                import {..., $SESSION, ...} from 'express-session'
                ...
            - pattern-inside: |
                import * as $SESSION from 'express-session'
                ...
        - patterns:
            - pattern-either:
                - pattern-inside: $APP.use($SESSION({...}))
                - pattern: |
                    $SECRET = $VALUE
                    ...
                    $APP.use($SESSION($SECRET))
            - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              secret: '$Y'
      severity: WARNING
    - id: javascript.express.security.audit.express-ssrf.express-ssrf
      languages:
        - javascript
        - typescript
      message: 'The following request $REQUEST.$METHOD() was found to be crafted from user-input
 `$REQ` which can lead to Server-Side Request Forgery (SSRF) vulnerabilities. It is recommended
where possible to not allow user-input to craft the base request, but to be treated as part of t
he path or query parameter. When user-input is necessary to craft the request, it is recommenede
d to follow OWASP best practices to prevent abuse. '
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevent
ion_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      options:
        taint_unify_mvars: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $REQUEST = require('request')
                    ...
                - pattern-inside: |
                    import * as $REQUEST from 'request'
                    ...
                - pattern-inside: |
                    import $REQUEST from 'request'
                    ...
            - pattern-either:
                - pattern: $REQUEST.$METHOD("$HTTP"+$REQ. ... .$VALUE)
                - pattern: $REQUEST.$METHOD("$HTTP"+$REQ. ... .$VALUE + $...A)
                - pattern: $REQUEST.$METHOD(`$HTTP${$REQ. ... .$VALUE}...`)
                - pattern: $REQUEST.$METHOD("$HTTP"+$REQ.$VALUE[...])
                - pattern: $REQUEST.$METHOD("$HTTP"+$REQ.$VALUE[...] + $...A)
                - pattern: $REQUEST.$METHOD(`$HTTP${$REQ.$VALUE[...]}...`)
            - metavariable-regex:
                metavariable: $METHOD
                regex: ^(get|post|put|patch|del|head|delete)$
            - metavariable-regex:
                metavariable: $HTTP
                regex: ^(https?:\/\/|//)$
            - pattern-either:
                - pattern: $REQ. ... .$VALUE
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $REQUEST = require('request')
                    ...
                - pattern-inside: |
                    import * as $REQUEST from 'request'
                    ...
                - pattern-inside: |
                    import $REQUEST from 'request'
                    ...
            - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: $REQUEST.$METHOD($REQ. ... .$VALUE,...)
            - pattern: $REQUEST.$METHOD($REQ. ... .$VALUE + $...A,...)
            - pattern: $REQUEST.$METHOD(`${$REQ. ... .$VALUE}...`,...)
        - pattern: $REQ. ... .$VALUE
        - metavariable-regex:
            metavariable: $METHOD
            regex: ^(get|post|put|patch|del|head|delete)$
    - patterns:
        - pattern-either:
            - pattern-inside: |
                $REQUEST = require('request')
                ...
            - pattern-inside: |
                import * as $REQUEST from 'request'
                ...
            - pattern-inside: |
                import $REQUEST from 'request'
                ...
        - pattern-either:
            - pattern: $REQUEST.$METHOD($REQ.$VALUE['...'],...)
            - pattern: $REQUEST.$METHOD($REQ.$VALUE['...'] + $...A,...)
            - pattern: $REQUEST.$METHOD(`${$REQ.$VALUE['...']}...`,...)
        - pattern: $REQ.$VALUE
        - metavariable-regex:
            metavariable: $METHOD
            regex: ^(get|post|put|patch|del|head|delete)$
    - patterns:
        - pattern-either:
            - pattern-inside: |
                $REQUEST = require('request')
                ...
            - pattern-inside: |
                import * as $REQUEST from 'request'
                ...
            - pattern-inside: |
                import $REQUEST from 'request'
                ...
        - pattern-either:
            - pattern-inside: |
                $ASSIGN = $REQ. ... .$VALUE
                ...
            - pattern-inside: |
                $ASSIGN = $REQ. ... .$VALUE['...']
                ...
            - pattern-inside: |
                $ASSIGN = $REQ. ... .$VALUE + $...A
                ...
            - pattern-inside: "$ASSIGN = $REQ. ... .$VALUE['...'] + $...A\n...     \n"
            - pattern-inside: |
                $ASSIGN = `${$REQ. ... .$VALUE}...`
                ...
            - pattern-inside: "$ASSIGN = `${$REQ. ... .$VALUE['...']}...`\n... \n"
            - patterns:
                - pattern-either:
                    - pattern-inside: |
                        $ASSIGN = "$HTTP"+ $REQ. ... .$VALUE
                        ...
                    - pattern-inside: |
                        $ASSIGN = "$HTTP"+$REQ. ... .$VALUE + $...A
                        ...
                    - pattern-inside: |
                        $ASSIGN = "$HTTP"+$REQ.$VALUE[...]
                        ...
                    - pattern-inside: |
                        $ASSIGN = "$HTTP"+$REQ.$VALUE[...] + $...A
                        ...
                    - pattern-inside: |
                        $ASSIGN = `$HTTP${$REQ.$VALUE[...]}...`
                        ...
                - metavariable-regex:
                    metavariable: $HTTP
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    regex: ^(https?:\/\/|//)$
            - pattern-either:
                - pattern: $REQUEST.$METHOD($ASSIGN,...)
                - pattern: $REQUEST.$METHOD($ASSIGN + $...FOO,...)
                - pattern: $REQUEST.$METHOD(`${$ASSIGN}...`,...)
                - patterns:
                    - pattern-either:
                        - pattern: $REQUEST.$METHOD("$HTTP"+$ASSIGN,...)
                        - pattern: $REQUEST.$METHOD("$HTTP"+$ASSIGN + $...A,...)
                        - pattern: $REQUEST.$METHOD(`$HTTP${$ASSIGN}...`,...)
                    - metavariable-regex:
                        metavariable: $HTTP
                        regex: ^(https?:\/\/|//)$
            - pattern: $ASSIGN
            - metavariable-regex:
                metavariable: $METHOD
                regex: ^(get|post|put|patch|del|head|delete)$
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, ...) {...}
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,...) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: $EXPRESS.Request,...) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: WARNING
    - id: javascript.express.security.audit.express-third-party-object-deserialization.express-t
hird-party-object-deserialization
      languages:
        - javascript
        - typescript
      message: The following function call $SER.$FUNC accepts user controlled data which can res
ult in Remote Code Execution (RCE) through Object Deserialization. It is recommended to use secu
re data processing alternatives such as JSON.parse() and Buffer.from().
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        interfile: true
        likelihood: HIGH
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html
        source_rule_url:
            - https://github.com/ajinabraham/njsscan/blob/75bfbeb9c8d72999e4d527dfa2548f7f0f3cc4
8a/njsscan/rules/semantic_grep/eval/eval_deserialize.yaml
        subcategory:
            - vuln
        technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - express
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern-inside: |
                        $SER = require('$IMPORT')
                        ...
                    - pattern-inside: |
                        import $SER from '$IMPORT'
                         ...
                    - pattern-inside: |
                        import * as $SER from '$IMPORT'
                        ...
                - metavariable-regex:
                    metavariable: $IMPORT
                    regex: ^(node-serialize|serialize-to-js)$
                - pattern: $SER.$FUNC(...)
                - metavariable-regex:
                    metavariable: $FUNC
                    regex: ^(unserialize|deserialize)$
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
                - pattern: $REQ.files.$ANYTHING.data.toString('utf8')
                - pattern: $REQ.files.$ANYTHING['data'].toString('utf8')
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
                - pattern: files.$ANYTHING.data.toString('utf8')
                - pattern: files.$ANYTHING['data'].toString('utf8')
      severity: WARNING
    - id: javascript.express.security.audit.express-xml2json-xxe-event.express-xml2json-xxe-even
t
      languages:
        - javascript
        - typescript
      message: Xml Parser is used inside Request Event. Make sure that unverified user data can
not reach the XML Parser, as it can result in XML External or Internal Entity (XXE) Processing v
ulnerabilities
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://www.npmjs.com/package/xml2json
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    require('xml2json');
                    ...
                - pattern-inside: |
                    import 'xml2json';
                    ...
            - pattern: $REQ.on('...', function(...) { ... $EXPAT.toJson($INPUT,...); ... })
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) => {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: WARNING
    - id: javascript.express.security.audit.res-render-injection.res-render-injection
      languages:
        - javascript
        - typescript
      message: User controllable data `$REQ` enters `$RES.render(...)` this can lead to the load
ing of other HTML/templating pages that they may not be authorized to render. An attacker may at
tempt to use directory traversal techniques e.g. `../folder/index` to access other HTML pages on
 the file system. Where possible, do not allow users to define what should be  loaded in $RES.re
nder or use an allow list for the existing application.
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-706: Use of Incorrectly-Resolved Name or Reference'
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - http://expressjs.com/en/4x/api.html#res.render
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: $RES.render($SINK, ...)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: WARNING
    - id: javascript.express.security.audit.xss.direct-response-write.direct-response-write
      languages:
        - javascript
        - typescript
      message: Detected directly writing to a Response object from user-defined input. This bypa
sses any HTML escaping and may expose your application to a Cross-Site-scripting (XSS) vulnerabi
lity. Instead, use 'resp.render()' to render safely escaped HTML.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Che
at_Sheet.html
        subcategory:
            - vuln
        technology:
            - express
        vulnerability_class:
            - Cross-Site-Scripting (XSS)
      mode: taint
      options:
        interfile: true
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import $S from "underscore.string"
                    ...
                - pattern-inside: |
                    import * as $S from "underscore.string"
                    ...
                - pattern-inside: |
                    import $S from "underscore.string"
                    ...
                - pattern-inside: |
                    $S = require("underscore.string")
                    ...
            - pattern-either:
                - pattern: $S.escapeHTML(...)
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import $S from "dompurify"
                    ...
                - pattern-inside: |
                    import { ..., $S,... } from "dompurify"
                    ...
                - pattern-inside: |
                    import * as $S from "dompurify"
                    ...
                - pattern-inside: |
                    $S = require("dompurify")
                    ...
                - pattern-inside: |
                    import $S from "isomorphic-dompurify"
                    ...
                - pattern-inside: |
                    import * as $S from "isomorphic-dompurify"
                    ...
                - pattern-inside: |
                    $S = require("isomorphic-dompurify")
                    ...
            - pattern-either:
                - patterns:
                    - pattern-inside: |
                        $VALUE = $S(...)
                        ...
                    - pattern: $VALUE.sanitize(...)
                - patterns:
                    - pattern-inside: |
                        $VALUE = $S.sanitize
                        ...
                    - pattern: $S(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: $S.sanitize(...)
            - pattern: $S(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                import $S from 'xss';
                ...
            - pattern-inside: |
                import * as $S from 'xss';
                ...
            - pattern-inside: |
                $S = require("xss")
                ...
        - pattern: $S(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                import $S from 'sanitize-html';
                ...
            - pattern-inside: |
                import * as $S from "sanitize-html";
                ...
            - pattern-inside: |
                $S = require("sanitize-html")
                ...
        - pattern: $S(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                $S = new Remarkable()
                ...
        - pattern: $S.render(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                import $S from 'express-xss-sanitizer';
                ...
            - pattern-inside: |
                import * as $S from "express-xss-sanitizer";
                ...
            - pattern-inside: |
                const { ..., $S, ... } = require('express-xss-sanitizer');
                ...
            - pattern-inside: |
                var { ..., $S, ... } = require('express-xss-sanitizer');
                ...
            - pattern-inside: |
                let { ...,$S,... } = require('express-xss-sanitizer');
                ...
            - pattern-inside: |
                $S = require("express-xss-sanitizer")
                ...
        - pattern: $S(...)
    - patterns:
        - pattern: $RES. ... .type('$F'). ... .send(...)
        - metavariable-regex:
            metavariable: $F
            regex: (?!.*text/html)
    - patterns:
        - pattern-inside: |
            $X = [...];
            ...
        - pattern: |
            if(<... !$X.includes($SOURCE)...>) {
                ...
                return ...
            }
            ...
        - pattern: $SOURCE
pattern-sinks:
  - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      - pattern-inside: function ... (..., $RES,...) {...}
      - pattern-either:
          - pattern: $RES.write($ARG)
          - pattern: $RES.send($ARG)
      - pattern-not: $RES. ... .set('...'). ... .send($ARG)
      - pattern-not: $RES. ... .type('...'). ... .send($ARG)
      - pattern-not-inside: $RES.$METHOD({ ... })
      - focus-metavariable: $ARG
  pattern-sources:
    - patterns:
      - pattern-either:
          - pattern-inside: function ... ($REQ, $RES) {...}
          - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
          - patterns:
              - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
              - metavariable-regex:
                  metavariable: $METHOD
                  regex: ^(get|post|put|head|delete|options)
      - pattern-not-inside: |
          function ... ($REQ, $RES) {
              ...
              $RES.$SET('Content-Type', '$TYPE')
          }
      - pattern-not-inside: |
          $APP.$METHOD(..., function $FUNC($REQ, $RES) {
              ...
              $RES.$SET('Content-Type', '$TYPE')
          })
      - pattern-not-inside: |
          function ... ($REQ, $RES, $NEXT) {
              ...
              $RES.$SET('Content-Type', '$TYPE')
          }
      - pattern-not-inside: |
          function ... ($REQ, $RES) {
              ...
              $RES.set('$TYPE')
          }
      - pattern-not-inside: |
          $APP.$METHOD(..., function $FUNC($REQ, $RES) {
              ...
              $RES.set('$TYPE')
          })
      - pattern-not-inside: |
          function ... ($REQ, $RES, $NEXT) {
              ...
              $RES.set('$TYPE')
          }
      - pattern-either:
          - pattern: $REQ.query
          - pattern: $REQ.body
          - pattern: $REQ.params
    - patterns:
      - pattern-either:
          - pattern-inside: |
              ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
              {...}
          - pattern-inside: |
              ({ $REQ }: Request,$RES: Response) => {...}
      - pattern-not-inside: |
          ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
          {
              ...
              $RES.$SET('Content-Type', '$TYPE')
          }
      - pattern-not-inside: |
          ({ $REQ }: Request,$RES: Response) => {
              ...
              $RES.$SET('Content-Type', '$TYPE')
          }
      - pattern-not-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                {
                    ...
                    $RES.set('$TYPE')
                }
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: body
      severity: WARNING
    - id: javascript.express.security.cors-misconfiguration.cors-misconfiguration
      languages:
        - javascript
        - typescript
      message: By letting user input control CORS parameters, there is a risk that software does
 not properly verify that the source of data or communication is valid. Use literal values for C
ORS settings.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-346: Origin Validation Error'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: $RES.set($HEADER, $X)
                - pattern: $RES.header($HEADER, $X)
                - pattern: $RES.setHeader($HEADER, $X)
                - pattern: |
                    $RES.set({$HEADER: $X}, ...)
                - pattern: |
                    $RES.writeHead($STATUS, {$HEADER: $X}, ...)
            - focus-metavariable: $X
            - metavariable-regex:
                metavariable: $HEADER
                regex: .*(Access-Control-Allow-Origin|access-control-allow-origin).*
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: WARNING
    - id: javascript.express.security.express-expat-xxe.express-expat-xxe
      languages:
        - javascript
        - typescript
      message: Make sure that unverified user data can not reach the XML Parser, as it can resul
t in XML External or Internal Entity (XXE) Processing vulnerabilities.
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://github.com/astro/node-expat
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $XML = require('node-expat')
                    ...
                - pattern-inside: |
                    import $XML from 'node-expat'
                    ...
                - pattern-inside: |
                    import * as $XML from 'node-expat'
                    ...
            - pattern-either:
                - pattern-inside: |
                    $PARSER = new $XML.Parser(...);
                    ...
            - pattern-either:
                - pattern: $PARSER.parse($QUERY)
                - pattern: $PARSER.write($QUERY)
            - focus-metavariable: $QUERY
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: ERROR
    - id: javascript.express.security.express-insecure-template-usage.express-insecure-template-
usage
      languages:
        - javascript
        - typescript
      message: User data from `$REQ` is being compiled into the template, which can lead to a Se
rver Side Template Injection (SSTI) vulnerability.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1336: Improper Neutralization of Special Elements Used in a Template Engine'
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
            - A01:2017 - Injection
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.ht
ml
        source_rule_url:
            - https://github.com/github/codeql/blob/2ba2642c7ab29b9eedef33bcc2b8cd1d203d0c10/jav
ascript/ql/test/query-tests/Security/CWE-094/CodeInjection/template-sinks.js
        subcategory:
            - vuln
        technology:
            - javascript
            - typescript
            - express
            - pug
            - jade
            - dot
            - ejs
            - nunjucks
            - lodash
            - handlbars
            - mustache
            - hogan.js
            - eta
            - squirrelly
      mode: taint
      options:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
  interfile: true
pattern-propagators:
  - from: $E
    pattern: $MODEL.$FIND($E).then((...,$S,...)=>{...})
    to: $S
pattern-sinks:
  - pattern-either:
      - patterns:
          - pattern-either:
              - pattern-inside: |
                  $PUG = require('pug')
                  ...
              - pattern-inside: |
                  import * as $PUG from 'pug'
                  ...
              - pattern-inside: |
                  $PUG = require('jade')
                  ...
              - pattern-inside: |
                  import * as $PUG from 'jade'
                  ...
          - pattern-either:
              - pattern: $PUG.compile(...)
              - pattern: $PUG.compileClient(...)
              - pattern: $PUG.compileClientWithDependenciesTracked(...)
              - pattern: $PUG.render(...)
      - patterns:
          - pattern-either:
              - pattern-inside: |
                  $PUG = require('dot')
                  ...
              - pattern-inside: |
                  import * as $PUG from 'dot'
                  ...
          - pattern-either:
              - pattern: $PUG.template(...)
              - pattern: $PUG.compile(...)
      - patterns:
          - pattern-either:
              - pattern-inside: |
                  $PUG = require('ejs')
                  ...
              - pattern-inside: |
                  import * as $PUG from 'ejs'
                  ...
          - pattern-either:
              - pattern: $PUG.render(...)
      - patterns:
          - pattern-either:
              - pattern-inside: |
                  $PUG = require('nunjucks')
                  ...
              - pattern-inside: |
                  import * as $PUG from 'nunjucks'
                  ...
          - pattern-either:
              - pattern: $PUG.renderString(...)
      - patterns:
          - pattern-either:
              - pattern-inside: |
                  $PUG = require('lodash')
                  ...
              - pattern-inside: |
                  import * as $PUG from 'lodash'
                  ...
          - pattern-either:
              - pattern: $PUG.template(...)
      - patterns:
          - pattern-either:
              - pattern-inside: |
                  $PUG = require('mustache')
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    ...
                - pattern-inside: |
                    import * as $PUG from 'mustache'
                    ...
                - pattern-inside: |
                    $PUG = require('eta')
                    ...
                - pattern-inside: |
                    import * as $PUG from 'eta'
                    ...
                - pattern-inside: |
                    $PUG = require('squirrelly')
                    ...
                - pattern-inside: |
                    import * as $PUG from 'squirrelly'
                    ...
            - pattern-either:
                - pattern: $PUG.render(...)
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $PUG = require('hogan.js')
                    ...
                - pattern-inside: |
                    import * as $PUG from 'hogan.js'
                    ...
                - pattern-inside: |
                    $PUG = require('handlebars')
                    ...
                - pattern-inside: |
                    import * as $PUG from 'handlebars'
                    ...
            - pattern-either:
                - pattern: $PUG.compile(...)
    pattern-sources:
      - patterns:
          - pattern-either:
              - pattern-inside: function ... ($REQ, $RES) {...}
              - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
              - patterns:
                  - pattern-either:
                      - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                      - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                  - metavariable-regex:
                      metavariable: $METHOD
                      regex: ^(get|post|put|head|delete|options)$
          - pattern-either:
              - pattern: $REQ.query
              - pattern: $REQ.body
              - pattern: $REQ.params
              - pattern: $REQ.cookies
              - pattern: $REQ.headers
      - patterns:
          - pattern-either:
              - pattern-inside: |
                  ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                  {...}
              - pattern-inside: |
                  ({ $REQ }: Request,$RES: Response) => {...}
          - focus-metavariable: $REQ
          - pattern-either:
              - pattern: params
              - pattern: query
              - pattern: cookies
              - pattern: headers
              - pattern: body
    severity: WARNING
  - id: javascript.express.security.express-jwt-hardcoded-secret.express-jwt-hardcoded-secret
    languages:
      - javascript
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - typescript
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        likelihood: HIGH
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
        subcategory:
            - audit
        technology:
            - express
            - secrets
      options:
        interfile: true
      patterns:
        - pattern-either:
            - pattern-inside: |
                $JWT = require('express-jwt');
                ...
            - pattern-inside: |
                import $JWT from 'express-jwt';
                ...
            - pattern-inside: |
                import * as $JWT from 'express-jwt';
                ...
            - pattern-inside: |
                import { ..., $JWT, ... } from 'express-jwt';
                ...
        - pattern-either:
            - pattern: |
                $JWT({...,secret: "$Y",...},...)
            - pattern: |
                $OPTS = "$Y";
                ...
                $JWT({...,secret: $OPTS},...);
        - focus-metavariable: $Y
      severity: WARNING
    - id: javascript.express.security.express-phantom-injection.express-phantom-injection
      languages:
        - javascript
        - typescript
      message: If unverified user data can reach the `phantom` methods it can result in Server-S
ide Request Forgery vulnerabilities
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://phantomjs.org/page-automation.html
        subcategory:
            - vuln
        technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    require('phantom');
                    ...
                - pattern-inside: |
                    import 'phantom';
                    ...
            - pattern-either:
                - pattern: $PAGE.open($SINK,...)
                - pattern: $PAGE.setContent($SINK,...)
                - pattern: $PAGE.openUrl($SINK,...)
                - pattern: $PAGE.evaluateJavaScript($SINK,...)
                - pattern: $PAGE.property("content",$SINK,...)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: ERROR
    - id: javascript.express.security.express-puppeteer-injection.express-puppeteer-injection
      languages:
        - javascript
        - typescript
      message: If unverified user data can reach the `puppeteer` methods it can result in Server
-Side Request Forgery vulnerabilities
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://pptr.dev/api/puppeteer.page
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    require('puppeteer');
                    ...
                - pattern-inside: |
                    import 'puppeteer';
                    ...
            - pattern-either:
                - pattern: $PAGE.goto($SINK,...)
                - pattern: $PAGE.setContent($SINK,...)
                - pattern: $PAGE.evaluate($SINK,...)
                - pattern: $PAGE.evaluate($CODE,$SINK,...)
                - pattern: $PAGE.evaluateHandle($SINK,...)
                - pattern: $PAGE.evaluateHandle($CODE,$SINK,...)
                - pattern: $PAGE.evaluateOnNewDocument($SINK,...)
                - pattern: $PAGE.evaluateOnNewDocument($CODE,$SINK,...)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: ERROR
    - id: javascript.express.security.express-sandbox-injection.express-sandbox-code-injection
      languages:
        - javascript
        - typescript
      message: Make sure that unverified user data can not reach `sandbox`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        owasp:
            - A03:2021 - Injection
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.ht
ml
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-inside: |
                $SANDBOX = require('sandbox');
                ...
            - pattern-either:
                - patterns:
                    - pattern-inside: |
                        $S = new $SANDBOX(...);
                        ...
                    - pattern: |
                        $S.run(...)
                - pattern: |
                    new $SANDBOX($OPTS).run(...)
                - pattern: new $SANDBOX().run(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: ERROR
    - id: javascript.express.security.express-vm-injection.express-vm-injection
      languages:
        - javascript
        - typescript
      message: Make sure that unverified user data can not reach `$VM`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
        impact: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.ht
ml
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-inside: |
                $VM = require('vm');
                ...
            - pattern-either:
                - pattern: |
                    $VM.runInContext(...)
                - pattern: |
                    $VM.runInNewContext(...)
                - pattern: |
                    $VM.compileFunction(...)
                - pattern: |
                    $VM.runInThisContext(...)
                - pattern: new $VM.Script(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: ERROR
    - id: javascript.express.security.express-vm2-injection.express-vm2-injection
      languages:
        - javascript
        - typescript
      message: Make sure that unverified user data can not reach `vm2`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.ht
ml
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-inside: |
                require('vm2')
                ...
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            $VM = new VM(...)
                            ...
                        - pattern-inside: |
                            $VM = new NodeVM(...)
                            ...
                    - pattern: |
                        $VM.run(...)
                - pattern: |
                    new VM(...).run(...)
                - pattern: |
                    new NodeVM(...).run(...)
                - pattern: |
                    new VMScript(...)
                - pattern: |
                    new VM(...)
                - pattern: new NodeVM(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: WARNING
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: javascript.express.security.express-xml2json-xxe.express-xml2json-xxe
      languages:
        - javascript
        - typescript
      message: Make sure that unverified user data can not reach the XML Parser, as it can resul
t in XML External or Internal Entity (XXE) Processing vulnerabilities
      metadata:
        asvs:
            control_id: 5.5.2 Insecue XML Deserialization
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v55-deserialization-prevention
            section: V5 Validation, Sanitization and Encoding
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://www.npmjs.com/package/xml2json
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    require('xml2json');
                    ...
                - pattern-inside: |
                    import 'xml2json';
                    ...
            - pattern: $EXPAT.toJson($SINK,...)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
                - pattern: $REQ.files.$ANYTHING.data.toString('utf8')
                - pattern: $REQ.files.$ANYTHING['data'].toString('utf8')
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
                - pattern: files.$ANYTHING.data.toString('utf8')
                - pattern: files.$ANYTHING['data'].toString('utf8')
      severity: ERROR
    - id: javascript.express.security.injection.raw-html-format.raw-html-format
      languages:
        - javascript
        - typescript
      message: User data flows into the host portion of this manually-constructed HTML. This can
 introduce a Cross-Site-Scripting (XSS) vulnerability if this comes from user-provided input. Co
nsider using a sanitization library such as DOMPurify to sanitize the HTML within.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Che
at_Sheet.html
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: '"$HTMLSTR" + $EXPR'
                        - pattern: '"$HTMLSTR".concat(...)'
                        - pattern: util.format($HTMLSTR, ...)
                    - metavariable-pattern:
                        language: generic
                        metavariable: $HTMLSTR
                        pattern: <$TAG ...
                - patterns:
                    - pattern: |
                        `...`
                    - pattern-regex: |
                        .*<\w+.*
          requires: (EXPRESS and not CLEAN) or (EXPRESSTS and not CLEAN)
      pattern-sources:
        - label: EXPRESS
          patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - label: EXPRESSTS
          patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
        - by-side-effect: true
          label: CLEAN
          patterns:
            - pattern-either:
                - pattern: $A($SOURCE)
                - pattern: $SANITIZE. ... .$A($SOURCE)
                - pattern: $A. ... .$SANITIZE($SOURCE)
            - focus-metavariable: $SOURCE
            - metavariable-regex:
                metavariable: $A
                regex: (?i)(.*valid|.*sanitiz)
      severity: WARNING
    - id: javascript.express.security.injection.tainted-sql-string.tainted-sql-string
      languages:
        - javascript
        - typescript
      message: Detected user input used to manually construct a SQL string. This is usually bad
practice because manual construction could accidentally result in a SQL injection. An attacker c
ould use a SQL injection to steal or modify contents of the database. Instead, use a parameteriz
ed query which is available by default in most database engines. Alternatively, consider using a
n object-relational mapper (ORM) such as Sequelize which will protect your queries.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/SQL_Injection
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            "$SQLSTR" + $EXPR
                        - pattern-inside: |
                            "$SQLSTR".concat($EXPR)
                        - pattern: util.format($SQLSTR, $EXPR)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern: |
                            `$SQLSTR${$EXPR}...`
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: .*\b(?i)(select|delete|insert|create|update\s+.+\sset|alter|drop)
\b.*
            - focus-metavariable: $EXPR
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... (...,$REQ, ...) {...}
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    (...,{ $REQ }: Request,...) => {...}
                - pattern-inside: |
                    (...,{ $REQ }: $EXPRESS.Request,...) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: ERROR
    - id: javascript.express.security.require-request.require-request
      languages:
        - javascript
        - typescript
      message: If an attacker controls the x in require(x) then they can cause code to load that
 was not intended to run on the server.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-706: Use of Incorrectly-Resolved Name or Reference'
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://github.com/google/node-sec-roadmap/blob/master/chapter-2/dynamism.md#dynam
ism-when-you-need-it
        source-rule-url: https://nodesecroadmap.fyi/chapter-1/threat-UIR.html
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern: require($SINK)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: ERROR
    - id: javascript.express.security.x-frame-options-misconfiguration.x-frame-options-misconfig
uration
      languages:
        - javascript
        - typescript
      message: By letting user input control `X-Frame-Options` header, there is a risk that soft
ware does not properly verify whether or not a browser should be allowed to render a page in an
`iframe`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-451: User Interface (UI) Misrepresentation of Critical Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A04:2021 - Insecure Design
        references:
            - https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: $RES.set($HEADER, ...)
                - pattern: $RES.header($HEADER, ...)
                - pattern: $RES.setHeader($HEADER, ...)
                - pattern: |
                    $RES.set({$HEADER: ...}, ...)
                - pattern: |
                    $RES.writeHead($STATUS, {$HEADER: ...}, ...)
            - metavariable-regex:
                metavariable: $HEADER
                regex: .*(X-Frame-Options|x-frame-options).*
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
      severity: WARNING
    - id: javascript.intercom.security.audit.intercom-settings-user-identifier-without-user-hash
.intercom-settings-user-identifier-without-user-hash
      languages:
        - js
      message: Found an initialization of the Intercom Messenger that identifies a User, but doe
s not specify a `user_hash`.This configuration allows users to impersonate one another. See the
Intercom Identity Verification docs for more context https://www.intercom.com/help/en/articles/1
83-set-up-identity-verification-for-web-and-mobile
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-287: Improper Authentication'
        impact: HIGH
        likelihood: MEDIUM
        references:
            - https://www.intercom.com/help/en/articles/183-set-up-identity-verification-for-web
-and-mobile
        subcategory:
            - guardrail
        technology:
            - intercom
      patterns:
        - pattern-either:
            - pattern: |
                window.intercomSettings = {..., email: $EMAIL, ...};
            - pattern: |
                window.intercomSettings = {..., user_id: $USER_ID, ...};
            - pattern: |
                Intercom('boot', {..., email: $EMAIL, ...});
            - pattern: |
                Intercom('boot', {..., user_id: $USER_ID, ...});
            - pattern: |
                $VAR = {..., email: $EMAIL, ...};
                ...
                Intercom('boot', $VAR);
            - pattern: |
                $VAR = {..., user_id: $EMAIL, ...};
                ...
                Intercom('boot', $VAR);
        - pattern-not: |
            window.intercomSettings = {..., user_hash: $USER_HASH, ...};
        - pattern-not: |
            Intercom('boot', {..., user_hash: $USER_HASH, ...});
        - pattern-not: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            $VAR = {..., user_hash: $USER_HASH, ...};
            ...
            Intercom('boot', $VAR);
      severity: WARNING
    - id: javascript.jose.security.jwt-hardcode.hardcoded-jwt-secret
      languages:
        - javascript
        - typescript
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
      metadata:
        asvs:
            control_id: 3.5.2 Static API keys or secret
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x12-V3-Session-manage
ment.md#v35-token-based-session-management
            section: 'V3: Session Management Verification Requirements'
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        likelihood: HIGH
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - jose
            - jwt
            - secrets
      options:
        interfile: true
        symbolic_propagation: true
      patterns:
        - pattern-inside: |
            $JOSE = require("jose");
            ...
        - pattern-either:
            - pattern-inside: |
                var {JWT} = $JOSE;
                ...
            - pattern-inside: |
                var {JWK, JWT} = $JOSE;
                ...
            - pattern-inside: |
                const {JWT} = $JOSE;
                ...
            - pattern-inside: |
                const {JWK, JWT} = $JOSE;
                ...
            - pattern-inside: |
                let {JWT} = $JOSE;
                ...
            - pattern-inside: |
                let {JWK, JWT} = $JOSE;
                ...
        - pattern-either:
            - pattern: |
                JWT.verify($P, "...", ...);
            - pattern: |
                JWT.sign($P, "...", ...);
            - pattern: "JWT.verify($P, JWK.asKey(\"...\"), ...);     \n"
            - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              $JWT.sign($P, JWK.asKey("..."), ...);
      severity: WARNING
    - id: javascript.jose.security.jwt-none-alg.jwt-none-alg
      languages:
        - javascript
        - typescript
      message: Detected use of the 'none' algorithm in a JWT token. The 'none' algorithm assumes
 the integrity of the token has already been verified. This would allow a malicious actor to for
ge a JWT token that will automatically be verified. Do not explicitly use the 'none' algorithm.
Instead, use an algorithm such as 'HS256'.
      metadata:
        asvs:
            control_id: 3.5.3 Insecue Stateless Session Tokens
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x12-V3-Session-manage
ment.md#v35-token-based-session-management
            section: 'V3: Session Management Verification Requirements'
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory:
            - vuln
        technology:
            - jose
            - jwt
      pattern-either:
        - pattern: |
            var $JOSE = require("jose");
            ...
            var { JWK, JWT } = $JOSE;
            ...
            var $T = JWT.verify($P, JWK.None,...);
        - pattern: |
            var $JOSE = require("jose");
            ...
            var { JWK, JWT } = $JOSE;
            ...
            $T = JWT.verify($P, JWK.None,...);
        - pattern: |
            var $JOSE = require("jose");
            ...
            var { JWK, JWT } = $JOSE;
            ...
            JWT.verify($P, JWK.None,...);
      severity: ERROR
    - id: javascript.jsonwebtoken.security.jwt-hardcode.hardcoded-jwt-secret
      languages:
        - javascript
        - typescript
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
      metadata:
        asvs:
            control_id: 3.5.2 Static API keys or secret
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x12-V3-Session-manage
ment.md#v35-token-based-session-management
            section: 'V3: Session Management Verification Requirements'
            version: "4"
        category: security
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        confidence: HIGH
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - jwt
            - javascript
            - secrets
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $JWT = require("jsonwebtoken")
                    ...
                - pattern-inside: |
                    import $JWT from "jsonwebtoken"
                    ...
                - pattern-inside: |
                    import * as $JWT from "jsonwebtoken"
                    ...
                - pattern-inside: |
                    import {...,$JWT,...} from "jsonwebtoken"
                    ...
            - pattern-either:
                - pattern-inside: |
                    $JWT.sign($DATA,$VALUE,...);
                - pattern-inside: |
                    $JWT.verify($DATA,$VALUE,...);
            - focus-metavariable: $VALUE
      pattern-sources:
        - patterns:
            - pattern: "$X = '...' \n"
            - pattern: "$X = '$Y' \n"
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $JWT.sign($DATA,"...",...);
                - pattern-inside: |
                    $JWT.verify($DATA,"...",...);
      severity: WARNING
    - id: javascript.jsonwebtoken.security.jwt-none-alg.jwt-none-alg
      languages:
        - javascript
        - typescript
      message: Detected use of the 'none' algorithm in a JWT token. The 'none' algorithm assumes
 the integrity of the token has already been verified. This would allow a malicious actor to for
ge a JWT token that will automatically be verified. Do not explicitly use the 'none' algorithm.
Instead, use an algorithm such as 'HS256'.
      metadata:
        asvs:
            control_id: 3.5.3 Insecue Stateless Session Tokens
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x12-V3-Session-manage
ment.md#v35-token-based-session-management
            section: 'V3: Session Management Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: HIGH
        likelihood: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory:
            - vuln
        technology:
            - jwt
      patterns:
        - pattern-inside: |
            $JWT = require("jsonwebtoken");
            ...
        - pattern: $JWT.verify($P, $X, {algorithms:[...,'none',...]},...)
      severity: ERROR
    - id: javascript.jwt-simple.security.jwt-simple-noverify.jwt-simple-noverify
      languages:
        - javascript
        - typescript
      message: Detected the decoding of a JWT token without a verify step. JWT tokens must be ve
rified before use, otherwise the token's integrity is unknown. This means a malicious actor coul
d forge a JWT token with any claims. Set 'verify' to `true` before using the token.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-287: Improper Authentication'
            - 'CWE-345: Insufficient Verification of Data Authenticity'
            - 'CWE-347: Improper Verification of Cryptographic Signature'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://www.npmjs.com/package/jwt-simple
            - https://cwe.mitre.org/data/definitions/287
            - https://cwe.mitre.org/data/definitions/345
            - https://cwe.mitre.org/data/definitions/347
        subcategory:
            - vuln
        technology:
            - jwt-simple
            - jwt
      patterns:
        - pattern-inside: |
            $JWT = require('jwt-simple');
            ...
        - pattern: $JWT.decode($TOKEN, $SECRET, $NOVERIFY, ...)
        - metavariable-pattern:
            metavariable: $NOVERIFY
            patterns:
                - pattern-either:
                    - pattern: |
                        true
                    - pattern: |
                        "..."
      severity: ERROR
    - id: javascript.lang.security.audit.code-string-concat.code-string-concat
      languages:
        - javascript
        - typescript
      message: Found data from an Express or Next web request flowing to `eval`. If this data is
 user-controllable this can lead to execution of arbitrary system commands in the context of you
r application process. Avoid `eval` whenever possible.
      metadata:
        category: security
        confidence: HIGH
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: MEDIUM
        interfile: true
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/e
val
            - https://nodejs.org/api/child_process.html#child_processexeccommand-options-callbac
k
            - https://www.stackhawk.com/blog/nodejs-command-injection-examples-and-prevention/
            - https://ckarande.gitbooks.io/owasp-nodegoat-tutorial/content/tutorial/a1_-_server_
side_js_injection.html
        subcategory:
            - vuln
        technology:
            - node.js
            - Express
            - Next.js
      mode: taint
      options:
        interfile: true
      pattern-sinks:
        - patterns:
            - pattern: |
                eval(...)
      pattern-sources:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern-inside: function ... ($REQ, $RES) {...}
                    - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                    - patterns:
                        - pattern-either:
                            - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...}
)
                            - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT
) {...})
                        - metavariable-regex:
                            metavariable: $METHOD
                            regex: ^(get|post|put|head|delete|options)$
                - pattern-either:
                    - pattern: $REQ.query
                    - pattern: $REQ.body
                    - pattern: $REQ.params
                    - pattern: $REQ.cookies
                    - pattern: $REQ.headers
            - patterns:
                - pattern-either:
                    - pattern-inside: |
                        import { ...,$IMPORT,... } from 'next/router'
                        ...
                    - pattern-inside: |
                        import $IMPORT from 'next/router';
                        ...
                - pattern-either:
                    - patterns:
                        - pattern-inside: |
                            $ROUTER = $IMPORT()
                            ...
                        - pattern-either:
                            - pattern-inside: |
                                const { ...,$PROPS,... } = $ROUTER.query
                                ...
                            - pattern-inside: |
                                var { ...,$PROPS,... } = $ROUTER.query
                                ...
                            - pattern-inside: |
                                let { ...,$PROPS,... } = $ROUTER.query
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                            ...
                        - focus-metavariable: $PROPS
                    - patterns:
                        - pattern-inside: |
                            $ROUTER = $IMPORT()
                            ...
                        - pattern: "$ROUTER.query.$VALUE \n"
                    - patterns:
                        - pattern: $IMPORT().query.$VALUE
      severity: ERROR
    - id: javascript.lang.security.audit.sqli.node-knex-sqli.node-knex-sqli
      languages:
        - javascript
        - typescript
      message: 'Detected SQL statement that is tainted by `$REQ` object. This could lead to SQL
injection if the variable is user-controlled and not properly sanitized. In order to prevent SQL
 injection, it is recommended to use parameterized queries or prepared statements. An example of
 parameterized queries like so: `knex.raw(''SELECT $1 from table'', [userinput])` can help preve
nt SQLi.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://knexjs.org/#Builder-fromRaw
            - https://knexjs.org/#Builder-whereRaw
            - https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Shee
t.html
        subcategory:
            - vuln
        technology:
            - express
            - nodejs
            - knex
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern: parseInt(...)
      pattern-sinks:
        - patterns:
            - focus-metavariable: $QUERY
            - pattern-either:
                - pattern-inside: $KNEX.fromRaw($QUERY, ...)
                - pattern-inside: $KNEX.whereRaw($QUERY, ...)
                - pattern-inside: $KNEX.raw($QUERY, ...)
            - pattern-either:
                - pattern-inside: |
                    require('knex')
                    ...
                - pattern-inside: |
                    import 'knex'
                    ...
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
                - pattern: $REQ.headers
                - pattern: $REQ.files.$ANYTHING.data.toString('utf8')
                - pattern: $REQ.files.$ANYTHING['data'].toString('utf8')
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
            - focus-metavariable: $REQ
            - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
                - pattern: files.$ANYTHING.data.toString('utf8')
                - pattern: files.$ANYTHING['data'].toString('utf8')
      severity: WARNING
    - id: javascript.lang.security.detect-eval-with-expression.detect-eval-with-expression
      languages:
        - javascript
        - typescript
      message: Detected use of dynamic execution of JavaScript which may come from user-input, w
hich can lead to Cross-Site-Scripting (XSS). Where possible avoid including user-input in functi
ons which dynamically execute user-input.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/e
val#never_use_eval!
        source-rule-url: https://github.com/nodesecurity/eslint-plugin-security/blob/master/rule
s/detect-eval-with-expression.js
        subcategory:
            - vuln
        technology:
            - javascript
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern: location.href = $FUNC(...)
                - pattern: location.hash = $FUNC(...)
                - pattern: location.search = $FUNC(...)
                - pattern: $WINDOW. ... .location.href = $FUNC(...)
                - pattern: $WINDOW. ... .location.hash = $FUNC(...)
                - pattern: $WINDOW. ... .location.search = $FUNC(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: eval(<... $SINK ...>)
                - pattern: window.eval(<... $SINK ...>)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: new Function(<... $SINK ...>)
                - pattern: new Function(<... $SINK ...>)(...)
                - pattern: setTimeout(<... $SINK ...>,...)
                - pattern: setInterval(<... $SINK ...>,...)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $PROP = new URLSearchParams($WINDOW. ... .location.search).get('...')
                      ...
                - pattern-inside: |
                    $PROP = new URLSearchParams(location.search).get('...')
                      ...
                - pattern-inside: |
                    $PROP = new URLSearchParams($WINDOW. ... .location.hash.substring(1)).get('.
..')
                      ...
                - pattern-inside: |
                    $PROP = new URLSearchParams(location.hash.substring(1)).get('...')
                      ...
            - focus-metavariable: $PROP
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $PROPS = new URLSearchParams($WINDOW. ... .location.search)
                      ...
                - pattern-inside: |
                    $PROPS = new URLSearchParams(location.search)
                      ...
                - pattern-inside: |
                    $PROPS = new
                    URLSearchParams($WINDOW. ... .location.hash.substring(1))
                      ...
                - pattern-inside: |
                    $PROPS = new URLSearchParams(location.hash.substring(1))
                    ...
            - pattern: $PROPS.get('...')
            - focus-metavariable: $PROPS
        - patterns:
            - pattern-either:
                - pattern: location.href
                - pattern: location.hash
                - pattern: location.search
                - pattern: $WINDOW. ... .location.href
                - pattern: $WINDOW. ... .location.hash
                - pattern: $WINDOW. ... .location.search
      severity: WARNING
    - id: javascript.passport-jwt.security.passport-hardcode.hardcoded-passport-secret
      languages:
        - javascript
        - typescript
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
      metadata:
        asvs:
            control_id: 3.5.2 Static API keys or secret
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x12-V3-Session-manage
ment.md#v35-token-based-session-management
            section: 'V3: Session Management Verification Requirements'
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
  owasp:
      - A07:2021 - Identification and Authentication Failures
  references:
      - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
  subcategory:
      - vuln
  technology:
      - jwt
      - nodejs
      - secrets
mode: taint
pattern-sinks:
  - patterns:
      - pattern-either:
          - pattern-inside: |
              $F = require("$I").Strategy
              ...
          - pattern-inside: |
              $F = require("$I")
              ...
          - pattern-inside: |
              import { $STRAT as $F } from '$I'
              ...
          - pattern-inside: |
              import $F from '$I'
              ...
      - metavariable-regex:
          metavariable: $I
          regex: (passport-.*)
      - pattern-inside: |
          new $F($VALUE,...)
      - focus-metavariable: $VALUE
pattern-sources:
  - by-side-effect: true
    patterns:
      - pattern-either:
          - pattern: |
              {..., clientSecret: "...", ...}
          - pattern: |
              {..., secretOrKey: "...", ...}
          - pattern: |
              {..., consumerSecret: "...", ...}
          - patterns:
              - pattern-inside: |
                  $OBJ = {}
                  ...
              - pattern-either:
                  - pattern: |
                      $OBJ.clientSecret = "..."
                  - pattern: |
                      $OBJ.secretOrKey = "..."
                  - pattern: |
                      $OBJ.consumerSecret = "..."
              - pattern: $OBJ
          - patterns:
              - pattern-inside: |
                  $SECRET = '...'
                  ...
              - pattern-either:
                  - pattern: |
                      {..., clientSecret: $SECRET, ...}
                  - pattern: |
                      {..., secretOrKey: $SECRET, ...}
                  - pattern: |
                      {..., consumerSecret: $SECRET, ...}
          - patterns:
              - pattern-inside: |
                  $SECRET = '...'
                  ...
              - pattern-either:
                  - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                          $VALUE = {..., clientSecret: $SECRET, ...}
                          ...
                      - pattern-inside: |
                          $VALUE = {..., secretOrKey: $SECRET, ...}
                          ...
                      - pattern-inside: |
                          $VALUE = {..., consumerSecret: $SECRET, ...}
                          ...
                  - pattern: $VALUE
      severity: WARNING
    - id: javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-inje
ction
      languages:
        - javascript
        - typescript
      message: Detected a sequelize statement that is tainted by user-input. This could lead to
SQL injection if the variable is user-controlled and is not properly sanitized. In order to prev
ent SQL injection, it is recommended to use parameterized queries or prepared statements.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        interfile: true
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://sequelize.org/docs/v6/core-concepts/raw-queries/#replacements
        subcategory:
            - vuln
        technology:
            - express
      mode: taint
      options:
        interfile: true
      pattern-sanitizers:
        - pattern-either:
            - pattern: parseInt(...)
            - pattern: $FUNC. ... .hash(...)
      pattern-sinks:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: sequelize.query($QUERY,...)
                    - pattern: $DB.sequelize.query($QUERY,...)
                - focus-metavariable: $QUERY
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: function ... ($REQ, $RES) {...}
                - pattern-inside: function ... ($REQ, $RES, $NEXT) {...}
                - patterns:
                    - pattern-either:
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES) {...})
                        - pattern-inside: $APP.$METHOD(..., function $FUNC($REQ, $RES, $NEXT) {.
..})
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: ^(get|post|put|head|delete|options)$
            - pattern-either:
                - pattern: $REQ.query
                - pattern: $REQ.body
                - pattern: $REQ.params
                - pattern: $REQ.cookies
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: $REQ.headers
                - pattern: $REQ.files.$ANYTHING.data.toString('utf8')
                - pattern: $REQ.files.$ANYTHING['data'].toString('utf8')
          - patterns:
              - pattern-either:
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response, $NEXT: NextFunction) =>
                    {...}
                - pattern-inside: |
                    ({ $REQ }: Request,$RES: Response) => {...}
              - focus-metavariable: $REQ
              - pattern-either:
                - pattern: params
                - pattern: query
                - pattern: cookies
                - pattern: headers
                - pattern: body
                - pattern: files.$ANYTHING.data.toString('utf8')
                - pattern: files.$ANYTHING['data'].toString('utf8')
        severity: ERROR
      - id: json.aws.security.public-s3-bucket.public-s3-bucket
        languages:
          - json
        message: Detected public S3 bucket. This policy allows anyone to have some kind of access
to the bucket. The exact level of access and types of actions allowed will depend on the configu
ration of bucket policy and ACLs. Please review the bucket configuration to make sure they are s
et with intended values.
        metadata:
          category: security
          confidence: MEDIUM
          cwe:
              - 'CWE-264: CWE CATEGORY: Permissions, Privileges, and Access Controls'
          impact: HIGH
          license: Commons Clause License Condition v1.0[LGPL-2.1-only]
          likelihood: LOW
          owasp:
              - A01:2021 - Broken Access Control
          references:
              - https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-
access.html
          subcategory:
              - vuln
          technology:
              - aws
        patterns:
          - pattern-inside: |
              $BUCKETNAME: {
                "Type": "AWS::S3::Bucket",
                "Properties": {
                ...,
                },
                ...,
              }
          - pattern-either:
              - pattern: |
                  "PublicAccessBlockConfiguration": {
                       ...,
                       "RestrictPublicBuckets": false,
                       ...,
                     },
              - pattern: |
                  "PublicAccessBlockConfiguration": {
                       ...,
                       "IgnorePublicAcls": false,
                       ...,
                     },
              - pattern: |
                  "PublicAccessBlockConfiguration": {
                       ...,
                       "BlockPublicAcls": false,
                       ...,
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                  },
            - pattern: |
                "PublicAccessBlockConfiguration": {
                    ...,
                    "BlockPublicPolicy": false,
                    ...,
                  },
      severity: WARNING
    - id: json.aws.security.public-s3-policy-statement.public-s3-policy-statement
      languages:
        - json
      message: Detected public S3 bucket policy. This policy allows anyone to access certain pro
perties of or items in the bucket. Do not do this unless you will never have sensitive data insi
de the bucket.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-264: CWE CATEGORY: Permissions, Privileges, and Access Controls'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://docs.aws.amazon.com/AmazonS3/latest/dev/WebsiteAccessPermissionsReqd.html
        subcategory:
            - vuln
        technology:
            - aws
      pattern: |
        {
          "Effect": "Allow",
          "Principal": "*",
          "Resource": [
            ..., "=~/arn:aws:s3.*/", ...
          ],
          ...
        }
      severity: WARNING
    - id: json.aws.security.wildcard-assume-role.wildcard-assume-role
      languages:
        - json
      message: 'Detected wildcard access granted to sts:AssumeRole. This means anyone with your
AWS account ID and the name of the role can assume the role. Instead, limit to a specific identi
ty in your account, like this: `arn:aws:iam::<account_id>:root`.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-250: Execution with Unnecessary Privileges'
        impact: HIGH
        likelihood: HIGH
        owasp:
            - A06:2017 - Security Misconfiguration
            - A05:2021 - Security Misconfiguration
        references:
            - https://rhinosecuritylabs.com/aws/assume-worst-aws-assume-role-enumeration/
        subcategory:
            - vuln
        technology:
            - aws
      patterns:
        - pattern-inside: |
            "Statement": [...]
        - pattern-inside: |
            {..., "Effect": "Allow", ..., "Action": "sts:AssumeRole", ...}
        - pattern: |
            "Principal": {..., "AWS": "*", ...}
      severity: ERROR
    - id: kotlin.lang.security.anonymous-ldap-bind.anonymous-ldap-bind
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - kt
      message: Detected anonymous LDAP bind. This permits anonymous users to execute LDAP statem
ents. Consider enforcing authentication for LDAP. See https://docs.oracle.com/javase/tutorial/jn
di/ldap/auth_mechs.html for more information.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-287: Improper Authentication'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A02:2017 - Broken Authentication
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#LDAP_ANONYMOUS
        subcategory:
            - vuln
        technology:
            - kotlin
      pattern: |
        $ENV.put($CTX.SECURITY_AUTHENTICATION, "none")
        ...
        $DCTX = InitialDirContext($ENV, ...)
      severity: WARNING
    - id: kotlin.lang.security.ecb-cipher.ecb-cipher
      languages:
        - kt
      message: Cipher in ECB mode is detected. ECB mode produces the same output for the same in
put each time which allows an attacker to intercept and replay the data. Further, ECB mode does
not provide any integrity checking. See https://find-sec-bugs.github.io/bugs.htm#CIPHER_INTEGRIT
Y.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#ECB_MODE
        subcategory:
            - vuln
        technology:
            - kotlin
      patterns:
        - pattern-either:
            - pattern: |
                val $VAR : Cipher = $CIPHER.getInstance($MODE)
            - pattern: |
                var $VAR : Cipher = $CIPHER.getInstance($MODE)
            - pattern: |
                val $VAR = $CIPHER.getInstance($MODE)
            - pattern: |
                var $VAR = $CIPHER.getInstance($MODE)
        - metavariable-regex:
            metavariable: $MODE
            regex: .*ECB.*
      severity: WARNING
    - id: kotlin.lang.security.no-null-cipher.no-null-cipher
      languages:
        - kt
        - scala
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: 'NullCipher was detected. This will not encrypt anything; the cipher text will be
 the same as the plain text. Use a valid, secure cipher: Cipher.getInstance("AES/CBC/PKCS7PADDIN
G"). See https://owasp.org/www-community/Using_the_Java_Cryptographic_Extensions for more inform
ation.'
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#NULL_CIPHER
        subcategory:
            - vuln
        technology:
            - kotlin
      pattern: NullCipher(...)
      severity: WARNING
    - id: kotlin.lang.security.use-of-md5.use-of-md5
      languages:
        - kt
      message: Detected MD5 hash algorithm which is considered insecure. MD5 is not collision re
sistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-328: Use of Weak Hash'
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#WEAK_MESSAGE_DIGEST_MD5
        subcategory:
            - vuln
        technology:
            - kotlin
      pattern-either:
        - pattern: |
            $VAR = $MD.getInstance("MD5")
        - pattern: |
            $DU.getMd5Digest().digest(...)
      severity: WARNING
    - id: kotlin.lang.security.use-of-sha1.use-of-sha1
      languages:
        - kt
      message: Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision
resistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead
.
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#WEAK_MESSAGE_DIGEST_SHA1
        subcategory:
            - vuln
        technology:
            - kotlin
      pattern-either:
        - patterns:
            - pattern: |
                $VAR = $MD.getInstance("$ALGO")
            - metavariable-regex:
                metavariable: $ALGO
                regex: (SHA1|SHA-1)
        - pattern: |
            $DU.getSha1Digest().digest(...)
      severity: WARNING
    - id: kotlin.lang.security.weak-rsa.use-of-weak-rsa-key
      languages:
        - kt
      message: RSA keys should be at least 2048 bits based on NIST recommendation.
      metadata:
        asvs:
            control_id: 6.2.5 Insecure Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.h
tml#algorithms
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#RSA_KEY_SIZE
        subcategory:
            - audit
        technology:
            - kotlin
      patterns:
        - pattern-either:
            - pattern: |
                $KEY = $G.getInstance("RSA")
                ...
                $KEY.initialize($BITS)
        - metavariable-comparison:
            comparison: $BITS < 2048
            metavariable: $BITS
      severity: WARNING
    - id: php.doctrine.security.audit.doctrine-orm-dangerous-query.doctrine-orm-dangerous-query
      languages:
        - php
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: ``$QUERY` Detected string concatenation with a non-literal variable in a Doctrine
 QueryBuilder method. This could lead to SQL injection if the variable is user-controlled and no
t properly sanitized. In order to prevent SQL injection, use parameterized queries or prepared s
tatements instead.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://www.doctrine-project.org/projects/doctrine-dbal/en/current/reference/query
-builder.html#security-safely-preventing-sql-injection
            - https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Shee
t.html
        subcategory:
            - vuln
        technology:
            - doctrine
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $SINK
            - pattern-either:
                - pattern: $QUERY->add(...,$SINK,...)
                - pattern: $QUERY->select(...,$SINK,...)
                - pattern: $QUERY->addSelect(...,$SINK,...)
                - pattern: $QUERY->delete(...,$SINK,...)
                - pattern: $QUERY->update(...,$SINK,...)
                - pattern: $QUERY->insert(...,$SINK,...)
                - pattern: $QUERY->from(...,$SINK,...)
                - pattern: $QUERY->join(...,$SINK,...)
                - pattern: $QUERY->innerJoin(...,$SINK,...)
                - pattern: $QUERY->leftJoin(...,$SINK,...)
                - pattern: $QUERY->rightJoin(...,$SINK,...)
                - pattern: $QUERY->where(...,$SINK,...)
                - pattern: $QUERY->andWhere(...,$SINK,...)
                - pattern: $QUERY->orWhere(...,$SINK,...)
                - pattern: $QUERY->groupBy(...,$SINK,...)
                - pattern: $QUERY->addGroupBy(...,$SINK,...)
                - pattern: $QUERY->having(...,$SINK,...)
                - pattern: $QUERY->andHaving(...,$SINK,...)
                - pattern: $QUERY->orHaving(...,$SINK,...)
                - pattern: $QUERY->orderBy(...,$SINK,...)
                - pattern: $QUERY->addOrderBy(...,$SINK,...)
                - pattern: $QUERY->set($SINK,...)
                - pattern: $QUERY->setValue($SINK,...)
            - pattern-either:
                - pattern-inside: |
                    $Q = $X->createQueryBuilder();
                    ...
                - pattern-inside: |
                    $Q = new QueryBuilder(...);
                    ...
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: sprintf(...)
                - pattern: |
                    "...".$SMTH
      severity: WARNING
    - id: php.lang.security.assert-use.assert-use
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - php
      message: Calling assert with user input is equivalent to eval'ing.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://www.php.net/manual/en/function.assert
            - https://github.com/FloeDesignTechnologies/phpcs-security-audit/blob/master/Securit
y/Sniffs/BadFunctions/AssertsSniff.php
        subcategory:
            - vuln
        technology:
            - php
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: assert($SINK, ...);
            - pattern-not: assert("...", ...);
            - pattern: $SINK
      pattern-sources:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: $_GET
                    - pattern: $_POST
                    - pattern: $_COOKIE
                    - pattern: $_REQUEST
                    - pattern: $_SERVER
            - patterns:
                - pattern: |
                    Route::$METHOD($ROUTENAME, function(..., $ARG, ...) { ... })
                - focus-metavariable: $ARG
      severity: ERROR
    - id: php.lang.security.base-convert-loses-precision.base-convert-loses-precision
      languages:
        - php
      message: The function base_convert uses 64-bit numbers internally, and does not correctly
convert large numbers. It is not suitable for random tokens such as those used for session token
s or CSRF tokens.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-190: Integer Overflow or Wraparound'
        impact: LOW
        likelihood: LOW
        references:
            - https://www.php.net/base_convert
            - https://www.sjoerdlangkemper.nl/2017/03/15/dont-use-base-convert-on-random-tokens/
        subcategory:
            - audit
        technology:
            - php
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern: substr(..., $LENGTH)
            - metavariable-comparison:
                comparison: $LENGTH <= 7
                metavariable: $LENGTH
      pattern-sinks:
        - pattern: base_convert(...)
      pattern-sources:
        - pattern: hash(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern: hash_hmac(...)
        - pattern: sha1(...)
        - pattern: md5(...)
        - patterns:
            - pattern: random_bytes($N)
            - metavariable-comparison:
                comparison: $N > 7
                metavariable: $N
        - patterns:
            - pattern: openssl_random_pseudo_bytes($N)
            - metavariable-comparison:
                comparison: $N > 7
                metavariable: $N
        - patterns:
            - pattern: $OBJ->get_random_bytes($N)
            - metavariable-comparison:
                comparison: $N > 7
                metavariable: $N
      severity: WARNING
    - id: php.lang.security.curl-ssl-verifypeer-off.curl-ssl-verifypeer-off
      languages:
        - php
      message: SSL verification is disabled but should not be (currently CURLOPT_SSL_VERIFYPEER=
 $IS_VERIFIED)
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: LOW
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.saotn.org/dont-turn-off-curlopt_ssl_verifypeer-fix-php-configuration/
        subcategory:
            - vuln
        technology:
            - php
      patterns:
        - pattern-either:
            - pattern: |
                $ARG = $IS_VERIFIED;
                ...
                curl_setopt(..., CURLOPT_SSL_VERIFYPEER, $ARG);
            - pattern: curl_setopt(..., CURLOPT_SSL_VERIFYPEER, $IS_VERIFIED)
        - metavariable-regex:
            metavariable: $IS_VERIFIED
            regex: 0|false|null
      severity: ERROR
    - id: php.lang.security.deserialization.extract-user-data
      languages:
        - php
      message: Do not call 'extract()' on user-controllable data. If you must, then you must als
o provide the EXTR_SKIP flag to prevent overwriting existing variables.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://www.php.net/manual/en/function.extract.php#refsect1-function.extract-notes
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - vuln
        technology:
            - php
      mode: taint
      pattern-sanitizers:
        - pattern: extract($VAR, EXTR_SKIP,...)
      pattern-sinks:
        - pattern: extract(...)
      pattern-sources:
        - pattern-either:
            - pattern: $_GET[...]
            - pattern: $_FILES[...]
            - pattern: $_POST[...]
      severity: ERROR
    - fix: echo htmlentities($...VARS);
      id: php.lang.security.injection.echoed-request.echoed-request
      languages:
        - php
      message: '`Echo`ing user input risks cross-site scripting vulnerability. You should use `h
tmlentities()` when showing data to users.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://www.php.net/manual/en/function.htmlentities.php
            - https://www.php.net/manual/en/reserved.variables.request.php
            - https://www.php.net/manual/en/reserved.variables.post.php
            - https://www.php.net/manual/en/reserved.variables.get.php
            - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Che
at_Sheet.html
        subcategory:
            - vuln
        technology:
            - php
      mode: taint
      pattern-sanitizers:
        - pattern: htmlentities(...)
        - pattern: htmlspecialchars(...)
        - pattern: strip_tags(...)
        - pattern: isset(...)
        - pattern: empty(...)
        - pattern: esc_html(...)
        - pattern: esc_attr(...)
        - pattern: wp_kses(...)
        - pattern: e(...)
        - pattern: twig_escape_filter(...)
        - pattern: xss_clean(...)
        - pattern: html_escape(...)
        - pattern: Html::escape(...)
        - pattern: Xss::filter(...)
        - pattern: escapeHtml(...)
        - pattern: escapeHtml(...)
        - pattern: escapeHtmlAttr(...)
      pattern-sinks:
        - pattern: echo $...VARS;
      pattern-sources:
        - pattern: $_REQUEST
        - pattern: $_GET
        - pattern: $_POST
      severity: ERROR
    - fix: print(htmlentities($...VARS));
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      id: php.lang.security.injection.printed-request.printed-request
      languages:
        - php
      message: '`Printing user input risks cross-site scripting vulnerability. You should use `h
tmlentities()` when showing data to users.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://www.php.net/manual/en/function.htmlentities.php
            - https://www.php.net/manual/en/reserved.variables.request.php
            - https://www.php.net/manual/en/reserved.variables.post.php
            - https://www.php.net/manual/en/reserved.variables.get.php
            - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Che
at_Sheet.html
        subcategory:
            - vuln
        technology:
            - php
      mode: taint
      pattern-sanitizers:
        - pattern: htmlentities(...)
        - pattern: htmlspecialchars(...)
        - pattern: strip_tags(...)
        - pattern: isset(...)
        - pattern: empty(...)
        - pattern: esc_html(...)
        - pattern: esc_attr(...)
        - pattern: wp_kses(...)
        - pattern: e(...)
        - pattern: twig_escape_filter(...)
        - pattern: xss_clean(...)
        - pattern: html_escape(...)
        - pattern: Html::escape(...)
        - pattern: Xss::filter(...)
        - pattern: escapeHtml(...)
        - pattern: escapeHtml(...)
        - pattern: escapeHtmlAttr(...)
      pattern-sinks:
        - pattern: print($...VARS);
      pattern-sources:
        - pattern: $_REQUEST
        - pattern: $_GET
        - pattern: $_POST
      severity: ERROR
    - id: php.lang.security.injection.tainted-filename.tainted-filename
      languages:
        - php
      message: File name based on user input risks server-side request forgery.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29
      subcategory:
        - vuln
      technology:
        - php
    mode: taint
    pattern-sanitizers:
      - patterns:
          - pattern-either:
              - pattern-inside: basename($PATH, ...)
              - pattern-inside: linkinfo($PATH, ...)
              - pattern-inside: readlink($PATH, ...)
              - pattern-inside: realpath($PATH, ...)
    pattern-sinks:
      - patterns:
          - pattern-either:
              - pattern-inside: opcache_compile_file($FILENAME, ...)
              - pattern-inside: opcache_invalidate($FILENAME, ...)
              - pattern-inside: opcache_is_script_cached($FILENAME, ...)
              - pattern-inside: runkit7_import($FILENAME, ...)
              - pattern-inside: readline_read_history($FILENAME, ...)
              - pattern-inside: readline_write_history($FILENAME, ...)
              - pattern-inside: rar_open($FILENAME, ...)
              - pattern-inside: zip_open($FILENAME, ...)
              - pattern-inside: gzfile($FILENAME, ...)
              - pattern-inside: gzopen($FILENAME, ...)
              - pattern-inside: readgzfile($FILENAME, ...)
              - pattern-inside: hash_file($ALGO, $FILENAME, ...)
              - pattern-inside: hash_update_file($CONTEXT, $FILENAME, ...)
              - pattern-inside: pg_trace($FILENAME, ...)
              - pattern-inside: dio_open($FILENAME, ...)
              - pattern-inside: finfo_file($FINFO, $FILENAME, ...)
              - pattern-inside: mime_content_type($FILENAME, ...)
              - pattern-inside: chgrp($FILENAME, ...)
              - pattern-inside: chmod($FILENAME, ...)
              - pattern-inside: chown($FILENAME, ...)
              - pattern-inside: clearstatcache($CLEAR_REALPATH_CACHE, $FILENAME, ...)
              - pattern-inside: file_exists($FILENAME, ...)
              - pattern-inside: file_get_contents($FILENAME, ...)
              - pattern-inside: file_put_contents($FILENAME, ...)
              - pattern-inside: file($FILENAME, ...)
              - pattern-inside: fileatime($FILENAME, ...)
              - pattern-inside: filectime($FILENAME, ...)
              - pattern-inside: filegroup($FILENAME, ...)
              - pattern-inside: fileinode($FILENAME, ...)
              - pattern-inside: filemtime($FILENAME, ...)
              - pattern-inside: fileowner($FILENAME, ...)
              - pattern-inside: fileperms($FILENAME, ...)
              - pattern-inside: filesize($FILENAME, ...)
              - pattern-inside: filetype($FILENAME, ...)
              - pattern-inside: fnmatch($PATTERN, $FILENAME, ...)
              - pattern-inside: fopen($FILENAME, ...)
              - pattern-inside: is_dir($FILENAME, ...)
              - pattern-inside: is_executable($FILENAME, ...)
              - pattern-inside: is_file($FILENAME, ...)
              - pattern-inside: is_link($FILENAME, ...)
              - pattern-inside: is_readable($FILENAME, ...)
              - pattern-inside: is_uploaded_file($FILENAME, ...)
              - pattern-inside: is_writable($FILENAME, ...)
              - pattern-inside: lchgrp($FILENAME, ...)
              - pattern-inside: lchown($FILENAME, ...)
              - pattern-inside: lstat($FILENAME, ...)
              - pattern-inside: parse_ini_file($FILENAME, ...)
              - pattern-inside: readfile($FILENAME, ...)
              - pattern-inside: stat($FILENAME, ...)
              - pattern-inside: touch($FILENAME, ...)
              - pattern-inside: unlink($FILENAME, ...)
              - pattern-inside: xattr_get($FILENAME, ...)
              - pattern-inside: xattr_list($FILENAME, ...)
              - pattern-inside: xattr_remove($FILENAME, ...)
              - pattern-inside: xattr_set($FILENAME, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern-inside: xattr_supported($FILENAME, ...)
                - pattern-inside: enchant_broker_request_pwl_dict($BROKER, $FILENAME, ...)
                - pattern-inside: pspell_config_personal($CONFIG, $FILENAME, ...)
                - pattern-inside: pspell_config_repl($CONFIG, $FILENAME, ...)
                - pattern-inside: pspell_new_personal($FILENAME, ...)
                - pattern-inside: exif_imagetype($FILENAME, ...)
                - pattern-inside: getimagesize($FILENAME, ...)
                - pattern-inside: image2wbmp($IMAGE, $FILENAME, ...)
                - pattern-inside: imagecreatefromavif($FILENAME, ...)
                - pattern-inside: imagecreatefrombmp($FILENAME, ...)
                - pattern-inside: imagecreatefromgd2($FILENAME, ...)
                - pattern-inside: imagecreatefromgd2part($FILENAME, ...)
                - pattern-inside: imagecreatefromgd($FILENAME, ...)
                - pattern-inside: imagecreatefromgif($FILENAME, ...)
                - pattern-inside: imagecreatefromjpeg($FILENAME, ...)
                - pattern-inside: imagecreatefrompng($FILENAME, ...)
                - pattern-inside: imagecreatefromtga($FILENAME, ...)
                - pattern-inside: imagecreatefromwbmp($FILENAME, ...)
                - pattern-inside: imagecreatefromwebp($FILENAME, ...)
                - pattern-inside: imagecreatefromxbm($FILENAME, ...)
                - pattern-inside: imagecreatefromxpm($FILENAME, ...)
                - pattern-inside: imageloadfont($FILENAME, ...)
                - pattern-inside: imagexbm($IMAGE, $FILENAME, ...)
                - pattern-inside: iptcembed($IPTC_DATA, $FILENAME, ...)
                - pattern-inside: mailparse_msg_extract_part_file($MIMEMAIL, $FILENAME, ...)
                - pattern-inside: mailparse_msg_extract_whole_part_file($MIMEMAIL, $FILENAME, ..
.)
                - pattern-inside: mailparse_msg_parse_file($FILENAME, ...)
                - pattern-inside: fdf_add_template($FDF_DOCUMENT, $NEWPAGE, $FILENAME, ...)
                - pattern-inside: fdf_get_ap($FDF_DOCUMENT, $FIELD, $FACE, $FILENAME, ...)
                - pattern-inside: fdf_open($FILENAME, ...)
                - pattern-inside: fdf_save($FDF_DOCUMENT, $FILENAME, ...)
                - pattern-inside: fdf_set_ap($FDF_DOCUMENT, $FIELD_NAME, $FACE, $FILENAME, ...)
                - pattern-inside: ps_add_launchlink($PSDOC, $LLX, $LLY, $URX, $URY, $FILENAME, .
..)
                - pattern-inside: ps_add_pdflink($PSDOC, $LLX, $LLY, $URX, $URY, $FILENAME, ...)
                - pattern-inside: ps_open_file($PSDOC, $FILENAME, ...)
                - pattern-inside: ps_open_image_file($PSDOC, $TYPE, $FILENAME, ...)
                - pattern-inside: posix_access($FILENAME, ...)
                - pattern-inside: posix_mkfifo($FILENAME, ...)
                - pattern-inside: posix_mknod($FILENAME, ...)
                - pattern-inside: ftok($FILENAME, ...)
                - pattern-inside: fann_cascadetrain_on_file($ANN, $FILENAME, ...)
                - pattern-inside: fann_read_train_from_file($FILENAME, ...)
                - pattern-inside: fann_train_on_file($ANN, $FILENAME, ...)
                - pattern-inside: highlight_file($FILENAME, ...)
                - pattern-inside: php_strip_whitespace($FILENAME, ...)
                - pattern-inside: stream_resolve_include_path($FILENAME, ...)
                - pattern-inside: swoole_async_read($FILENAME, ...)
                - pattern-inside: swoole_async_readfile($FILENAME, ...)
                - pattern-inside: swoole_async_write($FILENAME, ...)
                - pattern-inside: swoole_async_writefile($FILENAME, ...)
                - pattern-inside: swoole_load_module($FILENAME, ...)
                - pattern-inside: tidy_parse_file($FILENAME, ...)
                - pattern-inside: tidy_repair_file($FILENAME, ...)
                - pattern-inside: get_meta_tags($FILENAME, ...)
                - pattern-inside: yaml_emit_file($FILENAME, ...)
                - pattern-inside: yaml_parse_file($FILENAME, ...)
                - pattern-inside: curl_file_create($FILENAME, ...)
                - pattern-inside: ftp_chmod($FTP, $PERMISSIONS, $FILENAME, ...)
                - pattern-inside: ftp_delete($FTP, $FILENAME, ...)
                - pattern-inside: ftp_mdtm($FTP, $FILENAME, ...)
                - pattern-inside: ftp_size($FTP, $FILENAME, ...)
                - pattern-inside: rrd_create($FILENAME, ...)
                - pattern-inside: rrd_fetch($FILENAME, ...)
                - pattern-inside: rrd_graph($FILENAME, ...)
                - pattern-inside: rrd_info($FILENAME, ...)
                - pattern-inside: rrd_last($FILENAME, ...)
                - pattern-inside: rrd_lastupdate($FILENAME, ...)
                - pattern-inside: rrd_tune($FILENAME, ...)
                - pattern-inside: rrd_update($FILENAME, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern-inside: snmp_read_mib($FILENAME, ...)
                - pattern-inside: ssh2_sftp_chmod($SFTP, $FILENAME, ...)
                - pattern-inside: ssh2_sftp_realpath($SFTP, $FILENAME, ...)
                - pattern-inside: ssh2_sftp_unlink($SFTP, $FILENAME, ...)
                - pattern-inside: apache_lookup_uri($FILENAME, ...)
                - pattern-inside: md5_file($FILENAME, ...)
                - pattern-inside: sha1_file($FILENAME, ...)
                - pattern-inside: simplexml_load_file($FILENAME, ...)
            - pattern: $FILENAME
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: $_GET
                - pattern: $_POST
                - pattern: $_COOKIE
                - pattern: $_REQUEST
                - pattern: $_SERVER
      severity: WARNING
    - id: php.lang.security.injection.tainted-object-instantiation.tainted-object-instantiation
      languages:
        - php
      message: <- A new object is created where the class name is based on user input. This coul
d lead to remote code execution, as it allows to instantiate any class in the application.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-470: Use of Externally-Controlled Input to Select Classes or Code (''Unsafe R
eflection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - php
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: new $SINK(...)
            - pattern: $SINK
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: $_GET
                - pattern: $_POST
                - pattern: $_COOKIE
                - pattern: $_REQUEST
                - pattern: $_SERVER
      severity: WARNING
    - id: php.lang.security.injection.tainted-session.tainted-session
      languages:
        - php
      message: Session key based on user input risks session poisoning. The user can determine t
he key used for the session, and thus write any session variable. Session variables are typicall
y trusted to be set only by the application, and manipulating the session can result in access c
ontrol issues.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-284: Improper Access Control'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A01:2021 - Broken Access Control
        references:
            - https://en.wikipedia.org/wiki/Session_poisoning
        subcategory:
            - vuln
        technology:
            - php
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern: $A . $B
                - pattern: bin2hex(...)
                - pattern: crc32(...)
                - pattern: crypt(...)
                - pattern: filter_input(...)
                - pattern: filter_var(...)
                - pattern: hash(...)
                - pattern: md5(...)
                - pattern: preg_filter(...)
                - pattern: preg_grep(...)
                - pattern: preg_match_all(...)
                - pattern: sha1(...)
                - pattern: sprintf(...)
                - pattern: str_contains(...)
                - pattern: str_ends_with(...)
                - pattern: str_starts_with(...)
                - pattern: strcasecmp(...)
                - pattern: strchr(...)
                - pattern: stripos(...)
                - pattern: stristr(...)
                - pattern: strnatcasecmp(...)
                - pattern: strnatcmp(...)
                - pattern: strncmp(...)
                - pattern: strpbrk(...)
                - pattern: strpos(...)
                - pattern: strripos(...)
                - pattern: strrpos(...)
                - pattern: strspn(...)
                - pattern: strstr(...)
                - pattern: strtok(...)
                - pattern: substr_compare(...)
                - pattern: substr_count(...)
                - pattern: vsprintf(...)
      pattern-sinks:
        - patterns:
            - pattern-inside: $_SESSION[$KEY] = $VAL;
            - pattern: $KEY
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: $_GET
                - pattern: $_POST
                - pattern: $_COOKIE
                - pattern: $_REQUEST
      severity: WARNING
    - id: php.lang.security.injection.tainted-sql-string.tainted-sql-string
      languages:
        - php
      message: User data flows into this manually-constructed SQL string. User data can be safel
y inserted into SQL strings using prepared statements or an object-relational mapper (ORM). Manu
ally-constructed SQL strings is a possible indicator of SQL injection, which could let an attack
er steal or manipulate data from the database. Instead, use prepared statements (`$mysqli->prepa
re("INSERT INTO test(id, label) VALUES (?, ?)");`) or a safe library.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/SQL_Injection
        subcategory:
            - vuln
        technology:
            - php
      mode: taint
      pattern-sanitizers:
        - pattern-either:
            - pattern: mysqli_real_escape_string(...)
            - pattern: real_escape_string(...)
            - pattern: $MYSQLI->real_escape_string(...)
      pattern-sinks:
        - pattern-either:
            - patterns:
                - pattern: |
                    sprintf($SQLSTR, ...)
                - metavariable-regex:
                    metavariable: $SQLSTR
                    regex: .*\b(?i)(select|delete|insert|create|update|alter|drop)\b.*
            - patterns:
                - pattern: |
                    "...$EXPR..."
                - metavariable-regex:
                    metavariable: $EXPR
                    regex: .*\b(?i)(select|delete|insert|create|update|alter|drop)\b.*
            - patterns:
                - pattern: |
                    "$SQLSTR".$EXPR
                - metavariable-regex:
                    metavariable: $SQLSTR
                    regex: .*\b(?i)(select|delete|insert|create|update|alter|drop)\b.*
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: $_GET
                - pattern: $_POST
                - pattern: $_COOKIE
                - pattern: $_REQUEST
      severity: ERROR
    - id: php.lang.security.injection.tainted-url-host.tainted-url-host
      languages:
        - php
      message: User data flows into the host portion of this manually-constructed URL. This coul
d allow an attacker to send data to their own server, potentially exposing sensitive data such a
s cookies or authorization information sent with this request. They could also probe internal se
rvers or other resources that the server running this code can access. (This is called server-si
de request forgery, or SSRF.) Do not allow arbitrary hosts. Instead, create an allowlist for app
roved hosts, or hardcode the correct host.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevent
ion_Cheat_Sheet.html
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - vuln
          technology:
            - php
        mode: taint
        pattern-sinks:
          - pattern-either:
            - patterns:
                - pattern: |
                    sprintf($URLSTR, ...)
                - metavariable-pattern:
                    language: generic
                    metavariable: $URLSTR
                    pattern: $SCHEME://%s
            - patterns:
                - pattern: |
                    "...{$EXPR}..."
                - pattern-regex: |
                    .*://\{.*
            - patterns:
                - pattern: |
                    "...$EXPR..."
                - pattern-regex: |
                    .*://\$.*
            - patterns:
                - pattern: |
                    "...".$EXPR
                - pattern-regex: |
                    .*://["'].*
        pattern-sources:
          - patterns:
            - pattern-either:
                - pattern: $_GET
                - pattern: $_POST
                - pattern: $_COOKIE
                - pattern: $_REQUEST
        severity: WARNING
      - id: php.lang.security.md5-used-as-password.md5-used-as-password
        languages:
          - php
        message: It looks like MD5 is used as a password hash. MD5 is not considered a secure pass
word hash because it can be cracked by an attacker in a short amount of time. Use a suitable pas
sword hashing function such as bcrypt. You can use `password_hash($PASSWORD, PASSWORD_BCRYPT, $O
PTIONS);`.
        metadata:
          category: security
          confidence: MEDIUM
          cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
          impact: MEDIUM
          likelihood: HIGH
          owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
          references:
            - https://tools.ietf.org/html/rfc6151
            - https://crypto.stackexchange.com/questions/44151/how-does-the-flame-malware-take-a
dvantage-of-md5-collision
            - https://security.stackexchange.com/questions/211/how-to-securely-hash-passwords
            - https://github.com/returntocorp/semgrep-rules/issues/1609
            - https://www.php.net/password_hash
          subcategory:
            - vuln
          technology:
            - md5
        mode: taint
        pattern-sinks:
          - patterns:
            - pattern: $FUNCTION(...)
            - metavariable-regex:
                metavariable: $FUNCTION
                regex: (?i)(.*password.*)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: md5(...)
                - pattern: hash('md5', ...)
      severity: WARNING
    - id: php.lang.security.openssl-cbc-static-iv.openssl-cbc-static-iv
      languages:
        - php
      message: Static IV used with AES in CBC mode. Static IVs enable chosen-plaintext attacks a
gainst encrypted data.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-329: Generation of Predictable IV with CBC Mode'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://csrc.nist.gov/publications/detail/sp/800-38a/final
        subcategory:
            - vuln
        technology:
            - php
            - openssl
      patterns:
        - pattern-either:
            - pattern: openssl_encrypt($D, $M, $K, $FLAGS, "...",...);
            - pattern: openssl_decrypt($D, $M, $K, $FLAGS, "...",...);
        - metavariable-comparison:
            comparison: re.match(".*-CBC",$M)
            metavariable: $M
      severity: ERROR
    - id: php.lang.security.phpinfo-use.phpinfo-use
      languages:
        - php
      message: The 'phpinfo' function may reveal sensitive information about your environment.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://www.php.net/manual/en/function.phpinfo
            - https://github.com/FloeDesignTechnologies/phpcs-security-audit/blob/master/Securit
y/Sniffs/BadFunctions/PhpinfosSniff.php
        subcategory:
            - vuln
        technology:
            - php
      pattern: phpinfo(...);
      severity: ERROR
    - id: php.lang.security.redirect-to-request-uri.redirect-to-request-uri
      languages:
        - php
      message: Redirecting to the current request URL may redirect to another domain, if the cur
rent path starts with two slashes.  E.g. in https://www.example.com//attacker.com, the value of
REQUEST_URI is //attacker.com, and redirecting to it will redirect to that domain.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-601: URL Redirection to Untrusted Site (''Open Redirect'')'
        impact: LOW
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://www.php.net/manual/en/reserved.variables.server.php
            - https://owasp.org/www-project-top-ten/2017/A5_2017-Broken_Access_Control.html
        subcategory:
            - vuln
        technology:
            - php
      patterns:
        - pattern-either:
            - pattern: |
                header('$LOCATION' . $_SERVER['REQUEST_URI']);
            - pattern: |
                header('$LOCATION' . $_SERVER['REQUEST_URI'] . $MORE);
        - metavariable-regex:
            metavariable: $LOCATION
            regex: ^(?i)location:\s*$
      severity: WARNING
    - id: php.lang.security.tainted-exec.tainted-exec
      languages:
        - php
      message: Executing non-constant commands. This can lead to command injection. You should u
se `escapeshellarg()` when using command.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
        impact: HIGH
        likelihood: HIGH
        owasp:
            - A03:2021 - Injection
        references:
            - https://www.stackhawk.com/blog/php-command-injection/
            - https://brightsec.com/blog/code-injection-php/
            - https://www.acunetix.com/websitesecurity/php-security-2/
        subcategory:
            - vuln
        technology:
            - php
      mode: taint
      pattern-sanitizers:
        - pattern: escapeshellarg(...)
      pattern-sinks:
        - pattern: exec(...)
        - pattern: system(...)
        - pattern: popen(...)
        - pattern: passthru(...)
        - pattern: shell_exec(...)
        - pattern: pcntl_exec(...)
        - pattern: proc_open(...)
      pattern-sources:
        - pattern: $_REQUEST
        - pattern: $_GET
        - pattern: $_POST
        - pattern: $_COOKIE
      severity: ERROR
    - id: php.laravel.security.laravel-api-route-sql-injection.laravel-api-route-sql-injection
      languages:
        - php
      message: HTTP method [$METHOD] to Laravel route $ROUTE_NAME is vulnerable to SQL injection
 via string concatenation or unsafe interpolation.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Laravel_Cheat_Sh
eet.md
        subcategory:
            - vuln
        technology:
            - php
            - laravel
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern: |
                DB::raw("...",[...])
      pattern-sinks:
        - patterns:
            - pattern: |
                DB::raw(...)
      pattern-sources:
        - patterns:
            - focus-metavariable: $ARG
            - pattern-inside: |
                Route::$METHOD($ROUTE_NAME, function(...,$ARG,...){...})
      severity: WARNING
    - id: php.laravel.security.laravel-sql-injection.laravel-sql-injection
      languages:
        - php
      message: Detected a SQL query based on user input. This could lead to SQL injection, which
 could potentially result in sensitive data being exfiltrated by attackers. Instead, use paramet
erized queries and prepared statements.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://laravel.com/docs/8.x/queries
        subcategory:
            - vuln
        technology:
            - laravel
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: $SQL
                    - pattern-either:
                        - pattern-inside: DB::table(...)->whereRaw($SQL, ...)
                        - pattern-inside: DB::table(...)->orWhereRaw($SQL, ...)
                        - pattern-inside: DB::table(...)->groupByRaw($SQL, ...)
                        - pattern-inside: DB::table(...)->havingRaw($SQL, ...)
                        - pattern-inside: DB::table(...)->orHavingRaw($SQL, ...)
                        - pattern-inside: DB::table(...)->orderByRaw($SQL, ...)
                - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern: $EXPRESSION
                    - pattern-either:
                        - pattern-inside: DB::table(...)->selectRaw($EXPRESSION, ...)
                        - pattern-inside: DB::table(...)->fromRaw($EXPRESSION, ...)
                - patterns:
                    - pattern: $COLUMNS
                    - pattern-either:
                        - pattern-inside: DB::table(...)->whereNull($COLUMNS, ...)
                        - pattern-inside: DB::table(...)->orWhereNull($COLUMN)
                        - pattern-inside: DB::table(...)->whereNotNull($COLUMNS, ...)
                        - pattern-inside: DB::table(...)->whereRowValues($COLUMNS, ...)
                        - pattern-inside: DB::table(...)->orWhereRowValues($COLUMNS, ...)
                        - pattern-inside: DB::table(...)->find($ID, $COLUMNS)
                        - pattern-inside: DB::table(...)->paginate($PERPAGE, $COLUMNS, ...)
                        - pattern-inside: DB::table(...)->simplePaginate($PERPAGE, $COLUMNS, ...
)
                        - pattern-inside: DB::table(...)->cursorPaginate($PERPAGE, $COLUMNS, ...
)
                        - pattern-inside: DB::table(...)->getCountForPagination($COLUMNS)
                        - pattern-inside: DB::table(...)->aggregate($FUNCTION, $COLUMNS)
                        - pattern-inside: DB::table(...)->numericAggregate($FUNCTION, $COLUMNS)
                        - pattern-inside: DB::table(...)->insertUsing($COLUMNS, ...)
                        - pattern-inside: DB::table(...)->select($COLUMNS)
                        - pattern-inside: DB::table(...)->get($COLUMNS)
                        - pattern-inside: DB::table(...)->count($COLUMNS)
                - patterns:
                    - pattern: $COLUMN
                    - pattern-either:
                        - pattern-inside: DB::table(...)->whereIn($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereIn($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereNotIn($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereNotIn($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereIntegerInRaw($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereIntegerInRaw($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereIntegerNotInRaw($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereIntegerNotInRaw($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereBetweenColumns($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereBetween($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereBetweenColumns($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereNotBetween($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereNotBetweenColumns($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereNotBetween($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereNotBetweenColumns($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereNotNull($COLUMN)
                        - pattern-inside: DB::table(...)->whereDate($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereDate($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereTime($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereTime($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereDay($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereDay($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereMonth($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereMonth($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereYear($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereYear($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereJsonContains($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereJsonContains($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereJsonDoesntContain($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereJsonDoesntContain($COLUMN, ...)
                        - pattern-inside: DB::table(...)->whereJsonLength($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orWhereJsonLength($COLUMN, ...)
                        - pattern-inside: DB::table(...)->having($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orHaving($COLUMN, ...)
                        - pattern-inside: DB::table(...)->havingBetween($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orderBy($COLUMN, ...)
                        - pattern-inside: DB::table(...)->orderByDesc($COLUMN)
                        - pattern-inside: DB::table(...)->latest($COLUMN)
                        - pattern-inside: DB::table(...)->oldest($COLUMN)
                        - pattern-inside: DB::table(...)->forPageBeforeId($PERPAGE, $LASTID, $CO
LUMN)
                        - pattern-inside: DB::table(...)->forPageAfterId($PERPAGE, $LASTID, $COL
UMN)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                      - pattern-inside: DB::table(...)->value($COLUMN)
                      - pattern-inside: DB::table(...)->pluck($COLUMN, ...)
                      - pattern-inside: DB::table(...)->implode($COLUMN, ...)
                      - pattern-inside: DB::table(...)->min($COLUMN)
                      - pattern-inside: DB::table(...)->max($COLUMN)
                      - pattern-inside: DB::table(...)->sum($COLUMN)
                      - pattern-inside: DB::table(...)->avg($COLUMN)
                      - pattern-inside: DB::table(...)->average($COLUMN)
                      - pattern-inside: DB::table(...)->increment($COLUMN, ...)
                      - pattern-inside: DB::table(...)->decrement($COLUMN, ...)
                      - pattern-inside: DB::table(...)->where($COLUMN, ...)
                      - pattern-inside: DB::table(...)->orWhere($COLUMN, ...)
                      - pattern-inside: DB::table(...)->addSelect($COLUMN)
                - patterns:
                    - pattern: $QUERY
                    - pattern-inside: DB::unprepared($QUERY)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: $_GET
                - pattern: $_POST
                - pattern: $_COOKIE
                - pattern: $_REQUEST
                - pattern: $_SERVER
      severity: WARNING
    - id: php.laravel.security.laravel-unsafe-validator.laravel-unsafe-validator
      languages:
        - php
      message: Found a request argument passed to an `ignore()` definition in a Rule constraint.
This can lead to SQL injection.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://laravel.com/docs/9.x/validation#rule-unique
        subcategory:
            - vuln
        technology:
            - php
            - laravel
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                Illuminate\Validation\Rule::unique(...)->ignore(...,$IGNORE,...)
            - focus-metavariable: $IGNORE
      pattern-sources:
        - patterns:
            - pattern: |
                public function $F(...,Request $R,...){...}
            - focus-metavariable: $R
        - patterns:
            - pattern-either:
                - pattern: |
                    $this->$PROPERTY
                - pattern: |
                    $this->$PROPERTY->$GET
            - metavariable-pattern:
                metavariable: $PROPERTY
                patterns:
                    - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern: query
                        - pattern: request
                        - pattern: headers
                        - pattern: cookies
                        - pattern: cookie
                        - pattern: files
                        - pattern: file
                        - pattern: allFiles
                        - pattern: input
                        - pattern: all
                        - pattern: post
                        - pattern: json
            - pattern-either:
                - pattern-inside: |
                    class $CL extends Illuminate\Http\Request {...}
                - pattern-inside: |
                    class $CL extends Illuminate\Foundation\Http\FormRequest {...}
      severity: ERROR
    - id: problem-based-packs.insecure-transport.go-stdlib.bypass-tls-verification.bypass-tls-ve
rification
      languages:
        - go
      message: Checks for disabling of TLS/SSL certificate verification. This should only be use
d for debugging purposes because it leads to vulnerability to MTM attacks.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: HIGH
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://stackoverflow.com/questions/12122159/how-to-do-a-https-request-with-bad-ce
rtificate
        subcategory:
            - vuln
        technology:
            - go
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            tls.Config{..., InsecureSkipVerify: true, ...}
        - pattern: |
            $CONFIG = &tls.Config{...}
            ...
            $CONFIG.InsecureSkipVerify = true
      severity: WARNING
    - id: problem-based-packs.insecure-transport.go-stdlib.disallow-old-tls-versions.disallow-ol
d-tls-versions
      languages:
        - go
      message: Detects creations of tls configuration objects with an insecure MinVersion of TLS
. These protocols are deprecated due to POODLE, man in the middle attacks, and other vulnerabili
ties.
      metadata:
        category: security
        confidence: HIGH
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: HIGH
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://stackoverflow.com/questions/26429751/java-http-clients-and-poodle
        subcategory:
            - vuln
        technology:
            - go
        vulnerability: Insecure Transport
      patterns:
        - pattern-either:
            - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                tls.Config{..., MinVersion: $TLS.$VERSION, ...}
            - pattern: |
                $CONFIG = &tls.Config{...}
                ...
                $CONFIG.MinVersion = $TLS.$VERSION
        - metavariable-regex:
            metavariable: $VERSION
            regex: (VersionTLS10|VersionTLS11|VersionSSL30)
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[fF][tT][pP]://'
        replacement: sftp://
      id: problem-based-packs.insecure-transport.go-stdlib.ftp-request.ftp-request
      languages:
        - go
      message: Checks for outgoing connections to ftp servers with the ftp package. FTP does not
 encrypt traffic, possibly leading to PII being sent plaintext over the network. Instead, connec
t via the SFTP protocol.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://godoc.org/github.com/jlaffaye/ftp#Dial
            - https://github.com/jlaffaye/ftp
        subcategory:
            - vuln
        technology:
            - ftp
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            ftp.Dial("=~/^[fF][tT][pP]://.*/", ...)
        - pattern: |
            ftp.DialTimeout("=~/^[fF][tT][pP]://.*/", ...)
        - pattern: |
            ftp.Connect("=~/^[fF][tT][pP]://.*/")
        - pattern: |
            $URL = "=~/^[fF][tT][pP]://.*/"
            ...
            ftp.Dial($URL, ...)
        - pattern: |
            $URL = "=~/^[fF][tT][pP]://.*/"
            ...
            ftp.DialTimeout($URL, ...)
        - pattern: |
            $URL = "=~/^[fF][tT][pP]://.*/"
            ...
            ftp.Connect($URL)
      severity: WARNING
    - id: problem-based-packs.insecure-transport.go-stdlib.gorequest-http-request.gorequest-http
-request
      languages:
        - go
      message: Checks for requests to http (unencrypted) sites using gorequest, a popular HTTP c
lient library. This is dangerous because it could result in plaintext PII being passed around th
e network.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: HIGH
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://github.com/parnurzeal/gorequest
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - vuln
        technology:
            - gorequest
        vulnerability: Insecure Transport
      pattern-either:
        - patterns:
            - pattern-inside: |
                $REQ = gorequest.New()
                ...
                $RES = ...
            - pattern: |
                $REQ.$FUNC("=~/[hH][tT][tT][pP]://.*/")
            - metavariable-regex:
                metavariable: $FUNC
                regex: (Get|Post|Delete|Head|Put|Patch)
        - patterns:
            - pattern: gorequest.New().$FUNC("=~/[hH][tT][tT][pP]://.*/")
            - metavariable-regex:
                metavariable: $FUNC
                regex: (Get|Post|Delete|Head|Put|Patch)
      severity: WARNING
    - id: problem-based-packs.insecure-transport.go-stdlib.grequests-http-request.grequests-http
-request
      languages:
        - go
      message: Checks for requests to http (unencrypted) sites using grequests, a popular HTTP c
lient library. This is dangerous because it could result in plaintext PII being passed around th
e network.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://godoc.org/github.com/levigross/grequests#DoRegularRequest
            - https://github.com/levigross/grequests
        subcategory:
            - vuln
        technology:
            - grequests
        vulnerability: Insecure Transport
      patterns:
        - pattern-either:
            - pattern: |
                grequests.$FUNC(...,"=~/[hH][tT][tT][pP]://.*/", ...)
            - pattern: |
                $FUNC(...,"=~/[hH][tT][tT][pP]://.*/", ...)
        - metavariable-regex:
            metavariable: $FUNC
            regex: (Get|Head|Post|Put|Delete|Patch|Options|Req|DoRegularRequest)
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: problem-based-packs.insecure-transport.go-stdlib.http-customized-request.http-customiz
ed-request
      languages:
        - go
      message: Checks for requests sent via http.NewRequest to http:// URLS. This is dangerous b
ecause the server is attempting to connect to a website that does not encrypt traffic with TLS.
Instead, send requests only to https:// URLS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        references:
            - https://golang.org/pkg/net/http/#NewRequest
        subcategory:
            - vuln
        technology:
            - go
        vulnerability: Insecure Transport
      pattern: |
        http.NewRequest(..., "=~/[hH][tT][tT][pP]://.*/", ...)
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: problem-based-packs.insecure-transport.go-stdlib.http-request.http-request
      languages:
        - go
      message: Checks for requests sent via http.$FUNC to http:// URLS. This is dangerous becaus
e the server is attempting to connect to a website that does not encrypt traffic with TLS. Inste
ad, send requests only to https:// URLS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://golang.org/pkg/net/http/#Get
        subcategory:
            - vuln
        technology:
            - go
        vulnerability: Insecure Transport
      patterns:
        - pattern-either:
            - pattern: |
                http.$FUNC("=~/[hH][tT][tT][pP]://.*/", ...)
            - patterns:
                - pattern-inside: |
                    $CLIENT := &http.Client{...}
                    ...
                - pattern: |
                    client.$FUNC("=~/[hH][tT][tT][pP]://.*/", ...)
        - pattern-not: http.$FUNC("=~/[hH][tT][tT][pP]://127.0.0.1.*/", ...)
        - pattern-not: client.$FUNC("=~/[hH][tT][tT][pP]://127.0.0.1.*/", ...)
        - pattern-not: http.$FUNC("=~/[hH][tT][tT][pP]://localhost.*/", ...)
        - pattern-not: client.$FUNC("=~/[hH][tT][tT][pP]://localhost.*/", ...)
        - metavariable-regex:
            metavariable: $FUNC
            regex: (Get|Post|Head|PostForm)
      severity: WARNING
    - id: problem-based-packs.insecure-transport.go-stdlib.sling-http-request.sling-http-request
      languages:
        - go
      message: Checks for requests to http (unencrypted) sites using gorequest, a popular HTTP c
lient library. This is dangerous because it could result in plaintext PII being passed around th
e network.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://godoc.org/github.com/dghubble/sling#Sling.Add
            - https://github.com/dghubble/sling
        subcategory:
            - vuln
        technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - sling
        vulnerability: Insecure Transport
      pattern-either:
        - patterns:
            - pattern-inside: |
                $REQ = sling.New()
                ...
                $RES = ...
            - pattern: |
                $REQ.$FUNC("=~/[hH][tT][tT][pP]://.*/")
            - metavariable-regex:
                metavariable: $FUNC
                regex: (Get|Post|Delete|Head|Put|Options|Patch|Base|Connect)
        - patterns:
            - pattern: sling.New().$FUNC("=~/[hH][tT][tT][pP]://.*/")
            - metavariable-regex:
                metavariable: $FUNC
                regex: (Get|Post|Delete|Head|Put|Options|Patch|Base|Connect)
        - patterns:
            - pattern-inside: |
                $REQ = sling.New()
                ...
                $URL = "=~/[hH][tT][tT][pP]://.*/"
                ...
                $RES = ...
            - pattern: |
                $REQ.$FUNC($URL)
            - metavariable-regex:
                metavariable: $FUNC
                regex: (Get|Post|Delete|Head|Put|Options|Patch|Base|Connect)
        - patterns:
            - pattern-inside: |
                $URL = "=~/[hH][tT][tT][pP]://.*/"
                ...
                $RES = ...
            - pattern: |
                sling.New().$FUNC($URL)
            - metavariable-regex:
                metavariable: $FUNC
                regex: (Get|Post|Delete|Head|Put|Options|Patch|Base|Connect)
      severity: WARNING
    - id: problem-based-packs.insecure-transport.go-stdlib.telnet-request.telnet-request
      languages:
        - go
      message: Checks for attempts to connect to an insecure telnet server using the package tel
net. This is bad because it can lead to man in the middle attacks.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://godoc.org/github.com/reiver/go-telnet
        subcategory:
            - vuln
        technology:
            - go-telnet
        vulnerability: Insecure Transport
      pattern: |
        telnet.DialToAndCall(...)
      severity: WARNING
    - id: problem-based-packs.insecure-transport.java-spring.bypass-tls-verification.bypass-tls-
verification
      languages:
        - java
      message: Checks for redefinitions of functions that check TLS/SSL certificate verification
. This can lead to vulnerabilities, as simple errors in the code can result in lack of proper ce
rtificate validation. This should only be used for debugging purposes because it leads to vulner
ability to MTM attacks.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: HIGH
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://stackoverflow.com/questions/4072585/disabling-ssl-certificate-validation-i
n-spring-resttemplate
            - https://stackoverflow.com/questions/35530558/how-to-fix-unsafe-implementation-of-x
509trustmanager-in-android-app?rq=1
        subcategory:
            - vuln
        technology:
            - spring
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            new HostnameVerifier() {
              ...
              public boolean verify(String hostname, SSLSession session) {
                ...
              }
              ...
            };
        - pattern: |
            public RestTemplate restTemplate() throws KeyStoreException, NoSuchAlgorithmExceptio
n, KeyManagementException {
              ...
              TrustStrategy $FUNCNAME = (X509Certificate[] chain, String authType) -> ...;
              ...
            }
        - pattern: |
            TrustStrategy $FUNCNAME= new TrustStrategy() {
              ...
              public boolean isTrusted(X509Certificate[] x509Certificates, String s) throws Cert
ificateException {
                ...
              }
              ...
            };
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[fF][tT][pP]://'
        replacement: sftp://
      id: problem-based-packs.insecure-transport.java-spring.spring-ftp-request.spring-ftp-reque
st
      languages:
        - java
      message: Checks for outgoing connections to ftp servers via Spring plugin ftpSessionFactor
y. FTP does not encrypt traffic, possibly leading to PII being sent plaintext over the network.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://docs.spring.io/spring-integration/api/org/springframework/integration/ftp/
session/AbstractFtpSessionFactory.html#setClientMode-int-
        subcategory:
            - vuln
        technology:
            - spring
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            $SF = new DefaultFtpSessionFactory(...);
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
            $SF.setHost("=~/^[fF][tT][pP]://.*/");
            ...
            $SF.$FUNC(...);
        - pattern: |
            $SF = new DefaultFtpSessionFactory(...);
            ...
            String $URL = "=~/^[fF][tT][pP]://.*/";
            ...
            $SF.setHost($URL);
            ...
            $SF.$FUNC(...);
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: problem-based-packs.insecure-transport.java-spring.spring-http-request.spring-http-req
uest
      languages:
        - java
      message: Checks for requests sent via Java Spring RestTemplate API to http:// URLS. This i
s dangerous because the server is attempting to connect to a website that does not encrypt traff
ic with TLS. Instead, send requests only to https:// URLS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/web/cli
ent/RestTemplate.html#delete-java.lang.String-java.util.Map-
            - https://www.baeldung.com/rest-template
        subcategory:
            - vuln
        technology:
            - spring
        vulnerability: Insecure Transport
      patterns:
        - pattern-either:
            - pattern: |
                $RESTTEMP = new RestTemplate(...);
                ...
                $RESTTEMP.$FUNC("=~/[hH][tT][tT][pP]://.*/", ...);
            - pattern: |
                $RESTTEMP = new RestTemplate(...);
                ...
                String $URL = "=~/[hH][tT][tT][pP]://.*/";
                ...
                $RESTTEMP.$FUNC($URL, ...);
            - pattern: |
                $RESTTEMP = new RestTemplate(...);
                ...
                $URL = new URI(..., "=~/[hH][tT][tT][pP]://.*/", ...);
                ...
                $RESTTEMP.$FUNC($URL, ...);
        - metavariable-regex:
            metavariable: $FUNC
            regex: (delete|doExecute|exchange|getForEntity|getForObject|headForHeaders|optionsFo
rAllow|patchForObject|postForEntity|postForLocation|postForObject|put)
      severity: WARNING
    - id: problem-based-packs.insecure-transport.java-stdlib.bypass-tls-verification.bypass-tls-
verification
      languages:
        - java
      message: Checks for redefinitions of the checkServerTrusted function in the X509TrustManag
er class that disables TLS/SSL certificate verification. This should only be used for debugging
purposes because it leads to vulnerability to MTM attacks.
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://riptutorial.com/java/example/16517/temporarily-disable-ssl-verification--f
or-testing-purposes-
            - https://stackoverflow.com/questions/35530558/how-to-fix-unsafe-implementation-of-x
509trustmanager-in-android-app?rq=1
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
      patterns:
        - pattern: |
            new X509TrustManager() {
              ...
              public void checkClientTrusted(X509Certificate[] certs, String authType) {...}
              ...
            }
        - pattern-not: |
            new X509TrustManager() {
              ...
              public void checkServerTrusted(X509Certificate[] certs, String authType) {
                ...
                throw new CertificateException(...);
                ...
              }
              ...
            }
        - pattern-not: |
            new X509TrustManager() {
              ...
              public void checkServerTrusted(X509Certificate[] certs, String authType) {
                ...
                throw new IllegalArgumentException(...);
                ...
              }
              ...
            }
      severity: WARNING
    - id: problem-based-packs.insecure-transport.java-stdlib.disallow-old-tls-versions1.disallow
-old-tls-versions1
      languages:
        - java
      message: Detects direct creations of SSLConnectionSocketFactories that don't disallow SSL
v2, SSL v3, and TLS v1. SSLSocketFactory can be used to validate the identity of the HTTPS serve
r against a list of trusted certificates. These protocols are deprecated due to POODLE, man in t
he middle attacks, and other vulnerabilities.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: HIGH
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://stackoverflow.com/questions/26429751/java-http-clients-and-poodle
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
      patterns:
        - pattern: |
            new SSLConnectionSocketFactory(...);
        - pattern-not: |
            new SSLConnectionSocketFactory(..., new String[] {"TLSv1.2", "TLSv1.3"}, ...);
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-not: |
            new SSLConnectionSocketFactory(..., new String[] {"TLSv1.3", "TLSv1.2"}, ...);
        - pattern-not: |
            new SSLConnectionSocketFactory(..., new String[] {"TLSv1.3"}, ...);
        - pattern-not: |
            new SSLConnectionSocketFactory(..., new String[] {"TLSv1.2"}, ...);
        - pattern-not-inside: |
            (SSLConnectionSocketFactory $SF) = new SSLConnectionSocketFactory(...); ... (TlsConf
ig $TLSCONFIG) = TlsConfig.custom(). ... .setSupportedProtocols(TLS.V_1_2). ... .build(); ... Ht
tpClientConnectionManager cm = $CM.create(). ... .setSSLSocketFactory($SF). ... .setDefaultTlsCo
nfig($TLSCONFIG). ... .build();
        - pattern-not-inside: |
            (SSLConnectionSocketFactory $SF) = new SSLConnectionSocketFactory(...); ... (TlsConf
ig $TLSCONFIG) = TlsConfig.custom(). ... .setSupportedProtocols(TLS.V_1_3). ... .build(); ... Ht
tpClientConnectionManager cm = $CM.create(). ... .setSSLSocketFactory($SF). ... .setDefaultTlsCo
nfig($TLSCONFIG). ... .build();
      severity: WARNING
    - id: problem-based-packs.insecure-transport.java-stdlib.disallow-old-tls-versions2.disallow
-old-tls-versions2
      languages:
        - java
      message: Detects setting client protocols to insecure versions of TLS and SSL. These proto
cols are deprecated due to POODLE, man in the middle attacks, and other vulnerabilities.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://stackoverflow.com/questions/26504653/is-it-possible-to-disable-sslv3-for-a
ll-java-applications
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
      patterns:
        - pattern: $VALUE. ... .setProperty("jdk.tls.client.protocols", "$PATTERNS");
        - metavariable-pattern:
            language: generic
            metavariable: $PATTERNS
            patterns:
                - pattern-either:
                    - pattern: TLS1
                    - pattern-regex: ^(.*TLSv1|.*SSLv.*)$
                    - pattern-regex: ^(.*TLSv1,.*)
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[fF][tT][pP]://'
        replacement: sftp://
      id: problem-based-packs.insecure-transport.java-stdlib.ftp-request.ftp-request
      languages:
        - java
      message: Checks for outgoing connections to ftp servers. FTP does not encrypt traffic, pos
sibly leading to PII being sent plaintext over the network.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://www.codejava.net/java-se/ftp/connect-and-login-to-a-ftp-server
            - https://commons.apache.org/proper/commons-net/apidocs/org/apache/commons/net/ftp/F
TPClient.html
        subcategory:
            - vuln
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        technology:
            - java
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            FTPClient $FTPCLIENT = new FTPClient();
            ...
            $FTPCLIENT.connect(...);
        - pattern: |
            URL $URL = new URL("=~/^[fF][tT][pP]://.*/");
            ...
            URLConnection $CONN = $URL.openConnection(...);
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: problem-based-packs.insecure-transport.java-stdlib.http-components-request.http-compon
ents-request
      languages:
        - java
      message: Checks for requests sent via Apache HTTP Components to http:// URLS. This is dang
erous because the server is attempting to connect to a website that does not encrypt traffic wit
h TLS. Instead, send requests only to https:// URLS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://hc.apache.org/httpcomponents-client-ga/quickstart.html
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            $HTTPCLIENT = HttpClients.$CREATE(...);
            ...
            $HTTPREQ = new $HTTPFUNC("=~/[hH][tT][tT][pP]://.*/");
            ...
            $RESPONSE = $HTTPCLIENT.execute($HTTPREQ);
        - pattern: |
            $HTTPCLIENT = HttpClients.$CREATE(...);
            ...
            $RESPONSE = $HTTPCLIENT.execute(new $HTTPFUNC("=~/[hH][tT][tT][pP]://.*/"));
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: problem-based-packs.insecure-transport.java-stdlib.httpclient-http-request.httpclient-
http-request
      languages:
        - java
      message: Checks for requests sent via HttpClient to http:// URLS. This is dangerous becaus
e the server is attempting to connect to a website that does not encrypt traffic with TLS. Inste
ad, send requests only to https:// URLS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://openjdk.java.net/groups/net/httpclient/intro.html
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - vuln
        technology:
          - java
        vulnerability: Insecure Transport
      pattern-either:
        - patterns:
          - pattern: |
              URI.create("=~/[hH][tT][tT][pP]://.*/", ...)
          - pattern-inside: |
              HttpClient $CLIENT = ...;
              ...
              HttpRequest $REQ = ...;
              ...
              $CLIENT.sendAsync(...);
        - patterns:
          - pattern: |
              URI.create("=~/[hH][tT][tT][pP]://.*/", ...)
          - pattern-inside: |
              HttpClient $CLIENT = ...;
              ...
              HttpRequest $REQ = ...;
              ...
              $CLIENT.send(...);
        - patterns:
          - pattern: |
              URI.create($URI)
          - pattern-inside: |
              String $URI = "=~/[hH][tT][tT][pP]://.*/";
              ...
              HttpClient $CLIENT = ...;
              ...
              HttpRequest $REQ = ...;
              ...
              $CLIENT.send(...);
        - patterns:
          - pattern: |
              URI.create($URI)
          - pattern-inside: |
              String $URI = "=~/[hH][tT][tT][pP]://.*/";
              ...
              HttpClient $CLIENT = ...;
              ...
              HttpRequest $REQ = ...;
              ...
              $CLIENT.sendAsync(...);
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: problem-based-packs.insecure-transport.java-stdlib.httpget-http-request.httpget-http-r
equest
      languages:
        - java
      message: Detected an HTTP request sent via HttpGet. This could lead to sensitive informati
on being sent  over an insecure channel. Instead, it is recommended to send requests over HTTPS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
          - https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/net/URLConnectio
n.html
          - https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/net/URL.html#ope
nConnection()
        subcategory:
          - vuln
        technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - java
        vulnerability: Insecure Transport
      patterns:
        - pattern: |
            "=~/[Hh][Tt][Tt][Pp]://.*/"
        - pattern-inside: |
            $R = new HttpGet("=~/[Hh][Tt][Tt][Pp]://.*/");
            ...
            $CLIENT. ... .execute($R, ...);
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: problem-based-packs.insecure-transport.java-stdlib.httpurlconnection-http-request.http
urlconnection-http-request
      languages:
        - java
      message: Detected an HTTP request sent via HttpURLConnection. This could lead to sensitive
 information being sent over an insecure channel. Instead, it is recommended to send requests ov
er HTTPS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/net/URLConnectio
n.html
            - https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/net/URL.html#ope
nConnection()
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
      patterns:
        - pattern: |
            "=~/[Hh][Tt][Tt][Pp]://.*/"
        - pattern-either:
            - pattern-inside: |
                URL $URL = new URL ("=~/[Hh][Tt][Tt][Pp]://.*/", ...);
                ...
                $CON = (HttpURLConnection) $URL.openConnection(...);
                ...
                $CON.$FUNC(...);
            - pattern-inside: |
                URL $URL = new URL ("=~/[Hh][Tt][Tt][Pp]://.*/", ...);
                ...
                $CON = $URL.openConnection(...);
                ...
                $CON.$FUNC(...);
      severity: WARNING
    - id: problem-based-packs.insecure-transport.java-stdlib.telnet-request.telnet-request
      languages:
        - java
      message: Checks for attempts to connect through telnet. This is insecure as the telnet pro
tocol supports no encryption, and data passes through unencrypted.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://commons.apache.org/proper/commons-net/javadocs/api-3.6/org/apache/commons/
net/telnet/TelnetClient.html
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
      pattern: |
        $TELNETCLIENT = new TelnetClient(...);
        ...
        $TELNETCLIENT.connect(...);
      severity: WARNING
    - id: problem-based-packs.insecure-transport.java-stdlib.tls-renegotiation.tls-renegotiation
      languages:
        - java
      message: Checks for cases where java applications are allowing unsafe renegotiation. This
leaves the application vulnerable to a man-in-the-middle attack where chosen plain text is injec
ted as prefix to a TLS connection.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: LOW
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://www.oracle.com/java/technologies/javase/tlsreadme.html
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
      pattern: |
        java.lang.System.setProperty("sun.security.ssl.allowUnsafeRenegotiation", true);
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: problem-based-packs.insecure-transport.java-stdlib.unirest-http-request.unirest-http-r
equest
      languages:
        - java
      message: Checks for requests sent via Unirest to http:// URLS. This is dangerous because t
he server is attempting to connect to a website that does not encrypt traffic with TLS. Instead,
 send requests only to https:// URLS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://kong.github.io/unirest-java/#requests
        subcategory:
            - vuln
        technology:
            - unirest
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            Unirest.get("=~/[hH][tT][tT][pP]://.*/")
        - pattern: |
            Unirest.post("=~/[hH][tT][tT][pP]://.*/")
      severity: WARNING
    - id: problem-based-packs.insecure-transport.js-node.bypass-tls-verification.bypass-tls-veri
fication
      languages:
        - javascript
        - typescript
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Checks for setting the environment variable NODE_TLS_REJECT_UNAUTHORIZED to 0, wh
ich disables TLS verification. This should only be used for debugging purposes. Setting the opti
on rejectUnauthorized to false bypasses verification against the list of trusted CAs, which also
 leads to insecure transport. These options lead to vulnerability to MTM attacks, and should not
 be used.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://nodejs.org/api/https.html#https_https_request_options_callback
            - https://stackoverflow.com/questions/20433287/node-js-request-cert-has-expired#answ
er-29397100
        subcategory:
            - vuln
        technology:
            - node.js
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            process.env["NODE_TLS_REJECT_UNAUTHORIZED"] = 0;
        - pattern: |
            {rejectUnauthorized:false}
      severity: WARNING
    - id: problem-based-packs.insecure-transport.js-node.disallow-old-tls-versions1.disallow-old
-tls-versions1
      languages:
        - javascript
        - typescript
      message: Detects direct creations of $HTTPS servers that don't disallow SSL v2, SSL v3, an
d TLS v1. These protocols are deprecated due to POODLE, man in the middle attacks, and other vul
nerabilities.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://us-cert.cisa.gov/ncas/alerts/TA14-290A
            - https://stackoverflow.com/questions/40434934/how-to-disable-the-ssl-3-0-and-tls-1-
0-in-nodejs
            - https://nodejs.org/api/https.html#https_https_createserver_options_requestlistener
        subcategory:
            - vuln
        technology:
            - node.js
        vulnerability: Insecure Transport
      patterns:
        - pattern-either:
            - pattern-inside: |
                $CONST = require('crypto');
                ...
            - pattern-inside: |
                $CONST = require('constants');
                ...
        - pattern-inside: |
            $HTTPS = require('https');
            ...
        - pattern: |
            $HTTPS.createServer(...).$FUNC(...);
        - pattern-not: |
            $HTTPS.createServer({secureOptions: $CONST.SSL_OP_NO_TLSv1 | $CONST.SSL_OP_NO_SSLv3
| $CONST.SSL_OP_NO_SSLv2 }, ...).$FUNC(...);
        - pattern-not: |
            $HTTPS.createServer({secureOptions: $CONST.SSL_OP_NO_TLSv1 | $CONST.SSL_OP_NO_SSLv2
|$CONST.SSL_OP_NO_SSLv3 }, ...).$FUNC(...);
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-not: |
            $HTTPS.createServer({secureOptions: $CONST.SSL_OP_NO_SSLv2 |$CONST.SSL_OP_NO_SSLv3 |
$CONST.SSL_OP_NO_TLSv1 }, ...).$FUNC(...);
        - pattern-not: |
            $HTTPS.createServer({secureOptions: $CONST.SSL_OP_NO_SSLv2 |$CONST.SSL_OP_NO_TLSv1 |
 $CONST.SSL_OP_NO_SSLv3}, ...).$FUNC(...);
        - pattern-not: |
            $HTTPS.createServer({secureOptions:$CONST.SSL_OP_NO_SSLv3 | $CONST.SSL_OP_NO_SSLv2 |
$CONST.SSL_OP_NO_TLSv1}, ...).$FUNC(...);
        - pattern-not: |
            $HTTPS.createServer({secureOptions:$CONST.SSL_OP_NO_SSLv3 | $CONST.SSL_OP_NO_TLSv1|
$CONST.SSL_OP_NO_SSLv2}, ...).$FUNC(...);
      severity: WARNING
    - id: problem-based-packs.insecure-transport.js-node.disallow-old-tls-versions2.disallow-old
-tls-versions2
      languages:
        - javascript
        - typescript
      message: Detects creations of $HTTPS servers from option objects that don't disallow SSL v
2, SSL v3, and TLS v1. These protocols are deprecated due to POODLE, man in the middle attacks,
and other vulnerabilities.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://us-cert.cisa.gov/ncas/alerts/TA14-290A
            - https://stackoverflow.com/questions/40434934/how-to-disable-the-ssl-3-0-and-tls-1-
0-in-nodejs
            - https://nodejs.org/api/https.html#https_https_createserver_options_requestlistener
        subcategory:
            - vuln
        technology:
            - node.js
        vulnerability: Insecure Transport
      patterns:
        - pattern-either:
            - pattern-inside: |
                $CONST = require('crypto');
                ...
            - pattern-inside: |
                $CONST = require('constants');
                ...
        - pattern-inside: |
            $HTTPS = require('https');
            ...
        - pattern: |
            $OPTIONS = {};
            ...
            $HTTPS.createServer($OPTIONS, ...);
        - pattern-not: |
            $OPTIONS = {secureOptions: $CONST.SSL_OP_NO_TLSv1 | $CONST.SSL_OP_NO_SSLv3 | $CONST.
SSL_OP_NO_SSLv2};
            ...
            $HTTPS.createServer($OPTIONS, ...);
        - pattern-not: |
            $OPTIONS = {secureOptions: $CONST.SSL_OP_NO_TLSv1 | $CONST.SSL_OP_NO_SSLv2 | $CONST.
SSL_OP_NO_SSLv3};
            ...
            $HTTPS.createServer($OPTIONS, ...);
        - pattern-not: |
            $OPTIONS = {secureOptions: $CONST.SSL_OP_NO_SSLv2  | $CONST.SSL_OP_NO_TLSv1 | $CONST
.SSL_OP_NO_SSLv3};
            ...
            $HTTPS.createServer($OPTIONS, ...);
        - pattern-not: |
            $OPTIONS = {secureOptions: $CONST.SSL_OP_NO_SSLv2 | $CONST.SSL_OP_NO_SSLv3 | $CONST.
SSL_OP_NO_TLSv1};
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
            $HTTPS.createServer($OPTIONS, ...);
        - pattern-not: |
            $OPTIONS = {secureOptions: $CONST.SSL_OP_NO_SSLv3 | $CONST.SSL_OP_NO_SSLv2 | $CONST.
SSL_OP_NO_TLSv1};
            ...
            $HTTPS.createServer($OPTIONS, ...);
        - pattern-not: |
            $OPTIONS = {secureOptions: $CONST.SSL_OP_NO_SSLv3 | $CONST.SSL_OP_NO_TLSv1 | $CONST.
SSL_OP_NO_SSLv2};
            ...
            $HTTPS.createServer($OPTIONS, ...);
      severity: WARNING
    - id: problem-based-packs.insecure-transport.js-node.ftp-request.ftp-request
      languages:
        - javascript
        - typescript
      message: 'Checks for lack of usage of the "secure: true" option when sending ftp requests
through the nodejs ftp module. This leads to unencrypted traffic being sent to the ftp server. T
here are other options such as "implicit" that still does not encrypt all traffic. ftp is the mo
st utilized npm ftp module.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://www.npmjs.com/package/ftp
            - https://openbase.io/js/ftp
        subcategory:
            - vuln
        technology:
            - node.js
        vulnerability: Insecure Transport
      patterns:
        - pattern-inside: |
            $X = require('ftp');
            ...
            $C = new $X();
            ...
        - pattern-not-inside: |
            $OPTIONS = {secure: true};
            ...
        - pattern: |
            $C.connect($OPTIONS,...);
        - pattern-not: |
            $C.connect({...,secure: true});
      severity: WARNING
    - id: problem-based-packs.insecure-transport.js-node.http-request.http-request
      languages:
        - javascript
      message: Checks for requests sent to http:// URLs. This is dangerous as the server is atte
mpting to connect to a website that does not encrypt traffic with TLS. Instead, only send reques
ts to https:// URLs.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://nodejs.org/api/http.html#http_http_request_options_callback
        subcategory:
            - vuln
        technology:
            - node.js
        vulnerability: Insecure Transport
      patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-inside: |
            $HTTP = require('http');
            ...
        - pattern-either:
            - pattern: |
                $HTTP.request("=~/http://.*/",...);
            - pattern: |
                $HTTP.get("=~/http://.*/", ...)
            - pattern: |
                $VAR = new URL("=~/http://.*/");
                ...
                $HTTP.request($VAR, ...);
            - pattern: |
                $VAR = {...,hostname: "..."};
                ...
                $HTTP.request(..., $VAR, ...);
            - pattern: |
                $HTTP.request(..., {...,hostname: "..."}, ...);
        - pattern-not: |
            $VAR = {...,protocol: "https"};
            ...
            $HTTP.request(..., $VAR, ...);
        - pattern-not: |
            $HTTP.request(..., {...,protocol: "https"}, ...);
      severity: WARNING
    - id: problem-based-packs.insecure-transport.js-node.rest-http-client-support.rest-http-clie
nt-support
      languages:
        - javascript
      message: Checks for requests to http (unencrypted) sites using some of node js's most popu
lar REST/HTTP libraries, including node-rest-client, axios, and got.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://www.npmjs.com/package/axios
            - https://www.npmjs.com/package/got
            - https://www.npmjs.com/package/node-rest-client
        subcategory:
            - vuln
        technology:
            - node.js
        vulnerability: Insecure Transport
      patterns:
        - pattern-either:
            - pattern-inside: |
                $CLIENT = require('node-rest-client').Client;
                ...
                $C = new $CLIENT();
                ...
            - pattern-inside: |
                $C = require('axios');
                ...
            - pattern-inside: |
                $C = require('got');
                ...
        - pattern-either:
            - pattern: |
                $C.$REQ("=~/http://.*/", ...)
            - pattern: |
                $C("=~/http://.*/", ...)
            - pattern: |
                $C({...,url: "=~/http://.*/"})
            - pattern: |
                $C.$REQ({...,url: "=~/http://.*/"})
      severity: WARNING
    - id: problem-based-packs.insecure-transport.js-node.telnet-request.telnet-request
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      languages:
        - javascript
      message: Checks for creation of telnet servers or attempts to connect through telnet. This
 is insecure as the telnet protocol supports no encryption, and data passes through unencrypted.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://www.npmjs.com/package/telnet
            - https://www.npmjs.com/package/telnet-client
        subcategory:
            - vuln
        technology:
            - node.js
        vulnerability: Insecure Transport
      patterns:
        - pattern-either:
            - pattern-inside: |
                $TEL = require('telnet-client');
                ...
                $SERVER = new $TEL();
                ...
            - pattern-inside: |
                $SERVER = require('telnet');
                ...
        - pattern-either:
            - pattern: |
                $SERVER.on(...)
            - pattern: |
                $SERVER.connect(...)
            - pattern: |
                $SERVER.createServer(...)
      severity: WARNING
    - id: problem-based-packs.insecure-transport.ruby-stdlib.http-client-requests.http-client-re
quests
      languages:
        - ruby
      message: Checks for requests to http (unencrypted) sites using some of ruby's most popular
 REST/HTTP libraries, including httparty and restclient.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://github.com/rest-client/rest-client
            - https://github.com/jnunemaker/httparty/tree/master/docs
        subcategory:
            - vuln
        technology:
            - httparty
            - rest-client
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            HTTParty.$PARTYVERB("=~/[hH][tT][tT][pP]://.*/", ...)
        - pattern: |
            $STRING = "=~/[hH][tT][tT][pP]://.*/"
            ...
            HTTParty.$PARTYVERB($STRING, ...)
        - pattern: |
            RestClient.$RESTVERB "=~/[hH][tT][tT][pP]://.*/", ...
        - pattern: |
            RestClient::Request.execute(..., url: "=~/[hH][tT][tT][pP]://.*/", ...)
      severity: WARNING
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: problem-based-packs.insecure-transport.ruby-stdlib.net-ftp-request.net-ftp-request
      languages:
        - ruby
      message: Checks for outgoing connections to ftp servers with the 'net/ftp' package. FTP do
es not encrypt traffic, possibly leading to PII being sent plaintext over the network. Instead,
connect via the SFTP protocol.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://docs.ruby-lang.org/en/2.0.0/Net/FTP.html
        subcategory:
            - vuln
        technology:
            - ruby
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            $FTP = Net::FTP.new('...')
            ...
            $FTP.login
        - pattern: |
            Net::FTP.open('...') do |ftp|
              ...
              ftp.login
            end
      severity: WARNING
    - id: problem-based-packs.insecure-transport.ruby-stdlib.net-http-request.net-http-request
      languages:
        - ruby
      message: Checks for requests sent to http:// URLs. This is dangerous as the server is atte
mpting to connect to a website that does not encrypt traffic with TLS. Instead, only send reques
ts to https:// URLs.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://ruby-doc.org/stdlib-2.6.5/libdoc/net/http/rdoc/Net/
        subcategory:
            - vuln
        technology:
            - ruby
        vulnerability: Insecure Transport
      patterns:
        - pattern-either:
            - pattern: |
                $URI = URI('=~/[hH][tT][tT][pP]://.*/')
                ...
                Net::HTTP::$FUNC.new $URI
            - pattern: |
                $URI = URI('=~/[hH][tT][tT][pP]://.*/')
                ...
                Net::HTTP.$FUNC($URI, ...)
            - pattern: |
                Net::HTTP.$FUNC(URI('=~/[hH][tT][tT][pP]://.*/'), ...)
        - metavariable-regex:
            metavariable: $FUNC
            regex: ([gG]et|post_form|[pP]ost|get_response|get_print|Head|Patch|Put|Proppatch|Loc
k|Unlock|Options|Propfind|Delete|Move|Copy|Trace|Mkcol)
      severity: WARNING
    - id: problem-based-packs.insecure-transport.ruby-stdlib.net-telnet-request.net-telnet-reque
st
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - ruby
      message: Checks for creation of telnet servers or attempts to connect through telnet. This
 is insecure as the telnet protocol supports no encryption, and data passes through unencrypted.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://docs.ruby-lang.org/en/2.2.0/Net/Telnet.html
            - https://www.rubydoc.info/gems/net-ssh-telnet2/0.1.0/Net/SSH/Telnet
        subcategory:
            - vuln
        technology:
            - ruby
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            Net::Telnet::new(...)
        - pattern: |
            Net::SSH::Telnet.new(...)
      severity: WARNING
    - id: problem-based-packs.insecure-transport.ruby-stdlib.openuri-request.openuri-request
      languages:
        - ruby
      message: Checks for requests to http and ftp (unencrypted) sites using OpenURI.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A03:2017 - Sensitive Data Exposure
        references:
            - https://ruby-doc.org/stdlib-2.6.3/libdoc/open-uri/rdoc/OpenURI.html
        subcategory:
            - vuln
        technology:
            - open-uri
        vulnerability: Insecure Transport
      pattern-either:
        - pattern: |
            URI.open('=~/[hH][tT][tT][pP]://.*/', ...)
        - pattern: |
            $URI = URI.parse('=~/[hH][tT][tT][pP]://.*/', ...)
            ...
            $URI.open
        - pattern: |
            URI.open('=~/^[fF][tT][pP]://.*/', ...)
        - pattern: |
            $URI = URI.parse('=~/^[fF][tT][pP]://.*/', ...)
            ...
            $URI.open
      severity: WARNING
    - id: python.aws-lambda.security.dangerous-asyncio-create-exec.dangerous-asyncio-create-exec
      languages:
        - python
      message: Detected 'create_subprocess_exec' function with argument tainted by `event` objec
t. If this data can be controlled by a malicious actor, it may be an instance of command injecti
on. Audit the use of this call to ensure it is not controllable by an external resource. You may
 consider using 'shlex.escape()'.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.python.org/3/library/asyncio-subprocess.html#asyncio.create_subproces
s_exec
            - https://docs.python.org/3/library/shlex.html
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $CMD
            - pattern-either:
                - pattern: asyncio.create_subprocess_exec($PROG, $CMD, ...)
                - pattern: asyncio.create_subprocess_exec($PROG, [$CMD, ...], ...)
                - pattern: asyncio.subprocess.create_subprocess_exec($PROG, $CMD, ...)
                - pattern: asyncio.subprocess.create_subprocess_exec($PROG, [$CMD, ...], ...)
                - pattern: asyncio.create_subprocess_exec($PROG, "=~/(sh|bash|ksh|csh|tcsh|zsh)/
", "-c", $CMD, ...)
                - pattern: asyncio.create_subprocess_exec($PROG, ["=~/(sh|bash|ksh|csh|tcsh|zsh)
/", "-c", $CMD, ...], ...)
                - pattern: asyncio.subprocess.create_subprocess_exec($PROG, "=~/(sh|bash|ksh|csh
|tcsh|zsh)/", "-c", $CMD, ...)
                - pattern: asyncio.subprocess.create_subprocess_exec($PROG, ["=~/(sh|bash|ksh|cs
h|tcsh|zsh)/", "-c", $CMD, ...], ...)
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: ERROR
    - id: python.aws-lambda.security.dangerous-asyncio-exec.dangerous-asyncio-exec
      languages:
        - python
      message: Detected subprocess function '$LOOP.subprocess_exec' with argument tainted by `ev
ent` object. If this data can be controlled by a malicious actor, it may be an instance of comma
nd injection. Audit the use of this call to ensure it is not controllable by an external resourc
e. You may consider using 'shlex.escape()'.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        references:
            - https://docs.python.org/3/library/asyncio-eventloop.html#asyncio.loop.subprocess_e
xec
            - https://docs.python.org/3/library/shlex.html
        subcategory:
            - vuln
        technology:
            - python
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $CMD
            - pattern-either:
                - pattern: $LOOP.subprocess_exec($PROTOCOL, $CMD, ...)
                - pattern: $LOOP.subprocess_exec($PROTOCOL, [$CMD, ...], ...)
                - pattern: $LOOP.subprocess_exec($PROTOCOL, "=~/(sh|bash|ksh|csh|tcsh|zsh)/", "-
c", $CMD, ...)
                - pattern: $LOOP.subprocess_exec($PROTOCOL, ["=~/(sh|bash|ksh|csh|tcsh|zsh)/", "
-c", $CMD, ...], ...)
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: ERROR
    - id: python.aws-lambda.security.dangerous-asyncio-shell.dangerous-asyncio-shell
      languages:
        - python
      message: Detected asyncio subprocess function with argument tainted by `event` object. If
this data can be controlled by a malicious actor, it may be an instance of command injection. Au
dit the use of this call to ensure it is not controllable by an external resource. You may consi
der using 'shlex.escape()'.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.python.org/3/library/asyncio-subprocess.html
            - https://docs.python.org/3/library/shlex.html
        subcategory:
            - vuln
        technology:
            - python
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $CMD
            - pattern-either:
                - pattern: $LOOP.subprocess_shell($PROTOCOL, $CMD)
                - pattern: asyncio.subprocess.create_subprocess_shell($CMD, ...)
                - pattern: asyncio.create_subprocess_shell($CMD, ...)
      pattern-sources:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: ERROR
    - id: python.aws-lambda.security.dangerous-spawn-process.dangerous-spawn-process
      languages:
        - python
      message: Detected `os` function with argument tainted by `event` object. This is dangerous
 if external data can reach this function call because it allows a malicious actor to execute co
mmands. Ensure no external data reaches here.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/b605_start_process_with
_a_shell.html
        subcategory:
            - vuln
        technology:
            - python
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $CMD
            - pattern-either:
                - patterns:
                    - pattern: os.$METHOD($MODE, $CMD, ...)
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: (spawnl|spawnle|spawnlp|spawnlpe|spawnv|spawnve|spawnvp|spawnvp|s
pawnvpe|posix_spawn|posix_spawnp|startfile)
                - patterns:
                    - pattern-inside: os.$METHOD($MODE, $BASH, ["-c", $CMD,...],...)
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: (spawnv|spawnve|spawnvp|spawnvp|spawnvpe|posix_spawn|posix_spawnp
)
                    - metavariable-regex:
                        metavariable: $BASH
                        regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
                - patterns:
                    - pattern-inside: os.$METHOD($MODE, $BASH, "-c", $CMD,...)
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: (spawnl|spawnle|spawnlp|spawnlpe)
                    - metavariable-regex:
                        metavariable: $BASH
                        regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
      pattern-sources:
        - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: ERROR
    - id: python.aws-lambda.security.dangerous-subprocess-use.dangerous-subprocess-use
      languages:
        - python
      message: Detected subprocess function with argument tainted by an `event` object.  If this
 data can be controlled by a malicious actor, it may be an instance of command injection. The de
fault option for `shell` is False, and this is secure by default. Consider removing the `shell=T
rue` or setting it to False explicitely. Using `shell=False` means you have to split the command
 string into an array of strings for the command and its arguments. You may consider using 'shle
x.split()' for this purpose.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.python.org/3/library/subprocess.html
            - https://docs.python.org/3/library/shlex.html
        subcategory:
            - vuln
        technology:
            - python
            - aws-lambda
      mode: taint
      pattern-sanitizers:
        - pattern: shlex.split(...)
        - pattern: pipes.quote(...)
        - pattern: shlex.quote(...)
      pattern-sinks:
        - patterns:
            - pattern: subprocess.$FUNC(..., shell=True, ...)
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: ERROR
    - id: python.aws-lambda.security.dangerous-system-call.dangerous-system-call
      languages:
        - python
      message: Detected `os` function with argument tainted by `event` object. This is dangerous
 if external data can reach this function call because it allows a malicious actor to execute co
mmands. Use the 'subprocess' module instead, which is easier to use without accidentally exposin
g a command injection vulnerability.
      metadata:
        asvs:
            control_id: 5.2.4 Dyanmic Code Execution Features
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v52-sanitization-and-sandboxing-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/b605_start_process_with
_a_shell.html
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $CMD
            - pattern-either:
                - pattern: os.system($CMD,...)
                - pattern: os.popen($CMD,...)
                - pattern: os.popen2($CMD,...)
                - pattern: os.popen3($CMD,...)
                - pattern: os.popen4($CMD,...)
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: ERROR
    - id: python.aws-lambda.security.dynamodb-filter-injection.dynamodb-filter-injection
      languages:
        - python
      message: Detected DynamoDB query filter that is tainted by `$EVENT` object. This could lea
d to NoSQL injection if the variable is user-controlled and not properly sanitized. Explicitly a
ssign query params instead of passing data from `$EVENT` directly to DynamoDB client.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-943: Improper Neutralization of Special Elements in Data Query Logic'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
        references:
            - https://medium.com/appsecengineer/dynamodb-injection-1db99c2454ac
        subcategory:
            - vuln
        technology:
            - python
            - boto3
            - aws-lambda
            - dynamodb
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern: |
                {...}
      pattern-sinks:
        - patterns:
            - focus-metavariable: $SINK
            - pattern-either:
                - pattern: $TABLE.scan(..., ScanFilter = $SINK, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: $TABLE.query(..., QueryFilter = $SINK, ...)
            - pattern-either:
                - patterns:
                    - pattern-inside: |
                        $TABLE = $DB.Table(...)
                        ...
                    - pattern-inside: |
                        $DB = boto3.resource('dynamodb', ...)
                        ...
                - pattern-inside: |
                    $TABLE = boto3.client('dynamodb', ...)
                    ...
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: ERROR
    - id: python.aws-lambda.security.mysql-sqli.mysql-sqli
      languages:
        - python
      message: 'Detected SQL statement that is tainted by `event` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use parameterized
 statements like so: `cursor.execute(''SELECT * FROM projects WHERE status = %s'', (''active''))
`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqlcursor-exe
cute.html
            - https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqlcursor-exe
cutemany.html
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - mysql
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $QUERY
            - pattern-either:
                - pattern: $CURSOR.execute($QUERY,...)
                - pattern: $CURSOR.executemany($QUERY,...)
            - pattern-either:
                - pattern-inside: |
                    import mysql
                    ...
                - pattern-inside: |
                    import mysql.cursors
                    ...
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: WARNING
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: python.aws-lambda.security.psycopg-sqli.psycopg-sqli
      languages:
        - python
      message: 'Detected SQL statement that is tainted by `event` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use parameterized
 statements like so: `cursor.execute(''SELECT * FROM projects WHERE status = %s'', ''active'')`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://www.psycopg.org/docs/cursor.html#cursor.execute
            - https://www.psycopg.org/docs/cursor.html#cursor.executemany
            - https://www.psycopg.org/docs/cursor.html#cursor.mogrify
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - psycopg
            - psycopg2
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $QUERY
            - pattern-either:
                - pattern: $CURSOR.execute($QUERY,...)
                - pattern: $CURSOR.executemany($QUERY,...)
                - pattern: $CURSOR.mogrify($QUERY,...)
            - pattern-inside: |
                import psycopg2
                ...
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: WARNING
    - id: python.aws-lambda.security.pymssql-sqli.pymssql-sqli
      languages:
        - python
      message: 'Detected SQL statement that is tainted by `event` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use parameterized
 statements like so: `cursor.execute(''SELECT * FROM projects WHERE status = %s'', ''active'')`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://pypi.org/project/pymssql/
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - vuln
        technology:
            - aws-lambda
            - pymssql
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $QUERY
            - pattern: $CURSOR.execute($QUERY,...)
            - pattern-inside: |
                import pymssql
                ...
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: WARNING
    - id: python.aws-lambda.security.pymysql-sqli.pymysql-sqli
      languages:
        - python
      message: 'Detected SQL statement that is tainted by `event` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use parameterized
 statements like so: `cursor.execute(''SELECT * FROM projects WHERE status = %s'', (''active''))
`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://pypi.org/project/PyMySQL/#id4
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - pymysql
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $QUERY
            - pattern: $CURSOR.execute($QUERY,...)
            - pattern-either:
                - pattern-inside: |
                    import pymysql
                    ...
                - pattern-inside: |
                    import pymysql.cursors
                    ...
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: WARNING
    - id: python.aws-lambda.security.sqlalchemy-sqli.sqlalchemy-sqli
      languages:
        - python
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: 'Detected SQL statement that is tainted by `event` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use parameterized
 statements like so: `cursor.execute(''SELECT * FROM projects WHERE status = ?'', ''active'')`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.sqlalchemy.org/en/14/core/connections.html#sqlalchemy.engine.Connecti
on.execute
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - sqlalchemy
      mode: taint
      pattern-sinks:
        - patterns:
            - focus-metavariable: $QUERY
            - pattern: $CURSOR.execute($QUERY,...)
            - pattern-inside: |
                import sqlalchemy
                ...
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: WARNING
    - id: python.aws-lambda.security.tainted-code-exec.tainted-code-exec
      languages:
        - python
      message: Detected the use of `exec/eval`.This can be dangerous if used to evaluate dynamic
 content. If this content can be input from outside the program, this may be a code injection vu
lnerability. Ensure evaluated content is not definable by external sources.
      metadata:
        asvs:
            control_id: 5.2.4 Dyanmic Code Execution Features
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v52-sanitization-and-sandboxing-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      pattern-sinks:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - patterns:
            - pattern-either:
                - pattern: eval($CODE, ...)
                - pattern: exec($CODE, ...)
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: WARNING
    - id: python.aws-lambda.security.tainted-html-response.tainted-html-response
      languages:
        - python
      message: Detected user input flowing into an HTML response. You may be accidentally bypass
ing secure methods of rendering HTML by manually constructing HTML and this could create a cross
-site scripting vulnerability, which could let attackers steal sensitive user data.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: $BODY
            - pattern-inside: |
                {..., "headers": {..., "Content-Type": "text/html", ...}, "body": $BODY, ... }
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: WARNING
    - id: python.aws-lambda.security.tainted-html-string.tainted-html-string
      languages:
        - python
      message: Detected user input flowing into a manually constructed HTML string. You may be a
ccidentally bypassing secure methods of rendering HTML by manually constructing HTML and this co
uld create a cross-site scripting vulnerability, which could let attackers steal sensitive user
data. To be sure this is safe, check that the HTML is rendered safely. Otherwise, use templates
which will safely render HTML instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://owasp.org/Top10/A03_2021-Injection
          subcategory:
              - vuln
          technology:
              - aws-lambda
        mode: taint
        pattern-sinks:
          - patterns:
              - pattern-either:
                  - patterns:
                      - pattern-either:
                          - pattern: '"$HTMLSTR" % ...'
                          - pattern: '"$HTMLSTR".format(...)'
                          - pattern: '"$HTMLSTR" + ...'
                          - pattern: f"$HTMLSTR{...}..."
                  - patterns:
                      - pattern-inside: |
                          $HTML = "$HTMLSTR"
                          ...
                      - pattern-either:
                          - pattern: $HTML % ...
                          - pattern: $HTML.format(...)
                          - pattern: $HTML + ...
              - metavariable-pattern:
                  language: generic
                  metavariable: $HTMLSTR
                  pattern: <$TAG ...
              - pattern-not-inside: |
                  print(...)
        pattern-sources:
          - patterns:
              - pattern: event
              - pattern-inside: |
                  def $HANDLER(event, context):
                    ...
        severity: WARNING
      - id: python.aws-lambda.security.tainted-pickle-deserialization.tainted-pickle-deserializati
on
        languages:
          - python
        message: Avoid using `pickle`, which is known to lead to code execution vulnerabilities. W
hen unpickling, the serialized data could be manipulated to run arbitrary code. Instead, conside
r serializing the relevant data as JSON or a similar text-based serialization format.
        metadata:
          category: security
          confidence: MEDIUM
          cwe:
              - 'CWE-502: Deserialization of Untrusted Data'
          cwe2021-top25: true
          cwe2022-top25: true
          impact: MEDIUM
          likelihood: MEDIUM
          owasp:
              - A08:2017 - Insecure Deserialization
              - A08:2021 - Software and Data Integrity Failures
          references:
              - https://docs.python.org/3/library/pickle.html
              - https://davidhamann.de/2020/04/05/exploiting-python-pickle/
          subcategory:
              - vuln
          technology:
              - python
              - aws-lambda
        mode: taint
        pattern-sinks:
          - patterns:
              - focus-metavariable: $SINK
              - pattern-either:
                  - pattern: pickle.load($SINK,...)
                  - pattern: pickle.loads($SINK,...)
                  - pattern: _pickle.load($SINK,...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern: _pickle.loads($SINK,...)
                - pattern: cPickle.load($SINK,...)
                - pattern: cPickle.loads($SINK,...)
                - pattern: dill.load($SINK,...)
                - pattern: dill.loads($SINK,...)
                - pattern: shelve.open($SINK,...)
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: WARNING
    - id: python.aws-lambda.security.tainted-sql-string.tainted-sql-string
      languages:
        - python
      message: Detected user input used to manually construct a SQL string. This is usually bad
practice because manual construction could accidentally result in a SQL injection. An attacker c
ould use a SQL injection to steal or modify contents of the database. Instead, use a parameteriz
ed query which is available by default in most database engines. Alternatively, consider using a
n object-relational mapper (ORM) such as Sequelize which will protect your queries.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/SQL_Injection
        subcategory:
            - vuln
        technology:
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    "$SQLSTR" + ...
                - pattern: |
                    "$SQLSTR" % ...
                - pattern: |
                    "$SQLSTR".format(...)
                - pattern: |
                    f"$SQLSTR{...}..."
            - metavariable-regex:
                metavariable: $SQLSTR
                regex: \s*(?i)(select|delete|insert|create|update|alter|drop)\b.*=
            - pattern-not-inside: |
                print(...)
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context):
                  ...
      severity: ERROR
    - id: python.boto3.security.hardcoded-token.hardcoded-token
      languages:
        - python
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
            - https://bento.dev/checks/boto3/hardcoded-access-token/
            - https://aws.amazon.com/blogs/security/what-to-do-if-you-inadvertently-expose-an-aw
s-access-key/
        subcategory:
            - vuln
        technology:
            - boto3
            - secrets
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: $W(...,$TOKEN="$VALUE",...)
                - pattern: $BOTO. ... .$W(...,$TOKEN="$VALUE",...)
            - metavariable-regex:
                metavariable: $TOKEN
                regex: (aws_session_token|aws_access_key_id|aws_secret_access_key)
            - metavariable-pattern:
                language: generic
                metavariable: $VALUE
                patterns:
                    - pattern-either:
                        - pattern-regex: ^AKI
                        - pattern-regex: ^[A-Za-z0-9/+=]+$
            - metavariable-analysis:
                analyzer: entropy
                metavariable: $VALUE
      pattern-sources:
        - pattern: |
            "..."
      severity: WARNING
    - id: python.cryptography.security.empty-aes-key.empty-aes-key
      languages:
        - python
      message: Potential empty AES encryption key. Using an empty key in AES encryption can resu
lt in weak encryption and may allow attackers to easily decrypt sensitive data. Ensure that a st
rong, non-empty key is used for AES encryption.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
            - 'CWE-310: Cryptographic Issues'
        functional-categories:
            - crypto::search::key-length::pycrypto
            - crypto::search::key-length::pycryptodome
        impact: HIGH
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp: A6:2017 misconfiguration
        references:
            - https://cwe.mitre.org/data/definitions/327.html
            - https://cwe.mitre.org/data/definitions/310.html
        subcategory:
            - vuln
        technology:
            - python
            - pycrypto
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pycryptodome
      patterns:
        - pattern: AES.new("",...)
      severity: WARNING
    - fix: AES
      id: python.cryptography.security.insecure-cipher-algorithms-arc4.insecure-cipher-algorithm
-arc4
      languages:
        - python
      message: ARC4 (Alleged RC4) is a stream cipher with serious weaknesses in its initial stre
am output.  Its use is strongly discouraged. ARC4 does not use mode constructions. Use a strong
symmetric cipher such as EAS instead. With the `cryptography` package it is recommended to use t
he `Fernet` which is a secure implementation of AES in CBC mode with a 128-bit key.  Alternative
ly, keep using the `Cipher` class from the hazmat primitives but use the AES algorithm instead.
      metadata:
        bandit-code: B304
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::symmetric-algorithm::cryptography
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/#weak-cip
hers
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L98
        subcategory:
            - vuln
        technology:
            - cryptography
      patterns:
        - pattern: cryptography.hazmat.primitives.ciphers.algorithms.$ARC4($KEY)
        - pattern-inside: cryptography.hazmat.primitives.ciphers.Cipher(...)
        - metavariable-regex:
            metavariable: $ARC4
            regex: ^(ARC4)$
        - focus-metavariable: $ARC4
      severity: WARNING
    - fix: AES
      id: python.cryptography.security.insecure-cipher-algorithms-blowfish.insecure-cipher-algor
ithm-blowfish
      languages:
        - python
      message: Blowfish is a block cipher developed by Bruce Schneier. It is known to be suscept
ible to attacks when using weak keys.  The author has recommended that users of Blowfish move to
 newer algorithms such as AES. With the `cryptography` package it is recommended to use `Fernet`
 which is a secure implementation of AES in CBC mode with a 128-bit key.  Alternatively, keep us
ing the `Cipher` class from the hazmat primitives but use the AES algorithm instead.
      metadata:
        bandit-code: B304
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::symmetric-algorithm::cryptography
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/#weak-cip
hers
            - https://tools.ietf.org/html/rfc5469
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L98
        subcategory:
            - vuln
        technology:
            - cryptography
      patterns:
        - pattern: cryptography.hazmat.primitives.ciphers.algorithms.$BLOWFISH($KEY)
        - metavariable-regex:
            metavariable: $BLOWFISH
            regex: ^(Blowfish)$
        - focus-metavariable: $BLOWFISH
      severity: WARNING
    - fix: AES
      id: python.cryptography.security.insecure-cipher-algorithms.insecure-cipher-algorithm-idea
      languages:
        - python
      message: IDEA (International Data Encryption Algorithm) is a block cipher created in 1991.
  It is an optional component of the OpenPGP standard. This cipher is susceptible to attacks whe
n using weak keys.  It is recommended that you do not use this cipher for new applications. Use
a strong symmetric cipher such as EAS instead. With the `cryptography` package it is recommended
 to use `Fernet` which is a secure implementation of AES in CBC mode with a 128-bit key.  Altern
atively, keep using the `Cipher` class from the hazmat primitives but use the AES algorithm inst
ead.
      metadata:
        bandit-code: B304
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::symmetric-algorithm::cryptography
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://tools.ietf.org/html/rfc5469
            - https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/#cryptogr
aphy.hazmat.primitives.ciphers.algorithms.IDEA
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L98
        subcategory:
            - vuln
        technology:
            - cryptography
      patterns:
        - pattern: cryptography.hazmat.primitives.ciphers.algorithms.$IDEA($KEY)
        - metavariable-regex:
            metavariable: $IDEA
            regex: ^(IDEA)$
        - focus-metavariable: $IDEA
      severity: WARNING
    - fix: cryptography.hazmat.primitives.ciphers.modes.GCM($IV)
      id: python.cryptography.security.insecure-cipher-mode-ecb.insecure-cipher-mode-ecb
      languages:
        - python
      message: ECB (Electronic Code Book) is the simplest mode of operation for block ciphers.
Each block of data is encrypted in the same way.  This means identical plaintext blocks will alw
ays result in identical ciphertext blocks, which can leave significant patterns in the output. U
se a different, cryptographically strong mode instead, such as GCM.
      metadata:
        bandit-code: B305
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::mode::cryptography
        impact: LOW
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/#insecure
-modes
            - https://crypto.stackexchange.com/questions/20941/why-shouldnt-i-use-ecb-encryption
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L101
        subcategory:
            - audit
        technology:
            - cryptography
      pattern: cryptography.hazmat.primitives.ciphers.modes.ECB($IV)
      severity: WARNING
    - fix: SHA256
      id: python.cryptography.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5
      languages:
        - python
      message: Detected MD5 hash algorithm which is considered insecure. MD5 is not collision re
sistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
      metadata:
        bandit-code: B303
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::symmetric-algorithm::cryptography
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cryptography.io/en/latest/hazmat/primitives/cryptographic-hashes/#md5
            - https://www.schneier.com/blog/archives/2012/10/when_will_we_se.html
            - https://www.trendmicro.com/vinfo/us/security/news/vulnerabilities-and-exploits/sha
-1-collision-signals-the-end-of-the-algorithm-s-viability
            - http://2012.sharcs.org/slides/stevens.pdf
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L59
        subcategory:
            - vuln
        technology:
            - cryptography
      patterns:
        - pattern: cryptography.hazmat.primitives.hashes.$MD5()
        - metavariable-regex:
            metavariable: $MD5
            regex: ^(MD5)$
        - focus-metavariable: $MD5
      severity: WARNING
    - fix: |
        SHA256
      id: python.cryptography.security.insecure-hash-algorithms.insecure-hash-algorithm-sha1
      languages:
        - python
      message: Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision
resistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead
.
      metadata:
        bandit-code: B303
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        functional-categories:
            - crypto::search::symmetric-algorithm::cryptography
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cryptography.io/en/latest/hazmat/primitives/cryptographic-hashes/#sha-1
            - https://www.schneier.com/blog/archives/2012/10/when_will_we_se.html
            - https://www.trendmicro.com/vinfo/us/security/news/vulnerabilities-and-exploits/sha
-1-collision-signals-the-end-of-the-algorithm-s-viability
            - http://2012.sharcs.org/slides/stevens.pdf
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L59
        subcategory:
            - vuln
        technology:
            - cryptography
      patterns:
        - pattern: cryptography.hazmat.primitives.hashes.$SHA(...)
        - metavariable-pattern:
            metavariable: $SHA
            pattern: |
                SHA1
        - focus-metavariable: $SHA
      severity: WARNING
    - fix: |
        2048
      id: python.cryptography.security.insufficient-dsa-key-size.insufficient-dsa-key-size
      languages:
        - python
      message: Detected an insufficient key size for DSA. NIST recommends a key size of 2048 or
higher.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        functional-categories:
            - crypto::search::key-length::cryptography
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.cosic.esat.kuleuven.be/ecrypt/ecrypt2/documents/D.SPA.20.pdf
            - https://cryptography.io/en/latest/hazmat/primitives/asymmetric/dsa/
            - https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57Pt3r1.pdf
        source-rule-url: https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee9
54c2b1/bandit/plugins/weak_cryptographic_key.py
        subcategory:
            - vuln
        technology:
            - cryptography
      patterns:
        - pattern-either:
            - pattern: cryptography.hazmat.primitives.asymmetric.dsa.generate_private_key(..., k
ey_size=$SIZE, ...)
            - pattern: cryptography.hazmat.primitives.asymmetric.dsa.generate_private_key($SIZE,
 ...)
        - metavariable-comparison:
            comparison: $SIZE < 2048
            metavariable: $SIZE
        - focus-metavariable: $SIZE
      severity: WARNING
    - fix: |
        SECP256R1
      id: python.cryptography.security.insufficient-ec-key-size.insufficient-ec-key-size
      languages:
        - python
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Detected an insufficient curve size for EC. NIST recommends a key size of 224 or
higher. For example, use 'ec.SECP256R1'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        functional-categories:
            - crypto::search::key-length::cryptography
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57Pt3r1.pdf
            - https://cryptography.io/en/latest/hazmat/primitives/asymmetric/ec/#elliptic-curves
        source-rule-url: https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee9
54c2b1/bandit/plugins/weak_cryptographic_key.py
        subcategory:
            - audit
        technology:
            - cryptography
      patterns:
        - pattern-inside: cryptography.hazmat.primitives.asymmetric.ec.generate_private_key(...)
        - pattern: cryptography.hazmat.primitives.asymmetric.ec.$SIZE
        - metavariable-pattern:
            metavariable: $SIZE
            pattern-either:
                - pattern: SECP192R1
                - pattern: SECT163K1
                - pattern: SECT163R2
        - focus-metavariable: $SIZE
      severity: WARNING
    - fix: |
        2048
      id: python.cryptography.security.insufficient-rsa-key-size.insufficient-rsa-key-size
      languages:
        - python
      message: Detected an insufficient key size for RSA. NIST recommends a key size of 2048 or
higher.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        functional-categories:
            - crypto::search::key-length::cryptography
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cryptography.io/en/latest/hazmat/primitives/asymmetric/rsa/
            - https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57Pt3r1.pdf
        source-rule-url: https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee9
54c2b1/bandit/plugins/weak_cryptographic_key.py
        subcategory:
            - audit
        technology:
            - cryptography
      patterns:
        - pattern-either:
            - pattern: cryptography.hazmat.primitives.asymmetric.rsa.generate_private_key(..., k
ey_size=$SIZE, ...)
            - pattern: cryptography.hazmat.primitives.asymmetric.rsa.generate_private_key($EXP,
$SIZE, ...)
        - metavariable-comparison:
            comparison: $SIZE < 2048
            metavariable: $SIZE
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - focus-metavariable: $SIZE
      severity: WARNING
    - id: python.cryptography.security.mode-without-authentication.crypto-mode-without-authentic
ation
      languages:
        - python
      message: 'An encryption mode of operation is being used without proper message authenticat
ion. This can potentially result in the encrypted content to be decrypted by an attacker. Consid
er instead use an AEAD mode of operation like GCM. '
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - audit
        technology:
            - cryptography
      patterns:
        - pattern-either:
            - patterns:
                - pattern: |
                    Cipher(..., $HAZMAT_MODE(...),...)
                - pattern-not-inside: |
                    Cipher(..., $HAZMAT_MODE(...),...)
                    ...
                    HMAC(...)
                - pattern-not-inside: |
                    Cipher(..., $HAZMAT_MODE(...),...)
                    ...
                    hmac.HMAC(...)
        - metavariable-pattern:
            metavariable: $HAZMAT_MODE
            patterns:
                - pattern-either:
                    - pattern: modes.CTR
                    - pattern: modes.CBC
                    - pattern: modes.CFB
                    - pattern: modes.OFB
      severity: ERROR
    - fix: |
        True
      id: python.distributed.security.require-encryption
      languages:
        - python
      message: Initializing a security context for Dask (`distributed`) without "require_encrypt
ion" keyword argument may silently fail to provide security.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://distributed.dask.org/en/latest/tls.html?highlight=require_encryption#param
eters
        subcategory:
            - vuln
        technology:
            - distributed
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      patterns:
        - pattern: |
            distributed.security.Security(..., require_encryption=$VAL, ...)
        - metavariable-pattern:
            metavariable: $VAL
            pattern: |
                False
        - focus-metavariable: $VAL
      severity: WARNING
    - id: python.django.security.audit.avoid-insecure-deserialization.avoid-insecure-deserializa
tion
      languages:
        - python
      message: Avoid using insecure deserialization library, backed by `pickle`, `_pickle`, `cpi
ckle`, `dill`, `shelve`, or `yaml`, which are known to lead to remote code execution vulnerabili
ties.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://docs.python.org/3/library/pickle.html
        subcategory:
            - vuln
        technology:
            - django
      mode: taint
      pattern-sinks:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: |
                        pickle.$PICKLEFUNC(...)
                    - pattern: |
                        _pickle.$PICKLEFUNC(...)
                    - pattern: |
                        cPickle.$PICKLEFUNC(...)
                    - pattern: |
                        shelve.$PICKLEFUNC(...)
                - metavariable-regex:
                    metavariable: $PICKLEFUNC
                    regex: dumps|dump|load|loads
            - patterns:
                - pattern: dill.$DILLFUNC(...)
                - metavariable-regex:
                    metavariable: $DILLFUNC
                    regex: dump|dump_session|dumps|load|load_session|loads
            - patterns:
                - pattern: yaml.$YAMLFUNC(...)
                - pattern-not: yaml.$YAMLFUNC(..., Dumper=SafeDumper, ...)
                - pattern-not: yaml.$YAMLFUNC(..., Dumper=yaml.SafeDumper, ...)
                - pattern-not: yaml.$YAMLFUNC(..., Loader=SafeLoader, ...)
                - pattern-not: yaml.$YAMLFUNC(..., Loader=yaml.SafeLoader, ...)
                - metavariable-regex:
                    metavariable: $YAMLFUNC
                    regex: dump|dump_all|load|load_all
      pattern-sources:
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    def $INSIDE(..., $PARAM, ...):
                      ...
                - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern: request.$REQFUNC(...)
                    - pattern: request.$REQFUNC.get(...)
                    - pattern: request.$REQFUNC[...]
      severity: ERROR
    - id: python.django.security.django-no-csrf-token.django-no-csrf-token
      languages:
        - generic
      message: Manually-created forms in django templates should specify a csrf_token to prevent
 CSRF attacks
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-352: Cross-Site Request Forgery (CSRF)'
        impact: MEDIUM
        likelihood: MEDIUM
        references:
            - https://docs.djangoproject.com/en/4.2/howto/csrf/
        subcategory:
            - guardrail
        technology:
            - django
      paths:
        include:
            - '*.html'
      patterns:
        - pattern: <form...>...</form>
        - pattern-either:
            - pattern: |
                <form ... method="$METHOD" ...>...</form>
            - pattern: |
                <form ... method='$METHOD' ...>...</form>
            - pattern: |
                <form ... method=$METHOD ...>...</form>
        - metavariable-regex:
            metavariable: $METHOD
            regex: (?i)(post|put|delete|patch)
        - pattern-not-inside: <form...>...{% csrf_token %}...</form>
        - pattern-not-inside: <form...>...{{ $VAR.csrf_token }}...</form>
      severity: WARNING
    - id: python.django.security.django-using-request-post-after-is-valid.django-using-request-p
ost-after-is-valid
      languages:
        - python
      message: Use $FORM.cleaned_data[] instead of request.POST[] after form.is_valid() has been
 executed to only access sanitized data
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-20: Improper Input Validation'
        impact: MEDIUM
        likelihood: MEDIUM
        references:
            - https://docs.djangoproject.com/en/4.2/ref/forms/api/#accessing-clean-data
        subcategory:
            - guardrail
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(request, ...):
              ...
        - pattern-inside: |
            if $FORM.is_valid():
              ...
        - pattern-either:
            - pattern: request.POST[...]
            - pattern: request.POST.get(...)
      severity: WARNING
    - id: python.django.security.hashids-with-django-secret.hashids-with-django-secret
      languages:
        - python
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: The Django secret key is used as salt in HashIDs. The HashID mechanism is not sec
ure. By observing sufficient HashIDs, the salt used to construct them can be recovered. This mea
ns the Django secret key can be obtained by attackers, through the HashIDs.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A02:2021 – Cryptographic Failures
        references:
            - https://docs.djangoproject.com/en/4.2/ref/settings/#std-setting-SECRET_KEY
            - http://carnage.github.io/2015/08/cryptanalysis-of-hashids
        subcategory:
            - vuln
        technology:
            - django
      pattern-either:
        - pattern: hashids.Hashids(..., salt=django.conf.settings.SECRET_KEY, ...)
        - pattern: hashids.Hashids(django.conf.settings.SECRET_KEY, ...)
      severity: ERROR
    - id: python.django.security.injection.code.user-eval-format-string.user-eval-format-string
      languages:
        - python
      message: Found user data in a call to 'eval'. This is extremely dangerous because it can e
nable an attacker to execute remote code. See https://owasp.org/www-community/attacks/Code_Injec
tion for more information.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://nedbatchelder.com/blog/201206/eval_really_is_dangerous.html
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $F(...):
              ...
        - pattern-either:
            - pattern: eval(..., $STR % request.$W.get(...), ...)
            - pattern: |
                $V = request.$W.get(...)
                ...
                eval(..., $STR % $V, ...)
            - pattern: |
                $V = request.$W.get(...)
                ...
                $S = $STR % $V
                ...
                eval(..., $S, ...)
            - pattern: eval(..., "..." % request.$W(...), ...)
            - pattern: |
                $V = request.$W(...)
                ...
                eval(..., $STR % $V, ...)
            - pattern: |
                $V = request.$W(...)
                ...
                $S = $STR % $V
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    eval(..., $S, ...)
- pattern: eval(..., $STR % request.$W[...], ...)
- pattern: |
    $V = request.$W[...]
    ...
    eval(..., $STR % $V, ...)
- pattern: |
    $V = request.$W[...]
    ...
    $S = $STR % $V
    ...
    eval(..., $S, ...)
- pattern: eval(..., $STR.format(..., request.$W.get(...), ...), ...)
- pattern: |
    $V = request.$W.get(...)
    ...
    eval(..., $STR.format(..., $V, ...), ...)
- pattern: |
    $V = request.$W.get(...)
    ...
    $S = $STR.format(..., $V, ...)
    ...
    eval(..., $S, ...)
- pattern: eval(..., $STR.format(..., request.$W(...), ...), ...)
- pattern: |
    $V = request.$W(...)
    ...
    eval(..., $STR.format(..., $V, ...), ...)
- pattern: |
    $V = request.$W(...)
    ...
    $S = $STR.format(..., $V, ...)
    ...
    eval(..., $S, ...)
- pattern: eval(..., $STR.format(..., request.$W[...], ...), ...)
- pattern: |
    $V = request.$W[...]
    ...
    eval(..., $STR.format(..., $V, ...), ...)
- pattern: |
    $V = request.$W[...]
    ...
    $S = $STR.format(..., $V, ...)
    ...
    eval(..., $S, ...)
- pattern: |
    $V = request.$W.get(...)
    ...
    eval(..., f"...{$V}...", ...)
- pattern: |
    $V = request.$W.get(...)
    ...
    $S = f"...{$V}..."
    ...
    eval(..., $S, ...)
- pattern: |
    $V = request.$W(...)
    ...
    eval(..., f"...{$V}...", ...)
- pattern: |
    $V = request.$W(...)
    ...
    $S = f"...{$V}..."
    ...
    eval(..., $S, ...)
- pattern: |
    $V = request.$W[...]
    ...
    eval(..., f"...{$V}...", ...)
- pattern: |
    $V = request.$W[...]
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                $S = f"...{$V}..."
                ...
                eval(..., $S, ...)
      severity: WARNING
    - id: python.django.security.injection.code.user-eval.user-eval
      languages:
        - python
      message: Found user data in a call to 'eval'. This is extremely dangerous because it can e
nable an attacker to execute arbitrary remote code on the system. Instead, refactor your code to
 not use 'eval' and instead use a safe library for the specific functionality you need.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://nedbatchelder.com/blog/201206/eval_really_is_dangerous.html
            - https://owasp.org/www-community/attacks/Code_Injection
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $F(...):
              ...
        - pattern-either:
            - pattern: eval(..., request.$W.get(...), ...)
            - pattern: |
                $V = request.$W.get(...)
                ...
                eval(..., $V, ...)
            - pattern: eval(..., request.$W(...), ...)
            - pattern: |
                $V = request.$W(...)
                ...
                eval(..., $V, ...)
            - pattern: eval(..., request.$W[...], ...)
            - pattern: |
                $V = request.$W[...]
                ...
                eval(..., $V, ...)
      severity: WARNING
    - id: python.django.security.injection.code.user-exec-format-string.user-exec-format-string
      languages:
        - python
      message: Found user data in a call to 'exec'. This is extremely dangerous because it can e
nable an attacker to execute arbitrary remote code on the system. Instead, refactor your code to
 not use 'eval' and instead use a safe library for the specific functionality you need.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/Code_Injection
        subcategory:
            - vuln
        technology:
            - django
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
patterns:
  - pattern-inside: |
      def $F(...):
        ...
  - pattern-either:
      - pattern: exec(..., $STR % request.$W.get(...), ...)
      - pattern: |
          $V = request.$W.get(...)
          ...
          exec(..., $STR % $V, ...)
      - pattern: |
          $V = request.$W.get(...)
          ...
          $S = $STR % $V
          ...
          exec(..., $S, ...)
      - pattern: exec(..., "..." % request.$W(...), ...)
      - pattern: |
          $V = request.$W(...)
          ...
          exec(..., $STR % $V, ...)
      - pattern: |
          $V = request.$W(...)
          ...
          $S = $STR % $V
          ...
          exec(..., $S, ...)
      - pattern: exec(..., $STR % request.$W[...], ...)
      - pattern: |
          $V = request.$W[...]
          ...
          exec(..., $STR % $V, ...)
      - pattern: |
          $V = request.$W[...]
          ...
          $S = $STR % $V
          ...
          exec(..., $S, ...)
      - pattern: exec(..., $STR.format(..., request.$W.get(...), ...), ...)
      - pattern: |
          $V = request.$W.get(...)
          ...
          exec(..., $STR.format(..., $V, ...), ...)
      - pattern: |
          $V = request.$W.get(...)
          ...
          $S = $STR.format(..., $V, ...)
          ...
          exec(..., $S, ...)
      - pattern: exec(..., $STR.format(..., request.$W(...), ...), ...)
      - pattern: |
          $V = request.$W(...)
          ...
          exec(..., $STR.format(..., $V, ...), ...)
      - pattern: |
          $V = request.$W(...)
          ...
          $S = $STR.format(..., $V, ...)
          ...
          exec(..., $S, ...)
      - pattern: exec(..., $STR.format(..., request.$W[...], ...), ...)
      - pattern: |
          $V = request.$W[...]
          ...
          exec(..., $STR.format(..., $V, ...), ...)
      - pattern: |
          $V = request.$W[...]
          ...
          $S = $STR.format(..., $V, ...)
          ...
          exec(..., $S, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                $V = request.$W.get(...)
                ...
                exec(..., f"...{$V}...", ...)
            - pattern: |
                $V = request.$W.get(...)
                ...
                $S = f"...{$V}..."
                ...
                exec(..., $S, ...)
            - pattern: |
                $V = request.$W(...)
                ...
                exec(..., f"...{$V}...", ...)
            - pattern: |
                $V = request.$W(...)
                ...
                $S = f"...{$V}..."
                ...
                exec(..., $S, ...)
            - pattern: |
                $V = request.$W[...]
                ...
                exec(..., f"...{$V}...", ...)
            - pattern: |
                $V = request.$W[...]
                ...
                $S = f"...{$V}..."
                ...
                exec(..., $S, ...)
            - pattern: exec(..., base64.decodestring($S.format(..., request.$W.get(...), ...), .
..), ...)
            - pattern: exec(..., base64.decodestring($S % request.$W.get(...), ...), ...)
            - pattern: exec(..., base64.decodestring(f"...{request.$W.get(...)}...", ...), ...)
            - pattern: exec(..., base64.decodestring(request.$W.get(...), ...), ...)
            - pattern: exec(..., base64.decodestring(bytes($S.format(..., request.$W.get(...), .
..), ...), ...), ...)
            - pattern: exec(..., base64.decodestring(bytes($S % request.$W.get(...), ...), ...),
 ...)
            - pattern: exec(..., base64.decodestring(bytes(f"...{request.$W.get(...)}...", ...),
 ...), ...)
            - pattern: exec(..., base64.decodestring(bytes(request.$W.get(...), ...), ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                exec(..., base64.decodestring($DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = base64.decodestring($DATA, ...)
                ...
                exec(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                exec(..., base64.decodestring(bytes($DATA, ...), ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = base64.decodestring(bytes($DATA, ...), ...)
                ...
                exec(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                exec(..., base64.decodestring($DATA, ...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = base64.decodestring($DATA, ...)
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                exec(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                exec(..., base64.decodestring(bytes($DATA, ...), ...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = base64.decodestring(bytes($DATA, ...), ...)
                ...
                exec(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                exec(..., base64.decodestring($DATA, ...), ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = base64.decodestring($DATA, ...)
                ...
                exec(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                exec(..., base64.decodestring(bytes($DATA, ...), ...), ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = base64.decodestring(bytes($DATA, ...), ...)
                ...
                exec(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                exec(..., base64.decodestring($DATA, ...), ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = base64.decodestring($DATA, ...)
                ...
                exec(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                exec(..., base64.decodestring(bytes($DATA, ...), ...), ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = base64.decodestring(bytes($DATA, ...), ...)
                ...
                exec(..., $INTERM, ...)
      severity: WARNING
    - id: python.django.security.injection.code.user-exec.user-exec
      languages:
        - python
      message: Found user data in a call to 'exec'. This is extremely dangerous because it can e
nable an attacker to execute arbitrary remote code on the system. Instead, refactor your code to
 not use 'eval' and instead use a safe library for the specific functionality you need.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/Code_Injection
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - vuln
        technology:
          - django
      patterns:
        - pattern-inside: |
            def $F(...):
              ...
        - pattern-either:
            - pattern: exec(..., request.$W.get(...), ...)
            - pattern: |
                $V = request.$W.get(...)
                ...
                exec(..., $V, ...)
            - pattern: exec(..., request.$W(...), ...)
            - pattern: |
                $V = request.$W(...)
                ...
                exec(..., $V, ...)
            - pattern: exec(..., request.$W[...], ...)
            - pattern: |
                $V = request.$W[...]
                ...
                exec(..., $V, ...)
            - pattern: |
                loop = asyncio.get_running_loop()
                ...
                await loop.run_in_executor(None, exec, request.$W[...])
            - pattern: |
                $V = request.$W[...]
                ...
                loop = asyncio.get_running_loop()
                ...
                await loop.run_in_executor(None, exec, $V)
            - pattern: |
                loop = asyncio.get_running_loop()
                ...
                await loop.run_in_executor(None, exec, request.$W.get(...))
            - pattern: |
                $V = request.$W.get(...)
                ...
                loop = asyncio.get_running_loop()
                ...
                await loop.run_in_executor(None, exec, $V)
      severity: WARNING
    - id: python.django.security.injection.command.command-injection-os-system.command-injection
-os-system
      languages:
        - python
      message: Request data detected in os.system. This could be vulnerable to a command injecti
on and should be avoided. If this must be done, use the 'subprocess' module instead and pass the
 arguments as a list. See https://owasp.org/www-community/attacks/Command_Injection for more inf
ormation.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/Command_Injection
        subcategory:
            - vuln
        technology:
            - django
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
patterns:
  - pattern-inside: |
      def $FUNC(...):
        ...
  - pattern-either:
      - pattern: os.system(..., request.$W.get(...), ...)
      - pattern: os.system(..., $S.format(..., request.$W.get(...), ...), ...)
      - pattern: os.system(..., $S % request.$W.get(...), ...)
      - pattern: os.system(..., f"...{request.$W.get(...)}...", ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          os.system(..., $DATA, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $DATA
          ...
          os.system(..., $INTERM, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          os.system(..., $STR.format(..., $DATA, ...), ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $STR.format(..., $DATA, ...)
          ...
          os.system(..., $INTERM, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          os.system(..., $STR % $DATA, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $STR % $DATA
          ...
          os.system(..., $INTERM, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          os.system(..., f"...{$DATA}...", ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = f"...{$DATA}..."
          ...
          os.system(..., $INTERM, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          os.system(..., $STR + $DATA, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $STR + $DATA
          ...
          os.system(..., $INTERM, ...)
      - pattern: $A = os.system(..., request.$W.get(...), ...)
      - pattern: $A = os.system(..., $S.format(..., request.$W.get(...), ...), ...)
      - pattern: $A = os.system(..., $S % request.$W.get(...), ...)
      - pattern: $A = os.system(..., f"...{request.$W.get(...)}...", ...)
      - pattern: return os.system(..., request.$W.get(...), ...)
      - pattern: return os.system(..., $S.format(..., request.$W.get(...), ...), ...)
      - pattern: return os.system(..., $S % request.$W.get(...), ...)
      - pattern: return os.system(..., f"...{request.$W.get(...)}...", ...)
      - pattern: os.system(..., request.$W(...), ...)
      - pattern: os.system(..., $S.format(..., request.$W(...), ...), ...)
      - pattern: os.system(..., $S % request.$W(...), ...)
      - pattern: os.system(..., f"...{request.$W(...)}...", ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
- pattern: |
    $DATA = request.$W(...)
    ...
    os.system(..., $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $DATA
    ...
    os.system(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    os.system(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    os.system(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    os.system(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    os.system(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    os.system(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    os.system(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    os.system(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR + $DATA
    ...
    os.system(..., $INTERM, ...)
- pattern: $A = os.system(..., request.$W(...), ...)
- pattern: $A = os.system(..., $S.format(..., request.$W(...), ...), ...)
- pattern: $A = os.system(..., $S % request.$W(...), ...)
- pattern: $A = os.system(..., f"...{request.$W(...)}...", ...)
- pattern: return os.system(..., request.$W(...), ...)
- pattern: return os.system(..., $S.format(..., request.$W(...), ...), ...)
- pattern: return os.system(..., $S % request.$W(...), ...)
- pattern: return os.system(..., f"...{request.$W(...)}...", ...)
- pattern: os.system(..., request.$W[...], ...)
- pattern: os.system(..., $S.format(..., request.$W[...], ...), ...)
- pattern: os.system(..., $S % request.$W[...], ...)
- pattern: os.system(..., f"...{request.$W[...]}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    os.system(..., $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    os.system(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    os.system(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    os.system(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    os.system(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    os.system(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    os.system(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    os.system(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    os.system(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR + $DATA
    ...
    os.system(..., $INTERM, ...)
- pattern: $A = os.system(..., request.$W[...], ...)
- pattern: $A = os.system(..., $S.format(..., request.$W[...], ...), ...)
- pattern: $A = os.system(..., $S % request.$W[...], ...)
- pattern: $A = os.system(..., f"...{request.$W[...]}...", ...)
- pattern: return os.system(..., request.$W[...], ...)
- pattern: return os.system(..., $S.format(..., request.$W[...], ...), ...)
- pattern: return os.system(..., $S % request.$W[...], ...)
- pattern: return os.system(..., f"...{request.$W[...]}...", ...)
- pattern: os.system(..., request.$W, ...)
- pattern: os.system(..., $S.format(..., request.$W, ...), ...)
- pattern: os.system(..., $S % request.$W, ...)
- pattern: os.system(..., f"...{request.$W}...", ...)
- pattern: |
    $DATA = request.$W
    ...
    os.system(..., $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    os.system(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    os.system(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR.format(..., $DATA, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                os.system(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                os.system(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % $DATA
                ...
                os.system(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                os.system(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                os.system(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                os.system(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                os.system(..., $INTERM, ...)
            - pattern: $A = os.system(..., request.$W, ...)
            - pattern: $A = os.system(..., $S.format(..., request.$W, ...), ...)
            - pattern: $A = os.system(..., $S % request.$W, ...)
            - pattern: $A = os.system(..., f"...{request.$W}...", ...)
            - pattern: return os.system(..., request.$W, ...)
            - pattern: return os.system(..., $S.format(..., request.$W, ...), ...)
            - pattern: return os.system(..., $S % request.$W, ...)
            - pattern: return os.system(..., f"...{request.$W}...", ...)
      severity: ERROR
    - id: python.django.security.injection.command.subprocess-injection.subprocess-injection
      languages:
        - python
      message: Detected user input entering a `subprocess` call unsafely. This could result in a
 command injection vulnerability. An attacker could use this vulnerability to execute arbitrary
commands on the host, which allows them to download malware, scan sensitive data, or run any com
mand they wish on the server. Do not let users choose the command to run. In general, prefer to
use Python API versions of system commands. If you must use subprocess, use a dictionary to allo
wlist a set of commands.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - flask
      mode: taint
      options:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        symbolic_propagation: true
      pattern-sanitizers:
        - patterns:
            - pattern: $DICT[$KEY]
            - focus-metavariable: $KEY
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: subprocess.$FUNC(...)
                    - pattern-not: subprocess.$FUNC("...", ...)
                    - pattern-not: subprocess.$FUNC(["...", ...], ...)
                    - pattern-not-inside: |
                        $CMD = ["...", ...]
                        ...
                        subprocess.$FUNC($CMD, ...)
                - patterns:
                    - pattern: subprocess.$FUNC(["$SHELL", "-c", ...], ...)
                    - metavariable-regex:
                        metavariable: $SHELL
                        regex: ^(sh|bash|ksh|csh|tcsh|zsh)$
                - patterns:
                    - pattern: subprocess.$FUNC(["$INTERPRETER", ...], ...)
                    - metavariable-regex:
                        metavariable: $INTERPRETER
                        regex: ^(python|python\d)$
      pattern-sources:
        - patterns:
            - pattern-inside: |
                def $FUNC(..., $REQUEST, ...):
                  ...
            - focus-metavariable: $REQUEST
            - metavariable-pattern:
                metavariable: $REQUEST
                patterns:
                    - pattern: request
                    - pattern-not-inside: request.build_absolute_uri
      severity: ERROR
    - id: python.django.security.injection.csv-writer-injection.csv-writer-injection
      languages:
        - python
      message: Detected user input into a generated CSV file using the built-in `csv` module. If
 user data is used to generate the data in this file, it is possible that an attacker could inje
ct a formula when the CSV is imported into a spreadsheet application that runs an attacker scrip
t, which could steal data from the importing user or, at worst, install malware on the user's co
mputer. `defusedcsv` is a drop-in replacement with the same API that will attempt to mitigate fo
rmula injection attempts. You can use `defusedcsv` instead of `csv` to safely generate CSVs.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1236: Improper Neutralization of Formula Elements in a CSV File'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://github.com/raphaelm/defusedcsv
            - https://owasp.org/www-community/attacks/CSV_Injection
            - https://web.archive.org/web/20220516052229/https://www.contextis.com/us/blog/comma
-separated-vulnerabilities
        subcategory:
            - vuln
        technology:
            - django
            - python
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                $WRITER = csv.writer(...)

                ...

                $WRITER.$WRITE(...)
            - pattern: $WRITER.$WRITE(...)
            - metavariable-regex:
                metavariable: $WRITE
                regex: ^(writerow|writerows|writeheader)$
      pattern-sources:
        - patterns:
            - pattern-inside: |
                def $FUNC(..., $REQUEST, ...):
                  ...
            - focus-metavariable: $REQUEST
            - metavariable-pattern:
                metavariable: $REQUEST
                patterns:
                    - pattern: request
                    - pattern-not-inside: request.build_absolute_uri
      severity: ERROR
    - id: python.django.security.injection.email.xss-html-email-body.xss-html-email-body
      languages:
        - python
      message: Found request data in an EmailMessage that is set to use HTML. This is dangerous
because HTML emails are susceptible to XSS. An attacker could inject data into this HTML email,
causing XSS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-74: Improper Neutralization of Special Elements in Output Used by a Downstrea
m Component (''Injection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://www.damonkohler.com/2008/12/email-injection.html
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
              ...
              $EMAIL.content_subtype = "html"
              ...
        - pattern-either:
            - pattern: django.core.mail.EmailMessage($SUBJ, request.$W.get(...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.core.mail.EmailMessage($SUBJ, $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.core.mail.EmailMessage($SUBJ, $B.$C(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $B.$C(..., $DATA, ...)
                ...
                django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
- pattern: |
    $DATA = request.$W.get(...)
    ...
    django.core.mail.EmailMessage($SUBJ, $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $INTERM = $STR % $DATA
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    django.core.mail.EmailMessage($SUBJ, f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: $A = django.core.mail.EmailMessage($SUBJ, request.$W.get(...), ...)
- pattern: return django.core.mail.EmailMessage($SUBJ, request.$W.get(...), ...)
- pattern: django.core.mail.EmailMessage($SUBJ, request.$W(...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    django.core.mail.EmailMessage($SUBJ, $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $DATA
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    django.core.mail.EmailMessage($SUBJ, $B.$C(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $B.$C(..., $DATA, ...)
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    django.core.mail.EmailMessage($SUBJ, $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    django.core.mail.EmailMessage($SUBJ, f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: $A = django.core.mail.EmailMessage($SUBJ, request.$W(...), ...)
- pattern: return django.core.mail.EmailMessage($SUBJ, request.$W(...), ...)
- pattern: django.core.mail.EmailMessage($SUBJ, request.$W[...], ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.core.mail.EmailMessage($SUBJ, $DATA, ...)
- pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.core.mail.EmailMessage($SUBJ, $B.$C(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $B.$C(..., $DATA, ...)
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.core.mail.EmailMessage($SUBJ, $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.core.mail.EmailMessage($SUBJ, f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: $A = django.core.mail.EmailMessage($SUBJ, request.$W[...], ...)
- pattern: return django.core.mail.EmailMessage($SUBJ, request.$W[...], ...)
- pattern: django.core.mail.EmailMessage($SUBJ, request.$W, ...)
- pattern: |
    $DATA = request.$W
    ...
    django.core.mail.EmailMessage($SUBJ, $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    django.core.mail.EmailMessage($SUBJ, $B.$C(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $B.$C(..., $DATA, ...)
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    django.core.mail.EmailMessage($SUBJ, $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR % $DATA
    ...
    django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                django.core.mail.EmailMessage($SUBJ, f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.core.mail.EmailMessage($SUBJ, $INTERM, ...)
            - pattern: $A = django.core.mail.EmailMessage($SUBJ, request.$W, ...)
            - pattern: return django.core.mail.EmailMessage($SUBJ, request.$W, ...)
      severity: WARNING
    - id: python.django.security.injection.email.xss-send-mail-html-message.xss-send-mail-html-m
essage
      languages:
        - python
      message: Found request data in 'send_mail(...)' that uses 'html_message'. This is dangerou
s because HTML emails are susceptible to XSS. An attacker could inject data into this HTML email
, causing XSS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-74: Improper Neutralization of Special Elements in Output Used by a Downstrea
m Component (''Injection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://www.damonkohler.com/2008/12/email-injection.html
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
              ...
        - pattern-either:
            - pattern: django.core.mail.send_mail(..., html_message=request.$W.get(...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.core.mail.send_mail(..., html_message=$DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.core.mail.send_mail(..., html_message=$STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.core.mail.send_mail(..., html_message=$STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % $DATA
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                django.core.mail.send_mail(..., html_message=f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.core.mail.send_mail(..., html_message=$STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR + $DATA
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: $A = django.core.mail.send_mail(..., html_message=request.$W.get(...), ..
.)
            - pattern: return django.core.mail.send_mail(..., html_message=request.$W.get(...),
...)
            - pattern: django.core.mail.send_mail(..., html_message=request.$W(...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.core.mail.send_mail(..., html_message=$DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $DATA
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.core.mail.send_mail(..., html_message=$STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.core.mail.send_mail(..., html_message=$STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR % $DATA
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.core.mail.send_mail(..., html_message=f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.core.mail.send_mail(..., html_message=$STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR + $DATA
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    ...
    django.core.mail.send_mail(..., html_message=$INTERM, ...)
- pattern: $A = django.core.mail.send_mail(..., html_message=request.$W(...), ...)
- pattern: return django.core.mail.send_mail(..., html_message=request.$W(...), ...)
- pattern: django.core.mail.send_mail(..., html_message=request.$W[...], ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.core.mail.send_mail(..., html_message=$DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
    django.core.mail.send_mail(..., html_message=$INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.core.mail.send_mail(..., html_message=$STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    django.core.mail.send_mail(..., html_message=$INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.core.mail.send_mail(..., html_message=$STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    django.core.mail.send_mail(..., html_message=$INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.core.mail.send_mail(..., html_message=f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    django.core.mail.send_mail(..., html_message=$INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.core.mail.send_mail(..., html_message=$STR + $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR + $DATA
    ...
    django.core.mail.send_mail(..., html_message=$INTERM, ...)
- pattern: $A = django.core.mail.send_mail(..., html_message=request.$W[...], ...)
- pattern: return django.core.mail.send_mail(..., html_message=request.$W[...], ...)
- pattern: django.core.mail.send_mail(..., html_message=request.$W, ...)
- pattern: |
    $DATA = request.$W
    ...
    django.core.mail.send_mail(..., html_message=$DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    django.core.mail.send_mail(..., html_message=$INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                django.core.mail.send_mail(..., html_message=$STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.core.mail.send_mail(..., html_message=$STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % $DATA
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.core.mail.send_mail(..., html_message=f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.core.mail.send_mail(..., html_message=$STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                django.core.mail.send_mail(..., html_message=$INTERM, ...)
            - pattern: $A = django.core.mail.send_mail(..., html_message=request.$W, ...)
            - pattern: return django.core.mail.send_mail(..., html_message=request.$W, ...)
      severity: WARNING
    - id: python.django.security.injection.open-redirect.open-redirect
      languages:
        - python
      message: Data from request ($DATA) is passed to redirect(). This is an open redirect and c
ould be exploited. Ensure you are redirecting to safe URLs by using django.utils.http.is_safe_ur
l(). See https://cwe.mitre.org/data/definitions/601.html for more information.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-601: URL Redirection to Untrusted Site (''Open Redirect'')'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://www.djm.org.uk/posts/djangos-little-protections-word-redirect-dangers/
            - https://github.com/django/django/blob/d1b7bd030b1db111e1a3505b1fc029ab964382cc/dja
ngo/utils/http.py#L231
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
              ...
        - pattern-not-inside: |
            def $FUNC(...):
              ...
              django.utils.http.is_safe_url(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
        - pattern-not-inside: |
            def $FUNC(...):
              ...
              if <... django.utils.http.is_safe_url(...) ...>:
                ...
        - pattern-not-inside: |
            def $FUNC(...):
              ...
              django.utils.http.url_has_allowed_host_and_scheme(...)
              ...
        - pattern-not-inside: |
            def $FUNC(...):
              ...
              if <... django.utils.http.url_has_allowed_host_and_scheme(...) ...>:
                ...
        - pattern-either:
            - pattern: django.shortcuts.redirect(..., request.$W.get(...), ...)
            - pattern: django.shortcuts.redirect(..., $S.format(..., request.$W.get(...), ...),
...)
            - pattern: django.shortcuts.redirect(..., $S % request.$W.get(...), ...)
            - pattern: django.shortcuts.redirect(..., f"...{request.$W.get(...)}...", ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.shortcuts.redirect(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.shortcuts.redirect(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.shortcuts.redirect(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % $DATA
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.shortcuts.redirect(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.shortcuts.redirect(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR + $DATA
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              django.shortcuts.redirect(..., $INTERM, ...)
          - pattern: $A = django.shortcuts.redirect(..., request.$W.get(...), ...)
          - pattern: $A = django.shortcuts.redirect(..., $S.format(..., request.$W.get(...), .
..), ...)
          - pattern: $A = django.shortcuts.redirect(..., $S % request.$W.get(...), ...)
          - pattern: $A = django.shortcuts.redirect(..., f"...{request.$W.get(...)}...", ...)
          - pattern: return django.shortcuts.redirect(..., request.$W.get(...), ...)
          - pattern: return django.shortcuts.redirect(..., $S.format(..., request.$W.get(...),
 ...), ...)
          - pattern: return django.shortcuts.redirect(..., $S % request.$W.get(...), ...)
          - pattern: return django.shortcuts.redirect(..., f"...{request.$W.get(...)}...", ...
)
          - pattern: django.shortcuts.redirect(..., request.$W(...), ...)
          - pattern: django.shortcuts.redirect(..., $S.format(..., request.$W(...), ...), ...)
          - pattern: django.shortcuts.redirect(..., $S % request.$W(...), ...)
          - pattern: django.shortcuts.redirect(..., f"...{request.$W(...)}...", ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              django.shortcuts.redirect(..., $DATA, ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              $INTERM = $DATA
              ...
              django.shortcuts.redirect(..., $INTERM, ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              django.shortcuts.redirect(..., $STR.format(..., $DATA, ...), ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              $INTERM = $STR.format(..., $DATA, ...)
              ...
              django.shortcuts.redirect(..., $INTERM, ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              django.shortcuts.redirect(..., $STR % $DATA, ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              $INTERM = $STR % $DATA
              ...
              django.shortcuts.redirect(..., $INTERM, ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              django.shortcuts.redirect(..., f"...{$DATA}...", ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              $INTERM = f"...{$DATA}..."
              ...
              django.shortcuts.redirect(..., $INTERM, ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              django.shortcuts.redirect(..., $STR + $DATA, ...)
          - pattern: |
              $DATA = request.$W(...)
              ...
              $INTERM = $STR + $DATA
              ...
              django.shortcuts.redirect(..., $INTERM, ...)
          - pattern: $A = django.shortcuts.redirect(..., request.$W(...), ...)
          - pattern: $A = django.shortcuts.redirect(..., $S.format(..., request.$W(...), ...),
 ...)
          - pattern: $A = django.shortcuts.redirect(..., $S % request.$W(...), ...)
          - pattern: $A = django.shortcuts.redirect(..., f"...{request.$W(...)}...", ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: return django.shortcuts.redirect(..., request.$W(...), ...)
            - pattern: return django.shortcuts.redirect(..., $S.format(..., request.$W(...), ...
), ...)
            - pattern: return django.shortcuts.redirect(..., $S % request.$W(...), ...)
            - pattern: return django.shortcuts.redirect(..., f"...{request.$W(...)}...", ...)
            - pattern: django.shortcuts.redirect(..., request.$W[...], ...)
            - pattern: django.shortcuts.redirect(..., $S.format(..., request.$W[...], ...), ...)
            - pattern: django.shortcuts.redirect(..., $S % request.$W[...], ...)
            - pattern: django.shortcuts.redirect(..., f"...{request.$W[...]}...", ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.shortcuts.redirect(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $DATA
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.shortcuts.redirect(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.shortcuts.redirect(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR % $DATA
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.shortcuts.redirect(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.shortcuts.redirect(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR + $DATA
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: $A = django.shortcuts.redirect(..., request.$W[...], ...)
            - pattern: $A = django.shortcuts.redirect(..., $S.format(..., request.$W[...], ...),
 ...)
            - pattern: $A = django.shortcuts.redirect(..., $S % request.$W[...], ...)
            - pattern: $A = django.shortcuts.redirect(..., f"...{request.$W[...]}...", ...)
            - pattern: return django.shortcuts.redirect(..., request.$W[...], ...)
            - pattern: return django.shortcuts.redirect(..., $S.format(..., request.$W[...], ...
), ...)
            - pattern: return django.shortcuts.redirect(..., $S % request.$W[...], ...)
            - pattern: return django.shortcuts.redirect(..., f"...{request.$W[...]}...", ...)
            - pattern: django.shortcuts.redirect(..., request.$W, ...)
            - pattern: django.shortcuts.redirect(..., $S.format(..., request.$W, ...), ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: django.shortcuts.redirect(..., $S % request.$W, ...)
            - pattern: django.shortcuts.redirect(..., f"...{request.$W}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.shortcuts.redirect(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $DATA
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.shortcuts.redirect(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.shortcuts.redirect(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % $DATA
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.shortcuts.redirect(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.shortcuts.redirect(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                django.shortcuts.redirect(..., $INTERM, ...)
            - pattern: $A = django.shortcuts.redirect(..., request.$W, ...)
            - pattern: $A = django.shortcuts.redirect(..., $S.format(..., request.$W, ...), ...)
            - pattern: $A = django.shortcuts.redirect(..., $S % request.$W, ...)
            - pattern: $A = django.shortcuts.redirect(..., f"...{request.$W}...", ...)
            - pattern: return django.shortcuts.redirect(..., request.$W, ...)
            - pattern: return django.shortcuts.redirect(..., $S.format(..., request.$W, ...), ..
.)
            - pattern: return django.shortcuts.redirect(..., $S % request.$W, ...)
            - pattern: return django.shortcuts.redirect(..., f"...{request.$W}...", ...)
            - pattern: django.http.HttpResponseRedirect(..., request.$W.get(...), ...)
            - pattern: django.http.HttpResponseRedirect(..., $S.format(..., request.$W.get(...),
 ...), ...)
            - pattern: django.http.HttpResponseRedirect(..., $S % request.$W.get(...), ...)
            - pattern: django.http.HttpResponseRedirect(..., f"...{request.$W.get(...)}...", ...
)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponseRedirect(..., $DATA, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponseRedirect(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponseRedirect(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % $DATA
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponseRedirect(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponseRedirect(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR + $DATA
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., request.$W.get(...), ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., $S.format(..., request.$W.get(
...), ...), ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., $S % request.$W.get(...), ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., f"...{request.$W.get(...)}..."
, ...)
            - pattern: return django.http.HttpResponseRedirect(..., request.$W.get(...), ...)
            - pattern: return django.http.HttpResponseRedirect(..., $S.format(..., request.$W.ge
t(...), ...), ...)
            - pattern: return django.http.HttpResponseRedirect(..., $S % request.$W.get(...), ..
.)
            - pattern: return django.http.HttpResponseRedirect(..., f"...{request.$W.get(...)}..
.", ...)
            - pattern: django.http.HttpResponseRedirect(..., request.$W(...), ...)
            - pattern: django.http.HttpResponseRedirect(..., $S.format(..., request.$W(...), ...
), ...)
            - pattern: django.http.HttpResponseRedirect(..., $S % request.$W(...), ...)
            - pattern: django.http.HttpResponseRedirect(..., f"...{request.$W(...)}...", ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.http.HttpResponseRedirect(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            $INTERM = $DATA
            ...
            django.http.HttpResponseRedirect(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            django.http.HttpResponseRedirect(..., $STR.format(..., $DATA, ...), ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            $INTERM = $STR.format(..., $DATA, ...)
            ...
            django.http.HttpResponseRedirect(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            django.http.HttpResponseRedirect(..., $STR % $DATA, ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            $INTERM = $STR % $DATA
            ...
            django.http.HttpResponseRedirect(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            django.http.HttpResponseRedirect(..., f"...{$DATA}...", ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            $INTERM = f"...{$DATA}..."
            ...
            django.http.HttpResponseRedirect(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            django.http.HttpResponseRedirect(..., $STR + $DATA, ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            $INTERM = $STR + $DATA
            ...
            django.http.HttpResponseRedirect(..., $INTERM, ...)
        - pattern: $A = django.http.HttpResponseRedirect(..., request.$W(...), ...)
        - pattern: $A = django.http.HttpResponseRedirect(..., $S.format(..., request.$W(...)
, ...), ...)
        - pattern: $A = django.http.HttpResponseRedirect(..., $S % request.$W(...), ...)
        - pattern: $A = django.http.HttpResponseRedirect(..., f"...{request.$W(...)}...", ..
.)
        - pattern: return django.http.HttpResponseRedirect(..., request.$W(...), ...)
        - pattern: return django.http.HttpResponseRedirect(..., $S.format(..., request.$W(..
.), ...), ...)
        - pattern: return django.http.HttpResponseRedirect(..., $S % request.$W(...), ...)
        - pattern: return django.http.HttpResponseRedirect(..., f"...{request.$W(...)}...",
...)
        - pattern: django.http.HttpResponseRedirect(..., request.$W[...], ...)
        - pattern: django.http.HttpResponseRedirect(..., $S.format(..., request.$W[...], ...
), ...)
        - pattern: django.http.HttpResponseRedirect(..., $S % request.$W[...], ...)
        - pattern: django.http.HttpResponseRedirect(..., f"...{request.$W[...]}...", ...)
        - pattern: |
            $DATA = request.$W[...]
            ...
            django.http.HttpResponseRedirect(..., $DATA, ...)
        - pattern: |
            $DATA = request.$W[...]
            ...
            $INTERM = $DATA
            ...
            django.http.HttpResponseRedirect(..., $INTERM, ...)
        - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                $DATA = request.$W[...]
                ...
                django.http.HttpResponseRedirect(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.http.HttpResponseRedirect(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR % $DATA
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.http.HttpResponseRedirect(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.http.HttpResponseRedirect(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR + $DATA
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., request.$W[...], ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., $S.format(..., request.$W[...]
, ...), ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., $S % request.$W[...], ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., f"...{request.$W[...]}...", ..
.)
            - pattern: return django.http.HttpResponseRedirect(..., request.$W[...], ...)
            - pattern: return django.http.HttpResponseRedirect(..., $S.format(..., request.$W[..
.], ...), ...)
            - pattern: return django.http.HttpResponseRedirect(..., $S % request.$W[...], ...)
            - pattern: return django.http.HttpResponseRedirect(..., f"...{request.$W[...]}...",
...)
            - pattern: django.http.HttpResponseRedirect(..., request.$W, ...)
            - pattern: django.http.HttpResponseRedirect(..., $S.format(..., request.$W, ...), ..
.)
            - pattern: django.http.HttpResponseRedirect(..., $S % request.$W, ...)
            - pattern: django.http.HttpResponseRedirect(..., f"...{request.$W}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseRedirect(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $DATA
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseRedirect(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                $DATA = request.$W
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseRedirect(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % $DATA
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseRedirect(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseRedirect(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                django.http.HttpResponseRedirect(..., $INTERM, ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., request.$W, ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., $S.format(..., request.$W, ...
), ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., $S % request.$W, ...)
            - pattern: $A = django.http.HttpResponseRedirect(..., f"...{request.$W}...", ...)
            - pattern: return django.http.HttpResponseRedirect(..., request.$W, ...)
            - pattern: return django.http.HttpResponseRedirect(..., $S.format(..., request.$W, .
..), ...)
            - pattern: return django.http.HttpResponseRedirect(..., $S % request.$W, ...)
            - pattern: return django.http.HttpResponseRedirect(..., f"...{request.$W}...", ...)
        - metavariable-regex:
            metavariable: $W
            regex: (?!get_full_path)
      severity: WARNING
    - id: python.django.security.injection.path-traversal.path-traversal-open.path-traversal-ope
n
      languages:
        - python
      message: Found request data in a call to 'open'. Ensure the request data is validated or s
anitized, otherwise it could result in path traversal attacks and therefore sensitive data being
 leaked. To mitigate, consider using os.path.abspath or os.path.realpath or the pathlib library.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/www-community/attacks/Path_Traversal
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
  subcategory:
      - vuln
  technology:
      - django
patterns:
  - pattern-inside: |
      def $FUNC(...):
        ...
  - pattern-either:
      - pattern: open(..., request.$W.get(...), ...)
      - pattern: open(..., $S.format(..., request.$W.get(...), ...), ...)
      - pattern: open(..., $S % request.$W.get(...), ...)
      - pattern: open(..., f"...{request.$W.get(...)}...", ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          open(..., $DATA, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $DATA
          ...
          open(..., $INTERM, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $DATA
          ...
          with open(..., $INTERM, ...) as $FD:
            ...
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          open(..., $STR.format(..., $DATA, ...), ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $STR.format(..., $DATA, ...)
          ...
          open(..., $INTERM, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $STR.format(..., $DATA, ...)
          ...
          with open(..., $INTERM, ...) as $FD:
            ...
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          open(..., $STR % $DATA, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $STR % $DATA
          ...
          open(..., $INTERM, ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          $INTERM = $STR % $DATA
          ...
          with open(..., $INTERM, ...) as $FD:
            ...
      - pattern: |
          $DATA = request.$W.get(...)
          ...
          open(..., f"...{$DATA}...", ...)
      - pattern: |
          $DATA = request.$W.get(...)
          ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    $INTERM = f"...{$DATA}..."
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W.get(...)
    ...
    open(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $INTERM = $STR + $DATA
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $INTERM = $STR + $DATA
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: $A = open(..., request.$W.get(...), ...)
- pattern: $A = open(..., $S.format(..., request.$W.get(...), ...), ...)
- pattern: $A = open(..., $S % request.$W.get(...), ...)
- pattern: $A = open(..., f"...{request.$W.get(...)}...", ...)
- pattern: return open(..., request.$W.get(...), ...)
- pattern: return open(..., $S.format(..., request.$W.get(...), ...), ...)
- pattern: return open(..., $S % request.$W.get(...), ...)
- pattern: return open(..., f"...{request.$W.get(...)}...", ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    with open(..., $DATA, ...) as $FD:
      ...
- pattern: open(..., request.$W(...), ...)
- pattern: open(..., $S.format(..., request.$W(...), ...), ...)
- pattern: open(..., $S % request.$W(...), ...)
- pattern: open(..., f"...{request.$W(...)}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    open(..., $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $DATA
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $DATA
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W(...)
    ...
    open(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    open(..., $INTERM, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W(...)
    ...
    open(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W(...)
    ...
    open(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W(...)
    ...
    open(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR + $DATA
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR + $DATA
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: $A = open(..., request.$W(...), ...)
- pattern: $A = open(..., $S.format(..., request.$W(...), ...), ...)
- pattern: $A = open(..., $S % request.$W(...), ...)
- pattern: $A = open(..., f"...{request.$W(...)}...", ...)
- pattern: return open(..., request.$W(...), ...)
- pattern: return open(..., $S.format(..., request.$W(...), ...), ...)
- pattern: return open(..., $S % request.$W(...), ...)
- pattern: return open(..., f"...{request.$W(...)}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    with open(..., $DATA, ...) as $FD:
      ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
- pattern: open(..., request.$W[...], ...)
- pattern: open(..., $S.format(..., request.$W[...], ...), ...)
- pattern: open(..., $S % request.$W[...], ...)
- pattern: open(..., f"...{request.$W[...]}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    open(..., $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W[...]
    ...
    open(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W[...]
    ...
    open(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W[...]
    ...
    open(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    with open(..., $INTERM, ...) as $FD:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      ...
- pattern: |
    $DATA = request.$W[...]
    ...
    open(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR + $DATA
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR + $DATA
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: $A = open(..., request.$W[...], ...)
- pattern: $A = open(..., $S.format(..., request.$W[...], ...), ...)
- pattern: $A = open(..., $S % request.$W[...], ...)
- pattern: $A = open(..., f"...{request.$W[...]}...", ...)
- pattern: return open(..., request.$W[...], ...)
- pattern: return open(..., $S.format(..., request.$W[...], ...), ...)
- pattern: return open(..., $S % request.$W[...], ...)
- pattern: return open(..., f"...{request.$W[...]}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    with open(..., $DATA, ...) as $FD:
      ...
- pattern: open(..., request.$W, ...)
- pattern: open(..., $S.format(..., request.$W, ...), ...)
- pattern: open(..., $S % request.$W, ...)
- pattern: open(..., f"...{request.$W}...", ...)
- pattern: |
    $DATA = request.$W
    ...
    open(..., $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W
    ...
    open(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    open(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    with open(..., $INTERM, ...) as $FD:
      ...
- pattern: |
    $DATA = request.$W
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
            open(..., $STR % $DATA, ...)
        - pattern: |
            $DATA = request.$W
            ...
            $INTERM = $STR % $DATA
            ...
            open(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W
            ...
            $INTERM = $STR % $DATA
            ...
            with open(..., $INTERM, ...) as $FD:
              ...
        - pattern: |
            $DATA = request.$W
            ...
            open(..., f"...{$DATA}...", ...)
        - pattern: |
            $DATA = request.$W
            ...
            $INTERM = f"...{$DATA}..."
            ...
            open(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W
            ...
            $INTERM = f"...{$DATA}..."
            ...
            with open(..., $INTERM, ...) as $FD:
              ...
        - pattern: |
            $DATA = request.$W
            ...
            open(..., $STR + $DATA, ...)
        - pattern: |
            $DATA = request.$W
            ...
            $INTERM = $STR + $DATA
            ...
            open(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W
            ...
            $INTERM = $STR + $DATA
            ...
            with open(..., $INTERM, ...) as $FD:
              ...
        - pattern: $A = open(..., request.$W, ...)
        - pattern: $A = open(..., $S.format(..., request.$W, ...), ...)
        - pattern: $A = open(..., $S % request.$W, ...)
        - pattern: $A = open(..., f"...{request.$W}...", ...)
        - pattern: return open(..., request.$W, ...)
        - pattern: return open(..., $S.format(..., request.$W, ...), ...)
        - pattern: return open(..., $S % request.$W, ...)
        - pattern: return open(..., f"...{request.$W}...", ...)
        - pattern: |
            $DATA = request.$W
            ...
            with open(..., $DATA, ...) as $FD:
              ...
      severity: WARNING
    - id: python.django.security.injection.raw-html-format.raw-html-format
      languages:
        - python
      message: Detected user input flowing into a manually constructed HTML string. You may be a
ccidentally bypassing secure methods of rendering HTML by manually constructing HTML and this co
uld create a cross-site scripting vulnerability, which could let attackers steal sensitive user
data. To be sure this is safe, check that the HTML is rendered safely. Otherwise, use templates
(`django.shortcuts.render`) which will safely render HTML instead.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://docs.djangoproject.com/en/3.2/topics/http/shortcuts/#render
            - https://docs.djangoproject.com/en/3.2/topics/security/#cross-site-scripting-xss-pr
otection
        subcategory:
            - vuln
        technology:
            - django
      mode: taint
      pattern-sanitizers:
        - pattern: django.utils.html.escape(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: '"$HTMLSTR" % ...'
                        - pattern: '"$HTMLSTR".format(...)'
                        - pattern: '"$HTMLSTR" + ...'
                        - pattern: f"$HTMLSTR{...}..."
                - patterns:
                    - pattern-inside: |
                        $HTML = "$HTMLSTR"
                        ...
                    - pattern-either:
                        - pattern: $HTML % ...
                        - pattern: $HTML.format(...)
                        - pattern: $HTML + ...
            - metavariable-pattern:
                language: generic
                metavariable: $HTMLSTR
                pattern: <$TAG ...
      pattern-sources:
        - patterns:
            - pattern: request.$ANYTHING
            - pattern-not: request.build_absolute_uri
      severity: WARNING
    - id: python.django.security.injection.reflected-data-httpresponse.reflected-data-httprespon
se
      languages:
        - python
      message: Found user-controlled request data passed into HttpResponse. This could be vulner
able to XSS, leading to attackers gaining access to user cookies and protected information. Ensu
re that the request data is properly escaped or sanitzed.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        references:
            - https://django-book.readthedocs.io/en/latest/chapter20.html#cross-site-scripting-x
ss
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
              ...
        - pattern-either:
            - pattern: django.http.HttpResponse(..., $S.format(..., request.$W.get(...), ...), .
..)
            - pattern: django.http.HttpResponse(..., $S % request.$W.get(...), ...)
            - pattern: django.http.HttpResponse(..., f"...{request.$W.get(...)}...", ...)
            - pattern: django.http.HttpResponse(..., request.$W.get(...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponse(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                django.http.HttpResponse(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponse(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.http.HttpResponse(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponse(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % $DATA
                ...
                django.http.HttpResponse(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponse(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.http.HttpResponse(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponse(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR + $DATA
                ...
                django.http.HttpResponse(..., $INTERM, ...)
            - pattern: $A = django.http.HttpResponse(..., request.$W.get(...), ...)
            - pattern: return django.http.HttpResponse(..., request.$W.get(...), ...)
            - pattern: django.http.HttpResponse(..., $S.format(..., request.$W(...), ...), ...)
            - pattern: django.http.HttpResponse(..., $S % request.$W(...), ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
- pattern: django.http.HttpResponse(..., f"...{request.$W(...)}...", ...)
- pattern: django.http.HttpResponse(..., request.$W(...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    django.http.HttpResponse(..., $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $DATA
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    django.http.HttpResponse(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    django.http.HttpResponse(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    django.http.HttpResponse(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    django.http.HttpResponse(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR + $DATA
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: $A = django.http.HttpResponse(..., request.$W(...), ...)
- pattern: return django.http.HttpResponse(..., request.$W(...), ...)
- pattern: django.http.HttpResponse(..., $S.format(..., request.$W[...], ...), ...)
- pattern: django.http.HttpResponse(..., $S % request.$W[...], ...)
- pattern: django.http.HttpResponse(..., f"...{request.$W[...]}...", ...)
- pattern: django.http.HttpResponse(..., request.$W[...], ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.http.HttpResponse(..., $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    django.http.HttpResponse(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.http.HttpResponse(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.http.HttpResponse(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    django.http.HttpResponse(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR + $DATA
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: $A = django.http.HttpResponse(..., request.$W[...], ...)
- pattern: return django.http.HttpResponse(..., request.$W[...], ...)
- pattern: django.http.HttpResponse(..., $S.format(..., request.$W, ...), ...)
- pattern: django.http.HttpResponse(..., $S % request.$W, ...)
- pattern: django.http.HttpResponse(..., f"...{request.$W}...", ...)
- pattern: django.http.HttpResponse(..., request.$W, ...)
- pattern: |
    $DATA = request.$W
    ...
    django.http.HttpResponse(..., $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    django.http.HttpResponse(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    django.http.HttpResponse(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    django.http.HttpResponse(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR % $DATA
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                django.http.HttpResponse(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponse(..., f"...{$DATA}...", ...)
            - pattern: $A = django.http.HttpResponse(..., request.$W, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                $A = django.http.HttpResponse(..., $INTERM, ...)
            - pattern: return django.http.HttpResponse(..., request.$W, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.http.HttpResponse(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponse(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                django.http.HttpResponse(..., $INTERM, ...)
      severity: WARNING
    - id: python.django.security.injection.reflected-data-httpresponsebadrequest.reflected-data-
httpresponsebadrequest
      languages:
        - python
      message: Found user-controlled request data passed into a HttpResponseBadRequest. This cou
ld be vulnerable to XSS, leading to attackers gaining access to user cookies and protected infor
mation. Ensure that the request data is properly escaped or sanitzed.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://django-book.readthedocs.io/en/latest/chapter20.html#cross-site-scripting-x
ss
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
              ...
        - pattern-either:
            - pattern: django.http.HttpResponseBadRequest(..., $S.format(..., request.$W.get(...
), ...), ...)
            - pattern: django.http.HttpResponseBadRequest(..., $S % request.$W.get(...), ...)
            - pattern: django.http.HttpResponseBadRequest(..., f"...{request.$W.get(...)}...", .
..)
            - pattern: django.http.HttpResponseBadRequest(..., request.$W.get(...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                django.http.HttpResponseBadRequest(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponseBadRequest(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponseBadRequest(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponseBadRequest(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.HttpResponseBadRequest(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR + $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: $A = django.http.HttpResponseBadRequest(..., request.$W.get(...), ...)
            - pattern: return django.http.HttpResponseBadRequest(..., request.$W.get(...), ...)
            - pattern: django.http.HttpResponseBadRequest(..., $S.format(..., request.$W(...), .
..), ...)
            - pattern: django.http.HttpResponseBadRequest(..., $S % request.$W(...), ...)
            - pattern: django.http.HttpResponseBadRequest(..., f"...{request.$W(...)}...", ...)
            - pattern: django.http.HttpResponseBadRequest(..., request.$W(...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.http.HttpResponseBadRequest(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.http.HttpResponseBadRequest(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.http.HttpResponseBadRequest(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR % $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.http.HttpResponseBadRequest(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.http.HttpResponseBadRequest(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR + $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: $A = django.http.HttpResponseBadRequest(..., request.$W(...), ...)
            - pattern: return django.http.HttpResponseBadRequest(..., request.$W(...), ...)
            - pattern: django.http.HttpResponseBadRequest(..., $S.format(..., request.$W[...], .
..), ...)
            - pattern: django.http.HttpResponseBadRequest(..., $S % request.$W[...], ...)
            - pattern: django.http.HttpResponseBadRequest(..., f"...{request.$W[...]}...", ...)
            - pattern: django.http.HttpResponseBadRequest(..., request.$W[...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.http.HttpResponseBadRequest(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.http.HttpResponseBadRequest(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.http.HttpResponseBadRequest(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR % $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.http.HttpResponseBadRequest(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.http.HttpResponseBadRequest(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR + $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: $A = django.http.HttpResponseBadRequest(..., request.$W[...], ...)
            - pattern: return django.http.HttpResponseBadRequest(..., request.$W[...], ...)
            - pattern: django.http.HttpResponseBadRequest(..., $S.format(..., request.$W, ...),
...)
            - pattern: django.http.HttpResponseBadRequest(..., $S % request.$W, ...)
            - pattern: django.http.HttpResponseBadRequest(..., f"...{request.$W}...", ...)
            - pattern: django.http.HttpResponseBadRequest(..., request.$W, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseBadRequest(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseBadRequest(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseBadRequest(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseBadRequest(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.HttpResponseBadRequest(..., $STR + $DATA, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                django.http.HttpResponseBadRequest(..., $INTERM, ...)
            - pattern: $A = django.http.HttpResponseBadRequest(..., request.$W, ...)
            - pattern: return django.http.HttpResponseBadRequest(..., request.$W, ...)
      severity: WARNING
    - id: python.django.security.injection.request-data-fileresponse.request-data-fileresponse
      languages:
        - python
      message: Found user-controlled request data being passed into a file open, which is them p
assed as an argument into the FileResponse. This is dangerous because an attacker could specify
an arbitrary file to read, which could result in leaking important data. Be sure to validate or
sanitize the user-inputted filename in the request data before using it in FileResponse.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://django-book.readthedocs.io/en/latest/chapter20.html#cross-site-scripting-x
ss
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
              ...
        - pattern-either:
            - pattern: django.http.FileResponse(..., request.$W.get(...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.http.FileResponse(..., open($DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = open($DATA, ...)
                ...
                django.http.FileResponse(..., $INTERM, ...)
            - pattern: $A = django.http.FileResponse(..., request.$W.get(...), ...)
            - pattern: return django.http.FileResponse(..., request.$W.get(...), ...)
            - pattern: django.http.FileResponse(..., request.$W(...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.http.FileResponse(..., open($DATA, ...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = open($DATA, ...)
                ...
                django.http.FileResponse(..., $INTERM, ...)
            - pattern: $A = django.http.FileResponse(..., request.$W(...), ...)
            - pattern: return django.http.FileResponse(..., request.$W(...), ...)
            - pattern: django.http.FileResponse(..., request.$W[...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                django.http.FileResponse(..., open($DATA, ...), ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = open($DATA, ...)
                ...
                django.http.FileResponse(..., $INTERM, ...)
            - pattern: $A = django.http.FileResponse(..., request.$W[...], ...)
            - pattern: return django.http.FileResponse(..., request.$W[...], ...)
            - pattern: django.http.FileResponse(..., request.$W, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.http.FileResponse(..., open($DATA, ...), ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = open($DATA, ...)
                ...
                django.http.FileResponse(..., $INTERM, ...)
            - pattern: $A = django.http.FileResponse(..., request.$W, ...)
            - pattern: return django.http.FileResponse(..., request.$W, ...)
      severity: WARNING
    - id: python.django.security.injection.request-data-write.request-data-write
      languages:
        - python
      message: Found user-controlled request data passed into '.write(...)'. This could be dange
rous if a malicious actor is able to control data into sensitive files. For example, a malicious
 actor could force rolling of critical log files, or cause a denial-of-service by using up avail
able disk space. Instead, ensure that request data is properly escaped or sanitized.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-93: Improper Neutralization of CRLF Sequences (''CRLF Injection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - django
      pattern-either:
        - pattern: $F.write(..., request.$W.get(...), ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $F.write(..., $DATA, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $INTERM = $DATA
            ...
            $F.write(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $F.write(..., $B.$C(..., $DATA, ...), ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $INTERM = $B.$C(..., $DATA, ...)
            ...
            $F.write(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $F.write(..., $STR % $DATA, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $INTERM = $STR % $DATA
    ...
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $F.write(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    $F.write(..., $INTERM, ...)
- pattern: $A = $F.write(..., request.$W.get(...), ...)
- pattern: return $F.write(..., request.$W.get(...), ...)
- pattern: $F.write(..., request.$W(...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $F.write(..., $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $DATA
    ...
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $F.write(..., $B.$C(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $B.$C(..., $DATA, ...)
    ...
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $F.write(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $F.write(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    $F.write(..., $INTERM, ...)
- pattern: $A = $F.write(..., request.$W(...), ...)
- pattern: return $F.write(..., request.$W(...), ...)
- pattern: $F.write(..., request.$W[...], ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $F.write(..., $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $F.write(..., $B.$C(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $B.$C(..., $DATA, ...)
    ...
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $F.write(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $F.write(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    $F.write(..., $INTERM, ...)
- pattern: $A = $F.write(..., request.$W[...], ...)
- pattern: return $F.write(..., request.$W[...], ...)
- pattern: $F.write(..., request.$W, ...)
- pattern: |
    $DATA = request.$W
    ...
    $F.write(..., $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $F.write(..., $B.$C(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $B.$C(..., $DATA, ...)
    ...
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $F.write(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR % $DATA
    ...
    $F.write(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $F.write(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W
    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            $INTERM = f"...{$DATA}..."
            ...
            $F.write(..., $INTERM, ...)
        - pattern: $A = $F.write(..., request.$W, ...)
        - pattern: return $F.write(..., request.$W, ...)
      severity: WARNING
    - id: python.django.security.injection.sql.sql-injection-extra.sql-injection-using-extra-whe
re
      languages:
        - python
      message: User-controlled data from a request is passed to 'extra()'. This could lead to a
SQL injection and therefore protected information could be leaked. Instead, use parameterized qu
eries or escape the user-controlled data by using `params` and not using quote placeholders in t
he SQL string.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.djangoproject.com/en/3.0/ref/models/expressions/#.objects.extra
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
              ...
        - pattern-either:
            - pattern: $MODEL.objects.extra(..., where=[..., $S.format(..., request.$W.get(...),
 ...), ...], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., $S % request.$W.get(...), ...], ...
)
            - pattern: $MODEL.objects.extra(..., where=[..., f"...{request.$W.get(...)}...", ...
], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., request.$W.get(...), ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $MODEL.objects.extra(..., where=[..., $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $MODEL.objects.extra(..., where=[..., $STR.format(..., $DATA, ...), ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $MODEL.objects.extra(..., where=[..., $STR % $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                $INTERM = $STR % $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $MODEL.objects.extra(..., where=[..., f"...{$DATA}...", ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $MODEL.objects.extra(..., where=[..., $STR + $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR + $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: $A = $MODEL.objects.extra(..., where=[..., request.$W.get(...), ...], ...
)
            - pattern: return $MODEL.objects.extra(..., where=[..., request.$W.get(...), ...], .
..)
            - pattern: $MODEL.objects.extra(..., where=[..., $S.format(..., request.$W(...), ...
), ...], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., $S % request.$W(...), ...], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., f"...{request.$W(...)}...", ...], .
..)
            - pattern: $MODEL.objects.extra(..., where=[..., request.$W(...), ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $MODEL.objects.extra(..., where=[..., $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $MODEL.objects.extra(..., where=[..., $STR.format(..., $DATA, ...), ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $MODEL.objects.extra(..., where=[..., $STR % $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR % $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $MODEL.objects.extra(..., where=[..., f"...{$DATA}...", ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                $INTERM = f"...{$DATA}..."
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $MODEL.objects.extra(..., where=[..., $STR + $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR + $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: $A = $MODEL.objects.extra(..., where=[..., request.$W(...), ...], ...)
            - pattern: return $MODEL.objects.extra(..., where=[..., request.$W(...), ...], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., $S.format(..., request.$W[...], ...
), ...], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., $S % request.$W[...], ...], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., f"...{request.$W[...]}...", ...], .
..)
            - pattern: $MODEL.objects.extra(..., where=[..., request.$W[...], ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $MODEL.objects.extra(..., where=[..., $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $MODEL.objects.extra(..., where=[..., $STR.format(..., $DATA, ...), ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $MODEL.objects.extra(..., where=[..., $STR % $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR % $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $MODEL.objects.extra(..., where=[..., f"...{$DATA}...", ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = f"...{$DATA}..."
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $MODEL.objects.extra(..., where=[..., $STR + $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR + $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: $A = $MODEL.objects.extra(..., where=[..., request.$W[...], ...], ...)
            - pattern: return $MODEL.objects.extra(..., where=[..., request.$W[...], ...], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., $S.format(..., request.$W, ...), ..
.], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., $S % request.$W, ...], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., f"...{request.$W}...", ...], ...)
            - pattern: $MODEL.objects.extra(..., where=[..., request.$W, ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $MODEL.objects.extra(..., where=[..., $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $MODEL.objects.extra(..., where=[..., $STR.format(..., $DATA, ...), ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $MODEL.objects.extra(..., where=[..., $STR % $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $MODEL.objects.extra(..., where=[..., f"...{$DATA}...", ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $MODEL.objects.extra(..., where=[..., $STR + $DATA, ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: $A = $MODEL.objects.extra(..., where=[..., request.$W, ...], ...)
            - pattern: return $MODEL.objects.extra(..., where=[..., request.$W, ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $MODEL.objects.extra(..., where=[..., $STR % (..., $DATA, ...), ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $MODEL.objects.extra(..., where=[..., $STR % (..., $DATA, ...), ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $MODEL.objects.extra(..., where=[..., $STR % (..., $DATA, ...), ...], ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                $DATA = request.$W
                ...
                $MODEL.objects.extra(..., where=[..., $STR % (..., $DATA, ...), ...], ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % (..., $DATA, ...)
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR % (..., $DATA, ...)
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR % (..., $DATA, ...)
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % (..., $DATA, ...)
                ...
                $MODEL.objects.extra(..., where=[..., $INTERM, ...], ...)
      severity: WARNING
    - id: python.django.security.injection.sql.sql-injection-rawsql.sql-injection-using-rawsql
      languages:
        - python
      message: User-controlled data from request is passed to 'RawSQL()'. This could lead to a S
QL injection and therefore protected information could be leaked. Instead, use parameterized que
ries or escape the user-controlled data by using `params` and not using quote placeholders in th
e SQL string.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.djangoproject.com/en/3.0/ref/models/expressions/#django.db.models.exp
ressions.RawSQL
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
              ...
        - pattern-either:
            - pattern: django.db.models.expressions.RawSQL(..., $S.format(..., request.$W.get(..
.), ...), ...)
            - pattern: django.db.models.expressions.RawSQL(..., $S % request.$W.get(...), ...)
            - pattern: django.db.models.expressions.RawSQL(..., f"...{request.$W.get(...)}...",
...)
            - pattern: django.db.models.expressions.RawSQL(..., request.$W.get(...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.db.models.expressions.RawSQL(..., $DATA, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.db.models.expressions.RawSQL(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.db.models.expressions.RawSQL(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.db.models.expressions.RawSQL(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.db.models.expressions.RawSQL(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR + $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: $A = django.db.models.expressions.RawSQL(..., request.$W.get(...), ...)
            - pattern: return django.db.models.expressions.RawSQL(..., request.$W.get(...), ...)
            - pattern: django.db.models.expressions.RawSQL(..., $S.format(..., request.$W(...),
...), ...)
            - pattern: django.db.models.expressions.RawSQL(..., $S % request.$W(...), ...)
            - pattern: django.db.models.expressions.RawSQL(..., f"...{request.$W(...)}...", ...)
            - pattern: django.db.models.expressions.RawSQL(..., request.$W(...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.db.models.expressions.RawSQL(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.db.models.expressions.RawSQL(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.db.models.expressions.RawSQL(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR % $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.db.models.expressions.RawSQL(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.db.models.expressions.RawSQL(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR + $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: $A = django.db.models.expressions.RawSQL(..., request.$W(...), ...)
            - pattern: return django.db.models.expressions.RawSQL(..., request.$W(...), ...)
            - pattern: django.db.models.expressions.RawSQL(..., $S.format(..., request.$W[...],
...), ...)
            - pattern: django.db.models.expressions.RawSQL(..., $S % request.$W[...], ...)
            - pattern: django.db.models.expressions.RawSQL(..., f"...{request.$W[...]}...", ...)
            - pattern: django.db.models.expressions.RawSQL(..., request.$W[...], ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.db.models.expressions.RawSQL(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.db.models.expressions.RawSQL(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.db.models.expressions.RawSQL(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR % $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                django.db.models.expressions.RawSQL(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.db.models.expressions.RawSQL(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR + $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: $A = django.db.models.expressions.RawSQL(..., request.$W[...], ...)
            - pattern: return django.db.models.expressions.RawSQL(..., request.$W[...], ...)
            - pattern: django.db.models.expressions.RawSQL(..., $S.format(..., request.$W, ...),
...)
            - pattern: django.db.models.expressions.RawSQL(..., $S % request.$W, ...)
            - pattern: django.db.models.expressions.RawSQL(..., f"...{request.$W}...", ...)
            - pattern: django.db.models.expressions.RawSQL(..., request.$W, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.db.models.expressions.RawSQL(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.db.models.expressions.RawSQL(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.db.models.expressions.RawSQL(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.db.models.expressions.RawSQL(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.db.models.expressions.RawSQL(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                $INTERM = $STR + $DATA
                ...
                django.db.models.expressions.RawSQL(..., $INTERM, ...)
            - pattern: $A = django.db.models.expressions.RawSQL(..., request.$W, ...)
            - pattern: return django.db.models.expressions.RawSQL(..., request.$W, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                django.db.models.expressions.RawSQL($STR % (..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                django.db.models.expressions.RawSQL($STR % (..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                django.db.models.expressions.RawSQL($STR % (..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W
                ...
                django.db.models.expressions.RawSQL($STR % (..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % (..., $DATA, ...)
                ...
                django.db.models.expressions.RawSQL($INTERM, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $STR % (..., $DATA, ...)
                ...
                django.db.models.expressions.RawSQL($INTERM, ...)
            - pattern: |
                $DATA = request.$W[...]
                ...
                $INTERM = $STR % (..., $DATA, ...)
                ...
                django.db.models.expressions.RawSQL($INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % (..., $DATA, ...)
                ...
                django.db.models.expressions.RawSQL($INTERM, ...)
      severity: WARNING
    - id: python.django.security.injection.sql.sql-injection-using-db-cursor-execute.sql-injecti
on-db-cursor-execute
      languages:
        - python
      message: User-controlled data from a request is passed to 'execute()'. This could lead to
a SQL injection and therefore protected information could be leaked. Instead, use django's Query
Sets, which are built with query parameterization and therefore not vulnerable to sql injection.
 For example, you could use `Entry.objects.filter(date=2006)`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.djangoproject.com/en/3.0/topics/security/#sql-injection-protection
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - vuln
      technology:
        - django
    patterns:
      - pattern-inside: |
          def $FUNC(...):
            ...
      - pattern-either:
        - pattern: $CURSOR.execute(..., $S.format(..., request.$W.get(...), ...), ...)
        - pattern: $CURSOR.execute(..., $S % request.$W.get(...), ...)
        - pattern: $CURSOR.execute(..., f"...{request.$W.get(...)}...", ...)
        - pattern: $CURSOR.execute(..., request.$W.get(...), ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $CURSOR.execute(..., $DATA, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $INTERM = $DATA
            ...
            $CURSOR.execute(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $CURSOR.execute(..., $STR.format(..., $DATA, ...), ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $INTERM = $STR.format(..., $DATA, ...)
            ...
            $CURSOR.execute(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $CURSOR.execute(..., $STR % $DATA, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $INTERM = $STR % $DATA
            ...
            $CURSOR.execute(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $CURSOR.execute(..., f"...{$DATA}...", ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $INTERM = f"...{$DATA}..."
            ...
            $CURSOR.execute(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $CURSOR.execute(..., $STR + $DATA, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $INTERM = $STR + $DATA
            ...
            $CURSOR.execute(..., $INTERM, ...)
        - pattern: $A = $CURSOR.execute(..., request.$W.get(...), ...)
        - pattern: return $CURSOR.execute(..., request.$W.get(...), ...)
        - pattern: $CURSOR.execute(..., $S.format(..., request.$W(...), ...), ...)
        - pattern: $CURSOR.execute(..., $S % request.$W(...), ...)
        - pattern: $CURSOR.execute(..., f"...{request.$W(...)}...", ...)
        - pattern: $CURSOR.execute(..., request.$W(...), ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    $CURSOR.execute(..., $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $DATA
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $CURSOR.execute(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $CURSOR.execute(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $CURSOR.execute(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $CURSOR.execute(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR + $DATA
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: $A = $CURSOR.execute(..., request.$W(...), ...)
- pattern: return $CURSOR.execute(..., request.$W(...), ...)
- pattern: $CURSOR.execute(..., $S.format(..., request.$W[...], ...), ...)
- pattern: $CURSOR.execute(..., $S % request.$W[...], ...)
- pattern: $CURSOR.execute(..., f"...{request.$W[...]}...", ...)
- pattern: $CURSOR.execute(..., request.$W[...], ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $CURSOR.execute(..., $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $CURSOR.execute(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR.format(..., $DATA, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $CURSOR.execute(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $CURSOR.execute(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $CURSOR.execute(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR + $DATA
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: $A = $CURSOR.execute(..., request.$W[...], ...)
- pattern: return $CURSOR.execute(..., request.$W[...], ...)
- pattern: $CURSOR.execute(..., $S.format(..., request.$W, ...), ...)
- pattern: $CURSOR.execute(..., $S % request.$W, ...)
- pattern: $CURSOR.execute(..., f"...{request.$W}...", ...)
- pattern: $CURSOR.execute(..., request.$W, ...)
- pattern: |
    $DATA = request.$W
    ...
    $CURSOR.execute(..., $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $CURSOR.execute(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $CURSOR.execute(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR % $DATA
    ...
    $CURSOR.execute(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            $CURSOR.execute(..., f"...{$DATA}...", ...)
        - pattern: |
            $DATA = request.$W
            ...
            $INTERM = f"...{$DATA}..."
            ...
            $CURSOR.execute(..., $INTERM, ...)
        - pattern: |
            $DATA = request.$W
            ...
            $CURSOR.execute(..., $STR + $DATA, ...)
        - pattern: |
            $DATA = request.$W
            ...
            $INTERM = $STR + $DATA
            ...
            $CURSOR.execute(..., $INTERM, ...)
        - pattern: $A = $CURSOR.execute(..., request.$W, ...)
        - pattern: return $CURSOR.execute(..., request.$W, ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $CURSOR.execute($STR % (..., $DATA, ...), ...)
        - pattern: |
            $DATA = request.$W[...]
            ...
            $CURSOR.execute($STR % (..., $DATA, ...), ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            $CURSOR.execute($STR % (..., $DATA, ...), ...)
        - pattern: |
            $DATA = request.$W
            ...
            $CURSOR.execute($STR % (..., $DATA, ...), ...)
        - pattern: |
            $DATA = request.$W.get(...)
            ...
            $INTERM = $STR % (..., $DATA, ...)
            ...
            $CURSOR.execute($INTERM, ...)
        - pattern: |
            $DATA = request.$W(...)
            ...
            $INTERM = $STR % (..., $DATA, ...)
            ...
            $CURSOR.execute($INTERM, ...)
        - pattern: |
            $DATA = request.$W[...]
            ...
            $INTERM = $STR % (..., $DATA, ...)
            ...
            $CURSOR.execute($INTERM, ...)
        - pattern: |
            $DATA = request.$W
            ...
            $INTERM = $STR % (..., $DATA, ...)
            ...
            $CURSOR.execute($INTERM, ...)
    severity: WARNING
  - id: python.django.security.injection.sql.sql-injection-using-raw.sql-injection-using-raw
    languages:
      - python
    message: Data that is possible user-controlled from a python request is passed to `raw()`.
 This could lead to SQL injection and attackers gaining access to protected information. Instead
, use django's QuerySets, which are built with query parameterization and therefore not vulnerab
le to sql injection. For example, you could use `Entry.objects.filter(date=2006)`.
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
       cwe2021-top25: true
       cwe2022-top25: true
       impact: HIGH
       likelihood: MEDIUM
       owasp:
           - A01:2017 - Injection
           - A03:2021 - Injection
       references:
           - https://docs.djangoproject.com/en/3.0/topics/security/#sql-injection-protection
       subcategory:
           - vuln
       technology:
           - django
     patterns:
       - pattern-inside: |
           def $FUNC(...):
             ...
       - pattern-either:
           - pattern: $MODEL.objects.raw(..., $S.format(..., request.$W.get(...), ...), ...)
           - pattern: $MODEL.objects.raw(..., $S % request.$W.get(...), ...)
           - pattern: $MODEL.objects.raw(..., f"...{request.$W.get(...)}...", ...)
           - pattern: $MODEL.objects.raw(..., request.$W.get(...), ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
               $MODEL.objects.raw(..., $DATA, ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
               $INTERM = $DATA
               ...
               $MODEL.objects.raw(..., $INTERM, ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
               $MODEL.objects.raw(..., $STR.format(..., $DATA, ...), ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
               $INTERM = $STR.format(..., $DATA, ...)
               ...
               $MODEL.objects.raw(..., $INTERM, ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
               $MODEL.objects.raw(..., $STR % $DATA, ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
               $INTERM = $STR % $DATA
               ...
               $MODEL.objects.raw(..., $INTERM, ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
               $MODEL.objects.raw(..., f"...{$DATA}...", ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
               $INTERM = f"...{$DATA}..."
               ...
               $MODEL.objects.raw(..., $INTERM, ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
               $MODEL.objects.raw(..., $STR + $DATA, ...)
           - pattern: |
               $DATA = request.$W.get(...)
               ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    $INTERM = $STR + $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: $A = $MODEL.objects.raw(..., request.$W.get(...), ...)
- pattern: return $MODEL.objects.raw(..., request.$W.get(...), ...)
- pattern: $MODEL.objects.raw(..., $S.format(..., request.$W(...), ...), ...)
- pattern: $MODEL.objects.raw(..., $S % request.$W(...), ...)
- pattern: $MODEL.objects.raw(..., f"...{request.$W(...)}...", ...)
- pattern: $MODEL.objects.raw(..., request.$W(...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $MODEL.objects.raw(..., $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $MODEL.objects.raw(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $MODEL.objects.raw(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $MODEL.objects.raw(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $MODEL.objects.raw(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR + $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: $A = $MODEL.objects.raw(..., request.$W(...), ...)
- pattern: return $MODEL.objects.raw(..., request.$W(...), ...)
- pattern: $MODEL.objects.raw(..., $S.format(..., request.$W[...], ...), ...)
- pattern: $MODEL.objects.raw(..., $S % request.$W[...], ...)
- pattern: $MODEL.objects.raw(..., f"...{request.$W[...]}...", ...)
- pattern: $MODEL.objects.raw(..., request.$W[...], ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $MODEL.objects.raw(..., $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    ...
    $INTERM = $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $MODEL.objects.raw(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $MODEL.objects.raw(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $MODEL.objects.raw(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $MODEL.objects.raw(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR + $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: $A = $MODEL.objects.raw(..., request.$W[...], ...)
- pattern: return $MODEL.objects.raw(..., request.$W[...], ...)
- pattern: $MODEL.objects.raw(..., $S.format(..., request.$W, ...), ...)
- pattern: $MODEL.objects.raw(..., $S % request.$W, ...)
- pattern: $MODEL.objects.raw(..., f"...{request.$W}...", ...)
- pattern: $MODEL.objects.raw(..., request.$W, ...)
- pattern: |
    $DATA = request.$W
    ...
    $MODEL.objects.raw(..., $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $MODEL.objects.raw(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    $DATA = request.$W
    ...
    $MODEL.objects.raw(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR % $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $MODEL.objects.raw(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = f"...{$DATA}..."
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $MODEL.objects.raw(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR + $DATA
    ...
    $MODEL.objects.raw(..., $INTERM, ...)
- pattern: $A = $MODEL.objects.raw(..., request.$W, ...)
- pattern: return $MODEL.objects.raw(..., request.$W, ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $MODEL.objects.raw($STR % (..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $MODEL.objects.raw($STR % (..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $MODEL.objects.raw($STR % (..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W
    ...
    $MODEL.objects.raw($STR % (..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $INTERM = $STR % (..., $DATA, ...)
    ...
    $MODEL.objects.raw($INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % (..., $DATA, ...)
    ...
    $MODEL.objects.raw($INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % (..., $DATA, ...)
    ...
    $MODEL.objects.raw($INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR % (..., $DATA, ...)
    ...
    $MODEL.objects.raw($INTERM, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: WARNING
    - id: python.django.security.injection.ssrf.ssrf-injection-requests.ssrf-injection-requests
      languages:
        - python
      message: Data from request object is passed to a new server-side request. This could lead
to a server-side request forgery (SSRF). To mitigate, ensure that schemes and hosts are validate
d against an allowlist, do not forward the response to the user, and ensure proper authenticatio
n and transport-layer security in the proxied request. See https://owasp.org/www-community/attac
ks/Server_Side_Request_Forgery to learn more about SSRF vulnerabilities.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://owasp.org/www-community/attacks/Server_Side_Request_Forgery
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
              ...
        - pattern-either:
            - pattern: requests.$METHOD(..., $S.format(..., request.$W.get(...), ...), ...)
            - pattern: requests.$METHOD(..., $S % request.$W.get(...), ...)
            - pattern: requests.$METHOD(..., f"...{request.$W.get(...)}...", ...)
            - pattern: requests.$METHOD(..., request.$W.get(...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                requests.$METHOD(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                requests.$METHOD(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                requests.$METHOD(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                requests.$METHOD(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                requests.$METHOD(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % $DATA
                ...
                requests.$METHOD(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                requests.$METHOD(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W.get(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    ...
    $INTERM = f"...{$DATA}..."
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    requests.$METHOD(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W.get(...)
    ...
    $INTERM = $STR + $DATA
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: $A = requests.$METHOD(..., request.$W.get(...), ...)
- pattern: return requests.$METHOD(..., request.$W.get(...), ...)
- pattern: requests.$METHOD(..., $S.format(..., request.$W(...), ...), ...)
- pattern: requests.$METHOD(..., $S % request.$W(...), ...)
- pattern: requests.$METHOD(..., f"...{request.$W(...)}...", ...)
- pattern: requests.$METHOD(..., request.$W(...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    requests.$METHOD(..., $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $DATA
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    requests.$METHOD(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    requests.$METHOD(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    requests.$METHOD(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    requests.$METHOD(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR + $DATA
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: $A = requests.$METHOD(..., request.$W(...), ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
- pattern: return requests.$METHOD(..., request.$W(...), ...)
- pattern: requests.$METHOD(..., $S.format(..., request.$W[...], ...), ...)
- pattern: requests.$METHOD(..., $S % request.$W[...], ...)
- pattern: requests.$METHOD(..., f"...{request.$W[...]}...", ...)
- pattern: requests.$METHOD(..., request.$W[...], ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    requests.$METHOD(..., $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    requests.$METHOD(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    requests.$METHOD(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    requests.$METHOD(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    requests.$METHOD(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR + $DATA
    ...
    requests.$METHOD(..., $INTERM, ...)
- pattern: $A = requests.$METHOD(..., request.$W[...], ...)
- pattern: return requests.$METHOD(..., request.$W[...], ...)
- pattern: requests.$METHOD(..., $S.format(..., request.$W, ...), ...)
- pattern: requests.$METHOD(..., $S % request.$W, ...)
- pattern: requests.$METHOD(..., f"...{request.$W}...", ...)
- pattern: requests.$METHOD(..., request.$W, ...)
- pattern: |
    $DATA = request.$W
    ...
    requests.$METHOD(..., $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    requests.$METHOD(..., $INTERM, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                $DATA = request.$W
                ...
                requests.$METHOD(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                requests.$METHOD(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                requests.$METHOD(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR % $DATA
                ...
                requests.$METHOD(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                requests.$METHOD(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = f"...{$DATA}..."
                ...
                requests.$METHOD(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                requests.$METHOD(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                requests.$METHOD(..., $INTERM, ...)
            - pattern: $A = requests.$METHOD(..., request.$W, ...)
            - pattern: return requests.$METHOD(..., request.$W, ...)
      severity: ERROR
    - id: python.django.security.injection.ssrf.ssrf-injection-urllib.ssrf-injection-urllib
      languages:
        - python
      message: Data from request object is passed to a new server-side request. This could lead
to a server-side request forgery (SSRF), which could result in attackers gaining access to priva
te organization data. To mitigate, ensure that schemes and hosts are validated against an allowl
ist, do not forward the response to the user, and ensure proper authentication and transport-lay
er security in the proxied request.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://owasp.org/www-community/attacks/Server_Side_Request_Forgery
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-inside: |
            def $FUNC(...):
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
        - pattern-either:
            - pattern: urllib.request.urlopen(..., $S.format(..., request.$W.get(...), ...), ...
)
            - pattern: urllib.request.urlopen(..., $S % request.$W.get(...), ...)
            - pattern: urllib.request.urlopen(..., f"...{request.$W.get(...)}...", ...)
            - pattern: urllib.request.urlopen(..., request.$W.get(...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                urllib.request.urlopen(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $DATA
                ...
                urllib.request.urlopen(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                urllib.request.urlopen(..., $STR.format(..., $DATA, ...), ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR.format(..., $DATA, ...)
                ...
                urllib.request.urlopen(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                urllib.request.urlopen(..., $STR % $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR % $DATA
                ...
                urllib.request.urlopen(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                urllib.request.urlopen(..., f"...{$DATA}...", ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = f"...{$DATA}..."
                ...
                urllib.request.urlopen(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                urllib.request.urlopen(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W.get(...)
                ...
                $INTERM = $STR + $DATA
                ...
                urllib.request.urlopen(..., $INTERM, ...)
            - pattern: $A = urllib.request.urlopen(..., request.$W.get(...), ...)
            - pattern: return urllib.request.urlopen(..., request.$W.get(...), ...)
            - pattern: urllib.request.urlopen(..., $S.format(..., request.$W(...), ...), ...)
            - pattern: urllib.request.urlopen(..., $S % request.$W(...), ...)
            - pattern: urllib.request.urlopen(..., f"...{request.$W(...)}...", ...)
            - pattern: urllib.request.urlopen(..., request.$W(...), ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                urllib.request.urlopen(..., $DATA, ...)
            - pattern: |
                $DATA = request.$W(...)
                ...
                $INTERM = $DATA
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    urllib.request.urlopen(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    urllib.request.urlopen(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR % $DATA
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    urllib.request.urlopen(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = f"...{$DATA}..."
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    urllib.request.urlopen(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W(...)
    ...
    $INTERM = $STR + $DATA
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: $A = urllib.request.urlopen(..., request.$W(...), ...)
- pattern: return urllib.request.urlopen(..., request.$W(...), ...)
- pattern: urllib.request.urlopen(..., $S.format(..., request.$W[...], ...), ...)
- pattern: urllib.request.urlopen(..., $S % request.$W[...], ...)
- pattern: urllib.request.urlopen(..., f"...{request.$W[...]}...", ...)
- pattern: urllib.request.urlopen(..., request.$W[...], ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    urllib.request.urlopen(..., $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $DATA
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    urllib.request.urlopen(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    urllib.request.urlopen(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR % $DATA
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    urllib.request.urlopen(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = f"...{$DATA}..."
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    urllib.request.urlopen(..., $STR + $DATA, ...)
- pattern: |
    $DATA = request.$W[...]
    ...
    $INTERM = $STR + $DATA
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: $A = urllib.request.urlopen(..., request.$W[...], ...)
- pattern: return urllib.request.urlopen(..., request.$W[...], ...)
- pattern: urllib.request.urlopen(..., $S.format(..., request.$W, ...), ...)
- pattern: urllib.request.urlopen(..., $S % request.$W, ...)
- pattern: urllib.request.urlopen(..., f"...{request.$W}...", ...)
- pattern: urllib.request.urlopen(..., request.$W, ...)
- pattern: |
    $DATA = request.$W
    ...
    urllib.request.urlopen(..., $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $DATA
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    urllib.request.urlopen(..., $STR.format(..., $DATA, ...), ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR.format(..., $DATA, ...)
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    urllib.request.urlopen(..., $STR % $DATA, ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = $STR % $DATA
    ...
    urllib.request.urlopen(..., $INTERM, ...)
- pattern: |
    $DATA = request.$W
    ...
    urllib.request.urlopen(..., f"...{$DATA}...", ...)
- pattern: |
    $DATA = request.$W
    ...
    $INTERM = f"...{$DATA}..."
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                urllib.request.urlopen(..., $INTERM, ...)
            - pattern: |
                $DATA = request.$W
                ...
                urllib.request.urlopen(..., $STR + $DATA, ...)
            - pattern: |
                $DATA = request.$W
                ...
                $INTERM = $STR + $DATA
                ...
                urllib.request.urlopen(..., $INTERM, ...)
            - pattern: $A = urllib.request.urlopen(..., request.$W, ...)
            - pattern: return urllib.request.urlopen(..., request.$W, ...)
      severity: ERROR
    - id: python.django.security.nan-injection.nan-injection
      languages:
        - python
      message: Found user input going directly into typecast for bool(), float(), or complex().
This allows an attacker to inject Python's not-a-number (NaN) into the typecast. This results in
 undefind behavior, particularly when doing comparisons. Either cast to a different type, or add
 a guard checking for all capitalizations of the string 'nan'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-704: Incorrect Type Conversion or Cast'
        impact: MEDIUM
        likelihood: MEDIUM
        references:
            - https://discuss.python.org/t/nan-breaks-min-max-and-sorting-functions-a-solution/2
868
            - https://blog.bitdiscovery.com/2021/12/python-nan-injection/
        subcategory:
            - vuln
        technology:
            - django
      mode: taint
      pattern-sanitizers:
        - not_conflicting: true
          pattern: $ANYTHING(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: float(...)
                - pattern: bool(...)
                - pattern: complex(...)
            - pattern-not-inside: |
                if $COND:
                  ...
                ...
      pattern-sources:
        - patterns:
            - pattern-inside: |
                def $FUNC(request, ...):
                  ...
            - pattern-either:
                - pattern: request.$PROPERTY.get(...)
                - pattern: request.$PROPERTY[...]
      severity: ERROR
    - id: python.django.security.passwords.password-empty-string.password-empty-string
      languages:
        - python
      message: '''$VAR'' is the empty string and is being used to set the password on ''$MODEL''
. If you meant to set an unusable password, set the password to None or call ''set_unusable_pass
word()''.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-521: Weak Password Requirements'
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://docs.djangoproject.com/en/3.0/ref/contrib/auth/#django.contrib.auth.models
.User.set_password
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-either:
            - pattern: |
                $MODEL.set_password($EMPTY)
                ...
                $MODEL.save()
            - pattern: |
                $VAR = $EMPTY
                ...
                $MODEL.set_password($VAR)
                ...
                $MODEL.save()
        - metavariable-regex:
            metavariable: $EMPTY
            regex: (\'\'|\"\")
      severity: ERROR
    - fix: |
        None
      id: python.django.security.passwords.use-none-for-password-default.use-none-for-password-d
efault
      languages:
        - python
      message: '''$VAR'' is using the empty string as its default and is being used to set the p
assword on ''$MODEL''. If you meant to set an unusable password, set the default value to ''None
'' or call ''set_unusable_password()''.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-521: Weak Password Requirements'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://docs.djangoproject.com/en/3.0/ref/contrib/auth/#django.contrib.auth.models
.User.set_password
        subcategory:
            - vuln
        technology:
            - django
      patterns:
        - pattern-either:
            - pattern: |
                $VAR = request.$W.get($X, $EMPTY)
                ...
                $MODEL.set_password($VAR)
                ...
                $MODEL.save(...)
            - pattern: |
                def $F(..., $VAR=$EMPTY, ...):
                  ...
                  $MODEL.set_password($VAR)
        - metavariable-pattern:
            metavariable: $EMPTY
            pattern: '""'
        - focus-metavariable: $EMPTY
      severity: ERROR
    - id: python.fastapi.security.wildcard-cors.wildcard-cors
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - python
      message: CORS policy allows any origin (using wildcard '*'). This is insecure and should b
e avoided.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-942: Permissive Cross-domain Policy with Untrusted Domains'
        impact: LOW
        likelihood: HIGH
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
            - https://cwe.mitre.org/data/definitions/942.html
        subcategory:
            - vuln
        technology:
            - python
            - fastapi
        vulnerability_class:
            - Configuration
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                $APP.add_middleware(
                  CORSMiddleware,
                  allow_origins=$ORIGIN,
                  ...);
            - focus-metavariable: $ORIGIN
      pattern-sources:
        - pattern: '[..., "*", ...]'
      severity: WARNING
    - id: python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host
      languages:
        - python
      message: Running flask app with host 0.0.0.0 could expose the server publicly.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-668: Exposure of Resource to Wrong Sphere'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - flask
      pattern-either:
        - pattern: app.run(..., host="0.0.0.0", ...)
        - pattern: app.run(..., "0.0.0.0", ...)
      severity: WARNING
    - id: python.flask.security.audit.app-run-security-config.avoid_using_app_run_directly
      languages:
        - python
      message: top-level app.run(...) is ignored by flask. Consider putting app.run(...) behind
a guard, like inside a function
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-668: Exposure of Resource to Wrong Sphere'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - flask
      patterns:
        - pattern-not-inside: |
            if __name__ == '__main__':
              ...
        - pattern-not-inside: |
            def $X(...):
              ...
        - pattern: app.run(...)
      severity: WARNING
    - id: python.flask.security.audit.debug-enabled.debug-enabled
      languages:
        - python
      message: Detected Flask app with debug=True. Do not deploy to production with this flag en
abled as it will leak sensitive information. Instead, consider using Flask configuration variabl
es or setting 'debug' using system environment variables.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-489: Active Debug Code'
        impact: MEDIUM
        likelihood: HIGH
        owasp: A06:2017 - Security Misconfiguration
        references:
            - https://labs.detectify.com/2015/10/02/how-patreon-got-hacked-publicly-exposed-werk
zeug-debugger/
        subcategory:
            - vuln
        technology:
            - flask
      patterns:
        - pattern-inside: |
            import flask
            ...
        - pattern: $APP.run(..., debug=True, ...)
      severity: WARNING
    - id: python.flask.security.audit.directly-returned-format-string.directly-returned-format-s
tring
      languages:
        - python
      message: Detected Flask route directly returning a formatted string. This is subject to cr
oss-site scripting if user input can reach the string. Consider using the template engine instea
d and rendering pages with 'render_template()'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - flask
      mode: taint
      pattern-sinks:
        - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern-not-inside: return "..."
            - pattern-either:
                - pattern: return "...".format(...)
                - pattern: return "..." % ...
                - pattern: return "..." + ...
                - pattern: return ... + "..."
                - pattern: return f"...{...}..."
                - patterns:
                    - pattern: return $X
                    - pattern-either:
                        - pattern-inside: |
                            $X = "...".format(...)
                            ...
                        - pattern-inside: |
                            $X = "..." % ...
                            ...
                        - pattern-inside: |
                            $X = "..." + ...
                            ...
                        - pattern-inside: |
                            $X = ... + "..."
                            ...
                        - pattern-inside: |
                            $X = f"...{...}..."
                            ...
                    - pattern-not-inside: |
                        $X = "..."
                        ...
      pattern-sources:
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    @$APP.route(...)
                    def $FUNC(..., $PARAM, ...):
                      ...
                - pattern: $PARAM
            - pattern: |
                request.$FUNC.get(...)
            - pattern: |
                request.$FUNC(...)
            - pattern: request.$FUNC[...]
      severity: WARNING
    - id: python.flask.security.hashids-with-flask-secret.hashids-with-flask-secret
      languages:
        - python
      message: The Flask secret key is used as salt in HashIDs. The HashID mechanism is not secu
re. By observing sufficient HashIDs, the salt used to construct them can be recovered. This mean
s the Flask secret key can be obtained by attackers, through the HashIDs.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://flask.palletsprojects.com/en/2.2.x/config/#SECRET_KEY
            - http://carnage.github.io/2015/08/cryptanalysis-of-hashids
        subcategory:
            - vuln
        technology:
            - flask
      pattern-either:
        - pattern: hashids.Hashids(..., salt=flask.current_app.config['SECRET_KEY'], ...)
        - pattern: hashids.Hashids(flask.current_app.config['SECRET_KEY'], ...)
        - patterns:
            - pattern-inside: |
                $APP = flask.Flask(...)
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern-either:
                - pattern: hashids.Hashids(..., salt=$APP.config['SECRET_KEY'], ...)
                - pattern: hashids.Hashids($APP.config['SECRET_KEY'], ...)
      severity: ERROR
    - id: python.flask.security.injection.csv-writer-injection.csv-writer-injection
      languages:
        - python
      message: Detected user input into a generated CSV file using the built-in `csv` module. If
 user data is used to generate the data in this file, it is possible that an attacker could inje
ct a formula when the CSV is imported into a spreadsheet application that runs an attacker scrip
t, which could steal data from the importing user or, at worst, install malware on the user's co
mputer. `defusedcsv` is a drop-in replacement with the same API that will attempt to mitigate fo
rmula injection attempts. You can use `defusedcsv` instead of `csv` to safely generate CSVs.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1236: Improper Neutralization of Formula Elements in a CSV File'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://github.com/raphaelm/defusedcsv
            - https://owasp.org/www-community/attacks/CSV_Injection
            - https://web.archive.org/web/20220516052229/https://www.contextis.com/us/blog/comma
-separated-vulnerabilities
        subcategory:
            - vuln
        technology:
            - python
            - flask
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-inside: |
                $WRITER = csv.writer(...)

                ...

                $WRITER.$WRITE(...)
            - pattern: $WRITER.$WRITE(...)
            - metavariable-regex:
                metavariable: $WRITE
                regex: ^(writerow|writerows|writeheader)$
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: flask.request.form.get(...)
                        - pattern: flask.request.form[...]
                        - pattern: flask.request.args.get(...)
                        - pattern: flask.request.args[...]
                        - pattern: flask.request.values.get(...)
                        - pattern: flask.request.values[...]
                        - pattern: flask.request.cookies.get(...)
                        - pattern: flask.request.cookies[...]
                        - pattern: flask.request.stream
                        - pattern: flask.request.headers.get(...)
                        - pattern: flask.request.headers[...]
                        - pattern: flask.request.data
                        - pattern: flask.request.full_path
                        - pattern: flask.request.url
                        - pattern: flask.request.json
                        - pattern: flask.request.get_json()
                        - pattern: flask.request.view_args.get(...)
                        - pattern: flask.request.view_args[...]
                - patterns:
                    - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                      @$APP.route($ROUTE, ...)
                      def $FUNC(..., $ROUTEVAR, ...):
                        ...
                  - focus-metavariable: $ROUTEVAR
      severity: ERROR
    - id: python.flask.security.injection.nan-injection.nan-injection
      languages:
        - python
      message: Found user input going directly into typecast for bool(), float(), or complex().
This allows an attacker to inject Python's not-a-number (NaN) into the typecast. This results in
 undefind behavior, particularly when doing comparisons. Either cast to a different type, or add
 a guard checking for all capitalizations of the string 'nan'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-704: Incorrect Type Conversion or Cast'
        impact: MEDIUM
        likelihood: MEDIUM
        references:
            - https://discuss.python.org/t/nan-breaks-min-max-and-sorting-functions-a-solution/2
868
            - https://blog.bitdiscovery.com/2021/12/python-nan-injection/
        subcategory:
            - vuln
        technology:
            - flask
      mode: taint
      pattern-sanitizers:
        - not_conflicting: true
          pattern: $ANYTHING(...)
      pattern-sinks:
        - pattern-either:
            - pattern: float(...)
            - pattern: bool(...)
            - pattern: complex(...)
      pattern-sources:
        - pattern-either:
            - pattern: flask.request.$SOMETHING.get(...)
            - pattern: flask.request.$SOMETHING[...]
            - patterns:
                - pattern-inside: |
                    @$APP.route(...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                - pattern: $ROUTEVAR
      severity: ERROR
    - id: python.flask.security.injection.os-system-injection.os-system-injection
      languages:
        - python
      message: User data detected in os.system. This could be vulnerable to a command injection
and should be avoided. If this must be done, use the 'subprocess' module instead and pass the ar
guments as a list.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/Command_Injection
        subcategory:
            - audit
        technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - flask
      pattern-either:
        - patterns:
            - pattern: os.system(...)
            - pattern-either:
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                      os.system(..., <... $ROUTEVAR ...>, ...)
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                      $INTERM = <... $ROUTEVAR ...>
                      ...
                      os.system(..., <... $INTERM ...>, ...)
        - pattern: os.system(..., <... flask.request.$W.get(...) ...>, ...)
        - pattern: os.system(..., <... flask.request.$W[...] ...>, ...)
        - pattern: os.system(..., <... flask.request.$W(...) ...>, ...)
        - pattern: os.system(..., <... flask.request.$W ...>, ...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W.get(...) ...>
                ...
                os.system(<... $INTERM ...>)
            - pattern: os.system(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W[...] ...>
                ...
                os.system(<... $INTERM ...>)
            - pattern: os.system(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W(...) ...>
                ...
                os.system(<... $INTERM ...>)
            - pattern: os.system(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W ...>
                ...
                os.system(<... $INTERM ...>)
            - pattern: os.system(...)
      severity: ERROR
    - id: python.flask.security.injection.path-traversal-open.path-traversal-open
      languages:
        - python
      message: Found request data in a call to 'open'. Ensure the request data is validated or s
anitized, otherwise it could result in path traversal attacks.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/www-community/attacks/Path_Traversal
        subcategory:
            - audit
        technology:
            - flask
      pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
- patterns:
    - pattern: open(...)
    - pattern-either:
        - pattern-inside: |
            @$APP.route($ROUTE, ...)
            def $FUNC(..., $ROUTEVAR, ...):
              ...
              open(..., <... $ROUTEVAR ...>, ...)
        - pattern-inside: |
            @$APP.route($ROUTE, ...)
            def $FUNC(..., $ROUTEVAR, ...):
              ...
              with open(..., <... $ROUTEVAR ...>, ...) as $FD:
                ...
        - pattern-inside: |
            @$APP.route($ROUTE, ...)
            def $FUNC(..., $ROUTEVAR, ...):
              ...
              $INTERM = <... $ROUTEVAR ...>
              ...
              open(..., <... $INTERM ...>, ...)
- pattern: open(..., <... flask.request.$W.get(...) ...>, ...)
- pattern: open(..., <... flask.request.$W[...] ...>, ...)
- pattern: open(..., <... flask.request.$W(...) ...>, ...)
- pattern: open(..., <... flask.request.$W ...>, ...)
- patterns:
    - pattern-inside: |
        $INTERM = <... flask.request.$W.get(...) ...>
        ...
        open(<... $INTERM ...>, ...)
    - pattern: open(...)
- patterns:
    - pattern-inside: |
        $INTERM = <... flask.request.$W[...] ...>
        ...
        open(<... $INTERM ...>, ...)
    - pattern: open(...)
- patterns:
    - pattern-inside: |
        $INTERM = <... flask.request.$W(...) ...>
        ...
        open(<... $INTERM ...>, ...)
    - pattern: open(...)
- patterns:
    - pattern-inside: |
        $INTERM = <... flask.request.$W ...>
        ...
        open(<... $INTERM ...>, ...)
    - pattern: open(...)
- patterns:
    - pattern-inside: |
        $INTERM = <... flask.request.$W.get(...) ...>
        ...
        with open(<... $INTERM ...>, ...) as $F:
          ...
    - pattern: open(...)
- patterns:
    - pattern-inside: |
        $INTERM = <... flask.request.$W[...] ...>
        ...
        with open(<... $INTERM ...>, ...) as $F:
          ...
    - pattern: open(...)
- patterns:
    - pattern-inside: |
        $INTERM = <... flask.request.$W(...) ...>
        ...
        with open(<... $INTERM ...>, ...) as $F:
          ...
    - pattern: open(...)
- patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern-inside: |
                $INTERM = <... flask.request.$W ...>
                ...
                with open(<... $INTERM ...>, ...) as $F:
                  ...
            - pattern: open(...)
      severity: ERROR
    - id: python.flask.security.injection.raw-html-concat.raw-html-format
      languages:
        - python
      message: Detected user input flowing into a manually constructed HTML string. You may be a
ccidentally bypassing secure methods of rendering HTML by manually constructing HTML and this co
uld create a cross-site scripting vulnerability, which could let attackers steal sensitive user
data. To be sure this is safe, check that the HTML is rendered safely. Otherwise, use templates
(`flask.render_template`) which will safely render HTML instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://flask.palletsprojects.com/en/2.0.x/security/#cross-site-scripting-xss
        subcategory:
            - vuln
        technology:
            - flask
      mode: taint
      pattern-sanitizers:
        - pattern: jinja2.escape(...)
        - pattern: flask.escape(...)
        - pattern: flask.render_template("~=/.*\.html", ...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: '"$HTMLSTR" % ...'
                        - pattern: '"$HTMLSTR".format(...)'
                        - pattern: '"$HTMLSTR" + ...'
                        - pattern: f"$HTMLSTR{...}..."
                - patterns:
                    - pattern-inside: |
                        $HTML = "$HTMLSTR"
                        ...
                    - pattern-either:
                        - pattern: $HTML % ...
                        - pattern: $HTML.format(...)
                        - pattern: $HTML + ...
            - metavariable-pattern:
                language: generic
                metavariable: $HTMLSTR
                pattern: <$TAG ...
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: flask.request.$ANYTHING
                - patterns:
                    - pattern-inside: |
                        @$APP.route(...)
                        def $FUNC(..., $ROUTEVAR, ...):
                          ...
                    - pattern: $ROUTEVAR
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: WARNING
    - id: python.flask.security.injection.ssrf-requests.ssrf-requests
      languages:
        - python
      message: Data from request object is passed to a new server-side request. This could lead
to a server-side request forgery (SSRF). To mitigate, ensure that schemes and hosts are validate
d against an allowlist, do not forward the response to the user, and ensure proper authenticatio
n and transport-layer security in the proxied request.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://owasp.org/www-community/attacks/Server_Side_Request_Forgery
        subcategory:
            - vuln
        technology:
            - flask
      pattern-either:
        - patterns:
            - pattern: requests.$FUNC(...)
            - pattern-either:
                - pattern-inside: |
                    @$APP.$ROUTE_METHOD($ROUTE, ...)
                    def $ROUTE_FUNC(..., $ROUTEVAR, ...):
                      ...
                      requests.$FUNC(..., <... $ROUTEVAR ...>, ...)
                - pattern-inside: |
                    @$APP.$ROUTE_METHOD($ROUTE, ...)
                    def $ROUTE_FUNC(..., $ROUTEVAR, ...):
                      ...
                      $INTERM = <... $ROUTEVAR ...>
                      ...
                      requests.$FUNC(..., <... $INTERM ...>, ...)
            - metavariable-regex:
                metavariable: $ROUTE_METHOD
                regex: ^(route|get|post|put|delete|patch)$
        - pattern: requests.$FUNC(..., <... flask.request.$W.get(...) ...>, ...)
        - pattern: requests.$FUNC(..., <... flask.request.$W[...] ...>, ...)
        - pattern: requests.$FUNC(..., <... flask.request.$W(...) ...>, ...)
        - pattern: requests.$FUNC(..., <... flask.request.$W ...>, ...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W.get(...) ...>
                ...
                requests.$FUNC(<... $INTERM ...>, ...)
            - pattern: requests.$FUNC(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W[...] ...>
                ...
                requests.$FUNC(<... $INTERM ...>, ...)
            - pattern: requests.$FUNC(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W(...) ...>
                ...
                requests.$FUNC(<... $INTERM ...>, ...)
            - pattern: requests.$FUNC(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W ...>
                ...
                requests.$FUNC(<... $INTERM ...>, ...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: requests.$FUNC(...)
      severity: ERROR
    - id: python.flask.security.injection.subprocess-injection.subprocess-injection
      languages:
        - python
      message: Detected user input entering a `subprocess` call unsafely. This could result in a
 command injection vulnerability. An attacker could use this vulnerability to execute arbitrary
commands on the host, which allows them to download malware, scan sensitive data, or run any com
mand they wish on the server. Do not let users choose the command to run. In general, prefer to
use Python API versions of system commands. If you must use subprocess, use a dictionary to allo
wlist a set of commands.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - flask
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sanitizers:
        - patterns:
            - pattern: $DICT[$KEY]
            - focus-metavariable: $KEY
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: subprocess.$FUNC(...)
                    - pattern-not: subprocess.$FUNC("...", ...)
                    - pattern-not: subprocess.$FUNC(["...", ...], ...)
                    - pattern-not-inside: |
                        $CMD = ["...", ...]
                        ...
                        subprocess.$FUNC($CMD, ...)
                - patterns:
                    - pattern: subprocess.$FUNC(["$SHELL", "-c", ...], ...)
                    - metavariable-regex:
                        metavariable: $SHELL
                        regex: ^(sh|bash|ksh|csh|tcsh|zsh)$
                - patterns:
                    - pattern: subprocess.$FUNC(["$INTERPRETER", ...], ...)
                    - metavariable-regex:
                        metavariable: $INTERPRETER
                        regex: ^(python|python\d)$
      pattern-sources:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: flask.request.form.get(...)
                    - pattern: flask.request.form[...]
                    - pattern: flask.request.args.get(...)
                    - pattern: flask.request.args[...]
                    - pattern: flask.request.values.get(...)
                    - pattern: flask.request.values[...]
                    - pattern: flask.request.cookies.get(...)
                    - pattern: flask.request.cookies[...]
                    - pattern: flask.request.stream
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern: flask.request.headers.get(...)
                    - pattern: flask.request.headers[...]
                    - pattern: flask.request.data
                    - pattern: flask.request.full_path
                    - pattern: flask.request.url
                    - pattern: flask.request.json
                    - pattern: flask.request.get_json()
                    - pattern: flask.request.view_args.get(...)
                    - pattern: flask.request.view_args[...]
            - patterns:
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                - focus-metavariable: $ROUTEVAR
      severity: ERROR
    - id: python.flask.security.injection.tainted-sql-string.tainted-sql-string
      languages:
        - python
      message: Detected user input used to manually construct a SQL string. This is usually bad
practice because manual construction could accidentally result in a SQL injection. An attacker c
ould use a SQL injection to steal or modify contents of the database. Instead, use a parameteriz
ed query which is available by default in most database engines. Alternatively, consider using a
n object-relational mapper (ORM) such as SQLAlchemy which will protect your queries.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-704: Incorrect Type Conversion or Cast'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-textual-sql
            - https://www.tutorialspoint.com/sqlalchemy/sqlalchemy_quick_guide.htm
            - https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-more-specific-text-with
-table-expression-literal-column-and-expression-column
        subcategory:
            - vuln
        technology:
            - sqlalchemy
            - flask
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    "$SQLSTR" + ...
                - pattern: |
                    "$SQLSTR" % ...
                - pattern: |
                    "$SQLSTR".format(...)
                - pattern: |
                    f"$SQLSTR{...}..."
            - metavariable-regex:
                metavariable: $SQLSTR
                regex: \s*(?i)(select|delete|insert|create|update|alter|drop)\b.*
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: flask.request.$ANYTHING
                - patterns:
                    - pattern-inside: |
                        @$APP.route(...)
                        def $FUNC(..., $ROUTEVAR, ...):
                          ...
                    - pattern: $ROUTEVAR
      severity: ERROR
    - id: python.flask.security.injection.tainted-url-host.tainted-url-host
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      languages:
        - python
      message: User data flows into the host portion of this manually-constructed URL. This coul
d allow an attacker to send data to their own server, potentially exposing sensitive data such a
s cookies or authorization information sent with this request. They could also probe internal se
rvers or other resources that the server running this code can access. (This is called server-si
de request forgery, or SSRF.) Do not allow arbitrary hosts. Instead, create an allowlist for app
roved hosts, or hardcode the correct host.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevent
ion_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - flask
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: '"$URLSTR" % ...'
                    - metavariable-pattern:
                        language: generic
                        metavariable: $URLSTR
                        patterns:
                            - pattern-either:
                                - pattern: $SCHEME://%s
                                - pattern: $SCHEME://%r
                - patterns:
                    - pattern: '"$URLSTR".format(...)'
                    - metavariable-pattern:
                        language: generic
                        metavariable: $URLSTR
                        pattern: $SCHEME:// { ... }
                - patterns:
                    - pattern: '"$URLSTR" + ...'
                    - metavariable-regex:
                        metavariable: $URLSTR
                        regex: .*://$
                - patterns:
                    - pattern: f"$URLSTR{...}..."
                    - metavariable-regex:
                        metavariable: $URLSTR
                        regex: .*://$
                - patterns:
                    - pattern-inside: |
                        $URL = "$URLSTR"
                        ...
                    - pattern: $URL += ...
                    - metavariable-regex:
                        metavariable: $URLSTR
                        regex: .*://$
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: flask.request.$ANYTHING
                - patterns:
                    - pattern-inside: |
                        @$APP.route(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        def $FUNC(..., $ROUTEVAR, ...):
                          ...
                    - pattern: $ROUTEVAR
      severity: WARNING
    - id: python.flask.security.injection.user-eval.eval-injection
      languages:
        - python
      message: Detected user data flowing into eval. This is code injection and should be avoide
d.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://nedbatchelder.com/blog/201206/eval_really_is_dangerous.html
        subcategory:
            - vuln
        technology:
            - flask
      pattern-either:
        - patterns:
            - pattern: eval(...)
            - pattern-either:
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                      eval(..., <... $ROUTEVAR ...>, ...)
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                      $INTERM = <... $ROUTEVAR ...>
                      ...
                      eval(..., <... $INTERM ...>, ...)
        - pattern: eval(..., <... flask.request.$W.get(...) ...>, ...)
        - pattern: eval(..., <... flask.request.$W[...] ...>, ...)
        - pattern: eval(..., <... flask.request.$W(...) ...>, ...)
        - pattern: eval(..., <... flask.request.$W ...>, ...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W.get(...) ...>
                ...
                eval(..., <... $INTERM ...>, ...)
            - pattern: eval(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W[...] ...>
                ...
                eval(..., <... $INTERM ...>, ...)
            - pattern: eval(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W(...) ...>
                ...
                eval(..., <... $INTERM ...>, ...)
            - pattern: eval(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W ...>
                ...
                eval(..., <... $INTERM ...>, ...)
            - pattern: eval(...)
      severity: ERROR
    - id: python.flask.security.injection.user-exec.exec-injection
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      languages:
        - python
      message: Detected user data flowing into exec. This is code injection and should be avoide
d.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://nedbatchelder.com/blog/201206/exec_really_is_dangerous.html
        subcategory:
            - vuln
        technology:
            - flask
      pattern-either:
        - patterns:
            - pattern: exec(...)
            - pattern-either:
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                      exec(..., <... $ROUTEVAR ...>, ...)
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                      $INTERM = <... $ROUTEVAR ...>
                      ...
                      exec(..., <... $INTERM ...>, ...)
        - pattern: exec(..., <... flask.request.$W.get(...) ...>, ...)
        - pattern: exec(..., <... flask.request.$W[...] ...>, ...)
        - pattern: exec(..., <... flask.request.$W(...) ...>, ...)
        - pattern: exec(..., <... flask.request.$W ...>, ...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W.get(...) ...>
                ...
                exec(..., <... $INTERM ...>, ...)
            - pattern: exec(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W[...] ...>
                ...
                exec(..., <... $INTERM ...>, ...)
            - pattern: exec(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W(...) ...>
                ...
                exec(..., <... $INTERM ...>, ...)
            - pattern: exec(...)
        - patterns:
            - pattern-inside: |
                $INTERM = <... flask.request.$W ...>
                ...
                exec(..., <... $INTERM ...>, ...)
            - pattern: exec(...)
      severity: ERROR
    - fix: |
        True
      id: python.jinja2.security.audit.autoescape-disabled-false.incorrect-autoescape-disabled
      languages:
        - python
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Detected a Jinja2 environment with 'autoescaping' disabled. This is dangerous if
you are rendering to a browser because this allows for cross-site scripting (XSS) attacks. If yo
u are in a web context, enable 'autoescaping' by setting 'autoescape=True.' You may also conside
r using 'jinja2.select_autoescape()' to only enable automatic escaping for certain file extensio
ns.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-116: Improper Encoding or Escaping of Output'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2021 - Injection
        references:
            - https://jinja.palletsprojects.com/en/2.11.x/api/#basics
        source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/b701_jinja2_autoescape_
false.html
        subcategory:
            - vuln
        technology:
            - jinja2
      patterns:
        - pattern: jinja2.Environment(... , autoescape=$VAL, ...)
        - pattern-not: jinja2.Environment(... , autoescape=True, ...)
        - pattern-not: jinja2.Environment(... , autoescape=jinja2.select_autoescape(...), ...)
        - focus-metavariable: $VAL
      severity: WARNING
    - fix-regex:
        regex: (.*)\)
        replacement: \1, autoescape=True)
      id: python.jinja2.security.audit.missing-autoescape-disabled.missing-autoescape-disabled
      languages:
        - python
      message: Detected a Jinja2 environment without autoescaping. Jinja2 does not autoescape by
 default. This is dangerous if you are rendering to a browser because this allows for cross-site
 scripting (XSS) attacks. If you are in a web context, enable autoescaping by setting 'autoescap
e=True.' You may also consider using 'jinja2.select_autoescape()' to only enable automatic escap
ing for certain file extensions.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-116: Improper Encoding or Escaping of Output'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2021 - Injection
        references:
            - https://jinja.palletsprojects.com/en/2.11.x/api/#basics
        source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/b701_jinja2_autoescape_
false.html
        subcategory:
            - vuln
        technology:
            - jinja2
      patterns:
        - pattern-not: jinja2.Environment(..., autoescape=$VAL, ...)
        - pattern: jinja2.Environment(...)
      severity: WARNING
    - id: python.jwt.security.jwt-hardcode.jwt-python-hardcoded-secret
      languages:
        - python
      message: 'Hardcoded JWT secret or private key is used. This is a Insufficiently Protected
Credentials weakness: https://cwe.mitre.org/data/definitions/522.html Consider using an appropri
ate security mechanism to protect the credentials (e.g. keeping secrets in environment variables
)'
      metadata:
        category: security
        confidence: HIGH
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-other-common
-jwt-mistakes/
        subcategory:
            - vuln
        technology:
            - jwt
      patterns:
        - pattern: |
            jwt.encode($X, $SECRET, ...)
        - focus-metavariable: $SECRET
        - pattern: |
            "..."
      severity: ERROR
    - id: python.jwt.security.jwt-none-alg.jwt-python-none-alg
      languages:
        - python
      message: Detected use of the 'none' algorithm in a JWT token. The 'none' algorithm assumes
 the integrity of the token has already been verified. This would allow a malicious actor to for
ge a JWT token that will automatically be verified. Do not explicitly use the 'none' algorithm.
Instead, use an algorithm such as 'HS256'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory:
            - vuln
        technology:
            - jwt
      pattern-either:
        - pattern: |
            jwt.encode(...,algorithm="none",...)
        - pattern: jwt.decode(...,algorithms=[...,"none",...],...)
      severity: ERROR
    - fix: |
        True
      id: python.jwt.security.unverified-jwt-decode.unverified-jwt-decode
      languages:
        - python
      message: Detected JWT token decoded with 'verify=False'. This bypasses any integrity check
s for the token which means the token could be tampered with by malicious actors. Ensure that th
e JWT token is verified.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-287: Improper Authentication'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A02:2017 - Broken Authentication
            - A07:2021 - Identification and Authentication Failures
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        references:
            - https://github.com/we45/Vulnerable-Flask-App/blob/752ee16087c0bfb79073f68802d90756
9a1f0df7/app/app.py#L96
        subcategory:
            - audit
        technology:
            - jwt
      patterns:
        - pattern-either:
            - patterns:
                - pattern: |
                    jwt.decode(..., options={..., "verify_signature": $BOOL, ...}, ...)
                - metavariable-pattern:
                    metavariable: $BOOL
                    pattern: |
                        False
                - focus-metavariable: $BOOL
            - patterns:
                - pattern: |
                    $OPTS = {..., "verify_signature": $BOOL, ...}
                    ...
                    jwt.decode(..., options=$OPTS, ...)
                - metavariable-pattern:
                    metavariable: $BOOL
                    pattern: |
                        False
                - focus-metavariable: $BOOL
      severity: ERROR
    - id: python.lang.security.audit.dangerous-asyncio-exec-tainted-env-args.dangerous-asyncio-e
xec-tainted-env-args
      languages:
        - python
      message: Detected subprocess function '$LOOP.subprocess_exec' with user controlled data. Y
ou may consider using 'shlex.escape()'.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.python.org/3/library/asyncio-eventloop.html#asyncio.loop.subprocess_e
xec
            - https://docs.python.org/3/library/shlex.html
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - pattern-either:
            - patterns:
                - pattern-not: $LOOP.subprocess_exec($PROTOCOL, "...", ...)
                - pattern-not: $LOOP.subprocess_exec($PROTOCOL, ["...",...], ...)
                - pattern: $LOOP.subprocess_exec(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - patterns:
                - pattern-not: $LOOP.subprocess_exec($PROTOCOL, "=~/(sh|bash|ksh|csh|tcsh|zsh)/"
, "-c", "...", ...)
                - pattern: $LOOP.subprocess_exec($PROTOCOL, "=~/(sh|bash|ksh|csh|tcsh|zsh)/", "-
c",...)
            - patterns:
                - pattern-not: $LOOP.subprocess_exec($PROTOCOL, ["=~/(sh|bash|ksh|csh|tcsh|zsh)/
", "-c", "...", ...], ...)
                - pattern: $LOOP.subprocess_exec($PROTOCOL, ["=~/(sh|bash|ksh|csh|tcsh|zsh)/", "
-c", ...], ...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: os.environ
                        - pattern: os.environ.get('$FOO', ...)
                        - pattern: os.environb
                        - pattern: os.environb.get('$FOO', ...)
                        - pattern: os.getenv('$ANYTHING', ...)
                        - pattern: os.getenvb('$ANYTHING', ...)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-either:
                                - pattern: sys.argv
                                - pattern: sys.orig_argv
                        - patterns:
                            - pattern-inside: |
                                $PARSER = argparse.ArgumentParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-inside: |
                                $PARSER = optparse.OptionParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.getopt(...)
                                    ...
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.gnu_getopt(...)
                                    ...
                            - pattern-either:
                                - patterns:
                                    - pattern-inside: |
                                        for $O, $A in $OPTS:
                                          ...
                                    - pattern: $A
                                - pattern: $ARGS
      severity: ERROR
    - id: python.lang.security.audit.dangerous-asyncio-shell-tainted-env-args.dangerous-asyncio-
shell-tainted-env-args
      languages:
        - python
      message: Detected asyncio subprocess function with user controlled data. You may consider
using 'shlex.escape()'.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.python.org/3/library/asyncio-subprocess.html
            - https://docs.python.org/3/library/shlex.html
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: $LOOP.subprocess_shell($PROTOCOL, $CMD)
                - pattern-inside: asyncio.subprocess.create_subprocess_shell($CMD, ...)
                - pattern-inside: asyncio.create_subprocess_shell($CMD, ...)
            - focus-metavariable: $CMD
            - pattern-not-inside: |
                $CMD = "..."
                ...
            - pattern-not: $LOOP.subprocess_shell($PROTOCOL, "...")
            - pattern-not: asyncio.subprocess.create_subprocess_shell("...", ...)
            - pattern-not: asyncio.create_subprocess_shell("...", ...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: os.environ
                        - pattern: os.environ.get('$FOO', ...)
                        - pattern: os.environb
                        - pattern: os.environb.get('$FOO', ...)
                        - pattern: os.getenv('$ANYTHING', ...)
                        - pattern: os.getenvb('$ANYTHING', ...)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-either:
                                - pattern: sys.argv
                                - pattern: sys.orig_argv
                        - patterns:
                            - pattern-inside: |
                                $PARSER = argparse.ArgumentParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-inside: |
                                $PARSER = optparse.OptionParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.getopt(...)
                                    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.gnu_getopt(...)
                                    ...
                                - pattern-either:
                                    - patterns:
                                        - pattern-inside: |
                                            for $O, $A in $OPTS:
                                              ...
                                        - pattern: $A
                                    - pattern: $ARGS
      severity: ERROR
    - id: python.lang.security.audit.dangerous-code-run-tainted-env-args.dangerous-interactive-c
ode-run-tainted-env-args
      languages:
        - python
      message: Found user controlled data inside InteractiveConsole/InteractiveInterpreter metho
d. This is dangerous if external data can reach this function call because it allows a malicious
 actor to run arbitrary Python code.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $X = code.InteractiveConsole(...)
                    ...
                - pattern-inside: |
                    $X = code.InteractiveInterpreter(...)
                    ...
            - pattern-either:
                - pattern-inside: |
                    $X.push($PAYLOAD,...)
                - pattern-inside: |
                    $X.runsource($PAYLOAD,...)
                - pattern-inside: |
                    $X.runcode(code.compile_command($PAYLOAD),...)
                - pattern-inside: |
                    $PL = code.compile_command($PAYLOAD,...)
                    ...
                    $X.runcode($PL,...)
            - pattern: $PAYLOAD
            - pattern-not: |
                $X.push("...",...)
            - pattern-not: |
                $X.runsource("...",...)
            - pattern-not: |
                $X.runcode(code.compile_command("..."),...)
            - pattern-not: |
                $PL = code.compile_command("...",...)
                ...
                $X.runcode($PL,...)
      pattern-sources:
        - patterns:
            - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - patterns:
                    - pattern-either:
                        - pattern: os.environ
                        - pattern: os.environ.get('$FOO', ...)
                        - pattern: os.environb
                        - pattern: os.environb.get('$FOO', ...)
                        - pattern: os.getenv('$ANYTHING', ...)
                        - pattern: os.getenvb('$ANYTHING', ...)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-either:
                                - pattern: sys.argv
                                - pattern: sys.orig_argv
                        - patterns:
                            - pattern-inside: |
                                $PARSER = argparse.ArgumentParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-inside: |
                                $PARSER = optparse.OptionParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.getopt(...)
                                    ...
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.gnu_getopt(...)
                                    ...
                            - pattern-either:
                                - patterns:
                                    - pattern-inside: |
                                        for $O, $A in $OPTS:
                                          ...
                                    - pattern: $A
                                - pattern: $ARGS
      severity: WARNING
    - id: python.lang.security.audit.dangerous-os-exec-tainted-env-args.dangerous-os-exec-tainte
d-env-args
      languages:
        - python
      message: Found user controlled content when spawning a process. This is dangerous because
it allows a malicious actor to execute commands.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
  subcategory:
      - vuln
  technology:
      - python
mode: taint
options:
  symbolic_propagation: true
pattern-sinks:
  - patterns:
      - pattern-either:
          - patterns:
              - pattern-not: os.$METHOD("...", ...)
              - pattern: os.$METHOD(...)
              - metavariable-regex:
                  metavariable: $METHOD
                  regex: (execl|execle|execlp|execlpe|execv|execve|execvp|execvpe)
          - patterns:
              - pattern-not: os.$METHOD("...", [$PATH,"...","...",...],...)
              - pattern-inside: os.$METHOD($BASH,[$PATH,"-c",$CMD,...],...)
              - pattern: $CMD
              - metavariable-regex:
                  metavariable: $METHOD
                  regex: (execv|execve|execvp|execvpe)
              - metavariable-regex:
                  metavariable: $BASH
                  regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
          - patterns:
              - pattern-not: os.$METHOD("...", $PATH, "...", "...",...)
              - pattern-inside: os.$METHOD($BASH, $PATH, "-c", $CMD,...)
              - pattern: $CMD
              - metavariable-regex:
                  metavariable: $METHOD
                  regex: (execl|execle|execlp|execlpe)
              - metavariable-regex:
                  metavariable: $BASH
                  regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
pattern-sources:
  - patterns:
      - pattern-either:
          - patterns:
              - pattern-either:
                  - pattern: os.environ
                  - pattern: os.environ.get('$FOO', ...)
                  - pattern: os.environb
                  - pattern: os.environb.get('$FOO', ...)
                  - pattern: os.getenv('$ANYTHING', ...)
                  - pattern: os.getenvb('$ANYTHING', ...)
          - patterns:
              - pattern-either:
                  - patterns:
                      - pattern-either:
                          - pattern: sys.argv
                          - pattern: sys.orig_argv
                  - patterns:
                      - pattern-inside: |
                          $PARSER = argparse.ArgumentParser(...)
                          ...
                      - pattern-inside: |
                          $ARGS = $PARSER.parse_args()
                      - pattern: <... $ARGS ...>
                  - patterns:
                      - pattern-inside: |
                          $PARSER = optparse.OptionParser(...)
                          ...
                      - pattern-inside: |
                          $ARGS = $PARSER.parse_args()
                      - pattern: <... $ARGS ...>
                  - patterns:
                      - pattern-either:
                          - pattern-inside: |
                              $OPTS, $ARGS = getopt.getopt(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                                ...
                            - pattern-inside: |
                                $OPTS, $ARGS = getopt.gnu_getopt(...)
                                ...
                        - pattern-either:
                            - patterns:
                                - pattern-inside: |
                                    for $O, $A in $OPTS:
                                      ...
                                - pattern: $A
                            - pattern: $ARGS
      severity: ERROR
    - id: python.lang.security.audit.dangerous-spawn-process-tainted-env-args.dangerous-spawn-pr
ocess-tainted-env-args
      languages:
        - python
      message: Found user controlled content when spawning a process. This is dangerous because
it allows a malicious actor to execute commands.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/b605_start_process_with
_a_shell.html
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-not: os.$METHOD($MODE, "...", ...)
                    - pattern-inside: os.$METHOD($MODE, $CMD, ...)
                    - pattern: $CMD
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: (spawnl|spawnle|spawnlp|spawnlpe|spawnv|spawnve|spawnvp|spawnvp|s
pawnvpe|posix_spawn|posix_spawnp|startfile)
                - patterns:
                    - pattern-not: os.$METHOD($MODE, "...", ["...","..."...],...], ...)
                    - pattern-inside: os.$METHOD($MODE, $BASH, ["-c",$CMD,...],...)
                    - pattern: $CMD
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: (spawnv|spawnve|spawnvp|spawnvp|spawnvpe|posix_spawn|posix_spawnp
)
                    - metavariable-regex:
                        metavariable: $BASH
                        regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
                - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern-not: os.$METHOD($MODE, "...", "...", "...", ...)
                    - pattern-inside: os.$METHOD($MODE, $BASH, "-c", $CMD,...)
                    - pattern: $CMD
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: (spawnl|spawnle|spawnlp|spawnlpe)
                    - metavariable-regex:
                        metavariable: $BASH
                        regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: os.environ
                        - pattern: os.environ.get('$FOO', ...)
                        - pattern: os.environb
                        - pattern: os.environb.get('$FOO', ...)
                        - pattern: os.getenv('$ANYTHING', ...)
                        - pattern: os.getenvb('$ANYTHING', ...)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-either:
                                - pattern: sys.argv
                                - pattern: sys.orig_argv
                        - patterns:
                            - pattern-inside: |
                                $PARSER = argparse.ArgumentParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-inside: |
                                $PARSER = optparse.OptionParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.getopt(...)
                                    ...
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.gnu_getopt(...)
                                    ...
                            - pattern-either:
                                - patterns:
                                    - pattern-inside: |
                                        for $O, $A in $OPTS:
                                          ...
                                    - pattern: $A
                                - pattern: $ARGS
      severity: ERROR
    - id: python.lang.security.audit.dangerous-subinterpreters-run-string-tainted-env-args.dange
rous-subinterpreters-run-string-tainted-env-args
      languages:
        - python
      message: Found user controlled content in `run_string`. This is dangerous because it allow
s a malicious actor to run arbitrary Python code.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A03:2021 - Injection
        references:
            - https://bugs.python.org/issue43472
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-inside: |
                _xxsubinterpreters.run_string($ID, $PAYLOAD, ...)
            - pattern-not: |
                _xxsubinterpreters.run_string($ID, "...", ...)
            - pattern: $PAYLOAD
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: os.environ
                        - pattern: os.environ.get('$FOO', ...)
                        - pattern: os.environb
                        - pattern: os.environb.get('$FOO', ...)
                        - pattern: os.getenv('$ANYTHING', ...)
                        - pattern: os.getenvb('$ANYTHING', ...)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-either:
                                - pattern: sys.argv
                                - pattern: sys.orig_argv
                        - patterns:
                            - pattern-inside: |
                                $PARSER = argparse.ArgumentParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-inside: |
                                $PARSER = optparse.OptionParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.getopt(...)
                                    ...
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.gnu_getopt(...)
                                    ...
                            - pattern-either:
                                - patterns:
                                    - pattern-inside: |
                                        for $O, $A in $OPTS:
                                          ...
                                    - pattern: $A
                                - pattern: $ARGS
      severity: WARNING
    - id: python.lang.security.audit.dangerous-subprocess-use-tainted-env-args.dangerous-subproc
ess-use-tainted-env-args
      languages:
        - python
      message: Detected subprocess function '$FUNC' with user controlled data. A malicious actor
 could leverage this to perform command injection. You may consider using 'shlex.escape()'.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://stackoverflow.com/questions/3172470/actual-meaning-of-shell-true-in-subpro
cess
            - https://docs.python.org/3/library/subprocess.html
            - https://docs.python.org/3/library/shlex.html
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-not: subprocess.$FUNC("...", ...)
                    - pattern-not: subprocess.$FUNC(["...",...], ...)
                    - pattern-not: subprocess.$FUNC(("...",...), ...)
                    - pattern-not: subprocess.CalledProcessError(...)
                    - pattern-not: subprocess.SubprocessError(...)
                    - pattern: subprocess.$FUNC($CMD, ...)
                - patterns:
                    - pattern-not: subprocess.$FUNC("=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c","...",
...)
                    - pattern: subprocess.$FUNC("=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c", $CMD)
                - patterns:
                    - pattern-not: subprocess.$FUNC(["=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c","..."
,...],...)
                    - pattern-not: subprocess.$FUNC(("=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c","..."
,...),...)
                    - pattern-either:
                        - pattern: subprocess.$FUNC(["=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c", $CMD
], ...)
                        - pattern: subprocess.$FUNC(("=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c", $CMD
), ...)
                - patterns:
                    - pattern-not: subprocess.$FUNC("=~/(python)/","...",...)
                    - pattern: subprocess.$FUNC("=~/(python)/", $CMD)
                - patterns:
                    - pattern-not: subprocess.$FUNC(["=~/(python)/","...",...],...)
                    - pattern-not: subprocess.$FUNC(("=~/(python)/","...",...),...)
                    - pattern-either:
                        - pattern: subprocess.$FUNC(["=~/(python)/", $CMD],...)
                        - pattern: subprocess.$FUNC(("=~/(python)/", $CMD),...)
            - focus-metavariable: $CMD
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern: os.environ
                        - pattern: os.environ.get('$FOO', ...)
                        - pattern: os.environb
                        - pattern: os.environb.get('$FOO', ...)
                        - pattern: os.getenv('$ANYTHING', ...)
                        - pattern: os.getenvb('$ANYTHING', ...)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-either:
                                - pattern: sys.argv
                                - pattern: sys.orig_argv
                        - patterns:
                            - pattern-inside: |
                                $PARSER = argparse.ArgumentParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-inside: |
                                $PARSER = optparse.OptionParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.getopt(...)
                                    ...
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.gnu_getopt(...)
                                    ...
                            - pattern-either:
                                - patterns:
                                    - pattern-inside: |
                                        for $O, $A in $OPTS:
                                          ...
                                    - pattern: $A
                                - pattern: $ARGS
      severity: ERROR
    - id: python.lang.security.audit.dangerous-system-call-tainted-env-args.dangerous-system-cal
l-tainted-env-args
      languages:
        - python
      message: Found user-controlled data used in a system call. This could allow a malicious ac
tor to execute commands. Use the 'subprocess' module instead, which is easier to use without acc
identally exposing a command injection vulnerability.
      metadata:
        asvs:
            control_id: 5.2.4 Dyanmic Code Execution Features
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v52-sanitization-and-sandboxing-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/b605_start_process_with
_a_shell.html
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-not: os.$W("...", ...)
            - pattern-either:
                - pattern: os.system(...)
                - pattern: |
                    $X = __import__("os")
                    ...
                    $X.system(...)
                - pattern: |
                    $X = __import__("os")
                    ...
                    getattr($X, "system")(...)
                - pattern: |
                    $X = getattr(os, "system")
                    ...
                    $X(...)
                - pattern: |
                    $X = __import__("os")
                    ...
                    $Y = getattr($X, "system")
                    ...
                    $Y(...)
                - pattern: os.popen(...)
                - pattern: os.popen2(...)
                - pattern: os.popen3(...)
                - pattern: os.popen4(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: os.environ
                        - pattern: os.environ.get('$FOO', ...)
                        - pattern: os.environb
                        - pattern: os.environb.get('$FOO', ...)
                        - pattern: os.getenv('$ANYTHING', ...)
                        - pattern: os.getenvb('$ANYTHING', ...)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-either:
                                - pattern: sys.argv
                                - pattern: sys.orig_argv
                        - patterns:
                            - pattern-inside: |
                                $PARSER = argparse.ArgumentParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-inside: |
                                $PARSER = optparse.OptionParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.getopt(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                                ...
                            - pattern-inside: |
                                $OPTS, $ARGS = getopt.gnu_getopt(...)
                                ...
                        - pattern-either:
                            - patterns:
                                - pattern-inside: |
                                    for $O, $A in $OPTS:
                                      ...
                                - pattern: $A
                            - pattern: $ARGS
      severity: ERROR
    - id: python.lang.security.audit.dangerous-testcapi-run-in-subinterp-tainted-env-args.danger
ous-testcapi-run-in-subinterp-tainted-env-args
      languages:
        - python
      message: Found user controlled content in `run_in_subinterp`. This is dangerous because it
 allows a malicious actor to run arbitrary Python code.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    _testcapi.run_in_subinterp($PAYLOAD, ...)
                - pattern-inside: |
                    test.support.run_in_subinterp($PAYLOAD, ...)
            - pattern: $PAYLOAD
            - pattern-not: |
                _testcapi.run_in_subinterp("...", ...)
            - pattern-not: |
                test.support.run_in_subinterp("...", ...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: os.environ
                        - pattern: os.environ.get('$FOO', ...)
                        - pattern: os.environb
                        - pattern: os.environb.get('$FOO', ...)
                        - pattern: os.getenv('$ANYTHING', ...)
                        - pattern: os.getenvb('$ANYTHING', ...)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-either:
                                - pattern: sys.argv
                                - pattern: sys.orig_argv
                        - patterns:
                            - pattern-inside: |
                                $PARSER = argparse.ArgumentParser(...)
                                ...
                            - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                            $ARGS = $PARSER.parse_args()
                        - pattern: <... $ARGS ...>
                    - patterns:
                        - pattern-inside: |
                            $PARSER = optparse.OptionParser(...)
                            ...
                        - pattern-inside: |
                            $ARGS = $PARSER.parse_args()
                        - pattern: <... $ARGS ...>
                    - patterns:
                        - pattern-either:
                            - pattern-inside: |
                                $OPTS, $ARGS = getopt.getopt(...)
                                ...
                            - pattern-inside: |
                                $OPTS, $ARGS = getopt.gnu_getopt(...)
                                ...
                        - pattern-either:
                            - patterns:
                                - pattern-inside: |
                                    for $O, $A in $OPTS:
                                      ...
                                - pattern: $A
                            - pattern: $ARGS
      severity: WARNING
    - id: python.lang.security.audit.insecure-file-permissions.insecure-file-permissions
      languages:
        - python
      message: These permissions `$BITS` are widely permissive and grant access to more people t
han may be necessary. A good default is `0o644` which gives read and write access to yourself an
d read access to everyone else.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-276: Incorrect Default Permissions'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - python
      patterns:
        - pattern-inside: os.$METHOD(...)
        - metavariable-pattern:
            metavariable: $METHOD
            patterns:
                - pattern-either:
                    - pattern: chmod
                    - pattern: lchmod
                    - pattern: fchmod
        - pattern-either:
            - patterns:
                - pattern: os.$METHOD($FILE, $BITS, ...)
                - metavariable-comparison:
                    comparison: $BITS >= 0o650 and $BITS < 0o100000
                    metavariable: $BITS
            - patterns:
                - pattern: os.$METHOD($FILE, $BITS)
                - metavariable-comparison:
                    comparison: $BITS >= 0o100650
                    metavariable: $BITS
            - patterns:
                - pattern: os.$METHOD($FILE, $BITS, ...)
                - metavariable-pattern:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    metavariable: $BITS
                    patterns:
                        - pattern-either:
                            - pattern: <... stat.S_IWGRP ...>
                            - pattern: <... stat.S_IXGRP ...>
                            - pattern: <... stat.S_IWOTH ...>
                            - pattern: <... stat.S_IXOTH ...>
                            - pattern: <... stat.S_IRWXO ...>
                            - pattern: <... stat.S_IRWXG ...>
            - patterns:
                - pattern: os.$METHOD($FILE, $EXPR | $MOD, ...)
                - metavariable-comparison:
                    comparison: $MOD == 0o111
                    metavariable: $MOD
      severity: WARNING
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: python.lang.security.audit.insecure-transport.requests.request-session-http-in-with-co
ntext.request-session-http-in-with-context
      languages:
        - python
      message: Detected a request using 'http://'. This request will be unencrypted. Use 'https:
//' instead.
      metadata:
        asvs:
            control_id: 9.2.1 Weak TLS
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x17-V9-Communications
.md#v92-server-communications-security-requirements
            section: V9 Communications Verification Requirements
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: LOW
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - audit
        technology:
            - requests
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-inside: |
                with requests.Session(...) as $SESSION:
                  ...
            - pattern-either:
                - pattern: $SESSION.$W($SINK, ...)
                - pattern: $SESSION.request($METHOD, $SINK, ...)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern: |
                "$URL"
            - metavariable-pattern:
                language: regex
                metavariable: $URL
                patterns:
                    - pattern-regex: http://
                    - pattern-not-regex: .*://localhost
                    - pattern-not-regex: .*://127\.0\.0\.1
      severity: INFO
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: python.lang.security.audit.insecure-transport.requests.request-session-with-http.reque
st-session-with-http
      languages:
        - python
      message: Detected a request using 'http://'. This request will be unencrypted. Use 'https:
//' instead.
      metadata:
        asvs:
            control_id: 9.1.1 Weak TLS
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x17-V9-Communications
.md#v92-server-communications-security-requirements
            section: V9 Communications Verification Requirements
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: LOW
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - audit
        technology:
            - requests
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: requests.Session(...).$W($SINK, ...)
                - pattern: requests.Session(...).request($METHOD, $SINK, ...)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern: |
                "$URL"
            - metavariable-pattern:
                language: regex
                metavariable: $URL
                patterns:
                    - pattern-regex: http://
                    - pattern-not-regex: .*://localhost
                    - pattern-not-regex: .*://127\.0\.0\.1
      severity: INFO
    - fix-regex:
        count: 1
        regex: '[Hh][Tt][Tt][Pp]://'
        replacement: https://
      id: python.lang.security.audit.insecure-transport.requests.request-with-http.request-with-
http
      languages:
        - python
      message: Detected a request using 'http://'. This request will be unencrypted, and attacke
rs could listen into traffic on the network and be able to obtain sensitive information. Use 'ht
tps://' instead.
      metadata:
        asvs:
            control_id: 9.1.1 Weak TLS
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x17-V9-Communications
.md#v92-server-communications-security-requirements
            section: V9 Communications Verification Requirements
            version: "4"
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: LOW
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - audit
        technology:
            - requests
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: requests.$W($SINK, ...)
                - pattern: requests.request($METHOD, $SINK, ...)
                - pattern: requests.Request($METHOD, $SINK, ...)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern: |
                "$URL"
            - metavariable-pattern:
                language: regex
                metavariable: $URL
                patterns:
                    - pattern-regex: http://
                    - pattern-not-regex: .*://localhost
                    - pattern-not-regex: .*://127\.0\.0\.1
      severity: INFO
    - id: python.lang.security.audit.logging.logger-credential-leak.python-logger-credential-dis
closure
      languages:
        - python
      message: Detected a python logger call with a potential hardcoded secret $FORMAT_STRING be
ing logged. This may lead to secret credentials being exposed. Make sure that the logger is not
logging  sensitive information.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-532: Insertion of Sensitive Information into Log File'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A09:2021 - Security Logging and Monitoring Failures
        references:
            - https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures
        subcategory:
            - vuln
        technology:
            - python
      patterns:
        - pattern: |
            $LOGGER_OBJ.$LOGGER_CALL($FORMAT_STRING,...)
        - metavariable-regex:
            metavariable: $LOGGER_OBJ
            regex: (?i)(_logger|logger|self.logger|log)
        - metavariable-regex:
            metavariable: $LOGGER_CALL
            regex: (debug|info|warn|warning|error|exception|critical)
        - metavariable-regex:
            metavariable: $FORMAT_STRING
            regex: (?i).*(api.key|secret|credential|token|password).*\%s.*
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: WARNING
    - id: python.lang.security.audit.md5-used-as-password.md5-used-as-password
      languages:
        - python
      message: It looks like MD5 is used as a password hash. MD5 is not considered a secure pass
word hash because it can be cracked by an attacker in a short amount of time. Use a suitable pas
sword hashing function such as scrypt. You can use `hashlib.scrypt`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: LOW
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://tools.ietf.org/html/rfc6151
            - https://crypto.stackexchange.com/questions/44151/how-does-the-flame-malware-take-a
dvantage-of-md5-collision
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
            - https://security.stackexchange.com/questions/211/how-to-securely-hash-passwords
            - https://github.com/returntocorp/semgrep-rules/issues/1609
            - https://docs.python.org/3/library/hashlib.html#hashlib.scrypt
        subcategory:
            - vuln
        technology:
            - pycryptodome
            - hashlib
            - md5
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: $FUNCTION(...)
            - metavariable-regex:
                metavariable: $FUNCTION
                regex: (?i)(.*password.*)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: hashlib.md5
                - pattern: hashlib.new(..., name="MD5", ...)
                - pattern: Cryptodome.Hash.MD5
                - pattern: Crypto.Hash.MD5
                - pattern: cryptography.hazmat.primitives.hashes.MD5
      severity: WARNING
    - id: python.lang.security.audit.network.bind.avoid-bind-to-all-interfaces
      languages:
        - python
      message: Running `socket.bind` to 0.0.0.0, or empty string could unexpectedly expose the s
erver publicly as it binds to all available interfaces. Consider instead getting correct address
 from an environment variable or configuration file.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - python
      pattern-either:
        - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            $S = socket.socket(...)
            ...
            $S.bind(("0.0.0.0", ...))
        - pattern: |
            $S = socket.socket(...)
            ...
            $S.bind(("::", ...))
        - pattern: |
            $S = socket.socket(...)
            ...
            $S.bind(("", ...))
      severity: INFO
    - id: python.lang.security.audit.network.disabled-cert-validation.disabled-cert-validation
      languages:
        - python
      message: certificate verification explicitly disabled, insecure connections possible
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-295: Improper Certificate Validation'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures
        subcategory:
            - vuln
        technology:
            - python
      patterns:
        - pattern-either:
            - pattern: urllib3.PoolManager(..., cert_reqs=$REQS, ...)
            - pattern: urllib3.ProxyManager(..., cert_reqs=$REQS, ...)
            - pattern: urllib3.HTTPSConnectionPool(..., cert_reqs=$REQS, ...)
            - pattern: urllib3.connectionpool.HTTPSConnectionPool(..., cert_reqs=$REQS, ...)
            - pattern: urllib3.connection_from_url(..., cert_reqs=$REQS, ...)
            - pattern: urllib3.proxy_from_url(..., cert_reqs=$REQS, ...)
            - pattern: $CONTEXT.wrap_socket(..., cert_reqs=$REQS, ...)
            - pattern: ssl.wrap_socket(..., cert_reqs=$REQS, ...)
        - metavariable-regex:
            metavariable: $REQS
            regex: (NONE|CERT_NONE|CERT_OPTIONAL|ssl\.CERT_NONE|ssl\.CERT_OPTIONAL|\'NONE\'|\"NO
NE\"|\'OPTIONAL\'|\"OPTIONAL\")
      severity: ERROR
    - id: python.lang.security.audit.network.http-not-https-connection.http-not-https-connection
      languages:
        - python
      message: Detected HTTPConnectionPool. This will transmit data in cleartext. It is recommen
ded to use HTTPSConnectionPool instead for to encrypt communications.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://urllib3.readthedocs.io/en/1.2.1/pools.html#urllib3.connectionpool.HTTPSCon
nectionPool
        subcategory:
            - audit
        technology:
            - python
      pattern-either:
        - pattern: urllib3.HTTPConnectionPool(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern: urllib3.connectionpool.HTTPConnectionPool(...)
      severity: ERROR
    - id: python.lang.security.audit.ssl-wrap-socket-is-deprecated.ssl-wrap-socket-is-deprecated
      languages:
        - python
      message: '''ssl.wrap_socket()'' is deprecated. This function creates an insecure socket wi
thout server name indication or hostname matching. Instead, create an SSL context using ''ssl.SS
LContext()'' and use that to wrap a socket.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://docs.python.org/3/library/ssl.html#ssl.wrap_socket
            - https://docs.python.org/3/library/ssl.html#ssl.SSLContext.wrap_socket
        subcategory:
            - vuln
        technology:
            - python
      pattern: ssl.wrap_socket(...)
      severity: WARNING
    - fix-regex:
        regex: (shell\s*=\s*)True
        replacement: \1False
      id: python.lang.security.audit.subprocess-shell-true.subprocess-shell-true
      languages:
        - python
      message: Found 'subprocess' function '$FUNC' with 'shell=True'. This is dangerous because
this call will spawn the command using a shell process. Doing so propagates current shell settin
gs and variables, which makes it much easier for a malicious actor to execute commands. Use 'she
ll=False' instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: LOW
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://stackoverflow.com/questions/3172470/actual-meaning-of-shell-true-in-subpro
cess
            - https://docs.python.org/3/library/subprocess.html
        source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/b602_subprocess_popen_w
ith_shell_equals_true.html
        subcategory:
            - vuln
        technology:
            - python
      patterns:
        - pattern: subprocess.$FUNC(..., shell=True, ...)
        - pattern-not: subprocess.$FUNC("...", shell=True, ...)
      severity: ERROR
    - id: python.lang.security.audit.weak-ssl-version.weak-ssl-version
      languages:
        - python
      message: An insecure SSL version was detected. TLS versions 1.0, 1.1, and all SSL versions
 are considered weak encryption and are deprecated. Use 'ssl.PROTOCOL_TLSv1_2' or higher.
      metadata:
        asvs:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            control_id: 9.1.3 Weak TLS
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x17-V9-Communications
.md#v91-client-communications-security-requirements
            section: V9 Communications Verification Requirements
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://tools.ietf.org/html/rfc7568
            - https://tools.ietf.org/id/draft-ietf-tls-oldversions-deprecate-02.html
            - https://docs.python.org/3/library/ssl.html#ssl.PROTOCOL_TLSv1_2
        source-rule-url: https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee9
54c2b1/bandit/plugins/insecure_ssl_tls.py#L30
        subcategory:
            - audit
        technology:
            - python
      pattern-either:
        - pattern: ssl.PROTOCOL_SSLv2
        - pattern: ssl.PROTOCOL_SSLv3
        - pattern: ssl.PROTOCOL_TLSv1
        - pattern: ssl.PROTOCOL_TLSv1_1
        - pattern: pyOpenSSL.SSL.SSLv2_METHOD
        - pattern: pyOpenSSL.SSL.SSLv23_METHOD
        - pattern: pyOpenSSL.SSL.SSLv3_METHOD
        - pattern: pyOpenSSL.SSL.TLSv1_METHOD
        - pattern: pyOpenSSL.SSL.TLSv1_1_METHOD
      severity: WARNING
    - id: python.lang.security.dangerous-code-run.dangerous-interactive-code-run
      languages:
        - python
      message: Found user controlled data inside InteractiveConsole/InteractiveInterpreter metho
d. This is dangerous if external data can reach this function call because it allows a malicious
 actor to run arbitrary Python code.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $X = code.InteractiveConsole(...)
                    ...
                - pattern-inside: |
                    $X = code.InteractiveInterpreter(...)
                    ...
            - pattern-either:
                - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    $X.push($PAYLOAD,...)
                - pattern: |
                    $X.runsource($PAYLOAD,...)
                - pattern: |
                    $X.runcode(code.compile_command($PAYLOAD),...)
                - pattern: |
                    $PL = code.compile_command($PAYLOAD,...)
                    ...
                    $X.runcode($PL,...)
            - focus-metavariable: $PAYLOAD
            - pattern-not: |
                $X.push("...",...)
            - pattern-not: |
                $X.runsource("...",...)
            - pattern-not: |
                $X.runcode(code.compile_command("..."),...)
            - pattern-not: |
                $PL = code.compile_command("...",...)
                ...
                $X.runcode($PL,...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: flask.request.form.get(...)
                        - pattern: flask.request.form[...]
                        - pattern: flask.request.args.get(...)
                        - pattern: flask.request.args[...]
                        - pattern: flask.request.values.get(...)
                        - pattern: flask.request.values[...]
                        - pattern: flask.request.cookies.get(...)
                        - pattern: flask.request.cookies[...]
                        - pattern: flask.request.stream
                        - pattern: flask.request.headers.get(...)
                        - pattern: flask.request.headers[...]
                        - pattern: flask.request.data
                        - pattern: flask.request.full_path
                        - pattern: flask.request.url
                        - pattern: flask.request.json
                        - pattern: flask.request.get_json()
                        - pattern: flask.request.view_args.get(...)
                        - pattern: flask.request.view_args[...]
                        - patterns:
                            - pattern-inside: |
                                @$APP.route(...)
                                def $FUNC(..., $ROUTEVAR, ...):
                                  ...
                            - focus-metavariable: $ROUTEVAR
                - patterns:
                    - pattern-inside: |
                        def $FUNC(request, ...):
                          ...
                    - pattern-either:
                        - pattern: request.$PROPERTY.get(...)
                        - pattern: request.$PROPERTY[...]
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            @rest_framework.decorators.api_view(...)
                            def $FUNC($REQ, ...):
                              ...
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    class $VIEW(..., rest_framework.views.APIView, ...):
                                      ...
                                - pattern-inside: "class $VIEW(..., rest_framework.generics.Gene
ricAPIView, ...):\n  ...                                \n"
                            - pattern-inside: |
                                def $METHOD(self, $REQ, ...):
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                            ...
                        - metavariable-regex:
                            metavariable: $METHOD
                            regex: (get|post|put|patch|delete|head)
                    - pattern-either:
                        - pattern: $REQ.POST.get(...)
                        - pattern: $REQ.POST[...]
                        - pattern: $REQ.FILES.get(...)
                        - pattern: $REQ.FILES[...]
                        - pattern: $REQ.DATA.get(...)
                        - pattern: $REQ.DATA[...]
                        - pattern: $REQ.QUERY_PARAMS.get(...)
                        - pattern: $REQ.QUERY_PARAMS[...]
                        - pattern: $REQ.data.get(...)
                        - pattern: $REQ.data[...]
                        - pattern: $REQ.query_params.get(...)
                        - pattern: $REQ.query_params[...]
                        - pattern: $REQ.content_type
                        - pattern: $REQ.content_type
                        - pattern: $REQ.stream
                        - pattern: $REQ.stream
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            class $SERVER(..., http.server.BaseHTTPRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.StreamRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.DatagramRequestHandler, ...):
                              ...
                    - pattern-either:
                        - pattern: self.requestline
                        - pattern: self.path
                        - pattern: self.headers[...]
                        - pattern: self.headers.get(...)
                        - pattern: self.rfile
                - patterns:
                    - pattern-inside: |
                        @pyramid.view.view_config( ... )
                        def $VIEW($REQ):
                          ...
                    - pattern: $REQ.$ANYTHING
                    - pattern-not: $REQ.dbsession
      severity: WARNING
    - id: python.lang.security.dangerous-os-exec.dangerous-os-exec
      languages:
        - python
      message: Found user controlled content when spawning a process. This is dangerous because
it allows a malicious actor to execute commands.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
      subcategory:
        - vuln
      technology:
        - python
    mode: taint
    options:
      symbolic_propagation: true
    pattern-sinks:
      - patterns:
        - pattern-either:
            - patterns:
                - pattern-not: os.$METHOD("...", ...)
                - pattern: os.$METHOD(...)
                - metavariable-regex:
                    metavariable: $METHOD
                    regex: (execl|execle|execlp|execlpe|execv|execve|execvp|execvpe)
            - patterns:
                - pattern-not: os.$METHOD("...", [$PATH,"...","...",...],...)
                - pattern-inside: os.$METHOD($BASH,[$PATH,"-c",$CMD,...],...)
                - pattern: $CMD
                - metavariable-regex:
                    metavariable: $METHOD
                    regex: (execv|execve|execvp|execvpe)
                - metavariable-regex:
                    metavariable: $BASH
                    regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
            - patterns:
                - pattern-not: os.$METHOD("...", $PATH, "...", "...",...)
                - pattern-inside: os.$METHOD($BASH, $PATH, "-c", $CMD,...)
                - pattern: $CMD
                - metavariable-regex:
                    metavariable: $METHOD
                    regex: (execl|execle|execlp|execlpe)
                - metavariable-regex:
                    metavariable: $BASH
                    regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
    pattern-sources:
      - patterns:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: flask.request.form.get(...)
                    - pattern: flask.request.form[...]
                    - pattern: flask.request.args.get(...)
                    - pattern: flask.request.args[...]
                    - pattern: flask.request.values.get(...)
                    - pattern: flask.request.values[...]
                    - pattern: flask.request.cookies.get(...)
                    - pattern: flask.request.cookies[...]
                    - pattern: flask.request.stream
                    - pattern: flask.request.headers.get(...)
                    - pattern: flask.request.headers[...]
                    - pattern: flask.request.data
                    - pattern: flask.request.full_path
                    - pattern: flask.request.url
                    - pattern: flask.request.json
                    - pattern: flask.request.get_json()
                    - pattern: flask.request.view_args.get(...)
                    - pattern: flask.request.view_args[...]
                    - patterns:
                        - pattern-inside: |
                            @$APP.route(...)
                            def $FUNC(..., $ROUTEVAR, ...):
                              ...
                        - focus-metavariable: $ROUTEVAR
            - patterns:
                - pattern-inside: |
                    def $FUNC(request, ...):
                      ...
                - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern: request.$PROPERTY.get(...)
                        - pattern: request.$PROPERTY[...]
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            @rest_framework.decorators.api_view(...)
                            def $FUNC($REQ, ...):
                              ...
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    class $VIEW(..., rest_framework.views.APIView, ...):
                                      ...
                                - pattern-inside: "class $VIEW(..., rest_framework.generics.Gene
ricAPIView, ...):\n  ...                              \n"
                                - pattern-inside: |
                                    def $METHOD(self, $REQ, ...):
                                      ...
                                - metavariable-regex:
                                    metavariable: $METHOD
                                    regex: (get|post|put|patch|delete|head)
                        - pattern-either:
                            - pattern: $REQ.POST.get(...)
                            - pattern: $REQ.POST[...]
                            - pattern: $REQ.FILES.get(...)
                            - pattern: $REQ.FILES[...]
                            - pattern: $REQ.DATA.get(...)
                            - pattern: $REQ.DATA[...]
                            - pattern: $REQ.QUERY_PARAMS.get(...)
                            - pattern: $REQ.QUERY_PARAMS[...]
                            - pattern: $REQ.data.get(...)
                            - pattern: $REQ.data[...]
                            - pattern: $REQ.query_params.get(...)
                            - pattern: $REQ.query_params[...]
                            - pattern: $REQ.content_type
                            - pattern: $REQ.content_type
                            - pattern: $REQ.stream
                            - pattern: $REQ.stream
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            class $SERVER(..., http.server.BaseHTTPRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.StreamRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.DatagramRequestHandler, ...):
                              ...
                    - pattern-either:
                        - pattern: self.requestline
                        - pattern: self.path
                        - pattern: self.headers[...]
                        - pattern: self.headers.get(...)
                        - pattern: self.rfile
                - patterns:
                    - pattern-inside: |
                        @pyramid.view.view_config( ... )
                        def $VIEW($REQ):
                          ...
                    - pattern: $REQ.$ANYTHING
                    - pattern-not: $REQ.dbsession
      severity: ERROR
    - id: python.lang.security.dangerous-spawn-process.dangerous-spawn-process
      languages:
        - python
      message: Found user controlled content when spawning a process. This is dangerous because
it allows a malicious actor to execute commands.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/b605_start_process_with
_a_shell.html
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-not: os.$METHOD($MODE, "...", ...)
                    - pattern-inside: os.$METHOD($MODE, $CMD, ...)
                    - pattern: $CMD
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: (spawnl|spawnle|spawnlp|spawnlpe|spawnv|spawnve|spawnvp|spawnvp|s
pawnvpe|posix_spawn|posix_spawnp|startfile)
                - patterns:
                    - pattern-not: os.$METHOD($MODE, "...", ["...","..."...], ...)
                    - pattern-inside: os.$METHOD($MODE, $BASH, ["-c",$CMD,...],...)
                    - pattern: $CMD
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: (spawnv|spawnve|spawnvp|spawnvp|spawnvpe|posix_spawn|posix_spawnp
)
                    - metavariable-regex:
                        metavariable: $BASH
                        regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
                - patterns:
                    - pattern-not: os.$METHOD($MODE, "...", "...", "...", ...)
                    - pattern-inside: os.$METHOD($MODE, $BASH, "-c", $CMD,...)
                    - pattern: $CMD
                    - metavariable-regex:
                        metavariable: $METHOD
                        regex: (spawnl|spawnle|spawnlp|spawnlpe)
                    - metavariable-regex:
                        metavariable: $BASH
                        regex: (.*)(sh|bash|ksh|csh|tcsh|zsh)
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: flask.request.form.get(...)
                        - pattern: flask.request.form[...]
                        - pattern: flask.request.args.get(...)
                        - pattern: flask.request.args[...]
                        - pattern: flask.request.values.get(...)
                        - pattern: flask.request.values[...]
                        - pattern: flask.request.cookies.get(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern: flask.request.cookies[...]
                        - pattern: flask.request.stream
                        - pattern: flask.request.headers.get(...)
                        - pattern: flask.request.headers[...]
                        - pattern: flask.request.data
                        - pattern: flask.request.full_path
                        - pattern: flask.request.url
                        - pattern: flask.request.json
                        - pattern: flask.request.get_json()
                        - pattern: flask.request.view_args.get(...)
                        - pattern: flask.request.view_args[...]
                        - patterns:
                            - pattern-inside: |
                                @$APP.route(...)
                                def $FUNC(..., $ROUTEVAR, ...):
                                  ...
                            - pattern: $ROUTEVAR
                - patterns:
                    - pattern-inside: |
                        def $FUNC(request, ...):
                          ...
                    - pattern-either:
                        - pattern: request.$PROPERTY.get(...)
                        - pattern: request.$PROPERTY[...]
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            @rest_framework.decorators.api_view(...)
                            def $FUNC($REQ, ...):
                              ...
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    class $VIEW(..., rest_framework.views.APIView, ...):
                                      ...
                                - pattern-inside: "class $VIEW(..., rest_framework.generics.Gene
ricAPIView, ...):\n  ...                                \n"
                            - pattern-inside: |
                                def $METHOD(self, $REQ, ...):
                                  ...
                            - metavariable-regex:
                                metavariable: $METHOD
                                regex: (get|post|put|patch|delete|head)
                    - pattern-either:
                        - pattern: $REQ.POST.get(...)
                        - pattern: $REQ.POST[...]
                        - pattern: $REQ.FILES.get(...)
                        - pattern: $REQ.FILES[...]
                        - pattern: $REQ.DATA.get(...)
                        - pattern: $REQ.DATA[...]
                        - pattern: $REQ.QUERY_PARAMS.get(...)
                        - pattern: $REQ.QUERY_PARAMS[...]
                        - pattern: $REQ.data.get(...)
                        - pattern: $REQ.data[...]
                        - pattern: $REQ.query_params.get(...)
                        - pattern: $REQ.query_params[...]
                        - pattern: $REQ.content_type
                        - pattern: $REQ.content_type
                        - pattern: $REQ.stream
                        - pattern: $REQ.stream
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            class $SERVER(..., http.server.BaseHTTPRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.StreamRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.DatagramRequestHandler, ...):
                              ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern-either:
                        - pattern: self.requestline
                        - pattern: self.path
                        - pattern: self.headers[...]
                        - pattern: self.headers.get(...)
                        - pattern: self.rfile
                - patterns:
                    - pattern-inside: |
                        @pyramid.view.view_config( ... )
                        def $VIEW($REQ):
                          ...
                    - pattern: $REQ.$ANYTHING
                    - pattern-not: $REQ.dbsession
                - patterns:
                    - pattern-either:
                        - pattern: os.environ['$ANYTHING']
                        - pattern: os.environ.get('$FOO', ...)
                        - pattern: os.environb['$ANYTHING']
                        - pattern: os.environb.get('$FOO', ...)
                        - pattern: os.getenv('$ANYTHING', ...)
                        - pattern: os.getenvb('$ANYTHING', ...)
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern-either:
                                - pattern: sys.argv[...]
                                - pattern: sys.orig_argv[...]
                        - patterns:
                            - pattern-inside: |
                                $PARSER = argparse.ArgumentParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-inside: |
                                $PARSER = optparse.OptionParser(...)
                                ...
                            - pattern-inside: |
                                $ARGS = $PARSER.parse_args()
                            - pattern: <... $ARGS ...>
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.getopt(...)
                                    ...
                                - pattern-inside: |
                                    $OPTS, $ARGS = getopt.gnu_getopt(...)
                                    ...
                            - pattern-either:
                                - patterns:
                                    - pattern-inside: |
                                        for $O, $A in $OPTS:
                                          ...
                                    - pattern: $A
                                - pattern: $ARGS
      severity: ERROR
    - id: python.lang.security.dangerous-subinterpreters-run-string.dangerous-subinterpreters-ru
n-string
      languages:
        - python
      message: Found user controlled content in `run_string`. This is dangerous because it allow
s a malicious actor to run arbitrary Python code.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        owasp:
            - A03:2021 - Injection
        references:
            - https://bugs.python.org/issue43472
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern: |
                _xxsubinterpreters.run_string($ID, $PAYLOAD, ...)
            - pattern-not: |
                _xxsubinterpreters.run_string($ID, "...", ...)
            - focus-metavariable: $PAYLOAD
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: flask.request.form.get(...)
                        - pattern: flask.request.form[...]
                        - pattern: flask.request.args.get(...)
                        - pattern: flask.request.args[...]
                        - pattern: flask.request.values.get(...)
                        - pattern: flask.request.values[...]
                        - pattern: flask.request.cookies.get(...)
                        - pattern: flask.request.cookies[...]
                        - pattern: flask.request.stream
                        - pattern: flask.request.headers.get(...)
                        - pattern: flask.request.headers[...]
                        - pattern: flask.request.data
                        - pattern: flask.request.full_path
                        - pattern: flask.request.url
                        - pattern: flask.request.json
                        - pattern: flask.request.get_json()
                        - pattern: flask.request.view_args.get(...)
                        - pattern: flask.request.view_args[...]
                        - patterns:
                            - pattern-inside: |
                                @$APP.route(...)
                                def $FUNC(..., $ROUTEVAR, ...):
                                  ...
                            - focus-metavariable: $ROUTEVAR
                - patterns:
                    - pattern-inside: |
                        def $FUNC(request, ...):
                          ...
                    - pattern-either:
                        - pattern: request.$PROPERTY.get(...)
                        - pattern: request.$PROPERTY[...]
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            @rest_framework.decorators.api_view(...)
                            def $FUNC($REQ, ...):
                              ...
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    class $VIEW(..., rest_framework.views.APIView, ...):
                                      ...
                                - pattern-inside: "class $VIEW(..., rest_framework.generics.Gene
ricAPIView, ...):\n  ...                                \n"
                            - pattern-inside: |
                                def $METHOD(self, $REQ, ...):
                                  ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - metavariable-regex:
                            metavariable: $METHOD
                            regex: (get|post|put|patch|delete|head)
                    - pattern-either:
                        - pattern: $REQ.POST.get(...)
                        - pattern: $REQ.POST[...]
                        - pattern: $REQ.FILES.get(...)
                        - pattern: $REQ.FILES[...]
                        - pattern: $REQ.DATA.get(...)
                        - pattern: $REQ.DATA[...]
                        - pattern: $REQ.QUERY_PARAMS.get(...)
                        - pattern: $REQ.QUERY_PARAMS[...]
                        - pattern: $REQ.data.get(...)
                        - pattern: $REQ.data[...]
                        - pattern: $REQ.query_params.get(...)
                        - pattern: $REQ.query_params[...]
                        - pattern: $REQ.content_type
                        - pattern: $REQ.content_type
                        - pattern: $REQ.stream
                        - pattern: $REQ.stream
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            class $SERVER(..., http.server.BaseHTTPRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.StreamRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.DatagramRequestHandler, ...):
                              ...
                    - pattern-either:
                        - pattern: self.requestline
                        - pattern: self.path
                        - pattern: self.headers[...]
                        - pattern: self.headers.get(...)
                        - pattern: self.rfile
                - patterns:
                    - pattern-inside: |
                        @pyramid.view.view_config( ... )
                        def $VIEW($REQ):
                          ...
                    - pattern: $REQ.$ANYTHING
                    - pattern-not: $REQ.dbsession
      severity: WARNING
    - id: python.lang.security.dangerous-subprocess-use.dangerous-subprocess-use
      languages:
        - python
      message: Detected subprocess function '$FUNC' with user controlled data. A malicious actor
 could leverage this to perform command injection. You may consider using 'shlex.escape()'.
      metadata:
        asvs:
            control_id: 5.3.8 OS Command Injection
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v53-output-encoding-and-injection-prevention-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://stackoverflow.com/questions/3172470/actual-meaning-of-shell-true-in-subpro
cess
            - https://docs.python.org/3/library/subprocess.html
            - https://docs.python.org/3/library/shlex.html
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-not: subprocess.$FUNC("...", ...)
                    - pattern-not: subprocess.$FUNC(["...",...], ...)
                    - pattern-not: subprocess.$FUNC(("...",...), ...)
                    - pattern-not: subprocess.CalledProcessError(...)
                    - pattern-not: subprocess.SubprocessError(...)
                    - pattern: subprocess.$FUNC($CMD, ...)
                - patterns:
                    - pattern-not: subprocess.$FUNC("=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c","...",
...)
                    - pattern: subprocess.$FUNC("=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c", $CMD)
                - patterns:
                    - pattern-not: subprocess.$FUNC(["=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c","..."
,...],...)
                    - pattern-not: subprocess.$FUNC(("=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c","..."
,...),...)
                    - pattern-either:
                        - pattern: subprocess.$FUNC(["=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c", $CMD
], ...)
                        - pattern: subprocess.$FUNC(("=~/(sh|bash|ksh|csh|tcsh|zsh)/","-c", $CMD
), ...)
                - patterns:
                    - pattern-not: subprocess.$FUNC("=~/(python)/","...",...)
                    - pattern: subprocess.$FUNC("=~/(python)/", $CMD)
                - patterns:
                    - pattern-not: subprocess.$FUNC(["=~/(python)/","...",...],...)
                    - pattern-not: subprocess.$FUNC(("=~/(python)/","...",...),...)
                    - pattern-either:
                        - pattern: subprocess.$FUNC(["=~/(python)/", $CMD],...)
                        - pattern: subprocess.$FUNC(("=~/(python)/", $CMD),...)
            - focus-metavariable: $CMD
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: flask.request.form.get(...)
                        - pattern: flask.request.form[...]
                        - pattern: flask.request.args.get(...)
                        - pattern: flask.request.args[...]
                        - pattern: flask.request.values.get(...)
                        - pattern: flask.request.values[...]
                        - pattern: flask.request.cookies.get(...)
                        - pattern: flask.request.cookies[...]
                        - pattern: flask.request.stream
                        - pattern: flask.request.headers.get(...)
                        - pattern: flask.request.headers[...]
                        - pattern: flask.request.data
                        - pattern: flask.request.full_path
                        - pattern: flask.request.url
                        - pattern: flask.request.json
                        - pattern: flask.request.get_json()
                        - pattern: flask.request.view_args.get(...)
                        - pattern: flask.request.view_args[...]
                        - patterns:
                            - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                            @$APP.route(...)
                            def $FUNC(..., $ROUTEVAR, ...):
                              ...
                        - focus-metavariable: $ROUTEVAR
                - patterns:
                    - pattern-inside: |
                        def $FUNC(request, ...):
                          ...
                    - pattern-either:
                        - pattern: request.$PROPERTY.get(...)
                        - pattern: request.$PROPERTY[...]
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            @rest_framework.decorators.api_view(...)
                            def $FUNC($REQ, ...):
                              ...
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    class $VIEW(..., rest_framework.views.APIView, ...):
                                      ...
                                - pattern-inside: "class $VIEW(..., rest_framework.generics.Gene
ricAPIView, ...):\n  ...                                \n"
                            - pattern-inside: |
                                def $METHOD(self, $REQ, ...):
                                  ...
                            - metavariable-regex:
                                metavariable: $METHOD
                                regex: (get|post|put|patch|delete|head)
                    - pattern-either:
                        - pattern: $REQ.POST.get(...)
                        - pattern: $REQ.POST[...]
                        - pattern: $REQ.FILES.get(...)
                        - pattern: $REQ.FILES[...]
                        - pattern: $REQ.DATA.get(...)
                        - pattern: $REQ.DATA[...]
                        - pattern: $REQ.QUERY_PARAMS.get(...)
                        - pattern: $REQ.QUERY_PARAMS[...]
                        - pattern: $REQ.data.get(...)
                        - pattern: $REQ.data[...]
                        - pattern: $REQ.query_params.get(...)
                        - pattern: $REQ.query_params[...]
                        - pattern: $REQ.content_type
                        - pattern: $REQ.content_type
                        - pattern: $REQ.stream
                        - pattern: $REQ.stream
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            class $SERVER(..., http.server.BaseHTTPRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.StreamRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.DatagramRequestHandler, ...):
                              ...
                    - pattern-either:
                        - pattern: self.requestline
                        - pattern: self.path
                        - pattern: self.headers[...]
                        - pattern: self.headers.get(...)
                        - pattern: self.rfile
                - patterns:
                    - pattern-inside: |
                        @pyramid.view.view_config( ... )
                        def $VIEW($REQ):
                          ...
                    - pattern: $REQ.$ANYTHING
                    - pattern-not: $REQ.dbsession
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: ERROR
    - id: python.lang.security.dangerous-system-call.dangerous-system-call
      languages:
        - python
      message: Found user-controlled data used in a system call. This could allow a malicious ac
tor to execute commands. Use the 'subprocess' module instead, which is easier to use without acc
identally exposing a command injection vulnerability.
      metadata:
        asvs:
            control_id: 5.2.4 Dyanmic Code Execution Features
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x13-V5-Validation-San
itization-Encoding.md#v52-sanitization-and-sandboxing-requirements
            section: 'V5: Validation, Sanitization and Encoding Verification Requirements'
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        source-rule-url: https://bandit.readthedocs.io/en/latest/plugins/b605_start_process_with
_a_shell.html
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-not: os.$W("...", ...)
            - pattern-either:
                - pattern: os.system(...)
                - pattern: getattr(os, "system")(...)
                - pattern: __import__("os").system(...)
                - pattern: getattr(__import__("os"), "system")(...)
                - pattern: |
                    $X = __import__("os")
                    ...
                    $X.system(...)
                - pattern: |
                    $X = __import__("os")
                    ...
                    getattr($X, "system")(...)
                - pattern: |
                    $X = getattr(os, "system")
                    ...
                    $X(...)
                - pattern: |
                    $X = __import__("os")
                    ...
                    $Y = getattr($X, "system")
                    ...
                    $Y(...)
                - pattern: os.popen(...)
                - pattern: os.popen2(...)
                - pattern: os.popen3(...)
                - pattern: os.popen4(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern-either:
                        - pattern: flask.request.form.get(...)
                        - pattern: flask.request.form[...]
                        - pattern: flask.request.args.get(...)
                        - pattern: flask.request.args[...]
                        - pattern: flask.request.values.get(...)
                        - pattern: flask.request.values[...]
                        - pattern: flask.request.cookies.get(...)
                        - pattern: flask.request.cookies[...]
                        - pattern: flask.request.stream
                        - pattern: flask.request.headers.get(...)
                        - pattern: flask.request.headers[...]
                        - pattern: flask.request.data
                        - pattern: flask.request.full_path
                        - pattern: flask.request.url
                        - pattern: flask.request.json
                        - pattern: flask.request.get_json()
                        - pattern: flask.request.view_args.get(...)
                        - pattern: flask.request.view_args[...]
                        - patterns:
                            - pattern-inside: |
                                @$APP.route(...)
                                def $FUNC(..., $ROUTEVAR, ...):
                                  ...
                            - focus-metavariable: $ROUTEVAR
                - patterns:
                    - pattern-inside: |
                        def $FUNC(request, ...):
                          ...
                    - pattern-either:
                        - pattern: request.$PROPERTY.get(...)
                        - pattern: request.$PROPERTY[...]
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            @rest_framework.decorators.api_view(...)
                            def $FUNC($REQ, ...):
                              ...
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    class $VIEW(..., rest_framework.views.APIView, ...):
                                      ...
                                - pattern-inside: "class $VIEW(..., rest_framework.generics.Gene
ricAPIView, ...):\n  ...                            \n"
                            - pattern-inside: |
                                def $METHOD(self, $REQ, ...):
                                  ...
                            - metavariable-regex:
                                metavariable: $METHOD
                                regex: (get|post|put|patch|delete|head)
                    - pattern-either:
                        - pattern: $REQ.POST.get(...)
                        - pattern: $REQ.POST[...]
                        - pattern: $REQ.FILES.get(...)
                        - pattern: $REQ.FILES[...]
                        - pattern: $REQ.DATA.get(...)
                        - pattern: $REQ.DATA[...]
                        - pattern: $REQ.QUERY_PARAMS.get(...)
                        - pattern: $REQ.QUERY_PARAMS[...]
                        - pattern: $REQ.data.get(...)
                        - pattern: $REQ.data[...]
                        - pattern: $REQ.query_params.get(...)
                        - pattern: $REQ.query_params[...]
                        - pattern: $REQ.content_type
                        - pattern: $REQ.content_type
                        - pattern: $REQ.stream
                        - pattern: $REQ.stream
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                            class $SERVER(..., http.server.BaseHTTPRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.StreamRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.DatagramRequestHandler, ...):
                              ...
                    - pattern-either:
                        - pattern: self.requestline
                        - pattern: self.path
                        - pattern: self.headers[...]
                        - pattern: self.headers.get(...)
                        - pattern: self.rfile
                - patterns:
                    - pattern-inside: |
                        @pyramid.view.view_config( ... )
                        def $VIEW($REQ):
                          ...
                    - pattern: $REQ.$ANYTHING
                    - pattern-not: $REQ.dbsession
      severity: ERROR
    - id: python.lang.security.dangerous-testcapi-run-in-subinterp.dangerous-testcapi-run-in-sub
interp
      languages:
        - python
      message: Found user controlled content in `run_in_subinterp`. This is dangerous because it
 allows a malicious actor to run arbitrary Python code.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code (''Ev
al Injection'')'
        impact: HIGH
        likelihood: HIGH
        owasp:
            - A03:2021 - Injection
        references:
            - https://semgrep.dev/docs/cheat-sheets/python-command-injection/
        subcategory:
            - vuln
        technology:
            - python
      mode: taint
      options:
        symbolic_propagation: true
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    _testcapi.run_in_subinterp($PAYLOAD, ...)
                - pattern: |
                    test.support.run_in_subinterp($PAYLOAD, ...)
            - focus-metavariable: $PAYLOAD
            - pattern-not: |
                _testcapi.run_in_subinterp("...", ...)
            - pattern-not: |
                test.support.run_in_subinterp("...", ...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: flask.request.form.get(...)
                        - pattern: flask.request.form[...]
                        - pattern: flask.request.args.get(...)
                        - pattern: flask.request.args[...]
                        - pattern: flask.request.values.get(...)
                        - pattern: flask.request.values[...]
                        - pattern: flask.request.cookies.get(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern: flask.request.cookies[...]
                        - pattern: flask.request.stream
                        - pattern: flask.request.headers.get(...)
                        - pattern: flask.request.headers[...]
                        - pattern: flask.request.data
                        - pattern: flask.request.full_path
                        - pattern: flask.request.url
                        - pattern: flask.request.json
                        - pattern: flask.request.get_json()
                        - pattern: flask.request.view_args.get(...)
                        - pattern: flask.request.view_args[...]
                        - patterns:
                            - pattern-inside: |
                                @$APP.route(...)
                                def $FUNC(..., $ROUTEVAR, ...):
                                  ...
                            - focus-metavariable: $ROUTEVAR
                - patterns:
                    - pattern-inside: |
                        def $FUNC(request, ...):
                          ...
                    - pattern-either:
                        - pattern: request.$PROPERTY.get(...)
                        - pattern: request.$PROPERTY[...]
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            @rest_framework.decorators.api_view(...)
                            def $FUNC($REQ, ...):
                              ...
                        - patterns:
                            - pattern-either:
                                - pattern-inside: |
                                    class $VIEW(..., rest_framework.views.APIView, ...):
                                      ...
                                - pattern-inside: "class $VIEW(..., rest_framework.generics.Gene
ricAPIView, ...):\n  ...                                \n"
                            - pattern-inside: |
                                def $METHOD(self, $REQ, ...):
                                  ...
                            - metavariable-regex:
                                metavariable: $METHOD
                                regex: (get|post|put|patch|delete|head)
                    - pattern-either:
                        - pattern: $REQ.POST.get(...)
                        - pattern: $REQ.POST[...]
                        - pattern: $REQ.FILES.get(...)
                        - pattern: $REQ.FILES[...]
                        - pattern: $REQ.DATA.get(...)
                        - pattern: $REQ.DATA[...]
                        - pattern: $REQ.QUERY_PARAMS.get(...)
                        - pattern: $REQ.QUERY_PARAMS[...]
                        - pattern: $REQ.data.get(...)
                        - pattern: $REQ.data[...]
                        - pattern: $REQ.query_params.get(...)
                        - pattern: $REQ.query_params[...]
                        - pattern: $REQ.content_type
                        - pattern: $REQ.content_type
                        - pattern: $REQ.stream
                        - pattern: $REQ.stream
                - patterns:
                    - pattern-either:
                        - pattern-inside: |
                            class $SERVER(..., http.server.BaseHTTPRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.StreamRequestHandler, ...):
                              ...
                        - pattern-inside: |
                            class $SERVER(..., http.server.DatagramRequestHandler, ...):
                              ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern-either:
                        - pattern: self.requestline
                        - pattern: self.path
                        - pattern: self.headers[...]
                        - pattern: self.headers.get(...)
                        - pattern: self.rfile
                - patterns:
                    - pattern-inside: |
                        @pyramid.view.view_config( ... )
                        def $VIEW($REQ):
                          ...
                    - pattern: $REQ.$ANYTHING
                    - pattern-not: $REQ.dbsession
      severity: WARNING
    - fix-regex:
        count: 1
        regex: unsafe_load
        replacement: safe_load
      id: python.lang.security.deserialization.avoid-pyyaml-load.avoid-pyyaml-load
      languages:
        - python
      message: Detected a possible YAML deserialization vulnerability. `yaml.unsafe_load`, `yaml
.Loader`, `yaml.CLoader`, and `yaml.UnsafeLoader` are all known to be unsafe methods of deserial
izing YAML. An attacker with control over the YAML input could create special YAML input that al
lows the attacker to run arbitrary Python code. This would allow the attacker to steal files, do
wnload and install malware, or otherwise take over the machine. Use `yaml.safe_load` or `yaml.Sa
feLoader` instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://github.com/yaml/pyyaml/wiki/PyYAML-yaml.load(input)-Deprecation
            - https://nvd.nist.gov/vuln/detail/CVE-2017-18342
        subcategory:
            - audit
        technology:
            - pyyaml
      patterns:
        - pattern-inside: |
            import yaml
            ...
        - pattern-not-inside: |
            $YAML = ruamel.yaml.YAML(...)
            ...
        - pattern-either:
            - pattern: yaml.unsafe_load(...)
            - pattern: yaml.load(..., Loader=yaml.Loader, ...)
            - pattern: yaml.load(..., Loader=yaml.UnsafeLoader, ...)
            - pattern: yaml.load(..., Loader=yaml.CLoader, ...)
            - pattern: yaml.load_all(..., Loader=yaml.Loader, ...)
            - pattern: yaml.load_all(..., Loader=yaml.UnsafeLoader, ...)
            - pattern: yaml.load_all(..., Loader=yaml.CLoader, ...)
      severity: ERROR
    - id: python.lang.security.deserialization.avoid-unsafe-ruamel.avoid-unsafe-ruamel
      languages:
        - python
      message: Avoid using unsafe `ruamel.yaml.YAML()`. `ruamel.yaml.YAML` can create arbitrary
Python objects. A malicious actor could exploit this to run arbitrary code. Use `YAML(typ='rt')`
 or `YAML(typ='safe')` instead.
      metadata:
        category: security
        confidence: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://yaml.readthedocs.io/en/latest/basicuse.html?highlight=typ
        subcategory:
            - audit
        technology:
            - ruamel.yaml
      pattern-either:
        - pattern: ruamel.yaml.YAML(..., typ='unsafe', ...)
        - pattern: ruamel.yaml.YAML(..., typ='base', ...)
      severity: ERROR
    - id: python.lang.security.deserialization.pickle.avoid-shelve
      languages:
        - python
      message: Avoid using `shelve`, which uses `pickle`, which is known to lead to code executi
on vulnerabilities. When unpickling, the serialized data could be manipulated to run arbitrary c
ode. Instead, consider serializing the relevant data as JSON or a similar text-based serializati
on format.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://docs.python.org/3/library/pickle.html
        subcategory:
            - audit
        technology:
            - python
      pattern: shelve.$FUNC(...)
      severity: WARNING
    - id: python.lang.security.insecure-hash-algorithms-md5.insecure-hash-algorithm-md5
      languages:
        - python
      message: Detected MD5 hash algorithm which is considered insecure. MD5 is not collision re
sistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
      metadata:
        asvs:
            control_id: 6.2.2 Insecure Custom Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        bandit-code: B303
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.schneier.com/blog/archives/2012/10/when_will_we_se.html
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://www.trendmicro.com/vinfo/us/security/news/vulnerabilities-and-exploits/sha
-1-collision-signals-the-end-of-the-algorithm-s-viability
            - http://2012.sharcs.org/slides/stevens.pdf
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L59
        subcategory:
            - vuln
        technology:
            - python
      patterns:
        - pattern: hashlib.md5(...)
        - pattern-not: hashlib.md5(..., usedforsecurity=False, ...)
      severity: WARNING
    - fix-regex:
        regex: sha1
        replacement: sha256
      id: python.lang.security.insecure-hash-algorithms.insecure-hash-algorithm-sha1
      languages:
        - python
      message: Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision
resistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead
.
      metadata:
        asvs:
            control_id: 6.2.2 Insecure Custom Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        bandit-code: B303
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.schneier.com/blog/archives/2012/10/when_will_we_se.html
            - https://www.trendmicro.com/vinfo/us/security/news/vulnerabilities-and-exploits/sha
-1-collision-signals-the-end-of-the-algorithm-s-viability
            - http://2012.sharcs.org/slides/stevens.pdf
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L59
        subcategory:
            - vuln
        technology:
            - python
      pattern: hashlib.sha1(...)
      severity: WARNING
    - id: python.lang.security.insecure-hash-function.insecure-hash-function
      languages:
        - python
      message: Detected use of an insecure MD4 or MD5 hash function. These functions have known
vulnerabilities and are considered deprecated. Consider using 'SHA256' or a similar function ins
tead.
      metadata:
        asvs:
            control_id: 6.2.2 Insecure Custom Algorithm
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x14-V6-Cryptography.m
d#v62-algorithms
            section: V6 Stored Cryptography Verification Requirements
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://tools.ietf.org/html/rfc6151
            - https://crypto.stackexchange.com/questions/44151/how-does-the-flame-malware-take-a
dvantage-of-md5-collision
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee9
54c2b1/bandit/plugins/hashlib_new_insecure_functions.py
        subcategory:
            - audit
        technology:
            - python
      pattern-either:
        - pattern: hashlib.new("=~/[M|m][D|d][4|5]/", ...)
        - pattern: hashlib.new(..., name="=~/[M|m][D|d][4|5]/", ...)
      severity: WARNING
    - fix-regex:
        regex: _create_unverified_context
        replacement: create_default_context
      id: python.lang.security.unverified-ssl-context.unverified-ssl-context
      languages:
        - python
      message: Unverified SSL context detected. This will permit insecure connections without ve
rifying SSL certificates. Use 'ssl.create_default_context' instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-295: Improper Certificate Validation'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://docs.python.org/3/library/ssl.html#ssl-security
            - https://docs.python.org/3/library/http.client.html#http.client.HTTPSConnection
        subcategory:
            - audit
        technology:
            - python
      patterns:
        - pattern-either:
            - pattern: ssl._create_unverified_context(...)
            - pattern: ssl._create_default_https_context = ssl._create_unverified_context
      severity: ERROR
    - fix: defusedxml.etree.ElementTree.parse($...ARGS)
      id: python.lang.security.use-defused-xml-parse.use-defused-xml-parse
      languages:
        - python
      message: The native Python `xml` library is vulnerable to XML External Entity (XXE) attack
s.  These attacks can leak confidential data and "XML bombs" can cause denial of service. Do not
 use this library to parse untrusted input. Instead  the Python documentation recommends using `
defusedxml`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://docs.python.org/3/library/xml.html
            - https://github.com/tiran/defusedxml
            - https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processi
ng
        subcategory:
            - vuln
        technology:
            - python
      patterns:
        - pattern: xml.etree.ElementTree.parse($...ARGS)
        - pattern-not: xml.etree.ElementTree.parse("...")
      severity: ERROR
    - id: python.pycryptodome.security.insecure-cipher-algorithm-blowfish.insecure-cipher-algori
thm-blowfish
      languages:
        - python
      message: Detected Blowfish cipher algorithm which is considered insecure. This algorithm i
s not cryptographically secure and can be reversed easily. Use AES instead.
      metadata:
        bandit-code: B304
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://stackoverflow.com/questions/1135186/whats-wrong-with-xor-encryption
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L84
        subcategory:
            - vuln
        technology:
            - pycryptodome
      pattern-either:
        - pattern: Cryptodome.Cipher.Blowfish.new(...)
        - pattern: Crypto.Cipher.Blowfish.new(...)
      severity: WARNING
    - id: python.pycryptodome.security.insecure-cipher-algorithm-des.insecure-cipher-algorithm-d
es
      languages:
        - python
      message: Detected DES cipher algorithm which is considered insecure. This algorithm is not
 cryptographically secure and can be reversed easily. Use AES instead.
      metadata:
        bandit-code: B304
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cwe.mitre.org/data/definitions/326.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L84
        subcategory:
            - vuln
        technology:
            - pycryptodome
      pattern-either:
        - pattern: Cryptodome.Cipher.DES.new(...)
        - pattern: Crypto.Cipher.DES.new(...)
      severity: WARNING
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: python.pycryptodome.security.insecure-cipher-algorithm-rc2.insecure-cipher-algorithm-r
c2
      languages:
        - python
      message: Detected RC2 cipher algorithm which is considered insecure. This algorithm is not
 cryptographically secure and can be reversed easily. Use AES instead.
      metadata:
        bandit-code: B304
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cwe.mitre.org/data/definitions/326.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L84
        subcategory:
            - vuln
        technology:
            - pycryptodome
      pattern-either:
        - pattern: Cryptodome.Cipher.ARC2.new(...)
        - pattern: Crypto.Cipher.ARC2.new(...)
      severity: WARNING
    - id: python.pycryptodome.security.insecure-cipher-algorithm-rc4.insecure-cipher-algorithm-r
c4
      languages:
        - python
      message: Detected ARC4 cipher algorithm which is considered insecure. This algorithm is no
t cryptographically secure and can be reversed easily. Use AES instead.
      metadata:
        bandit-code: B304
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cwe.mitre.org/data/definitions/326.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L84
        subcategory:
            - vuln
        technology:
            - pycryptodome
      pattern-either:
        - pattern: Cryptodome.Cipher.ARC4.new(...)
        - pattern: Crypto.Cipher.ARC4.new(...)
      severity: WARNING
    - id: python.pycryptodome.security.insecure-cipher-algorithm.insecure-cipher-algorithm-xor
      languages:
        - python
      message: Detected XOR cipher algorithm which is considered insecure. This algorithm is not
 cryptographically secure and can be reversed easily. Use AES instead.
      metadata:
        bandit-code: B304
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://stackoverflow.com/questions/1135186/whats-wrong-with-xor-encryption
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L84
        subcategory:
            - vuln
        technology:
            - pycryptodome
      pattern-either:
        - pattern: Cryptodome.Cipher.XOR.new(...)
        - pattern: Crypto.Cipher.XOR.new(...)
      severity: WARNING
    - id: python.pycryptodome.security.insecure-hash-algorithm-md2.insecure-hash-algorithm-md2
      languages:
        - python
      message: Detected MD2 hash algorithm which is considered insecure. MD2 is not collision re
sistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.schneier.com/blog/archives/2012/10/when_will_we_se.html
            - https://www.trendmicro.com/vinfo/us/security/news/vulnerabilities-and-exploits/sha
-1-collision-signals-the-end-of-the-algorithm-s-viability
            - http://2012.sharcs.org/slides/stevens.pdf
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L59
        subcategory:
            - vuln
        technology:
            - pycryptodome
      pattern-either:
        - pattern: Crypto.Hash.MD2.new(...)
        - pattern: Cryptodome.Hash.MD2.new (...)
      severity: WARNING
    - id: python.pycryptodome.security.insecure-hash-algorithm-md4.insecure-hash-algorithm-md4
      languages:
        - python
      message: Detected MD4 hash algorithm which is considered insecure. MD4 is not collision re
sistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.schneier.com/blog/archives/2012/10/when_will_we_se.html
            - https://www.trendmicro.com/vinfo/us/security/news/vulnerabilities-and-exploits/sha
-1-collision-signals-the-end-of-the-algorithm-s-viability
            - http://2012.sharcs.org/slides/stevens.pdf
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L59
        subcategory:
            - vuln
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        technology:
            - pycryptodome
      pattern-either:
        - pattern: Crypto.Hash.MD4.new(...)
        - pattern: Cryptodome.Hash.MD4.new (...)
      severity: WARNING
    - id: python.pycryptodome.security.insecure-hash-algorithm-md5.insecure-hash-algorithm-md5
      languages:
        - python
      message: Detected MD5 hash algorithm which is considered insecure. MD5 is not collision re
sistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.schneier.com/blog/archives/2012/10/when_will_we_se.html
            - https://www.trendmicro.com/vinfo/us/security/news/vulnerabilities-and-exploits/sha
-1-collision-signals-the-end-of-the-algorithm-s-viability
            - http://2012.sharcs.org/slides/stevens.pdf
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L59
        subcategory:
            - vuln
        technology:
            - pycryptodome
      pattern-either:
        - pattern: Crypto.Hash.MD5.new(...)
        - pattern: Cryptodome.Hash.MD5.new (...)
      severity: WARNING
    - id: python.pycryptodome.security.insecure-hash-algorithm.insecure-hash-algorithm-sha1
      languages:
        - python
      message: Detected SHA1 hash algorithm which is considered insecure. SHA1 is not collision
resistant and is therefore not suitable as a cryptographic signature. Use SHA256 or SHA3 instead
.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.schneier.com/blog/archives/2012/10/when_will_we_se.html
            - https://www.trendmicro.com/vinfo/us/security/news/vulnerabilities-and-exploits/sha
-1-collision-signals-the-end-of-the-algorithm-s-viability
            - http://2012.sharcs.org/slides/stevens.pdf
            - https://pycryptodome.readthedocs.io/en/latest/src/hash/sha3_256.html
        source-rule-url: https://github.com/PyCQA/bandit/blob/d5f8fa0d89d7b11442fc6ec80ca4295397
4354c8/bandit/blacklists/calls.py#L59
        subcategory:
            - vuln
        technology:
            - pycryptodome
      pattern-either:
        - pattern: Crypto.Hash.SHA.new(...)
        - pattern: Cryptodome.Hash.SHA.new (...)
      severity: WARNING
    - id: python.pycryptodome.security.insufficient-dsa-key-size.insufficient-dsa-key-size
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - python
      message: Detected an insufficient key size for DSA. NIST recommends a key size of 2048 or
higher.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57Pt3r1.pdf
        source-rule-url: https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee9
54c2b1/bandit/plugins/weak_cryptographic_key.py
        subcategory:
            - vuln
        technology:
            - pycryptodome
      patterns:
        - pattern-either:
            - pattern: Crypto.PublicKey.DSA.generate(..., bits=$SIZE, ...)
            - pattern: Crypto.PublicKey.DSA.generate($SIZE, ...)
            - pattern: Cryptodome.PublicKey.DSA.generate(..., bits=$SIZE, ...)
            - pattern: Cryptodome.PublicKey.DSA.generate($SIZE, ...)
        - metavariable-comparison:
            comparison: $SIZE < 2048
            metavariable: $SIZE
      severity: WARNING
    - id: python.pycryptodome.security.insufficient-rsa-key-size.insufficient-rsa-key-size
      languages:
        - python
      message: Detected an insufficient key size for RSA. NIST recommends a key size of 2048 or
higher.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57Pt3r1.pdf
        source-rule-url: https://github.com/PyCQA/bandit/blob/b1411bfb43795d3ffd268bef17a839dee9
54c2b1/bandit/plugins/weak_cryptographic_key.py
        subcategory:
            - vuln
        technology:
            - pycryptodome
      patterns:
        - pattern-either:
            - pattern: Crypto.PublicKey.RSA.generate(..., bits=$SIZE, ...)
            - pattern: Crypto.PublicKey.RSA.generate($SIZE, ...)
            - pattern: Cryptodome.PublicKey.RSA.generate(..., bits=$SIZE, ...)
            - pattern: Cryptodome.PublicKey.RSA.generate($SIZE, ...)
        - metavariable-comparison:
            comparison: $SIZE < 2048
            metavariable: $SIZE
      severity: WARNING
    - id: python.pycryptodome.security.mode-without-authentication.crypto-mode-without-authentic
ation
      languages:
        - python
      message: 'An encryption mode of operation is being used without proper message authenticat
ion. This can potentially result in the encrypted content to be decrypted by an attacker. Consid
er instead use an AEAD mode of operation like GCM. '
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - cryptography
      patterns:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: |
                        AES.new(..., $PYCRYPTODOME_MODE)
                - pattern-not-inside: |
                    AES.new(..., $PYCRYPTODOME_MODE)
                    ...
                    HMAC.new
                - metavariable-pattern:
                    metavariable: $PYCRYPTODOME_MODE
                    patterns:
                        - pattern-either:
                            - pattern: AES.MODE_CBC
                            - pattern: AES.MODE_CTR
                            - pattern: AES.MODE_CFB
                            - pattern: AES.MODE_OFB
      severity: ERROR
    - fix-regex:
        regex: MONGODB-CR
        replacement: SCRAM-SHA-256
      id: python.pymongo.security.mongodb.mongo-client-bad-auth
      languages:
        - python
      message: Warning MONGODB-CR was deprecated with the release of MongoDB 3.6 and is no longe
r supported by MongoDB 4.0 (see https://api.mongodb.com/python/current/examples/authentication.h
tml for details).
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-477: Use of Obsolete Function'
        impact: LOW
        likelihood: LOW
        references:
            - https://cwe.mitre.org/data/definitions/477.html
        subcategory:
            - vuln
        technology:
            - pymongo
      pattern: |
        pymongo.MongoClient(..., authMechanism='MONGODB-CR')
      severity: WARNING
    - fix: |
        $...PARAMS, httponly=True
      id: python.pyramid.audit.authtkt-cookie-httponly-unsafe-default.pyramid-authtkt-cookie-htt
ponly-unsafe-default
      languages:
        - python
      message: Found a Pyramid Authentication Ticket cookie without the httponly option correctl
y set. Pyramid cookies should be handled securely by setting httponly=True. If this parameter is
 not properly set, your cookies are not properly protected and are at risk of being stolen by an
 attacker.
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1004: Sensitive Cookie Without ''HttpOnly'' Flag'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern: pyramid.authentication.$FUNC($...PARAMS)
        - metavariable-pattern:
            metavariable: $FUNC
            pattern-either:
                - pattern: AuthTktCookieHelper
                - pattern: AuthTktAuthenticationPolicy
        - pattern-not: pyramid.authentication.$FUNC(..., httponly=$HTTPONLY, ...)
        - pattern-not: pyramid.authentication.$FUNC(..., **$PARAMS, ...)
        - focus-metavariable: $...PARAMS
      severity: WARNING
    - fix: |
        True
      id: python.pyramid.audit.authtkt-cookie-httponly-unsafe-value.pyramid-authtkt-cookie-httpo
nly-unsafe-value
      languages:
        - python
      message: Found a Pyramid Authentication Ticket cookie without the httponly option correctl
y set. Pyramid cookies should be handled securely by setting httponly=True. If this parameter is
 not properly set, your cookies are not properly protected and are at risk of being stolen by an
 attacker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1004: Sensitive Cookie Without ''HttpOnly'' Flag'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-either:
            - patterns:
                - pattern-not: pyramid.authentication.AuthTktCookieHelper(..., **$PARAMS)
                - pattern: pyramid.authentication.AuthTktCookieHelper(..., httponly=$HTTPONLY, .
..)
            - patterns:
                - pattern-not: pyramid.authentication.AuthTktAuthenticationPolicy(..., **$PARAMS
)
                - pattern: pyramid.authentication.AuthTktAuthenticationPolicy(..., httponly=$HTT
PONLY, ...)
        - pattern: $HTTPONLY
        - metavariable-pattern:
            metavariable: $HTTPONLY
            pattern: |
                False
      severity: WARNING
    - fix: |
        'Lax'
      id: python.pyramid.audit.authtkt-cookie-samesite.pyramid-authtkt-cookie-samesite
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - python
      message: Found a Pyramid Authentication Ticket without the samesite option correctly set.
Pyramid cookies should be handled securely by setting samesite='Lax'. If this parameter is not p
roperly set, your cookies are not properly protected and are at risk of being stolen by an attac
ker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1275: Sensitive Cookie with Improper SameSite Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-either:
            - pattern: pyramid.authentication.AuthTktCookieHelper(..., samesite=$SAMESITE, ...)
            - pattern: pyramid.authentication.AuthTktAuthenticationPolicy(..., samesite=$SAMESIT
E, ...)
        - pattern: $SAMESITE
        - metavariable-regex:
            metavariable: $SAMESITE
            regex: (?!'Lax')
      severity: WARNING
    - fix-regex:
        regex: (.*)\)
        replacement: \1, secure=True)
      id: python.pyramid.audit.authtkt-cookie-secure-unsafe-default.pyramid-authtkt-cookie-secur
e-unsafe-default
      languages:
        - python
      message: Found a Pyramid Authentication Ticket cookie using an unsafe default for the secu
re option. Pyramid cookies should be handled securely by setting secure=True. If this parameter
is not properly set, your cookies are not properly protected and are at risk of being stolen by
an attacker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-614: Sensitive Cookie in HTTPS Session Without ''Secure'' Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-either:
            - patterns:
                - pattern-not: pyramid.authentication.AuthTktCookieHelper(..., secure=$SECURE, .
..)
                - pattern-not: pyramid.authentication.AuthTktCookieHelper(..., **$PARAMS)
                - pattern: pyramid.authentication.AuthTktCookieHelper(...)
            - patterns:
                - pattern-not: pyramid.authentication.AuthTktAuthenticationPolicy(..., secure=$S
ECURE, ...)
                - pattern-not: pyramid.authentication.AuthTktAuthenticationPolicy(..., **$PARAMS
)
                - pattern: pyramid.authentication.AuthTktAuthenticationPolicy(...)
      severity: WARNING
    - fix: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        True
      id: python.pyramid.audit.authtkt-cookie-secure-unsafe-value.pyramid-authtkt-cookie-secure-
unsafe-value
      languages:
        - python
      message: Found a Pyramid Authentication Ticket cookie without the secure option correctly
set. Pyramid cookies should be handled securely by setting secure=True. If this parameter is not
 properly set, your cookies are not properly protected and are at risk of being stolen by an att
acker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-614: Sensitive Cookie in HTTPS Session Without ''Secure'' Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-either:
            - patterns:
                - pattern-not: pyramid.authentication.AuthTktCookieHelper(..., **$PARAMS)
                - pattern: pyramid.authentication.AuthTktCookieHelper(..., secure=$SECURE, ...)
            - patterns:
                - pattern-not: pyramid.authentication.AuthTktAuthenticationPolicy(..., **$PARAMS
)
                - pattern: pyramid.authentication.AuthTktAuthenticationPolicy(..., secure=$SECUR
E, ...)
        - pattern: $SECURE
        - metavariable-pattern:
            metavariable: $SECURE
            pattern: |
                False
      severity: WARNING
    - fix: |
        True
      id: python.pyramid.audit.csrf-origin-check-disabled-globally.pyramid-csrf-origin-check-dis
abled-globally
      languages:
        - python
      message: Automatic check of the referrer for cross-site request forgery tokens has been ex
plicitly disabled globally, which might leave views unprotected when an unsafe CSRF storage poli
cy is used. Use 'pyramid.config.Configurator.set_default_csrf_options(check_origin=True)' to tur
n the automatic check for all unsafe methods (per RFC2616).
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-352: Cross-Site Request Forgery (CSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-inside: |
            $CONFIG.set_default_csrf_options(..., check_origin=$CHECK_ORIGIN, ...)
        - pattern: $CHECK_ORIGIN
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - metavariable-comparison:
            comparison: $CHECK_ORIGIN == False
            metavariable: $CHECK_ORIGIN
      severity: ERROR
    - fix: |
        True
      id: python.pyramid.audit.csrf-origin-check-disabled.pyramid-csrf-origin-check-disabled
      languages:
        - python
      message: Origin check for the CSRF token is disabled for this view. This might represent a
 security risk if the CSRF storage policy is not known to be secure.
      metadata:
        asvs:
            control_id: 4.2.2 CSRF
            control_url: https://github.com/OWASP/ASVS/blob/master/4.0/en/0x12-V4-Access-Control
.md#v42-operation-level-access-control
            section: V4 Access Control
            version: "4"
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-352: Cross-Site Request Forgery (CSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-inside: |
            from pyramid.view import view_config
            ...
            @view_config(..., check_origin=$CHECK_ORIGIN, ...)
            def $VIEW(...):
              ...
        - pattern: $CHECK_ORIGIN
        - metavariable-comparison:
            comparison: $CHECK_ORIGIN == False
            metavariable: $CHECK_ORIGIN
      severity: WARNING
    - fix-regex:
        regex: (.*)\)
        replacement: \1, httponly=True)
      id: python.pyramid.audit.set-cookie-httponly-unsafe-default.pyramid-set-cookie-httponly-un
safe-default
      languages:
        - python
      message: Found a Pyramid cookie using an unsafe default for the httponly option. Pyramid c
ookies should be handled securely by setting httponly=True in response.set_cookie(...). If this
parameter is not properly set, your cookies are not properly protected and are at risk of being
stolen by an attacker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1004: Sensitive Cookie Without ''HttpOnly'' Flag'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        subcategory:
            - vuln
        technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pyramid
      patterns:
        - pattern-either:
            - pattern-inside: |
                @pyramid.view.view_config(...)
                def $VIEW($REQUEST):
                    ...
                    $RESPONSE = $REQUEST.response
                    ...
            - pattern-inside: |
                def $VIEW(...):
                    ...
                    $RESPONSE = pyramid.httpexceptions.HTTPFound(...)
                    ...
        - pattern-not: $RESPONSE.set_cookie(..., httponly=$HTTPONLY, ...)
        - pattern-not: $RESPONSE.set_cookie(..., **$PARAMS)
        - pattern: $RESPONSE.set_cookie(...)
      severity: WARNING
    - fix: |
        True
      id: python.pyramid.audit.set-cookie-httponly-unsafe-value.pyramid-set-cookie-httponly-unsa
fe-value
      languages:
        - python
      message: Found a Pyramid cookie without the httponly option correctly set. Pyramid cookies
 should be handled securely by setting httponly=True in response.set_cookie(...). If this parame
ter is not properly set, your cookies are not properly protected and are at risk of being stolen
 by an attacker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1004: Sensitive Cookie Without ''HttpOnly'' Flag'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/www-community/controls/SecureCookieAttribute
            - https://owasp.org/www-community/HttpOnly
            - https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
#httponly-attribute
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-either:
            - pattern-inside: |
                @pyramid.view.view_config(...)
                def $VIEW($REQUEST):
                    ...
                    $RESPONSE = $REQUEST.response
                    ...
            - pattern-inside: |
                def $VIEW(...):
                    ...
                    $RESPONSE = pyramid.httpexceptions.HTTPFound(...)
                    ...
        - pattern-not: $RESPONSE.set_cookie(..., **$PARAMS)
        - pattern: $RESPONSE.set_cookie(..., httponly=$HTTPONLY, ...)
        - pattern: $HTTPONLY
        - metavariable-pattern:
            metavariable: $HTTPONLY
            pattern: |
                False
      severity: WARNING
    - fix-regex:
        regex: (.*)\)
        replacement: \1, samesite='Lax')
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      id: python.pyramid.audit.set-cookie-samesite-unsafe-default.pyramid-set-cookie-samesite-un
safe-default
      languages:
        - python
      message: Found a Pyramid cookie using an unsafe value for the samesite option. Pyramid coo
kies should be handled securely by setting samesite='Lax' in response.set_cookie(...). If this p
arameter is not properly set, your cookies are not properly protected and are at risk of being s
tolen by an attacker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1275: Sensitive Cookie with Improper SameSite Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-either:
            - pattern-inside: |
                @pyramid.view.view_config(...)
                def $VIEW($REQUEST):
                    ...
                    $RESPONSE = $REQUEST.response
                    ...
            - pattern-inside: |
                def $VIEW(...):
                    ...
                    $RESPONSE = pyramid.httpexceptions.HTTPFound(...)
                    ...
        - pattern-not: $RESPONSE.set_cookie(..., samesite=$SAMESITE, ...)
        - pattern-not: $RESPONSE.set_cookie(..., **$PARAMS)
        - pattern: $RESPONSE.set_cookie(...)
      severity: WARNING
    - fix: |
        'Lax'
      id: python.pyramid.audit.set-cookie-samesite-unsafe-value.pyramid-set-cookie-samesite-unsa
fe-value
      languages:
        - python
      message: Found a Pyramid cookie without the samesite option correctly set. Pyramid cookies
 should be handled securely by setting samesite='Lax' in response.set_cookie(...). If this param
eter is not properly set, your cookies are not properly protected and are at risk of being stole
n by an attacker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1275: Sensitive Cookie with Improper SameSite Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-either:
            - pattern-inside: |
                @pyramid.view.view_config(...)
                def $VIEW($REQUEST):
                    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    $RESPONSE = $REQUEST.response
                    ...
            - pattern-inside: |
                def $VIEW(...):
                    ...
                    $RESPONSE = pyramid.httpexceptions.HTTPFound(...)
                    ...
        - pattern-not: $RESPONSE.set_cookie(..., **$PARAMS)
        - pattern: $RESPONSE.set_cookie(..., samesite=$SAMESITE, ...)
        - pattern: $SAMESITE
        - metavariable-regex:
            metavariable: $SAMESITE
            regex: (?!'Lax')
      severity: WARNING
    - fix-regex:
        regex: (.*)\)
        replacement: \1, secure=True)
      id: python.pyramid.audit.set-cookie-secure-unsafe-default.pyramid-set-cookie-secure-unsafe
-default
      languages:
        - python
      message: Found a Pyramid cookie using an unsafe default for the secure option. Pyramid coo
kies should be handled securely by setting secure=True in response.set_cookie(...). If this para
meter is not properly set, your cookies are not properly protected and are at risk of being stol
en by an attacker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-614: Sensitive Cookie in HTTPS Session Without ''Secure'' Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-either:
            - pattern-inside: |
                @pyramid.view.view_config(...)
                def $VIEW($REQUEST):
                    ...
                    $RESPONSE = $REQUEST.response
                    ...
            - pattern-inside: |
                def $VIEW(...):
                    ...
                    $RESPONSE = pyramid.httpexceptions.HTTPFound(...)
                    ...
        - pattern-not: $RESPONSE.set_cookie(..., secure=$SECURE, ...)
        - pattern-not: $RESPONSE.set_cookie(..., **$PARAMS)
        - pattern: $RESPONSE.set_cookie(...)
      severity: WARNING
    - fix: |
        True
      id: python.pyramid.audit.set-cookie-secure-unsafe-value.pyramid-set-cookie-secure-unsafe-v
alue
      languages:
        - python
      message: Found a Pyramid cookie without the secure option correctly set. Pyramid cookies s
hould be handled securely by setting secure=True in response.set_cookie(...). If this parameter
is not properly set, your cookies are not properly protected and are at risk of being stolen by
an attacker.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-614: Sensitive Cookie in HTTPS Session Without ''Secure'' Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-either:
            - pattern-inside: |
                @pyramid.view.view_config(...)
                def $VIEW($REQUEST):
                    ...
                    $RESPONSE = $REQUEST.response
                    ...
            - pattern-inside: |
                def $VIEW(...):
                    ...
                    $RESPONSE = pyramid.httpexceptions.HTTPFound(...)
                    ...
        - pattern-not: $RESPONSE.set_cookie(..., **$PARAMS)
        - pattern: $RESPONSE.set_cookie(..., secure=$SECURE, ...)
        - pattern: $SECURE
        - metavariable-pattern:
            metavariable: $SECURE
            pattern: |
                False
      severity: WARNING
    - fix: |
        True
      id: python.pyramid.security.csrf-check-disabled-globally.pyramid-csrf-check-disabled-globa
lly
      languages:
        - python
      message: Automatic check of cross-site request forgery tokens has been explicitly disabled
 globally, which might leave views unprotected. Use 'pyramid.config.Configurator.set_default_csr
f_options(require_csrf=True)' to turn the automatic check for all unsafe methods (per RFC2616).
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-352: Cross-Site Request Forgery (CSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - pyramid
      patterns:
        - pattern-inside: |
            $CONFIG.set_default_csrf_options(..., require_csrf=$REQUIRE_CSRF, ...)
        - pattern: $REQUIRE_CSRF
        - metavariable-comparison:
            comparison: $REQUIRE_CSRF == False
            metavariable: $REQUIRE_CSRF
      severity: ERROR
    - id: python.pyramid.security.direct-use-of-response.pyramid-direct-use-of-response
      languages:
        - python
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Detected data rendered directly to the end user via 'Response'. This bypasses Pyr
amid's built-in cross-site scripting (XSS) defenses and could result in an XSS vulnerability. Us
e Pyramid's template engines to safely render HTML.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - pyramid
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    pyramid.request.Response.text($SINK)
                - pattern: |
                    pyramid.request.Response($SINK)
                - pattern: |
                    $REQ.response.body = $SINK
                - pattern: |
                    $REQ.response.text = $SINK
                - pattern: |
                    $REQ.response.ubody = $SINK
                - pattern: |
                    $REQ.response.unicode_body = $SINK
            - pattern: $SINK
      pattern-sources:
        - patterns:
            - pattern-inside: |
                @pyramid.view.view_config( ... )
                def $VIEW($REQ):
                  ...
            - pattern: $REQ.$ANYTHING
            - pattern-not: $REQ.dbsession
      severity: ERROR
    - fix-regex:
        regex: format
        replacement: bindparams
      id: python.pyramid.security.sqlalchemy-sql-injection.pyramid-sqlalchemy-sql-injection
      languages:
        - python
      message: Distinct, Having, Group_by, Order_by, and Filter in SQLAlchemy can cause sql inje
ctions if the developer inputs raw SQL into the before-mentioned clauses. This pattern captures
relevant cases in which the developer inputs raw SQL into the distinct, having, group_by, order_
by or filter clauses and injects user-input into the raw SQL with any function besides "bindpara
ms". Use bindParams to securely bind user-input to SQL statements.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A03:2021 - Injection
        references:
            - https://docs.sqlalchemy.org/en/14/tutorial/data_select.html#tutorial-selecting-dat
a
        subcategory:
            - vuln
        technology:
            - pyramid
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-inside: |
                $QUERY = $REQ.dbsession.query(...)
                ...
            - pattern-either:
                - pattern: |
                    $QUERY.$SQLFUNC("...".$FORMATFUNC(..., $SINK, ...))
                - pattern: |
                    $QUERY.join(...).$SQLFUNC("...".$FORMATFUNC(..., $SINK, ...))
            - pattern: $SINK
            - metavariable-regex:
                metavariable: $SQLFUNC
                regex: (group_by|order_by|distinct|having|filter)
            - metavariable-regex:
                metavariable: $FORMATFUNC
                regex: (?!bindparams)
      pattern-sources:
        - patterns:
            - pattern-inside: |
                from pyramid.view import view_config
                ...
                @view_config( ... )
                def $VIEW($REQ):
                  ...
            - pattern: $REQ.$ANYTHING
            - pattern-not: $REQ.dbsession
      severity: ERROR
    - id: python.sqlalchemy.security.audit.avoid-sqlalchemy-text.avoid-sqlalchemy-text
      languages:
        - python
      message: sqlalchemy.text passes the constructed SQL statement to the database mostly uncha
nged. This means that the usual SQL injection protections are not applied and this function is v
ulnerable to SQL injection if user input can reach here. Use normal SQLAlchemy operators (such a
s or_, and_, etc.) to construct SQL.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.sqlalchemy.org/en/14/core/tutorial.html#using-textual-sql
        subcategory:
            - audit
        technology:
            - sqlalchemy
      mode: taint
      pattern-sinks:
        - pattern: |
            sqlalchemy.text(...)
      pattern-sources:
        - patterns:
            - pattern: |
                $X + $Y
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - metavariable-type:
                metavariable: $X
                type: string
        - patterns:
            - pattern: |
                $X + $Y
            - metavariable-type:
                metavariable: $Y
                type: string
        - patterns:
            - pattern: |
                f"..."
        - patterns:
            - pattern: |
                $X.format(...)
            - metavariable-type:
                metavariable: $X
                type: string
        - patterns:
            - pattern: |
                $X % $Y
            - metavariable-type:
                metavariable: $X
                type: string
      severity: ERROR
    - fix-regex:
        regex: format
        replacement: bindparams
      id: python.sqlalchemy.security.sqlalchemy-sql-injection.sqlalchemy-sql-injection
      languages:
        - python
      message: Distinct, Having, Group_by, Order_by, and Filter in SQLAlchemy can cause sql inje
ctions if the developer inputs raw SQL into the before-mentioned clauses. This pattern captures
relevant cases in which the developer inputs raw SQL into the distinct, having, group_by, order_
by or filter clauses and injects user-input into the raw SQL with any function besides "bindpara
ms". Use bindParams to securely bind user-input to SQL statements.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: LOW
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
            - sqlalchemy
      patterns:
        - pattern-either:
            - pattern: |
                def $FUNC(...,$VAR,...):
                  ...
                  $SESSION.query(...).$SQLFUNC("...".$FORMATFUNC(...,$VAR,...))
            - pattern: |
                def $FUNC(...,$VAR,...):
                  ...
                  $SESSION.query.join(...).$SQLFUNC("...".$FORMATFUNC(...,$VAR,...))
            - pattern: |
                def $FUNC(...,$VAR,...):
                  ...
                  $SESSION.query.$SQLFUNC("...".$FORMATFUNC(...,$VAR,...))
            - pattern: |
                def $FUNC(...,$VAR,...):
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                query.$SQLFUNC("...".$FORMATFUNC(...,$VAR,...))
        - metavariable-regex:
            metavariable: $SQLFUNC
            regex: (group_by|order_by|distinct|having|filter)
        - metavariable-regex:
            metavariable: $FORMATFUNC
            regex: (?!bindparams)
      severity: WARNING
    - id: python.twilio.security.twiml-injection.twiml-injection
      languages:
        - python
      message: Using non-constant TwiML (Twilio Markup Language) argument when creating a Twilio
 conversation could allow the injection of additional TwiML commands
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-91: XML Injection'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2021 - Injection
        references:
            - https://codeberg.org/fennix/funjection
        subcategory: vuln
        technology:
            - python
            - twilio
            - twiml
      mode: taint
      pattern-sanitizers:
        - pattern: xml.sax.saxutils.escape(...)
        - pattern: html.escape(...)
      pattern-sinks:
        - patterns:
            - pattern: |
                $CLIENT.calls.create(..., twiml=$SINK, ...)
            - focus-metavariable: $SINK
      pattern-sources:
        - pattern: |
            f"..."
        - pattern: |
            "..." % ...
        - pattern: |
            "...".format(...)
        - patterns:
            - pattern: $ARG
            - pattern-inside: |
                def $F(..., $ARG, ...):
                    ...
      severity: WARNING
    - id: ruby.aws-lambda.security.activerecord-sqli.activerecord-sqli
      languages:
        - ruby
      message: 'Detected SQL statement that is tainted by `event` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use parameterized
 statements like so: `Example.find_by_sql ["SELECT title FROM posts WHERE author = ? AND created
 > ?", author_id, start_date]`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://guides.rubyonrails.org/active_record_querying.html#finding-by-sql
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - active-record
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: $QUERY
            - pattern-either:
                - pattern: ActiveRecord::Base.connection.execute($QUERY,...)
                - pattern: $MODEL.find_by_sql($QUERY,...)
                - pattern: $MODEL.select_all($QUERY,...)
            - pattern-inside: |
                require 'active_record'
                ...
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context)
                  ...
                end
      severity: WARNING
    - id: ruby.aws-lambda.security.mysql2-sqli.mysql2-sqli
      languages:
        - ruby
      message: 'Detected SQL statement that is tainted by `event` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use sanitize stat
ements like so: `escaped = client.escape(user_input)`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://github.com/brianmario/mysql2
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - mysql2
      mode: taint
      pattern-sanitizers:
        - pattern: $CLIENT.escape(...)
      pattern-sinks:
        - patterns:
            - pattern: $QUERY
            - pattern-either:
                - pattern: $CLIENT.query($QUERY,...)
                - pattern: $CLIENT.prepare($QUERY,...)
            - pattern-inside: |
                require 'mysql2'
                ...
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              def $HANDLER(event, context)
                ...
              end
    severity: WARNING
  - id: ruby.aws-lambda.security.pg-sqli.pg-sqli
    languages:
      - ruby
    message: 'Detected SQL statement that is tainted by `event` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use parameterized
 statements like so: `conn.exec_params(''SELECT $1 AS a, $2 AS b, $3 AS c'', [1, 2, nil])`'
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
          - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
      cwe2021-top25: true
      cwe2022-top25: true
      impact: HIGH
      likelihood: MEDIUM
      owasp:
          - A01:2017 - Injection
          - A03:2021 - Injection
      references:
          - https://www.rubydoc.info/gems/pg/PG/Connection
      subcategory:
          - vuln
      technology:
          - aws-lambda
          - postgres
          - pg
    mode: taint
    pattern-sinks:
      - patterns:
          - pattern: $QUERY
          - pattern-either:
              - pattern: $CONN.exec($QUERY,...)
              - pattern: $CONN.exec_params($QUERY,...)
              - pattern: $CONN.exec_prepared($QUERY,...)
              - pattern: $CONN.async_exec($QUERY,...)
              - pattern: $CONN.async_exec_params($QUERY,...)
              - pattern: $CONN.async_exec_prepared($QUERY,...)
          - pattern-inside: |
              require 'pg'
              ...
    pattern-sources:
      - patterns:
          - pattern: event
          - pattern-inside: |
              def $HANDLER(event, context)
                ...
              end
    severity: WARNING
  - id: ruby.aws-lambda.security.sequel-sqli.sequel-sqli
    languages:
      - ruby
    message: 'Detected SQL statement that is tainted by `event` object. This could lead to SQL
 injection if the variable is user-controlled and not properly sanitized. In order to prevent SQ
L injection, use parameterized queries or prepared statements instead. You can use parameterized
 statements like so: `DB[''select * from items where name = ?'', name]`'
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
          - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
      cwe2021-top25: true
      cwe2022-top25: true
      impact: HIGH
      likelihood: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://github.com/jeremyevans/sequel#label-Arbitrary+SQL+queries
        subcategory:
            - vuln
        technology:
            - aws-lambda
            - sequel
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: $QUERY
            - pattern-either:
                - pattern: DB[$QUERY,...]
                - pattern: DB.run($QUERY,...)
            - pattern-inside: |
                require 'sequel'
                ...
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context)
                  ...
                end
      severity: WARNING
    - id: ruby.aws-lambda.security.tainted-deserialization.tainted-deserialization
      languages:
        - ruby
      message: Deserialization of a string tainted by `event` object found. Objects in Ruby can
be serialized into strings, then later loaded from strings. However, uses of `load` can cause re
mote code execution. Loading user input with MARSHAL, YAML or CSV can potentially be dangerous.
If you need to deserialize untrusted data, you should use JSON as it is only capable of returnin
g 'primitive' types such as strings, arrays, hashes, numbers and nil.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://ruby-doc.org/core-3.1.2/doc/security_rdoc.html
            - https://groups.google.com/g/rubyonrails-security/c/61bkgvnSGTQ/m/nehwjA8tQ8EJ
            - https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks/check_dese
rialize.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: $SINK
            - pattern-either:
                - pattern-inside: |
                    YAML.load($SINK,...)
                - pattern-inside: |
                    CSV.load($SINK,...)
                - pattern-inside: |
                    Marshal.load($SINK,...)
                - pattern-inside: |
                    Marshal.restore($SINK,...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context)
                  ...
                end
      severity: WARNING
    - id: ruby.aws-lambda.security.tainted-sql-string.tainted-sql-string
      languages:
        - ruby
      message: Detected user input used to manually construct a SQL string. This is usually bad
practice because manual construction could accidentally result in a SQL injection. An attacker c
ould use a SQL injection to steal or modify contents of the database. Instead, use a parameteriz
ed query which is available by default in most database engines. Alternatively, consider using a
n object-relational mapper (ORM) such as Sequelize which will protect your queries.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://rorsecurity.info/portfolio/ruby-on-rails-sql-injection-cheat-sheet
        subcategory:
            - vuln
        technology:
            - aws-lambda
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: |
                        "...#{...}..."
                    - pattern-regex: (?i)(select|delete|insert|create|update|alter|drop)\b|\w+\s
*!?[<>=].*
                - patterns:
                    - pattern-either:
                        - pattern: Kernel::sprintf("$SQLSTR", ...)
                        - pattern: |
                            "$SQLSTR" + $EXPR
                        - pattern: |
                            "$SQLSTR" % $EXPR
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: (?i)(select|delete|insert|create|update|alter|drop)\b|\w+\s*!?[<>
=].*
            - pattern-not-inside: |
                puts(...)
      pattern-sources:
        - patterns:
            - pattern: event
            - pattern-inside: |
                def $HANDLER(event, context)
                  ...
                end
      severity: ERROR
    - id: ruby.lang.security.bad-deserialization.bad-deserialization
      languages:
        - ruby
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Checks for unsafe deserialization. Objects in Ruby can be serialized into strings
, then later loaded from strings. However, uses of load and object_load can cause remote code ex
ecution. Loading user input with MARSHAL or CSV can potentially be dangerous. Use JSON in a secu
re fashion instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-502: Deserialization of Untrusted Data'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A08:2017 - Insecure Deserialization
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://groups.google.com/g/rubyonrails-security/c/61bkgvnSGTQ/m/nehwjA8tQ8EJ
            - https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks/check_dese
rialize.rb
        subcategory:
            - vuln
        technology:
            - ruby
      mode: taint
      pattern-sinks:
        - pattern-either:
            - pattern: |
                CSV.load(...)
            - pattern: |
                Marshal.load(...)
            - pattern: |
                Marshal.restore(...)
            - pattern: |
                Oj.object_load(...)
            - pattern: |
                Oj.load($X)
      pattern-sources:
        - pattern-either:
            - pattern: params
            - pattern: cookies
      severity: ERROR
    - id: ruby.lang.security.dangerous-exec.dangerous-exec
      languages:
        - ruby
      message: Detected non-static command inside $EXEC. Audit the input to '$EXEC'. If unverifi
ed user data can reach this call site, this is a code injection vulnerability. A malicious actor
 can inject a malicious script to execute arbitrary code.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://guides.rubyonrails.org/security.html#command-line-injection
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_execute.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              $EXEC(...)
            - pattern-not: |
                $EXEC("...","...","...",...)
            - pattern-not: |
                $EXEC(["...","...","...",...],...)
            - pattern-not: |
                $EXEC({...},"...","...","...",...)
            - pattern-not: |
                $EXEC({...},["...","...","...",...],...)
            - metavariable-regex:
                metavariable: $EXEC
                regex: ^(system|exec|spawn|Process.exec|Process.spawn|Open3.capture2|Open3.captu
re2e|Open3.capture3|Open3.popen2|Open3.popen2e|Open3.popen3|IO.popen|Gem::Util.popen|PTY.spawn)$
      pattern-sources:
        - patterns:
            - pattern: |
                def $F(...,$ARG,...)
                    ...
                end
            - focus-metavariable: $ARG
        - pattern: params
        - pattern: cookies
      severity: WARNING
    - id: ruby.lang.security.divide-by-zero.divide-by-zero
      languages:
        - ruby
      message: Detected a possible ZeroDivisionError.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-369: Divide By Zero'
        impact: MEDIUM
        likelihood: MEDIUM
        references:
            - https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks/check_divi
de_by_zero.rb
        subcategory:
            - vuln
        technology:
            - ruby
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-inside: $NUMER / 0
            - pattern: $NUMER
      pattern-sources:
        - patterns:
            - pattern: $VAR
            - metavariable-regex:
                metavariable: $VAR
                regex: ^\d*(?!\.)$
      severity: WARNING
    - fix-regex:
        regex: =\s*false
        replacement: = true
      id: ruby.lang.security.force-ssl-false.force-ssl-false
      languages:
        - ruby
      message: Checks for configuration setting of force_ssl to false. Force_ssl forces usage of
 HTTPS, which could lead to network interception of unencrypted application traffic. To fix, set
 config.force_ssl = true.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-311: Missing Encryption of Sensitive Data'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A04:2021 - Insecure Design
        references:
            - https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks/check_forc
e_ssl.rb
        subcategory:
            - vuln
        technology:
            - ruby
      pattern: config.force_ssl = false
      severity: WARNING
    - id: ruby.lang.security.hardcoded-http-auth-in-controller.hardcoded-http-auth-in-controller
      languages:
        - ruby
      message: Detected hardcoded password used in basic authentication in a controller class. I
ncluding this password in version control could expose this credential. Consider refactoring to
use environment variables or configuration files.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/docs/warning_types/
basic_auth/index.markdown
        subcategory:
            - audit
        technology:
            - ruby
            - secrets
      patterns:
        - pattern-inside: |
            class $CONTROLLER < ApplicationController
              ...
              http_basic_authenticate_with ..., :password => "$SECRET", ...
            end
        - focus-metavariable: $SECRET
      severity: WARNING
    - id: ruby.lang.security.hardcoded-secret-rsa-passphrase.hardcoded-secret-rsa-passphrase
      languages:
        - ruby
      message: Found the use of an hardcoded passphrase for RSA. The passphrase can be easily di
scovered, and therefore should not be stored in source-code. It is recommended to remove the pas
sphrase from source-code, and use system environment variables or a restricted configuration fil
e.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cwe.mitre.org/data/definitions/522.html
        subcategory:
            - vuln
        technology:
            - ruby
            - secrets
      patterns:
        - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
- pattern: OpenSSL::PKey::RSA.new(..., '...')
- pattern: OpenSSL::PKey::RSA.new(...).to_pem(..., '...')
- pattern: OpenSSL::PKey::RSA.new(...).export(..., '...')
- patterns:
    - pattern-inside: |
        $OPENSSL = OpenSSL::PKey::RSA.new(...)
        ...
    - pattern-either:
        - pattern: |
            $OPENSSL.export(...,'...')
        - pattern: |
            $OPENSSL.to_pem(...,'...')
- patterns:
    - pattern-either:
        - patterns:
            - pattern-inside: |
                $ASSIGN = '...'
                ...
            - pattern: OpenSSL::PKey::RSA.new(..., $ASSIGN)
        - patterns:
            - pattern-inside: |
                def $METHOD1(...)
                ...
                $ASSIGN = '...'
                ...
                end
                ...
                def $METHOD2(...)
                ...
                end
            - pattern: OpenSSL::PKey::RSA.new(..., $ASSIGN)
        - patterns:
            - pattern-inside: |
                $ASSIGN = '...'
                ...
                def $METHOD(...)
                  $OPENSSL = OpenSSL::PKey::RSA.new(...)
                ...
                end
                ...
            - pattern-either:
                - pattern: $OPENSSL.export(...,$ASSIGN)
                - pattern: $OPENSSL.to_pem(...,$ASSIGN)
        - patterns:
            - pattern-inside: |
                def $METHOD1(...)
                ...
                $OPENSSL = OpenSSL::PKey::RSA.new(...)
                ...
                $ASSIGN = '...'
                ...
                end
                ...
            - pattern-either:
                - pattern: $OPENSSL.export(...,$ASSIGN)
                - pattern: $OPENSSL.to_pem(...,$ASSIGN)
        - patterns:
            - pattern-inside: |
                def $METHOD1(...)
                ...
                $ASSIGN = '...'
                ...
                end
                ...
                def $METHOD2(...)
                ...
                $OPENSSL = OpenSSL::PKey::RSA.new(...)
                ...
                end
                ...
            - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                          - pattern: $OPENSSL.export(...,$ASSIGN)
                          - pattern: $OPENSSL.to_pem(...,$ASSIGN)
      severity: WARNING
    - id: ruby.lang.security.insufficient-rsa-key-size.insufficient-rsa-key-size
      languages:
        - ruby
      message: The RSA key size $SIZE is insufficent by NIST standards. It is recommended to use
 a key length of 2048 or higher.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57Pt3r1.pdf
        subcategory:
            - vuln
        technology:
            - ruby
      patterns:
        - pattern-either:
            - pattern: OpenSSL::PKey::RSA.generate($SIZE,...)
            - pattern: OpenSSL::PKey::RSA.new($SIZE, ...)
            - patterns:
                - pattern-either:
                    - patterns:
                        - pattern-inside: |
                            $ASSIGN = $SIZE
                            ...
                        - pattern-either:
                            - pattern: OpenSSL::PKey::RSA.new($ASSIGN, ...)
                            - pattern: OpenSSL::PKey::RSA.generate($ASSIGN, ...)
                    - patterns:
                        - pattern-inside: |
                            def $METHOD1(...)
                            ...
                            $ASSIGN = $SIZE
                            ...
                            end
                            ...
                        - pattern-either:
                            - pattern: OpenSSL::PKey::RSA.new($ASSIGN, ...)
                            - pattern: OpenSSL::PKey::RSA.generate($ASSIGN, ...)
        - metavariable-comparison:
            comparison: $SIZE < 2048
            metavariable: $SIZE
      severity: WARNING
    - id: ruby.lang.security.md5-used-as-password.md5-used-as-password
      languages:
        - ruby
      message: It looks like MD5 is used as a password hash. MD5 is not considered a secure pass
word hash because it can be cracked by an attacker in a short amount of time. Instead, use a sui
table password hashing function such as bcrypt. You can use the `bcrypt` gem.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-327: Use of a Broken or Risky Cryptographic Algorithm'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://tools.ietf.org/id/draft-lvelvindron-tls-md5-sha1-deprecate-01.html
            - https://security.stackexchange.com/questions/211/how-to-securely-hash-passwords
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://github.com/returntocorp/semgrep-rules/issues/1609
          subcategory:
            - vuln
          technology:
            - md5
        mode: taint
        pattern-sinks:
          - patterns:
            - pattern: $FUNCTION(...);
            - metavariable-regex:
                metavariable: $FUNCTION
                regex: (?i)(.*password.*)
        pattern-sources:
          - pattern: Digest::MD5
        severity: WARNING
      - id: ruby.lang.security.no-eval.ruby-eval
        languages:
          - ruby
        message: Use of eval with user-controllable input detected. This can lead  to attackers ru
nning arbitrary code. Ensure external data does not  reach here, otherwise this is a security vu
lnerability. Consider  other ways to do this without eval.
        metadata:
          category: security
          confidence: MEDIUM
          cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
          cwe2021-top25: true
          cwe2022-top25: true
          impact: MEDIUM
          likelihood: HIGH
          owasp:
            - A03:2021 - Injection
          references:
            - https://owasp.org/Top10/A03_2021-Injection
          source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_evaluation.rb
          subcategory:
            - vuln
          technology:
            - ruby
            - rails
        mode: taint
        pattern-sinks:
          - patterns:
            - pattern-either:
                - pattern: $X.eval
                - pattern: $X.class_eval
                - pattern: $X.instance_eval
                - pattern: $X.module_eval
                - pattern: $X.eval(...)
                - pattern: $X.class_eval(...)
                - pattern: $X.instance_eval(...)
                - pattern: $X.module_eval(...)
                - pattern: eval(...)
                - pattern: class_eval(...)
                - pattern: module_eval(...)
                - pattern: instance_eval(...)
            - pattern-not: $M("...",...)
        pattern-sources:
          - pattern-either:
            - pattern: params
            - pattern: cookies
            - patterns:
                - pattern: |
                    RubyVM::InstructionSequence.compile(...)
                - pattern-not: |
                    RubyVM::InstructionSequence.compile("...")
        severity: WARNING
      - fix-regex:
          regex: VERIFY_NONE
          replacement: VERIFY_PEER
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      id: ruby.lang.security.ssl-mode-no-verify.ssl-mode-no-verify
      languages:
        - ruby
      message: Detected SSL that will accept an unverified connection. This makes the connection
s susceptible to man-in-the-middle attacks. Use 'OpenSSL::SSL::VERIFY_PEER' instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-295: Improper Certificate Validation'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures
        subcategory:
            - vuln
        technology:
            - ruby
      pattern: OpenSSL::SSL::VERIFY_NONE
      severity: WARNING
    - id: ruby.lang.security.weak-hashes-md5.weak-hashes-md5
      languages:
        - ruby
      message: Should not use md5 to generate hashes. md5 is proven to be vulnerable through the
 use of brute-force attacks. Could also result in collisions, leading to potential collision att
acks. Use SHA256 or other hashing functions instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-328: Use of Weak Hash'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.ibm.com/support/pages/security-bulletin-vulnerability-md5-signature-an
d-hash-algorithm-affects-sterling-integrator-and-sterling-file-gateway-cve-2015-7575
        subcategory:
            - vuln
        technology:
            - ruby
      pattern-either:
        - pattern: Digest::MD5.base64digest $X
        - pattern: Digest::MD5.hexdigest $X
        - pattern: Digest::MD5.digest $X
        - pattern: Digest::MD5.new
        - pattern: OpenSSL::Digest::MD5.base64digest $X
        - pattern: OpenSSL::Digest::MD5.hexdigest $X
        - pattern: OpenSSL::Digest::MD5.digest $X
        - pattern: OpenSSL::Digest::MD5.new
      severity: WARNING
    - id: ruby.lang.security.weak-hashes-sha1.weak-hashes-sha1
      languages:
        - ruby
      message: Should not use SHA1 to generate hashes. There is a proven SHA1 hash collision by
Google, which could lead to vulnerabilities. Use SHA256, SHA3 or other hashing functions instead
.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-328: Use of Weak Hash'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A02:2021 - Cryptographic Failures
        references:
            - https://security.googleblog.com/2017/02/announcing-first-sha1-collision.html
            - https://shattered.io/
        subcategory:
            - vuln
        technology:
            - ruby
      pattern-either:
        - pattern: Digest::SHA1.$FUNC
        - pattern: OpenSSL::Digest::SHA1.$FUNC
        - pattern: OpenSSL::HMAC.$FUNC("sha1",...)
      severity: WARNING
    - id: ruby.rails.security.audit.avoid-session-manipulation.avoid-session-manipulation
      languages:
        - ruby
      message: This gets data from session using user inputs. A malicious user may be able to re
trieve information from your session that you didn't intend them to. Do not use user input as a
session key.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-276: Incorrect Default Permissions'
        cwe2021-top25: true
        cwe2022-top25: true
        help: |
            ## Remediation
            Session manipulation can occur when an application allows user-input in session keys
. Since sessions are typically considered a source of truth (e.g. to check the logged-in user or
 to match CSRF tokens), allowing an attacker to manipulate the session may lead to unintended be
havior.

            ## References
            [Session Manipulation](https://brakemanscanner.org/docs/warning_types/session_manipu
lation/)
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://brakemanscanner.org/docs/warning_types/session_manipulation/
        shortDescription: Allowing an attacker to manipulate the session may lead to unintended
behavior.
        subcategory:
            - vuln
        tags:
            - security
        technology:
            - rails
      mode: taint
      pattern-sinks:
        - pattern: session[...]
      pattern-sources:
        - pattern: params
        - pattern: cookies
        - pattern: request.env
      severity: WARNING
    - id: ruby.rails.security.audit.avoid-tainted-file-access.avoid-tainted-file-access
      languages:
        - ruby
      message: Using user input when accessing files is potentially dangerous. A malicious actor
 could use this to modify or access files they have no right to.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://github.com/presidentbeef/brakeman/blob/main/docs/warning_types/file_access
/index.markdown
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: Dir.$X(...)
                - pattern: File.$X(...)
                - pattern: IO.$X(...)
                - pattern: Kernel.$X(...)
                - pattern: PStore.$X(...)
                - pattern: Pathname.$X(...)
            - metavariable-pattern:
                metavariable: $X
                patterns:
                    - pattern-either:
                        - pattern: chdir
                        - pattern: chroot
                        - pattern: delete
                        - pattern: entries
                        - pattern: foreach
                        - pattern: glob
                        - pattern: install
                        - pattern: lchmod
                        - pattern: lchown
                        - pattern: link
                        - pattern: load
                        - pattern: load_file
                        - pattern: makedirs
                        - pattern: move
                        - pattern: new
                        - pattern: open
                        - pattern: read
                        - pattern: readlines
                        - pattern: rename
                        - pattern: rmdir
                        - pattern: safe_unlink
                        - pattern: symlink
                        - pattern: syscopy
                        - pattern: sysopen
                        - pattern: truncate
                        - pattern: unlink
      pattern-sources:
        - pattern: params
        - pattern: cookies
        - pattern: request.env
      severity: WARNING
    - id: ruby.rails.security.audit.avoid-tainted-ftp-call.avoid-tainted-ftp-call
      languages:
        - ruby
      message: Using user input when accessing files is potentially dangerous. A malicious actor
 could use this to modify or access files they have no right to.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: HIGH
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://github.com/presidentbeef/brakeman/blob/main/docs/warning_types/file_access
/index.markdown
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sinks:
        - pattern-either:
            - pattern: Net::FTP.$X(...)
            - patterns:
                - pattern-inside: |
                    $FTP = Net::FTP.$OPEN(...)
                    ...
                    $FTP.$METHOD(...)
                - pattern: $FTP.$METHOD(...)
      pattern-sources:
        - pattern: params
        - pattern: cookies
        - pattern: request.env
      severity: WARNING
    - id: ruby.rails.security.audit.avoid-tainted-http-request.avoid-tainted-http-request
      languages:
        - ruby
      message: Using user input when accessing files is potentially dangerous. A malicious actor
 could use this to modify or access files they have no right to.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://github.com/presidentbeef/brakeman/blob/main/docs/warning_types/file_access
/index.markdown
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sinks:
        - pattern-either:
            - patterns:
                - pattern: Net::HTTP::$METHOD.new(...)
                - metavariable-pattern:
                    metavariable: $METHOD
                    patterns:
                        - pattern-either:
                            - pattern: Copy
                            - pattern: Delete
                            - pattern: Get
                            - pattern: Head
                            - pattern: Lock
                            - pattern: Mkcol
                            - pattern: Move
                            - pattern: Options
                            - pattern: Patch
                            - pattern: Post
                            - pattern: Propfind
                            - pattern: Proppatch
                            - pattern: Put
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                            - pattern: Trace
                            - pattern: Unlock
            - patterns:
                - pattern: Net::HTTP.$X(...)
                - metavariable-pattern:
                    metavariable: $X
                    patterns:
                        - pattern-either:
                            - pattern: get
                            - pattern: get2
                            - pattern: head
                            - pattern: head2
                            - pattern: options
                            - pattern: patch
                            - pattern: post
                            - pattern: post2
                            - pattern: post_form
                            - pattern: put
                            - pattern: request
                            - pattern: request_get
                            - pattern: request_head
                            - pattern: request_post
                            - pattern: send_request
                            - pattern: trace
                            - pattern: get_print
                            - pattern: get_response
                            - pattern: start
      pattern-sources:
        - pattern: params
        - pattern: cookies
        - pattern: request.env
      severity: WARNING
    - id: ruby.rails.security.audit.avoid-tainted-shell-call.avoid-tainted-shell-call
      languages:
        - ruby
      message: Using user input when accessing files is potentially dangerous. A malicious actor
 could use this to modify or access files they have no right to.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://github.com/presidentbeef/brakeman/blob/main/docs/warning_types/file_access
/index.markdown
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: Kernel.$X(...)
                - patterns:
                    - pattern-either:
                        - pattern: Shell.$X(...)
                        - patterns:
                            - pattern-inside: |
                                $SHELL = Shell.$ANY(...)
                                ...
                                $SHELL.$X(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern: $SHELL.$X(...)
            - metavariable-pattern:
                metavariable: $X
                patterns:
                    - pattern-either:
                        - pattern: cat
                        - pattern: chdir
                        - pattern: chroot
                        - pattern: delete
                        - pattern: entries
                        - pattern: exec
                        - pattern: foreach
                        - pattern: glob
                        - pattern: install
                        - pattern: lchmod
                        - pattern: lchown
                        - pattern: link
                        - pattern: load
                        - pattern: load_file
                        - pattern: makedirs
                        - pattern: move
                        - pattern: new
                        - pattern: open
                        - pattern: read
                        - pattern: readlines
                        - pattern: rename
                        - pattern: rmdir
                        - pattern: safe_unlink
                        - pattern: symlink
                        - pattern: syscopy
                        - pattern: sysopen
                        - pattern: system
                        - pattern: truncate
                        - pattern: unlink
      pattern-sources:
        - pattern-either:
            - pattern: params[...]
            - pattern: cookies
            - pattern: request.env
      severity: ERROR
    - id: ruby.rails.security.audit.sqli.ruby-pg-sqli.ruby-pg-sqli
      languages:
        - ruby
      message: 'Detected string concatenation with a non-literal variable in a pg Ruby SQL state
ment. This could lead to SQL injection if the variable is user-controlled and not properly sanit
ized. In order to prevent SQL injection, use parameterized queries or prepared statements instea
d. You can use parameterized queries like so: `conn.exec_params(''SELECT $1 AS a, $2 AS b, $3 AS
 c'', [1, 2, nil])` And you can use prepared statements with `exec_prepared`.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://www.rubydoc.info/gems/pg/PG/Connection
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-propagators:
        - from: $Y
          pattern: $X << $Y
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          to: $X
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $CON = PG.connect(...)
                    ...
                - pattern-inside: |
                    $CON = PG::Connection.open(...)
                    ...
                - pattern-inside: |
                    $CON = PG::Connection.new(...)
                    ...
            - pattern-either:
                - pattern: |
                    $CON.$METHOD($X,...)
                - pattern: |
                    $CON.$METHOD $X, ...
            - focus-metavariable: $X
            - metavariable-regex:
                metavariable: $METHOD
                regex: ^(exec|exec_params)$
      pattern-sources:
        - pattern-either:
            - pattern: |
                params
            - pattern: |
                cookies
      severity: WARNING
    - id: ruby.rails.security.audit.xss.avoid-link-to.avoid-link-to
      languages:
        - ruby
      message: This code includes user input in `link_to`. In Rails 2.x, the body of `link_to` i
s not escaped. This means that user input which reaches the body will be executed when the HTML
is rendered. Even in other versions, values starting with `javascript:` or `data:` are not escap
ed. It is better to create and use a safer function which checks the body argument.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://brakemanscanner.org/docs/warning_types/link_to/
            - https://brakemanscanner.org/docs/warning_types/link_to_href/
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_link_to.rb
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern: |
                "...#{...}..."
            - pattern-not: |
                "#{...}..."
      pattern-sinks:
        - pattern: link_to(...)
      pattern-sources:
        - pattern: params
        - pattern: cookies
        - pattern: request.env
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-either:
            - pattern: $MODEL.url(...)
            - pattern: $MODEL.uri(...)
            - pattern: $MODEL.link(...)
            - pattern: $MODEL.page(...)
            - pattern: $MODEL.site(...)
      severity: WARNING
    - id: ruby.rails.security.audit.xss.avoid-redirect.avoid-redirect
      languages:
        - ruby
      message: When a redirect uses user input, a malicious user can spoof a website under a tru
sted URL or access restricted parts of a site. When using user-supplied values, sanitize the val
ue before using it for the redirect.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-601: URL Redirection to Untrusted Site (''Open Redirect'')'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://brakemanscanner.org/docs/warning_types/redirect/
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sanitizers:
        - pattern: params.merge(:only_path => true)
        - pattern: params.merge(:host => ...)
      pattern-sinks:
        - pattern: redirect_to(...)
      pattern-sources:
        - pattern: params
        - pattern: cookies
        - pattern: request.env
        - patterns:
            - pattern: $MODEL.$X(...)
            - pattern-not: $MODEL.$X("...")
            - metavariable-pattern:
                metavariable: $X
                pattern-either:
                    - pattern: all
                    - pattern: create
                    - pattern: create!
                    - pattern: find
                    - pattern: find_by_sql
                    - pattern: first
                    - pattern: last
                    - pattern: new
                    - pattern: from
                    - pattern: group
                    - pattern: having
                    - pattern: joins
                    - pattern: lock
                    - pattern: order
                    - pattern: reorder
                    - pattern: select
                    - pattern: where
                    - pattern: find_by
                    - pattern: find_by!
                    - pattern: take
      severity: WARNING
    - id: ruby.rails.security.audit.xss.avoid-render-dynamic-path.avoid-render-dynamic-path
      languages:
        - ruby
      message: Avoid rendering user input. It may be possible for a malicious user to input a pa
th that lets them access a template they shouldn't. To prevent this, check dynamic template path
s against a predefined allowlist to make sure it's an allowed template.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://brakemanscanner.org/docs/warning_types/dynamic_render_paths/
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-inside: render($X => $INPUT, ...)
            - pattern: $INPUT
            - metavariable-pattern:
                metavariable: $X
                pattern-either:
                    - pattern: action
                    - pattern: template
                    - pattern: partial
                    - pattern: file
      pattern-sources:
        - pattern: params
        - pattern: cookies
        - pattern: request.env
      severity: WARNING
    - id: ruby.rails.security.brakeman.check-before-filter.check-before-filter
      languages:
        - ruby
      message: 'Disabled-by-default Rails controller checks make it much easier to introduce acc
ess control mistakes. Prefer an allowlist approach with `:only => [...]` rather than `except: =>
 [...]`'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-284: Improper Access Control'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_skip_before_filter.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: search
      patterns:
        - pattern-either:
            - pattern: |
                skip_filter ..., :except => $ARGS
            - pattern: |
                skip_before_filter ..., :except => $ARGS
            - pattern: |
                skip_before_action ..., :except => $ARGS
      severity: ERROR
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: ruby.rails.security.brakeman.check-dynamic-render-local-file-include.check-dynamic-ren
der-local-file-include
      languages:
        - generic
      message: Found request parameters in a call to `render` in a dynamic context. This can all
ow end users to request arbitrary local files which may result in leaking sensitive information
persisted on disk.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Sec
urity_Testing/07-Input_Validation_Testing/11.1-Testing_for_Local_File_Inclusion
            - https://github.com/presidentbeef/brakeman/blob/f74cb53ead47f0af821d98b5b41e16d6310
0c240/test/apps/rails2/app/views/home/test_render.html.erb
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_render.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: search
      paths:
        include:
            - '*.erb'
      patterns:
        - pattern: |
            params[...]
        - pattern-inside: |
            render :file => ...
      severity: WARNING
    - id: ruby.rails.security.brakeman.check-http-verb-confusion.check-http-verb-confusion
      languages:
        - ruby
      message: Found an improperly constructed control flow block with `request.get?`. Rails wil
l route HEAD requests as GET requests but they will fail the `request.get?` check, potentially c
ausing unexpected behavior unless an `elif` condition is used.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-650: Trusting HTTP Permission Methods on the Server Side'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A04:2021 - Insecure Design
        references:
            - https://github.com/presidentbeef/brakeman/blob/main/test/apps/rails6/app/controlle
rs/accounts_controller.rb
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_verb_confusion.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: search
      patterns:
        - pattern: |
            if request.get?
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
          else
            ...
          end
      - pattern-not-inside: |
          if ...
          elsif ...
            ...
          end
    severity: ERROR
  - id: ruby.rails.security.brakeman.check-rails-session-secret-handling.check-rails-session-s
ecret-handling
    languages:
      - ruby
    message: Found a string literal assignment to a Rails session secret `$KEY`. Do not commit
 secret values to source control! Any user in possession of this value may falsify arbitrary ses
sion data in your application. Read this value from an environment variable, KMS, or file on dis
k outside of source control.
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
          - 'CWE-540: Inclusion of Sensitive Information in Source Code'
      impact: MEDIUM
      likelihood: MEDIUM
      owasp:
          - A01:2021 - Broken Access Control
      references:
          - https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_
Security_Testing/06-Session_Management_Testing/02-Testing_for_Cookies_Attributes
          - https://github.com/presidentbeef/brakeman/blob/main/test/apps/rails4_with_engines/
config/initializers/secret_token.rb
          - https://github.com/presidentbeef/brakeman/blob/main/test/apps/rails3/config/initia
lizers/secret_token.rb
      source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_session_settings.rb
      subcategory:
          - vuln
      technology:
          - ruby
          - rails
    patterns:
      - pattern-either:
          - patterns:
              - pattern: |
                  :$KEY => "$LITERAL"
              - pattern-inside: |
                  ActionController::Base.session = {...}
          - pattern: |
              $RAILS::Application.config.$KEY = "$LITERAL"
          - pattern: |
              Rails.application.config.$KEY = "$LITERAL"
      - metavariable-regex:
          metavariable: $KEY
          regex: ^secret(_(token|key_base))?$
    severity: WARNING
  - id: ruby.rails.security.brakeman.check-redirect-to.check-redirect-to
    languages:
      - ruby
    message: Found potentially unsafe handling of redirect behavior $X. Do not pass `params` t
o `redirect_to` without the `:only_path => true` hash value.
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
          - 'CWE-601: URL Redirection to Untrusted Site (''Open Redirect'')'
      impact: MEDIUM
      likelihood: MEDIUM
      owasp:
          - A01:2021 - Broken Access Control
      references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_
Cheat_Sheet.html
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_redirect.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: |
                        $F(...)
                    - metavariable-pattern:
                        metavariable: $F
                        patterns:
                            - pattern-not-regex: (params|url_for|cookies|request.env|permit|redi
rect_to)
                - pattern: |
                    params.merge! :only_path => true
                    ...
                - pattern: |
                    params.slice(...)
                    ...
                - pattern: |
                    redirect_to [...]
                - patterns:
                    - pattern: |
                        $MODEL. ... .$M(...)
                        ...
                    - metavariable-regex:
                        metavariable: $MODEL
                        regex: '[A-Z]\w+'
                    - metavariable-regex:
                        metavariable: $M
                        regex: (all|create|find|find_by|find_by_sql|first|last|new|from|group|ha
ving|joins|lock|order|reorder|select|where|take)
                - patterns:
                    - pattern: |
                        params.$UNSAFE_HASH.merge(...,:only_path => true,...)
                        ...
                    - metavariable-regex:
                        metavariable: $UNSAFE_HASH
                        regex: to_unsafe_h(ash)?
                - patterns:
                    - pattern: params.permit(...,$X,...)
                    - metavariable-pattern:
                        metavariable: $X
                        patterns:
                            - pattern-not-regex: (host|port|(sub)?domain)
      pattern-sinks:
        - patterns:
            - pattern: $X
            - pattern-inside: |
                redirect_to $X, ...
            - pattern-not-regex: params\.\w+(?<!permit)\(.*?\)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: params
                - pattern: cookies
                - pattern: request.env
                - pattern: url_for(params[...],...,:only_path => false,...)
      severity: WARNING
    - id: ruby.rails.security.brakeman.check-regex-dos.check-regex-dos
      languages:
        - ruby
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Found a potentially user-controllable argument in the construction of a regular e
xpressions. This may result in excessive resource consumption when applied to certain inputs, or
 when the user is allowed to control the match target. Avoid allowing users to specify regular e
xpressions processed by the server. If you must support user-controllable input in a regular exp
ression, use an allow-list to restrict the expressions users may supply to limit catastrophic ba
cktracking.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1333: Inefficient Regular Expression Complexity'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2017 - Sensitive Data Exposure
        references:
            - https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReD
oS
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_regex_dos.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: $Y
                    - pattern-inside: |
                        /...#{...}.../
                - patterns:
                    - pattern: $Y
                    - pattern-inside: |
                        Regexp.new(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    cookies[...]
                - patterns:
                    - pattern: |
                        cookies. ... .$PROPERTY[...]
                    - metavariable-regex:
                        metavariable: $PROPERTY
                        regex: (?!signed|encrypted)
                - pattern: |
                    params[...]
                - pattern: |
                    request.env[...]
                - patterns:
                    - pattern: $Y
                    - pattern-either:
                        - pattern-inside: |
                            $RECORD.read_attribute($Y)
                        - pattern-inside: |
                            $RECORD[$Y]
                    - metavariable-regex:
                        metavariable: $RECORD
                        regex: '[A-Z][a-z]+'
      severity: ERROR
    - id: ruby.rails.security.brakeman.check-render-local-file-include.check-render-local-file-i
nclude
      languages:
        - ruby
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Found request parameters in a call to `render`. This can allow end users to reque
st arbitrary local files which may result in leaking sensitive information persisted on disk. Wh
ere possible, avoid letting users specify template paths for `render`. If you must allow user in
put, use an allow-list of known templates or normalize the user-supplied value with `File.basena
me(...)`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-22: Improper Limitation of a Pathname to a Restricted Directory (''Path Trave
rsal'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Sec
urity_Testing/07-Input_Validation_Testing/11.1-Testing_for_Local_File_Inclusion
            - https://github.com/presidentbeef/brakeman/blob/f74cb53/test/apps/rails2/app/contro
llers/home_controller.rb#L48-L60
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_render.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
        vulnerability_class:
            - Path Traversal
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern: $MAP[...]
            - metavariable-pattern:
                metavariable: $MAP
                patterns:
                    - pattern-not-regex: params
        - pattern: File.basename(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    render ..., file: $X
                - pattern: |
                    render ..., inline: $X
                - pattern: |
                    render ..., template: $X
                - pattern: |
                    render ..., action: $X
                - pattern: |
                    render $X, ...
            - focus-metavariable: $X
      pattern-sources:
        - patterns:
            - pattern: params[...]
      severity: WARNING
    - id: ruby.rails.security.brakeman.check-reverse-tabnabbing.check-reverse-tabnabbing
      languages:
        - generic
      message: Setting an anchor target of `_blank` without the `noopener` or `noreferrer` attri
bute allows reverse tabnabbing on Internet Explorer, Opera, and Android Webview.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1022: Use of Web Link to Untrusted Target with window.opener Access'
        impact: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: MEDIUM
        references:
            - https://developer.mozilla.org/en-US/docs/Web/HTML/Element/a#browser_compatibility
            - https://github.com/presidentbeef/brakeman/blob/3f5d5d5/test/apps/rails5/app/views/
users/show.html.erb
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_reverse_tabnabbing.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: search
      paths:
        include:
            - '*.erb'
      patterns:
        - pattern: |
            _blank
        - pattern-inside: |
            target: ...
        - pattern-not-inside: |
            <%= ... rel: 'noopener noreferrer' ...%>
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    <%= $...INLINERUBYDO do -%>
                    ...
                    <% end %>
                - metavariable-pattern:
                    language: ruby
                    metavariable: $...INLINERUBYDO
                    patterns:
                        - pattern: |
                            link_to ...
                        - pattern-not: |
                            link_to "...", "...", ...
            - patterns:
                - pattern-not-inside: |
                    <%= ... do - %>
                - pattern-inside: |
                    <%= $...INLINERUBY %>
                - metavariable-pattern:
                    language: ruby
                    metavariable: $...INLINERUBY
                    patterns:
                        - pattern: |
                            link_to ...
                        - pattern-not: |
                            link_to '...', '...', ...
                        - pattern-not: |
                            link_to '...', target: ...
      severity: WARNING
    - id: ruby.rails.security.brakeman.check-secrets.check-secrets
      languages:
        - ruby
      message: Found a Brakeman-style secret - a variable with the name password/secret/api_key/
rest_auth_site_key and a non-empty string literal value.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://github.com/presidentbeef/brakeman/blob/3f5d5d5f00864cdf7769c50f5bd26f1769a
4ba75/test/apps/rails3.1/app/controllers/users_controller.rb
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_secrets.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      patterns:
        - pattern: $VAR = "$VALUE"
        - metavariable-regex:
            metavariable: $VAR
            regex: (?i)password|secret|(rest_auth_site|api)_key$
        - metavariable-regex:
            metavariable: $VALUE
            regex: .+
      severity: WARNING
    - id: ruby.rails.security.brakeman.check-send-file.check-send-file
      languages:
        - ruby
      message: Allowing user input to `send_file` allows a malicious user to potentially read ar
bitrary files from the server. Avoid accepting user input in `send_file` or normalize with `File
.basename(...)`
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-73: External Control of File Name or Path'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/www-community/attacks/Path_Traversal
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control/
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_send_file.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                send_file ...
      pattern-sources:
        - pattern-either:
            - pattern: |
                cookies[...]
            - patterns:
                - pattern: |
                    cookies. ... .$PROPERTY[...]
                - metavariable-regex:
                    metavariable: $PROPERTY
                    regex: (?!signed|encrypted)
            - pattern: |
                params[...]
            - pattern: |
                request.env[...]
      severity: ERROR
    - id: ruby.rails.security.brakeman.check-sql.check-sql
      languages:
        - ruby
      message: Found potential SQL injection due to unsafe SQL query construction via $X. Where
possible, prefer parameterized queries.
      metadata:
        category: security
        confidence: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://owasp.org/www-community/attacks/SQL_Injection
            - https://github.com/presidentbeef/brakeman/blob/main/test/apps/rails3.1/app/models/
product.rb
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_sql.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: $X
                    - pattern-either:
                        - pattern-inside: |
                            :$KEY => $X
                        - pattern-inside: |
                            ["...",$X,...]
                - pattern: |
                    params[...].to_i
                - pattern: |
                    params[...].to_f
                - patterns:
                    - pattern: |
                        params[...] ? $A : $B
                    - metavariable-pattern:
                        metavariable: $A
                        patterns:
                            - pattern-not: |
                                params[...]
                    - metavariable-pattern:
                        metavariable: $B
                        patterns:
                            - pattern-not: |
                                params[...]
      pattern-sinks:
        - patterns:
            - pattern: $X
            - pattern-not-inside: |
                $P.where("...",...)
            - pattern-not-inside: |
                $P.where(:$KEY => $VAL,...)
            - pattern-either:
                - pattern-inside: |
                    $P.$M(...)
                - pattern-inside: |
                    $P.$M("...",...)
            - pattern-inside: |
                class $P < ActiveRecord::Base
                  ...
                end
            - metavariable-regex:
                metavariable: $M
                regex: (where|find|first|last|select|minimum|maximum|calculate|sum|average)
      pattern-sources:
        - pattern-either:
            - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              cookies[...]
          - patterns:
              - pattern: |
                  cookies. ... .$PROPERTY[...]
              - metavariable-regex:
                  metavariable: $PROPERTY
                  regex: (?!signed|encrypted)
          - pattern: |
              params[...]
          - pattern: |
              request.env[...]
      severity: ERROR
    - id: ruby.rails.security.brakeman.check-unsafe-reflection-methods.check-unsafe-reflection-m
ethods
      languages:
        - ruby
      message: Found user-controllable input to a reflection method. This may allow a user to al
ter program behavior and potentially execute arbitrary instructions in the context of the proces
s. Do not provide arbitrary user input to `tap`, `method`, or `to_proc`
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://github.com/presidentbeef/brakeman/blob/main/test/apps/rails6/app/controlle
rs/groups_controller.rb
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_unsafe_reflection_methods.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: $X
            - pattern-either:
                - pattern-inside: |
                    $X. ... .to_proc
                - patterns:
                    - pattern-inside: |
                        $Y.method($Z)
                    - focus-metavariable: $Z
                - patterns:
                    - pattern-inside: |
                        $Y.tap($Z)
                    - focus-metavariable: $Z
                - patterns:
                    - pattern-inside: |
                        $Y.tap{ |$ANY| $Z }
                    - focus-metavariable: $Z
      pattern-sources:
        - pattern-either:
            - pattern: |
                cookies[...]
            - patterns:
                - pattern: |
                    cookies. ... .$PROPERTY[...]
                - metavariable-regex:
                    metavariable: $PROPERTY
                    regex: (?!signed|encrypted)
            - pattern: |
                params[...]
            - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                request.env[...]
      severity: ERROR
    - id: ruby.rails.security.brakeman.check-unsafe-reflection.check-unsafe-reflection
      languages:
        - ruby
      message: Found user-controllable input to Ruby reflection functionality. This allows a rem
ote user to influence runtime behavior, up to and including arbitrary remote code execution. Do
not provide user-controllable input to reflection functionality. Do not call symbol conversion o
n user-controllable input.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://github.com/presidentbeef/brakeman/blob/main/test/apps/rails2/app/controlle
rs/application_controller.rb
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_unsafe_reflection.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: $X
            - pattern-either:
                - pattern-inside: |
                    $X.constantize
                - pattern-inside: |
                    $X. ... .safe_constantize
                - pattern-inside: |
                    const_get(...)
                - pattern-inside: |
                    qualified_const_get(...)
      pattern-sources:
        - pattern-either:
            - pattern: |
                cookies[...]
            - patterns:
                - pattern: |
                    cookies. ... .$PROPERTY[...]
                - metavariable-regex:
                    metavariable: $PROPERTY
                    regex: (?!signed|encrypted)
            - pattern: |
                params[...]
            - pattern: |
                request.env[...]
      severity: ERROR
    - id: ruby.rails.security.brakeman.check-unscoped-find.check-unscoped-find
      languages:
        - ruby
      message: Found an unscoped `find(...)` with user-controllable input. If the ActiveRecord m
odel being searched against is sensitive, this may lead to Insecure Direct Object Reference (IDO
R) behavior and allow users to read arbitrary records. Scope the find to the current user, e.g.
`current_user.accounts.find(params[:id])`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-639: Authorization Bypass Through User-Controlled Key'
        impact: HIGH
        likelihood: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://brakemanscanner.org/docs/warning_types/unscoped_find/
            - https://github.com/presidentbeef/brakeman/blob/main/test/apps/rails3.1/app/control
lers/users_controller.rb
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_unscoped_find.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: $MODEL.find(...)
                - pattern: $MODEL.find_by_id(...)
                - pattern: $MODEL.find_by_id!(...)
            - metavariable-regex:
                metavariable: $MODEL
                regex: '[A-Z]\S+'
      pattern-sources:
        - pattern-either:
            - pattern: |
                cookies[...]
            - patterns:
                - pattern: |
                    cookies. ... .$PROPERTY[...]
                - metavariable-regex:
                    metavariable: $PROPERTY
                    regex: (?!signed|encrypted)
            - pattern: |
                params[...]
            - pattern: |
                request.env[...]
      severity: WARNING
    - id: ruby.rails.security.brakeman.check-validation-regex.check-validation-regex
      languages:
        - ruby
      message: $V Found an incorrectly-bounded regex passed to `validates_format_of` or `validat
e ... format => ...`. Ruby regex behavior is multiline by default and lines should be terminated
 by `\A` for beginning of line and `\Z` for end of line, respectively.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-185: Incorrect Regular Expression'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://brakemanscanner.org/docs/warning_types/format_validation/
            - https://github.com/presidentbeef/brakeman/blob/aef6253a8b7bcb97116f2af1ed2a561a6ae
35bd5/test/apps/rails3/app/models/account.rb
            - https://github.com/presidentbeef/brakeman/blob/main/test/apps/rails3.1/app/models/
account.rb
        source-rule-url: https://github.com/presidentbeef/brakeman/blob/main/lib/brakeman/checks
/check_validation_regex.rb
        subcategory:
            - vuln
        technology:
            - ruby
            - rails
      mode: search
      patterns:
        - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                validates ..., :format => <... $V ...>,...
            - pattern: |
                validates_format_of ..., :with => <... $V ...>,...
        - metavariable-regex:
            metavariable: $V
            regex: /(.{2}(?<!\\A)[^\/]+|[^\/]+(?<!\\[Zz]))\/
      severity: ERROR
    - id: ruby.rails.security.injection.raw-html-format.raw-html-format
      languages:
        - ruby
      message: Detected user input flowing into a manually constructed HTML string. You may be a
ccidentally bypassing secure methods of rendering HTML by manually constructing HTML and this co
uld create a cross-site scripting vulnerability, which could let attackers steal sensitive user
data. Use the `render template` and make template files which will safely render HTML instead, o
r inspect that the HTML is absolutely rendered safely with a function like `sanitize`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://www.netsparker.com/blog/web-security/preventing-xss-ruby-on-rails-web-appl
ications/
            - https://api.rubyonrails.org/classes/ActionView/Helpers/SanitizeHelper.html
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sanitizers:
        - pattern-either:
            - pattern: sanitize(...)
            - pattern: strip_tags(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: |
                        $HTMLSTR
                    - pattern-regex: <\w+.*
                - patterns:
                    - pattern-either:
                        - pattern: Kernel::sprintf("$HTMLSTR", ...)
                        - pattern: |
                            "$HTMLSTR" + $EXPR
                        - pattern: |
                            "$HTMLSTR" % $EXPR
                    - metavariable-pattern:
                        language: generic
                        metavariable: $HTMLSTR
                        pattern: <$TAG ...
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: params
                - pattern: request
      severity: WARNING
    - id: ruby.rails.security.injection.tainted-sql-string.tainted-sql-string
      languages:
        - ruby
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Detected user input used to manually construct a SQL string. This is usually bad
practice because manual construction could accidentally result in a SQL injection. An attacker c
ould use a SQL injection to steal or modify contents of the database. Instead, use a parameteriz
ed query which is available by default in most database engines. Alternatively, consider using a
n object-relational mapper (ORM) such as ActiveRecord which will protect your queries.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://rorsecurity.info/portfolio/ruby-on-rails-sql-injection-cheat-sheet
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sanitizers:
        - pattern: |
            $PARAMS.slice(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - patterns:
                            - pattern: |
                                $RECORD.where($X,...)
                        - pattern: |
                            $RECORD.find(..., :conditions => $X,...)
                    - focus-metavariable: $X
                - patterns:
                    - pattern: |
                        "$SQLVERB#{$EXPR}..."
                    - pattern-not-inside: |
                        $FUNC("...",  "...#{$EXPR}...",...)
                    - focus-metavariable: $SQLVERB
                    - pattern-regex: (?i)(select|delete|insert|create|update|alter|drop)\b
                - patterns:
                    - pattern-either:
                        - pattern: Kernel::sprintf("$SQLSTR", $EXPR)
                        - pattern: |
                            "$SQLSTR" + $EXPR
                        - pattern: |
                            "$SQLSTR" % $EXPR
                    - pattern-not-inside: |
                        $FUNC("...",  "...#{$EXPR}...",...)
                    - focus-metavariable: $EXPR
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: (?i)(select|delete|insert|create|update|alter|drop)\b
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: params
                - pattern: request
      severity: ERROR
    - id: ruby.rails.security.injection.tainted-url-host.tainted-url-host
      languages:
        - ruby
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: User data flows into the host portion of this manually-constructed URL. This coul
d allow an attacker to send data to their own server, potentially exposing sensitive data such a
s cookies or authorization information sent with this request. They could also probe internal se
rvers or other resources that the server running this code can access. (This is called server-si
de request forgery, or SSRF.) Do not allow arbitrary hosts. Use the `ssrf_filter` gem and guard
the url construction with `SsrfFilter(...)`, or create an allowlist for approved hosts.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevent
ion_Cheat_Sheet.html
            - https://github.com/arkadiyt/ssrf_filter
        subcategory:
            - vuln
        technology:
            - rails
      mode: taint
      pattern-sanitizers:
        - pattern: SsrfFilter
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: |
                        $URLSTR
                    - pattern-regex: \w+:\/\/#{.*}
                - patterns:
                    - pattern-either:
                        - pattern: Kernel::sprintf("$URLSTR", ...)
                        - pattern: |
                            "$URLSTR" + $EXPR
                        - pattern: |
                            "$URLSTR" % $EXPR
                    - metavariable-pattern:
                        language: generic
                        metavariable: $URLSTR
                        pattern: $SCHEME:// ...
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: params
                - pattern: request
      severity: WARNING
    - id: rust.lang.security.args-os.args-os
      languages:
        - rust
      message: 'args_os should not be used for security operations. From the docs: "The first el
ement is traditionally the path of the executable, but it can be set to arbitrary text, and migh
t not even exist. This means this property should not be relied upon for security purposes."'
      metadata:
        category: security
        confidence: HIGH
        cwe: 'CWE-807: Reliance on Untrusted Inputs in a Security Decision'
        impact: LOW
        likelihood: LOW
        references:
            - https://doc.rust-lang.org/stable/std/env/fn.args_os.html
        subcategory: audit
        technology:
            - rust
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      pattern: std::env::args_os()
      severity: INFO
    - id: rust.lang.security.args.args
      languages:
        - rust
      message: 'args should not be used for security operations. From the docs: "The first eleme
nt is traditionally the path of the executable, but it can be set to arbitrary text, and might n
ot even exist. This means this property should not be relied upon for security purposes."'
      metadata:
        category: security
        confidence: HIGH
        cwe: 'CWE-807: Reliance on Untrusted Inputs in a Security Decision'
        impact: LOW
        likelihood: LOW
        references:
            - https://doc.rust-lang.org/stable/std/env/fn.args.html
        subcategory: audit
        technology:
            - rust
      pattern: std::env::args()
      severity: INFO
    - id: rust.lang.security.current-exe.current-exe
      languages:
        - rust
      message: 'current_exe should not be used for security operations. From the docs: "The outp
ut of this function should not be trusted for anything that might have security implications. Ba
sically, if users can run the executable, they can change the output arbitrarily."'
      metadata:
        category: security
        confidence: HIGH
        cwe: 'CWE-807: Reliance on Untrusted Inputs in a Security Decision'
        impact: LOW
        likelihood: LOW
        references:
            - https://doc.rust-lang.org/stable/std/env/fn.current_exe.html#security
        subcategory: audit
        technology:
            - rust
      pattern: std::env::current_exe()
      severity: INFO
    - id: rust.lang.security.insecure-hashes.insecure-hashes
      languages:
        - rust
      message: Detected cryptographically insecure hashing function
      metadata:
        category: security
        confidence: HIGH
        cwe: 'CWE-328: Use of Weak Hash'
        impact: MEDIUM
        likelihood: LOW
        references:
            - https://github.com/RustCrypto/hashes
            - https://docs.rs/md2/latest/md2/
            - https://docs.rs/md4/latest/md4/
            - https://docs.rs/md5/latest/md5/
            - https://docs.rs/sha-1/latest/sha1/
        subcategory: audit
        technology:
            - rust
      pattern-either:
        - pattern: md2::Md2::new(...)
        - pattern: md4::Md4::new(...)
        - pattern: md5::Md5::new(...)
        - pattern: sha1::Sha1::new(...)
      severity: WARNING
    - id: rust.lang.security.reqwest-accept-invalid.reqwest-accept-invalid
      languages:
        - rust
      message: Dangerously accepting invalid TLS information
      metadata:
        category: security
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        confidence: HIGH
        cwe: 'CWE-295: Improper Certificate Validation'
        impact: MEDIUM
        likelihood: LOW
        references:
            - https://docs.rs/reqwest/latest/reqwest/struct.ClientBuilder.html#method.danger_acc
ept_invalid_hostnames
            - https://docs.rs/reqwest/latest/reqwest/struct.ClientBuilder.html#method.danger_acc
ept_invalid_certs
        subcategory: vuln
        technology:
            - reqwest
      pattern-either:
        - pattern: reqwest::Client::builder(). ... .danger_accept_invalid_hostnames(true)
        - pattern: reqwest::Client::builder(). ... .danger_accept_invalid_certs(true)
      severity: WARNING
    - id: rust.lang.security.reqwest-set-sensitive.reqwest-set-sensitive
      languages:
        - rust
      message: Set sensitive flag on security headers with 'set_sensitive' to treat data with sp
ecial care
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-921: Storage of Sensitive Data in a Mechanism without Access Control'
        impact: LOW
        likelihood: LOW
        references:
            - https://docs.rs/reqwest/latest/reqwest/header/struct.HeaderValue.html#method.set_s
ensitive
        subcategory: audit
        technology:
            - reqwest
      patterns:
        - pattern: |
            let mut $HEADERS = header::HeaderMap::new();
            ...
            let $HEADER_VALUE = <... header::HeaderValue::$FROM_FUNC(...) ...>;
            ...
            $HEADERS.insert($HEADER, $HEADER_VALUE);
        - pattern-not: |
            let mut $HEADERS = header::HeaderMap::new();
            ...
            let $HEADER_VALUE = <... header::HeaderValue::$FROM_FUNC(...) ...>;
            ...
            $HEADER_VALUE.set_sensitive(true);
            ...
            $HEADERS.insert($HEADER, $HEADER_VALUE);
        - metavariable-pattern:
            metavariable: $FROM_FUNC
            pattern-either:
                - pattern: from_static
                - pattern: from_str
                - pattern: from_name
                - pattern: from_bytes
                - pattern: from_maybe_shared
        - metavariable-pattern:
            metavariable: $HEADER
            pattern-either:
                - pattern: header::AUTHORIZATION
                - pattern: '"Authorization"'
      severity: INFO
    - id: rust.lang.security.rustls-dangerous.rustls-dangerous
      languages:
        - rust
      message: Dangerous client config used, ensure SSL verification
      metadata:
        category: security
        confidence: HIGH
        cwe: 'CWE-295: Improper Certificate Validation'
        impact: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: LOW
        references:
            - https://docs.rs/rustls/latest/rustls/client/struct.DangerousClientConfig.html
            - https://docs.rs/rustls/latest/rustls/client/struct.ClientConfig.html#method.danger
ous
        subcategory: vuln
        technology:
            - rustls
      pattern-either:
        - pattern: rustls::client::DangerousClientConfig
        - pattern: $CLIENT.dangerous().set_certificate_verifier(...)
        - pattern: |
            let $CLIENT = rustls::client::ClientConfig::dangerous(...);
            ...
            $CLIENT.set_certificate_verifier(...);
      severity: WARNING
    - id: rust.lang.security.ssl-verify-none.ssl-verify-none
      languages:
        - rust
      message: SSL verification disabled, this allows for MitM attacks
      metadata:
        category: security
        confidence: HIGH
        cwe: 'CWE-295: Improper Certificate Validation'
        impact: MEDIUM
        likelihood: LOW
        references:
            - https://docs.rs/openssl/latest/openssl/ssl/struct.SslContextBuilder.html#method.se
t_verify
        subcategory: vuln
        technology:
            - openssl
      pattern: $BUILDER.set_verify(openssl::ssl::SSL_VERIFY_NONE)
      severity: WARNING
    - id: rust.lang.security.temp-dir.temp-dir
      languages:
        - rust
      message: 'temp_dir should not be used for security operations. From the docs: ''The tempor
ary directory may be shared among users, or between processes with different privileges; thus, t
he creation of any files or directories in the temporary directory must use a secure method to c
reate a uniquely named file. Creating a file or directory with a fixed or predictable name may r
esult in “insecure temporary file” security vulnerabilities.'''
      metadata:
        category: security
        confidence: HIGH
        cwe: 'CWE-807: Reliance on Untrusted Inputs in a Security Decision'
        impact: LOW
        likelihood: LOW
        references:
            - https://doc.rust-lang.org/stable/std/env/fn.temp_dir.html
        subcategory: audit
        technology:
            - rust
      pattern: std::env::temp_dir()
      severity: INFO
    - id: rust.lang.security.unsafe-usage.unsafe-usage
      languages:
        - rust
      message: Detected 'unsafe' usage, please audit for secure usage
      metadata:
        category: security
        confidence: HIGH
        cwe: 'CWE-242: Use of Inherently Dangerous Function'
        impact: LOW
        likelihood: LOW
        references:
            - https://doc.rust-lang.org/std/keyword.unsafe.html
        subcategory: audit
        technology:
            - rust
      pattern: unsafe { ... }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: INFO
    - id: scala.jwt-scala.security.jwt-scala-hardcode.jwt-scala-hardcode
      languages:
        - scala
      message: 'Hardcoded JWT secret or private key is used. This is a Insufficiently Protected
Credentials weakness: https://cwe.mitre.org/data/definitions/522.html Consider using an appropri
ate security mechanism to protect the credentials (e.g. keeping secrets in environment variables
)'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://jwt-scala.github.io/jwt-scala/
        subcategory:
            - vuln
        technology:
            - scala
      patterns:
        - pattern-inside: |
            import pdi.jwt.$DEPS
            ...
        - pattern-either:
            - pattern: $JWT.encode($X, "...", ...)
            - pattern: $JWT.decode($X, "...", ...)
            - pattern: $JWT.decodeRawAll($X, "...", ...)
            - pattern: $JWT.decodeRaw($X, "...", ...)
            - pattern: $JWT.decodeAll($X, "...", ...)
            - pattern: $JWT.validate($X, "...", ...)
            - pattern: $JWT.isValid($X, "...", ...)
            - pattern: $JWT.decodeJson($X, "...", ...)
            - pattern: $JWT.decodeJsonAll($X, "...", ...)
            - patterns:
                - pattern-either:
                    - pattern: $JWT.encode($X, $KEY, ...)
                    - pattern: $JWT.decode($X, $KEY, ...)
                    - pattern: $JWT.decodeRawAll($X, $KEY, ...)
                    - pattern: $JWT.decodeRaw($X, $KEY, ...)
                    - pattern: $JWT.decodeAll($X, $KEY, ...)
                    - pattern: $JWT.validate($X, $KEY, ...)
                    - pattern: $JWT.isValid($X, $KEY, ...)
                    - pattern: $JWT.decodeJson($X, $KEY, ...)
                    - pattern: $JWT.decodeJsonAll($X, $KEY, ...)
                    - pattern: $JWT.encode($X, this.$KEY, ...)
                    - pattern: $JWT.decode($X, this.$KEY, ...)
                    - pattern: $JWT.decodeRawAll($X, this.$KEY, ...)
                    - pattern: $JWT.decodeRaw($X, this.$KEY, ...)
                    - pattern: $JWT.decodeAll($X, this.$KEY, ...)
                    - pattern: $JWT.validate($X, this.$KEY, ...)
                    - pattern: $JWT.isValid($X, this.$KEY, ...)
                    - pattern: $JWT.decodeJson($X, this.$KEY, ...)
                    - pattern: $JWT.decodeJsonAll($X, this.$KEY, ...)
                - pattern-either:
                    - pattern-inside: |
                        class $CL {
                          ...
                          $KEY = "..."
                          ...
                        }
                    - pattern-inside: |
                        object $CL {
                          ...
                          $KEY = "..."
                          ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    }
        - metavariable-pattern:
            metavariable: $JWT
            patterns:
                - pattern-either:
                    - pattern: Jwt
                    - pattern: JwtArgonaut
                    - pattern: JwtCirce
                    - pattern: JwtJson4s
                    - pattern: JwtJson
                    - pattern: JwtUpickle
      severity: WARNING
    - id: scala.lang.correctness.positive-number-index-of.positive-number-index-of
      languages:
        - scala
      message: Flags scala code that look for values that are greater than 0. This ignores the f
irst element, which is most likely a bug. Instead, use indexOf with -1. If the intent is to chec
k the inclusion of a value, use the contains method instead.
      metadata:
        category: correctness
        confidence: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        references:
            - https://blog.codacy.com/9-scala-security-issues/
        technology:
            - scala
      patterns:
        - pattern-either:
            - patterns:
                - pattern: |
                    $OBJ.indexOf(...) > $VALUE
                - metavariable-comparison:
                    comparison: $VALUE >= 0
                    metavariable: $VALUE
            - patterns:
                - pattern: |
                    $OBJ.indexOf(...) >= $SMALLERVAL
                - metavariable-comparison:
                    comparison: $SMALLERVAL > 0
                    metavariable: $SMALLERVAL
      severity: WARNING
    - id: scala.lang.security.audit.documentbuilder-dtd-enabled.documentbuilder-dtd-enabled
      languages:
        - scala
      message: Document Builder being instantiated without calling the `setFeature` functions th
at are generally used for disabling entity processing. User controlled data in XML Document buil
der can result in XML Internal Entity Processing vulnerabilities like the disclosure of confiden
tial data, denial of service, Server Side Request Forgery (SSRF), port scanning. Make sure to di
sable entity processing functionality.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        source-rule-url: https://cheatsheetseries.owasp.org//cheatsheets/XML_External_Entity_Pre
vention_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - scala
      patterns:
        - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                $DF = DocumentBuilderFactory.newInstance(...)
                ...
                $DB = $DF.newDocumentBuilder(...)
            - patterns:
                - pattern: $DB = DocumentBuilderFactory.newInstance(...)
                - pattern-not-inside: |
                    ...
                    $X = $DB.newDocumentBuilder(...)
            - pattern: $DB = DocumentBuilderFactory.newInstance(...).newDocumentBuilder(...)
        - pattern-not-inside: |
            ...
            $DB.setXIncludeAware(true)
            ...
            $DB.setNamespaceAware(true)
            ...
            $DB.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
            ...
            $DB.setFeature("http://xml.org/sax/features/external-general-entities", false)
            ...
            $DB.setFeature("http://xml.org/sax/features/external-parameter-entities", false)
        - pattern-not-inside: |
            ...
            $DB.setXIncludeAware(true)
            ...
            $DB.setNamespaceAware(true)
            ...
            $DB.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
            ...
            $DB.setFeature("http://xml.org/sax/features/external-parameter-entities", false)
            ...
            $DB.setFeature("http://xml.org/sax/features/external-general-entities", false)
        - pattern-not-inside: |
            ...
            $DB.setXIncludeAware(true)
            ...
            $DB.setNamespaceAware(true)
            ...
            $DB.setFeature("http://xml.org/sax/features/external-general-entities", false)
            ...
            $DB.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
            ...
            $DB.setFeature("http://xml.org/sax/features/external-parameter-entities", false)
        - pattern-not-inside: |
            ...
            $DB.setXIncludeAware(true)
            ...
            $DB.setNamespaceAware(true)
            ...
            $DB.setFeature("http://xml.org/sax/features/external-general-entities", false)
            ...
            $DB.setFeature("http://xml.org/sax/features/external-parameter-entities", false)
            ...
            $DB.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
      severity: WARNING
    - id: scala.lang.security.audit.io-source-ssrf.io-source-ssrf
      languages:
        - scala
      message: A parameter being passed directly into `fromURL` most likely lead to SSRF. This c
ould allow an attacker to send data to their own server, potentially exposing sensitive data sen
t with this request. They could also probe internal servers or other resources that the server r
unning this code can access. Do not allow arbitrary hosts. Instead, create an allowlist for appr
oved hosts, or hardcode the correct host.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: LOW
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevent
ion_Cheat_Sheet.html
            - https://www.scala-lang.org/api/current/scala/io/Source$.html#fromURL(url:java.net.
URL)(implicitcodec:scala.io.Codec):scala.io.BufferedSource
        subcategory:
            - audit
        technology:
            - scala
      patterns:
        - pattern-either:
            - pattern: Source.fromURL($URL,...)
            - pattern: Source.fromURI($URL,...)
        - pattern-inside: |
            import scala.io.$SOURCE
            ...
        - pattern-either:
            - pattern-inside: |
                def $FUNC(..., $URL: $T, ...) = $A {
                  ...
                }
            - pattern-inside: |
                def $FUNC(..., $URL: $T, ...) = {
                  ...
                }
      severity: WARNING
    - id: scala.lang.security.audit.rsa-padding-set.rsa-padding-set
      languages:
        - scala
      message: Usage of RSA without OAEP (Optimal Asymmetric Encryption Padding) may weaken encr
yption. This could lead to sensitive data exposure. Instead, use RSA with `OAEPWithMD5AndMGF1Pad
ding` instead.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-780: Use of RSA Algorithm without OAEP'
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        resources:
            - https://blog.codacy.com/9-scala-security-issues/
        subcategory:
            - audit
        technology:
            - scala
            - cryptography
      patterns:
        - pattern: |
            $VAR = $CIPHER.getInstance($MODE)
        - metavariable-regex:
            metavariable: $MODE
            regex: .*RSA/.*/NoPadding.*
      severity: WARNING
    - id: scala.lang.security.audit.sax-dtd-enabled.sax-dtd-enabled
      languages:
        - scala
      message: XML processor being instantiated without calling the `setFeature` functions that
are generally used for disabling entity processing. User controlled data in XML Parsers can resu
lt in XML Internal Entity Processing vulnerabilities like the disclosure of confidential data, d
enial of service, Server Side Request Forgery (SSRF), port scanning. Make sure to disable entity
 processing functionality.
      metadata:
        category: security
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        source-rule-url: https://cheatsheetseries.owasp.org//cheatsheets/XML_External_Entity_Pre
vention_Cheat_Sheet.html
        subcategory:
            - audit
        technology:
            - scala
      patterns:
        - pattern-either:
            - pattern: $SR = new SAXReader(...)
            - pattern: |
                $SF = SAXParserFactory.newInstance(...)
                ...
                $SR = $SF.newSAXParser(...)
            - patterns:
                - pattern: $SR = SAXParserFactory.newInstance(...)
                - pattern-not-inside: |
                    ...
                    $X = $SR.newSAXParser(...)
            - pattern: $SR = SAXParserFactory.newInstance(...).newSAXParser(...)
            - pattern: $SR = new SAXBuilder(...)
        - pattern-not-inside: |
            ...
            $SR.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
            ...
            $SR.setFeature("http://xml.org/sax/features/external-general-entities", false)
            ...
            $SR.setFeature("http://xml.org/sax/features/external-parameter-entities", false)
        - pattern-not-inside: |
            ...
            $SR.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
            ...
            $SR.setFeature("http://xml.org/sax/features/external-parameter-entities", false)
            ...
            $SR.setFeature("http://xml.org/sax/features/external-general-entities", false)
        - pattern-not-inside: |
            ...
            $SR.setFeature("http://xml.org/sax/features/external-general-entities", false)
            ...
            $SR.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
            ...
            $SR.setFeature("http://xml.org/sax/features/external-parameter-entities", false)
        - pattern-not-inside: |
            ...
            $SR.setFeature("http://xml.org/sax/features/external-general-entities", false)
            ...
            $SR.setFeature("http://xml.org/sax/features/external-parameter-entities", false)
            ...
            $SR.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true)
      severity: WARNING
    - id: scala.lang.security.audit.scalac-debug.scalac-debug
      languages:
        - generic
      message: Scala applications built with `debug` set to true in production may leak debug in
formation to attackers. Debug mode also affects performance and reliability. Remove it from conf
iguration.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-489: Active Debug Code'
        impact: LOW
        likelihood: LOW
        owasp: A05:2021 - Security Misconfiguration
        references:
            - https://docs.scala-lang.org/overviews/compiler-options/index.html
        subcategory:
            - audit
        technology:
            - scala
            - sbt
      paths:
        include:
            - '*.sbt*'
      patterns:
        - pattern-either:
            - pattern: scalacOptions ... "-Vdebug"
            - pattern: scalacOptions ... "-Ydebug"
      severity: WARNING
    - id: scala.lang.security.audit.tainted-sql-string.tainted-sql-string
      languages:
        - scala
      message: User data flows into this manually-constructed SQL string. User data can be safel
y inserted into SQL strings using prepared statements or an object-relational mapper (ORM). Manu
ally-constructed SQL strings is a possible indicator of SQL injection, which could let an attack
er steal or manipulate data from the database. Instead, use prepared statements (`connection.Pre
paredStatement`) or a safe library.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.oracle.com/javase/7/docs/api/java/sql/PreparedStatement.html
        subcategory:
            - vuln
        technology:
            - scala
      mode: taint
      pattern-sanitizers:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: $LOGGER.$METHOD(...)
                    - pattern: $LOGGER(...)
                - metavariable-regex:
                    metavariable: $LOGGER
                    regex: (i?)log.*
            - patterns:
                - pattern: $LOGGER.$METHOD(...)
                - metavariable-regex:
                    metavariable: $METHOD
                    regex: (i?)(trace|info|warn|warning|warnToError|error|debug)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern-either:
                        - pattern: |
                            "$SQLSTR" + ...
                        - pattern: |
                            "$SQLSTR".format(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - patterns:
                            - pattern-inside: |
                                $SB = new StringBuilder("$SQLSTR");
                                ...
                            - pattern: $SB.append(...)
                        - patterns:
                            - pattern-inside: |
                                $VAR = "$SQLSTR"
                                ...
                            - pattern: $VAR += ...
                    - metavariable-regex:
                        metavariable: $SQLSTR
                        regex: (?i)(select|delete|insert|create|update|alter|drop)\b
                - patterns:
                    - pattern-either:
                        - pattern: s"..."
                        - pattern: f"..."
                    - pattern-regex: |
                        .*\b(?i)(select|delete|insert|create|update|alter|drop)\b.*
            - pattern-not-inside: println(...)
      pattern-sources:
        - patterns:
            - pattern: $PARAM
            - pattern-either:
                - pattern-inside: |
                    def $CTRL(..., $PARAM: $TYPE, ...) = {
                      ...
                    }
                - pattern-inside: |
                    def $CTRL(..., $PARAM: $TYPE, ...) = $A {
                      ...
                    }
                - pattern-inside: |
                    def $CTRL(..., $PARAM: $TYPE, ...) = $A(...) {
                      ...
                    }
      severity: ERROR
    - id: scala.lang.security.audit.xmlinputfactory-dtd-enabled.xmlinputfactory-dtd-enabled
      languages:
        - scala
      message: XMLInputFactory being instantiated without calling the setProperty functions that
 are generally used for disabling entity processing. User controlled data in XML Document builde
r can result in XML Internal Entity Processing vulnerabilities like the disclosure of confidenti
al data, denial of service, Server Side Request Forgery (SSRF), port scanning. Make sure to disa
ble entity processing functionality.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-611: Improper Restriction of XML External Entity Reference'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A04:2017 - XML External Entities (XXE)
            - A05:2021 - Security Misconfiguration
        references:
            - https://owasp.org/Top10/A05_2021-Security_Misconfiguration
        source-rule-url: https://cheatsheetseries.owasp.org//cheatsheets/XML_External_Entity_Pre
vention_Cheat_Sheet.html
        subcategory:
            - audit
        technology:
            - scala
      patterns:
        - pattern-not-inside: |
            ...
            $XMLFACTORY.setProperty("javax.xml.stream.isSupportingExternalEntities", false)
        - pattern-either:
            - pattern: $XMLFACTORY = XMLInputFactory.newFactory(...)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: $XMLFACTORY = XMLInputFactory.newInstance(...)
            - pattern: $XMLFACTORY = new XMLInputFactory(...)
      severity: WARNING
    - id: scala.play.security.conf-csrf-headers-bypass.conf-csrf-headers-bypass
      languages:
        - generic
      message: Possibly bypassable CSRF configuration found. CSRF is an attack that forces an en
d user to execute unwanted actions on a web application in which they’re currently authenticated
. Make sure that Content-Type black list is configured and CORS filter is turned on.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-352: Cross-Site Request Forgery (CSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: LOW
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://www.playframework.com/documentation/2.8.x/Migration25#CSRF-changes
            - https://owasp.org/www-community/attacks/csrf
        subcategory:
            - vuln
        technology:
            - scala
            - play
      paths:
        include:
            - '*.conf'
      patterns:
        - pattern-either:
            - pattern: X-Requested-With = "*"
            - pattern: Csrf-Token = "..."
        - pattern-inside: |
            bypassHeaders {...
            ...
            ...}
        - pattern-not-inside: |
            {...
              ...
              ...blackList = [..."application/x-www-form-urlencoded"..."multipart/form-data"..."
text/plain"...]
              ...
            ...}
        - pattern-not-inside: |
            {...
              ...
              ...blackList = [..."application/x-www-form-urlencoded"..."text/plain"..."multipart
/form-data"...]
              ...
            ...}
        - pattern-not-inside: |
            {...
              ...
              ...blackList = [..."multipart/form-data"..."application/x-www-form-urlencoded"..."
text/plain"...]
              ...
            ...}
        - pattern-not-inside: |
            {...
              ...
              ...blackList = [..."multipart/form-data"..."text/plain"..."application/x-www-form-
urlencoded"...]
              ...
            ...}
        - pattern-not-inside: |
            {...
              ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              ...blackList = [..."text/plain"..."application/x-www-form-urlencoded"..."multipart
/form-data"...]
              ...
            ...}
        - pattern-not-inside: |
            {...
              ...
              ...blackList = [..."text/plain"..."multipart/form-data"..."application/x-www-form-
urlencoded"...]
              ...
            ...}
      severity: ERROR
    - id: scala.play.security.conf-insecure-cookie-settings.conf-insecure-cookie-settings
      languages:
        - generic
      message: Session cookie `Secure` flag is explicitly disabled. The `secure` flag for cookie
s prevents the client from transmitting the cookie over insecure channels such as HTTP. Set the
`Secure` flag by setting `secure` to `true` in configuration file.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-614: Sensitive Cookie in HTTPS Session Without ''Secure'' Attribute'
        impact: LOW
        likelihood: LOW
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies#security
            - https://www.playframework.com/documentation/2.8.x/SettingsSession#Session-Configur
ation
        subcategory:
            - vuln
        technology:
            - play
            - scala
      paths:
        include:
            - '*.conf'
      patterns:
        - pattern: secure = false
        - pattern-inside: |
            session = {
              ...
            }
      severity: WARNING
    - id: scala.play.security.tainted-html-response.tainted-html-response
      languages:
        - scala
      message: Detected a request with potential user-input going into an `Ok()` response. This
bypasses any view or template environments, including HTML escaping, which may expose this appli
cation to cross-site scripting (XSS) vulnerabilities. Consider using a view technology such as T
wirl which automatically escapes HTML views.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        subcategory:
            - vuln
        technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - scala
            - play
      mode: taint
      pattern-sanitizers:
        - pattern-either:
            - pattern: org.apache.commons.lang3.StringEscapeUtils.escapeHtml4(...)
            - pattern: org.owasp.encoder.Encode.forHtml(...)
      pattern-sinks:
        - pattern-either:
            - pattern: Html.apply(...)
            - pattern: Ok(...).as(HTML)
            - pattern: Ok(...).as(ContentTypes.HTML)
            - patterns:
                - pattern: Ok(...).as($CTYPE)
                - metavariable-regex:
                    metavariable: $CTYPE
                    regex: '"[tT][eE][xX][tT]/[hH][tT][mM][lL]"'
            - patterns:
                - pattern: Ok(...).as($CTYPE)
                - pattern-not: Ok(...).as("...")
                - pattern-either:
                    - pattern-inside: |
                        def $FUNC(..., $URL: $T, ...) = $A {
                          ...
                        }
                    - pattern-inside: |
                        def $FUNC(..., $URL: $T, ...) = {
                          ...
                        }
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: $REQ
                    - pattern-either:
                        - pattern-inside: "Action {\n  $REQ: Request[$T] => \n    ...\n}\n"
                        - pattern-inside: "Action(...) {\n  $REQ: Request[$T] => \n    ...\n}\n"
                        - pattern-inside: "Action.async {\n  $REQ: Request[$T] => \n    ...\n}\n
"
                        - pattern-inside: "Action.async(...) {\n  $REQ: Request[$T] => \n    ...
\n}\n"
                - patterns:
                    - pattern: $PARAM
                    - pattern-either:
                        - pattern-inside: |
                            def $CTRL(..., $PARAM: $TYPE, ...) = Action {
                              ...
                            }
                        - pattern-inside: |
                            def $CTRL(..., $PARAM: $TYPE, ...) = Action(...) {
                              ...
                            }
                        - pattern-inside: |
                            def $CTRL(..., $PARAM: $TYPE, ...) = Action.async {
                              ...
                            }
                        - pattern-inside: |
                            def $CTRL(..., $PARAM: $TYPE, ...) = Action.async(...) {
                              ...
                            }
      severity: WARNING
    - id: scala.play.security.tainted-slick-sqli.tainted-slick-sqli
      languages:
        - scala
      message: Detected a tainted SQL statement. This could lead to SQL injection if variables i
n the SQL statement are not properly sanitized. Avoid using using user input for generating SQL
strings.
      metadata:
        category: security
        confidence: HIGH
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://scala-slick.org/doc/3.3.3/sql.html#splicing-literal-values
            - https://scala-slick.org/doc/3.2.0/sql-to-slick.html#non-optimal-sql-code
        subcategory:
            - vuln
        technology:
            - scala
            - slick
            - play
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: $MODEL.overrideSql(...)
                - pattern: sql"..."
            - pattern-inside: |
                import slick.$DEPS
                ...
      pattern-sources:
        - patterns:
            - pattern-either:
                - patterns:
                    - pattern: $REQ
                    - pattern-either:
                        - pattern-inside: "Action {\n  $REQ: Request[$T] => \n    ...\n}\n"
                        - pattern-inside: "Action(...) {\n  $REQ: Request[$T] => \n    ...\n}\n"
                        - pattern-inside: "Action.async {\n  $REQ: Request[$T] => \n    ...\n}\n
"
                        - pattern-inside: "Action.async(...) {\n  $REQ: Request[$T] => \n    ...
\n}\n"
                - patterns:
                    - pattern: $PARAM
                    - pattern-either:
                        - pattern-inside: |
                            def $CTRL(..., $PARAM: $TYPE, ...) = Action {
                              ...
                            }
                        - pattern-inside: |
                            def $CTRL(..., $PARAM: $TYPE, ...) = Action(...) {
                              ...
                            }
                        - pattern-inside: |
                            def $CTRL(..., $PARAM: $TYPE, ...) = Action.async {
                              ...
                            }
                        - pattern-inside: |
                            def $CTRL(..., $PARAM: $TYPE, ...) = Action.async(...) {
                              ...
                            }
      severity: ERROR
    - id: scala.play.security.tainted-sql-from-http-request.tainted-sql-from-http-request
      languages:
        - scala
      message: User data flows into this manually-constructed SQL string. User data can be safel
y inserted into SQL strings using prepared statements or an object-relational mapper (ORM). Manu
ally-constructed SQL strings is a possible indicator of SQL injection, which could let an attack
er steal or manipulate data from the database. Instead, use prepared statements (`connection.Pre
paredStatement`) or a safe library.
      metadata:
        category: security
        confidence: HIGH
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - 'CWE-89: Improper Neutralization of Special Elements used in an SQL Command (''SQL
 Injection'')'
       cwe2021-top25: true
       cwe2022-top25: true
       impact: MEDIUM
       likelihood: HIGH
       owasp:
           - A01:2017 - Injection
           - A03:2021 - Injection
       references:
           - https://docs.oracle.com/javase/7/docs/api/java/sql/PreparedStatement.html
       subcategory:
           - vuln
       technology:
           - scala
           - play
     mode: taint
     pattern-sinks:
       - patterns:
           - pattern-either:
               - patterns:
                   - pattern-either:
                       - pattern: |
                           "$SQLSTR" + ...
                       - pattern: |
                           "$SQLSTR".format(...)
                       - patterns:
                           - pattern-inside: |
                               $SB = new StringBuilder("$SQLSTR");
                               ...
                           - pattern: $SB.append(...)
                       - patterns:
                           - pattern-inside: |
                               $VAR = "$SQLSTR"
                               ...
                           - pattern: $VAR += ...
                   - metavariable-regex:
                       metavariable: $SQLSTR
                       regex: (?i)(select|delete|insert|create|update|alter|drop)\b
               - patterns:
                   - pattern: s"..."
                   - pattern-regex: |
                       .*\b(?i)(select|delete|insert|create|update|alter|drop)\b.*
           - pattern-not-inside: println(...)
     pattern-sources:
       - patterns:
           - pattern-either:
               - patterns:
                   - pattern: $REQ
                   - pattern-either:
                       - pattern-inside: "Action {\n  $REQ: Request[$T] => \n    ...\n}\n"
                       - pattern-inside: "Action(...) {\n  $REQ: Request[$T] => \n    ...\n}\n"
                       - pattern-inside: "Action.async {\n  $REQ: Request[$T] => \n    ...\n}\n
"
                       - pattern-inside: "Action.async(...) {\n  $REQ: Request[$T] => \n    ...
\n}\n"
               - patterns:
                   - pattern: $PARAM
                   - pattern-either:
                       - pattern-inside: |
                           def $CTRL(..., $PARAM: $TYPE, ...) = Action {
                             ...
                           }
                       - pattern-inside: |
                           def $CTRL(..., $PARAM: $TYPE, ...) = Action(...) {
                             ...
                           }
                       - pattern-inside: |
                           def $CTRL(..., $PARAM: $TYPE, ...) = Action.async {
                             ...
                           }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        - pattern-inside: |
                            def $CTRL(..., $PARAM: $TYPE, ...) = Action.async(...) {
                              ...
                            }
      severity: ERROR
    - id: scala.scala-jwt.security.jwt-hardcode.scala-jwt-hardcoded-secret
      languages:
        - scala
      message: 'Hardcoded JWT secret or private key is used. This is a Insufficiently Protected
Credentials weakness: https://cwe.mitre.org/data/definitions/522.html Consider using an appropri
ate security mechanism to protect the credentials (e.g. keeping secrets in environment variables
)'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory:
            - audit
        technology:
            - jwt
      pattern-either:
        - pattern: |
            com.auth0.jwt.algorithms.Algorithm.HMAC256("...");
        - pattern: |
            $SECRET = "...";
            ...
            com.auth0.jwt.algorithms.Algorithm.HMAC256($SECRET);
        - pattern: |
            class $CLASS {
              ...
              $DECL $SECRET = "...";
              ...
              def $FUNC (...): $RETURNTYPE = {
                ...
                com.auth0.jwt.algorithms.Algorithm.HMAC256($SECRET);
                ...
              }
              ...
            }
        - pattern: |
            com.auth0.jwt.algorithms.Algorithm.HMAC384("...");
        - pattern: |
            $SECRET = "...";
            ...
            com.auth0.jwt.algorithms.Algorithm.HMAC384($SECRET);
        - pattern: |
            class $CLASS {
              ...
              $DECL $SECRET = "...";
              ...
              def $FUNC (...): $RETURNTYPE = {
                ...
                com.auth0.jwt.algorithms.Algorithm.HMAC384($SECRET);
                ...
              }
              ...
            }
        - pattern: |
            com.auth0.jwt.algorithms.Algorithm.HMAC512("...");
        - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            $SECRET = "...";
            ...
            com.auth0.jwt.algorithms.Algorithm.HMAC512($SECRET);
        - pattern: |
            class $CLASS {
              ...
              $DECL $SECRET = "...";
              ...
              def $FUNC (...): $RETURNTYPE = {
                ...
                com.auth0.jwt.algorithms.Algorithm.HMAC512($SECRET);
                ...
              }
              ...
            }
      severity: ERROR
    - id: swift.lang.storage.sensitive-storage-userdefaults.swift-user-defaults
      languages:
        - swift
      message: Potentially sensitive data was observed to be stored in UserDefaults, which is no
t adequate protection of sensitive information. For data of a sensitive nature, applications sho
uld leverage the Keychain.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-311: Missing Encryption of Sensitive Data'
        impact: HIGH
        likelihood: LOW
        masvs:
            - 'MASVS-STORAGE-1: The app securely stores sensitive data'
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A04:2021 - Insecure Design
        references:
            - https://developer.apple.com/library/archive/documentation/Security/Conceptual/Secu
reCodingGuide/Articles/ValidatingInput.html
            - https://mas.owasp.org/MASVS/controls/MASVS-STORAGE-1/
        subcategory:
            - vuln
        technology:
            - ios
            - macos
      options:
        symbolic_propagation: true
      patterns:
        - pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: |
                        UserDefaults.standard.set("$VALUE", forKey: "$KEY")
                    - pattern: |
                        UserDefaults.standard.set("$VALUE", forKey: $KEY)
                    - pattern: |
                        UserDefaults.standard.set($VALUE, forKey: "$KEY")
                    - pattern: |
                        UserDefaults.standard.set($VALUE, forKey: $KEY)
                - metavariable-regex:
                    metavariable: $VALUE
                    regex: (?i).*(passcode|password|pass_word|passphrase|pass_code|pass_word|pas
s_phrase)$
                - focus-metavariable: $VALUE
            - patterns:
                - pattern-either:
                    - pattern: |
                        UserDefaults.standard.set("$VALUE", forKey: "$KEY")
                    - pattern: |
                        UserDefaults.standard.set("$VALUE", forKey: $KEY)
                    - pattern: |
                        UserDefaults.standard.set($VALUE, forKey: "$KEY")
                    - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    UserDefaults.standard.set($VALUE, forKey: $KEY)
            - metavariable-regex:
                metavariable: $KEY
                regex: (?i).*(passcode|password|pass_word|passphrase|pass_code|pass_word|pas
s_phrase)$
            - focus-metavariable: $KEY
        - patterns:
            - pattern-either:
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: $KEY)
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: $KEY)
            - metavariable-regex:
                metavariable: $VALUE
                regex: (?i).*(api_key|apikey)$
            - focus-metavariable: $VALUE
        - patterns:
            - pattern-either:
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: $KEY)
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: $KEY)
            - metavariable-regex:
                metavariable: $KEY
                regex: (?i).*(api_key|apikey)$
            - focus-metavariable: $KEY
        - patterns:
            - pattern-either:
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: $KEY)
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: $KEY)
            - metavariable-regex:
                metavariable: $VALUE
                regex: (?i).*(secretkey|secret_key|secrettoken|secret_token|clientsecret|cli
ent_secret)$
            - focus-metavariable: $VALUE
        - patterns:
            - pattern-either:
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: $KEY)
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: $KEY)
            - metavariable-regex:
                metavariable: $KEY
                regex: (?i).*(secretkey|secret_key|secrettoken|secret_token|clientsecret|cli
ent_secret)$
            - focus-metavariable: $KEY
        - patterns:
            - pattern-either:
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: $KEY)
                - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    UserDefaults.standard.set($VALUE, forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: $KEY)
            - metavariable-regex:
                metavariable: $VALUE
                regex: (?i).*(cryptkey|cryptokey|crypto_key|cryptionkey|symmetrickey|private
key|symmetric_key|private_key)$
            - focus-metavariable: $VALUE
        - patterns:
            - pattern-either:
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set("$VALUE", forKey: $KEY)
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: "$KEY")
                - pattern: |
                    UserDefaults.standard.set($VALUE, forKey: $KEY)
            - metavariable-regex:
                metavariable: $KEY
                regex: (?i).*(cryptkey|cryptokey|crypto_key|cryptionkey|symmetrickey|private
key|symmetric_key|private_key)$
            - focus-metavariable: $KEY
  severity: WARNING
- id: swift.webview.webview-js-window.swift-webview-config-allows-js-open-windows
  languages:
    - swift
  message: Webviews were observed that explictly allow JavaScript in an WKWebview to open wi
ndows automatically. Consider disabling this functionality if not required, following the princi
ple of least privelege.
  metadata:
    category: security
    confidence: HIGH
    cwe:
        - 'CWE-272: Least Privilege Violation'
    impact: LOW
    likelihood: LOW
    masvs:
        - 'MASVS-PLATFORM-2: The app uses WebViews securely'
    references:
        - https://mas.owasp.org/MASVS/controls/MASVS-PLATFORM-2/
        - https://developer.apple.com/documentation/webkit/wkpreferences/1536573-javascriptc
anopenwindowsautomati
    subcategory:
        - audit
    technology:
        - ios
        - macos
  patterns:
    - pattern: |
        $P = WKPreferences()
        ...
    - pattern-either:
        - patterns:
            - pattern-inside: |
                $P.JavaScriptCanOpenWindowsAutomatically  = $FALSE
                ...
                $P.JavaScriptCanOpenWindowsAutomatically  = $TRUE
            - pattern-not-inside: |
                ...
                $P.JavaScriptCanOpenWindowsAutomatically  = $TRUE
                ...
                $P.JavaScriptCanOpenWindowsAutomatically = $FALSE
            - pattern: |
                $P.JavaScriptCanOpenWindowsAutomatically  = true
            - metavariable-regex:
                metavariable: $TRUE
                regex: ^(true)$
            - metavariable-regex:
                metavariable: $TRUE
                regex: (.*(?!true))
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - patterns:
                - pattern: |
                    $P.JavaScriptCanOpenWindowsAutomatically  = true
                - pattern-not-inside: |
                    ...
                    $P.JavaScriptCanOpenWindowsAutomatically  = ...
                    ...
                    $P.JavaScriptCanOpenWindowsAutomatically  = ...
      severity: WARNING
    - id: terraform.aws.correctness.subscription-filter-missing-depends.subscription-filter-miss
ing-depends
      languages:
        - hcl
      message: The `aws_cloudwatch_log_subscription_filter` resource "$NAME" needs a `depends_on
` clause on the `aws_lambda_permission`, otherwise Terraform may try to create these out-of-orde
r and fail.
      metadata:
        category: correctness
        confidence: MEDIUM
        references:
            - https://stackoverflow.com/questions/38407660/terraform-configuring-cloudwatch-log-
subscription-delivery-to-lambda/38428834#38428834
        technology:
            - aws
            - terraform
            - aws-lambda
            - cloudwatch
      patterns:
        - pattern: |
            resource "aws_cloudwatch_log_subscription_filter" $NAME {
              ...
              destination_arn = aws_lambda_function.$LAMBDA_NAME.arn
            }
        - pattern-not-inside: |
            resource "aws_cloudwatch_log_subscription_filter" $NAME {
              ...
              depends_on = [..., aws_lambda_permission.$PERMISSION_NAME, ...]
            }
      severity: WARNING
    - id: terraform.aws.security.aws-cloudfront-insecure-tls.aws-insecure-cloudfront-distributio
n-tls-version
      languages:
        - hcl
      message: Detected an AWS CloudFront Distribution with an insecure TLS version. TLS version
s less than 1.2 are considered insecure because they can be broken. To fix this, set your `minim
um_protocol_version` to `"TLSv1.2_2018", "TLSv1.2_2019" or "TLSv1.2_2021"`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      patterns:
        - pattern: |
            resource "aws_cloudfront_distribution" $ANYTHING {
              ...
              viewer_certificate {
                ...
              }
              ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            }
        - pattern-not-inside: |
            resource "aws_cloudfront_distribution" $ANYTHING {
              ...
              viewer_certificate {
                ...
                minimum_protocol_version = "TLSv1.2_2018"
                ...
              }
              ...
            }
        - pattern-not-inside: |
            resource "aws_cloudfront_distribution" $ANYTHING {
              ...
              viewer_certificate {
                ...
                minimum_protocol_version = "TLSv1.2_2019"
                ...
              }
              ...
            }
        - pattern-not-inside: |
            resource "aws_cloudfront_distribution" $ANYTHING {
              ...
              viewer_certificate {
                ...
                minimum_protocol_version = "TLSv1.2_2021"
                ...
              }
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-cloudwatch-log-group-no-retention.aws-cloudwatch-log-group-
no-retention
      languages:
        - hcl
      message: The AWS CloudWatch Log Group has no retention. Missing retention in log groups ca
n cause losing important event information.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-320: CWE CATEGORY: Key Management Errors'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern: |
            resource "aws_cloudwatch_log_group" $ANYTHING {
              ...
            }
        - pattern-not-inside: |
            resource "aws_cloudwatch_log_group" $ANYTHING {
              ...
              retention_in_days = ...
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-codebuild-project-unencrypted.aws-codebuild-project-unencry
pted
      languages:
        - hcl
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: The AWS CodeBuild Project is unencrypted. The AWS KMS encryption key protects pro
jects in the CodeBuild. To create your own, create a aws_kms_key resource or use the ARN string
of a key in your account.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-320: CWE CATEGORY: Key Management Errors'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern: |
            resource "aws_codebuild_project" $ANYTHING {
              ...
            }
        - pattern-not-inside: |
            resource "aws_codebuild_project" $ANYTHING {
              ...
              encryption_key = ...
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-config-aggregator-not-all-regions.aws-config-aggregator-not
-all-regions
      languages:
        - hcl
      message: The AWS configuration aggregator does not aggregate all AWS Config region. This m
ay result in unmonitored configuration in regions that are thought to be unused. Configure the a
ggregator with all_regions for the source.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-778: Insufficient Logging'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A09:2021 - Security Logging and Monitoring Failures
        references:
            - https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/
        subcategory:
            - audit
        technology:
            - terraform
            - aws
      pattern-either:
        - pattern: |
            resource "aws_config_configuration_aggregator" $ANYTHING {
              ...
              account_aggregation_source {
                ...
                regions = ...
                ...
              }
              ...
            }
        - pattern: |
            resource "aws_config_configuration_aggregator" $ANYTHING {
              ...
              organization_aggregation_source {
                ...
                regions = ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              ...
            }
            ...
          }
      severity: WARNING
    - id: terraform.aws.security.aws-db-instance-no-logging.aws-db-instance-no-logging
      languages:
        - hcl
      message: Database instance has no logging. Missing logs can cause missing important event
information.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-311: Missing Encryption of Sensitive Data'
        impact: LOW
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern: |
            resource "aws_db_instance" $ANYTHING {
              ...
            }
        - pattern-not-inside: |
            resource "aws_db_instance" $ANYTHING {
              ...
              enabled_cloudwatch_logs_exports = [$SOMETHING, ...]
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-documentdb-auditing-disabled.aws-documentdb-auditing-disabl
ed
      languages:
        - hcl
      message: Auditing is not enabled for DocumentDB. To ensure that you are able to accurately
 audit the usage of your DocumentDB cluster, you should enable auditing and export logs to Cloud
Watch.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-778: Insufficient Logging'
        impact: LOW
        likelihood: LOW
        owasp:
            - A09:2021 - Security Logging and Monitoring Failures
        references:
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/docdb_
cluster#enabled_cloudwatch_logs_exports
            - https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/
        subcategory:
            - audit
        technology:
            - terraform
            - aws
      patterns:
        - pattern: |
            resource "aws_docdb_cluster" $ANYTHING {
              ...
            }
        - pattern-not-inside: |
            resource "aws_docdb_cluster" $ANYTHING {
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              ...
              enabled_cloudwatch_logs_exports = [..., "audit", ...]
              ...
            }
      severity: INFO
    - id: terraform.aws.security.aws-dynamodb-table-unencrypted.aws-dynamodb-table-unencrypted
      languages:
        - hcl
      message: By default, AWS DynamoDB Table is encrypted using AWS-managed keys. However, for
added security, it's recommended to configure your own AWS KMS encryption key to protect your da
ta in the DynamoDB table. You can either create a new aws_kms_key resource or use the ARN of an
existing key in your AWS account to do so.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern: |
            resource "aws_dynamodb_table" $ANYTHING {
              ...
            }
        - pattern-not-inside: |
            resource "aws_dynamodb_table" $ANYTHING {
              ...
              server_side_encryption {
                enabled = true
                kms_key_arn = ...
              }
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-ebs-snapshot-encrypted-with-cmk.aws-ebs-snapshot-encrypted-
with-cmk
      languages:
        - hcl
      message: Ensure EBS Snapshot is encrypted at rest using KMS CMKs. CMKs gives you control o
ver the encryption key in terms of access and rotation.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-320: CWE CATEGORY: Key Management Errors'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      patterns:
        - pattern: |
            resource "aws_ebs_snapshot_copy" $ANYTHING {
              ...
              encrypted = true
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
          }
      - pattern-not-inside: |
          resource "aws_ebs_snapshot_copy" $ANYTHING {
            ...
            encrypted = true
            kms_key_id = ...
            ...
          }
    severity: WARNING
  - id: terraform.aws.security.aws-ebs-unencrypted.aws-ebs-unencrypted
    languages:
      - hcl
    message: The AWS EBS is unencrypted. The AWS EBS encryption protects data in the EBS.
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
          - 'CWE-320: CWE CATEGORY: Key Management Errors'
      impact: MEDIUM
      likelihood: MEDIUM
      owasp:
          - A03:2017 - Sensitive Data Exposure
      references:
          - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
      subcategory:
          - vuln
      technology:
          - aws
          - terraform
    patterns:
      - pattern: |
          resource "aws_ebs_encryption_by_default" $ANYTHING {
            ...
            enabled = false
            ...
          }
    severity: WARNING
  - id: terraform.aws.security.aws-ebs-volume-unencrypted.aws-ebs-volume-unencrypted
    languages:
      - hcl
    message: The AWS EBS volume is unencrypted. The volume, the disk I/O and any derived snaps
hots could be read if compromised. Volumes should be encrypted to ensure sensitive data is store
d securely.
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
          - 'CWE-311: Missing Encryption of Sensitive Data'
      impact: HIGH
      likelihood: MEDIUM
      owasp:
          - A03:2017 - Sensitive Data Exposure
          - A04:2021 - Insecure Design
      references:
          - https://owasp.org/Top10/A04_2021-Insecure_Design
          - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/ebs_vo
lume#encrypted
          - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSEncryption.html
      subcategory:
          - audit
      technology:
          - terraform
          - aws
    patterns:
      - pattern: |
          resource "aws_ebs_volume" $ANYTHING {
            ...
          }
      - pattern-not: |
          resource "aws_ebs_volume" $ANYTHING {
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
              ...
              encrypted = true
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-ec2-has-public-ip.aws-ec2-has-public-ip
      languages:
        - hcl
      message: EC2 instances should not have a public IP address attached in order to block publ
ic access to the instances. To fix this, set your `associate_public_ip_address` to `"false"`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-284: Improper Access Control'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      patterns:
        - pattern-either:
            - pattern: |
                resource "aws_instance" $ANYTHING {
                  ...
                  associate_public_ip_address = true
                  ...
                }
            - pattern: |
                resource "aws_launch_template" $ANYTHING {
                  ...
                  network_interfaces {
                    ...
                    associate_public_ip_address = true
                    ...
                  }
                  ...
                }
      severity: WARNING
    - id: terraform.aws.security.aws-ec2-launch-template-metadata-service-v1-enabled.aws-ec2-lau
nch-template-metadata-service-v1-enabled
      languages:
        - hcl
      message: The EC2 launch template has Instance Metadata Service Version 1 (IMDSv1) enabled.
 IMDSv2 introduced session authentication tokens which improve security when talking to IMDS. Yo
u should either disable IMDS or require the use of IMDSv2.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-1390: Weak Authentication'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/launch
_configuration#metadata_options
            - https://aws.amazon.com/blogs/security/defense-in-depth-open-firewalls-reverse-prox
ies-ssrf-vulnerabilities-ec2-instance-metadata-service
        subcategory:
            - audit
        technology:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - terraform
          - aws
      patterns:
        - pattern: |
            resource "aws_launch_template" $ANYTHING {
              ...
            }
        - pattern-not-inside: |
            resource "aws_launch_template" $ANYTHING {
              ...
              metadata_options {
                ...
                http_endpoint = "disabled"
                ...
              }
              ...
            }
        - pattern-not-inside: |
            resource "aws_launch_template" $ANYTHING {
              ...
              metadata_options {
                ...
                http_tokens = "required"
                ...
              }
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-ecr-mutable-image-tags.aws-ecr-mutable-image-tags
      languages:
        - hcl
      message: The ECR repository allows tag mutability. Image tags could be overwritten with co
mpromised images. ECR images should be set to IMMUTABLE to prevent code injection through image
mutation. This can be done by setting `image_tag_mutability` to IMMUTABLE.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-345: Insufficient Verification of Data Authenticity'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A08:2021 - Software and Data Integrity Failures
        references:
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/ecr_re
pository#image_tag_mutability
            - https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/
        subcategory:
            - audit
        technology:
            - terraform
            - aws
      patterns:
        - pattern: |
            resource "aws_ecr_repository" $ANYTHING {
              ...
            }
        - pattern-not-inside: |
            resource "aws_ecr_repository" $ANYTHING {
              ...
              image_tag_mutability = "IMMUTABLE"
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-ecr-repository-wildcard-principal.aws-ecr-repository-wildca
rd-principal
      languages:
        - hcl
      message: Detected wildcard access granted in your ECR repository policy principal. This gr
ants access to all users, including anonymous users (public access). Instead, limit principals,
actions and resources to what you need according to least privilege.
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        cwe2021-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/ecr_re
pository_policy
            - https://docs.aws.amazon.com/lambda/latest/operatorguide/wildcard-permissions-iam.h
tml
            - https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/monitor-amazon-e
cr-repositories-for-wildcard-permissions-using-aws-cloudformation-and-aws-config.html
            - https://cwe.mitre.org/data/definitions/732.html
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern-inside: |
            resource "aws_ecr_repository_policy" $ANYTHING {
              ...
            }
        - pattern-either:
            - patterns:
                - pattern: policy = "$JSONPOLICY"
                - metavariable-pattern:
                    language: json
                    metavariable: $JSONPOLICY
                    patterns:
                        - pattern-not-inside: |
                            {..., "Effect": "Deny", ...}
                        - pattern-either:
                            - pattern: |
                                {..., "Principal": "*", ...}
                            - pattern: |
                                {..., "Principal": [..., "*", ...], ...}
                            - pattern: |
                                {..., "Principal": { "AWS": "*" }, ...}
                            - pattern: |
                                {..., "Principal": { "AWS": [..., "*", ...] }, ...}
            - patterns:
                - pattern-inside: policy = jsonencode(...)
                - pattern-not-inside: |
                    {..., Effect = "Deny", ...}
                - pattern-either:
                    - pattern: |
                        {..., Principal = "*", ...}
                    - pattern: |
                        {..., Principal = [..., "*", ...], ...}
                    - pattern: |
                        {..., Principal = { AWS = "*" }, ...}
                    - pattern: |
                        {..., Principal = { AWS = [..., "*", ...] }, ...}
      severity: WARNING
    - id: terraform.aws.security.aws-efs-filesystem-encrypted-with-cmk.aws-efs-filesystem-encryp
ted-with-cmk
      languages:
        - hcl
      message: Ensure EFS filesystem is encrypted at rest using KMS CMKs. CMKs gives you control
 over the encryption key in terms of access and rotation.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-320: CWE CATEGORY: Key Management Errors'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - audit
        technology:
            - terraform
            - aws
      patterns:
        - pattern: |
            resource "aws_efs_file_system" $ANYTHING {
              ...
              encrypted  = true
              ...
            }
        - pattern-not-inside: |
            resource "aws_efs_file_system" $ANYTHING {
              ...
              encrypted  = true
              kms_key_id = ...
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-elasticsearch-insecure-tls-version.aws-elasticsearch-insecu
re-tls-version
      languages:
        - terraform
      message: Detected an AWS Elasticsearch domain using an insecure version of TLS. To fix thi
s, set "tls_security_policy" equal to "Policy-Min-TLS-1-2-2019-07".
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      pattern: |
        resource "aws_elasticsearch_domain" $ANYTHING {
          ...
          domain_endpoint_options {
            ...
            enforce_https = true
            tls_security_policy = "Policy-Min-TLS-1-0-2019-07"
            ...
          }
          ...
        }
      severity: WARNING
    - id: terraform.aws.security.aws-elasticsearch-nodetonode-encryption.aws-elasticsearch-nodet
onode-encryption-not-enabled
      languages:
        - hcl
      message: "Ensure all Elasticsearch has node-to-node encryption enabled.\t"
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      patterns:
        - pattern-either:
            - pattern: |
                resource "aws_elasticsearch_domain" $ANYTHING {
                  ...
                  node_to_node_encryption {
                    ...
                    enabled = false
                    ...
                  }
                  ...
                }
            - pattern: |
                resource "aws_elasticsearch_domain" $ANYTHING {
                  ...
                  cluster_config {
                    ...
                    instance_count = $COUNT
                    ...
                  }
                }
        - pattern-not-inside: |
            resource "aws_elasticsearch_domain" $ANYTHING {
              ...
              cluster_config {
                ...
                instance_count = $COUNT
                ...
              }
              node_to_node_encryption {
                ...
                enabled = true
                ...
              }
            }
        - metavariable-comparison:
            comparison: $COUNT > 1
            metavariable: $COUNT
      severity: WARNING
    - id: terraform.aws.security.aws-glacier-vault-any-principal.aws-glacier-vault-any-principal
      languages:
        - hcl
      message: 'Detected wildcard access granted to Glacier Vault. This means anyone within your
 AWS account ID can perform actions on Glacier resources. Instead, limit to a specific identity
in your account, like this: `arn:aws:iam::<account_id>:<identity>`.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://cwe.mitre.org/data/definitions/732.html
        subcategory:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - vuln
        technology:
          - aws
      patterns:
        - pattern-inside: |
            resource "aws_glacier_vault" $ANYTHING {
              ...
            }
        - pattern: access_policy = "$STATEMENT"
        - metavariable-pattern:
            language: json
            metavariable: $STATEMENT
            patterns:
              - pattern-inside: |
                  {..., "Effect": "Allow", ...}
              - pattern-either:
                  - pattern: |
                      "Principal": "*"
                  - pattern: |
                      "Principal": {..., "AWS": "*", ...}
                  - pattern-inside: |
                      "Principal": {..., "AWS": ..., ...}
                  - pattern-regex: |
                      (^\"arn:aws:iam::\*:(.*)\"$)
      severity: ERROR
    - id: terraform.aws.security.aws-iam-admin-policy-ssoadmin.aws-iam-admin-policy-ssoadmin
      languages:
        - hcl
      message: Detected admin access granted in your policy. This means anyone with this policy
can perform administrative actions. Instead, limit actions and resources to what you need accord
ing to least privilege.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
          - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
          - A05:2021 - Security Misconfiguration
        references:
          - https://cwe.mitre.org/data/definitions/732.html
        subcategory:
          - vuln
        technology:
          - aws
      patterns:
        - pattern-inside: |
            resource "aws_ssoadmin_permission_set_inline_policy" $ANYTHING {
              ...
            }
        - pattern: inline_policy = "$STATEMENT"
        - metavariable-pattern:
            language: json
            metavariable: $STATEMENT
            patterns:
              - pattern-not-inside: |
                  {..., "Effect": "Deny", ...}
              - pattern-either:
                  - pattern: |
                      {..., "Action": [..., "*", ...], "Resource": [..., "*", ...], ...}
                  - pattern: |
                      {..., "Action": "*", "Resource": "*", ...}
                  - pattern: |
                      {..., "Action": "*", "Resource": [...], ...}
                  - pattern: |
                      {..., "Action": [...], "Resource": "*", ...}
      severity: ERROR
    - id: terraform.aws.security.aws-iam-admin-policy.aws-iam-admin-policy
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - hcl
      message: Detected admin access granted in your policy. This means anyone with this policy
can perform administrative actions. Instead, limit actions and resources to what you need accord
ing to least privilege.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://cwe.mitre.org/data/definitions/732.html
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern-inside: |
            resource "aws_iam_policy" $ANYTHING {
              ...
            }
        - pattern: policy = "$STATEMENT"
        - metavariable-pattern:
            language: json
            metavariable: $STATEMENT
            patterns:
                - pattern-not-inside: |
                    {..., "Effect": "Deny", ...}
                - pattern-either:
                    - pattern: |
                        {..., "Action": [..., "*", ...], "Resource": [..., "*", ...], ...}
                    - pattern: |
                        {..., "Action": "*", "Resource": "*", ...}
                    - pattern: |
                        {..., "Action": "*", "Resource": [...], ...}
                    - pattern: |
                        {..., "Action": [...], "Resource": "*", ...}
      severity: ERROR
    - id: terraform.aws.security.aws-insecure-api-gateway-tls-version.aws-insecure-api-gateway-t
ls-version
      languages:
        - terraform
      message: Detected AWS API Gateway to be using an insecure version of TLS. To fix this issu
e make sure to set "security_policy" equal to "TLS_1_2".
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern-either:
            - pattern: |
                resource "aws_api_gateway_domain_name" $ANYTHING {
                    ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    security_policy = "..."
                    ...
                }
            - pattern: |
                resource "aws_apigatewayv2_domain_name" $ANYTHING {
                    ...
                    domain_name_configuration {...}
                    ...
                }
        - pattern-not: |
            resource "aws_api_gateway_domain_name" $ANYTHING {
                    ...
                    security_policy = "TLS_1_2"
                    ...
                }
        - pattern-not: |
            resource "aws_apigatewayv2_domain_name" $ANYTHING {
                    ...
                    domain_name_configuration {
                        ...
                        security_policy = "TLS_1_2"
                        ...
                    }
                }
      severity: WARNING
    - id: terraform.aws.security.aws-insecure-redshift-ssl-configuration.aws-insecure-redshift-s
sl-configuration
      languages:
        - hcl
      message: Detected an AWS Redshift configuration with a SSL disabled. To fix this, set your
 `require_ssl` to `"true"`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      patterns:
        - pattern: |
            resource "aws_redshift_parameter_group" $ANYTHING {
              ...
            }
        - pattern-not-inside: |
            resource "aws_redshift_parameter_group" $ANYTHING {
              ...
              parameter {
                name  = "require_ssl"
                value = "true"
              }
              ...
            }
        - pattern-not-inside: |
            resource "aws_redshift_parameter_group" $ANYTHING {
              ...
              parameter {
                name  = "require_ssl"
                value = true
              }
              ...
            }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: WARNING
    - id: terraform.aws.security.aws-kinesis-stream-unencrypted.aws-kinesis-stream-unencrypted
      languages:
        - hcl
      message: The AWS Kinesis stream does not encrypt data at rest. The data could be read if t
he Kinesis stream storage layer is compromised. Enable Kinesis stream server-side encryption.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-311: Missing Encryption of Sensitive Data'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A04:2021 - Insecure Design
        references:
            - https://owasp.org/Top10/A04_2021-Insecure_Design
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/kinesi
s_stream#encryption_type
            - https://docs.aws.amazon.com/streams/latest/dev/server-side-encryption.html
        rule-origin-note: published from /src/aws-kinesis-stream-unencrypted.yml in None
        subcategory:
            - audit
        technology:
            - terraform
            - aws
      patterns:
        - pattern: |
            resource "aws_kinesis_stream" $ANYTHING {
              ...
            }
        - pattern-not: |
            resource "aws_kinesis_stream" $ANYTHING {
              ...
              encryption_type = "KMS"
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.aws-kms-key-wildcard-principal.aws-kms-key-wildcard-principal
      languages:
        - hcl
      message: Detected wildcard access granted in your KMS key. This means anyone with this pol
icy can perform administrative actions over the keys. Instead, limit principals, actions and res
ources to what you need according to least privilege.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://cwe.mitre.org/data/definitions/732.html
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern-inside: |
            resource "aws_kms_key" $ANYTHING {
              ...
            }
        - pattern: policy = "$STATEMENT"
        - metavariable-pattern:
            language: json
            metavariable: $STATEMENT
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            patterns:
                - pattern-not-inside: |
                    {..., "Effect": "Deny", ...}
                - pattern-either:
                    - pattern: |
                        {..., "Principal": "*", "Action": "kms:*", "Resource": "*", ...}
                    - pattern: |
                        {..., "Principal": [..., "*", ...], "Action": "kms:*", "Resource": "*",
...}
                    - pattern: |
                        {..., "Principal": { "AWS": "*" }, "Action": "kms:*", "Resource": "*", .
..}
                    - pattern: |
                        {..., "Principal": { "AWS": [..., "*", ...] }, "Action": "kms:*", "Resou
rce": "*", ...}
      severity: ERROR
    - id: terraform.aws.security.aws-kms-no-rotation.aws-kms-no-rotation
      languages:
        - hcl
      message: The AWS KMS has no rotation. Missing rotation can cause leaked key to be used by
attackers. To fix this, set a `enable_key_rotation`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern-either:
            - pattern: |
                resource "aws_kms_key" $ANYTHING {
                  ...
                  enable_key_rotation = false
                  ...
                }
            - pattern: |
                resource "aws_kms_key" $ANYTHING {
                  ...
                  customer_master_key_spec = "SYMMETRIC_DEFAULT"
                  enable_key_rotation = false
                  ...
                }
            - pattern: |
                resource "aws_kms_key" $ANYTHING {
                  ...
                }
        - pattern-not-inside: |
            resource "aws_kms_key" $ANYTHING {
              ...
              enable_key_rotation = true
              ...
            }
        - pattern-not-inside: |
            resource "aws_kms_key" $ANYTHING {
              ...
              customer_master_key_spec = "RSA_2096"
              ...
            }
      severity: WARNING
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    - id: terraform.aws.security.aws-lambda-environment-credentials.aws-lambda-environment-crede
ntials
      languages:
        - hcl
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
            - secrets
      patterns:
        - pattern-inside: |
            resource "$ANYTING" $ANYTHING {
              ...
              environment {
                variables = {
                  ...
                }
              }
              ...
            }
        - pattern-either:
            - pattern-inside: |
                AWS_ACCESS_KEY_ID = "$Y"
            - pattern-regex: |
                (?<![A-Z0-9])[A-Z0-9]{20}(?![A-Z0-9])
            - pattern-inside: |
                AWS_SECRET_ACCESS_KEY = "$Y"
            - pattern-regex: |
                (?<![A-Za-z0-9/+=])[A-Za-z0-9/+=]{40}(?![A-Za-z0-9/+=])
        - focus-metavariable: $Y
      severity: ERROR
    - id: terraform.aws.security.aws-lambda-permission-unrestricted-source-arn.aws-lambda-permis
sion-unrestricted-source-arn
      languages:
        - hcl
      message: The AWS Lambda permission has an AWS service principal but does not specify a sou
rce ARN. If you grant permission to a service principal without specifying the source, other acc
ounts could potentially configure resources in their account to invoke your Lambda function. Set
 the source_arn value to the ARN of the AWS resource that invokes the function, eg. an S3 bucket
, CloudWatch Events Rule, API Gateway, or SNS topic.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://cwe.mitre.org/data/definitions/732.html
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/lambda
_permission
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-resource-lambda
-permission.html
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      patterns:
        - pattern: |
            resource "aws_lambda_permission" $ANYTHING {
              ...
              principal = "$PRINCIPAL"
              ...
            }
        - pattern-not: |
            resource "aws_lambda_permission" $ANYTHING {
              ...
              source_arn = ...
              ...
            }
        - metavariable-regex:
            metavariable: $PRINCIPAL
            regex: .*[.]amazonaws[.]com$
      severity: ERROR
    - id: terraform.aws.security.aws-lambda-x-ray-tracing-not-active.aws-lambda-x-ray-tracing-no
t-active
      languages:
        - hcl
      message: The AWS Lambda function does not have active X-Ray tracing enabled. X-Ray tracing
 enables end-to-end debugging and analysis of all function activity. This makes it easier to tra
ce the flow of logs and identify bottlenecks, slow downs and timeouts.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-778: Insufficient Logging'
        impact: LOW
        likelihood: LOW
        owasp:
            - A09:2021 Security Logging and Monitoring Failures
        references:
            - https://cwe.mitre.org/data/definitions/778.html
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/lambda
_function#mode
            - https://docs.aws.amazon.com/lambda/latest/dg/services-xray.html
        subcategory:
            - audit
        technology:
            - aws
            - terraform
      patterns:
        - pattern: |
            resource "aws_lambda_function" $ANYTHING {
              ...
            }
        - pattern-not: |
            resource "aws_lambda_function" $ANYTHING {
              ...
              tracing_config {
                ...
                mode = "Active"
                ...
              }
              ...
            }
      severity: INFO
    - id: terraform.aws.security.aws-provider-static-credentials.aws-provider-static-credentials
      languages:
        - hcl
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: A hard-coded credential was detected. It is not recommended to store credentials
in source-code, as this risks secrets being leaked and used by either an internal or external ma
licious adversary. It is recommended to use environment variables to securely provide credential
s or retrieve credentials from a secure vault or HSM (Hardware Security Module).
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - secrets
            - aws
            - terraform
      patterns:
        - pattern-inside: |
            provider "aws" {
            ...
                secret_key = "$SECRET"
            }
        - focus-metavariable: $SECRET
      severity: WARNING
    - id: terraform.aws.security.aws-provisioner-exec.aws-provisioner-exec
      languages:
        - terraform
      message: Provisioners are a tool of last resort and should be avoided where possible. Prov
isioner behavior cannot be mapped by Terraform as part of a plan, and execute arbitrary shell co
mmands by design.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-77: Improper Neutralization of Special Elements used in a Command (''Command
Injection'')'
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        impact: MEDIUM
        likelihood: HIGH
        owasp:
            - A03:2021 - Injection
            - A01:2017 - Injection
        references:
            - https://developer.hashicorp.com/terraform/language/resources/provisioners/remote-e
xec
            - https://developer.hashicorp.com/terraform/language/resources/provisioners/local-ex
ec
        subcategory:
            - guardrail
        technology:
            - terraform
      patterns:
        - pattern-either:
            - pattern: |
                provisioner "remote-exec" {
                    ...
                }
            - pattern: |
                provisioner "local-exec" {
                    ...
                }
        - pattern-inside: |
            resource "aws_instance" "..." {
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            }
      severity: WARNING
    - id: terraform.aws.security.aws-rds-backup-no-retention.aws-rds-backup-no-retention
      languages:
        - hcl
      message: The AWS RDS has no retention. Missing retention can cause losing important event
information. To fix this, set a `backup_retention_period`.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-320: CWE CATEGORY: Key Management Errors'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
        - pattern-either:
            - pattern: |
                resource "aws_rds_cluster" $ANYTHING {
                  ...
                  backup_retention_period = 0
                  ...
                }
            - pattern: |
                resource "aws_db_instance" $ANYTHING {
                  ...
                  backup_retention_period = 0
                  ...
                }
      severity: WARNING
    - id: terraform.aws.security.aws-sqs-queue-policy-wildcard-principal.aws-sqs-queue-policy-wi
ldcard-principal
      languages:
        - hcl
      message: Wildcard used in your SQS queue policy principal. This grants access to all users
, including anonymous users (public access). Unless you explicitly require anyone on the interne
t to be able to read or write to your queue, limit principals, actions and resources to what you
 need according to least privilege.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
        references:
            - https://cwe.mitre.org/data/definitions/732.html
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/sqs_qu
eue
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/sqs_qu
eue_policy
            - https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-sec
urity-best-practices.html
        rule-origin-note: published from /src/aws-sqs-queue-policy-wildcard-principal.yml in Non
e
        subcategory:
            - vuln
        technology:
            - aws
            - terraform
      patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-either:
            - pattern-inside: |
                resource "aws_sqs_queue_policy" $ANYTHING {
                  ...
                }
            - pattern-inside: |
                resource "aws_sqs_queue" $ANYTHING {
                  ...
                }
        - pattern-either:
            - patterns:
                - pattern: policy = "$JSONPOLICY"
                - metavariable-pattern:
                    language: json
                    metavariable: $JSONPOLICY
                    patterns:
                        - pattern-not-inside: |
                            {..., "Effect": "Deny", ...}
                        - pattern-either:
                            - pattern: |
                                {..., "Principal": "*", ...}
                            - pattern: |
                                {..., "Principal": [..., "*", ...], ...}
                            - pattern: |
                                {..., "Principal": { "AWS": "*" }, ...}
                            - pattern: |
                                {..., "Principal": { "AWS": [..., "*", ...] }, ...}
            - patterns:
                - pattern-inside: policy = jsonencode(...)
                - pattern-not-inside: |
                    {..., Effect = "Deny", ...}
                - pattern-either:
                    - pattern: |
                        {..., Principal = "*", ...}
                    - pattern: |
                        {..., Principal = [..., "*", ...], ...}
                    - pattern: |
                        {..., Principal = { AWS = "*" }, ...}
                    - pattern: |
                        {..., Principal = { AWS = [..., "*", ...] }, ...}
      severity: ERROR
    - id: terraform.aws.security.aws-subnet-has-public-ip-address.aws-subnet-has-public-ip-addre
ss
      languages:
        - hcl
      message: Resources in the AWS subnet are assigned a public IP address. Resources should no
t be exposed on the public internet, but should have access limited to consumers required for th
e function of your application. Set `map_public_ip_on_launch` to false so that resources are not
 publicly-accessible.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-284: Improper Access Control'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control/
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/subnet
#map_public_ip_on_launch
            - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-instance-addressing.html
#concepts-public-addresses
        subcategory:
            - audit
        technology:
            - terraform
            - aws
      patterns:
        - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: |
                resource "aws_subnet" $ANYTHING {
                  ...
                  map_public_ip_on_launch = true
                  ...
                }
            - pattern: |
                resource "aws_default_subnet" $ANYTHING {
                  ...
                }
        - pattern-not: |
            resource "aws_default_subnet" $ANYTHING {
              ...
              map_public_ip_on_launch = false
              ...
            }
      severity: WARNING
    - id: terraform.aws.security.insecure-load-balancer-tls-version.insecure-load-balancer-tls-v
ersion
      languages:
        - hcl
      message: Detected an AWS load balancer with an insecure TLS version. TLS versions less tha
n 1.2 are considered insecure because they can be broken. To fix this, set your `ssl_policy` to
`"ELBSecurityPolicy-TLS13-1-2-2021-06"`, or include a default action to redirect to HTTPS.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://www.ietf.org/rfc/rfc5246.txt
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      patterns:
        - pattern-either:
            - patterns:
                - pattern: ssl_policy = $ANYTHING
                - pattern-not-regex: ELBSecurityPolicy-TLS13-1-[23]-[0-9-]+
                - pattern-not-regex: ELBSecurityPolicy-FS-1-2-[(Res)0-9-]+
            - patterns:
                - pattern: protocol = "HTTP"
                - pattern-not-inside: |
                    resource $ANYTHING $NAME {
                      ...
                      default_action {
                        ...
                        redirect {
                          ...
                          protocol = "HTTPS"
                          ...
                        }
                        ...
                      }
                      ...
                    }
        - pattern-inside: |
            resource $RESOURCE $X {
              ...
            }
        - metavariable-pattern:
            metavariable: $RESOURCE
            patterns:
                - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    - pattern: |
                        "aws_lb_listener"
                    - pattern: |
                        "aws_alb_listener"
      severity: WARNING
    - id: terraform.aws.security.unrestricted-github-oidc-policy.unrestricted-github-oidc-policy
      languages:
        - hcl
      match:
        all:
            - inside: |
                data "aws_iam_policy_document" $POLICY {
                  ...
                }
            - |
              statement {
                ...
                principals {
                  ...
                  type = "Federated"
                  identifiers = [..., $IDENTIFIER, ...]
                }
              }
            - not: |
                statement {
                  ...
                  condition {
                    ...
                    variable = "token.actions.githubusercontent.com:sub"
                  }
                }
        where:
            - metavariable: $IDENTIFIER
              regex: .*oidc-provider/token\.actions\.githubusercontent\.com
      message: '`$POLICY` is missing a `condition` block which scopes users of this policy to sp
ecific GitHub repositories. Without this, `$POLICY` is open to all users on GitHub. Add a `condi
tion` block on the variable `token.actions.githubusercontent.com:sub` which scopes it to prevent
 this.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-284: Improper Access Control'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Sensitive Data Exposure
            - A01:2021 - Broken Access Control
        references:
            - https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/
configuring-openid-connect-in-amazon-web-services#configuring-the-role-and-trust-policy
            - https://dagrz.com/writing/aws-security/hacking-github-aws-oidc/
        subcategory:
            - audit
        technology:
            - terraform
            - aws
      severity: WARNING
    - id: terraform.aws.security.wildcard-assume-role.wildcard-assume-role
      languages:
        - hcl
      message: 'Detected wildcard access granted to sts:AssumeRole. This means anyone with your
AWS account ID and the name of the role can assume the role. Instead, limit to a specific identi
ty in your account, like this: `arn:aws:iam::<account_id>:root`.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-250: Execution with Unnecessary Privileges'
        impact: MEDIUM
        likelihood: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        owasp:
            - A06:2017 - Security Misconfiguration
            - A05:2021 - Security Misconfiguration
        references:
            - https://rhinosecuritylabs.com/aws/assume-worst-aws-assume-role-enumeration/
        subcategory:
            - vuln
        technology:
            - aws
      patterns:
        - pattern-inside: |
            resource "aws_iam_role" $NAME {
              ...
            }
        - pattern: assume_role_policy = "$STATEMENT"
        - metavariable-pattern:
            language: json
            metavariable: $STATEMENT
            patterns:
                - pattern-inside: |
                    {..., "Effect": "Allow", ..., "Action": "sts:AssumeRole", ...}
                - pattern: |
                    "Principal": {..., "AWS": "*", ...}
      severity: ERROR
    - id: terraform.azure.security.appservice.appservice-authentication-enabled.appservice-authe
ntication-enabled
      languages:
        - hcl
      message: Enabling authentication ensures that all communications in the application are au
thenticated. The `auth_settings` block needs to be filled out with the appropriate auth backend
settings
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-287: Improper Authentication'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A02:2017 - Broken Authentication
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/ap
p_service#auth_settings
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_app_service" "..." {
            ...
              auth_settings {
                ...
                enabled = true
                ...
              }
            ...
            }
        - pattern-either:
            - pattern-inside: |
                resource "azurerm_app_service" "..." {
                ...
                }
            - pattern-inside: |
                resource "azurerm_app_service" "..." {
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                  auth_settings {
                    ...
                    enabled = false
                    ...
                  }
                ...
                }
      severity: ERROR
    - id: terraform.azure.security.appservice.appservice-enable-http2.appservice-enable-http2
      languages:
        - hcl
      message: Use the latest version of HTTP to ensure you are benefiting from security fixes.
Add `http2_enabled = true` to your appservice resource block
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-444: Inconsistent Interpretation of HTTP Requests (''HTTP Request/Response Sm
uggling'')'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A04:2021 - Insecure Design
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/ap
p_service#http2_enabled
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_app_service" "..." {
            ...
              site_config {
                ...
                http2_enabled = true
                ...
              }
            ...
            }
        - pattern-either:
            - pattern-inside: |
                resource "azurerm_app_service" "..." {
                ...
                }
            - pattern-inside: |
                resource "azurerm_app_service" "..." {
                ...
                  site_config {
                    ...
                    http2_enabled = false
                    ...
                  }
                ...
                }
      severity: INFO
    - id: terraform.azure.security.appservice.appservice-enable-https-only.appservice-enable-htt
ps-only
      languages:
        - hcl
      message: By default, clients can connect to App Service by using both HTTP or HTTPS. HTTP
should be disabled enabling the HTTPS Only setting.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/ap
p_service#https_only
            - https://docs.microsoft.com/en-us/azure/app-service/configure-ssl-bindings#enforce-
https
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_app_service" "..." {
            ...
              https_only = true
            ...
            }
        - pattern-either:
            - pattern-inside: |
                resource "azurerm_app_service" "..." {
                ...
                }
            - pattern-inside: |
                resource "azurerm_app_service" "..." {
                ...
                  https_only = false
                ...
                }
      severity: ERROR
    - id: terraform.azure.security.appservice.appservice-require-client-cert.appservice-require-
client-cert
      languages:
        - hcl
      message: Detected an AppService that was not configured to use a client certificate. Add `
client_cert_enabled = true` in your resource block.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-295: Improper Certificate Validation'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/ap
p_service#client_cert_enabled
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_app_service" "..." {
            ...
              client_cert_enabled = true
            ...
            }
        - pattern-either:
            - pattern-inside: |
                resource "azurerm_app_service" "..." {
                ...
                }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern-inside: |
                resource "azurerm_app_service" "..." {
                ...
                  client_cert_enabled = false
                ...
                }
      severity: INFO
    - id: terraform.azure.security.appservice.appservice-use-secure-tls-policy.appservice-use-se
cure-tls-policy
      languages:
        - hcl
      message: Detected an AppService that was not configured to use TLS 1.2. Add `site_config.m
in_tls_version = "1.2"` in your resource block.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/ap
p_service#min_tls_version
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: min_tls_version = $ANYTHING
        - pattern-inside: |
            resource "azurerm_app_service" "$NAME" {
              ...
            }
        - pattern-not-inside: min_tls_version = "1.2"
      severity: ERROR
    - id: terraform.azure.security.appservice.azure-appservice-detailed-errormessages-enabled.az
ure-appservice-detailed-errormessages-enabled
      languages:
        - hcl
      message: Ensure that App service enables detailed error messages
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-778: Insufficient Logging'
        impact: LOW
        likelihood: LOW
        owasp:
            - A10:2017 - Insufficient Logging & Monitoring
            - A09:2021 - Security Logging and Monitoring Failures
        references:
            - https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_app_service" "..." {
            ...
            logs {
              ...
              detailed_error_messages_enabled = true
              ...
            }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
            }
        - pattern-inside: |
            resource "azurerm_app_service" "..." {
            ...
            }
      severity: WARNING
    - id: terraform.azure.security.appservice.azure-appservice-https-only.azure-appservice-https
-only
      languages:
        - hcl
      message: Ensure web app redirects all HTTP traffic to HTTPS in Azure App Service Slot
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_app_service" "..." {
            ...
            https_only = true
            ...
            }
        - pattern-inside: |
            resource "azurerm_app_service" "..." {
            ...
            }
      severity: WARNING
    - id: terraform.azure.security.appservice.azure-appservice-min-tls-version.azure-appservice-
min-tls-version
      languages:
        - hcl
      message: Ensure web app is using the latest version of TLS encryption
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - audit
        technology:
            - terraform
            - azure
      patterns:
        - pattern-either:
            - pattern: |
                "1.0"
            - pattern: |
                "1.1"
        - pattern-inside: min_tls_version = ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern-inside: |
            $RESOURCE "azurerm_app_service" "..." {
            ...
            }
      severity: WARNING
    - id: terraform.azure.security.azure-key-no-expiration-date.azure-key-no-expiration-date
      languages:
        - hcl
      message: Ensure that the expiration date is set on all keys
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-320: CWE CATEGORY: Key Management Errors'
        impact: LOW
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_key_vault_key" "..." {
            ...
            expiration_date = "..."
            ...
            }
        - pattern-inside: |
            resource "azurerm_key_vault_key" "..." {
            ...
            }
      severity: WARNING
    - id: terraform.azure.security.azure-mssql-service-mintls-version.azure-mssql-service-mintls
-version
      languages:
        - hcl
      message: Ensure MSSQL is using the latest version of TLS encryption
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern-either:
            - pattern: |
                "1.0"
            - pattern: |
                "1.1"
        - pattern-inside: minimum_tls_version = ...
        - pattern-inside: |
            $RESOURCE "azurerm_mssql_server" "..." {
            ...
            }
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: WARNING
    - id: terraform.azure.security.azure-mysql-encryption-enabled.azure-mysql-encryption-enabled
      languages:
        - hcl
      message: Ensure that MySQL server enables infrastructure encryption
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-320: CWE CATEGORY: Key Management Errors'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-inside: |
            resource "azurerm_mysql_server" "..." {
            ...
            }
        - pattern-not-inside: |
            resource "azurerm_mysql_server" "..." {
            ...
            infrastructure_encryption_enabled = true
            ...
            }
      severity: WARNING
    - id: terraform.azure.security.azure-mysql-mintls-version.azure-mysql-mintls-version
      languages:
        - hcl
      message: Ensure MySQL is using the latest version of TLS encryption
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern-either:
            - pattern: |
                "TLS1_0"
            - pattern: |
                "TLS1_1"
        - pattern-inside: ssl_minimal_tls_version_enforced = ...
        - pattern-inside: |
            $RESOURCE "azurerm_mysql_server" "..." {
            ...
            }
      severity: WARNING
    - id: terraform.azure.security.keyvault.keyvault-ensure-key-expires.keyvault-ensure-key-expi
res
      languages:
        - hcl
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: Ensure that the expiration date is set on all keys
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-262: Not Using Password Aging'
        impact: MEDIUM
        likelihood: LOW
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/ke
y_vault_key#expiration_date
            - https://docs.microsoft.com/en-us/powershell/module/az.keyvault/update-azkeyvaultke
y?view=azps-5.8.0#example-1--modify-a-key-to-enable-it--and-set-the-expiration-date-and-tags
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_key_vault_key" "..." {
            ...
            expiration_date = "..."
            ...
            }
        - pattern-inside: |
            resource "azurerm_key_vault_key" "..." {
            ...
            }
      severity: INFO
    - id: terraform.azure.security.keyvault.keyvault-ensure-secret-expires.keyvault-ensure-secre
t-expires
      languages:
        - hcl
      message: Ensure that the expiration date is set on all secrets
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-262: Not Using Password Aging'
        impact: MEDIUM
        likelihood: LOW
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/ke
y_vault_secret#expiration_date
            - https://docs.microsoft.com/en-us/azure/key-vault/secrets/about-secrets
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_key_vault_secret" "..." {
            ...
            expiration_date = "..."
            ...
            }
        - pattern-inside: |
            resource "azurerm_key_vault_secret" "..." {
            ...
            }
      severity: INFO
    - id: terraform.azure.security.keyvault.keyvault-purge-enabled.keyvault-purge-enabled
      languages:
        - hcl
      message: Key vault should have purge protection enabled
      metadata:
        category: security
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        confidence: MEDIUM
        cwe:
            - 'CWE-693: Protection Mechanism Failure'
        impact: MEDIUM
        likelihood: MEDIUM
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/ke
y_vault#purge_protection_enabled
            - https://docs.microsoft.com/en-us/azure/key-vault/general/soft-delete-overview#purg
e-protection
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern: resource
        - pattern-not-inside: |
            resource "azurerm_key_vault" "..." {
            ...
             purge_protection_enabled = true
            ...
            }
        - pattern-either:
            - pattern-inside: |
                resource "azurerm_key_vault" "..." {
                ...
                }
            - pattern-inside: |
                resource "azurerm_key_vault" "..." {
                ...
                  purge_protection_enabled = false
                ...
                }
      severity: WARNING
    - id: terraform.azure.security.storage.storage-enforce-https.storage-enforce-https
      languages:
        - hcl
      message: Detected a Storage that was not configured to deny action by default. Add `enable
_https_traffic_only = true` in your resource block.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/st
orage_account#enable_https_traffic_only
            - https://docs.microsoft.com/en-us/azure/storage/common/storage-require-secure-trans
fer
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern-not-inside: |
            resource "azurerm_storage_account" "..." {
            ...
              enable_https_traffic_only = true
            ...
            }
        - pattern-inside: |
            resource "azurerm_storage_account" "..." {
            ...
              enable_https_traffic_only = false
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
            }
      severity: WARNING
    - id: terraform.azure.security.storage.storage-use-secure-tls-policy.storage-use-secure-tls-
policy
      languages:
        - hcl
      message: 'Azure Storage currently supports three versions of the TLS protocol: 1.0, 1.1, a
nd 1.2. Azure Storage uses TLS 1.2 on public HTTPS endpoints, but TLS 1.0 and TLS 1.1 are still
supported for backward compatibility. This check will warn if the minimum TLS is not set to TLS1
_2.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs/resources/st
orage_account#min_tls_version
            - https://docs.microsoft.com/en-us/azure/storage/common/transport-layer-security-con
figure-minimum-version
        subcategory:
            - vuln
        technology:
            - terraform
            - azure
      patterns:
        - pattern-either:
            - pattern-inside: |
                resource "azurerm_storage_account" "..." {
                  ...
                  min_tls_version = "$ANYTHING"
                  ...
                }
            - pattern-inside: |
                resource "azurerm_storage_account" "..." {
                  ...
                }
        - pattern-not-inside: |
            resource "azurerm_storage_account" "..." {
              ...
              min_tls_version = "TLS1_2"
              ...
            }
      severity: ERROR
    - id: terraform.gcp.security.gcp-cloud-storage-logging.gcp-cloud-storage-logging
      languages:
        - hcl
      message: Ensure bucket logs access.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-778: Insufficient Logging'
        impact: LOW
        likelihood: LOW
        owasp:
            - A10:2017 - Insufficient Logging & Monitoring
            - A09:2021 - Security Logging and Monitoring Failures
        references:
            - https://docs.bridgecrew.io/docs/google-cloud-policy-index
        subcategory:
            - vuln
        technology:
            - terraform
            - gcp
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      patterns:
        - pattern: |
            resource "google_storage_bucket" $ANYTHING {
              ...
            }
        - pattern-not-inside: "resource \"google_storage_bucket\" $ANYTHING {\n  ...\n  logging
{\n      log_bucket = ...\n  }          \n  ...\n}\n"
      severity: WARNING
    - id: terraform.gcp.security.gcp-dns-key-specs-rsasha1.gcp-dns-key-specs-rsasha1
      languages:
        - hcl
      message: "Ensure that RSASHA1 is not used for the zone-signing and key-signing keys in Clo
ud DNS DNSSEC\t"
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-326: Inadequate Encryption Strength'
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        subcategory:
            - vuln
        technology:
            - terraform
            - gcp
      patterns:
        - pattern: resource
        - pattern-inside: |
            resource "google_dns_managed_zone" "..." {
            ...
            dnssec_config {
                ...
                default_key_specs {
                    ...
                    algorithm  = "rsasha1"
                    key_type   = "zoneSigning"
                    ...
                }
                ...
            }
            ...
            }
        - pattern-inside: |
            resource "google_dns_managed_zone" "..." {
            ...
            dnssec_config {
                ...
                default_key_specs {
                    ...
                    algorithm  = "rsasha1"
                    key_type   = "keySigning"
                    ...
                }
                ...
            }
            ...
            }
      severity: WARNING
    - id: terraform.gcp.security.gcp-sql-database-require-ssl.gcp-sql-database-require-ssl
      languages:
        - hcl
      message: Ensure all Cloud SQL database instance requires all incoming connections to use S
SL
      metadata:
        category: security
        confidence: MEDIUM
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
    cwe:
        - 'CWE-326: Inadequate Encryption Strength'
    impact: MEDIUM
    likelihood: LOW
    owasp:
        - A03:2017 - Sensitive Data Exposure
        - A02:2021 - Cryptographic Failures
    references:
        - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
    subcategory:
        - vuln
    technology:
        - terraform
        - gcp
  patterns:
    - pattern: resource
    - pattern-inside: |
        resource "google_sql_database_instance" "..." {
            ...
        }
    - pattern-not-inside: |
        resource "google_sql_database_instance" "..." {
            ...
            ip_configuration {
                ...
                require_ssl = true
                ...
            }
            ...
        }
  severity: WARNING
- id: terraform.gcp.security.gcp-sql-public-database.gcp-sql-public-database
  languages:
    - hcl
  message: Ensure that Cloud SQL database Instances are not open to the world
  metadata:
    category: security
    confidence: MEDIUM
    cwe:
        - 'CWE-284: Improper Access Control'
    impact: MEDIUM
    likelihood: MEDIUM
    owasp:
        - A05:2017 - Broken Access Control
        - A01:2021 - Broken Access Control
    references:
        - https://owasp.org/Top10/A01_2021-Broken_Access_Control
    subcategory:
        - vuln
    technology:
        - terraform
        - gcp
  patterns:
    - pattern: resource
    - pattern-either:
        - pattern-inside: |
            resource "google_sql_database_instance" "..." {
            ...
            ip_configuration {
              ...
              authorized_networks {
                ...
                value = "0.0.0.0/0"
                ...
              }
              ...
            }
            ...
            }
        - pattern-inside: |
            resource "google_sql_database_instance" "..." {
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
                ip_configuration {
                  ...
                  dynamic "authorized_networks" {
                    ...
                    content {
                      ...
                      value = "0.0.0.0/0"
                      ...
                    }
                    ...
                  }
                  ...
                }
                ...
                }
      severity: WARNING
    - id: terraform.lang.security.ec2-imdsv1-optional.ec2-imdsv1-optional
      languages:
        - hcl
      message: AWS EC2 Instance allowing use of the IMDSv1
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-918: Server-Side Request Forgery (SSRF)'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A10:2021 - Server-Side Request Forgery (SSRF)
        references:
            - https://aws.amazon.com/blogs/security/defense-in-depth-open-firewalls-reverse-prox
ies-ssrf-vulnerabilities-ec2-instance-metadata-service
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/instan
ce#metadata-options
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      pattern-either:
        - patterns:
            - pattern: http_tokens = "optional"
            - pattern-inside: |
                metadata_options { ... }
        - patterns:
            - pattern: |
                resource "aws_instance" "$NAME" {
                  ...
                }
            - pattern-not: |
                resource "aws_instance" "$NAME" {
                  ...
                  metadata_options {
                    ...
                    http_tokens = "required"
                    ...
                  }
                  ...
                }
            - pattern-not: |
                resource "aws_instance" "$NAME" {
                  ...
                  metadata_options {
                    ...
                    http_tokens = "optional"
                    ...
                  }
                  ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                }
            - pattern-not: |
                resource "aws_instance" "$NAME" {
                  ...
                  metadata_options {
                    ...
                    http_endpoint = "disabled"
                    ...
                  }
                  ...
                }
      severity: ERROR
    - id: terraform.lang.security.rds-insecure-password-storage-in-source-code.rds-insecure-pass
word-storage-in-source-code
      languages:
        - hcl
      message: RDS instance or cluster with hardcoded credentials in source code. It is recommen
ded to pass the credentials at runtime, or generate random credentials using the random_password
 resource.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-522: Insufficiently Protected Credentials'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A02:2017 - Broken Authentication
            - A04:2021 - Insecure Design
        references:
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/db_ins
tance#master_password
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/rds_cl
uster#master_password
            - https://registry.terraform.io/providers/hashicorp/random/latest/docs/resources/pas
sword
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      pattern-either:
        - patterns:
            - pattern: password = "..."
            - pattern-inside: |
                resource "aws_db_instance" "..." {
                  ...
                }
        - patterns:
            - pattern: master_password = "..."
            - pattern-inside: |
                resource "aws_rds_cluster" "..." {
                  ...
                }
      severity: WARNING
    - id: terraform.lang.security.s3-public-rw-bucket.s3-public-rw-bucket
      languages:
        - hcl
      message: S3 bucket with public read-write access detected.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-200: Exposure of Sensitive Information to an Unauthorized Actor'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: LOW
        owasp:
            - A01:2021 - Broken Access Control
        references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/s3_buc
ket#acl
            - https://docs.aws.amazon.com/AmazonS3/latest/dev/acl-overview.html#canned-acl
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      pattern: acl = "public-read-write"
      severity: ERROR
    - id: terraform.lang.security.s3-unencrypted-bucket.s3-unencrypted-bucket
      languages:
        - hcl
      message: This rule has been deprecated, as all s3 buckets are encrypted by default with no
 way to disable it. See https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resour
ces/s3_bucket_server_side_encryption_configuration for more info.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-311: Missing Encryption of Sensitive Data'
        deprecated: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A04:2021 - Insecure Design
        references:
            - https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/s3_buc
ket#server_side_encryption_configuration
            - https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-encryption.html
        subcategory:
            - vuln
        technology:
            - terraform
            - aws
      patterns:
        - pattern: a
        - pattern: b
      severity: INFO
    - id: typescript.angular.security.audit.angular-domsanitizer.angular-bypasssecuritytrust
      languages:
        - typescript
      message: Detected the use of `$TRUST`. This can introduce a Cross-Site-Scripting (XSS) vul
nerability if this comes from user-provided input. If you have to use `$TRUST`, ensure it does n
ot come from user-input or use the appropriate prevention mechanism e.g. input validation or san
itization depending on the context.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://angular.io/api/platform-browser/DomSanitizer
            - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Che
at_Sheet.html
        subcategory:
            - vuln
        technology:
            - angular
            - browser
      mode: taint
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
pattern-sanitizers:
  - patterns:
      - pattern-either:
          - pattern-inside: |
              import $S from "underscore.string"
              ...
          - pattern-inside: |
              import * as $S from "underscore.string"
              ...
          - pattern-inside: |
              import $S from "underscore.string"
              ...
          - pattern-inside: |
              $S = require("underscore.string")
              ...
      - pattern-either:
          - pattern: $S.escapeHTML(...)
  - patterns:
      - pattern-either:
          - pattern-inside: |
              import $S from "dompurify"
              ...
          - pattern-inside: |
              import { ..., $S,... } from "dompurify"
              ...
          - pattern-inside: |
              import * as $S from "dompurify"
              ...
          - pattern-inside: |
              $S = require("dompurify")
              ...
          - pattern-inside: |
              import $S from "isomorphic-dompurify"
              ...
          - pattern-inside: |
              import * as $S from "isomorphic-dompurify"
              ...
          - pattern-inside: |
              $S = require("isomorphic-dompurify")
              ...
      - pattern-either:
          - patterns:
              - pattern-inside: |
                  $VALUE = $S(...)
                  ...
              - pattern: $VALUE.sanitize(...)
          - patterns:
              - pattern-inside: |
                  $VALUE = $S.sanitize
                  ...
              - pattern: $S(...)
          - pattern: $S.sanitize(...)
          - pattern: $S(...)
  - patterns:
      - pattern-either:
          - pattern-inside: |
              import $S from 'xss';
              ...
          - pattern-inside: |
              import * as $S from 'xss';
              ...
          - pattern-inside: |
              $S = require("xss")
              ...
      - pattern: $S(...)
  - patterns:
      - pattern-either:
          - pattern-inside: |
              import $S from 'sanitize-html';
              ...
          - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    import * as $S from "sanitize-html";
                    ...
                - pattern-inside: |
                    $S = require("sanitize-html")
                    ...
            - pattern: $S(...)
        - patterns:
            - pattern: sanitizer.sanitize(...)
            - pattern-not: sanitizer.sanitize(SecurityContext.NONE, ...);
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: $X.$TRUST($Y)
            - focus-metavariable: $Y
            - pattern-not: |
                $X.$TRUST(`...`)
            - pattern-not: |
                $X.$TRUST("...")
            - metavariable-regex:
                metavariable: $TRUST
                regex: (bypassSecurityTrustHtml|bypassSecurityTrustStyle|bypassSecurityTrustScri
pt|bypassSecurityTrustUrl|bypassSecurityTrustResourceUrl)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    function ...({..., $X: string, ...}) { ... }
                - pattern-inside: |
                    function ...(..., $X: string, ...) { ... }
            - focus-metavariable: $X
      severity: WARNING
    - id: typescript.aws-cdk.security.audit.awscdk-bucket-encryption.awscdk-bucket-encryption
      languages:
        - typescript
      message: 'Add "encryption: $Y.BucketEncryption.KMS_MANAGED" or "encryption: $Y.BucketEncry
ption.S3_MANAGED" to the bucket props for Bucket construct $X'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-311: Missing Encryption of Sensitive Data'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A04:2021 - Insecure Design
        references:
            - https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-best-practices.html
        subcategory:
            - vuln
        technology:
            - AWS-CDK
      pattern-either:
        - patterns:
            - pattern-inside: |
                import {Bucket} from '@aws-cdk/aws-s3'
                ...
            - pattern: const $X = new Bucket(...)
            - pattern-not: |
                const $X = new Bucket(..., {..., encryption: BucketEncryption.KMS_MANAGED, ...})
            - pattern-not: |
                const $X = new Bucket(..., {..., encryption: BucketEncryption.KMS, ...})
            - pattern-not: |
                const $X = new Bucket(..., {..., encryption: BucketEncryption.S3_MANAGED, ...})
        - patterns:
            - pattern-inside: |
                import * as $Y from '@aws-cdk/aws-s3'
                ...
            - pattern: const $X = new $Y.Bucket(...)
            - pattern-not: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                const $X = new $Y.Bucket(..., {..., encryption: $Y.BucketEncryption.KMS_MANAGED,
 ...})
            - pattern-not: |
                const $X = new $Y.Bucket(..., {..., encryption: $Y.BucketEncryption.KMS, ...})
            - pattern-not: |
                const $X = new $Y.Bucket(..., {..., encryption: $Y.BucketEncryption.S3_MANAGED,
...})
      severity: ERROR
    - id: typescript.aws-cdk.security.audit.awscdk-bucket-enforcessl.aws-cdk-bucket-enforcessl
      languages:
        - ts
      message: Bucket $X is not set to enforce encryption-in-transit, if not explictly setting t
his on the bucket policy - the property "enforceSSL" should be set to true
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-best-practices.html
        subcategory:
            - vuln
        technology:
            - AWS-CDK
      pattern-either:
        - patterns:
            - pattern-inside: |
                import {Bucket} from '@aws-cdk/aws-s3';
                ...
            - pattern: const $X = new Bucket(...)
            - pattern-not: |
                const $X = new Bucket(..., {enforceSSL: true}, ...)
        - patterns:
            - pattern-inside: |
                import * as $Y from '@aws-cdk/aws-s3';
                ...
            - pattern: const $X = new $Y.Bucket(...)
            - pattern-not: |
                const $X = new $Y.Bucket(..., {..., enforceSSL: true, ...})
      severity: ERROR
    - id: typescript.aws-cdk.security.audit.awscdk-sqs-unencryptedqueue.awscdk-sqs-unencryptedqu
eue
      languages:
        - ts
      message: 'Queue $X is missing encryption at rest. Add "encryption: $Y.QueueEncryption.KMS"
 or "encryption: $Y.QueueEncryption.KMS_MANAGED" to the queue props to enable encryption at rest
 for the queue.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-311: Missing Encryption of Sensitive Data'
        impact: HIGH
        likelihood: LOW
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A04:2021 - Insecure Design
        references:
            - https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dat
a-protection.html
        subcategory:
            - vuln
        technology:
            - AWS-CDK
      pattern-either:
        - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern-inside: |
                import {Queue} from '@aws-cdk/aws-sqs'
                ...
            - pattern: const $X = new Queue(...)
            - pattern-not: |
                const $X = new Queue(..., {..., encryption: QueueEncryption.KMS_MANAGED, ...})
            - pattern-not: |
                const $X = new Queue(..., {..., encryption: QueueEncryption.KMS, ...})
        - patterns:
            - pattern-inside: |
                import * as $Y from '@aws-cdk/aws-sqs'
                ...
            - pattern: const $X = new $Y.Queue(...)
            - pattern-not: |
                const $X = new $Y.Queue(..., {..., encryption: $Y.QueueEncryption.KMS_MANAGED, .
..})
            - pattern-not: |
                const $X = new $Y.Queue(..., {..., encryption: $Y.QueueEncryption.KMS, ...})
      severity: WARNING
    - id: typescript.aws-cdk.security.awscdk-bucket-grantpublicaccessmethod.awscdk-bucket-grantp
ublicaccessmethod
      languages:
        - ts
      message: Using the GrantPublicAccess method on bucket contruct $X will make the objects in
 the bucket world accessible. Verify if this is intentional.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-306: Missing Authentication for Critical Function'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: HIGH
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-overview.html
        subcategory:
            - vuln
        technology:
            - AWS-CDK
      pattern-either:
        - patterns:
            - pattern-inside: |
                import {Bucket} from '@aws-cdk/aws-s3'
                ...
            - pattern: |
                const $X = new Bucket(...)
                ...
                $X.grantPublicAccess(...)
        - patterns:
            - pattern-inside: |
                import * as $Y from '@aws-cdk/aws-s3'
                ...
            - pattern: |
                const $X = new $Y.Bucket(...)
                ...
                $X.grantPublicAccess(...)
      severity: WARNING
    - id: typescript.aws-cdk.security.awscdk-codebuild-project-public.awscdk-codebuild-project-p
ublic
      languages:
        - ts
      message: CodeBuild Project $X is set to have a public URL. This will make the build result
s, logs, artifacts publically accessible, including builds prior to the project being public. En
sure this is acceptable for the project.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - 'CWE-306: Missing Authentication for Critical Function'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://docs.aws.amazon.com/codebuild/latest/userguide/public-builds.html
        subcategory:
            - vuln
        technology:
            - AWS-CDK
      pattern-either:
        - patterns:
            - pattern-inside: |
                import {Project} from '@aws-cdk/aws-codebuild'
                ...
            - pattern: |
                const $X = new Project(..., {..., badge: true, ...})
        - patterns:
            - pattern-inside: |
                import * as $Y from '@aws-cdk/aws-codebuild'
                ...
            - pattern: |
                const $X = new $Y.Project(..., {..., badge: true, ...})
      severity: WARNING
    - id: typescript.react.security.audit.react-dangerouslysetinnerhtml.react-dangerouslysetinne
rhtml
      languages:
        - typescript
        - javascript
      message: Detection of dangerouslySetInnerHTML from non-constant definition. This can inadv
ertently expose users to cross-site scripting (XSS) attacks if this comes from user-provided inp
ut. If you have to use dangerouslySetInnerHTML, consider using a sanitization library such as DO
MPurify to sanitize your HTML.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://react.dev/reference/react-dom/components/common#dangerously-setting-the-in
ner-html
        subcategory:
            - vuln
        technology:
            - react
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import $S from "underscore.string"
                    ...
                - pattern-inside: |
                    import * as $S from "underscore.string"
                    ...
                - pattern-inside: |
                    import $S from "underscore.string"
                    ...
                - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                $S = require("underscore.string")
                ...
        - pattern-either:
            - pattern: $S.escapeHTML(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                import $S from "dompurify"
                ...
            - pattern-inside: |
                import { ..., $S,... } from "dompurify"
                ...
            - pattern-inside: |
                import * as $S from "dompurify"
                ...
            - pattern-inside: |
                $S = require("dompurify")
                ...
            - pattern-inside: |
                import $S from "isomorphic-dompurify"
                ...
            - pattern-inside: |
                import * as $S from "isomorphic-dompurify"
                ...
            - pattern-inside: |
                $S = require("isomorphic-dompurify")
                ...
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    $VALUE = $S(...)
                    ...
                - pattern: $VALUE.sanitize(...)
            - patterns:
                - pattern-inside: |
                    $VALUE = $S.sanitize
                    ...
                - pattern: $S(...)
            - pattern: $S.sanitize(...)
            - pattern: $S(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                import $S from 'xss';
                ...
            - pattern-inside: |
                import * as $S from 'xss';
                ...
            - pattern-inside: |
                $S = require("xss")
                ...
        - pattern: $S(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                import $S from 'sanitize-html';
                ...
            - pattern-inside: |
                import * as $S from "sanitize-html";
                ...
            - pattern-inside: |
                $S = require("sanitize-html")
                ...
        - pattern: $S(...)
    - patterns:
        - pattern-either:
            - pattern-inside: |
                $S = new Remarkable()
                ...
        - pattern: $S.render(...)
  pattern-sinks:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - patterns:
            - focus-metavariable: $X
            - pattern-either:
                - pattern: |
                    {...,dangerouslySetInnerHTML: {__html: $X},...}
                - pattern: |
                    <$Y ... dangerouslySetInnerHTML={{__html: $X}} />
            - pattern-not: |
                <$Y ... dangerouslySetInnerHTML={{__html: "..."}} />
            - pattern-not: |
                {...,dangerouslySetInnerHTML:{__html: "..."},...}
            - metavariable-pattern:
                metavariable: $X
                patterns:
                    - pattern-not: |
                        {...}
            - pattern-not: |
                <... {__html: "..."} ...>
            - pattern-not: |
                <... {__html: `...`} ...>
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    function ...({..., $X, ...}) { ... }
                - pattern-inside: |
                    function ...(..., $X, ...) { ... }
            - focus-metavariable: $X
            - pattern-not-inside: |
                $F. ... .$SANITIZEUNC(...)
      severity: WARNING
    - id: typescript.react.security.audit.react-unsanitized-method.react-unsanitized-method
      languages:
        - typescript
        - javascript
      message: Detection of $HTML from non-constant definition. This can inadvertently expose us
ers to cross-site scripting (XSS) attacks if this comes from user-provided input. If you have to
 use $HTML, consider using a sanitization library such as DOMPurify to sanitize your HTML.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://developer.mozilla.org/en-US/docs/Web/API/Document/writeln
            - https://developer.mozilla.org/en-US/docs/Web/API/Document/write
            - https://developer.mozilla.org/en-US/docs/Web/API/Element/insertAdjacentHTML
        subcategory:
            - vuln
        technology:
            - react
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import $S from "underscore.string"
                    ...
                - pattern-inside: |
                    import * as $S from "underscore.string"
                    ...
                - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            import $S from "underscore.string"
            ...
        - pattern-inside: |
            $S = require("underscore.string")
            ...
    - pattern-either:
        - pattern: $S.escapeHTML(...)
- patterns:
    - pattern-either:
        - pattern-inside: |
            import $S from "dompurify"
            ...
        - pattern-inside: |
            import { ..., $S,... } from "dompurify"
            ...
        - pattern-inside: |
            import * as $S from "dompurify"
            ...
        - pattern-inside: |
            $S = require("dompurify")
            ...
        - pattern-inside: |
            import $S from "isomorphic-dompurify"
            ...
        - pattern-inside: |
            import * as $S from "isomorphic-dompurify"
            ...
        - pattern-inside: |
            $S = require("isomorphic-dompurify")
            ...
    - pattern-either:
        - patterns:
            - pattern-inside: |
                $VALUE = $S(...)
                ...
            - pattern: $VALUE.sanitize(...)
        - patterns:
            - pattern-inside: |
                $VALUE = $S.sanitize
                ...
            - pattern: $S(...)
        - pattern: $S.sanitize(...)
        - pattern: $S(...)
- patterns:
    - pattern-either:
        - pattern-inside: |
            import $S from 'xss';
            ...
        - pattern-inside: |
            import * as $S from 'xss';
            ...
        - pattern-inside: |
            $S = require("xss")
            ...
    - pattern: $S(...)
- patterns:
    - pattern-either:
        - pattern-inside: |
            import $S from 'sanitize-html';
            ...
        - pattern-inside: |
            import * as $S from "sanitize-html";
            ...
        - pattern-inside: |
            $S = require("sanitize-html")
            ...
    - pattern: $S(...)
- patterns:
    - pattern-either:
        - pattern-inside: |
            $S = new Remarkable()
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                ...
            - pattern: $S.render(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: "this.window.document. ... .$HTML('...',$SINK) \n"
                - pattern: "window.document. ... .$HTML('...',$SINK) \n"
                - pattern: "document.$HTML($SINK)  \n"
            - metavariable-regex:
                metavariable: $HTML
                regex: (writeln|write)
            - focus-metavariable: $SINK
        - patterns:
            - pattern-either:
                - pattern: "$PROP. ... .$HTML('...',$SINK) \n"
            - metavariable-regex:
                metavariable: $HTML
                regex: (insertAdjacentHTML)
            - focus-metavariable: $SINK
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    function ...({..., $X, ...}) { ... }
                - pattern-inside: |
                    function ...(..., $X, ...) { ... }
            - focus-metavariable: $X
            - pattern-either:
                - pattern: $X.$Y
                - pattern: $X[...]
      severity: WARNING
    - id: typescript.react.security.audit.react-unsanitized-property.react-unsanitized-property
      languages:
        - typescript
        - javascript
      message: Detection of $HTML from non-constant definition. This can inadvertently expose us
ers to cross-site scripting (XSS) attacks if this comes from user-provided input. If you have to
 use $HTML, consider using a sanitization library such as DOMPurify to sanitize your HTML.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-79: Improper Neutralization of Input During Web Page Generation (''Cross-site
 Scripting'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A07:2017 - Cross-Site Scripting (XSS)
            - A03:2021 - Injection
        references:
            - https://react.dev/reference/react-dom/components/common#dangerously-setting-the-in
ner-html
        subcategory:
            - vuln
        technology:
            - react
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import $S from "underscore.string"
                    ...
                - pattern-inside: |
                    import * as $S from "underscore.string"
                    ...
                - pattern-inside: |
                    import $S from "underscore.string"
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
        - pattern-inside: |
            $S = require("underscore.string")
            ...
    - pattern-either:
        - pattern: $S.escapeHTML(...)
- patterns:
    - pattern-either:
        - pattern-inside: |
            import $S from "dompurify"
            ...
        - pattern-inside: |
            import { ..., $S,... } from "dompurify"
            ...
        - pattern-inside: |
            import * as $S from "dompurify"
            ...
        - pattern-inside: |
            $S = require("dompurify")
            ...
        - pattern-inside: |
            import $S from "isomorphic-dompurify"
            ...
        - pattern-inside: |
            import * as $S from "isomorphic-dompurify"
            ...
        - pattern-inside: |
            $S = require("isomorphic-dompurify")
            ...
    - pattern-either:
        - patterns:
            - pattern-inside: |
                $VALUE = $S(...)
                ...
            - pattern: $VALUE.sanitize(...)
        - patterns:
            - pattern-inside: |
                $VALUE = $S.sanitize
                ...
            - pattern: $S(...)
        - pattern: $S.sanitize(...)
        - pattern: $S(...)
- patterns:
    - pattern-either:
        - pattern-inside: |
            import $S from 'xss';
            ...
        - pattern-inside: |
            import * as $S from 'xss';
            ...
        - pattern-inside: |
            $S = require("xss")
            ...
    - pattern: $S(...)
- patterns:
    - pattern-either:
        - pattern-inside: |
            import $S from 'sanitize-html';
            ...
        - pattern-inside: |
            import * as $S from "sanitize-html";
            ...
        - pattern-inside: |
            $S = require("sanitize-html")
            ...
    - pattern: $S(...)
- patterns:
    - pattern-either:
        - pattern-inside: |
            $S = new Remarkable()
            ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - pattern: $S.render(...)
    pattern-sinks:
      - patterns:
          - pattern-either:
              - pattern-inside: |
                  $BODY = $REACT.useRef(...)
                  ...
              - pattern-inside: |
                  $BODY = useRef(...)
                  ...
              - pattern-inside: |
                  $BODY = findDOMNode(...)
                  ...
              - pattern-inside: |
                  $BODY = createRef(...)
                  ...
              - pattern-inside: |
                  $BODY = $REACT.findDOMNode(...)
                  ...
              - pattern-inside: |
                  $BODY = $REACT.createRef(...)
                  ...
          - pattern-either:
              - pattern: "$BODY. ... .$HTML = $SINK \n"
              - pattern: "$BODY.$HTML = $SINK  \n"
          - metavariable-regex:
              metavariable: $HTML
              regex: (innerHTML|outerHTML)
          - focus-metavariable: $SINK
      - patterns:
          - pattern-either:
              - pattern: ReactDOM.findDOMNode(...).$HTML = $SINK
          - metavariable-regex:
              metavariable: $HTML
              regex: (innerHTML|outerHTML)
          - focus-metavariable: $SINK
    pattern-sources:
      - patterns:
          - pattern-either:
              - pattern-inside: |
                  function ...({..., $X, ...}) { ... }
              - pattern-inside: |
                  function ...(..., $X, ...) { ... }
          - focus-metavariable: $X
          - pattern-either:
              - pattern: $X.$Y
              - pattern: $X[...]
    severity: WARNING
  - id: typescript.react.security.react-insecure-request.react-insecure-request
    languages:
      - typescript
      - javascript
    message: Unencrypted request over HTTP detected.
    metadata:
      category: security
      confidence: MEDIUM
      cwe:
          - 'CWE-319: Cleartext Transmission of Sensitive Information'
      impact: MEDIUM
      likelihood: LOW
      owasp:
          - A03:2017 - Sensitive Data Exposure
          - A02:2021 - Cryptographic Failures
      references:
          - https://www.npmjs.com/package/axios
      subcategory:
          - vuln
      technology:
          - react
      vulnerability: Insecure Transport
    pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import $AXIOS from 'axios';
                    ...
                    $AXIOS.$METHOD(...)
                - pattern-inside: |
                    $AXIOS = require('axios');
                    ...
                    $AXIOS.$METHOD(...)
            - pattern-either:
                - pattern: $AXIOS.get("=~/[Hh][Tt][Tt][Pp]:\/\/.*/",...)
                - pattern: $AXIOS.post("=~/[Hh][Tt][Tt][Pp]:\/\/.*/",...)
                - pattern: $AXIOS.delete("=~/[Hh][Tt][Tt][Pp]:\/\/.*/",...)
                - pattern: $AXIOS.head("=~/[Hh][Tt][Tt][Pp]:\/\/.*/",...)
                - pattern: $AXIOS.patch("=~/[Hh][Tt][Tt][Pp]:\/\/.*/",...)
                - pattern: $AXIOS.put("=~/[Hh][Tt][Tt][Pp]:\/\/.*/",...)
                - pattern: $AXIOS.options("=~/[Hh][Tt][Tt][Pp]:\/\/.*/",...)
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    import $AXIOS from 'axios';
                    ...
                    $AXIOS(...)
                - pattern-inside: |
                    $AXIOS = require('axios');
                    ...
                    $AXIOS(...)
            - pattern-either:
                - pattern: '$AXIOS({url: "=~/[Hh][Tt][Tt][Pp]:\/\/.*/"}, ...)'
                - pattern: |
                    $OPTS = {url: "=~/[Hh][Tt][Tt][Pp]:\/\/.*/"}
                    ...
                    $AXIOS($OPTS, ...)
        - pattern: fetch("=~/[Hh][Tt][Tt][Pp]:\/\/.*/", ...)
      severity: ERROR
    - id: yaml.argo.security.argo-workflow-parameter-command-injection.argo-workflow-parameter-c
ommand-injection
      languages:
        - yaml
      message: Using input or workflow parameters in here-scripts can lead to command injection
or code injection. Convert the parameters to env variables instead.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A03:2021 - Injection
        references:
            - https://github.com/argoproj/argo-workflows/issues/5061
            - https://github.com/argoproj/argo-workflows/issues/5114#issue-808865370
        subcategory:
            - vuln
        technology:
            - ci
            - argo
      patterns:
        - pattern-inside: |
            apiVersion: $VERSION
            ...
        - metavariable-regex:
            metavariable: $VERSION
            regex: (argoproj.io.*)
        - pattern-either:
            - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                - pattern-inside: "command:\n  ...\n  - python\n  ...\n...\nsource: \n  $SCRIPT\
n"
                - focus-metavariable: $SCRIPT
                - metavariable-pattern:
                    language: python
                    metavariable: $SCRIPT
                    patterns:
                        - pattern: |
                            $FUNC(..., $PARAM, ...)
                        - metavariable-pattern:
                            metavariable: $PARAM
                            pattern-either:
                                - pattern-regex: (.*{{.*inputs.parameters.*}}.*)
                                - pattern-regex: (.*{{.*workflow.parameters.*}}.*)
            - patterns:
                - pattern-inside: "command:\n  ...\n  - $LANG\n  ...\n...\nsource: \n  $SCRIPT\n
"
                - metavariable-regex:
                    metavariable: $LANG
                    regex: (bash|sh)
                - focus-metavariable: $SCRIPT
                - metavariable-pattern:
                    language: bash
                    metavariable: $SCRIPT
                    patterns:
                        - pattern: |
                            $CMD ... $PARAM  ...
                        - metavariable-pattern:
                            metavariable: $PARAM
                            pattern-either:
                                - pattern-regex: (.*{{.*inputs.parameters.*}}.*)
                                - pattern-regex: (.*{{.*workflow.parameters.*}}.*)
            - patterns:
                - pattern-inside: |
                    container:
                      ...
                      command: $LANG
                      ...
                      args: $PARAM
                - metavariable-regex:
                    metavariable: $LANG
                    regex: .*(sh|bash|ksh|csh|tcsh|zsh).*
                - metavariable-pattern:
                    metavariable: $PARAM
                    pattern-either:
                        - pattern-regex: (.*{{.*inputs.parameters.*}}.*)
                        - pattern-regex: (.*{{.*workflow.parameters.*}}.*)
                - focus-metavariable: $PARAM
      severity: ERROR
    - fix: |
        false
      id: yaml.docker-compose.security.privileged-service.privileged-service
      languages:
        - yaml
      message: Service '$SERVICE' is running in privileged mode. This grants the container the e
quivalent of root capabilities on the host machine. This can lead to container escapes, privileg
e escalation, and other security concerns. Remove the 'privileged' key to disable this capabilit
y.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-250: Execution with Unnecessary Privileges'
        impact: HIGH
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: HIGH
        owasp:
            - A06:2017 - Security Misconfiguration
            - A05:2021 - Security Misconfiguration
        references:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://www.trendmicro.com/en_us/research/19/l/why-running-a-privileged-container-
in-docker-is-a-bad-idea.html
            - https://containerjournal.com/topics/container-security/why-running-a-privileged-co
ntainer-is-not-a-good-idea/
        subcategory:
            - vuln
        technology:
            - docker-compose
      patterns:
        - pattern-inside: |
            version: ...
            ...
            services:
              ...
              $SERVICE:
                ...
                privileged: $TRUE
        - focus-metavariable: $TRUE
        - metavariable-regex:
            metavariable: $TRUE
            regex: (true)
      severity: WARNING
    - id: yaml.github-actions.security.allowed-unsecure-commands.allowed-unsecure-commands
      languages:
        - yaml
      message: The environment variable `ACTIONS_ALLOW_UNSECURE_COMMANDS` grants this workflow p
ermissions to use the `set-env` and `add-path` commands. There is a vulnerability in these comma
nds that could result in environment variables being modified by an attacker. Depending on the u
se of the environment variable, this could enable an attacker to, at worst, modify the system pa
th to run a different command than intended, resulting in arbitrary code execution. This could r
esult in stolen code or secrets. Don't use `ACTIONS_ALLOW_UNSECURE_COMMANDS`. Instead, use Envir
onment Files. See https://github.com/actions/toolkit/blob/main/docs/commands.md#environment-file
s for more information.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-749: Exposed Dangerous Method or Function'
        impact: MEDIUM
        likelihood: LOW
        owasp: A06:2017 - Security Misconfiguration
        references:
            - https://github.blog/changelog/2020-10-01-github-actions-deprecating-set-env-and-ad
d-path-commands/
            - https://github.com/actions/toolkit/security/advisories/GHSA-mfwh-5m23-j46w
            - https://github.com/actions/toolkit/blob/main/docs/commands.md#environment-files
        subcategory:
            - vuln
        technology:
            - github-actions
      patterns:
        - pattern-either:
            - patterns:
                - pattern-inside: '{env: ...}'
                - pattern: 'ACTIONS_ALLOW_UNSECURE_COMMANDS: true'
      severity: WARNING
    - id: yaml.github-actions.security.github-script-injection.github-script-injection
      languages:
        - yaml
      message: 'Using variable interpolation `${{...}}` with `github` context data in a `actions
/github-script`''s `script:` step could allow an attacker to inject their own code into the runn
er. This would allow them to steal secrets and code. `github` context data can have arbitrary us
er input and should be treated as untrusted. Instead, use an intermediate environment variable w
ith `env:` to store the data and use the environment variable in the `run:` script. Be sure to u
se double-quotes the environment variable, like this: "$ENVVAR".'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-94: Improper Control of Generation of Code (''Code Injection'')'
        cwe2022-top25: true
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        impact: HIGH
        likelihood: HIGH
        owasp:
            - A03:2021 - Injection
        references:
            - https://docs.github.com/en/actions/learn-github-actions/security-hardening-for-git
hub-actions#understanding-the-risk-of-script-injections
            - https://securitylab.github.com/research/github-actions-untrusted-input/
            - https://github.com/actions/github-script
        subcategory:
            - vuln
        technology:
            - github-actions
      patterns:
        - pattern-inside: 'steps: [...]'
        - pattern-inside: |
            uses: $ACTION
            ...
        - pattern-inside: |
            with:
              ...
              script: ...
              ...
        - pattern: 'script: $SHELL'
        - metavariable-regex:
            metavariable: $ACTION
            regex: actions/github-script@.*
        - metavariable-pattern:
            language: generic
            metavariable: $SHELL
            patterns:
                - pattern-either:
                    - pattern: ${{ github.event.issue.title }}
                    - pattern: ${{ github.event.issue.body }}
                    - pattern: ${{ github.event.pull_request.title }}
                    - pattern: ${{ github.event.pull_request.body }}
                    - pattern: ${{ github.event.comment.body }}
                    - pattern: ${{ github.event.review.body }}
                    - pattern: ${{ github.event.review_comment.body }}
                    - pattern: ${{ github.event.pages. ... .page_name}}
                    - pattern: ${{ github.event.head_commit.message }}
                    - pattern: ${{ github.event.head_commit.author.email }}
                    - pattern: ${{ github.event.head_commit.author.name }}
                    - pattern: ${{ github.event.commits ... .author.email }}
                    - pattern: ${{ github.event.commits ... .author.name }}
                    - pattern: ${{ github.event.pull_request.head.ref }}
                    - pattern: ${{ github.event.pull_request.head.label }}
                    - pattern: ${{ github.event.pull_request.head.repo.default_branch }}
                    - pattern: ${{ github.head_ref }}
                    - pattern: ${{ github.event.inputs ... }}
      severity: ERROR
    - id: yaml.github-actions.security.run-shell-injection.run-shell-injection
      languages:
        - yaml
      message: 'Using variable interpolation `${{...}}` with `github` context data in a `run:` s
tep could allow an attacker to inject their own code into the runner. This would allow them to s
teal secrets and code. `github` context data can have arbitrary user input and should be treated
 as untrusted. Instead, use an intermediate environment variable with `env:` to store the data a
nd use the environment variable in the `run:` script. Be sure to use double-quotes the environme
nt variable, like this: "$ENVVAR".'
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-78: Improper Neutralization of Special Elements used in an OS Command (''OS C
ommand Injection'')'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: HIGH
        likelihood: HIGH
        owasp:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A01:2017 - Injection
            - A03:2021 - Injection
        references:
            - https://docs.github.com/en/actions/learn-github-actions/security-hardening-for-git
hub-actions#understanding-the-risk-of-script-injections
            - https://securitylab.github.com/research/github-actions-untrusted-input/
        subcategory:
            - vuln
        technology:
            - github-actions
      patterns:
        - pattern-inside: 'steps: [...]'
        - pattern-inside: |
            - run: ...
              ...
        - pattern: 'run: $SHELL'
        - metavariable-pattern:
            language: generic
            metavariable: $SHELL
            patterns:
                - pattern-either:
                    - pattern: ${{ github.event.issue.title }}
                    - pattern: ${{ github.event.issue.body }}
                    - pattern: ${{ github.event.pull_request.title }}
                    - pattern: ${{ github.event.pull_request.body }}
                    - pattern: ${{ github.event.comment.body }}
                    - pattern: ${{ github.event.review.body }}
                    - pattern: ${{ github.event.review_comment.body }}
                    - pattern: ${{ github.event.pages. ... .page_name}}
                    - pattern: ${{ github.event.head_commit.message }}
                    - pattern: ${{ github.event.head_commit.author.email }}
                    - pattern: ${{ github.event.head_commit.author.name }}
                    - pattern: ${{ github.event.commits ... .author.email }}
                    - pattern: ${{ github.event.commits ... .author.name }}
                    - pattern: ${{ github.event.pull_request.head.ref }}
                    - pattern: ${{ github.event.pull_request.head.label }}
                    - pattern: ${{ github.event.pull_request.head.repo.default_branch }}
                    - pattern: ${{ github.head_ref }}
                    - pattern: ${{ github.event.inputs ... }}
      severity: ERROR
    - id: yaml.github-actions.security.third-party-action-not-pinned-to-commit-sha.third-party-a
ction-not-pinned-to-commit-sha
      languages:
        - yaml
      message: An action sourced from a third-party repository on GitHub is not pinned to a full
 length commit SHA. Pinning an action to a full length commit SHA is currently the only way to u
se an action as an immutable release. Pinning to a particular SHA helps mitigate the risk of a b
ad actor adding a backdoor to the action's repository, as they would need to generate a SHA-1 co
llision for a valid Git object payload.
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-1357: Reliance on Insufficiently Trustworthy Component'
            - 'CWE-353: Missing Support for Integrity Check'
        impact: LOW
        likelihood: LOW
        owasp: A06:2021 - Vulnerable and Outdated Components
        references:
            - https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components
            - https://docs.github.com/en/actions/security-guides/security-hardening-for-github-a
ctions#using-third-party-actions
        subcategory:
            - vuln
        technology:
            - github-actions
      patterns:
        - pattern-inside: '{steps: ...}'
        - pattern: |
            uses: "$USES"
        - metavariable-pattern:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            language: generic
            metavariable: $USES
            patterns:
                - pattern-not-regex: ^[.]/
                - pattern-not-regex: ^actions/
                - pattern-not-regex: ^github/
                - pattern-not-regex: '@[0-9a-f]{40}$'
                - pattern-not-regex: ^docker://.*@sha256:[0-9a-f]{64}$
      severity: WARNING
    - id: yaml.github-actions.security.workflow-run-target-code-checkout.workflow-run-target-cod
e-checkout
      languages:
        - yaml
      message: This GitHub Actions workflow file uses `workflow_run` and checks out code from th
e incoming pull request. When using `workflow_run`, the Action runs in the context of the target
 repository, which includes access to all repository secrets. Normally, this is safe because the
 Action only runs code from the target repository, not the incoming PR. However, by checking out
 the incoming PR code, you're now using the incoming code for the rest of the action. You may be
 inadvertently executing arbitrary code from the incoming PR with access to repository secrets,
which would let an attacker steal repository secrets. This normally happens by running build scr
ipts (e.g., `npm build` and `make`) or dependency installation scripts (e.g., `python setup.py i
nstall`). Audit your workflow file to make sure no code from the incoming PR is executed. Please
 see https://securitylab.github.com/research/github-actions-preventing-pwn-requests/ for additio
nal mitigations.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: 'CWE-913: Improper Control of Dynamically-Managed Code Resources'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp: A01:2017 - Injection
        references:
            - https://securitylab.github.com/research/github-actions-preventing-pwn-requests/
            - https://github.com/justinsteven/advisories/blob/master/2021_github_actions_checksp
elling_token_leak_via_advice_symlink.md
            - https://www.legitsecurity.com/blog/github-privilege-escalation-vulnerability
        subcategory:
            - vuln
        technology:
            - github-actions
      patterns:
        - pattern-inside: |
            on:
              ...
              workflow_run: ...
              ...
            ...
        - pattern-inside: |
            jobs:
              ...
              $JOBNAME:
                ...
                steps:
                  ...
        - pattern: |
            ...
            uses: "$ACTION"
            with:
              ...
              ref: $EXPR
        - metavariable-regex:
            metavariable: $ACTION
            regex: actions/checkout@.*
        - metavariable-pattern:
            language: generic
            metavariable: $EXPR
            patterns:
                - pattern: ${{ github.event.workflow_run ... }}
      severity: WARNING
    - fix: |
        securityContext:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          allowPrivilegeEscalation: false
        $NAME
      id: yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege
-escalation-no-securitycontext
      languages:
        - yaml
      message: In Kubernetes, each pod runs in its own isolated environment with its own set of
security policies. However, certain container images may contain `setuid` or `setgid` binaries t
hat could allow an attacker to perform privilege escalation and gain access to sensitive resourc
es. To mitigate this risk, it's recommended to add a `securityContext` to the container in the p
od, with the parameter `allowPrivilegeEscalation` set to `false`. This will prevent the containe
r from running any privileged processes and limit the impact of any potential attacks. By adding
 a `securityContext` to your Kubernetes pod, you can help to ensure that your containerized appl
ications are more secure and less vulnerable to privilege escalation attacks.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
            - A06:2017 - Security Misconfiguration
        references:
            - https://kubernetes.io/docs/concepts/policy/pod-security-policy/#privilege-escalati
on
            - https://kubernetes.io/docs/tasks/configure-pod-container/security-context/
            - https://www.kernel.org/doc/Documentation/prctl/no_new_privs.txt
            - https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html#ru
le-4-add-no-new-privileges-flag
        subcategory:
            - vuln
        technology:
            - kubernetes
      patterns:
        - pattern-inside: |
            containers:
              ...
        - pattern-inside: |
            - $NAME: $CONTAINER
              ...
        - pattern: |
            image: ...
            ...
        - pattern-not: |
            image: ...
            ...
            securityContext:
              ...
        - metavariable-regex:
            metavariable: $NAME
            regex: name
        - focus-metavariable: $NAME
      severity: WARNING
    - fix: |
        false
      id: yaml.kubernetes.security.allow-privilege-escalation-true.allow-privilege-escalation-tr
ue
      languages:
        - yaml
      message: In Kubernetes, each pod runs in its own isolated environment with its own  set of
 security policies. However, certain container images may contain  `setuid` or `setgid` binaries
 that could allow an attacker to perform  privilege escalation and gain access to sensitive reso
urces. To mitigate  this risk, it's recommended to add a `securityContext` to the container in
the pod, with the parameter `allowPrivilegeEscalation` set to `false`.  This will prevent the co
ntainer from running any privileged processes and  limit the impact of any potential attacks.  I
n the container `$CONTAINER` this parameter is set to `true` which makes this container much mor
e vulnerable to privelege escalation attacks.
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
            - A06:2017 - Security Misconfiguration
        references:
            - https://kubernetes.io/docs/concepts/policy/pod-security-policy/#privilege-escalati
on
            - https://kubernetes.io/docs/tasks/configure-pod-container/security-context/
            - https://www.kernel.org/doc/Documentation/prctl/no_new_privs.txt
            - https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html#ru
le-4-add-no-new-privileges-flag
        subcategory:
            - vuln
        technology:
            - kubernetes
      patterns:
        - pattern-inside: |
            containers:
              ...
        - pattern-inside: |
            - name: $CONTAINER
              ...
        - pattern-inside: |
            image: ...
            ...
        - pattern-inside: |
            securityContext:
              ...
        - pattern: |
            allowPrivilegeEscalation: $TRUE
        - metavariable-pattern:
            metavariable: $TRUE
            pattern: |
                true
        - focus-metavariable: $TRUE
      severity: WARNING
    - fix: |
        securityContext:
          allowPrivilegeEscalation: false #
      id: yaml.kubernetes.security.allow-privilege-escalation.allow-privilege-escalation
      languages:
        - yaml
      message: In Kubernetes, each pod runs in its own isolated environment with its own set of
security policies. However, certain container images may contain `setuid` or `setgid` binaries t
hat could allow an attacker to perform privilege escalation and gain access to sensitive resourc
es. To mitigate this risk, it's recommended to add a `securityContext` to the container in the p
od, with the parameter `allowPrivilegeEscalation` set to `false`. This will prevent the containe
r from running any privileged processes and limit the impact of any potential attacks. By adding
 the `allowPrivilegeEscalation` parameter to your the `securityContext`, you can help to ensure
that your containerized applications are more secure and less vulnerable to privilege escalation
 attacks.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-732: Incorrect Permission Assignment for Critical Resource'
        cwe2021-top25: true
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
            - A06:2017 - Security Misconfiguration
        references:
            - https://kubernetes.io/docs/concepts/policy/pod-security-policy/#privilege-escalati
on
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://kubernetes.io/docs/tasks/configure-pod-container/security-context/
            - https://www.kernel.org/doc/Documentation/prctl/no_new_privs.txt
            - https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html#ru
le-4-add-no-new-privileges-flag
        subcategory:
            - vuln
        technology:
            - kubernetes
      patterns:
        - pattern-inside: |
            containers:
              ...
        - pattern-inside: |
            - name: $CONTAINER
              ...
        - pattern: |
            image: ...
            ...
        - pattern-inside: |
            image: ...
            ...
            $SC:
              ...
        - metavariable-regex:
            metavariable: $SC
            regex: ^(securityContext)$
        - pattern-not-inside: |
            image: ...
            ...
            securityContext:
              ...
              allowPrivilegeEscalation: $VAL
        - focus-metavariable: $SC
      severity: WARNING
    - id: yaml.kubernetes.security.exposing-docker-socket-hostpath.exposing-docker-socket-hostpa
th
      languages:
        - yaml
      message: Exposing host's Docker socket to containers via a volume. The owner of this socke
t is root. Giving someone access to it is equivalent to giving unrestricted root access to your
host. Remove 'docker.sock' from hostpath to prevent this.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-250: Execution with Unnecessary Privileges'
        impact: HIGH
        likelihood: LOW
        references:
            - https://kubernetes.io/docs/concepts/storage/volumes/#hostpath
            - https://kubernetes.io/docs/concepts/policy/pod-security-policy/#volumes-and-file-s
ystems
            - https://kubernetes.io/docs/tasks/configure-pod-container/security-context/
            - https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html#ru
le-1-do-not-expose-the-docker-daemon-socket-even-to-the-containers
        subcategory:
            - vuln
        technology:
            - kubernetes
      patterns:
        - pattern-inside: |
            volumes:
              ...
        - pattern: |
            hostPath:
              ...
              path: /var/run/docker.sock
      severity: WARNING
    - id: yaml.kubernetes.security.legacy-api-clusterrole-excessive-permissions.legacy-api-clust
errole-excessive-permissions
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - yaml
      message: 'Semgrep detected a Kubernetes core API ClusterRole with excessive permissions. A
ttaching excessive permissions to a ClusterRole associated with the core namespace allows the V1
 API to perform arbitrary actions on arbitrary resources attached to the cluster. Prefer explici
t allowlists of verbs/resources when configuring the core API namespace. '
      metadata:
        category: security
        confidence: HIGH
        cwe:
            - 'CWE-269: Improper Privilege Management'
        cwe2021-top25: false
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
            - A06:2017 - Security Misconfiguration
        references:
            - https://kubernetes.io/docs/reference/access-authn-authz/rbac/#role-and-clusterrole
            - https://kubernetes.io/docs/concepts/security/rbac-good-practices/#general-good-pra
ctice
            - https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.28/#api-groups
        subcategory:
            - vuln
        technology:
            - kubernetes
      patterns:
        - pattern: |
            "*"
        - pattern-inside: |
            resources: $A
            ...
        - pattern-inside: |
            verbs: $A
            ...
        - pattern-inside: |
            - apiGroups: [""]
              ...
        - pattern-inside: |
            apiVersion: rbac.authorization.k8s.io/v1
            ...
        - pattern-inside: |
            kind: ClusterRole
            ...
      severity: WARNING
    - id: yaml.kubernetes.security.privileged-container.privileged-container
      languages:
        - yaml
      message: Container or pod is running in privileged mode. This grants the container the equ
ivalent of root capabilities on the host machine. This can lead to container escapes, privilege
escalation, and other security concerns. Remove the 'privileged' key to disable this capability.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-250: Execution with Unnecessary Privileges'
        impact: MEDIUM
        likelihood: MEDIUM
        references:
            - https://kubernetes.io/docs/concepts/policy/pod-security-policy/#privileged
            - https://kubernetes.io/docs/tasks/configure-pod-container/security-context/
            - https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
        subcategory:
            - vuln
        technology:
            - kubernetes
      pattern-either:
        - patterns:
            - pattern-inside: |
                containers:
                  ...
            - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                image: ...
                ...
                securityContext:
                  ...
                  privileged: true
        - patterns:
            - pattern-inside: |
                spec:
                  ...
            - pattern-not-inside: |
                image: ...
                ...
            - pattern: |
                privileged: true
      severity: WARNING
    - fix: |
        true
      id: yaml.kubernetes.security.run-as-non-root-unsafe-value.run-as-non-root-unsafe-value
      languages:
        - yaml
      message: When running containers in Kubernetes, it's important to ensure that they  are pr
operly secured to prevent privilege escalation attacks.  One potential vulnerability is when a c
ontainer is allowed to run  applications as the root user, which could allow an attacker to gain
  access to sensitive resources. To mitigate this risk, it's recommended to  add a `securityCont
ext` to the container, with the parameter `runAsNonRoot`  set to `true`. This will ensure that t
he container runs as a non-root user,  limiting the damage that could be caused by any potential
 attacks. By  adding a `securityContext` to the container in your Kubernetes pod, you can  help
to ensure that your containerized applications are more secure and  less vulnerable to privilege
 escalation attacks.
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-250: Execution with Unnecessary Privileges'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A05:2021 - Security Misconfiguration
            - A06:2017 - Security Misconfiguration
        references:
            - https://kubernetes.io/blog/2016/08/security-best-practices-kubernetes-deployment/
            - https://kubernetes.io/docs/concepts/policy/pod-security-policy/
            - https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html#ru
le-2-set-a-user
        subcategory:
            - audit
        technology:
            - kubernetes
      patterns:
        - pattern-either:
            - pattern: |
                spec:
                  ...
                  securityContext:
                    ...
                    runAsNonRoot: $VALUE
            - patterns:
                - pattern-inside: |
                    containers:
                      ...
                - pattern: |
                    image: ...
                    ...
                    securityContext:
                      ...
                      runAsNonRoot: $VALUE
        - metavariable-pattern:
            metavariable: $VALUE
            pattern: |
                false
        - focus-metavariable: $VALUE
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      severity: INFO
    - id: yaml.kubernetes.security.seccomp-confinement-disabled.seccomp-confinement-disabled
      languages:
        - yaml
      message: 'Container is explicitly disabling seccomp confinement. This runs the service in
an unrestricted state. Remove ''seccompProfile: unconfined'' to prevent this.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-284: Improper Access Control'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A05:2017 - Broken Access Control
            - A01:2021 - Broken Access Control
        references:
            - https://kubernetes.io/docs/concepts/policy/pod-security-policy/#seccomp
            - https://kubernetes.io/docs/tasks/configure-pod-container/security-context/
        subcategory:
            - vuln
        technology:
            - kubernetes
      patterns:
        - pattern-inside: |
            containers:
              ...
        - pattern: |
            image: ...
            ...
            securityContext:
              ...
              seccompProfile: unconfined
      severity: WARNING
    - id: yaml.kubernetes.security.secrets-in-config-file.secrets-in-config-file
      languages:
        - yaml
      message: 'Secrets ($VALUE) should not be stored in infrastructure as code files. Use an al
ternative such as Bitnami Sealed Secrets or KSOPS to encrypt Kubernetes Secrets. '
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-798: Use of Hard-coded Credentials'
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A07:2021 - Identification and Authentication Failures
        references:
            - https://kubernetes.io/docs/concepts/configuration/secret/
            - https://media.defense.gov/2021/Aug/03/2002820425/-1/-1/0/CTR_Kubernetes_Hardening_
Guidance_1.1_20220315.PDF
            - https://docs.gitlab.com/ee/user/clusters/agent/gitops/secrets_management.html
            - https://www.cncf.io/blog/2021/04/22/revealing-the-secrets-of-kubernetes-secrets/
            - https://github.com/bitnami-labs/sealed-secrets
            - https://www.cncf.io/blog/2022/01/25/secrets-management-essential-when-using-kubern
etes/
            - https://blog.oddbit.com/post/2021-03-09-getting-started-with-ksops/
        subcategory:
            - vuln
        technology:
            - kubernetes
      patterns:
        - pattern: |
            $KEY: $VALUE
        - pattern-inside: |
            data: ...
        - pattern-inside: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            kind: Secret
            ...
        - metavariable-regex:
            metavariable: $VALUE
            regex: (?i)^[aA-zZ0-9+/]+={0,2}$
        - metavariable-analysis:
            analyzer: entropy
            metavariable: $VALUE
      severity: WARNING
    - id: yaml.kubernetes.security.skip-tls-verify-cluster.skip-tls-verify-cluster
      languages:
        - yaml
      message: 'Cluster is disabling TLS certificate verification when communicating with the se
rver. This makes your HTTPS connections insecure. Remove the ''insecure-skip-tls-verify: true''
key to secure communication.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://kubernetes.io/docs/reference/config-api/client-authentication.v1beta1/#cli
ent-authentication-k8s-io-v1beta1-Cluster
        subcategory:
            - vuln
        technology:
            - kubernetes
      pattern: |
        cluster:
          ...
          insecure-skip-tls-verify: true
      severity: WARNING
    - id: yaml.kubernetes.security.skip-tls-verify-service.skip-tls-verify-service
      languages:
        - yaml
      message: 'Service is disabling TLS certificate verification when communicating with the se
rver. This makes your HTTPS connections insecure. Remove the ''insecureSkipTLSVerify: true'' key
 to secure communication.'
      metadata:
        category: security
        confidence: MEDIUM
        cwe:
            - 'CWE-319: Cleartext Transmission of Sensitive Information'
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A03:2017 - Sensitive Data Exposure
            - A02:2021 - Cryptographic Failures
        references:
            - https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.20/#apiservice-v1
-apiregistration-k8s-io
        subcategory:
            - vuln
        technology:
            - kubernetes
      pattern: |
        spec:
          ...
          insecureSkipTLSVerify: true
      severity: WARNING
    - id: yaml.openapi.security.use-of-basic-authentication.use-of-basic-authentication
      languages:
        - yaml
      message: Basic authentication is considered weak and should be avoided.  Use a different a
uthentication scheme, such of OAuth2, OpenID Connect, or mTLS.
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: HIGH
        cwe: 'CWE-287: Improper Authentication'
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A04:2021 Insecure Design
            - A07:2021 Identification and Authentication Failures
        references:
            - https://cwe.mitre.org/data/definitions/287.html
            - https://owasp.org/Top10/A04_2021-Insecure_Design/
            - https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
        subcategory:
            - vuln
        technology:
            - openapi
      patterns:
        - pattern-inside: |
            openapi: $VERSION
            ...
            components:
              ...
              securitySchemes:
                ...
                $SCHEME:
                  ...
        - metavariable-regex:
            metavariable: $VERSION
            regex: 3.*
        - pattern: |
            type: http
            ...
            scheme: basic
      severity: ERROR
    - id: java_perm_rule-DangerousPermissions
      languages:
        - java
      message: |
        The application was found to permit the `RuntimePermission` of `createClassLoader`,
        `ReflectPermission` of `suppressAccessChecks`, or both.

        By granting the `RuntimePermission` of `createClassLoader`, a compromised application
        could instantiate their own class loaders and load arbitrary classes.

        By granting the `ReflectPermission` of `suppressAccessChecks` an application will no lon
ger
        check Java language access checks on fields and methods of a class. This will effectivel
y
        grant access to protected and private members.

        For more information on `RuntimePermission` see:
        https://docs.oracle.com/javase/8/docs/api/java/lang/RuntimePermission.html

        For more information on `ReflectPermission` see:
        https://docs.oracle.com/javase/8/docs/api/java/lang/reflect/ReflectPermission.html
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-732
        owasp:
            - A5:2017-Broken Access Control
            - A01:2021-Broken Access Control
        security-severity: Medium
        shortDescription: Incorrect permission assignment for critical resource
      pattern-either:
        - pattern: |
            $RUNVAR = new RuntimePermission("createClassLoader");
            ...
            (PermissionCollection $PC).add($RUNVAR);
        - pattern: |
            $REFVAR = new ReflectPermission("suppressAccessChecks");
            ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            (PermissionCollection $PC).add($REFVAR);
        - pattern: (PermissionCollection $PC).add(new ReflectPermission("suppressAccessChecks"))
        - pattern: (PermissionCollection $PC).add(new RuntimePermission("createClassLoader"))
      severity: WARNING
    - id: java_perm_rule-OverlyPermissiveFilePermissionInline
      languages:
        - java
      message: |
        The application was found setting file permissions to overly permissive values. Consider
        using the following values if the application user is the only process to access
        the file:

        - `r--` - read only access to the file
        - `w--` - write only access to the file
        - `rw-` - read/write access to the file

        Example setting read/write permissions for only the owner of a `Path`:
        ```
        // Get a reference to the path
        Path path = Paths.get("/tmp/somefile");
        // Create a PosixFilePermission set from java.nio.file.attribute
        Set<PosixFilePermission> permissions =
        java.nio.file.attribute.PosixFilePermissions.fromString("rw-------");
        // Set the permissions
        java.nio.file.Files.setPosixFilePermissions(path, permissions);
        ```

        For all other values please see:
        https://en.wikipedia.org/wiki/File-system_permissions#Symbolic_notation
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-732
        owasp:
            - A5:2017-Broken Access Control
            - A01:2021-Broken Access Control
        security-severity: Medium
        shortDescription: Incorrect permission assignment for critical resource
      patterns:
        - pattern-either:
            - pattern: java.nio.file.Files.setPosixFilePermissions(..., java.nio.file.attribute.
PosixFilePermissions.fromString("$PERM_STRING"));
            - pattern: |
                $PERMISSIONS = java.nio.file.attribute.PosixFilePermissions.fromString("$PERM_ST
RING");
                ...
                java.nio.file.Files.setPosixFilePermissions(..., $PERMISSIONS);
        - metavariable-regex:
            metavariable: $PERM_STRING
            regex: '[rwx-]{6}[rwx]{1,}'
      severity: WARNING
    - id: java_strings_rule-BadHexConversion
      languages:
        - java
      message: |
        The application is using `Integer.toHexString` on a digest array buffer which
        may lead to an incorrect version of values.

        Consider using the `java.util.HexFormat` object introduced in Java 17. For older Java ap
plications
        consider using the `javax.xml.bind.DatatypeConverter`.

        Example using `HexFormat` to create a human-readable string:
        ```
        // Create a MessageDigest using the SHA-384 algorithm
        MessageDigest sha384Digest = MessageDigest.getInstance("SHA-384");
        // Call update with your data
        sha384Digest.update("some input".getBytes(StandardCharsets.UTF_8));
        // Only call digest once all data has been fed into the update sha384digest instance
        byte[] output = sha384Digest.digest();
        // Create a JDK 17 HexFormat object
        HexFormat hex = HexFormat.of();
```

CODEX/.codacy/tools-configs/semgrep.yaml
```

```
        // Use formatHex on the byte array to create a string (note that alphabet characters are
        lowercase)
        String hexString = hex.formatHex(output);
        ```

        For more information on DatatypeConverter see:
        https://docs.oracle.com/javase/9/docs/api/javax/xml/bind/DatatypeConverter.html#printHex
Binary-byte:A-
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-704
        owasp:
            - A6:2017-Security Misconfiguration
            - A05:2021-Security Misconfiguration
        security-severity: Info
        shortDescription: Incorrect type conversion or cast
      patterns:
        - pattern-inside: |
            $B_ARR = (java.security.MessageDigest $MD).digest(...);
            ...
        - pattern-either:
            - pattern: |
                for(...) {
                  ...
                  $B = $B_ARR[...];
                  ...
                  Integer.toHexString($B);
                }
            - pattern: |
                for(...) {
                  ...
                  Integer.toHexString($B_ARR[...]);
                }
            - pattern: |
                for(byte $B :$B_ARR) {
                  ...
                  Integer.toHexString($B);
                }
            - pattern: |
                while(...) {
                  ...
                  Integer.toHexString($B_ARR[...])
                }
            - pattern: |
                do {
                  ...
                  Integer.toHexString($B_ARR[...])
                } while(...)
            - pattern: |
                while(...) {
                  ...
                  $B = $B_ARR[...];
                  ...
                  Integer.toHexString($B);
                }
            - pattern: |
                do {
                  ...
                  $B = $B_ARR[...];
                  ...
                  Integer.toHexString($B);
                } while(...)
      severity: WARNING
    - id: java_strings_rule-FormatStringManipulation
      languages:
        - java
      message: |
        The application allows user input to control format string parameters. By passing invali
d
        format
```

CODEX/.codacy/tools-configs/semgrep.yaml
```

```
        string specifiers an adversary could cause the application to throw exceptions or possib
ly
        leak
        internal information depending on application logic.

        Never allow user-supplied input to be used to create a format string. Replace all format
        string
        arguments with hardcoded format strings containing the necessary specifiers.

        Example of using `String.format` safely:
        ```
        // Get untrusted user input
        String userInput = request.getParameter("someInput");
        // Ensure that user input is not included in the first argument to String.format
        String.format("Hardcoded string expecting a string: %s", userInput);
        // ...
        ```
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-134
        owasp:
            - A1:2017-Injection
            - A03:2021-Injection
        security-severity: Medium
        shortDescription: Use of externally-controlled format string
      patterns:
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    String $INPUT = (HttpServletRequest $REQ).getParameter(...);
                    ...
                - pattern-inside: |
                    String $FORMAT_STR = ... + $INPUT;
                    ...
            - patterns:
                - pattern-inside: |
                    String $INPUT = (HttpServletRequest $REQ).getParameter(...);
                    ...
                - pattern-inside: |
                    String $FORMAT_STR = ... + $INPUT + ...;
                    ...
            - pattern-inside: |
                String $FORMAT_STR = ... + (HttpServletRequest $REQ).getParameter(...) + ...;
                ...
            - pattern-inside: |
                String $FORMAT_STR = ... + (HttpServletRequest $REQ).getParameter(...);
                ...
        - pattern-either:
            - pattern: String.format($FORMAT_STR, ...);
            - pattern: String.format(java.util.Locale.$LOCALE, $FORMAT_STR, ...);
            - pattern: (java.util.Formatter $F).format($FORMAT_STR, ...);
            - pattern: (java.util.Formatter $F).format(java.util.Locale.$LOCALE, $FORMAT_STR, ..
.);
            - pattern: (java.io.PrintStream $F).printf($FORMAT_STR, ...);
            - pattern: (java.io.PrintStream $F).printf(java.util.Locale.$LOCALE, $FORMAT_STR, ..
.);
            - pattern: (java.io.PrintStream $F).format($FORMAT_STR, ...);
            - pattern: (java.io.PrintStream $F).format(java.util.Locale.$LOCALE, $FORMAT_STR, ..
.);
            - pattern: System.out.printf($FORMAT_STR, ...);
            - pattern: System.out.printf(java.util.Locale.$LOCALE, $FORMAT_STR, ...);
            - pattern: System.out.format($FORMAT_STR, ...);
            - pattern: System.out.format(java.util.Locale.$LOCALE, $FORMAT_STR, ...);
      severity: ERROR
    - id: java_strings_rule-ModifyAfterValidation
      languages:
        - java
      message: |+
        The application was found matching a variable during a regular expression
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        pattern match, and then calling string modification functions after validation has occur
red.
        This is usually indicative of a poor input validation strategy as an adversary may attem
pt to
        exploit the removal of characters.

        For example a common mistake in attempting to remove path characters to protect against
path
        traversal is to match '../' and then remove any matches. However, if an adversary were t
o
        include in their input: '..../' then the `replace`  method would replace the first `../
` but
        cause the leading `..` and trailing `/` to join into the final string of `../`, effectiv
ely
        bypassing the check.

        To remediate this issue always perform string modifications before any validation of a s
tring.
        It is strongly recommended that strings be encoded instead of replaced or removed prior
to
        validation.


        Example replaces `..` before validation. Do note this is still not a recommended method
for
        protecting against directory traversal, always use randomly generated IDs or filenames i
nstead:
        ```
        // This is ONLY for demonstration purpose, never use untrusted input
        // in paths, always use randomly generated filenames or IDs.
        String input = "test../..../dir";
        // Use replaceAll _not_ replace
        input = input.replaceAll("\\.\\.", "");
        // Input would be test///dir at this point
        // Create a pattern to match on
        Pattern pattern = Pattern.compile("\\.\\.");
        // Create a matcher
        Matcher match = pattern.matcher(input);
        // Call find to see if .. is still in our string
        if (match.find()) {
            throw new Exception(".. detected");
        }
        // Use the input (but do not modify the string)
        System.out.println(input + " safe");
        ```

        For more information see Carnegie Mellon University's Secure Coding Guide:
        https://wiki.sei.cmu.edu/confluence/display/java/IDS11-J.+Perform+any+string+modificatio
ns+before+validation

      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-182
        owasp:
            - A1:2017-Injection
            - A03:2021-Injection
        security-severity: Info
        shortDescription: Collapse of data into unsafe value
      patterns:
        - pattern: |
            (java.util.regex.Pattern $Y).matcher($VAR);
            ...
            $VAR.$METHOD(...);
        - metavariable-regex:
            metavariable: $METHOD
            regex: (replace|replaceAll|replaceFirst|concat)
      severity: WARNING
    - id: java_strings_rule-NormalizeAfterValidation
      languages:
        - java
      message: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        The application was found matching a variable during a regular expression
        pattern match, and then calling a Unicode normalize function after validation has occurr
ed.
        This is usually indicative of a poor input validation strategy as an adversary may attem
pt to
        exploit the normalization process.

        To remediate this issue, always perform Unicode normalization before any validation of a
        string.

        Example of normalizing a string before validation:
        ```
        // User input possibly containing malicious unicode
        String userInput = "\uFE64" + "tag" + "\uFE65";
        // Normalize the input
        userInput = Normalizer.normalize(userInput, Normalizer.Form.NFKC);
        // Compile our regex pattern looking for < or > characters
        Pattern pattern = Pattern.compile("[<>]");
        // Create a matcher from the userInput
        Matcher matcher = pattern.matcher(userInput);
        // See if the matcher matches
        if (matcher.find()) {
            // It did so throw an error
            throw new Exception("found banned characters in input");
        }
        ```

        For more information see Carnegie Mellon University's Secure Coding Guide:
        https://wiki.sei.cmu.edu/confluence/display/java/IDS01-J.+Normalize+strings+before+valid
ating+them
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-180
        owasp:
            - A1:2017-Injection
            - A03:2021-Injection
        security-severity: Info
        shortDescription: 'Incorrect behavior order: validate before canonicalize'
      patterns:
        - pattern: |
            $Y = java.util.regex.Pattern.compile("[<>]");
            ...
            $Y.matcher($VAR);
            ...
            java.text.Normalizer.normalize($VAR, ...);
      severity: WARNING
    - id: java_crypto_rule-DisallowOldTLSVersion
      languages:
        - java
      message: "This application sets the `jdk.tls.client.protocols` system property to\ninclude
 insecure TLS or SSL versions (SSLv3, TLSv1, TLSv1.1), which are\ndeprecated due to serious secu
rity vulnerabilities like POODLE attacks and\nsusceptibility to man-in-the-middle attacks. Conti
nuing to use these\nprotocols can expose data to interception or manipulation. \n\nTo mitigate t
he issue, upgrade to TLSv1.2 or higher, which provide stronger \nencryption and improved securit
y. Refrain from using any SSL versions as they \nare entirely deprecated.\n\nSecure Code Example
:\n```\npublic void safe() {\n  java.lang.System.setProperty(\"jdk.tls.client.protocols\", \"TLS
v1.3\");\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-326
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://stackoverflow.com/questions/26504653/is-it-possible-to-disable-sslv3-for-a
ll-java-applications
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        security-severity: MEDIUM
        shortDescription: Inadequate encryption strength
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
        vulnerability_class:
            - Mishandled Sensitive Information
      patterns:
        - pattern: $VALUE. ... .setProperty("jdk.tls.client.protocols", "$PATTERNS");
        - metavariable-pattern:
            language: generic
            metavariable: $PATTERNS
            patterns:
                - pattern-either:
                    - pattern-regex: ^(.*TLSv1|.*SSLv.*)$
                    - pattern-regex: ^(.*TLSv1,.*|.*TLSv1.1.*)
      severity: WARNING
    - id: java_crypto_rule-HTTPUrlConnectionHTTPRequest
      languages:
        - java
      message: "Detected an HTTP request sent via HttpURLConnection or URLConnection.\nThis coul
d lead to sensitive information being sent over an insecure \nchannel, as HTTP does not encrypt
data. Transmitting data over HTTP \nexposes it to potential interception by attackers, risking d
ata \nintegrity and confidentiality. Using HTTP for transmitting sensitive \ndata such as passwo
rds, personal information, or financial details can \nlead to information disclosure.\n\nTo miti
gate the issue, switch to HTTPS to ensure all data transmitted \nis securely encrypted. This hel
ps protect against eavesdropping and \nman-in-the-middle attacks. Modify the URL in your code fr
om HTTP to \nHTTPS and ensure the server supports HTTPS.\n\nSecure Code Example:\n```\nprivate s
tatic void safe() {\n    try {\n        URL url = new URL(\"https://example.com/api/data\"); //
Changed to HTTPS\n        HttpURLConnection con = (HttpURLConnection) url.openConnection();\n
     con.setRequestMethod(\"GET\");\n\n        int status = con.getResponseCode();\n        if (
status == HttpURLConnection.HTTP_OK) { \n            BufferedReader in = new BufferedReader(new
InputStreamReader(con.getInputStream()));\n            String inputLine;\n            StringBuil
der response = new StringBuilder();\n            while ((inputLine = in.readLine()) != null) {\n
                response.append(inputLine);\n            }\n            in.close();\n
 System.out.println(\"Response: \" + response.toString());\n        } else {\n            System
.out.println(\"HTTP error code: \" + status);\n        }\n        con.disconnect();\n    } catch
 (Exception e) {\n        e.printStackTrace();\n    }\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-319
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/net/URLConnectio
n.html
            - https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/net/URL.html#ope
nConnection()
        security-severity: MEDIUM
        shortDescription: Cleartext transmission of sensitive information
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
        vulnerability_class:
            - Mishandled Sensitive Information
      patterns:
        - pattern: |
            "=~/[Hh][Tt][Tt][Pp]://.*/"
        - pattern-either:
            - pattern-inside: |
                URL $URL = new URL ("=~/[Hh][Tt][Tt][Pp]://.*/", ...);
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                $CON = (HttpURLConnection) $URL.openConnection(...);
                ...
                $CON.$FUNC(...);
            - pattern-inside: |
                URL $URL = new URL ("=~/[Hh][Tt][Tt][Pp]://.*/", ...);
                ...
                $CON = $URL.openConnection(...);
                ...
                $CON.$FUNC(...);
      severity: WARNING
    - id: java_crypto_rule-HttpComponentsRequest
      languages:
        - java
      message: "Detected an HTTP GET request sent via Apache HTTP Components. Sending data\nover
 HTTP can expose sensitive information to interception or modification\nby attackers, as HTTP do
es not encrypt the data transmitted. It is critical\nto use HTTPS, which encrypts the communicat
ion, to protect the confidentiality\nand integrity of data in transit.\n\nTo mitigate the issue,
 ensure all data transmitted between the client and \nserver is sent over HTTPS. Update all HTTP
 URLs to HTTPS and configure your \nserver to redirect HTTP requests to HTTPS. Additionally, imp
lement HSTS \n(HTTP Strict Transport Security) to enforce secure connections.\nSecure Code Examp
le:\n```\nprivate static void safe() {\n  CloseableHttpClient httpclient = HttpClients.createDef
ault();\n  CloseableHttpResponse response = httpclient.execute(new HttpPost(\"https://example.co
m\"));\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-319
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://hc.apache.org/httpcomponents-client-ga/quickstart.html
        security-severity: MEDIUM
        shortDescription: Cleartext transmission of sensitive information
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
        vulnerability_class:
            - Mishandled Sensitive Information
      mode: taint
      pattern-sinks:
        - pattern: (org.apache.http.impl.client.CloseableHttpClient $A).execute($HTTPREQ);
      pattern-sources:
        - pattern: |
            "=~/^http://.+/i"
      severity: WARNING
    - id: java_crypto_rule-HttpGetHTTPRequest
      languages:
        - java
      message: "Detected an HTTP GET request sent via HttpGet. Sending data over HTTP can\nexpos
e sensitive information to interception or modification by attackers,\nas HTTP does not encrypt
the data transmitted. It is critical to use\nHTTPS, which encrypts the communication, to protect
 the confidentiality\nand integrity of data in transit.\n\nTo mitigate the issue, ensure all dat
a transmitted between the client and \nserver is sent over HTTPS. Update all HTTP URLs to HTTPS
and configure your \nserver to redirect HTTP requests to HTTPS. Additionally, implement HSTS \n(
HTTP Strict Transport Security) to enforce secure connections.\n\nSecure Code Example:\n```\npri
vate static void safe() throws IOException {\n  HttpGet httpGet = new HttpGet(\"https://example.
com\");\n  HttpClients.createDefault().execute(httpGet);\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-319
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/net/URLConnectio
n.html
            - https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/net/URL.html#ope
nConnection()
        security-severity: MEDIUM
        shortDescription: Cleartext transmission of sensitive information
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
        vulnerability_class:
            - Mishandled Sensitive Information
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                $R = new org.apache.http.client.methods.HttpGet($PROT);
                ...
                $CLIENT. ... .execute($R, ...);
            - focus-metavariable: $PROT
      pattern-sources:
        - pattern: |
            "=~/^http:\/\/.+/i"
      severity: WARNING
    - id: java_crypto_rule_JwtDecodeWithoutVerify
      languages:
        - java
      message: Detected the decoding of a JWT token without a verify step. JWT tokens must be ve
rified before use, otherwise the token's integrity is unknown. This means a malicious actor coul
d forge a JWT token with any claims. Call '.verify()' before using the token.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-347
        impact: HIGH
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A8:2017-Insecure Deserialization
            - A08:2021-Software and Data Integrity Failures
        references: https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures
        security-severity: MEDIUM
        shortDescription: Improper verification of cryptographic signature
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory: vuln
        technology: jwt
        vulnerability_class: Improper Authentication
      patterns:
        - pattern: |
            com.auth0.jwt.JWT.decode(...);
        - pattern-not-inside: |-
            class $CLASS {
              ...
              $RETURNTYPE $FUNC (...) {
                ...
                $VERIFIER.verify(...);
                ...
              }
            }
      severity: WARNING
    - id: java_crypto_rule-SpringFTPRequest
      languages:
        - java
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: "This pattern detects configurations where the Spring Integration FTP plugin \nis
 used to set up connections to FTP servers. FTP is an insecure protocol \nthat transmits data, i
ncluding potentially sensitive information, in plaintext. \nThis can expose personal identifiabl
e information (PII) or other sensitive data \nto interception by attackers during transmission.
\n\nTo mitigate the vulnerability, switch to a secure protocol such as SFTP or FTPS \nthat encry
pts the connection to prevent data exposure. Ensure that any method \nused to set the host for a
n FTP session does not use plaintext FTP. \n\nSecure Code Example:\n```\npublic SessionFactory<F
TPFile> safe(FtpSessionFactoryProperties properties) {\n  DefaultFtpSessionFactory ftpSessionFac
tory = new DefaultFtpSessionFactory();\n  ftpSessionFactory.setHost(\"sftp://example.com\");\n
ftpSessionFactory.setPort(properties.getPort());\n  ftpSessionFactory.setUsername(properties.get
Username());\n  ftpSessionFactory.setPassword(properties.getPassword());\n  ftpSessionFactory.se
tClientMode(properties.getClientMode().getMode());\n  return ftpSessionFactory;\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-319
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://docs.spring.io/spring-integration/api/org/springframework/integration/ftp/
session/AbstractFtpSessionFactory.html#setClientMode-int-
        security-severity: MEDIUM
        shortDescription: Cleartext transmission of sensitive information
        subcategory:
            - vuln
        technology:
            - spring
        vulnerability: Insecure Transport
        vulnerability_class:
            - Mishandled Sensitive Information
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                (org.springframework.integration.ftp.session.DefaultFtpSessionFactory
                $SF).setHost($URL);
            - focus-metavariable: $URL
      pattern-sources:
        - pattern: |
            "=~/^ftp://.+/i"
      severity: WARNING
    - id: java_crypto_rule-SpringHTTPRequestRestTemplate
      languages:
        - java
      message: "This rule detects instances where Java Spring's RestTemplate API sends \nrequest
s to non-secure (http://) URLs. Sending data over HTTP is vulnerable \nas it does not use TLS en
cryption, exposing the data to interception, \nmodification, or redirection by attackers. \n\nTo
 mitigate this vulnerability, modify the request URLs to use HTTPS instead, \nwhich ensures that
 the data is encrypted during transit and prevents from\nMITM attacks. \n\nSecure Code Example:\
n```\npublic void safe(Object obj) throws Exception {\n  RestTemplate restTemplate = new RestTem
plate();\n  restTemplate.put(URI.create(\"https://example.com\"), obj);\n}\n```  \n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-319
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/web/cli
ent/RestTemplate.html#delete-java.lang.String-java.util.Map-
            - https://www.baeldung.com/rest-template
        security-severity: MEDIUM
        shortDescription: Cleartext transmission of sensitive information
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        subcategory:
            - vuln
        technology:
            - spring
        vulnerability: Insecure Transport
        vulnerability_class:
            - Mishandled Sensitive Information
      mode: taint
      pattern-sinks:
        - patterns:
            - pattern: |
                (org.springframework.web.client.RestTemplate $RESTTEMP).$FUNC($URL, ...);
            - focus-metavariable: $URL
            - metavariable-regex:
                metavariable: $FUNC
                regex: (delete|doExecute|exchange|getForEntity|getForObject|headForHeaders|optio
nsForAllow|patchForObject|postForEntity|postForLocation|postForObject|put|execute)
      pattern-sources:
        - pattern: |
            "=~/^http:\/\/.+/i"
      severity: WARNING
    - id: java_crypto_rule-TLSUnsafeRenegotiation
      languages:
        - java
      message: "This code enables unsafe renegotiation in SSL/TLS connections, which is\nvulnera
ble to man-in-the-middle attacks. In such attacks, an attacker\ncould inject chosen plaintext at
 the beginning of the secure\ncommunication, potentially compromising the security of data trans
mission. If \nexploited, this vulnerability can lead to unauthorized access to sensitive \ndata,
 data manipulation, and potentially full system compromise depending on \nthe data and operation
s protected by the TLS session.\n\nTo mitigate this vulnerability, disable unsafe renegotiation
in the Java \napplication. Ensure that only secure renegotiation is allowed by setting the \nsys
tem property `sun.security.ssl.allowUnsafeRenegotiation` to `false`. \n\nSecure code example:\n`
``\npublic void safe() {\n  java.lang.System.setProperty(\"sun.security.ssl.allowUnsafeRenegotia
tion\", false);\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-319
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://www.oracle.com/java/technologies/javase/tlsreadme.html
        security-severity: MEDIUM
        shortDescription: Cleartext transmission of sensitive information
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
        vulnerability_class:
            - Mishandled Sensitive Information
      patterns:
        - pattern: |
            java.lang.System.setProperty("sun.security.ssl.allowUnsafeRenegotiation", $TRUE);
        - metavariable-pattern:
            metavariable: $TRUE
            pattern-either:
                - pattern: |
                    true
                - pattern: |
                    "true"
                - pattern: |
                    Boolean.TRUE
      severity: WARNING
    - id: java_crypto_rule-TelnetRequest
      languages:
        - java
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: "Checks for attempts to connect through telnet. Telnet is an outdated\nprotocol t
hat transmits all data, including sensitive information like\npasswords, in clear text. This exp
oses it to interception and\neavesdropping on unsecured networks.\n\nTo mitigate this issue, rep
lace Telnet usage with more secure protocols \nsuch as SSH (Secure Shell), which provides encryp
ted communication. Use \nthe SSH functionality provided by libraries like JSch or Apache MINA SS
HD \nfor secure data transmission.\n\nSecure Code Example:\n```\nimport com.jcraft.jsch.JSch;\ni
mport com.jcraft.jsch.Session;\n\npublic class SecureConnector {\n    public static void main(St
ring[] args) {\n        try {\n            JSch jsch = new JSch();\n            Session session
= jsch.getSession(\"username\", \"hostname\", 22);\n            session.setPassword(\"password\"
);\n            session.setConfig(\"StrictHostKeyChecking\", \"no\");\n            session.conne
ct();\n            System.out.println(\"Connected securely.\");\n        } catch (Exception e) {
\n            System.err.println(\"Secure connection failed: \" + e.getMessage());\n        }\n
   }\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-319
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://commons.apache.org/proper/commons-net/javadocs/api-3.6/org/apache/commons/
net/telnet/TelnetClient.html
        security-severity: MEDIUM
        shortDescription: Cleartext transmission of sensitive information
        subcategory:
            - vuln
        technology:
            - java
        vulnerability: Insecure Transport
        vulnerability_class:
            - Mishandled Sensitive Information
      pattern: |
        (org.apache.commons.net.telnet.TelnetClient $TELNETCLIENT).connect(...);
      severity: WARNING
    - id: java_crypto_rule-UnirestHTTPRequest
      languages:
        - java
      message: "This application uses the Unirest library to send\nnetwork requests to URLs star
ting with 'http://'. Communicating over HTTP\nis considered insecure because it does not encrypt
 traffic with TLS\n(Transport Layer Security), exposing data to potential interception or\nmanip
ulation by attackers.\n\nTo mitigate the issue, modify the request URL to begin with 'https://'
\ninstead of 'http://'. Using HTTPS ensures that the data is encrypted and \nsecure during trans
mission. Review all instances where HTTP is used and \nupdate them to use HTTPS to prevent secur
ity risks.\n\nSecure Code Example:\n```\nimport kong.unirest.core.Unirest;\n\npublic void safe()
 {\n  Unirest.get(\"https://httpbin.org\")\n      .queryString(\"fruit\", \"apple\")\n      .que
ryString(\"droid\", \"R2D2\")\n      .asString();\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-319
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://kong.github.io/unirest-java/#requests
        security-severity: MEDIUM
        shortDescription: Cleartext transmission of sensitive information
        subcategory:
            - vuln
        technology:
            - unirest
        vulnerability: Insecure Transport
        vulnerability_class:
            - Mishandled Sensitive Information
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      patterns:
        - pattern: |
            Unirest.$METHOD("=~/[hH][tT][tT][pP]://.*/")
      severity: WARNING
    - id: java_crypto_rule-UseOfRC2
      languages:
        - java
      message: "Use of RC2, a deprecated cryptographic algorithm vulnerable to related-key\natta
cks, was detected. Modern cryptographic standards recommend the\nadoption of algorithms that int
egrate message integrity to ensure the\nciphertext remains unaltered.\n\nTo mitigate the issue,
use any of the below algorithms instead:\n1. `ChaCha20Poly1305` - Preferred for its simplicity a
nd speed, suitable for \nenvironments where cryptographic acceleration is absent.\n2. `AES-256-G
CM` - Highly recommended when hardware support is available, \ndespite being somewhat slower tha
n `ChaCha20Poly1305`. It is crucial to avoid \nnonce reuse with AES-256-GCM to prevent security
compromises.\n\nSecure code example using `ChaCha20Poly1305` in Java:\n```\npublic void encryptA
ndDecrypt() throws Exception {\n    SecureRandom random = new SecureRandom();\n    byte[] secret
Key = new byte[32];  // 256-bit key\n    byte[] nonce = new byte[12];      // 96-bit nonce\n
random.nextBytes(secretKey);\n    random.nextBytes(nonce);\n\n    Cipher cipher = Cipher.getInst
ance(\"ChaCha20-Poly1305/None/NoPadding\");\n    SecretKeySpec keySpec = new SecretKeySpec(secre
tKey, \"ChaCha20\");\n    GCMParameterSpec spec = new GCMParameterSpec(128, nonce);\n\n    ciphe
r.init(Cipher.ENCRYPT_MODE, keySpec, spec);\n    byte[] plaintext = \"Secret text\".getBytes(Sta
ndardCharsets.UTF_8);\n    byte[] ciphertext = cipher.doFinal(plaintext);\n    System.out.printl
n(\"Encrypted: \" + Base64.getEncoder().encodeToString(ciphertext));\n\n    cipher.init(Cipher.D
ECRYPT_MODE, keySpec, spec);\n    byte[] decrypted = cipher.doFinal(ciphertext);\n    System.out
.println(\"Decrypted: \" + new String(decrypted, StandardCharsets.UTF_8));\n}\n```\nFor more on
Java Cryptography, refer:\nhttps://docs.oracle.com/en/java/javase/15/security/java-cryptography-
architecture-jca-reference-guide.html\n"
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-327
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        security-severity: MEDIUM
        shortDescription: Use of a broken or risky cryptographic algorithm
        subcategory:
            - vuln
        technology:
            - java
      pattern-either:
        - pattern: |
            javax.crypto.Cipher.getInstance("RC2")
        - patterns:
            - pattern-inside: |
                class $CLS{
                  ...
                  String $ALG = "RC2";
                  ...
                }
            - pattern: |
                javax.crypto.Cipher.getInstance($ALG);
      severity: WARNING
    - id: java_crypto_rule-UseOfRC4
      languages:
        - java
```

```
      message: "Use of RC4 was detected. RC4 is vulnerable to several types of attacks,\nincludi
ng stream cipher attacks where attackers can recover plaintexts by\nanalyzing a large number of
encrypted messages, and bit-flipping attacks\nthat can alter messages without knowing the encryp
tion key.\n\nTo mitigate the issue, use any of the below algorithms instead:\n1. `ChaCha20Poly13
05` - Preferred for its simplicity and speed, suitable for \nenvironments where cryptographic ac
celeration is absent.\n2. `AES-256-GCM` - Highly recommended when hardware support is available,
 \ndespite being somewhat slower than `ChaCha20Poly1305`. It is crucial to avoid \nnonce reuse w
ith AES-256-GCM to prevent security compromises.\n\nSecure code example using `ChaCha20Poly1305`
 in Java:\n```\npublic void encryptAndDecrypt() throws Exception {\n    SecureRandom random = ne
w SecureRandom();\n    byte[] secretKey = new byte[32];  // 256-bit key\n    byte[] nonce = new
byte[12];      // 96-bit nonce\n    random.nextBytes(secretKey);\n    random.nextBytes(nonce);\n
\n    Cipher cipher = Cipher.getInstance(\"ChaCha20-Poly1305/None/NoPadding\");\n    SecretKeySp
ec keySpec = new SecretKeySpec(secretKey, \"ChaCha20\");\n    GCMParameterSpec spec = new GCMPar
ameterSpec(128, nonce);\n\n    cipher.init(Cipher.ENCRYPT_MODE, keySpec, spec);\n    byte[] plai
ntext = \"Secret text\".getBytes(StandardCharsets.UTF_8);\n    byte[] ciphertext = cipher.doFina
l(plaintext);\n    System.out.println(\"Encrypted: \" + Base64.getEncoder().encodeToString(ciphe
rtext));\n\n    cipher.init(Cipher.DECRYPT_MODE, keySpec, spec);\n    byte[] decrypted = cipher.
doFinal(ciphertext);\n    System.out.println(\"Decrypted: \" + new String(decrypted, StandardCha
rsets.UTF_8));\n}\n```\nFor more information, refer:\nhttps://owasp.org/www-community/Using_the_
Java_Cryptographic_Extensions\n"
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-327
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
            - https://googleprojectzero.blogspot.com/2022/10/rc4-is-still-considered-harmful.htm
l
        security-severity: MEDIUM
        shortDescription: Use of a broken or risky cryptographic algorithm
        subcategory:
            - vuln
        technology:
            - java
      pattern-either:
        - pattern: |
            javax.crypto.Cipher.getInstance("RC4")
        - patterns:
            - pattern-inside: |
                class $CLS{
                  ...
                  String $ALG = "RC4";
                  ...
                }
            - pattern: |
                javax.crypto.Cipher.getInstance($ALG);
      severity: WARNING
    - id: java_deserialization_rule-InsecureJmsDeserialization
      languages:
        - java
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: "Deserialization of untrusted JMS ObjectMessage can lead to arbitrary \ncode exec
ution. This vulnerability occurs when `ObjectMessage.getObject()` \nis called to deserialize the
 payload of an ObjectMessage, potentially \nallowing remote attackers to execute arbitrary code
with the permissions \nof the JMS MessageListener application. \n\nTo mitigate the issue, avoid
deserialization of untrusted data and \nconsider alternative message formats or explicit whiteli
sting of \nallowable classes for deserialization.\n\nTo implement allowlisting, override the Obj
ectInputStream#resolveClass() \nmethod to limit deserialization to allowed classes only. This pr
events \ndeserialization of any class except those explicitly permitted, such as \nin the follow
ing example that restricts deserialization to the Bicycle \nclass only:\n\n```\n// Code from htt
ps://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html\npublic class LookA
headObjectInputStream extends ObjectInputStream {\n    public LookAheadObjectInputStream(InputSt
ream inputStream) throws IOException {\n        super(inputStream);\n    }\n    /**\n    * Only
deserialize instances of our expected Bicycle class\n    */\n    @Override\n    protected Class<
?> resolveClass(ObjectStreamClass desc) throws IOException, ClassNotFoundException {\n        if
 (!desc.getName().equals(Bicycle.class.getName())) {\n            throw new InvalidClassExceptio
n(\"Unauthorized deserialization attempt\", desc.getName());\n        }\n        return super.re
solveClass(desc);\n    }\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-502
        cwe2021-top25: "true"
        cwe2022-top25: "true"
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A8:2017-Insecure Deserialization
            - A08:2021-Software and Data Integrity Failures
        references:
            - https://www.blackhat.com/docs/us-16/materials/us-16-Kaiser-Pwning-Your-Java-Messag
ing-With-Deserialization-Vulnerabilities-wp.pdf
        security-severity: High
        shortDescription: Deserialization of untrusted data
        subcategory:
            - vuln
        technology:
            - java
        vulnerability_class:
            - 'Insecure Deserialization '
      patterns:
        - pattern-inside: |
            class $JMS_LISTENER implements MessageListener {
              ...
              public void onMessage(Message $JMS_MSG) {
                  ...
              }
            }
        - pattern: $Y.getObject(...);
      severity: ERROR
    - id: java_endpoint_rule-ManuallyConstructedURLs
      languages:
        - java
      message: |
        User data flows into the host portion of this manually-constructed URL.
        This could allow an attacker to send data to their own server, potentially
        exposing sensitive data such as cookies or authorization information sent
        with this request. They could also probe internal servers or other
        resources that the server running this code can access. (This is called
        server-side request forgery, or SSRF.) Do not allow arbitrary hosts.
        Instead, create an allowlist for approved hosts hardcode the correct host,
        or ensure that the user data can only affect the path or parameters.

        Example of using allowlist:
        ```
        ArrayList<String> allowlist = (ArrayList<String>)
            Arrays.asList(new String[] { "https://example.com/api/1", "https://example.com/api/2
", "https://example.com/api/3"});

        if(allowlist.contains(url)){
          ...
```

CODEX/.codacy/tools-configs/semgrep.yaml
```

```
        }
        ```
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-918
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        interfile: true
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A1:2017-Injection
            - A10:2021-Server-Side Request Forgery
        references:
            - https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevent
ion_Cheat_Sheet.html
        security-severity: CRITICAL
        shortDescription: Detect manually constructed URLs
        subcategory:
            - vuln
        technology:
            - java
            - spring
        vulnerability_class:
            - Server-Side Request Forgery (SSRF)
      mode: taint
      options:
        interfile: true
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern: "if($VALIDATION){\n ...\n new URL($ONEARG);\n ...\n} \n"
                - pattern: |
                    $A = $VALIDATION;
                    ...
                    if($A){
                      ...
                      new URL($ONEARG);
                      ...
                    }
            - metavariable-pattern:
                metavariable: $VALIDATION
                pattern-either:
                    - pattern: "$AL.contains(...) \n"
                    - pattern: |
                        $AL.indexOf(...) != -1
      pattern-sinks:
        - pattern-either:
            - pattern: new URL($ONEARG)
            - patterns:
                - pattern-either:
                    - pattern: |
                        "$URLSTR" + ...
                    - pattern: |
                        "$URLSTR".concat(...)
                    - patterns:
                        - pattern-inside: |
                            StringBuilder $SB = new StringBuilder("$URLSTR");
                            ...
                        - pattern: $SB.append(...)
                    - patterns:
                        - pattern-inside: |
                            $VAR = "$URLSTR";
                            ...
                        - pattern: $VAR += ...
                    - patterns:
                        - pattern: String.format("$URLSTR", ...)
                        - pattern-not: String.format("$URLSTR", "...", ...)
                    - patterns:
```

CODEX/.codacy/tools-configs/semgrep.yaml
```

```
                        - pattern-inside: |
                            String $VAR = "$URLSTR";
                            ...
                        - pattern: String.format($VAR, ...)
                - metavariable-regex:
                    metavariable: $URLSTR
                    regex: http(s?)://%(v|s|q).*
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern-inside: |
                    $METHODNAME(..., @$REQ(...) $TYPE $SOURCE,...) {
                      ...
                    }
                - pattern-inside: |
                    $METHODNAME(..., @$REQ $TYPE $SOURCE,...) {
                      ...
                    }
            - metavariable-regex:
                metavariable: $TYPE
                regex: ^(?!(Integer|Long|Float|Double|Char|Boolean|int|long|float|double|char|bo
olean))
            - metavariable-regex:
                metavariable: $REQ
                regex: (RequestBody|PathVariable|RequestParam|RequestHeader|CookieValue|ModelAtt
ribute)
            - focus-metavariable: $SOURCE
      severity: ERROR
    - id: java_file_rule_rule-FilePathTraversalHttpServlet
      languages:
        - java
      message: "Detected a potential path traversal. A malicious actor could control\nthe locati
on of this file, to include going backwards in the directory\nwith '../'. \n\nTo address this, e
nsure that user-controlled variables in file\npaths are sanitized. You may also consider using a
 utility method such as\norg.apache.commons.io.FilenameUtils.getName(...) to only retrieve the f
ile\nname from the path.\n\nExample code using FilenameUtils.getName(...)\n\n```\npublic void ok
(HttpServletRequest request, HttpServletResponse response)\n        throws ServletException, IOE
xception {\n    String image = request.getParameter(\"image\");\n    File file = new File(\"stat
ic/images/\", FilenameUtils.getName(image));\n\n    if (!file.exists()) {\n        log.info(imag
e + \" could not be created.\");\n        response.sendError();\n    }\n\n    response.sendRedir
ect(\"/index.html\");\n}\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-22
        cwe2021-top25: true
        cwe2022-top25: true
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        owasp:
            - A5:2017-Broken Access Control
            - A01:2021-Broken Access Control
        references:
            - https://www.owasp.org/index.php/Path_Traversal
        security-severity: CRITICAL
        shortDescription: Improper Limitation of a Pathname to a Restricted Directory ('Path Tra
versal')
        source-rule-url: https://find-sec-bugs.github.io/bugs.htm#PATH_TRAVERSAL_IN
        technology:
            - java
        vulnerability_class:
            - Path Traversal
      mode: taint
      pattern-sanitizers:
        - pattern: org.apache.commons.io.FilenameUtils.getName(...)
      pattern-sinks:
        - patterns:
            - pattern-either:
                - pattern: |
                    (java.io.File $FILE) = ...
                - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    (java.io.FileOutputStream $FOS) = ...
                - pattern: |
                    new java.io.FileInputStream(...)
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    (HttpServletRequest $REQ)
                - patterns:
                    - pattern-inside: |
                        (javax.servlet.http.Cookie[] $COOKIES) = (HttpServletRequest $REQ).getCo
okies(...); ...
                        for (javax.servlet.http.Cookie $COOKIE: $COOKIES) {
                          ...
                        }
                    - pattern: |
                        $COOKIE.getValue(...)
                - patterns:
                    - pattern-inside: |
                        $TYPE[] $VALS = (HttpServletRequest $REQ).$GETFUNC(...);
                        ...
                    - pattern: |
                        $PARAM = $VALS[$INDEX];
      severity: ERROR
    - id: java_inject_rule-EnvInjection
      languages:
        - java
      message: "Detected input from a HTTPServletRequest going into the environment\nvariables o
f an 'exec' command. The user input is passed directly to\nthe Runtime.exec() function to set an
 environment variable. This allows \nmalicious input from the user to modify the command that wi
ll be executed.\nTo remediate this, do not pass user input directly to Runtime.exec().\nValidate
 any user input before using it to set environment variables \nor command arguments. Consider us
ing an allow list of allowed values\nrather than a deny list. If dynamic commands must be constr
ucted, use\na map to look up valid values based on user input instead of using \nthe input direc
tly.\nExample of safely executing an OS command:\n```\npublic void doPost(HttpServletRequest req
uest, HttpServletResponse response)\n      throws ServletException, IOException {\n  response.se
tContentType(\"text/html;charset=UTF-8\");\n\n  String param = \"\";\n  if (request.getHeader(\"
UserDefined\") != null) {\n      param = request.getHeader(\"UserDefined\");\n  }\n\n  param = j
ava.net.URLDecoder.decode(param, \"UTF-8\");\n  String cmd = \"/bin/cmd\";\n\n  String[] allowLi
st = {\"FOO=true\",\"FOO=false\",\"BAR=true\", \"BAR=false\"}\n  if(Arrays.asList(allowList).con
tains(param)){\n      String[] argsEnv = {param};\n  }\n  \n  Runtime r = Runtime.getRuntime();\
n\n  try {\n      Process p = r.exec(cmd, argsEnv);\n      printOSCommandResults(p, response);
      \n  } catch (IOException e) {\n      System.out.println(\"Problem executing command\");\n
     response.getWriter()\n              .println(org.owasp.esapi.ESAPI.encoder().encodeForHTML(
e.getMessage()));\n      return;\n  }\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-78
        impact: MEDIUM
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: MEDIUM
        owasp:
            - A1:2017-Injection
            - A03:2021-Injection
        references:
            - https://owasp.org/Top10/A03_2021-Injection
        security-severity: HIGH
        shortDescription: Improper neutralization of special elements used in an OS command ('OS
 Command Injection')
        subcategory:
            - vuln
        technology:
            - java
        vulnerability_class:
            - Other
      mode: taint
      pattern-sanitizers:
        - patterns:
            - pattern-either:
                - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                    if($VALIDATION){
                    ...
                    }
                - patterns:
                    - pattern-inside: |
                        $A = $VALIDATION;
                        ...
                    - pattern: |
                        if($A){
                        ...
                        }
            - metavariable-pattern:
                metavariable: $VALIDATION
                pattern-either:
                    - pattern: |
                        $AL.contains(...)
      pattern-sinks:
        - pattern-either:
            - patterns:
                - pattern: (java.lang.Runtime $R).exec($CMD, $ENV_ARGS, ...);
                - focus-metavariable: $ENV_ARGS
            - patterns:
                - pattern: (ProcessBuilder $PB).environment().put($...ARGS);
                - focus-metavariable: $...ARGS
            - patterns:
                - pattern: |
                    $ENV = (ProcessBuilder $PB).environment();
                    ...
                    $ENV.put($...ARGS);
                - focus-metavariable: $...ARGS
      pattern-sources:
        - patterns:
            - pattern-either:
                - pattern: |
                    (HttpServletRequest $REQ)
        - patterns:
            - pattern-inside: |
                $FUNC(..., $VAR, ...) {
                ...
                }
            - pattern: $VAR
      severity: ERROR
    - id: java_xxe_rule-DisallowDoctypeDeclFalse
      languages:
        - java
      message: "DOCTYPE declarations are enabled for $DBFACTORY. Without prohibiting\nexternal e
ntity declarations, this is vulnerable to XML external entity\nattacks. In an XXE attack, an att
acker can exploit the processing of external \nentity references within an XML document to acces
s internal files, conduct \ndenial-of-service attacks, or SSRF (Server Side Request Forgery), po
tentially \nleading to sensitive information disclosure or system compromise.\n\nTo mitigate thi
s vulnerability, disable this by setting the feature\n\"http://apache.org/xml/features/disallow-
doctype-decl\" to true.\nAlternatively, allow DOCTYPE declarations and only prohibit external\ne
ntities declarations. This can be done by setting the features\n\"http://xml.org/sax/features/ex
ternal-general-entities\" and\n\"http://xml.org/sax/features/external-parameter-entities\" to fa
lse.\n\nSecure Code Example: \n``` \npublic void GoodXMLInputFactory() throws  ParserConfigurati
onException {\n  DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();\n  dbf.setFe
ature(\"http://apache.org/xml/features/disallow-doctype-decl\", true);\n} \n```\n"
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-611
        cwe2021-top25: "true"
        cwe2022-top25: "true"
        impact: HIGH
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A4:2017-XML External Entities (XXE)
            - A05:2021-Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://blog.sonarsource.com/secure-xml-processor
            - https://xerces.apache.org/xerces2-j/features.html
        security-severity: MEDIUM
        shortDescription: Improper restriction of XML external entity reference
        technology:
            - java
            - xml
        vulnerability_class:
            - XML Injection
      patterns:
        - pattern: |
            $DBFACTORY.setFeature("http://apache.org/xml/features/disallow-doctype-decl",
            false);
        - pattern-not-inside: |
            $RETURNTYPE $METHOD(...){
              ...
              $DBF.setFeature("http://xml.org/sax/features/external-general-entities", false);
              ...
              $DBF.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
              ...
            }
        - pattern-not-inside: |
            $RETURNTYPE $METHOD(...){
              ...
              $DBF.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
              ...
              $DBF.setFeature("http://xml.org/sax/features/external-general-entities", false);
              ...
            }
        - pattern-not-inside: |
            $RETURNTYPE $METHOD(...){
              ...
              $DBF.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
              ...
              $DBF.setAttribute(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");
              ...
            }
        - pattern-not-inside: |
            $RETURNTYPE $METHOD(...){
              ...
              $DBF.setAttribute(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");
              ...
              $DBF.setAttribute(XMLConstants.ACCESS_EXTERNAL_DTD, "");
              ...
            }
      severity: WARNING
    - id: java_xxe_rule-DocumentBuilderFactoryDisallowDoctypeDeclMissing
      languages:
        - java
      message: "DOCTYPE declarations are enabled for this DocumentBuilderFactory. Enabling \nDOC
TYPE declarations without proper restrictions can make your application \nvulnerable to XML Exte
rnal Entity (XXE) attacks. \nIn an XXE attack, an attacker can exploit the processing of externa
l entity \nreferences within an XML document to access internal files, conduct \ndenial-of-servi
ce attacks, or SSRF (Server Side Request Forgery), potentially \nleading to sensitive informatio
n disclosure or system compromise. \n\nTo mitigate this vulnerability, disable this by setting t
he\nfeature \"http://apache.org/xml/features/disallow-doctype-decl\" to true.\nAlternatively, al
low DOCTYPE declarations and only prohibit external\nentities declarations. This can be done by
setting the features\n\"http://xml.org/sax/features/external-general-entities\" and\n\"http://xm
l.org/sax/features/external-parameter-entities\" to false.\n\nSecure Code Example (You can do ei
ther of the following):\n```\npublic void GoodDocumentBuilderFactory() throws  ParserConfigurati
onException {\n  DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();\n  dbf.setFe
ature(\"http://apache.org/xml/features/disallow-doctype-decl\", true);\n  dbf.newDocumentBuilder
();\n}\n\npublic void GoodDocumentBuilderFactory2() throws  ParserConfigurationException {\n  Do
cumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();\n  dbf.setFeature(\"http://xml.
org/sax/features/external-parameter-entities\", false);\n  dbf.setFeature(\"http://xml.org/sax/f
eatures/external-general-entities\", false);\n  dbf.newDocumentBuilder();\n}\n```\n"
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-611
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        cwe2021-top25: "true"
        cwe2022-top25: "true"
        impact: HIGH
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A4:2017-XML External Entities (XXE)
            - A05:2021-Security Misconfiguration
        references:
            - https://semgrep.dev/blog/2022/xml-security-in-java
            - https://semgrep.dev/docs/cheat-sheets/java-xxe/
            - https://blog.sonarsource.com/secure-xml-processor
            - https://xerces.apache.org/xerces2-j/features.html
        security-severity: MEDIUM
        shortDescription: Improper restriction of XML external entity reference
        subcategory:
            - vuln
        technology:
            - java
            - xml
        vulnerability_class:
            - XML Injection
      mode: taint
      pattern-sanitizers:
        - by-side-effect: true
          pattern-either:
            - patterns:
                - pattern-either:
                    - pattern: |
                        $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype-dec
l", true);
                    - pattern: |
                        $FACTORY.setFeature("http://xml.org/sax/features/external-general-entiti
es", false); ... $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-entities",
false);
                    - pattern: |
                        $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-enti
ties", false); ... $FACTORY.setFeature("http://xml.org/sax/features/external-general-entities",
false);
                - focus-metavariable: $FACTORY
            - patterns:
                - pattern-either:
                    - pattern-inside: |
                        class $C {
                          ...
                          $T $M(...) {
                            ...
                            $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype
-decl",
                            true);
                            ...
                          }
                          ...
                        }
                    - pattern-inside: |
                        class $C {
                          ...
                          $T $M(...) {
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities", false);
                            ...
                            $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                            ...
                          }
                          ...
                        }
                    - pattern-inside: |
                        class $C {
                          ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                        $T $M(...) {
                          ...
                          $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
                          ...
                          $FACTORY.setFeature("http://xml.org/sax/features/external-general-en
tities",false);
                          ...
                        }
                        ...
                      }
              - pattern: $M($X)
              - focus-metavariable: $X
      pattern-sinks:
        - patterns:
            - pattern: |
                $FACTORY.newDocumentBuilder();
      pattern-sources:
        - by-side-effect: true
          patterns:
            - pattern: |
                $FACTORY
            - pattern-inside: |
                $FACTORY = DocumentBuilderFactory.newInstance();
            - pattern-not-inside: |
                class $C {
                  ...
                  $V $FACTORY = DocumentBuilderFactory.newInstance();
                  ...
                  static {
                    ...
                    $FACTORY.setFeature("http://apache.org/xml/features/disallow-doctype-decl",
true);
                    ...
                  }
                  ...
                }
            - pattern-not-inside: |
                class $C {
                  ...
                  $V $FACTORY = DocumentBuilderFactory.newInstance();
                  ...
                  static {
                    ...
                    $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-entities
", false);
                    ...
                    $FACTORY.setFeature("http://xml.org/sax/features/external-general-entities",
 false);
                    ...
                  }
                  ...
                }
            - pattern-not-inside: |
                class $C {
                  ...
                  $V $FACTORY = DocumentBuilderFactory.newInstance();
                  ...
                  static {
                    ...
                    $FACTORY.setFeature("http://xml.org/sax/features/external-general-entities",
 false);
                    ...
                    $FACTORY.setFeature("http://xml.org/sax/features/external-parameter-entities
", false);
                    ...
                  }
                  ...
                }
      severity: WARNING
    - id: properties_spring_rule-SpringActuatorFullyEnabled
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      languages:
        - generic
      message: "Spring Boot Actuator is fully enabled. This exposes sensitive endpoints\nsuch as
 /actuator/env, /actuator/logfile, /actuator/heapdump and others.\nIf the application lacks prop
er security measures (e.g., authentication and \nauthorization), sensitive data could be accesse
d, compromising the application and \nits infrastructure. This configuration poses a serious ris
k in production \nenvironments or public-facing deployments.\n\nTo mitigate the risks, take the
following measures:\n  - Expose only the Actuator endpoints required for your use case\n  - For
production environments, restrict exposure to non-sensitive endpoints \n  like `health` or `info
`\n  - Ensure Actuator endpoints are protected with authentication and authorization \n  (e.g.,
via Spring Security)\n  - Use environment-specific configurations to limit exposure in productio
n\n\nSecure Code Example:\nInstead of include: \"*\", list only the endpoints you need to expose
:\n```\nmanagement.endpoints.web.exposure.include=\"health,info,metrics\"\n```\n\nReferences:\n-
 https://docs.spring.io/spring-boot/reference/actuator/endpoints.html#actuator.endpoints.exposin
g\n- https://medium.com/walmartglobaltech/perils-of-spring-boot-actuators-misconfiguration-185c4
3a0f785\n- https://blog.maass.xyz/spring-actuator-security-part-1-stealing-secrets-using-spring-
actuators\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-497
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A01:2021-Broken Access Control
            - A3:2017-Sensitive Data Exposure
        security-severity: Medium
        shortDescription: Exposure of sensitive system information to an unauthorized control sp
here
        technology:
            - java
      paths:
        include:
            - '*properties'
      pattern: management.endpoints.web.exposure.include=*
      severity: WARNING
    - id: python_crypto_rule-HTTPConnectionPool
      languages:
        - python
      message: "The application is using HTTPConnectionPool method. This method transmits\ndata
in cleartext, which is vulnerable to MITM (Man in the middle)\nattacks. In MITM attacks, the dat
a transmitted over the unencrypted\nconnection can be intercepted, read and/or modified by unaut
horized\nparties which can lead to data integrity and confidentiality loss. \n\nTo mitigate this
 issue, use HTTPSConnectionPool instead, which encrypts \ncommunications and enhances security.\
n\nSecure Code Example:\n```\nimport urllib3\nspool = urllib3.connectionpool.HTTPSConnectionPool
(\"example.com\")\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-319
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://urllib3.readthedocs.io/en/1.2.1/pools.html#urllib3.connectionpool.HTTPSCon
nectionPool
        security-severity: MEDIUM
        shortDescription: Cleartext transmission of sensitive information
        subcategory:
            - audit
        technology:
            - python
      pattern-either:
        - pattern: urllib3.HTTPConnectionPool(...)
        - pattern: urllib3.connectionpool.HTTPConnectionPool(...)
      severity: WARNING
    - id: python_flask_rule-path-traversal-open
      languages:
        - python
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
      message: "Found request data in a call to 'open'. An attacker can manipulate this input to
 access files outside the intended \ndirectory. This can lead to unauthorized access to sensitiv
e files or directories. To prevent path traversal attacks, \navoid using user-controlled input i
n file paths. If you must use user-controlled input, validate and sanitize the \ninput to ensure
 it does not contain any path traversal sequences. For example, you can use the `os.path.join` f
unction \nto safely construct file paths or validate that the absolute path starts with the dire
ctory which is whitelisted for \naccessing file. The following code snippet demonstrates how to
validate a file path from user-controlled input:\n```\nimport os\n\ndef safe_open_file(filename,
 base_path):\n  # Resolve the absolute path of the user-supplied filename\n  absolute_path = os.
path.abspath(filename)\n\n  # Check that the absolute path starts with the base path\n  if not a
bsolute_path.startswith(base_path):\n      raise ValueError(\"Invalid file path\")\n\n  return o
pen(absolute_path, 'r')\n```\nFor more information, see the OWASP Path Traversal page: https://o
wasp.org/www-community/attacks/Path_Traversal\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-22
        impact: HIGH
        likelihood: MEDIUM
        owasp:
            - A5:2017-Broken Access Control
            - A01:2021-Broken Access Control
        references:
            - https://owasp.org/www-community/attacks/Path_Traversal
        security-severity: CRITICAL
        shortDescription: Improper Limitation of a Pathname to a Restricted Directory ('Path Tra
versal')
        technology:
            - flask
      pattern-either:
        - patterns:
            - pattern: open(...)
            - pattern-either:
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                      open(..., <... $ROUTEVAR ...>, ...)
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                      with open(..., <... $ROUTEVAR ...>, ...) as $FD:
                        ...
                - pattern-inside: |
                    @$APP.route($ROUTE, ...)
                    def $FUNC(..., $ROUTEVAR, ...):
                      ...
                      $INTERIM = <... $ROUTEVAR ...>
                      ...
                      open(..., <... $INTERIM ...>, ...)
        - pattern: open(..., <... flask.request.$W.get(...) ...>, ...)
        - pattern: open(..., <... flask.request.$W[...] ...>, ...)
        - pattern: open(..., <... flask.request.$W(...) ...>, ...)
        - pattern: open(..., <... flask.request.$W ...>, ...)
        - patterns:
            - pattern-inside: |
                $INTERIM = <... flask.request.$W.get(...) ...>
                ...
                open(<... $INTERIM ...>, ...)
            - pattern: open(...)
        - patterns:
            - pattern-inside: |
                $INTERIM = <... flask.request.$W[...] ...>
                ...
                open(<... $INTERIM ...>, ...)
            - pattern: open(...)
        - patterns:
            - pattern-inside: |
                $INTERIM = <... flask.request.$W(...) ...>
                ...
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                open(<... $INTERIM ...>, ...)
            - pattern: open(...)
        - patterns:
            - pattern-inside: |
                $INTERIM = <... flask.request.$W ...>
                ...
                open(<... $INTERIM ...>, ...)
            - pattern: open(...)
        - patterns:
            - pattern-inside: |
                $INTERIM = <... flask.request.$W.get(...) ...>
                ...
                with open(<... $INTERIM ...>, ...) as $F:
                  ...
            - pattern: open(...)
        - patterns:
            - pattern-inside: |
                $INTERIM = <... flask.request.$W[...] ...>
                ...
                with open(<... $INTERIM ...>, ...) as $F:
                  ...
            - pattern: open(...)
        - patterns:
            - pattern-inside: |
                $INTERIM = <... flask.request.$W(...) ...>
                ...
                with open(<... $INTERIM ...>, ...) as $F:
                  ...
            - pattern: open(...)
        - patterns:
            - pattern-inside: |
                $INTERIM = <... flask.request.$W ...>
                ...
                with open(<... $INTERIM ...>, ...) as $F:
                  ...
            - pattern: open(...)
      severity: ERROR
    - id: python_jwt_rule-jwt-none-alg
      languages:
        - python
      message: |
        Detected use of the 'none' algorithm in a JWT token.
        The 'none' algorithm assumes the integrity of the token has already
        been verified. This would allow a malicious actor to forge a JWT token
        that will automatically be verified. Do not explicitly use the 'none'
        algorithm. Instead, use an algorithm such as 'HS256'.
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-327
        impact: MEDIUM
        likelihood: MEDIUM
        owasp:
            - A3:2017-Sensitive Data Exposure
            - A02:2021-Cryptographic Failures
        references:
            - https://owasp.org/Top10/A02_2021-Cryptographic_Failures
        security-severity: MEDIUM
        shortDescription: Use of a Broken or Risky Cryptographic Algorithm
        source-rule-url: https://semgrep.dev/blog/2020/hardcoded-secrets-unverified-tokens-and-o
ther-common-jwt-mistakes/
        subcategory:
            - vuln
        technology:
            - jwt
      pattern-either:
        - pattern: jwt.encode(...,algorithm="none",...)
        - pattern: jwt.decode(...,algorithms=[...,"none",...],...)
      severity: ERROR
    - id: python_pyramid_rule-pyramid-csrf-origin-check
      languages:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        - python
      message: "Automatic check of the referrer for cross-site request forgery tokens\nhas been
explicitly disabled globally, which might leave views unprotected\nwhen an unsafe CSRF storage p
olicy is used. By passing `check_origin=False` \nto `set_default_csrf_options()` method, you opt
 out of checking the origin \nof the domain in the referrer header or the origin header, which c
an make \nthe application vulnerable to CSRF attacks, specially if CSRF token is not \nproperly
implemented.\nCSRF attacks are a type of exploit where an attacker tricks a user into \nexecutin
g unwanted actions on a web application in which they are authenticated. \nIf a user is logged i
nto a web application, an attacker could create a malicious \nlink or script on another site tha
t causes the user's browser to make a request \nto the web application, carrying out an action w
ithout the user's consent.\n\nTo mitigate this vulnerability, use \n'pyramid.config.Configurator
.set_default_csrf_options(check_origin=True)'\nto turn the automatic check for all unsafe method
s (per RFC2616).\n\nSecure Code Example:\n```\ndef safe(config):\n  config.set_csrf_storage_poli
cy(CookieCSRFStoragePolicy())\n  config.set_default_csrf_options(check_origin=True)\n```\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-352
        cwe2021-top25: "true"
        cwe2022-top25: "true"
        impact: LOW
        license: Commons Clause License Condition v1.0[LGPL-2.1-only]
        likelihood: LOW
        owasp:
            - A5:2017-Broken Access Control
            - A01:2021-Broken Access Control
        references:
            - https://owasp.org/Top10/A01_2021-Broken_Access_Control
            - https://docs.pylonsproject.org/projects/pyramid/en/latest/narr/security.html
        security-severity: MEDIUM
        shortDescription: Cross-site request forgery (CSRF)
        subcategory:
            - vuln
        technology:
            - pyramid
        vulnerability_class:
            - Cross-Site Request Forgery (CSRF)
      patterns:
        - pattern-inside: |
            $CONFIG.set_default_csrf_options(..., check_origin=$CHECK_ORIGIN, ...)
        - pattern: |
            $CHECK_ORIGIN
        - metavariable-comparison:
            comparison: $CHECK_ORIGIN == False
            metavariable: $CHECK_ORIGIN
      severity: WARNING
    - id: yaml_spring_rule-SpringActuatorFullyEnabled
      languages:
        - yaml
      message: "Spring Boot Actuator is fully enabled. This exposes sensitive endpoints\nsuch as
 /actuator/env, /actuator/logfile, /actuator/heapdump and others.\nIf the application lacks prop
er security measures (e.g., authentication and \nauthorization), sensitive data could be accesse
d, compromising the application and \nits infrastructure. This configuration poses a serious ris
k in production \nenvironments or public-facing deployments.\n\nTo mitigate the risks, take the
following measures:\n  - Expose only the Actuator endpoints required for your use case\n  - For
production environments, restrict exposure to non-sensitive endpoints \n  like `health` or `info
`\n  - Ensure Actuator endpoints are protected with authentication and authorization \n  (e.g.,
via Spring Security)\n  - Use environment-specific configurations to limit exposure in productio
n\n\nSecure Code Example:\nInstead of include: \"*\", list only the endpoints you need to expose
:\n```\nmanagement:\n  endpoints:\n    web:\n      exposure:\n        include: \"health,info,met
rics\"\n```\n\nReferences:\n- https://docs.spring.io/spring-boot/reference/actuator/endpoints.ht
ml#actuator.endpoints.exposing\n- https://medium.com/walmartglobaltech/perils-of-spring-boot-act
uators-misconfiguration-185c43a0f785\n- https://blog.maass.xyz/spring-actuator-security-part-1-s
tealing-secrets-using-spring-actuators\n"
      metadata:
        category: security
        confidence: MEDIUM
        cwe: CWE-497
        impact: HIGH
        likelihood: MEDIUM
        owasp:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - A01:2021-Broken Access Control
            - A3:2017-Sensitive Data Exposure
        security-severity: Medium
        shortDescription: Exposure of sensitive system information to an unauthorized control sp
here
        technology:
            - java
      patterns:
        - pattern: |
            management:
              ...
              endpoints:
                ...
                web:
                  ...
                  exposure:
                    ...
                    include: "*"
                    ...
      severity: WARNING
    - id: kotlin_perm_rule-DangerousPermissions
      languages:
        - kotlin
      message: |
        Do not grant dangerous combinations of permissions.
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-277
        owasp:
            - A5:2017-Broken Access Control
            - A01:2021-Broken Access Control
        security-severity: MEDIUM
        shortDescription: Insecure inherited permissions
      patterns:
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    $PC = $X.getPermissions(...)
                    ...
                - pattern: $PC.add($PERMISSION)
            - pattern: |
                $REFVAR = $PERMISSION
                ...;
                ($PC: PermissionCollection).add($REFVAR)
            - pattern: '($PC: PermissionCollection).add($PERMISSION)'
        - metavariable-pattern:
            metavariable: $PERMISSION
            pattern-either:
                - pattern: ReflectPermission("suppressAccessChecks")
                - pattern: RuntimePermission("createClassLoader")
      severity: WARNING
    - id: kotlin_perm_rule-OverlyPermissiveFilePermissionInline
      languages:
        - kotlin
      message: |
        Overly permissive file permission
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-732
        owasp:
            - A5:2017-Broken Access Control
            - A01:2021-Broken Access Control
        security-severity: MEDIUM
        shortDescription: Incorrect permission assignment for critical resource
      patterns:
        - pattern-either:
            - pattern: java.nio.file.Files.setPosixFilePermissions(..., java.nio.file.attribute.
PosixFilePermissions.fromString("$PERM_STRING"));
            - pattern: |
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
                $PERMISSIONS = java.nio.file.attribute.PosixFilePermissions.fromString("$PERM_ST
RING");
                ...
                java.nio.file.Files.setPosixFilePermissions(..., $PERMISSIONS);
        - metavariable-regex:
            metavariable: $PERM_STRING
            regex: '[rwx-]{6}[rwx]{1,}'
      severity: WARNING
    - id: kotlin_strings_rule-BadHexConversion
      languages:
        - kotlin
      message: |
        When converting a byte array containing a hash signature to a human readable string, a
        conversion mistake can be made if the array is read byte by byte.
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-704
        owasp:
            - A6:2017-Security Misconfiguration
            - A05:2021-Security Misconfiguration
        security-severity: MEDIUM
        shortDescription: Incorrect type conversion or cast
      patterns:
        - pattern-inside: |
            $B_ARR = ($MD: java.security.MessageDigest).digest(...);
            ...
        - pattern-either:
            - pattern: |
                for($B in $B_ARR) {
                  ...
                  $B_TOSTR
                }
            - pattern: |
                while(...) {
                  ...
                  $B_TOSTR
                }
            - pattern: |
                do {
                  ...
                  $B_TOSTR
                } while(...)
        - metavariable-pattern:
            metavariable: $B_TOSTR
            patterns:
                - pattern-either:
                    - pattern: java.lang.Integer.toHexString($B_TOINT)
                    - pattern: Integer.toHexString($B_TOINT)
                    - pattern: $B_TOINT.toHexString(...)
                - metavariable-pattern:
                    metavariable: $B_TOINT
                    pattern-either:
                        - pattern: $B_ARR[...].toInt()
                        - pattern: $B_ARR[...]
                        - pattern: $B.toInt()
                        - pattern: $B
      severity: WARNING
    - id: kotlin_strings_rule-FormatStringManipulation
      languages:
        - kotlin
      message: |
        Allowing user input to control format parameters could enable an attacker to cause excep
tions
        to be thrown or leak information.Attackers may be able  to modify the format string argu
ment,
        such that an exception is thrown. If this exception is left uncaught, it may crash the
        application. Alternatively, if sensitive information is used within the unused arguments
,
        attackers may change the format string to reveal this information.
      metadata:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        category: security
        confidence: HIGH
        cwe: CWE-134
        owasp:
            - A1:2017-Injection
            - A03:2021-Injection
        security-severity: CRITICAL
        shortDescription: Use of externally-controlled format string
      patterns:
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    $INPUT = ($REQ: HttpServletRequest).getParameter(...)
                    ...
                - pattern-inside: |
                    $FORMAT_STR = ... + $INPUT
                    ...
            - patterns:
                - pattern-inside: |
                    $INPUT = ($REQ: HttpServletRequest).getParameter(...)
                    ...
                - pattern-inside: |
                    $FORMAT_STR = ... + $INPUT + ...
                    ...
            - pattern-inside: |
                $FORMAT_STR = ... + ($REQ: HttpServletRequest).getParameter(...) + ...
                ...
            - pattern-inside: |
                $FORMAT_STR = ... + ($REQ: HttpServletRequest).getParameter(...)
                ...
        - pattern-either:
            - pattern: String.format($FORMAT_STR, ...)
            - pattern: String.format(java.util.Locale.$LOCALE, $FORMAT_STR, ...)
            - patterns:
                - pattern-inside: |
                    $F = java.util.Formatter(...)
                    ...
                - pattern-either:
                    - pattern: $F.format($FORMAT_STR, ...)
                    - pattern: $F.format(java.util.Locale.$LOCALE, $FORMAT_STR, ...)
            - pattern: '($F: java.io.PrintStream).printf($FORMAT_STR, ...)'
            - pattern: '($F: java.io.PrintStream).printf(java.util.Locale.$LOCALE, $FORMAT_STR,
...)'
            - pattern: '($F: java.io.PrintStream).format($FORMAT_STR, ...)'
            - pattern: '($F: java.io.PrintStream).format(java.util.Locale.$LOCALE, $FORMAT_STR,
...)'
            - pattern: System.out.printf($FORMAT_STR, ...)
            - pattern: System.out.printf(java.util.Locale.$LOCALE, $FORMAT_STR, ...)
            - pattern: System.out.format($FORMAT_STR, ...)
            - pattern: System.out.format(java.util.Locale.$LOCALE, $FORMAT_STR, ...)
      severity: ERROR
    - id: kotlin_strings_rule-ModifyAfterValidation
      languages:
        - kotlin
      message: |
        CERT: IDS11-J. Perform any string modifications before validation
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-182
        owasp:
            - A1:2017-Injection
            - A03:2021-Injection
        security-severity: MEDIUM
        shortDescription: Collapse of data into unsafe value
      patterns:
        - pattern-inside: |
            $PATTERN = Pattern.compile(...)
            ...
        - pattern-inside: |
            $PATTERN.matcher($VAR)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            ...
        - pattern-either:
            - pattern: |
                $VAR + $OTHER
            - patterns:
                - pattern: |
                    $VAR.$METHOD(...)
                - metavariable-regex:
                    metavariable: $METHOD
                    regex: (replace|replaceAll|replaceFirst|concat)
      severity: WARNING
    - id: kotlin_strings_rule-NormalizeAfterValidation
      languages:
        - kotlin
      message: |
        IDS01-J. Normalize strings before validating them
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-180
        owasp:
            - A1:2017-Injection
            - A03:2021-Injection
        security-severity: MEDIUM
        shortDescription: 'Incorrect behavior order: validate before canonicalize'
      patterns:
        - pattern: |
            $Y = java.util.regex.Pattern.compile("[<>]");
            ...
            $Y.matcher($VAR);
            ...
            java.text.Normalizer.normalize($VAR, ...);
      severity: WARNING
    - id: scala_perm_rule-DangerousPermissions
      languages:
        - scala
      message: |
        Do not grant dangerous combinations of permissions.
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-277
        security-severity: Info
        shortDescription: Insecure inherited permissions
      pattern-either:
        - pattern: |
            $RUNVAR = new RuntimePermission("createClassLoader");
            ...
            ($PC: PermissionCollection).add($RUNVAR);
        - pattern: |
            $REFVAR = new ReflectPermission("suppressAccessChecks");
            ...
            ($PC: PermissionCollection).add($REFVAR);
        - pattern: '($PC: PermissionCollection).add(new ReflectPermission ("suppressAccessChecks
"))'
        - pattern: '($PC: PermissionCollection).add(new RuntimePermission("createClassLoader"))'
      severity: WARNING
    - id: scala_perm_rule-OverlyPermissiveFilePermissionInline
      languages:
        - scala
      message: |
        Overly permissive file permission
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-732
        security-severity: High
        shortDescription: Incorrect Permission Assignment for Critical Resource
      patterns:
        - pattern-either:
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
            - pattern: java.nio.file.Files.setPosixFilePermissions(..., java.nio.file.attribute.
PosixFilePermissions.fromString("$PERM_STRING"));
            - pattern: |
                $PERMISSIONS = java.nio.file.attribute.PosixFilePermissions.fromString("$PERM_ST
RING");
                ...
                java.nio.file.Files.setPosixFilePermissions(..., $PERMISSIONS);
        - metavariable-regex:
            metavariable: $PERM_STRING
            regex: '[rwx-]{6}[rwx]{1,}'
      severity: WARNING
    - id: scala_perm_rule-OverlyPermissiveFilePermissionObj
      languages:
        - scala
      message: |
        Overly permissive file permission
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-732
        security-severity: Medium
        shortDescription: Incorrect Permission Assignment for Critical Resource
      patterns:
        - pattern-inside: |
            ...
            java.nio.file.Files.setPosixFilePermissions(..., $PERMS);
        - pattern-either:
            - pattern: $PERMS.add($P);
            - pattern: $A = $B + $P;
        - metavariable-regex:
            metavariable: $P
            regex: (PosixFilePermission.){0,1}(OTHERS_)
      severity: WARNING
    - id: scala_strings_rule-BadHexConversion
      languages:
        - scala
      message: |
        When converting a byte array containing a hash signature to a human readable string, a
        conversion mistake can be made if the array is read byte by byte.
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-704
        security-severity: Medium
        shortDescription: Incorrect Type Conversion or Cast
      pattern-either:
        - pattern: |
            $B_ARR = ($MD: java.security.MessageDigest).digest(...);
            ...
            for(...) {
              ...
              Integer.toHexString(...);
            }
        - pattern: |
            $B_ARR = ($MD: java.security.MessageDigest).digest(...);
            ...
            while(...) {
              ...
              Integer.toHexString(...);
            }
      severity: WARNING
    - id: scala_strings_rule-FormatStringManipulation
      languages:
        - scala
      message: |
        Allowing user input to control format parameters could enable an attacker to cause excep
tions
        to be thrown or leak information.Attackers may be able  to modify the format string argu
ment,
        such that an exception is thrown. If this exception is left uncaught, it may crash the
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
        application. Alternatively, if sensitive information is used within the unused arguments
,
        attackers may change the format string to reveal this information.
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-134
        security-severity: Info
        shortDescription: Use of Externally-Controlled Format String
      patterns:
        - pattern-either:
            - patterns:
                - pattern-inside: |
                    $INPUT = ($REQ: javax.servlet.http.HttpServletRequest).getParameter(...);
                    ...
                - pattern-inside: |
                    $FORMAT_STR = <... $INPUT ...>;
            - patterns:
                - pattern-inside: |
                    val $INPUT = ($REQ: javax.servlet.http.HttpServletRequest).getParameter(...)
;
                    ...
                - pattern-inside: |
                    val $FORMAT_STR = <... $INPUT ...>;
                    ...
            - pattern-inside: |
                val $FORMAT_STR = ... + ($REQ: javax.servlet.http.HttpServletRequest).getParamet
er(...) + ...; ...
            - pattern-inside: |
                val $FORMAT_STR = ... + ($REQ: javax.servlet.http.HttpServletRequest).getParamet
er(...); ...
        - pattern-either:
            - pattern: $VAL = <... $INPUT ...>
            - pattern: String.format($FORMAT_STR, ...);
            - pattern: String.format(java.util.Locale.$LOCALE, $FORMAT_STR, ...);
            - pattern: '($F: java.util.Formatter).format($FORMAT_STR, ...);'
            - pattern: '($F: java.util.Formatter).format(java.util.Locale.$LOCALE, $FORMAT_STR,
...);'
            - pattern: '($F: java.io.PrintStream).printf($FORMAT_STR, ...);'
            - pattern: '($F: java.io.PrintStream).printf(java.util.Locale.$LOCALE, $FORMAT_STR,
...);'
            - pattern: '($F: java.io.PrintStream).format($FORMAT_STR, ...);'
            - pattern: '($F: java.io.PrintStream).format(java.util.Locale.$LOCALE, $FORMAT_STR,
...);'
            - pattern: System.out.printf($FORMAT_STR, ...);
            - pattern: System.out.printf(java.util.Locale.$LOCALE, $FORMAT_STR, ...);
            - pattern: System.out.format($FORMAT_STR, ...);
            - pattern: System.out.format(java.util.Locale.$LOCALE, $FORMAT_STR, ...);
      severity: ERROR
    - id: scala_strings_rule-ImproperUnicode
      languages:
        - scala
      message: |
        Improper Handling of Unicode Encoding
      metadata:
        category: security
        confidence: HIGH
        cwe: CWE-176
        security-severity: Medium
        shortDescription: Improper Handling of Unicode Encoding
      pattern-either:
        - patterns:
            - pattern-either:
                - pattern: |
                    $S = ($INPUT: String).$TRANSFORM(...);
                    ...
                    $S.$METHOD(...);
                - pattern: '($INPUT: String).$TRANSFORM().$METHOD(...);'
            - metavariable-regex:
                metavariable: $METHOD
                regex: (equals|equalsIgnoreCase|indexOf)
```

CODEX/.codacy/tools-configs/semgrep.yaml

```
          - metavariable-regex:
              metavariable: $TRANSFORM
              regex: (toLowerCase|toUpperCase)
      - pattern: java.text.Normalizer.normalize(...);
      - pattern: java.net.IDN.toASCII(...);
      - pattern: '($U: URI).toASCIIString()'
    severity: ERROR
  - id: scala_strings_rule-ModifyAfterValidation
    languages:
      - scala
    message: |
      CERT: IDS11-J. Perform any string modifications before validation
    metadata:
      category: security
      confidence: HIGH
      cwe: CWE-182
      security-severity: Info
      shortDescription: Collapse of data into unsafe value
    patterns:
      - pattern: |
          $Y.matcher($VAR);
          ...
          $VAR.$METHOD(...);
      - metavariable-regex:
          metavariable: $METHOD
          regex: (replace)
    severity: WARNING
  - id: scala_strings_rule-NormalizeAfterValidation
    languages:
      - scala
    message: |
      IDS01-J. Normalize strings before validating them
    metadata:
      category: security
      confidence: HIGH
      cwe: CWE-182
      security-severity: Info
      shortDescription: Collapse of data into unsafe value
    patterns:
      - pattern: |
          $Y = java.util.regex.Pattern.compile("[<>]");
          ...
          $Y.matcher($VAR);
          ...
          java.text.Normalizer.normalize($VAR, ...);
    severity: WARNING
```

CODEX/.codacy/tools-configs/semgrep.yaml

SOURCE WITNESS 018

Trivy

SHA-256 11f651209bdd9f9dad62d567f43ff615ab5b1d099f9eb7c4212f8549fbe7b1b3

```
severity:
  - LOW
  - MEDIUM
  - HIGH
  - CRITICAL

scan:
  scanners:
    - vuln
    - secret
```

CODEX/.codacy/tools-configs/trivy.yaml